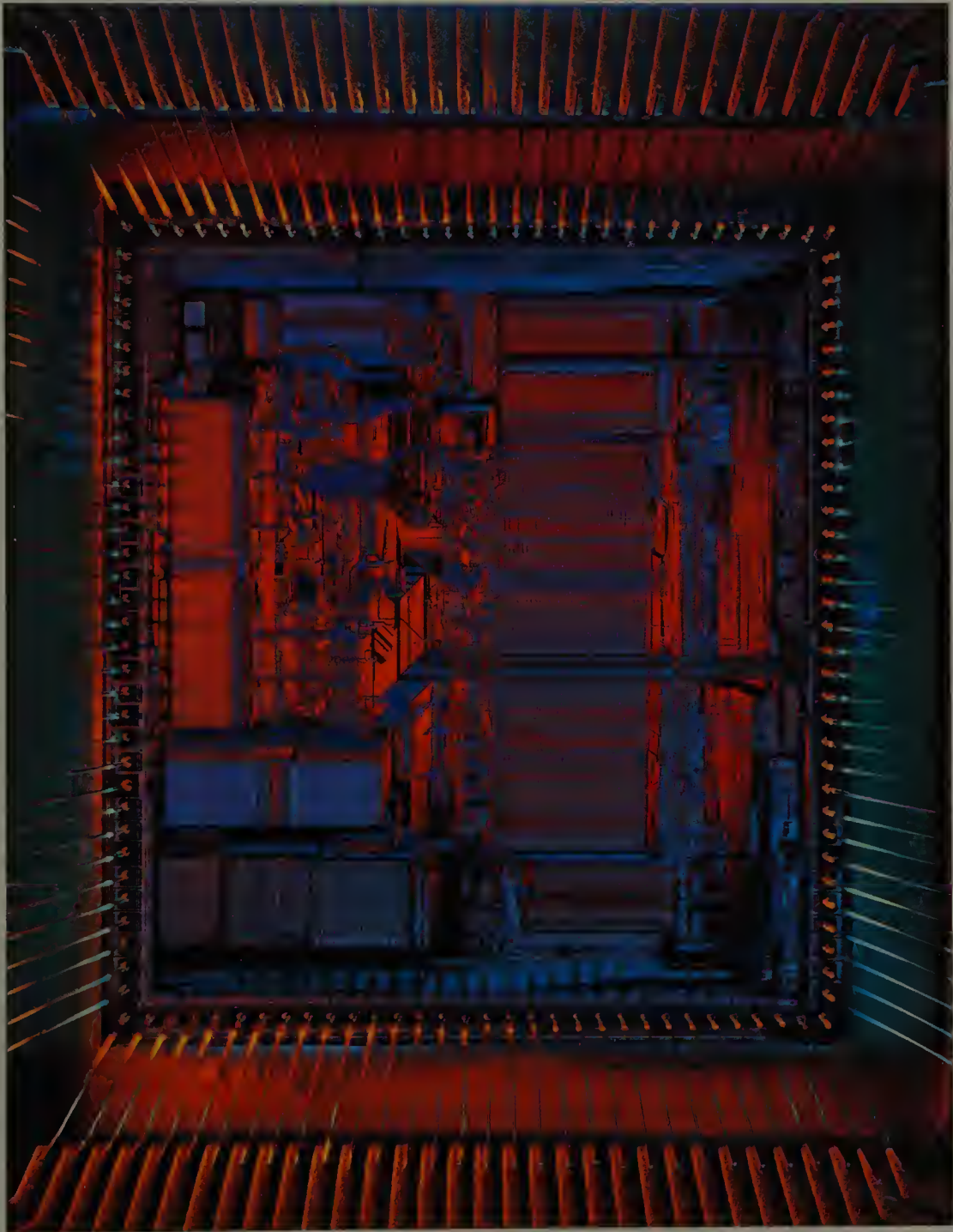

Modern Computer Architecture



Rafiquzzaman

Chandra

MODERN COMPUTER ARCHITECTURE

MODERN COMPUTER ARCHITECTURE

Mohamed Rafiquzzaman

California State Polytechnic University, Pomona, California

AND

Rajan Chandra

California State Polytechnic University, Pomona, California

WEST PUBLISHING COMPANY

ST. PAUL • NEW YORK • SAN FRANCISCO • LOS ANGELES

Project Management: Maureen P. Conway, Spectrum Publisher Services
Designer: Rita Naughton
Illustrator: Scientific Illustrators
Compositor: The Clarinda Company
Cover design: Roslyn M. Stendahl, Dapper Design, Minneapolis, MN
Cover photograph: © Phillip A. Harrington/Fran Heyl Associates.
The Motorola 68030, the newest member of the 68000 family, has on-chip data and instruction caches, parallel (Harvard-style) architecture, and dual modes of address.

COPYRIGHT © 1988

By WEST PUBLISHING COMPANY
50 W. Kellogg Boulevard
P.O. Box 64526
St. Paul, MN 55164-1003

All rights reserved

Printed in the United States of America

Library of Congress Cataloging-in-Publication Data

Rafiquzzaman, Mohamed.
Modern computer architecture.

Bibliography: p.
Includes index.

1. Computer architecture. I. Chandra, Rajan.
II. Title.

QA76.9.A73R34 1988

004.2'2

88-222

ISBN 0-314-60714-0



**To Our Parents
and
Families**

Contents

Preface xi

**1 INTRODUCTION TO COMPUTERS—EVOLUTION,
PROGRAMMING AND ARCHITECTURE** **1**

1.1	EVOLUTION OF COMPUTERS	1
1.2	HARDWARE AND FIRMWARE	5
1.3	COMPUTER SOFTWARE	5
1.3.1	Introduction to Computer Programming	5
1.3.2	The Operating System	6
1.4	BASICS OF COMPUTER ARCHITECTURE	6
1.4.1	Introduction	7
1.4.2	Computer Structures	10
1.5	TYPES OF COMPUTERS	17
1.6	FUTURE TRENDS	18
	<i>Questions and Problems</i>	18
	<i>References</i>	21

2 COMPUTER INSTRUCTION SET**22**

2.1	INTRODUCTION	22
2.2	OP-CODE ENCODING	24
2.3	ADDRESSING MODES	31
2.4	INSTRUCTION TYPES	39
2.4.1	Data-transfer Instructions	40
2.4.2	Arithmetic Instructions	40
2.4.3	Logical Instructions	41
2.4.4	Program-control Instructions	42
2.4.5	System-control Instructions	48
2.4.6	I/O Instructions	50
2.5	REDUCED INSTRUCTION SET COMPUTER (RISC)	54
2.5.1	Case Study: RISC I (University of California, Berkeley)	52
2.5.2	RISC Assessment	54
	<i>Questions and Problems</i>	56
	<i>References</i>	59

3 EXECUTION UNIT**60**

3.1	INTRODUCTION	60
3.2	REGISTER SECTIONS	61
3.2.1	General Register Design	62
3.2.2	Combinational Shifter Design	66
3.2.3	Flag Register	70
3.3	ADDERS	74
3.4	ALU DESIGN	84
3.5	BIT-SLICE PROCESSOR	88
3.6	MULTIPLICATION OF UNSIGNED AND SIGNED INTEGERS	94
3.7	DIVISION OF UNSIGNED INTEGERS	110
3.8	COPROCESSORS	118
3.8.1	Intelligent Monitor Interface	118
3.8.2	Interface Using Special Bus Signals and Instructions	119
3.8.3	Coprocessor Interface Using Special Instructions	120
	<i>Questions and Problems</i>	121
	<i>References</i>	126

4 CONTROL UNIT**127**

4.1	INTRODUCTION	127
4.2	BASIC CONCEPTS	128
4.3	DESIGN METHODS	139
4.3.1	Hardwired Control Design	140
4.3.2	Microprogrammed Control Unit	155
	<i>Questions and Problems</i>	192
	<i>References</i>	202

5 MEMORY ORGANIZATION 202

5.1	INTRODUCTION	202
5.2	CHARACTERISTICS OF MEMORY SYSTEMS	203
5.3	MAIN MEMORY DESIGN	208
5.4	POPULAR ELECTROMECHANICAL MEMORY DEVICES	213
5.5	MEMORY HIERARCHY	216
5.6	CACHE MEMORIES	220
5.7	ASSOCIATIVE MEMORY	232
5.8	VIRTUAL MEMORY AND MEMORY MANAGEMENT CONCEPTS	240
	<i>Questions and Problems</i>	267
	<i>References</i>	274

6 INPUT/OUTPUT 276

6.1	BASIC CONCEPTS	276
6.2	PROGRAMMED I/O	278
6.3	STANDARD I/O VERSUS MEMORY-MAPPED I/O	279
6.4	UNCONDITIONAL AND CONDITIONAL PROGRAMMED I/O	280
6.5	INTERRUPT I/O	282
6.5.1	Basic Concepts	282
6.5.2	Main Features of Interrupt I/O	284
6.6	DIRECT MEMORY ACCESS (DMA)	289
6.7	MOTOROLA MC68000 I/O	291
6.7.1	MC68000 Programmed I/O	291
6.7.2	MC68000 Interrupt System	294
6.7.3	MC68000 DMA	297
6.7.4	MC68000 Exception Handling	298
6.8	I/O PROCESSOR	299
	<i>Questions and Problems</i>	301
	<i>References</i>	302

7 FUNDAMENTALS OF PARALLEL PROCESSING 304

7.1	INTRODUCTION	304
7.2	PARALLELISM IN CONVENTIONAL COMPUTERS	305
7.3	GENERAL CLASSIFICATION OF COMPUTER ARCHITECTURES	306
7.4	ARRAY PROCESSORS	308
7.4.1	Systolic Arrays	310
7.4.2	Wavefront Array Processors	311
7.5	PIPELINE PROCESSING	314
7.5.1	Basic Concepts	315
7.5.2	Arithmetic Pipelines	317
7.5.3	Instruction Pipelines	320
7.6	MULTIPROCESSORS	326
7.6.1	Single Bus	328

7.6.2	Multibus	329
7.6.3	Crossbar	330
7.6.4	Multiport Memory	333
7.7	DATA-FLOW COMPUTER SYSTEMS	334
	<i>Questions and Problems</i>	339
	<i>References</i>	342

APPENDIX A	INTRODUCTION TO LOGIC DESIGN	344
-------------------	-------------------------------------	------------

APPENDIX B	MOTOROLA MC68000 AND ITS INSTRUCTION SET	380
-------------------	---	------------

INDEX	407
--------------	------------

Preface

The evolution of computers has revolutionized the design of electronic systems. Most industrial applications are now controlled by computers. Typical computer-based applications include process control, communication systems, home computers, and video games.

In order to apply both hardware and software capabilities of a computer effectively, it is necessary for engineers and scientists to be familiar with modern computer architecture. The aim of this book is to provide readers with the hardware and basic software aspects of typical computer systems. This includes computer organization, design of CPU, I/O and memory, typical instruction sets, and parallel computers.

The book is divided into seven chapters:

- *Chapter 1* presents basic programming techniques; concepts of hardware, software, and firmware; basics of computer architecture; and typical CPU architectures.
- *Chapter 2* contains basics of instruction sets. The fundamentals of op-code encoding, addressing modes, and typical instruction sets are described. These concepts are then related to a modern processor such as the Motorola 68000, which is included as an appendix.

- *Chapter 3* introduces design of registers and ALU. The basic features of MSI ALU chips, bit-slice processors, and coprocessors are covered.
- *Chapter 4* presents control unit design, using both hardwired and microprogrammed approach. These concepts are related by nanomemory implementation of the 68000 control unit.
- *Chapter 5* explains the basics of memory design. Basic concepts associated with both main and secondary memory are covered. Cache Memory Implementation in MC 68020 is also included.
- *Chapter 6* is devoted to the design of computer I/O systems. Typical I/O techniques such as programmed I/O, interrupt I/O, and DMA are included. These concepts are then related to the 68000 I/O. Finally, 68000 exception handling and structure of a typical I/O processor such as the Intel 8089 are discussed.
- *Chapter 7* covers fundamentals of parallel processing.

Computer architecture is a wide-open area. Also, the computer field is changing so rapidly that it is not possible to cover all topics thoroughly in a book like this. The book has evolved from classroom notes on computer architecture for EE 457 at the University of Southern California, Los Angeles, and ECE 342 at California State Polytechnic University, Pomona.

The electrical engineering majors are expected to have basic digital logic background, while computer science majors should have background in both digital hardware logic and assembly language programming. The authors have tried to cover all major topics in adequate detail so that this book can be used as an undergraduate text for electrical engineering architecture courses.

The materials in the text can also be used as a reference document for engineers, scientists, and other users in industry. A solutions' manual for this book will be available from the publisher.

The authors would like to express their appreciation to Ari Danesh, Kathy Zeller, and Ben Nise for reviewing the manuscript and making constructive suggestions and to Bryce Nakatane, Harjit Singh, Todd Norville, Seta Boghikian, Art Densmore, Greg Fox, and Albert Scalise.

Finally the authors would like to thank Mary MacDonald, Ann MacDonald, and Gayle Brumley for typing the manuscript.

Mohamed Rafiquzzaman
Rajan Chandra

CHAPTER 1

Introduction to Computers— Evolution, Programming, and Architecture

This chapter provides the fundamental concepts associated with computer hardware and software. Included topics are: computer evolution, programming, computer structures, and types of computers.

1.1 EVOLUTION OF COMPUTERS

To provide some perspective on digital computers, their historical development is discussed in this section. The first electronic computer, called ENIAC, was invented in 1946 at the Moore School of Engineering, University of Pennsylvania. ENIAC was designed using vacuum tubes and relays. The computer could execute 5000 additions or subtractions per second. These additions, subtractions, and other operations were carried out by special wiring rather than by programming. Rewiring was necessary for modifications to the existing operations. The concept of programs stored in memory became feasible later.

Von Neumann, a consultant at the Moore School, designed the first conceptual architecture of a stored-program computer, called the EDVAC. Soon afterwards, M. V. Wilkes of Cambridge University implemented the EDSAC, the first operational stored-memory computer.

Using the stored-program concept, Von Neumann alone designed a new, unique, and feasible architecture called the Von Neumann architecture. Nearly all modern computers still use this architecture.

A typical Von Neumann computer is depicted in Figure 1.1. It consists of five basic elements: input, memory, control unit, arithmetic logic unit (ALU), and output.

An input device, such as a keyboard, is used to enter into memory a program containing instructions and data. The memory stores this program. The arithmetic logic unit carries out arithmetic and logic operations such as ADD, SUBTRACT, OR, AND, and NOT. The control unit decodes or translates each instruction. Based on the desired operations, the control unit generates enable signals for the ALU and other elements in the system to complete the task. An output device, such as a cathode-ray tube (CRT), displays the result.

To execute instructions, the computer performs the following steps:

- The control unit reads or fetches an instruction from memory and decodes or translates it.
- For arithmetic or logic-type instructions, the control unit generates enable signals for the ALU to perform the required operations.
- For input/output (I/O) instructions, the control unit generates enable signals for the I/O either to input data from or output data to external devices.

The Von Neumann architecture was the first computer that allowed storing of instructions and data in the same memory. The memory of this computer was 4096 (2^{12}) words, which were 40 bits wide. The control unit was designed using a hardwired approach, with an instruction decoding capability of 64 operation codes or op-codes. Soon afterwards, a number of other computers—namely, ILLIAC at the University of Illinois and JOHNIAC at the Rand Corporation—were designed. All the computers discussed up to this point were designed for scientific computations.

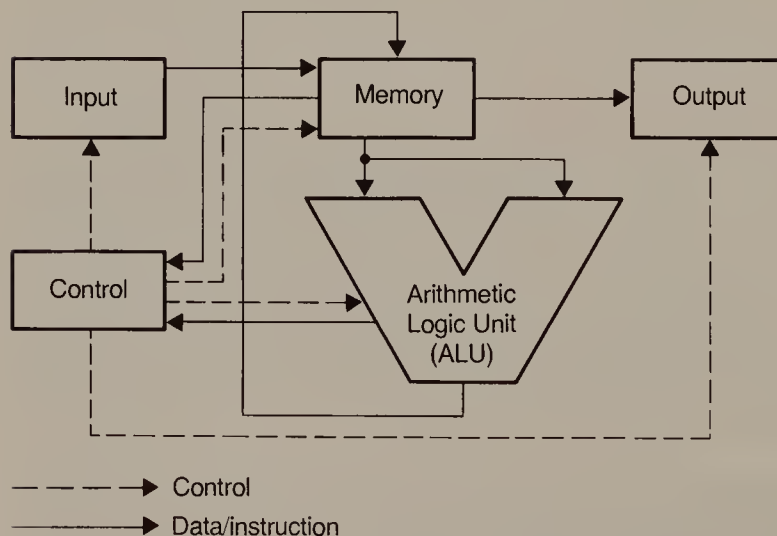


Figure 1.1 Von Neumann Architecture (Modified version of Figure 1.1, page 5 in J. L. Baer, *Computer Systems Architecture* © by Computer Science Press, 1803 Research Blvd, Rockville, MD. Reprinted with permission.)

With the introduction of transistors in the 1950s, the computer industry grew more rapidly. NCR, RCA, IBM, and other designed fully transistorized computers, and the concept of real-time computers evolved. Real-time computers are loosely defined as the computers that provide fast responses to process request.

The first computer for real-time application was designed at MIT in 1951 after 4 years of extensive research. The computer was called WHIRLWIND I (WWI), and its main areas of application were in process control and aircraft simulation. WWI had a 16-bit word length with 27 operation codes, or op-codes. It was capable of performing 20,000 instructions in 1 second. WWI was the first core-memory-based computer. Before this, electrostatic storage tubes had been used for memory.

These early computers had no operating systems and were obviously not user-friendly. The operating system is the collection of programs, usually written by the manufacturer, that controls the computer system. The operating system allows the operator to communicate with the computer.

The computer industry made major strides in the 1950s. In 1951, the first tape-based system, UNIVAC I (universal automatic computer) was released by Eckert and Mauchly (formerly with the University of Pennsylvania). This tape could be read forward or backward, and error-checking capabilities were included. UNIVAC I used a mercury delay line for memory, which quickly made it obsolete. UNIVAC II, a core-memory-based computer, was soon released. It was primarily designed for commercial applications.

The entry of IBM into the computer industry happened in 1953 with the development of a desk calculator called the IBM 701. The IBM 701 was designed using Von Neumann architecture with 2K 36-bit words of unreliable electrostatic memory. IBM's next computer, the 704, was announced in 1956 with 4K words of core memory, three index registers, and hardwired floating-point arithmetic.

In 1954, IBM announced its first drum-based computer, called the IBM 650. With the introduction of drum-based computers, the use of assemblers, compilers, and other system-oriented programs became feasible.

The advent of integrated circuits and advanced operating systems, such as those with multiprogramming features, put computer manufacturers under pressure to implement these features into their computers in the 1960s. IBM announced the IBM 360 scientific computer in the mid-1960-s. This computer had more memory and I/O devices than earlier computers and was faster.

In 1970, IBM announced System 370, which is similar to the 360 except for a different circuit technology. Several other manufacturers, such as RCA and UNIVAC, introduced computers based on the IBM 360 architecture.

Since the 1960s, Digital Equipment Corporation (DEC) has developed its program data processor (PDP) line of computers, such as the PDP8 and PDP11. The DEC computers provided the basis for future trends in real-time minicomputers.

In the 1970s, microprocessors became popular. The invention that paved the way for the evolution of microprocessors [the central processing unit (CPU) of the microcomputer] was the planar transistor in the late 1950s. The planar process led to the small-scale integrated circuits (SSI) of the early 1960s. Early usage of SSI (which soon gave way to medium- and large-scale integration, or MSI and LSI) was for developing electronic calculators. The first electronic calculators were constructed from discrete devices, but soon calculator chip sets were developed using MSI and LSI technology.

In an attempt to create a general-purpose computational chip set, the Intel Corporation designed the first microprocessor, the 4004, in 1969. The Intel 4004 was a 4-bit device that contained the ALU, control unit circuitry, and the registers that define current microprocessors. The Intel 4004 was rapidly replaced by the Intel 8008 and then the Intel 8080/8085. Very quickly, other 8-bit microprocessors such as the Motorola 6800/6809, Zilog Z80, and MOS 6502 appeared.

Intel 8080/8085 was the microprocessor that provided the basis of the future trends in 8-bit microprocessors. Typical 8-bit microprocessors use an 8-bit data path and a 16-bit address range. The powerful instruction set, (for its size, price, and power consumption) of 8-bit microprocessors allowed the development of general-purpose microcomputer systems. Advances in IC technology have led to the development of powerful 16-bit (Intel 8086 and Motorola 68000) and 32-bit (Intel 80386 and Motorola 68020) microprocessors. Microcomputers designed using these microprocessors have the capability of processing programs written in high-level languages. In addition, a 16-bit or 32-bit microprocessor can directly address memory space in the order of 1 megabyte to 4 gigabytes.

The early microcomputer systems typically consisted of an 8-bit microprocessor, 8K to 16K of main memory for program and data storage, and a simple I/O device such as the ASR-33 teletype. Modern operating systems were not available for these microcomputers, but they typically contained system monitors. The system monitor contained the basic software routines needed to operate the microcomputer. These routines usually included I/O handlers, number base conversion routines, memory access routines, and a primitive command line interpreter (CLI). The CLI allowed the user to enter simple English commands to the microcomputer.

Most of the first system monitors and application programs for microcomputers were written in machine, or assembly, language. As the power of microcomputers grew and the price dropped, high-level languages were developed for use on microcomputers. One of the first microcomputer high-level languages was PL/M, which was developed for the Intel 8080. PL/M was a powerful block-structured language, which was then used to create further system software such as the de facto standard for microcomputer operating systems, CP/M.

As the need for more program and data storage grew, typical main memory sizes grew from 64K bytes for 8-bit microprocessors to 16 megabytes for 16-bit microprocessors. The early secondary storage devices used were paper tapes, punched cards and readers. Cassette recorders replaced paper tape; the current floppy disk provides a good compromise between system performance and system cost. The most current microcomputer systems have one or more floppy-disk devices for secondary storage. The current trend for secondary storage is towards the hard-disk systems, which typically hold 10 to 20 million bytes of information.

The ASR-33 Teletypewriters were replaced by high-speed terminals, usually referred to as CRTs. Current CRTs communicate with the microcomputer by data rates of 9600 to 19,200 baud. (Baud refers to bits per second.)

The current trend for microprocessor and minicomputer applications is toward distributed systems, where several microprocessors usually perform dedicated real-time activities. These microprocessors are connected to a host minicomputer via a data bus such as a coaxial cable. High reliability is one of the main characteristics of such a system.

1.2 HARDWARE, SOFTWARE, AND FIRMWARE

Hardware refers to the physical components of a computer such as memory, CPU, transistors, nuts and bolts, and so on. When a computer is said to have the hardware capability to add, it has a hardwired electronic circuit that is capable of adding two numbers. Changes in the hardware cannot be accomplished by programmers.

Software consists of a collection of programs. Programs contain a sequence of instructions for performing a specified task. *Firmware* falls between hardware and software. Firmware consists of programs that are included in electronic circuits during their manufacture. Firmware is used when the programs must be retained in case of power failure and when programs are not expected to be changed.

In general, most computers are regarded as clocked sequential systems. In these computers, almost all activities pertaining to instruction execution are synchronized with master clocks. Those operations that can be executed within one clock period are referred to as *microoperations*, or *microinstructions*. A program written using such microinstructions is called a *microprogram*. This microprogram is held in a separate memory called the *control memory*. It is executed in the same way as a conventional program is executed, by reading microinstructions from the read-only memory (ROM) inside the control unit. This concept is called *microprogramming* and was originated by M. V. Wilkes of England. This technique allows greater flexibility and a structured approach in designing CPU control algorithms, floating-point arithmetic routines, and diagnostic routines.

In modern computers, the control unit is normally microprogrammed. The microprogram is contained in the ROM in the control unit of the computer. There, it interprets the computer's instruction set. The microprogram facilitates the implementation of hardware and is, therefore, called firmware to distinguish it from software.

Recently, firmware has been receiving considerable attention in computer design. For example, some of the important features of many operating systems are accomplished these days using firmware rather than software. This is done for faster execution time, better system security, and cost reduction in the area of software development. Functions such as subroutine calls or returns, servicing of interrupts, and context switching (switching a single computer at high speed among several users) are implemented in firmware.

1.3 COMPUTER SOFTWARE

This section briefly discusses the basic concepts of computer software. An overview of programming and operating systems is included.

1.3.1 Introduction to Computer Programming

A digital computer is capable of storing large amounts of information that include instructions and data. The computer executes these instructions and uses the data to produce results. A set of instructions and data specifying the solution of a particular problem is called a *program*.

The computer uses memory to store programs in binary. The computer can exe-

only programs that are represented in binary forms. A program written using binary numbers is called a *machine-language program*. Programmers find it difficult to work with machine language. Therefore, most computer manufacturers include a program called an *assembler* with the computer system. The assembler translates a program written in a symbolic programming language (assembly language) into machine language. Assembly language programming is used when a programmer needs exposure to the internal operation of a computer. Assembly language programming is chosen under the following situations:

- When memory space is critical
- For short- to medium-sized programs
- For more control operations than computations

Assembly language is not suitable for writing well-structured, reliable programs.

High-level languages, such as FORTRAN 77 and Pascal, can be used efficiently to write structured programs. Most newer software is written in high-level languages. High-level languages require translators (compilers) for translating the high-level language program into machine language.

1.3.2 The Operating System

The operating system performs resource management and human-to-machine translation functions. A resource may be the CPU, memory, or an I/O device. Basically, an operating system is another computer program that tells the machine what to do under a variety of conditions. Major operating system functions include efficient sharing of the memory, I/O peripherals, and CPU among several users. In summary, an operating system is:

- An interface between hardware and users
- A manager of system resources in accordance with system policy to achieve system objectives.

System administrators formulate a set of policies to run a computer installation efficiently. These policies are the decisions governing the memory space allocation, disk space allocation, and the price paid by a user to use these resources. In this sense, an operating system can be viewed as a mechanism that implements all such system policies. When several users share a computer installation, conflicts such as resource contention are more likely to occur. At this point, the operating system resolves the conflict by enforcing system policies. Another important consequence of resource sharing is the possibility that there is a threat to the system's integrity by one or more malicious users. Thus an operating system must protect the system from the users and a user from others.

1.4 BASICS OF COMPUTER ARCHITECTURE

This section contains the fundamental concepts of computer architecture along with a basic overview of computer organization.

1.4.1 Introduction

A computer is an electronic device capable of processing information. The computer processes information by executing a program consisting of a sequence of instructions stored in memory. A block-diagram representation is shown in Figure 1.2. A computer system of this kind is called a *stored-program computer* because the instructions and data are held in the same memory.

As mentioned before, instruction execution takes place within the CPU and is controlled by the control unit. Typically, the control unit has to perform the following tasks to carry out the instruction execution process:

- Instruction interpretation
- Instruction sequencing

In the instruction interpretation phase, the control unit reads an instruction and recognizes the instruction operation code. It then retrieves the required operands or data items and performs the desired operation by activating the ALU. Finally, the result is routed to a specified destination. The control unit determines the address of the next instruction to be executed during instruction sequencing. Normally, the instructions of a program are executed in a strictly sequential fashion. The next instruction to be executed is stored in the memory word that immediately follows the word where the current instruction is held. However, the following are samples of two exceptional situations to the sequential-control flow mentioned earlier:

- Branch instruction
- Subroutine calls

For the first case, the control unit jumps to an instruction stored in some arbitrary address B as the next instruction to be executed and continues from the jump address. The control unit begins executing a complete program (called a *subprogram*, or *procedure*) starting at some address S for a subroutine call, and it returns to the next instruction after the subroutine-call instruction, when a return instruction is inserted in the body of the subroutine.

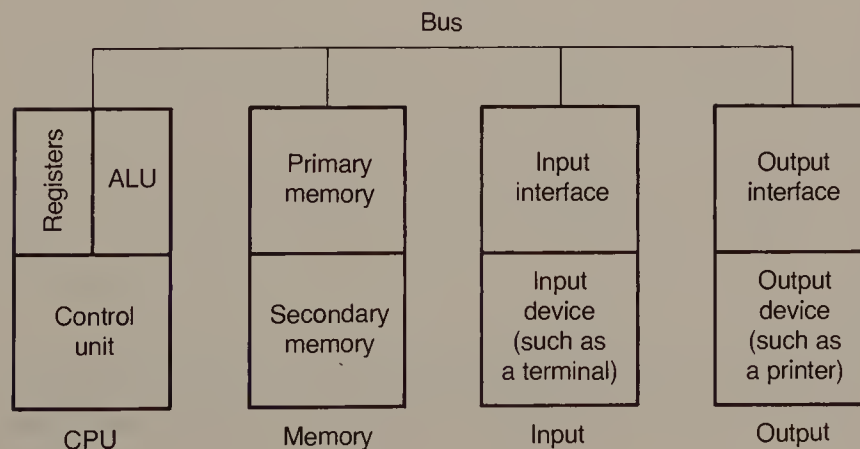


Figure 1.2 Organization of a Stored-program Computer System

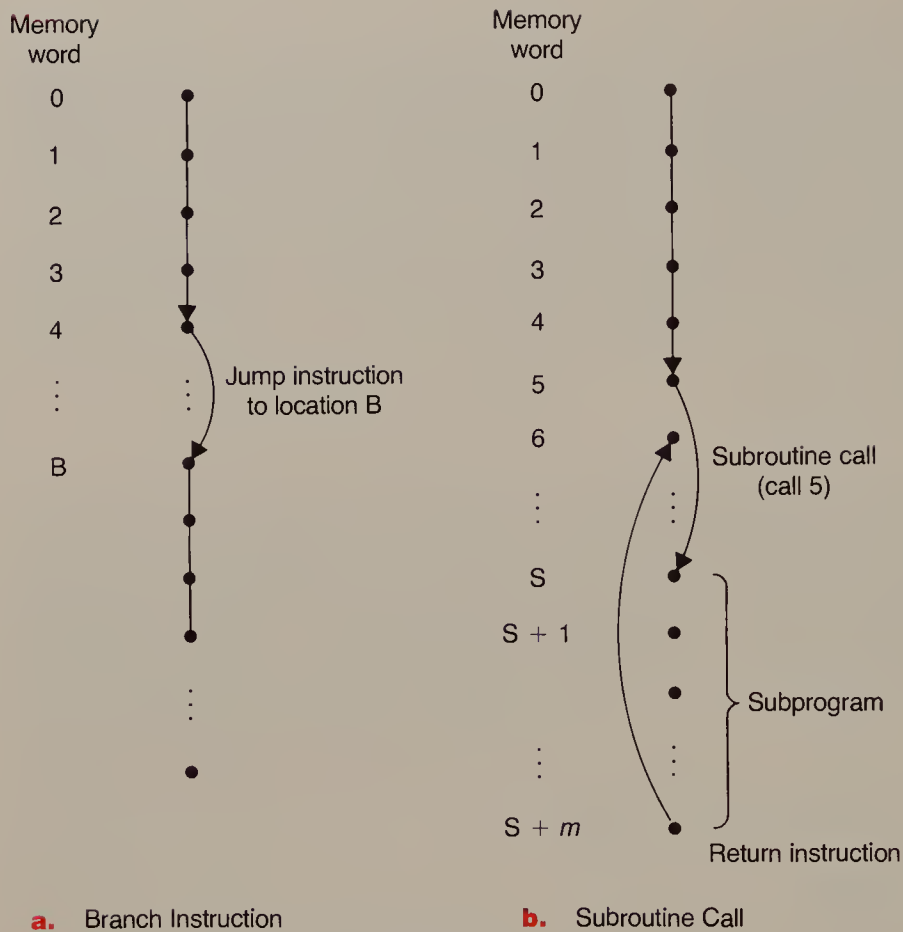


Figure 1.3 Typical Control Flow Sequences

These two situations are illustrated in Figure 1.3.

The CPU of a computer includes a set of high-speed registers. These registers may hold data, temporary results, or memory addresses when a computation is in progress. The control unit typically uses the following registers to accomplish a task:

REGISTER NAME	PURPOSE
Program counter (PC)	Holds address of the next instruction to be executed
Instruction register (IR)	Holds the instruction currently being executed
Effective address register (EAR)	Holds the address of the data to be retrieved from memory

These registers are usually referred to as *dedicated registers* because they are available for the exclusive use of the control unit and cannot be accessed by a user program.

As mentioned earlier, programs and data are held in the memory unit. This unit can be divided into two sections:

- Primary memory
- Secondary memory

The primary memory is often designed using solid-state technology. Sometimes this portion of the memory is called *executable memory* because any program in this memory can be executed by the CPU. In many contemporary computers, the primary memory is further divided into two types:

- Read-write memory (RWM)
- Read-only memory (ROM)

As the name implies, a RWM allows one to read from and write into the memory. A ROM provides only a read access to the stored data. The inclusion of a ROM can be justified by the following reasons:

- Advances in IC technology allow manufacturers to produce ROMs at a low price.
- Certain information stored in a primary memory is unalterable in nature. This includes: monitor programs, mathematical tables, and look-up tables, such as an op-code table (used by an assembler). Therefore, use of a ROM significantly lowers the system cost.

Both RWM and ROM are usually organized as a set of 8-bit words.

RWMs and ROMs are also called *random-access storage devices* because the time required to access a memory word is independent of its position in the memory device. These days, a computer system holds large data files and programs such as a text editor and an assembler. These items need large memory space, and thus the use of solid-state memory increases the system cost tremendously. Therefore, to bring down the storage cost, electromechanical memory devices are included in a computer system. These devices include magnetic disks and tapes and collectively constitute the secondary memory of a computer system. The consequence of using these devices is a low speed of operation. However, this may not severely affect the system performance because the CPU does not often require information stored in the secondary memory devices.

The use of efficient memory-management algorithms allows one to maintain the system performance at an optimum level. These algorithms swap data between the primary and secondary memory devices in a piecemeal fashion, so the item required by the CPU is always confined to the primary memory. This topic is addressed in Chapter 5.

The I/O devices allow a user to communicate with the computer. A user terminal and a line printer are typical examples. In general, the operational characteristics of these I/O devices vary considerably from the CPU. Therefore, to facilitate communication between the CPU and an I/O device, there must be interface circuitry between them.

The key elements of a stored-program computer system are connected to each other by a bus. A bus consists of a set of wires carrying address, data, and control information. The computer system bus may be divided into three subcategories, as follows:

- Address bus
- Data bus
- Control bus

The contents of an address bus specify the address of the memory word or I/O device with which the processor wishes to communicate. Since the address information always originates from the processor, the address bus is unidirectional. (The size of this bus varies from 16 to 32 bits.) With a 32-bit address bus, the processor can directly access over 4 billion (2^{32}) different external locations. The data between the processor and external unit is transferred via the data bus. The data bus is bidirectional in nature, and its size typically lies in the range of 8 to 32 bits. The control bus carries necessary commands and other signals. These signals are primarily used to synchronize the I/O activities.

1.4.2 Computer Structures

Most contemporary computers may be classified into three groups:

- General register machines
- Accumulator based machines
- Stack machines

The organization of a general register processor is shown in Figure 1.4. There are eight general registers (R0 through R7). They are called general registers because any one of these registers can be used to hold data, memory addresses, or the results of arithmetic or logic operations.

The stack pointer (SP) is a dedicated register that usually holds the address of the top element of the stack. The mechanics of a stack are addressed later in this section.

The F register is known as the *flag register*. It holds status flags such as the zero flag (Z-flag) and the carry flag. These flags are usually updated after executing an arithmetic or logic instruction to indicate the status of a computation in progress. For example, if the Z-flag is set to 1, the most recently executed instruction has produced a zero result.

General register processors support three- and two-address instructions. Some examples of three-address instructions are summarized next:

INSTRUCTION	OPERATION
ADD x, y, z	$z \leftarrow (x) + (y)$
SUB y, x, z	$z \leftarrow (x) - (y)$

MUL x, y, z $z \leftarrow (x) * (y)$

DIV y, x, z $z \leftarrow (x) / (y)$

Typical examples of two-address instructions are as follows:

INSTRUCTION	OPERATION
MOV x, y	$y \leftarrow (x)$
ADD x, y	$y \leftarrow (x) + (y)$
SUB y, x	$x \leftarrow (x) - (y)$
MUL y, x	$x \leftarrow (x) * (y)$
DIV y, x	$x \leftarrow (x) / (y)$

In the preceding, the symbols x, y, and z may refer to a memory word or a general register. $z \leftarrow (x) + (y)$ means the contents of x and y are added and the result is saved in the address z.

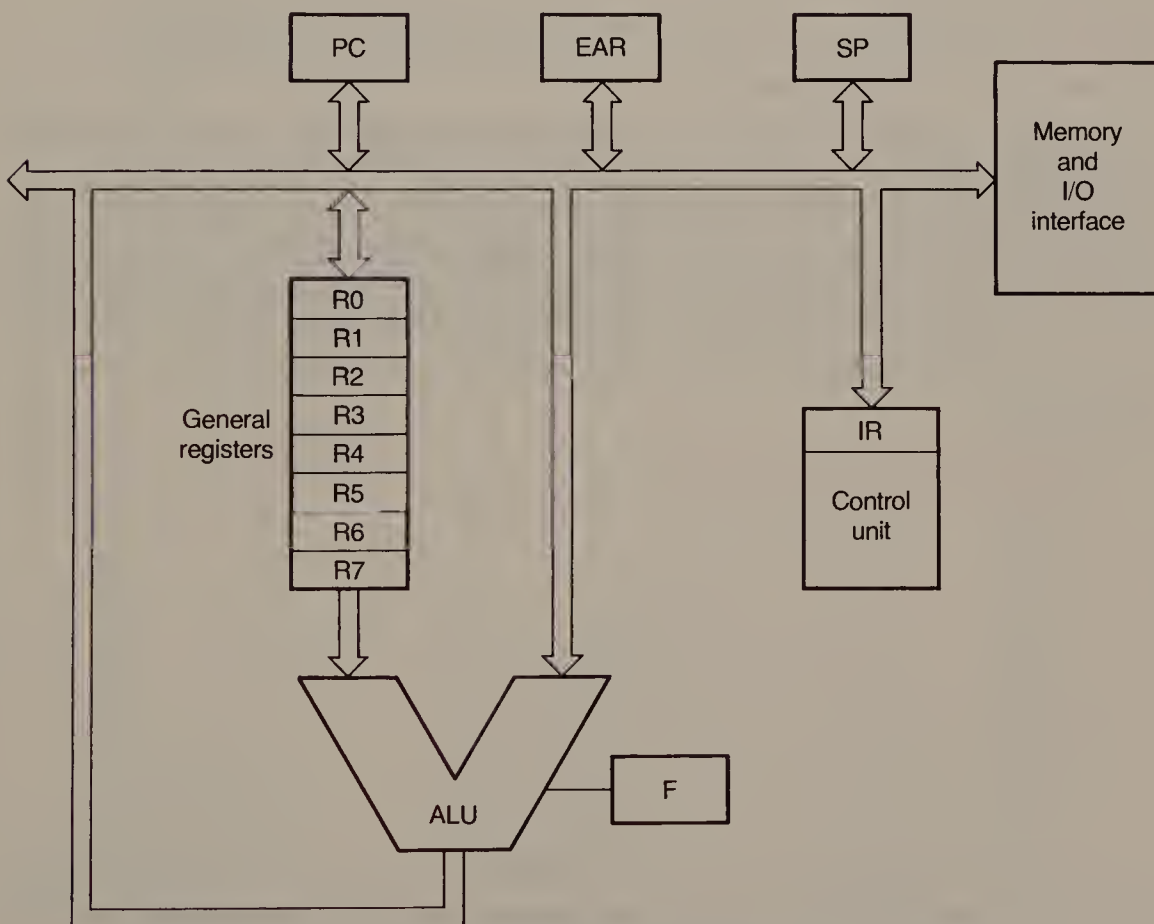


Figure 1.4 Organization of a Typical General Register Processor (From J. F. Wakerly *Microcomputer Architecture and Programming*, John Wiley and Sons, 1981. Reprinted by permission of John Wiley and Sons, Inc.)

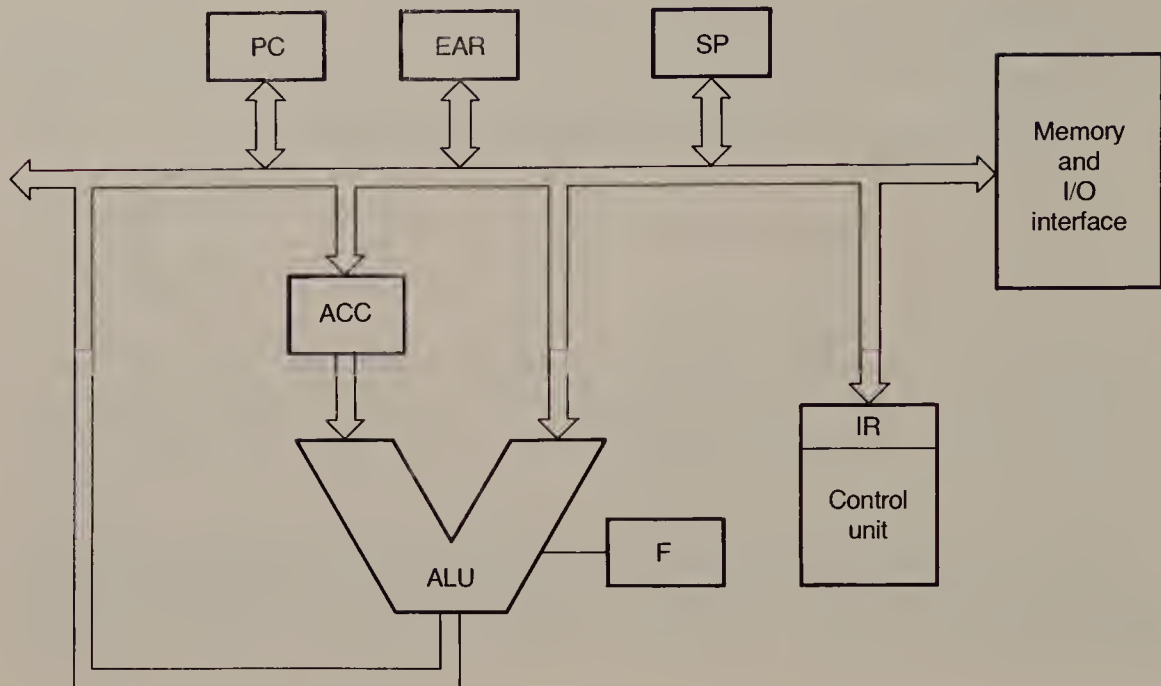


Figure 1.5 Organization of an Accumulator-based Processor (From J. F. Wakerly, *Microcomputer Architecture and Programming*, John Wiley and Sons, 1981. Reprinted by permission of John Wiley and Sons, Inc.)

IBM's 370, DEC's VAX-11, and Motorola's MC68000 are examples of a general register processor.

The organization of an accumulator-based processor is shown in Figure 1.5. One of the operands is assumed to be held in the accumulator register (ACC) for arithmetic or logic operation. The results of all arithmetic and logic operations are routed to the ACC. One-address instructions are very predominant in this organization.

Some typical one-address instructions available in an accumulator-based machine are as follows:

INSTRUCTION		OPERATION
LDA	x	$\text{Acc} \leftarrow (x)$
STA	x	$x \leftarrow (\text{Acc})$
ADD	x	$\text{Acc} \leftarrow (\text{Acc}) + (x)$
SUB	x	$\text{Acc} \leftarrow (\text{Acc}) - (x)$
MUL	x	$\text{Acc} \leftarrow (\text{Acc}) * (x)$
DIV	x	$\text{Acc} \leftarrow (\text{Acc}) / (x)$

Rockwell's M6502 and Motorola's MC6809 are examples of accumulator-based processors. The MC6809 processor chip contains two 8-bit accumulator registers, ACCA and ACCB. In this system, these two registers may be combined to form one 16-bit accumulator for all 16-bit arithmetic and logic operations. The following example shows how the Pascal assignment statement $D := A + B * C$ can be written using these instruction formats:

THREE ADDRESS

MUL B,C,D ; $D \leftarrow (B) * (C)$
 ADD A,D,D ; $D \leftarrow (A) + (D)$

TWO ADDRESS

MOV B,D ; $D \leftarrow (B)$
 MUL C,D ; $D \leftarrow (C) * (D)$
 ADD A,D ; $D \leftarrow (A) + (D)$

ONE ADDRESS

LDA B ; $Acc \leftarrow (B)$
 MUL C ; $Acc \leftarrow (Acc) * (C)$
 ADD A ; $Acc \leftarrow (Acc) + (A)$
 STA D ; $D \leftarrow (Acc)$

A stack machine is a computer system in which the primary element of the CPU is a stack. To appreciate this organization, one must have some understanding of the basic concepts associated with the stack.

The stack is a data structure where data items are inserted and removed from the same end. Because of this restriction, the stack possesses the last-in first-out (LIFO) property. Consider the conceptual organization of the stack with three elements shown in Figure 1.6.

If two elements, D and E, are added to this stack, the stack will appear as shown in Figure 1.6b. At this point if one element is removed from the stack, item E, which was the last element entered, will be removed.

The stack is typically used for the following purposes:

- To implement subroutine calls and returns and to pass parameters from a main program to a subroutine.
- To handle interrupts.

These applications are covered in more detail later in the next chapter.

A stack may be implemented in hardware or software. A hardware stack with a storage capacity of p words can be obtained using p shift registers. The primary advantage of this organization is that the stack manipulations can be carried out at a high speed. High speed is a desirable feature in a computer that is required to provide a fast service for interrupts. This idea is implemented in the IBM 370 computer.

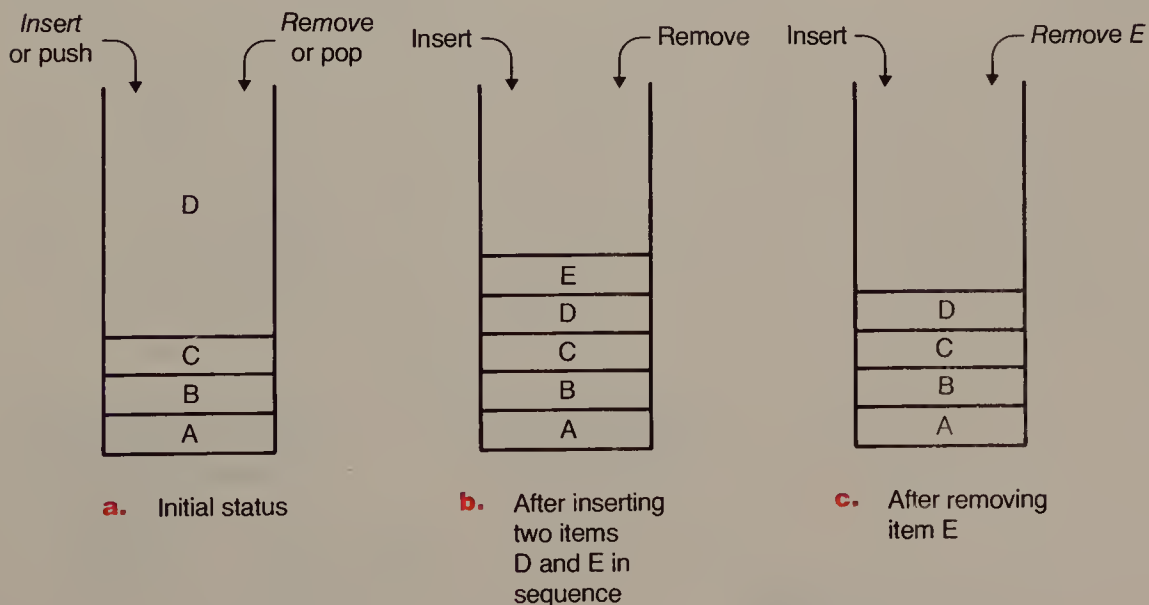


Figure 1.6 Three Instances of a Stack

The stack can be implemented by configuring some portion of the read-write memory as stack area. This is the key feature of the software stack. One of the CPU registers, called the stack pointer (SP), is used to hold the address of the most recently entered item into the stack. Hence, SP points to the top of the stack. This is illustrated in Figure 1.7.

There are two basic operations associated with the stack. These are PUSH (writing into the stack), and POP (reading from the stack). An item can be inserted into the stack by executing the PUSH instruction, which assumes the following general format:

PUSH <Mem adr>

The semantics of this instruction can be formally described as follows:

$SP \leftarrow SP - 1$; decrement the SP by 1
 $(SP) \leftarrow (\text{Mem adr})$; copy the contents of the specified memory address into the location whose address is the current contents of the SP

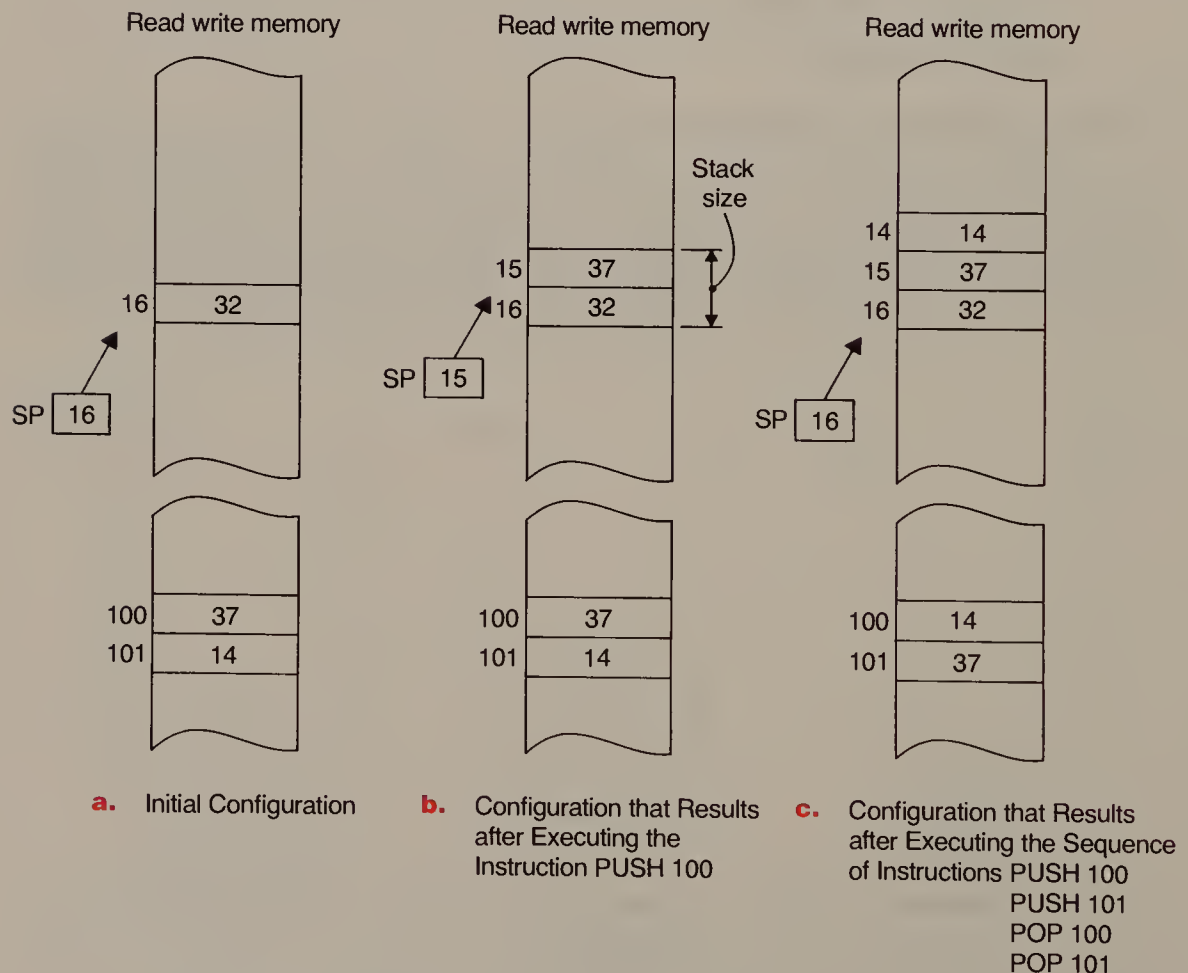


Figure 1.7 Basic Stack Operations

If the instruction PUSH 100 is executed in the configuration of Figure 1.7, a new configuration is obtained, as shown in Figure 1.7(b).

In this arrangement, the stack grows in the direction of lower addresses. This characteristic is typical of many processors, including the PDP-11, Intel 8085, Z80, and MC6809. In contrast, the HP3000 stack grows in the direction of higher addresses.

A data item can be removed from the stack by executing a POP instruction, whose general form is

POP <Mem Adr>

The semantics of this instruction can be formally expressed as follows:

<Mem Adr> $\leftarrow$ (SP) : Copy the contents of the memory location pointed to by the SP into the specified memory address

SP $\leftarrow$ SP + 1; Increment the Sp by 1

If the sequence of instructions

```
PUSH 100
PUSH 101
POP 100
POP 101
```

is executed in the configuration of Figure 1.7(a), a new configuration (Figure 1.7(c)) is obtained.

Close examination reveals that the preceding sequence has actually exchanged the contents of the memory locations 100 and 101.

In a software stack, the stack size is determined by the initial value stored in the SP. Since a programmer can initialize the SP using the instruction MOV initial value, SP;, stack size is a program-controlled parameter. Theoretically, the size of a stack can be as large as the total available space in the RWM.

The organization of a stack machine is shown in Figure 1.8.

In Figure 1.8, assume the stack is implemented using the software approach discussed earlier. In the event of a binary arithmetic or logic operation, the first operand will be removed from the stack and held in the register TEMP. The second operand will be directly routed from the stack to the right input of the ALU. The result of the operation is stored at the top of the stack. Typical instructions for a stack machine are summarized in Figure 1.9.

The following example illustrates how to model the Pascal assignment statement D: = A + B*C using the instruction set of Figure 1.9

```
PUSH A
PUSH B
PUSH C
MUL    ; calculate B*C
ADD    ; Add A to B*C
POP D  ; save result.
```

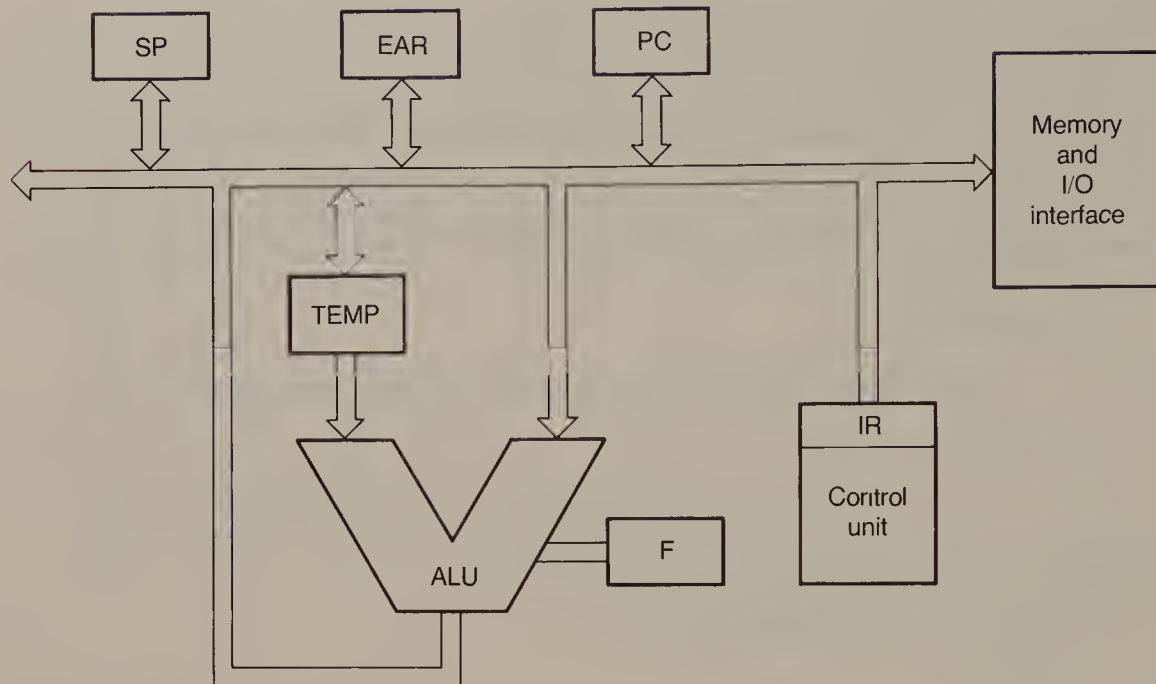



Figure 1.8 Organization of a Typical Stack Machine. (From J. F. Wakerly, *Microcomputer Architecture and Programming*, John Wiley and Sons, 1981. Reprinted by permission of John Wiley and Sons, Inc.)

One of the main advantages of the stack machine is its efficient evaluation of complicated arithmetic expressions. Typical examples of stack machines include Hewlett-Packard's HP3000 and Burroughs B1700. General register machines, such as the PDP-11 and MC68000, possess the capabilities to perform some stack operations. A programmer can simulate the behavior of the stack machine on these processors by writing a suitable program.

INSTRUCTION	OPERATION
PUSH X	$(TOS) \leftarrow (X)$
POP Z	$Z \leftarrow (TOS)$
ADD	$(NOS) \leftarrow (TOS) + (NOS)$
SUB	$(NOS) \leftarrow (TOS) - (NOS)$
MUL	$(NOS) \leftarrow (TOS) * (NOS)$
DIV	$(NOS) \leftarrow (TOS) / (NOS)$

Note: TOS = top of stack

NOS = next on the stack

Figure 1.9 Some Typical Stack Machine Instructions

1.5 TYPES OF COMPUTERS

Computers can be divided into five different types, based on cost and performance. These are supercomputers, maxicomputers, midicomputers, minicomputers, and microcomputers. Supercomputers are very powerful, extremely high-performance machines that cost millions of dollars. These computers are usually used for scientific computations. Cyber 205 and Cray X-MP are examples of this type. Supercomputers provide a tremendous amount of raw computing power for applications such as weather prediction, nuclear energy problems (where solutions of large partial differential equations are necessary), image processing, real-time speech recognition, and medical applications. Since the applications just mentioned usually need the processing of ordered data such as vectors or matrices, a speed advantage can be achieved by performing arithmetic operations in parallel. This is the basic concept behind the Vector processors such as Texas Instrument's Advanced Scientific Computer (TIASC), Control Data's STAR-100, Fijutsu's VP-200, and Data West's MATP.

Maxicomputers are high-performance, general-purpose computers that cost about \$1 million. CDC 7600 and the IBM 370 series are typical examples. These computers were the first ones to implement IC technology and multiprogramming operating systems.

Midicomputers are general-purpose computers costing about \$650,000. Examples of such systems are Digital's VAX 8600, Data General's Eclipse and IBM's 4381 Model group 2. These systems can directly address in excess of 16M bytes of main memory and are able to support more than 190 work stations. They are built using sophisticated technologies, such as gate arrays, and use high-speed memories, such as cache memories (see Chapter 5). These computers are frequently used in artificial intelligence projects and for running large CAD (computer-aided design) software.

Minicomputers are designed primarily for real-time dedicated applications or as high-performance, general-purpose computers. Digital's PDP-11/45 and PDP-11/70 are high-performance, general-purpose 16-bit minicomputers. These systems can typically support 20 users and allow implementation of popular operating systems such as UNIX. The PDP-11/02 and PDP-11/04 16-bit minicomputers are primarily used in dedicated applications. The cost of minicomputers ranges from \$25,000 to \$300,000.

Microcomputers are usually designed for dedicated real-time applications in a distributed system. The CPU of a microcomputer (the *microprocessor*) is usually fabricated in a single chip. With the advent of 16- and 32-bit microprocessors, it became feasible to use microprocessors as general-purpose computers. Today 32-bit, microcomputer systems usually cost around \$5000. Typical applications of microcomputers include the following:

- Controllers for appliances, video games, and automobiles.
- Distributed process control, data-acquisition systems, and associated fabricated interfaces.
- Smart terminals, hobby markets, office automation, word processing, and personal computers.

A realistic indication of classification involves scale or packaging, sophistication of software support from the manufacturer, and level of maintenance support.

1.6 FUTURE TRENDS

The advent of large-scale integration (LSI) technology has made it possible to fabricate a complete CPU on a single chip. Examples include Intel 8085 and Motorola 68000. This technology also allows expansion of the capacity of memory chips. It is expected that there will be significant impact of very large-scale integration (VLSI) technology on future architectures.

Using VLSI technology, 16- and 32-bit microprocessors, such as Motorola 68000, 68020, and Intel 80286 and 80386, have already been fabricated. The architectures of these various types of microprocessors do not deviate significantly from earlier processors, due to compatibility reasons. In fact, the Intel 80286 and Motorola 68000 have incorporated some of the best features of the IBM 370 and DEC PDP-11. Future microprocessors are expected to contain most of the features of the VAX 11/780. VLSI technology allows 32-bit microprocessors such as the 80386 and MC68020 to incorporate cache memory in the CPU chip. The cache memory concept is addressed in Chapter 5.

Distributed function architecture with special-purpose functional chips, such as text editors and language translators of these chips, will depend mostly on the cost of connecting various active elements. Computer engineers and scientists will be devising new algorithms and communication mechanisms for these computers with VLSI technology. The use of microprocessors implementing the CPUs of minicomputers and mid-computers will be more common. Radical changes in programming languages and methodologies will be required.

QUESTIONS AND PROBLEMS

- 1.1.** Explain the functions of a computer control unit.
- 1.2.** In the processors such as Texas Instrument's TMS 320XX (Digital signal processor series), there is a separate memory for code (program) and a separate memory to hold data. This type of architecture is known as the "Harvard architecture." Compare this with the Von Neumann architecture or stored-program architecture discussed in this chapter.
- 1.3.** Explain the functions of the following dedicated CPU registers: a) PC, b) IR, c) SP, d) Flag register.
- 1.4.** Using a minimum number of 0-address instructions, write a program that is equivalent to
 - a)** $Z := A * B + C * D + E * F$
 - b)** $Z := A * A + E * A + A * B + E * B$
- *1.5.** Assume there are four different machines with the characteristics shown in the table.

Machine name	Instruction set
Machine 0 (zero-address machine)	PUSHM POP M ADD SUB MUL DIV
Machine 1 (one-address machine)	LDA M ; ACC $\leftarrow$ (M) STA M ; M $\leftarrow$ (ACC) ADD M ; ACC $\leftarrow$ (ACC) + (M) SUB M ; ACC $\leftarrow$ (ACC) - (M) DIV M ; ACC $\leftarrow$ (ACC) / (M) MUL M ; ACC $\leftarrow$ (ACC) * (M)
Machine 2 (two-address machine)	MOV Y, X; X $\leftarrow$ (Y) ADD Y, X; X $\leftarrow$ (X) + (Y) SUB Y, X; X $\leftarrow$ (X) - (Y) MUL Y, X; X $\leftarrow$ (X) * (Y) DIV Y, X; X $\leftarrow$ (X) / (Y)
Machine 3 (three-address machine)	MOV Y, X ; X $\leftarrow$ (Y) ADD X, Y,Z; Z $\leftarrow$ (X) + (Y) SUB X, Y,Z; Z $\leftarrow$ (X) - (Y) MUL X, Y,Z; Z $\leftarrow$ (X) * (Y) DIV X, Y,Z ; Z $\leftarrow$ (X) / (Y)

In the preceding description, the symbol M refers to a 20-bit memory address. However, the symbols X, Y, and Z may refer to a 20-bit memory address or 3-bit register address. (There are 8 registers, R0 through R7.)

Assume the size of the op-code field is 8 bits. Compare these four machines by writing a program equivalent to:

$$Z := (A + B * C) / (D - E * F)$$

Summarize your results as follows:

	Machine 0	Machine 1	Machine 2	Machine 3
Number of instructions				
Program size (in bits)				
Number of distinct registers used				

- 1.6.** Consider the following programs that produce the same result

<i>Program 1</i>	<i>Program 2</i>
<i>PUSH A</i>	<i>PUSH A</i>
<i>PUSH B</i>	<i>PUSH B</i>
<i>ADD</i>	<i>PUSH C</i>
<i>PUSH C</i>	<i>ADD</i>
<i>ADD</i>	<i>ADD</i>
<i>POP Z</i>	<i>POP Z</i>

Among these two programs which one is superior? Explain.

- 1.7.** Suppose that the following instructions are added to the stack machine discussed earlier.

a) PUSHC $\langle \text{datavalue} \rangle$

This instruction pushes a constant data ($\langle \text{datavalue} \rangle$) into the stack.

b) TST

In this case we pop the top element of the stack (say B), and the next element of the stack (say A), and calculate the difference $A - B$. If the difference is 0, then we push a constant value 1 into the stack, otherwise we push a 0 into the stack.

c) BOT $\langle \text{address} \rangle$

Here, we pop the top element of the stack (say A). If $A = 1$, then we branch to the specified address, otherwise we simply execute the following instruction.

Using these instruction along with the others, write a program that is equivalent to the following Pascal fragments:

	<u>Fragment 1</u>	<u>Fragment 2</u>
if C = D	then $E := E + F$	$S := 0; I := 1;$
	else $E := E * F$	while $I < ID$ do
		begin
		$S := S + I;$
		$I := I + 1$
		end;

- 1.8.** The CPU of a computer contains 8 registers R0 through R7. The register R7 is a read-only register and it always contains a zero. The instruction set of this machine includes the following instructions:

ADD $R_i, X, R_j; \leftarrow (R_i) + (X)$ where $i = 0, 1, 2, \dots, 7$ but $j = 0, 1, 2, 4, 5, \text{or}, 6$.

ADD $R_i, R_j, X; (X) \leftarrow (R_i) + (R_j)$

where $i, j = 0, 1, 2, \dots, 6, 7$

Note: The instruction set does not have any explicit memory to register or register to memory load and store instruction.

Write a program segment that will add the contents of memory locations 2400 and 2500 and save the sum in memory location 2600.

Compare the performance of the minicomputer with that of the microcomputer.

Provide a brief note on the future trends in computers.

REFERENCES

1. Allison, D. R. "A Design Philosophy for Microcomputer Architectures." *IEEE Transaction on Computers* (February 1977).
2. Baer, J. L. *Computer System Architecture*. Rockville, Md.: Computer Science Press, 1980.
3. Bell, C. G., and A. Newell. *Computer Structures: Readings and Examples*. New York: McGraw-Hill, 1971.
4. Deitel, H. M. *Introduction to Operating Systems*. Reading, Mass.: Addison-Wesley, 1982.
5. Faggin, F. "How VLSI Impacts Computer Architecture." *IEEE Spectrum* (May 1978): 28–31.
6. Hayes, J. P. *Computer Architecture and Organization*. New York: McGraw-Hill, 1978.
7. Hayes, J. P. *Digital System Design and Micro-processors*. New York: McGraw-Hill, 1984.
8. Hill, F. J., and G. R. Peterson. *Digital Systems: Hardware Organization and Design*. New York: John Wiley, 1978.
9. Hwang, K., and F. A. Briggs. *Computer Architecture and Parallel Processing*. New York: McGraw-Hill, 1984.
10. Madnick, S. E., and J. J. Donovan. *Operating Systems*. New York: McGraw-Hill, 1974.
11. Meyer, R., and L. Seawright, "A Virtual Machine Time-Sharing System." *IBM System Journal* 9, no. 3 (1970).
12. Rafiquzzaman, M. *Microcomputer Theory and Applications with Intel SDK-85*. New York: John Wiley, 1987.
13. Rafiquzzaman, M. *Microcomputer and Microcomputer Development Systems: Designing Microprocessor Based Systems*. New York: Harper & Row, 1984.
14. Shaw, A. C. *The Logical Design of Operating Systems*. Englewood Cliffs, N. J. Prentice-Hall, 1974.
15. Tanenbaum, A. S., *Structured Computer Organization*, Englewood Cliffs, N. J.: Prentice-Hall, 1984.
16. Wakerly, J. F. *Microcomputer Architecture and Programming*. New York: John Wiley, 1981.
17. Weizer, N. "A History of Operating Systems," *Datamation* (January 1981):119–126.

CHAPTER 2

Computer Instruction Set

Computer architecture is defined as the study of the components and their interconnections that form a computer system. The computer supports the instruction types and data, which become primary architectural considerations. In this chapter some important characteristics and properties of computer instruction sets are discussed. Topics will include: op-code encoding, addressing modes, and instruction types.

2.1 INTRODUCTION

An instruction manipulates the stored data, and a sequence of instructions constitutes a program. In general, an instruction has two components:

- Op-code field
- Address field(s)

The op-code field specifies how data is to be manipulated. The data items may reside within a CPU register or in the main memory. The purpose of the address field is to indicate the data address. When operations require data to be read from or stored

into two or more addresses, the address field may contain more than one address. For example, consider the following instruction:

ADD	R1, R0
op-code field	address field

Assume that this computer uses R1 as the source register and R0 as the destination register. The preceding instruction then adds the contents of CPU registers R0 and R1 and saves the sum in register R0. The number and types of instructions supported by a computer vary from one computer to another and depend primarily on the architecture of a particular machine.

Depending on the number of addresses specified, one can have the following instruction formats:

- Three-address
- Two-address
- One-address
- Zero-address

Since all instructions are stored in the main memory, instruction formats are designed so that instruction sizes are optimized and have powerful processing capabilities. The CPU architecture has considerable influence on a specific instruction format. For example, zero-address instructions are very predominant in stack machines.

The following are some important technical points that have to be considered when designing an instruction format:

- The size of an instruction word is chosen by the designer to specify several operations. For example, with 4- and 8-bit op-code fields, 16 and 256 distinct operations, respectively, can be specified.
- Instructions are used to manipulate various data elements, such as integers, floating-point numbers, and character strings. In particular, all programs written in a symbolic language such as FORTRAN or Pascal are internally stored as characters. Therefore, memory space will not be wasted if the word length of the machine is some integral multiple of the number of bits needed to represent a character. Since all characters are represented using typical 8-bit character codes such as ASCII or EBCDIC, it is desirable to have an 8-, 16-, 32-, or 64-bit word length.
- The size of the address field is chosen to guarantee high resolution. In any computer, the ultimate resolution is a bit. Memory resolution is a function of the instructional length and, in particular, of short instructions that provide less resolution. For example, in a computer with 32K 16-bit memory words, at least 19 bits are required to access each bit of the word. However, the resolution achieved by some processors lies at the extremes. For example, the Burrough's B1700 processor achieves a 1-bit resolution (each bit is addressable). CDC's CYBER 70 series computer, however, has a minimum addressable unit of one 60-bit memory word.

Instruction	Meaning
LDA addr	$\text{Acc} \leftarrow (\text{addr})$
STA addr	$M(\text{addr}) \leftarrow (\text{Acc})$
ADD addr	$\text{Acc} \leftarrow (\text{Acc}) + (\text{addr})$
AND addr	$\text{Acc} \leftarrow (\text{Acc}) \wedge (\text{addr})$
CMA	$\text{Acc} \leftarrow (\text{Acc})'$
INCA	$\text{Acc} \leftarrow (\text{Acc}) + 1$
JMP addr	Unconditionally branch to addr
HLT	Halt CPU

Figure 2.1 A Hypothetical Instruction Set

2.2 OP-CODE ENCODING

A processor can execute an instruction only if it is represented as a binary sequence. A unique binary pattern must be assigned to each op-code. This process is known as *op-code encoding*. Some popular op-code encoding techniques are addressed in this section.

The simplest way to carry out op-code encoding is to assign a fixed length of binary pattern to each op-code. For example, a k -bit binary pattern can represent 2^k distinct op-codes. This method is known as the *block-code technique*. To illustrate this concept, consider the hypothetical instruction set shown in Figure 2.1. In this figure, there are 8 different instructions that can be encoded using a 3-bit binary pattern $p_2p_1p_0$ (Figure 2.2).

Op-Code	Binary Pattern
	$p_2p_1p_0$
LDA	000
STA	001
ADD	010
AND	011
CMA	100
INCA	101
JMP	110
HLT	111

Figure 2.2 Op-code Encoding Using a 3-bit Block Code

The op-codes of the hypothetical instruction set can be decoded using a 3-to-8 decoder such as the 74LS138 shown in Figure 2.3. An n -to- 2^n decoder is required for an n -bit op-code. As the value of n increases, the cost of the decoder and decoding time also increases.

Some op-code encoding techniques are considered in which the length of the op-code is a function of parameters such as the number of addresses or the relative frequency of its usage. The following approaches are discussed:

- Expanding op-code technique
- Huffman encoding

The rationale behind expanding op-code technique is to find a compromise between the instruction length and memory resolution. Consider an instruction in which the lengths of the op-code and address fields are 4 and 12 bits, respectively. Using such a format, 16 operations can be specified that will allow access to 4096 memory locations. If the size of the address field is increased to 13 bits and the instruction length is kept at 16 bits, the op-code length is three bits. However, this change will reduce the number of possible operations by 50%. At the same time, it will increase memory resolution by 100%. Conversely, the original instruction format can be changed so that the sizes of the op-code and address field are 5 and 11 bits, respectively.

With this new format, 16 more operations can be specified, which will be a 100% increase. However, this gain results in a 50% reduction in memory resolution (because the 11-bit address field allows access to only 2048 different memory locations). This is the basic concept behind the expanding op-code technique. The following example is provided to explain the usefulness of this approach.

Consider an instruction format with an instruction length and address field size of 8 and 3 bits, respectively. Only 4 distinct, two-address instructions can be formed because the op-code field has only 2 bits. This result is illustrated in Figure 2.4.

If there are 3 two-address instructions rather than 4, 8 one-address instructions can also be specified. This happens because each one-address instruction requires only one address field, and the other 3-bit address-field can be used for specifying the 8 op-codes. This idea is illustrated in Figure 2.5.

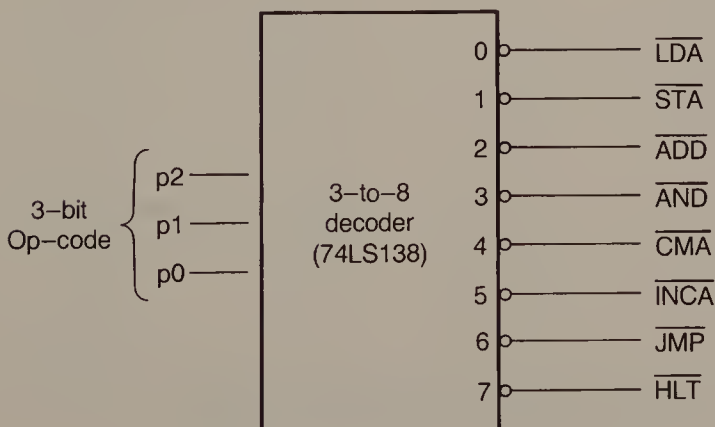


Figure 2.3 Instruction Decoder

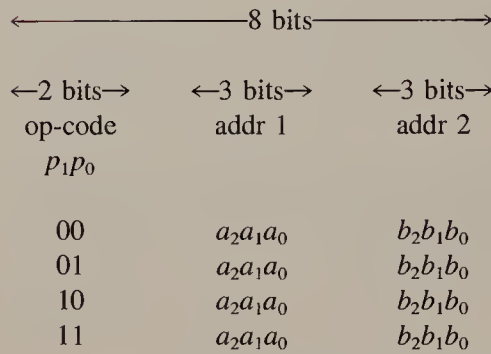


Figure 2.4 Four, two-address Instructions Are Derived Using a 2-bit Op-code Field

The length of the op-code for each one-address instruction is 5 bits. This means the length of the op-code field increases as the number of address fields is decreased. For this reason, this technique is referred to as the expanding op-code technique.

In a typical instruction set, it is necessary to include some zero-address instructions. Suppose the number of one-address instructions is reduced from 8 to 7. Then up to 8 zero-address instructions can be accommodated in the same instruction format. This is illustrated in Figure 2.6.

For zero-address instructions, 8 bits are used for op-code specification. The expanding op-code technique is employed to encode the instruction set of the PDP-11 computer.

Huffman's encoding scheme is discussed next. The block-code technique assumes all instructions are used with equal probability. In practice, not all instructions are used with the same relative frequency count. On the average, 40% of the instructions used in a program are load and store instructions. This pattern is similar to the occurrence of

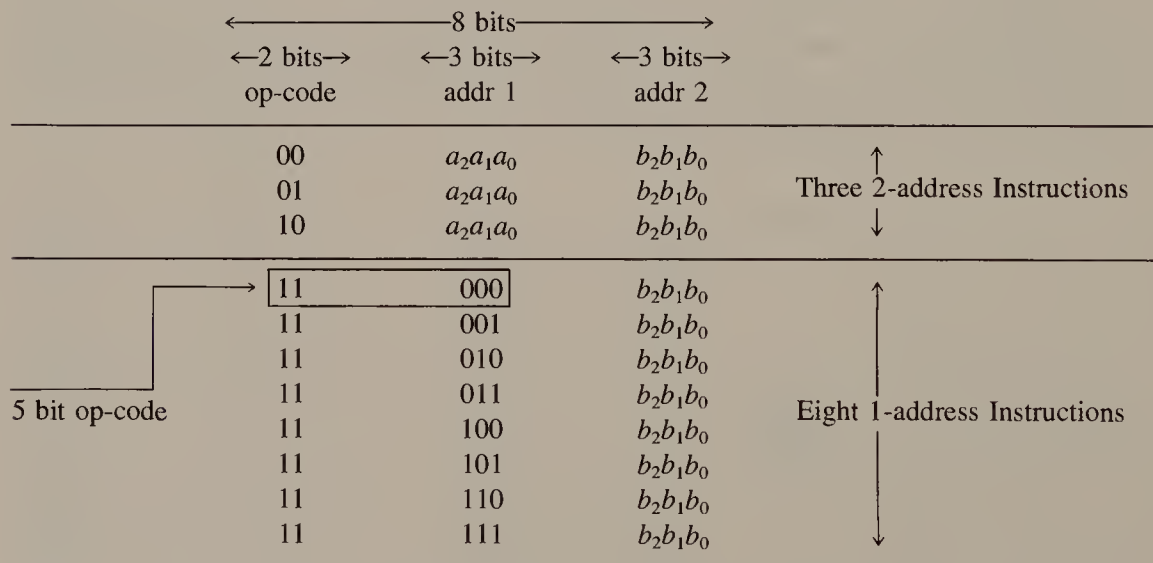


Figure 2.5 Three 2-address and Eight 1-address Instructions Using 8-bit Instruction Format

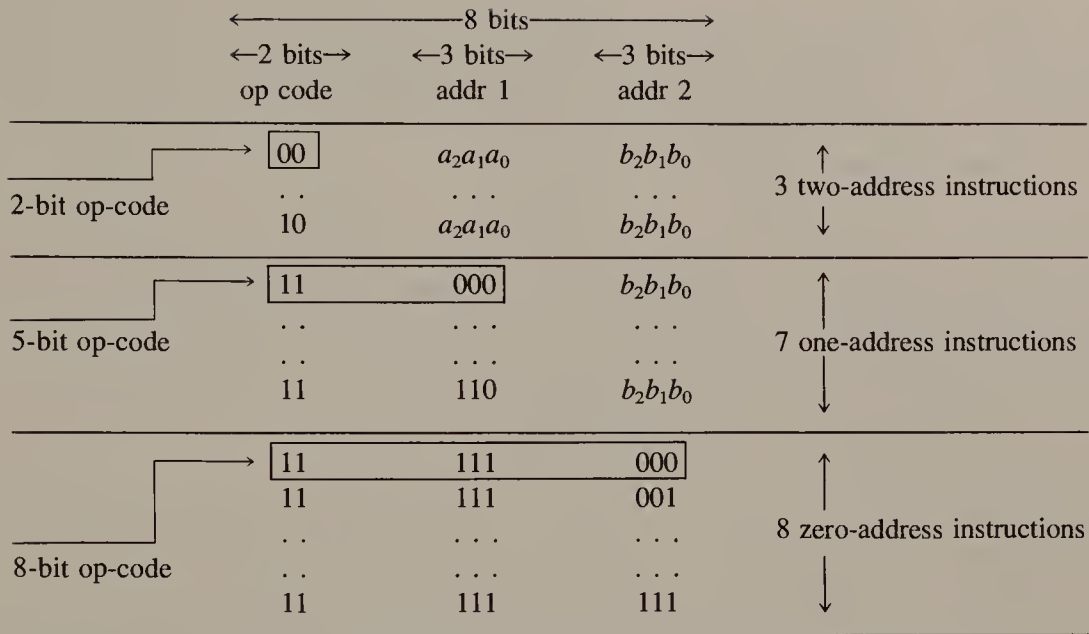


Figure 2.6 Illustration of 3 two-address, 7 one-address, and 8 zero-address instructions in an 8-bit instruction format

vowels in an English text. Therefore, this study warrants an encoding scheme that will encode the op-code of the most frequently used instructions with fewer bits and the least frequently used instructions with more bits. This allows the average number of bits required to encode a typical program to be optimum.

Huffman's procedure carries out this idea, as explained by the following example. Suppose it is desired to encode the hypothetical instruction set shown in Figure 2.7. A

Instruction Mnemonic	Relative Frequency Count
LOAD	$\frac{1}{4}$
STO	$\frac{1}{4}$
ADD	$\frac{1}{8}$
AND	$\frac{1}{8}$
NOT	$\frac{1}{16}$
RSHIFT	$\frac{1}{16}$
JUMP	$\frac{1}{16}$
HALT	$\frac{1}{16}$

Figure 2.7 An Instruction Set with Relative Frequency Count

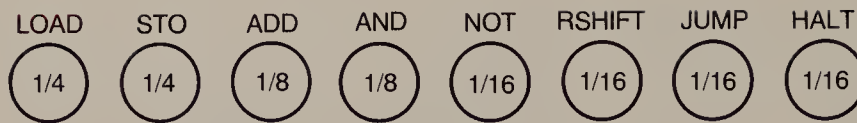


Figure 2.8 Initial Arrangement of Instruction Mnemonics

relative frequency count of each instruction is shown in this figure. These values are obtained by inspecting the occurrence of each instruction in a set of representative programs. In this procedure, first arrange the instructions to obtain a graph, as shown in Figure 2.8. In this graph, there is one node for each instruction mnemonic, and these nodes are labeled with the corresponding relative frequency count. The nodes are arranged in the ascending order of their relative frequency counts.

Next, scan all the nodes of the graph and select two nodes with minimum values. Create a new node with a value equal to the sum of these minimum values. Exclude the two nodes that were picked up from the subsequent scanning process. This result is shown in Figure 2.9. This figure is developed by scanning the nodes of Figure 2.8. Here, the nodes corresponding to the instruction mnemonics JUMP and HALT are shown to have the least values. So these two nodes are selected, and new node with a value $\frac{1}{8}$ ($\frac{1}{16} + \frac{1}{16} = \frac{1}{8}$) is created. After this, these nodes are excluded from the subsequent process by crossing them out. The nodes corresponding to the mnemonics NOT and RSHIFT or RSHIFT and JUMP could have been chosen. When there are several possibilities, the choice is arbitrary. The important aspect of this procedure is to select two nodes with minimum values and form a new node with a value equal to the sum of the two chosen nodes. If the scanning process is continued, a new graph develops, as shown in Figure 2.10.

This scanning process is continued until a single node with a value equal to 1 is formed. At this point, a tree is formed, as shown in Figure 2.11.

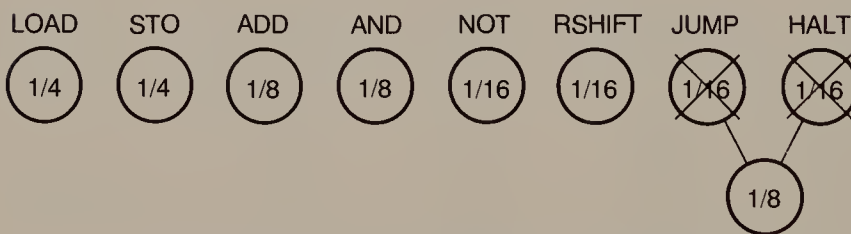


Figure 2.9 The Result of the Initial Scanning of the Nodes in Figure 2.8

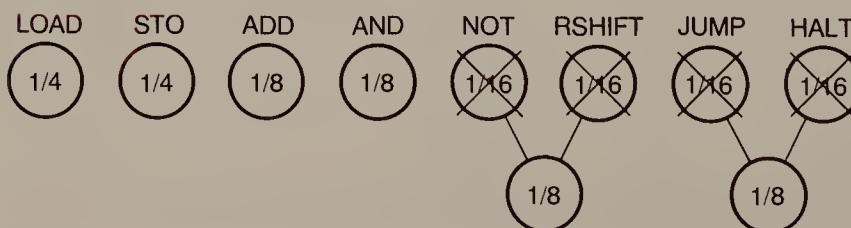


Figure 2.10 The Result of Scanning of Nodes of the Figure 2.9

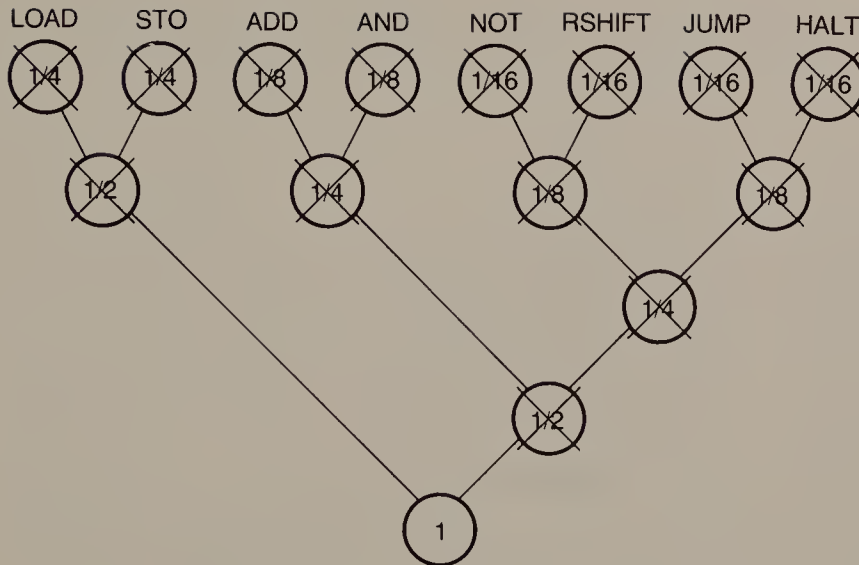


Figure 2.11 The Tree Obtained by Continuously Scanning the Graph of Figure 2.10

Now the right and left branches of this tree are labeled with 0 and 1, respectively, to obtain the Huffman tree shown in Figure 2.12.

To find the op-code for a mnemonic, a path from the root to the leaf node corresponding to this mnemonic must be found; the 0s and 1s are picked up from the path. For example, the op-code corresponding to the mnemonic LOAD in Figure 2.12 is 11. Starting from the root, first move to the left; and move again to the left to reach the node corresponding to LOAD. The values on the left branches are 1. The op-codes for other mnemonics can be found in a similar manner; they are tabulated as follows:

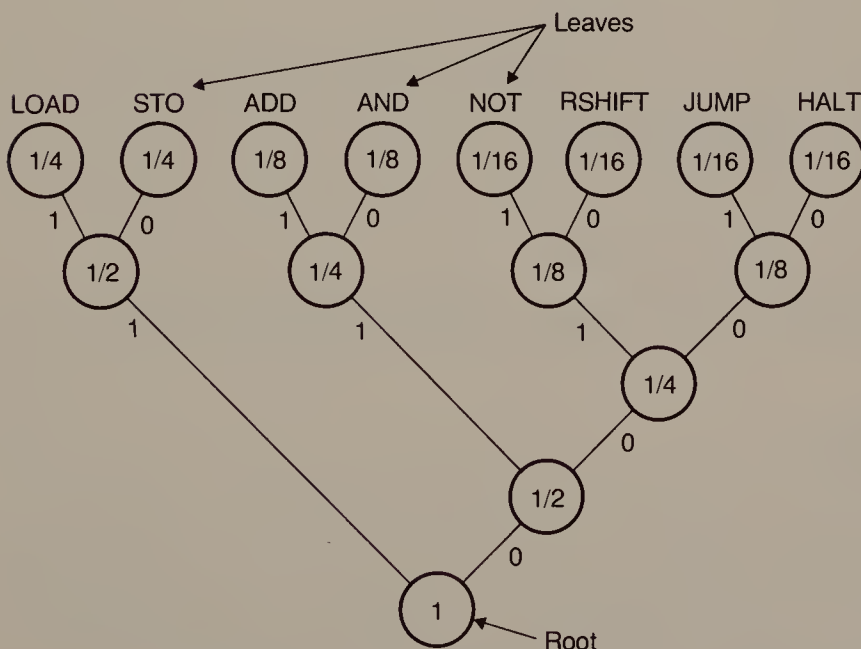


Figure 2.12 Huffman Tree

MNEMONIC	OP-CODE	PATH FROM THE ROOT
LOAD	11	Left→left
STO	10	Left→right
ADD	011	Right→left→left
AND	010	Right→left→right
NOT	0011	Right→right→left→left
RSHIFT	0010	Right→right→left→right
JUMP	0001	Right→right→right→left
HALT	0000	Right→right→right→right

From the preceding result, it is easy to see that Huffman's procedure encodes the most-frequently used instructions with short op-codes and the least-frequently used with long op-codes.

The average number of bits needed per instruction can be calculated using the formula

$$\sum_{i=1}^n l_i f_i \quad 2.1$$

Where l_i and f_i are op-code length and the relative frequency count of the i th instruction, respectively. For this example, the average number of bits is:

$$\begin{aligned}
 & 2(1/4) + 2(1/4) + 3(1/8) + 3(1/8) + 4(1/16) + 4(1/16) + 4(1/16) + 4(1/16) \\
 &= 1/4(2 + 2) + 1/8(3 + 3) + 1/16(4 + 4 + 4 + 4) \\
 &= 1 + 3/4 + 1 = 2.75 \text{ bits}
 \end{aligned}$$

Using the block-code scheme, each instruction will be encoded with a 3-bit op-code ($2^3 = 8$), giving an average value of:

$$\begin{aligned}
 & 3(1/4 + 1/4) + 3(1/8 + 1/8) + 3(1/16 + 1/16 + 1/16 + 1/16) \\
 &= 1.5 + 0.75 + 0.75 = 3.0 \text{ bits}
 \end{aligned}$$

From the information theory, the optimum number of bits needed to encode a set of messages is

$$- \sum_{i=1}^n f_i \log_2(f_i) \quad 2.2$$

The difference between the actual average length and the optimum length is called *redundancy* (R), and it can be written as

$$R = \frac{\text{actual length} - \text{optimum length}}{\text{actual length}} \quad 2.3$$

If equation (2.2) is applied to the example, the following results:

$$\begin{aligned}
 & -[2^{(1/4)}\log_2(1/4) + 2^{(1/8)}\log_2(1/8) + 4^{(1/16)}\log_2(1/16)] \\
 & = -[-1 - 0.75 - 1] \\
 & = 2.75 \text{ bits}
 \end{aligned}$$

If equation (2.3) is applied to the result of Huffman's scheme, R is found to be zero. However, the block-code scheme introduces a redundancy of $1/12$, or 8.33%.

Therefore, Huffman's scheme achieves an optimal result by keeping the redundancy to a minimum value. However, when op-codes are encoded using Huffman's scheme, the decoding process takes more time because a search must be conducted on the Huffman tree. This idea is used in Burrough's B1700 computer. The op-codes of the Zilog's Z 80 microprocessor's instruction set are encoded using a scheme that is very close to the Huffman's scheme. Even though block-code encoding takes extra storage space, it is widely used because of the simplicity of the decoding procedure.

2.3 ADDRESSING MODES

A processor executes a sequence of instructions in the following manner:

```

begin
    read next instruction from the memory
    decode the instruction and recognize the type
      of operation
    while the required operation is not a halt op-
      eration do

begin
    determine operand addresses
    retrieve the operands
    perform the desired operation
    determine the destination address
    save the result of the operation in the desti-
      nation
    read the next instruction from memory
    decode the instruction and recognize the type
      of operation
end
    while there is a hardware reset do

    skip    (/ this is a dummy statement that allows /)
           (/ the processor to execute an infinite loop/)
end

```

The sequence of operations that a processor has to carry out while executing an instruction is called its *instruction cycle*. The most important activity in an instruction cycle is the determination of the addresses of the operands involved in that instruction.

The manner in which a processor accomplishes this task is called the *addressing mode*. The typical addressing modes supported by the instruction sets of popular contemporary processors such as the 8085, Z 80, MC6809, MC68000, PDP-11, and VAX-11 will be examined.

An instruction is said to have an *inherent* addressing mode if its op-code indicates the address of the operand, which is usually the contents of a CPU register. For example, consider the following instruction:

STC. Set the carry flag in the status register. Since the op-code implies the address of the operand, the processor does not have to compute the operand address. This mode is very common with 8-bit microprocessors such as the 8085, Z 80, and MC6809.

Whenever an instruction contains the operand value, it is called an *immediate-mode* instruction. For example, consider the following instruction:

Add #25, R1; $R1 \leftarrow R1 + 25$

In this instruction, the symbol # indicates that it is an immediate-mode instruction. This convention is adopted in the assemblers for processors such as the MC6809, MC68000, PDP-11, and VAX-11. In these systems, the machine representation of this instruction occupies two consecutive memory words:

The first word holds the op-code, whereas the next word holds the data value. (For the preceding case, it is 25).

To execute this instruction, the processor has to access memory twice.

An instruction is said to have an *absolute addressing mode* if it contains the address of the operand. For example, consider the following move instruction:

MOV @#5000, R2; $R2 \leftarrow (5000)$

This instruction copies the contents of memory location 5000 in the CPU-register R2. As in the previous case, the object code corresponding to this instruction occupies two consecutive memory words. However, in this case, the contents of the memory word that follows the op-code is interpreted as the address of the operand. To execute this instruction, the processor has to make three memory accesses: one for the op-code, one for the address, and one for the data. The execution time of an absolute-mode instruction is more than the corresponding immediate-mode instruction.

An instruction is said to have a *register mode* if it contains a register address as opposed to a memory address. In this mode, the operand values are held in the CPU registers. For example, consider the following register mode and instruction:

ADD R2, R3; $R3 \leftarrow R2 + R3$

In the register-addressing mode, the effective address (EA) of an operand is a CPU register. Since many contemporary CPUs have a small number of registers, the machine representation of a register mode instruction requires only a few bits. Memory space can be conserved by using register-mode instructions. In this mode, the processor does not require any memory reference for data retrieval. Hence, the instruction exe-

cution rate can be considerably increased. Since there is always a limit on the number of CPU registers, it is not possible to handle a large number of operands by the exclusive use of the register-addressing mode. However, having a CPU with large number of registers is a key characteristic of the reduced-instruction-set computers (RISC). The RISC architecture is discussed in a later section of this chapter.

Whenever an instruction specifies the address of a CPU register that holds the address of an operand, the resulting addressing mode is known as the *register indirect* mode. From this definition, it follows that the EA of an operand in the register-indirect mode is the contents of the CPU register R. More formally, this result is written as follows:

$$EA = (R)$$

To illustrate this idea clearly, consider the following instruction:

MOV (R2), (R3); (R3) $\leftarrow$ (R2)

Assume that the following configuration exists:

(R2)	=	5000 ₁₆
(R3)	=	4000 ₁₆
(5000)	=	1256 ₁₆
(4000)	=	4629 ₁₆

This instruction copies the contents of the memory location, whose address is specified by the CPU register R2, into the location whose address is specified by the CPU-register R3. Thus, after the execution of this instruction, the memory location 4000 will contain the value 1256. Whenever a CPU register is used as a data pointer, the assembler convention is to enclose that register using a set of parentheses. Alternatively, an indirect register may be specified by using the prefix @:

MOV, @R2, @R3

This addressing mode is very useful whenever there is a need to manipulate two different memory areas.

Two variations of the register indirect mode are the *auto-increment* and *auto-decrement* modes. In auto-increment mode, first the contents of the specified CPU register are used as the address of the operand, and then data transfer takes place. After this, the register contents are automatically incremented by some constant k . To indicate this mode, the register involved will be enclosed by parentheses and immediately followed by the plus sign. For example, consider the following instruction:

MOV (R2) +, R3

In this instruction, the source operand is in the auto-increment mode, and the action taken by this instruction can be described as follows:

$$R3 \leftarrow (R2)$$

$$R2 \leftarrow R2 + k$$

The auto-decrement mode is similar to the auto-increment mode, except the contents of the specified CPU register are first decremented by k , and the resulting value is used as the address of the operand. This action is symbolically represented by surrounding the CPU register with parentheses together with a minus sign just before the left parentheses. For example, consider the auto-decrement mode clear instruction:

CLR – (R5)

This instruction can be precisely described as follows:

$R5 \leftarrow R5 - k$

$(R5) \leftarrow 0$

The constant value k used in the auto-increment and auto-decrement modes is actually a function of the number of bits involved in the data transfer. Typically, this value is 1 for 8-bit, 2 for 16-bit, and 4 for 32-bit operands.

These modes are useful in array manipulations. For example, assume the CPU registers R2 and R3 are initialized with the starting addresses of two arrays X and Y, respectively. Then, the following instruction transfers the first element of the array X into the first element of the array Y:

MOV (R2) +, (R3) +

After this data transfer, the CPU registers are automatically pointed to the next corresponding elements of these arrays. If the system does not support an auto-increment mode, then the same result can be obtained by using a sequence of three instructions:

MOV (R2), (R3)

ADD #k, R2

ADD #k, R3

The auto-increment mode allows one to write programs that are more efficient both in terms of space and time.

By using the stack pointer in auto-decrement and auto-increment modes, PUSH and POP operations can be obtained. For example, consider the following instructions:

MOV R2, – (SP)

MOV R3, – (SP)

MOV R4, – (SP)

This sequence pushes the contents of the CPU registers R1, R2, and R3 into the stack. To restore the register contents, three successive POP operations are performed, as follows:

MOV (SP) +, R4

MOV (SP) +, R3

MOV (SP) +, R2

Another important concept used in the context of addressing modes is address modification. In this approach the EA of an operand is expressed as the sum of two parameters: reference address (RA) and modifier (M), formally written as:

$$EA = RA + M$$

The modifier M is also called the *offset*, or displacement. Such an address-modification principle is the basic concept associated with the following addressing modes:

- Indexed mode
- Base-register mode
- Relative mode

In the indexed mode, the value of RA is included in the instruction, and a CPU register contains the value M. The CPU register X is called the *index register*. This mode is useful for accessing arrays. For example, consider the following Pascal integer array y:

```
var  
    y: array [0..9] of integer;
```

Assume that each element of this array requires 1 byte and that the entire array is configured in the memory as shown in Figure 2.13. From this figure, notice the array starts at the memory address 0100. Now, assume the index register X contains the value 0002, and execute the following indexed-mode load instruction:

```
LDA 0100 (X)
```

This instruction indicates that the register X is the index register. Its machine representation includes the reference address, 0100, which is the starting address of array Y (see Figure 2.13). For this situation, the EA of the operand is:

$$\begin{aligned} EA &= RA + X \\ &= 0100 + 2 \\ &= 0102 \end{aligned}$$

Therefore, when this instruction is executed, the contents of memory location 0102 are transferred to the A register. This memory address actually holds the array element y [2]. Since the register X contains the required index value of the array element, it is referred to as the index register. To access the third element of array y, register X needs to be incremented by 1. This operation can be performed quickly. Therefore, the indexed mode allows a programmer to carry out array manipulations in an efficient manner.

In the base-register addressing mode, the parameter RA is held in a separate register called the base register, and the modifier M is included in the instruction. This mode is very significant in a system that provides a virtual memory support. In particular, the base-register mode has application in segmented memory systems. In these systems, the base register holds the base address (or the starting address) of a segment.

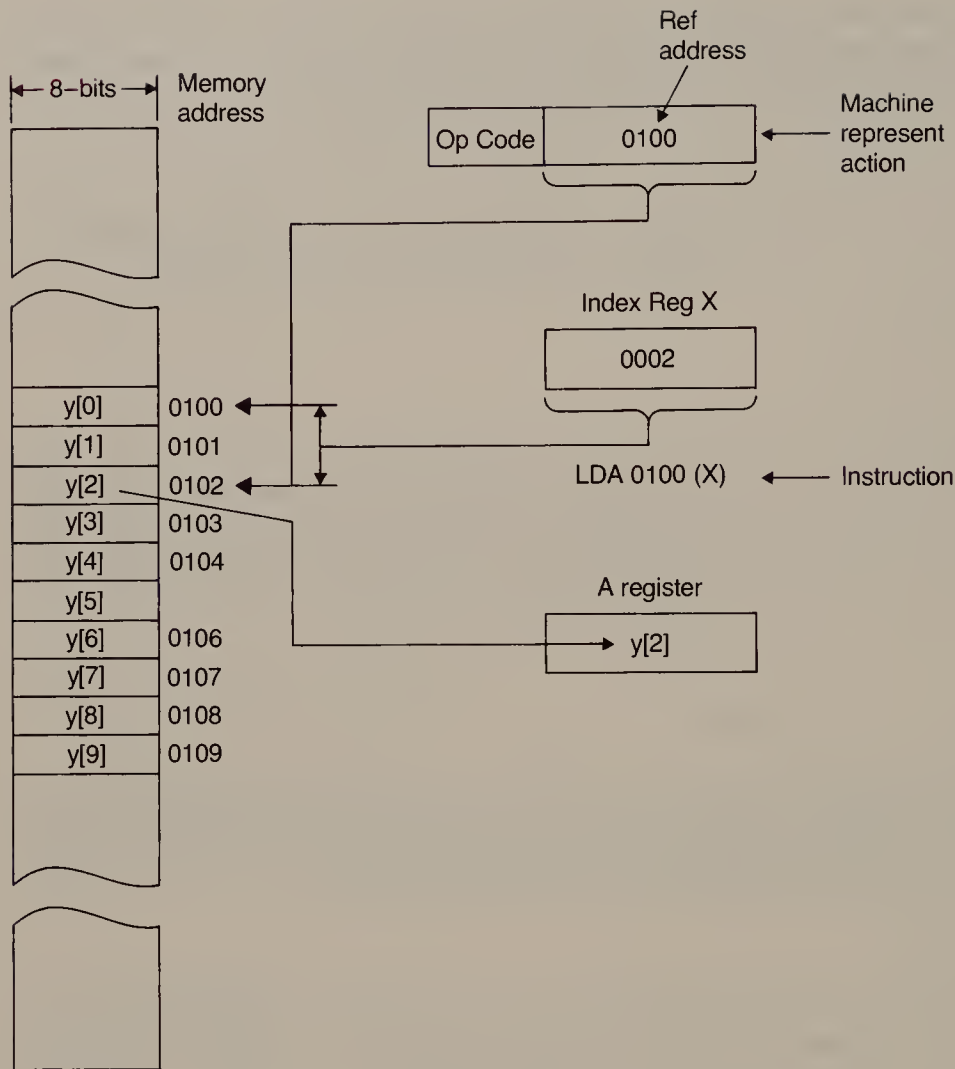
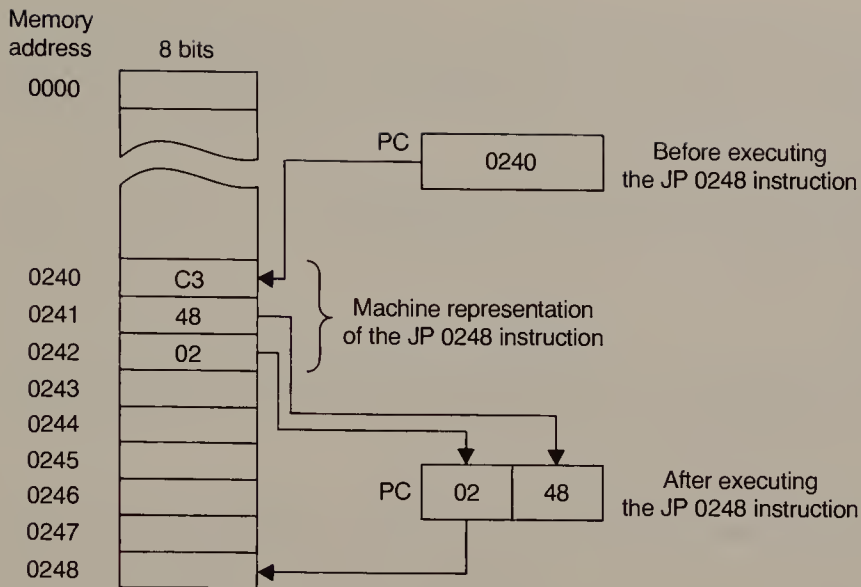


Figure 2.13 Use of the Indexed Addressing Mode in Accessing Arrays

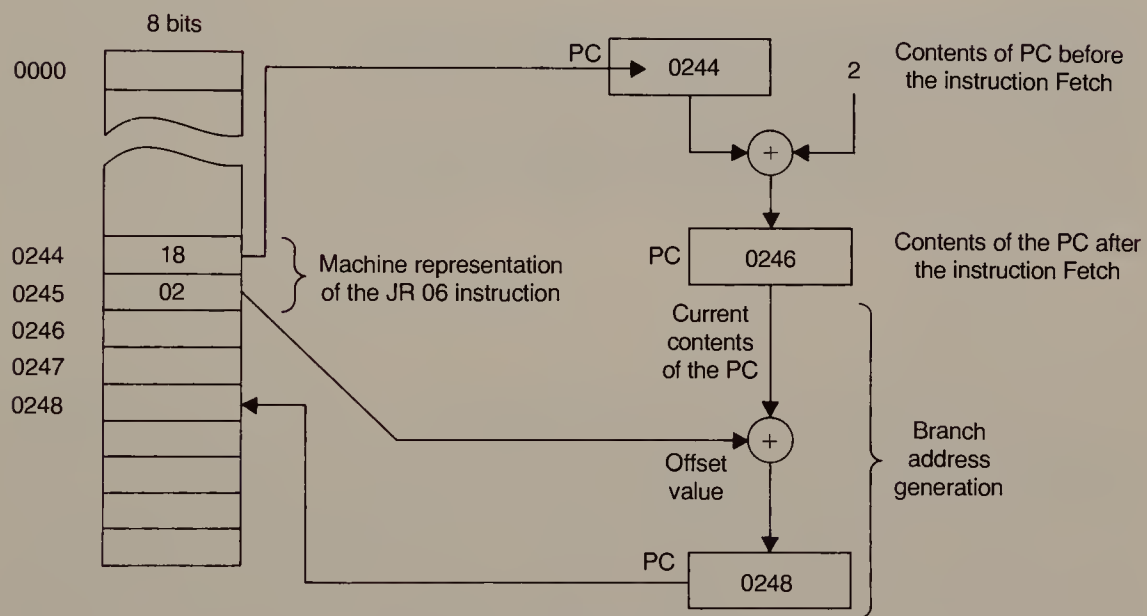
In general, the size of the indexed mode modifier (M) is the same as the number of bits required to specify a memory address. In the base-register mode, the number of bits in the modifier field (M) may be less than the number of bits required for a direct memory reference.

The contents of the M field are often interpreted as a 2's complement number. Whenever the sizes of the modifier and the memory address fields are unequal, the sign-extended form of M is used in the effective address calculation.

If the PC is configured as the base register, the *relative addressing mode* results. This mode is particularly useful in designing short branch instructions. For example, consider the Z 80 branch instruction JP 0248. This instruction is equivalent to the high-level language statement GOTO 0248. The machine representation of this instruction requires 3 bytes: 1 byte for the op-code (C3) and 2 bytes for the branch address (0248). Assume this instruction is stored as shown in Figure 2.14(a). Since PC always holds the address of the next instruction to be executed, implementation of a branch instruction implies loading the PC with the branch address.



a. Operation of an Absolute Jump Instruction



b. Operation of a Relative Branch Instruction

Figure 2.14 Mechanics of an absolute and a relative branch instructions (All Numbers Are in Hex Decimal Form)

Alternatively, consider the Z 80 relative branch instruction JR 02. The numerical value 02 represents the value of the modifier or offset. The machine representation of this instruction required only 2 bytes: 1 byte for the op-code (18) and 1 more to hold the offset value 02. If we assume that this instruction is stored as shown in Figure 2.14(b), the execution of this instruction can be explained as follows:

- First, the entire instruction is fetched.
- Since this is a 2-byte instruction, after instruction Fetch, the PC will contain 0246 (0244 + 2).
- The branch address is computed as follows:

$$\begin{aligned}
 \text{branch} &= \text{current contents of the PC} + \text{offset value PC} \\
 &= 0246 + 02 \\
 &= 0248
 \end{aligned}$$

- Finally, the PC is loaded with this branch address, and the next instruction to be executed is located in the memory location 0248.

When the offset is a negative number, reverse branching takes place. For example, consider JR 06 with all other data (in hex) as in Figure 2.14(b). In this case, 06 will be subtracted from 0246. Since this is a subtraction of an 8-bit signed number from a 16-bit signed number, the correct result is obtained if the 8-bit number (−06) is sign-extended and then subtracted from 0242 using 2's complement (ignoring the final carry) as follows:

$$\begin{array}{rcl}
 0246 & = & 0000 \quad 0010 \quad 0100 \quad 0110 \\
 \text{2's complement} & = & \boxed{1111 \quad 1111} \quad 1111 \quad 1010 \\
 \text{of 6} & & \\
 \hline
 & 1 & \underbrace{0000} \quad \underbrace{0010} \quad \underbrace{0100} \quad \underbrace{0000} \\
 & \nearrow & 0 \quad 2 \quad 4 \quad 0 \\
 & \text{Ignore carry} & \\
 & \text{Sign extension} &
 \end{array}$$

Since the offset value is an 8-bit quantity, only −128 to 127 bytes can be branched relative to the current contents of the PC. In most computers, conditional branching instructions use the relative mode. If one needs to go beyond the −128 to +127 range with conditional branching, then the unconditional branching instruction can be used anywhere in this range to branch to any location within the computer's directly addressable memory. Another important aspect of the relative addressing mode is its ability to produce a relocatable program. For example, consider the situation in Figure 2.14(b). In this configuration, assume that the branch address is 2 bytes away from the current contents of the PC. This means when this program is relocated, the JR 02 instruction will be placed in a different location, and different PC contents will result. Nevertheless, the program execution will produce the same result because the system identifies the branch address as the one located 2 bytes away from the current contents of the PC.

The usefulness of various addressing modes will now be discussed. The implementation of a particular addressing mode is largely dependent on the processor organization. In a processor with many general registers, indexed and indirect addressing modes are easy to obtain. If a computer supports powerful addressing modes, the task

of designing language translators, operating systems, and efficient application programs can be greatly simplified. For example, the auto-increment and auto-decrement modes allow one to implement stack operations and program loops efficiently. For a compiler designer, efficient stack operations are very important for implementing procedure calls.

The indexed addressing mode allows an applications programmer to manipulate arrays, whereas absolute and relative modes allow the programmers to write position-independent programs. A program is said to be *position-independent* if it can be placed anywhere in memory. This is a desirable feature for operating system designers because it allows the operating system to relocate programs in a dynamic manner. In a multiuser system, many different users may need the same library program provided by the operating system. If this library routine is position-independent, the operating system can load the machine code of this routine into any available portion of the main memory.

2.4 INSTRUCTION TYPES

The purpose of a computer instruction set is to provide a virtual machine with all features so that compilers, operating systems, and library subroutines can be designed quickly (see Figure 2.15).

From Figure 2.15, it can be seen that a program using the instruction set is translated into the machine code by the translator program *assembler*. Finally, the machine code is interpreted by the microcode so that the hardware produces the desired result. In other words, the microcode directs the hardware by generating necessary commands (control signals). In general, instructions available in a processor may be broadly classified into six groups. These are:

Data transfer instructions

Arithmetic instructions

Logical instructions

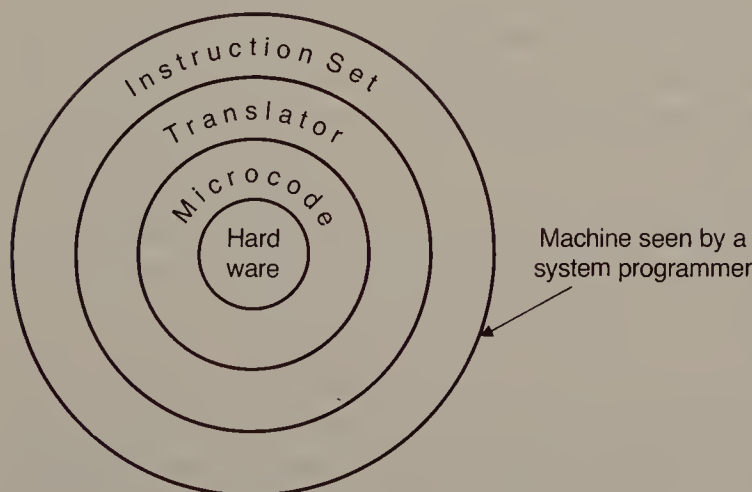


Figure 2.15 Creation of a Virtual Machine via Instruction Set

Program-control instructions

System-control instructions

I/O instructions

These instruction types are discussed in the following sections.

2.4.1 Data-transfer Instructions

Data-transfer instructions are concerned primarily with data transfers between the processor and main memory. Typically, an ideal instruction set must be able to handle the following transfers:

- Register to register
- Register to memory
- Memory to register
- Memory to memory

In contemporary computers, the system architecture is configured so the preceding possibilities can be implemented in an efficient manner. For example, in a three-bus architecture, simultaneous data transfers can occur. Various bus architectures are included in Chapter 4. This group is given primary consideration while designing the control unit because results reported by D. E. Knuth [5] and S. H. Fuller [2] have shown that, on the average, 40% of the instructions in many user programs are data-transfer instructions.

2.4.2 Arithmetic Instructions

All instruction sets typically include ADD and SUBTRACT instructions. The instruction sets of the VLSI microprocessors such as the Intel 8086 and Motorola's MC68000/68020 include multiplication and division instructions. The CPUs of these processors must include provisions for performing multiplication and division. The design of the hardware required for these operations is covered in Chapter 3. Many data-processing applications require the system to manipulate decimal data quickly. For this purpose, the instruction set of modern processors include instructions that can exclusively process BCD numbers. For example, the ABCD instruction of the MC68000 processor is capable of adding two BCD numbers stored in the main memory. Similarly, floating-point arithmetic operations are vital with respect to scientific programming. To facilitate this manipulation, some processors include floating-point instructions. For example, the extended instruction set of the PDP-11/70 computer includes instructions such as MULF and DIVF to multiply and divide floating-point numbers, respectively. To speed up the floating-point arithmetic operations, the PDP-11/70 CPU includes 6 additional 32-bit general-purpose floating-point registers (FO through F5) and the floating-point coprocessor, FP-11. With the advent of VLSI technology, it is possible to obtain numerous coprocessors compatible with microprocessors. For example, the I8087, AM9511, and

MC68881 coprocessor chips are compatible with I8086, I8085, and MC68000 microprocessors, respectively. Likewise, Motorola's floating-point ROM chip, the MC6839, can be interfaced with the MC6809 microprocessor to handle floating-point computations.

In general, the following assignment statements are common with respect to high-level language:

X: = X + 1;

Y: = Y - 2;

Z: = 2;

As a consequence, the instruction repertoire of the state-of-the-art processors includes special instructions to increment or decrement by small quantities and to assign a small value to a specified memory address. According to the Motorola designers, these instructions are called *quick mode* instructions. With VAX-11 computers, these instructions are referred to as *short immediate* or *literal* mode instructions. For example, the assignment statement Z: = Z - 4 can be implemented in the MC68000 and VAX-11 assembly languages as follows:

MC68000

LEA Z, A2	;	Load the address of Z into the address register A2
SUBQ.B #4, (A2)	;	Subtract 4 from the memory byte whose address is; specified in the register A2.

↑

This suffix indicates
that this is a quick
mode instruction.

VAX-11

MOVEAB Z, R6	;	Move the address of Z into the CPU
	;	register R6
SUBB S^#4 (R6)	;	Subtract 4 from the memory
	;	byte whose address is specified in the register R6.

↙ ↘

This prefix indicates
that the operand
is in the literal
mode.

2.4.3 Logical Instructions

Invariably, the instruction sets of all contemporary processors include instructions to perform Boolean AND, OR, NOT, and EXCLUSIVE-OR operations on a bit-by-bit basis. These instruction sets also include the following shift instructions:

- Arithmetic shift (left or right)
- Logical shift (left or right)
- Rotate shift (left or right) through or without the carry flag

In fact, the MC68000 and the VAX-11 instruction sets include even more elegant shift instructions. For example, consider the following shift instructions:

MC68000

LSR.W #3,D5 This instruction performs a left logical shift of the low-order 16-bits of the data register D5 by three places.

VAX-11

ROTL #16,R7

These instructions rotate the contents of the CPU register R7 to the left by 16 places. After this instruction is executed, the low-order 16-bits and the higher-order 16-bits of the 32-bits register R7 are exchanged. The VAX-11 instruction set also allow a programmer to perform memory-to-memory compare operations. The test instruction tests the specified operand with zero and sets the zero and sign flags, depending on the status of the tested operand. These flags are held in a dedicated register called condition code register. Its design is covered in the next chapter.

2.4.4 Program-control Instructions

In a conventional computer, instructions are always executed in the same order they are presented. In reality, the flow of control depends on the result of a computation. In this situation, a program can select a particular sequence of instructions to execute based on the results of a computation. Instructions that perform this are called *program-control instructions*. In general, these instructions may be classified into four groups:

- Unconditional branch instructions
- Conditional branch instructions
- Subroutine call instructions
- Interrupt-handling instructions

An unconditional branch instruction transfers the control to the specified address regardless of the status of the computation. Processors such as the Z 80, MC68000, PDP-11, and VAX-11 include both the absolute and relative branch instructions.

A conditional branch instruction works as follows:

If ⟨condition⟩ *then* branch to execute a new instruction *else* execute the following instruction.

In this situation, assume the condition flags are already set by some instruction that immediately precedes the conditional branch instruction. Typically, the instruction

may be an arithmetic instruction (such as ADD, SUBTRACT, INCREMENT, DECREMENT, or COMPARE) or a logical instruction (such as TEST or COMPARE). Using the condition flag settings, traditional relational operators (such as equal to, not equal to, greater than, greater than or equal to, less than, or less than or equal to) can be implemented. For example, consider the following VAX-11 instruction sequence:

```

MOVAB    X,R7;           Move the address of X register into R7
MOVAB    Y,R8;           Move the address of Y into register R8
TSTB, (R7) ;             Check whether X = 0
BEQL UPDTY ;             If it is then update Y
BRB NEXT ;               otherwise go to next instruction
UPDTY: ADDB S^ #2, (R8):  perform Y: = Y + 2
NEXT: .....

```

This sequence is equivalent to the Pascal *if* statement:

```
If X = 0 then Y: = Y + 2
```

In this example, the variables X and Y are assumed to be 8-bit 2's complement numbers. The test instruction sets the Z-flag (zero-status flag) only if X is zero, and the branch instruction BEQL causes a branch only if the Z-flag is set to 1. PDP-11, MC68000, and VAX-11 conditional branch instructions can handle both signed and unsigned operand values.

The preceding numerical example suggests that by utilizing the logical and conditional branching instructions, the following control structures, which are regarded as the primitive components of a structured programming language such as Pascal or C; can be implemented:

- If <cond> then <statement>
 else <statement>
- While <cond> do
 <statement>
- Repeat
 <statement>
 Until <cond>
- Case <label> of
 id1 : <statement>
 id2 : <statement>
end

The VAX-11 instruction set includes useful instructions such as SOBGTR (subtract 1 from the loop index and branch if the index is still greater than zero) and AOBLEQ (add 1 to the loop index and branch if index is less than or equal to limit) to implement Pascal *repeat* and *for* loops, respectively.

For example, the VAX-11 code:

```

MOV B    #10, R5    ; Initialize R5 with 10
CVTBL    R5, R5      ; Sign extend R5 to 32 bits
CLRB     R6          ; Clear R6 to 0
MOVAB    SUM, R7     ; Point R7 to SUM

LOOP:    ADDB2       R5, R6
         SOBGTR      R5, LOOP    ; Repeat until (R5) = 0
         MOV B       R6, (R7)   ; Transfer the result to SUM

```

is equivalent to the Pascal program:

```

i := 10;
SUM := 0;
repeat
    SUM := SUM + i;
    i := i - 1;
until i = 0

```

Similarly, the VAX-11 program fragment

```

MOV B    #1, R5      ; Initialize R5 with the initial
CVTBL    R5, R5      ; Value of the loop index
MOV B    #10, R6     ; Initialize R6 with
CVTBL    R6, R6      ; Limiting value of the loop index
MOVAB    SUM, R7     ; Point R7 to SUM
CLRB     R8          ; Clear R8

LOOP:    ADDB2       R5, R8    ; Increment index by 1
         ADBLEQ      R6, R5, LOOP; and repeat until Loop index >
                                     limit
         MOV B       R8, (R7)  ; save result

```

is equivalent to this C program segment:

```

{
    sum = 0;
    for (i = 1;    i  <= 10;    i + +)
        sum + = i;
}

```

We leave the task of verifying the correctness of the preceding results as an exercise. With respect to VAX-11 programming, all loop parameters must be specified as 32-bit quantities. This is why we use the CVTBL instruction to sign-extend a byte into 32-bit operand (long-word operand).

A subroutine is a program segment for carrying out repeatedly needed tasks such as converting code from binary to ASCII, searching, and sorting. A subroutine may be written and tested separately. A subroutine can be linked with a user program so that the latter can call the former as many times as needed. Thus the use of subroutines can save programmer's time as well as the memory space needed by an application program.

A large program can be thought of as a collection of independent program modules, where each module may be a subroutine or a set of subroutines. This is the key feature of the modern software approach called *modular programming*.

In this method, a programmer has a global view of all components of a large program, and efficient programs can be developed within a short period of time. Since each subroutine can be independently tested, it follows that the modular programming approach considerably improves the overall software reliability. The subroutine concept strongly encourages the idea of program sharing. For example, in a multiuser system, several user programs may share the same I/O subroutine provided by the operating system. Therefore, a user does not have to spend time developing I/O routines.

Subroutine calls and returns from subroutines are usually handled by two special instructions, CALL and RET, respectively. The CALL instruction is of the form CALL <addr>, where the parameter <addr> refers to the address of the first instruction of the subroutine. When this instruction is executed, the current contents of the PC are saved in the stack, and the PC is loaded with <addr>. The current contents of the PC provide the address of the instruction that immediately follows the CALL instruction. This address is also called the *return address* because this is the point where execution of the calling program will take place after exiting from the subroutine. The CALL instruction is functionally equivalent to the following instruction sequence:

```

PUSH PC      ; save the return address in
              ; the stack (SP) + ← PC
JP addr      ; branch to the subroutine

```

The RET instruction is usually the last instruction of the subroutine. When this instruction is executed, the return address previously saved in the stack is retrieved and loaded into the PC. The control is then transferred to the calling program. A RET instruction is functionally equivalent to POP PC ; PC ← — (SP)

At this point, one may be wondering why the return address is not saved in a

CPU register rather than the stack. This arrangement fails to work if nested subroutine calls are to be implemented. Subroutine nesting refers to one subroutine calling another.

For example, consider the main program M and two subroutines P and Q shown in Figure 2.16(a): The main program calls subroutine P, and this subroutine in turn calls subroutine Q. The expected control flow sequence is shown in Figure 2.16(b). Parameters MR and PR refer to the return address of the main program (M) and the subroutine (P) respectively.

When the main program calls subroutine P, the return address (MR) is pushed into the stack (see Figure 2.16(c)) and the control is transferred to subroutine P. Similarly, when subroutine P calls subroutine Q, the return address (PR) is pushed into the stack (see Figure 2.16(d)), and the control is transferred to subroutine Q. When subroutine Q completes its execution, the return address is retrieved from the stack and loaded into the PC. Since the return address is PR, the execution of subroutine P is resumed. Similarly, when subroutine P terminates, the return address (MR) (see Figure 2.16(e)) is retrieved from the stack and loaded into the PC. The execution of the main program is then resumed.

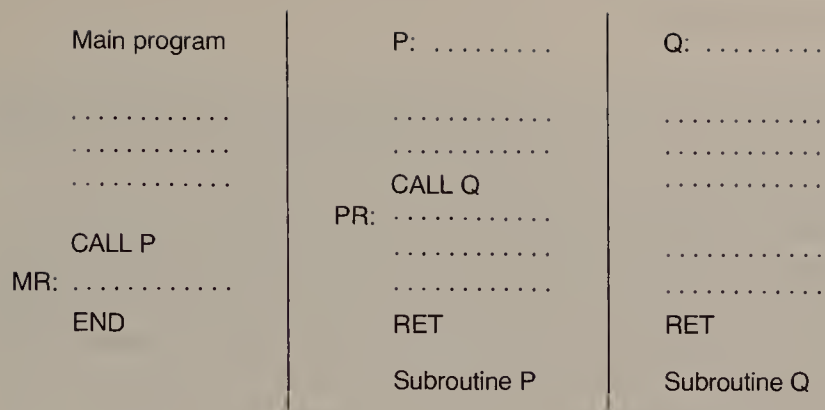
To implement subroutine nesting, the return addresses must be retrieved exactly in the reverse order in which they are saved. Since a stack is a LIFO data structure, its use is a natural solution to this problem. Suppose a CPU register is used to save the return address. The return address (PR) will write over the return address (MR), and control will not be transferred back to the main program at all.

Solutions to some problems such as traversing a binary tree are naturally recursive and are precisely solved by writing recursive subroutines. A subroutine is said to be recursive if it calls itself. A recursive evaluation involves a descent process all the way to the basis part of the recursive definition. Then it involves an ascent process all the way up in exactly the reverse order. Therefore, the use of a stack offers a solution for implementing recursive subroutines.

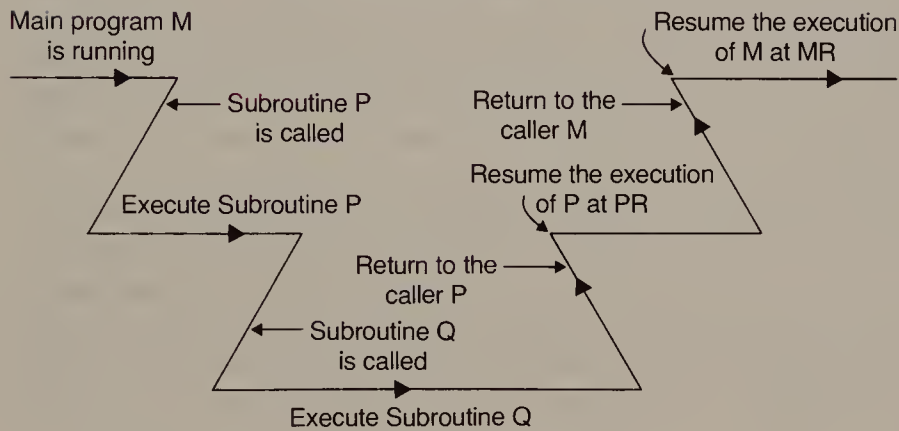
The registers involved in implementing a subroutine call are called *linkage registers*, and the process is known as the *subroutine linkage convention*. In all microprocessors, the PC is used as the linkage register. In the PDP-11 computer, any CPU register can be configured as a linkage register under program control. In the majority of the processors, the system hardware automatically uses the SP in response to CALL and RET instructions. An exception to this is Motorola's 8-bit microprocessor, the MC6809. The CPU of this processor entails two dedicated 16-bit registers: U (user stack pointer) and S (system stack pointer). In this microprocessor, the system hardware automatically uses the stack pointer S for handling subroutine calls and returns. A user can use the stack pointer U to configure a temporary data storage area in a different portion of the stack, preventing the user stack area from causing any interference to the subroutine linkage.

The effective use of subroutines calls for a method for transferring data from the caller to the subroutine, and vice versa. This aspect is often referred to as *parameter- or argument-passing* conventions. High-level languages such as Pascal and Ada adopt typical parameter-passing conventions such as call-by-value and call-by-reference.

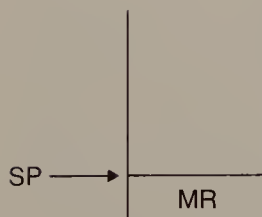
In the *call-by-value* approach, the main program transfers a parameter to a subroutine by copying the parameter value in the local variable that is created when the subroutine becomes active. When the subroutine changes the value of this local variable,



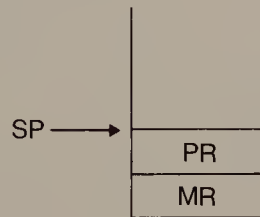
a. A Typical Two-Level Subroutine Nesting



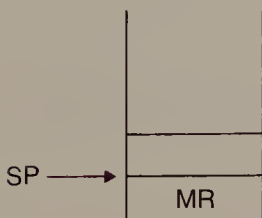
b. Expected Control Flow



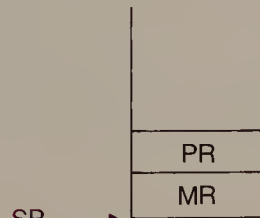
c. Contents of the stack after Executing CALL P



d. Contents of the stack after Executing CALL Q



e. Contents of the stack Executing the RET Instruction in the Subroutine Q



f. Contents of the stack after Executing the RET Instruction in the Subroutine P

Figure 2.16 Implementation of a Two-level Subroutine Nesting

the actual parameter of the main program does not change. This is a desirable property for function subprograms because it keeps a function from altering the value of the argument. Hence, the function returns a unique value to the caller.

In the *call-by-reference* approach, the main program transfers the address of a parameter to a subroutine. A subroutine can alter the value of the parameter that belongs to a caller. This approach allows a subroutine to transmit the results to the caller. The ability of a subroutine to alter the variables of the calling program is known to have a side effect. There is no side effect with the call-by-value approach. However, the call-by-reference approach uses the side effect for passing the results to the caller.

To implement these features, contemporary processors include either special instructions or additional hardware. For example, the LINK and UNLK (unlink) instructions of the MC68000 processors allow a compiler designer to implement Pascal functions and procedures with minimum effort. The VAX-11 CPU includes two dedicated registers: AP (argument pointer) and FP (frame pointer). These hardware elements greatly simplify the task of argument passing. A thorough discussion of this topic is beyond the scope of this book.

An interrupt may be defined as a hardware-initiated subroutine call. For example, in a microprocessor-based system, an I/O device such as a keyboard may generate an interrupt to inform the processor that valid data is available. When this interrupt is recognized, the processor suspends the execution of the currently running program, saves the contents of the PC in the stack, and transfers the control to a service routine dedicated to serve the keyboard. The service routine in this case reads the keyboard data, saves it in the main memory for further processing, and returns the control to the suspended program.

If the service routine needs the CPU registers, their previous contents must be saved in the stack before the service routine actually uses them. This is a time-consuming operation, and to speed it up, different microprocessors use different approaches. For example, in the Z 80 microprocessor, all CPU registers are duplicated. For each CPU register X, there is an alternate companion register X'. At any given time, either the original or alternate set of CPU registers can remain active. Normally, a user program is run using the original set of CPU registers. When an interrupt occurs, the system automatically switches to the alternate register set so the service routine can immediately use all the registers in the alternate set. However, after the interrupt is serviced, the system automatically switches back to the original set of registers so the execution of the suspended program can continue. This concept provides a fast response to external interrupts.

Modern processors such as the MC68000 and VAX-11 provide a number of software internal interrupts called TRAP instructions. These are covered in detail in Chapter 6.

2.4.5 System-control Instructions

With the advent of low-cost VLSI microprocessors, designing systems with several processors is feasible. Such a multiprocessor system has significant advantages, such as high processing speed and more reliability. In a typical multiprocessor system, several processors will be allowed to share a single memory unit. In this situation, at any given

time only one processor should be allowed to access the shared memory. This problem is known as the *mutual exclusion problem*.

Some processors such as the Motorola MC68000 have a special instruction called test and set (TAS) to provide a hardware-based solution to the above problem. The TAS instruction is used to prevent access to a shared resource by other programs when one program has control of the resource. This is sometimes called *lockout*. The TAS instruction is used to test and modify a byte length held either in a data register or in memory. For example, consider TAS (memory). If (memory) = 0, then set the zero flag Z = 1, else set Z = 0, N = 1, and then set bit 7 of the memory location to 1. Now, consider TAS (A1). Suppose (A1) = 00₁₆; then after execution of TAS, Z = 1 and (A1) is changed to 80₁₆. If the initial zero value indicated the memory area was free for use, a subsequent test of the memory area with the value 80₁₆ would indicate the memory was in use. To prevent accessing a shared memory in a multiprocessor system simultaneously by more than one processor, the TAS instruction has an indivisible read-modify-write cycle. Once the operand is addressed by the 68000 executing the TAS instruction, the bus is not available to another processor until the TAS instruction is completed.

As an illustration of synchronization by the TAS instruction, consider two 68000 processors that are interfaced through a shared RAM. It is desired to transfer (D₀) from Processor 1 to (D₂) by Processor 2 using the RAM byte TRDATA, as shown in Figure 2.17. To accomplish this transfer and provide proper synchronization, Processor 1 can execute the following routine for writing (D₀) into the RAM location TRDATA:

```

LOOP1    TAS      TEST
          BNE      LOOP1
          MOVE     D0, (TRDATA)
          CLR.B    TEST
          -
          -
          -

```

Processor 2 can then transfer (TRDATA) to D₂ by executing the following routine:

```

LOOP2    TAS      TEST
          BNE      LOOP2
          MOVE     (TRDATA), D2
          CLR.B    TEST
          -
          -
          -

```



Figure 2.17 Interfacing Two Processors Via Shared RAM

Initially the contents of the location TEST is zero. When Processor 1 executes the TAS instruction, it finds that (TEST) = 0; it sets (TEST) to 80₁₆ and falls through the wait loop “LOOP1”. Then it executes the MOVE D0 (TRDATA) instruction. At this point, assume that Processor 2 executes the TAS instruction. Processor 2 finds that (TEST) = 80, and it will not pass LOOP2 (because Z = 0). When Processor 1 is done with data transfer, it will execute the CLRB TEST instruction and so (TEST) are cleared to zero. This causes Processor 2 to fall through LOOP2, and hence it can access the shared memory. Observe that at any given time, the value of the memory byte TEST is 00 or 80H. For this reason, this byte often is referred to as *binary semaphore*.

2.4.6 I/O Instructions

I/O instructions allow a processor to perform input and output operations. An input instruction allows a peripheral to transfer a word to either a CPU register or memory. Similarly, an output instruction enables a processor to transfer a word into the buffer register of a peripheral device. In processors such as PDP-11 and MC68000, a peripheral device is mapped to a main memory address. In this situation, data moves from CPU to a memory location and vice versa constitute output and input operations, respectively. This method is known as *memory-mapped I/O*. The instruction set of any processor employing this approach does not include any special I/O instructions. Also, in this environment, a programmer can exploit all the available addressing modes for performing I/O operations in an efficient manner. In processors such as 8085, Z80, and 8086, a peripheral device can be mapped to an address in a separate address space called *I/O space*. The instruction set of each one of these processors include special input (IN) and output (OUT) instructions. These instructions are shorter than the regular move instructions, and so program execution time is expedited. Also, since the memory address space does not conflict with the I/O space, the entire memory can be completely utilized exclusively for storing code and data.

From the preceding discussion, we notice that the processor is completely tied up with I/O operations. This limits the system's *throughput* (number of tasks processed per unit time) because the operational speed of a peripheral device is typically 20 to 50 times lower than the processor. In order to overcome this difficulty, a hardware approach called *direct memory access* (DMA) is employed. In this method, the processor initiates an I/O operation by transferring typical parameters such as starting address in the memory, block size, and the type of operation to a hardware device called a DMA controller. After this, the DMA controller takes control of the I/O transfer. The processor is free to do any computation after initializing the DMA controller. Since I/O and CPU activities overlap, the DMA approach significantly increases a system's throughput.

2.5 REDUCED INSTRUCTION SET COMPUTER (RISC)

RISC, which stands for reduced instruction set computer, promises a new generation of faster and cheaper machines. The initial application of RISC principles has been in desktop workstations and minicomputers aimed at the fast-growing scientific and tech-

nical computing market. This market is currently worth \$8 billion, and sales could leap to \$28 billion by 1989. The major RISC vendors are Pyramid Technology, which makes UNIX-based supermini computers, and RIDGE computers, which produces computers for engineering workstations.

The basic idea behind RISC is for machines to cost less yet run faster, by using a small set of simple instructions for their operations. Also, RISC allows a balance between hardware and software based on functions to be achieved to make a program run faster and more efficiently. The philosophy of RISC embraces six principles: reliance on optimizing compilers, few instruction and addressing modes, fixed instruction format, instructions executed in one machine cycle, only call/return instructions accessing memory, and hardwired control.

The trend has always been to build CISCs (complex instruction set computers), which use many detailed instructions. However, because of their complexity, more hardware would have to be used, which actually would slow down the computer. The more instructions, the more hardware logic is needed to implement and support them. For example, in a RISC machine, an ADD instruction takes its data from registers. On a VAX, each operand can be stored in any of 14 different forms, so the compiler must check 14 possibilities.

The principles of understanding optimizing compilers and what actually happens when a program is executed lead to RISC. It turns out that RISC is really as much a philosophical approach as an architecture. To many implementers, RISC is just common sense.

However, not everyone in the industry favors using RISC as opposed to CISC. Some computer architects see RISC as a fad, or misleading claim. Their claim is that the advantages of RISC have nothing to do with reduced instruction sets. A study done by D. Patterson [7] at the University of California, Berkeley, showed that much of the performance of RISC and CISC machines has come from having lots of registers rather than from having few instructions. Critics also noted that RISC designs need to keep juggling program requirements with the number of available registers. If those factors got out of balance, there would be a lot of time-consuming memory swaps made by the processor, hence negating much of the performance advantage. According to Hewlett-Packard's Spectrum machine, which uses only 32 registers, this is not always the case.

The Spectrum line was the result of the reduction and simplification of the HP 300 instruction set. To exploit the increased power of the RISC-type architecture, users would employ an optimizing compiler to recompile automatically their existing application software. The software can run efficiently on everything from personal computers to mainframes. Hewlett-Packard was able to only use 32 registers based on a tremendous amount of analysis and simulation. Therefore, it achieved optimal performance with a very few registers.

Another argument critics have against RISC is the claim that the RISC technology is not well suited for modern general-purpose computing jobs. One example is floating-point arithmetic capability for high-precision numerical calculations. These operations require more than one machine cycle to execute; thus they do not fit into the single-cycle-per-instruction RISC philosophy. It is also very difficult to perform memory management or swapping chunks of data between devices such as disk drives and CPU memory with simple instructions. Therefore, none of the general-purpose computers

built with RISC principles are completely RISC machines. They are called “RISC-like.” This means most include at least a few complex instructions as well as instructions taking more than one machine cycle. However, there are some “pure” RISC machines that are built by universities, that have only 30 or 40 instructions.

Critics of RISC also claim RISC relies too heavily on compiler design. Without a good optimizing compiler, RISC is not better than CISC and may be worse. Opposing arguments claim writing the optimizing compiler for a RISC machine is easier than for a CISC machine because of the simple RISC design.

2.5.1 Case Study: RISC I (University of California, Berkeley)

The RISC machine presented in this section is the one investigated by D. Patterson and C. Sequin [8]. The authors proposed the computer RISC I with the following design constraints:

1. Only one instruction is executed per cycle.
2. All instructions have the same size.
3. Only load and store instructions can access memory.
4. High-level languages (HLL) are supported.

Owing to the larger user community of C and Pascal, these are the two languages that are considered for RISC I. A simple architecture implies a lesser number of transistors, and this leads to the fact that most pieces of a RISC HLL system are in software. Hardware is utilized for time-consuming operations. Using C and Pascal, a comparison study was made to determine the frequency of occurrence of particular variable and statement types. Studies revealed that integer constants appeared most frequently, and a study of the code produced revealed that the procedure calls are the most time-consuming operations.

Basic RISC Architecture

The RISC I instruction set contains a few simple operations (arithmetic, logical, and shift operations). These instructions operate on registers. Instruction, data, addresses and registers are all 32 bits long. RISC instructions fall in four categories: ALU, memory access, branch, and miscellaneous. The execution time is given by the time taken to read a register, perform an ALU operation, and store the result in a register. Register 0 always contains a 0. Load and store instructions move data between registers and memory. These instructions use two CPU cycles. Variations of memory-access instructions exist in order to accommodate sign-extended or zero-extended 8-bit, 16-bit and 32-bit data. Though absolute and register indirect addressing are not directly available, they may be synthesized using register 0. Branch instructions include CALL, RETURN, and conditional and unconditional jumps. The conditional instructions available are the standard ones used in PDP-11.

Instruction Format

opcode(7)	scc(1)	dest(5)	source1(5)	imm(1)	source2(13)
-----------	--------	---------	------------	--------	-------------

For register-to-register instructions, *dest* selects one of the 32 registers as destination of the result of the operation that is itself performed on registers *source1* and *source2*. If *imm* equals 0, the low-order 5 bits of *source2* specify another register. If *imm* equals 1, then *source2* is regarded as a sign-extended 13-bit constant. Since the frequency of integer constants is high, the immediate field has been made an option in every instruction. Also, *scc* determines whether the condition codes are set. Memory-access instructions use *source1* to specify the index register and *source2* to specify offset.

Register Windows

As mentioned earlier, the procedure-call statements take the maximum execution time. A RISC program has more call statements, since the complex instructions available in CISC are subroutines in RISC. The RISC register window scheme strives to make the call operation as fast as possible and also to reduce the number of accesses to data memory. The scheme works as follows.

Using procedures involve two groups of time-consuming operations, namely, saving or restoring registers on each call/return and passing parameters and results to and from the procedure. Statistics indicate that local scalars are the most frequent operands.

This creates a need to support the allocation of locals in the registers. One available scheme is to provide multiple banks of registers on the chip to avoid saving and restoring of registers. Thus each procedure call results in a new set of registers being allocated for use by that procedure. The return alters a pointer that restores the old set. A similar scheme is adopted by RISC. However, there are some registers that are not saved or restored; these are called *global registers*. In addition, the sets of registers used by different processes are overlapped in order to allow parameters to be passed. In other machines, parameters are usually passed on the stack with the calling procedure using a register to point to the beginning of the parameters (and also to the end of the locals). Thus all references to parameters are indexed references to memory. In RISC I the set of window registers (r10 to r31) is divided into three parts. Registers r26 to r31 (HIGH) contain parameters passed from the calling procedure. Registers r16 to r25 (LOCAL) are for local scalar storage. Registers r10 to r15 (LOW) are for local storage and for parameters to be passed to the called procedure. On each call, a new set of r10 to r31 registers is allocated. The LOW registers of the caller are required to become the HIGH registers of the called procedure. This is accomplished by having the hardware overlap the LOW registers of the calling frame with the HIGH registers of the called frame. Thus without actually moving the information, parameters are transferred (Refer to Figure 2.18 for an illustration.)

Multiple register banks require a mechanism to handle the case in which there are no free register banks available. RISC handles this problem with a separate register-overflow stack in memory and a stack pointer to it. Overflow and underflow are handled with a trap to a software routine that adjusts the stack. The final step in allocating variables in registers is handling the problem of pointers. RISC resolves this by giving addresses to the window registers. If a portion of the address space is reserved, we can determine with one comparison whether an address points to a register or to memory. Load and store are the only instructions that access memory and they take an extra cycle already. Hence this feature may be added without reducing the performance of the load and store instructions. This permits the use of straightforward computer technology and still leaves a large fraction of the variables in registers.

	<u>proc A</u>	<u>proc B</u>	<u>proc C</u>
HIGH _A	r31 _A		
	r26 _A		
LOCAL _A	r25 _A		
	r16 _A	r31 _B	
LOWA/HIGHB	r15 _A	r26 _B	
	r10 _A	r25 _B	
LOCAL _B		r16 _B	
		r15 _B	r31 _C
LOWB/HIGHC		r10 _B	r26 _C
			r25 _C
LOCAL _C			r16 _C
			r15 _C
LOW _C			r10 _C
GLOBAL			

Figure 2.18 Usage of Overlapped Register Windows (From Patterson, D., and C. Sequin *A VLSI RISC, Computer*, September 1982. Reprinted with permission.)

Delayed Jump

A normal RISC I instruction cycle is long enough to execute the following sequence of operations:

1. Read a register.
2. Perform an ALU operation.
3. Store the result back into a register.

Performance is increased by *prefetching* the *next instruction* during the current instruction. To facilitate this, jumps are redefined such that they do not occur until after the following instruction. This is called *delayed jump*, and its significance is explained in Chapter 7.

Evaluation

A study was made to compare RISC I and VAX and PDP-11 and MC68000. The results of the test are illustrated in Figure 2.19. This data is collected by studying the code generated by a C compiler for these four machines for the call return routine.

In the preceding test, two parameters are assumed to have been passed and three registers are saved.

2.5.2 RISC Assessment

The following material strives to show that RISC can provide as good a high-level language (HLL) environment as CISC. Two metrics are used to compare the performance of HLL programs on RISC and CISC. The first is speed and the second is the

System	Instructions executed	Size (bytes)	Register Accesses	Data Memory accesses
VAX	5	16	59	19
MC68000	9	30	41	12
PDP-11	19	44	51	15
RISC I	6	24	12	0.2

Figure 2.19 Performance Assessment of RISC I (From Patterson D., and R. Piepho, *Assessing RISCs in High-level Language Support*, IEEE Micro, 1982. Reprinted with permission.)

penalty of using HLL on a given machine. The index of evaluation used in the latter is the ratio of speed of execution of a program written in assembly to the speed of the same when written in HLL. This ratio is known as the HLL execution support factor (HLLSF). A system with HLLSF close to 0 penalizes the use of HLL, whereas an HLLSF close to 1 does not reward the use of assembly language.

The results of an experiment conducted to evaluate the relative performance of RISC, 68000, Z80, and VAX-11/780 with respect to the preceding two metrics are reproduced in Figure 2.20. Five variables, or benchmark programs, have been utilized

Benchmark	Number of times slower than RISC			
	RISC (ms)	6800	Z80	VAX 11/780
E—String search	0.46	2.8	1.6	1.3
F—Bit test	0.06	4.8	7.2	4.8
H—Linked list	0.10	1.6	2.4	1.2
K—Bit matrix	0.43	4.0	5.2	3.0
I—Quicksort	50.40	4.1	5.2	3.0

HLLSF - HLL Execution Support Factor				
Benchmark	RISC	6800	Z80	VAX11/780
E	0.62	0.17	0.32	0.23
F	1.00	0.23	0.27	0.34
H	1.00	0.92	0.96	0.88
K	0.94	0.21	0.29	0.34
I	0.92	0.16	0.44	0.47

Figure 2.20 Two Metrics to Assess the RISC's Effectiveness in HLL Support (From Patterson, D., and R. Piepho, *Assessing RISCs in High-level Language Support*, IEEE Micro, 1982. Reprinted with permission.)

for this experiment. These are selected as being representatives of frequent “real-world” problems. They allow manipulation of characters, integers, and floating-point data besides testing interrupt handling and addressing modes. Actually there are 12 such programs, but 7 were omitted either due to a lack of virtual memory or the difficulty involved in writing them in HLL. A brief description of the remaining 5 follows:

1. *String search* Examines a long character string for the first occurrence of a substring.
2. *Bit test, set and reset* Tests, sets, or resets a bit within a tightly packed bit string.
3. *Linked-list insertion*: Inserts a new entry into a doubly linked list.
4. *Quicksort*: Performs a nonrecursive quicksort algorithm on large vectors of fixed length records.
5. *Bit-matrix transportation*: Takes a tightly packed square bit matrix and transposes it.

RISC is a combination of scientific and philosophic rule called *Occam's razor*, which states that among competing theories, the simplest should be preferred to the complex.

RISC, which is a trend toward simplicity, will eventually have a definite influence on the computers of the future.

QUESTIONS AND PROBLEMS

- 2.1** What are the characteristics of a good instruction format?
- 2.2** What are the merits and demerits of the block-code encoding technique?
- 2.3** Explain the key idea behind the expanding op-code and Huffman encoding techniques.
- 2.4** In a computer instruction format, the instruction length and the size of an address field are 11 and 4 bits, respectively. Is it possible to have
 - 5 two-address instructions
 - 45 one-address instructions
 - 32 zero-address instructionsusing this format? Justify your answer.
- 2.5** Using the instruction format of Problem 2.4, determine whether or not is it possible to have

- 7 two-address instructions
- 15 one-address instructions
- 18 zero-address instructions

Justify your answer.

- 2.6** Assume that a computer architect has already designed 6 two-address and 24 zero-address instructions using the instruction format of the problem 2.4. What is the maximum number of one-address instructions that can be added to the instruction set?
- 2.7** The instruction repertoire of a computer consists of 8 instructions, $I_0, I_1, I_2, \dots, I_7$. The relative frequency of these instructions are as follows:

INSTRUCTION	RELATIVE FREQUENCY
I_0	$\frac{1}{4}$
I_1	$\frac{1}{8}$
I_2	$\frac{1}{8}$
I_3	$\frac{1}{8}$
I_4	$\frac{1}{8}$
I_5	$\frac{1}{8}$
I_6	$\frac{1}{16}$
I_7	$\frac{1}{16}$

Encode these instructions using Huffman's method.

Calculate the redundancy introduced by Huffman's scheme.

- 2.8** What is the essential difference between base and index register addressing modes?
- 2.9** Mention some important uses of each one of the following addressing modes.
- a)** Immediate
 - b)** Postincrement
 - c)** Predecrement
 - d)** Indexed
 - e)** Base register
 - f)** Relative
- 2.10** In a hypothetical computer, the system can directly address 16M bytes. The CPU of this computer has 16 base registers. Determine the size of the displacement

field so that we can address 6.25% of the total memory addresses (without changing the contents of the base registers).

- *2.11** In a computer, the main memory is divided into 256 pages and each page contains 256 bytes. The instruction format of this computer is as follows:

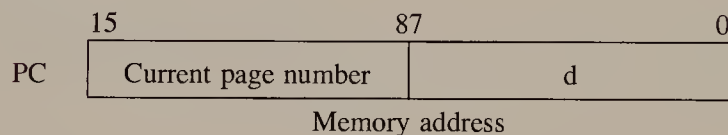
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
----	----	----	----	----	----	---	---	---	---	---	---	---	---	---	---

Bits 0–7 Address of a byte within a page (d)

Bit 8 0—direct addressing
 1—Indirect addressing

Bit 9 0—a reference to page 0
 1—a reference to the current page which is determined by the high-order 8-bits of the PC as shown below

Bits 10–15 Op-code



- a. How much of main memory can be directly addressed?
- b. How much of main memory can be indirectly addressed?

- 2.12** A computer instruction set supports the following Boolean instructions:

XOR X, Y ; $Y \leftarrow X \text{ Exclusive or } Y$

AND X, Y ; $Y \leftarrow X \text{ AND } Y$

Explain how will you perform the Boolean-OR operation $Y \leftarrow X \text{ OR } Y$ using the above two instructions.

- 2.13** Assume that a CPU has only two registers, R_1 and R_2 , and that only the following instruction is available.

XOR R_i, R_j ; $R_j \leftarrow R_i + R_j$
 ; $i, j = 1, 2$

Using this XOR instruction, find an instruction sequence in order to exchange the contents of the registers R_1 and R_2 .

*From Danhof, K. J. and C. L. Smith, *Computing Systems Fundamentals: An Approach Based on Microcomputers*, Addison-Wesley, 1981, p. 24. Reprinted with permission.

- 2.14** Explain why position independent codes are very important with operating system-supported routines.
- 2.15** What are the salient features of a RISC machine?

REFERENCES

1. Danhof, K. J., and C. L. Smith. *Computing System Fundamentals: An Approach Based on Microcomputers*. Reading, Mass.: Addison-Wesley, 1981.
2. Fuller, S. H., "Performance Evaluation," in *Introduction to Computer Architecture* edited by H. S. Stone, New York: SRA, 1980.
3. Hayes, J. P. *Computer Architecture and Organization*. New York: McGraw-Hill, 1978.
4. Hayes, J. P. *Digital System Design and Microprocessors*. New York: McGraw-Hill, 1984.
5. Knuth, D. E. *The Art of Computer Programming*. Vol. 1. New York: Addison-Wesley, 1969.
6. Mano, M. M. *Computer Systems Architecture*, Englewood Cliffs, N.J.: Prentice-Hall, 1983.
7. Patterson, D. "Reduced Instruction Set Computers." *CACM* (January 1985).
8. Patterson, D., and C. Sequin. "A VLSI RISC." *Computer* (September 1982).
9. Patterson, D., and R. Piepho. "Assessing RISCs in High-Level Language Support." *IEEE Micro*, (November 1982).
10. Rafiquzzaman, M. *Microcomputer Theory and Applications with the Intel SDK-85.* 2d. ed. New York: John Wiley, 1987.
11. Tanenbaum, A. S. *Structured Computer Organization*. Englewood Cliffs, N.J.: Prentice-Hall, 1984.

CHAPTER 3

Execution Unit

In this chapter, the typical components of a computer execution unit are discussed:

1. General Purpose registers
2. ALU
3. Dedicated hardware or firmware to perform special operations such as multiplication and division

In addition, the advanced topics of coprocessors, which are used to enhance the performance of the main execution unit, and LSI bit-slice processors are discussed.

3.1 INTRODUCTION

As mentioned previously, in a conventional stored-program computer, instruction codes are brought into the CPU from the main memory, where they are executed by the execution unit. Unless otherwise specified, instructions are always fetched in a strictly sequential fashion. The control unit, as the word implies, controls the sequencing of data and instructions coming into the ALU to be executed.

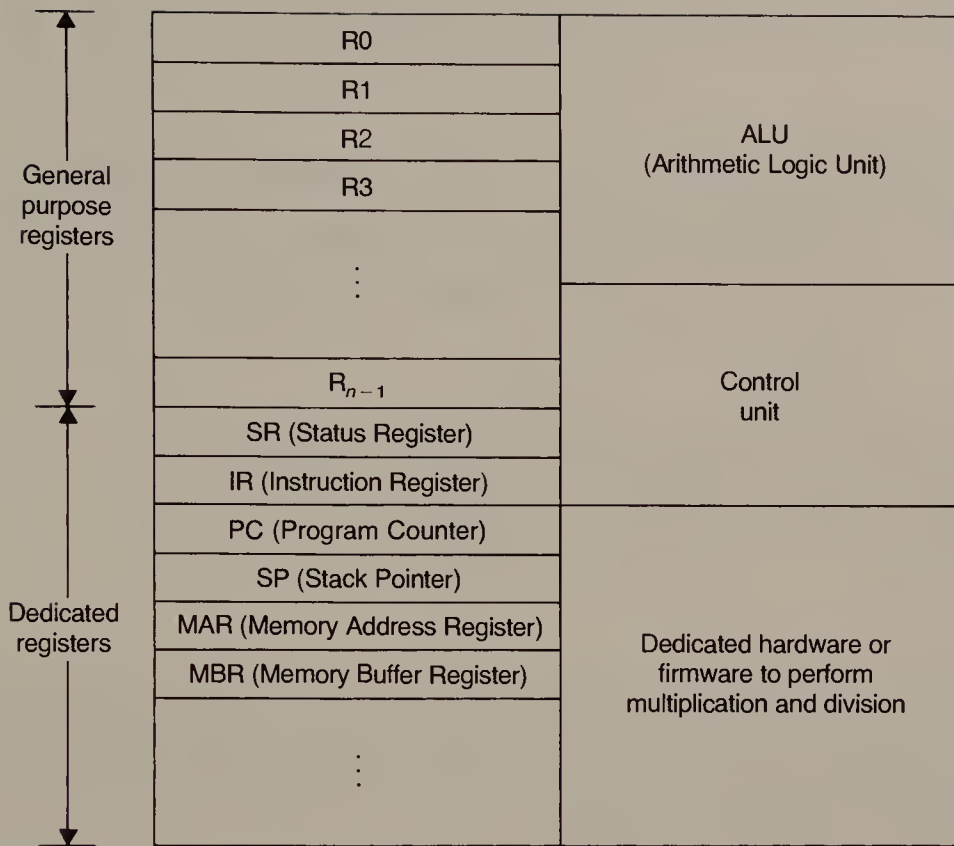


Figure 3.1 A Typical CPU Model

For the purpose of discussion, a typical CPU is modeled as shown in Figure 3.1. Figure 3.1 shows that a conventional CPU consists of the following:

1. General-purpose registers
2. Dedicated registers
3. An ALU
4. Dedicated hardware or firmware elements that perform a special operation such as multiplication or division.
5. A control unit

3.2 REGISTER SECTION

The register section is an important part of any CPU. A CPU with many registers reduces the number of visits to the main memory, therefore simplifying the programming task and shortening execution time. All CPU registers can be classified into two groups: general-purpose and dedicated registers. A general-purpose register may be configured as an accumulator, address pointer, or a data pointer. The contents of dedicated registers such as a program counter and stack pointer, are reserved for program execu-

tion. A summary of some commonly known special-purpose registers and their tasks follows:

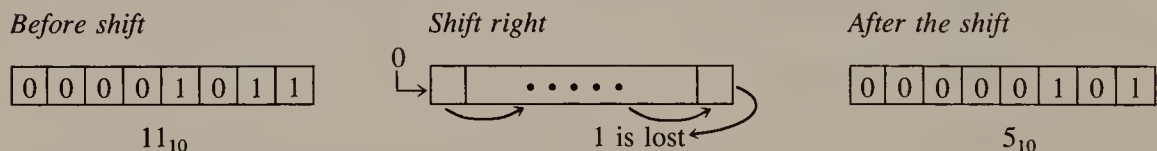
REGISTER	TASK
PC	Usually holds the address of the next instruction to be executed.
SP	Usually holds the address of the top element of the stack.
IR	Holds the instruction code currently being executed.
MAR	Holds the address of the data item to be retrieved from the main memory.
MBR	Holds the data item retrieved from the main memory.
SR or PSW	Holds the condition code flags and other information that describe the status of the running program.

3.2.1 General Register Design

The primary task of a general-purpose register (GPR) is to store address or data for an indefinite amount of time, then to be able to retrieve the data when needed. A GPR is also capable of manipulating the stored data by shift left or right operations. Figure 3.2 contains a summary of typical shift operations.

In a logical shift operation, a bit that is shifted out will be lost, and the vacant position will be filled with a 0. For example, if we have the number $(11)_{10}$, after right shift, the following occurs:

8-bit word

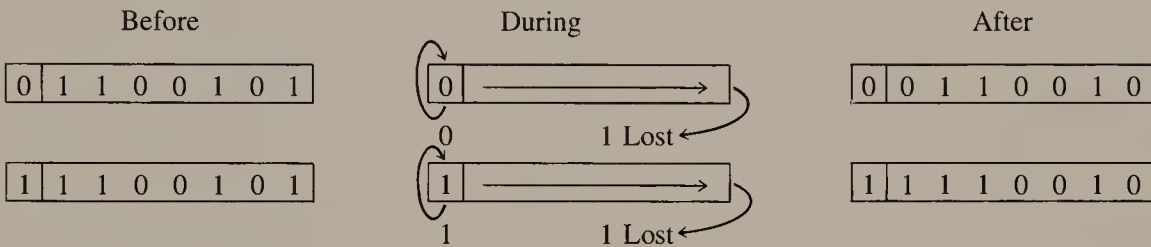


Shift type	Logical	Arithmetic	Rotate
Right			
Left			

Figure 3.2 Summary of Typical Shift Operations

It must be emphasized that a logical left or right shift of an unsigned number by n positions implies multiplication or division of the number by 2^n , respectively, provided that a 1 is not shifted out during the operation.

In case of true arithmetic left or right shift operations, the sign bit of the number to be shifted must be retained. However, in most typical computers, this is true for right shift and not for left shift operation. For example, if a register is shifted right arithmetically, the most-significant bit (MSB) of the register is preserved, thus ensuring that the sign of the number will remain constant. This is illustrated next:



There is no difference between arithmetic and logical left shift operations. If the most-significant bit changes from 0 to 1, or vice versa, in an arithmetic left shift, the result is incorrect. For example, if the original value of the register is $(3)_{10}$, the results of two successive arithmetic left shift operations are interpreted as follows:

ORIGINAL	AFTER FIRST SHIFT	AFTER SECOND SHIFT
$0011_2 = (3)_{10}$	$0110_2 = (6)_{10}$	$1100_2 = (-4)$
	$3 \times 2 = 6$, correct	$6 \times 2 = 12$ not -4 , incorrect

For this reason, in processors such as the MC68000, the overflow flag bit (V-bit) of the status register(SR) is set when the MSB is changed at any time during an arithmetic shift left (ASL); otherwise the flag is cleared. The overflow flag is not affected during a logical left shift.

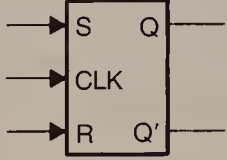
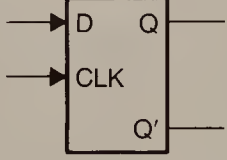
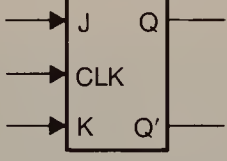
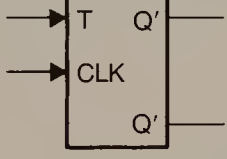
Rotation is useful for the serialization of parallel data. Peripheral devices such as a cassette recorders will expect serial 8-bit data. In this situation, the CPU transfers the contents of an 8-bit CPU register to the cassette recorder by rotating the register 8 times.

Let us now discuss the hardware aspects of a register. From a hardware perspective, a register can be viewed as an interconnection of a set of flip-flops. The characteristics of standard flip-flops are summarized in Table 3.1.

In order to design a general purpose register, first let us propose a basic cell S.

The internal organization of the S cell is shown in Figure 3.3. A 4-input multiplexer selects one of the external inputs as the D flip-flop input, and the selected input appears at the flip-flop output Q after the clock pulse. The CLR input is an asynchronous clear input, and whenever this input is asserted (held low), the flip-flop is cleared to zero. Using the basic cell S as the building block, a 4-bit GPR can be designed. Its schematic representation is shown in Figure 3.4.

Table 3.1 A SUMMARY OF STANDARD FLIP-FLOPS

Flip-flop Type	Block Diagram Representation	Function Table	Characteristic Equation	Excitation Table
S – R		S R Q ⁺	$Q^+ = S + R' \cdot Q$ with $S \cdot R = 0$	Q Q ⁺ S R
		0 0 Q		0 0 0 X
		0 1 0		0 1 1 0
		1 0 1		1 0 0 1
		1 1 ?		1 1 X 0
D Flip-flop		D Q ⁺	$Q^+ = D$	Q Q ⁺ D
		0 0		0 0 0
		1 1		0 1 1
				1 0 0
				1 1 1
JK Flip-flop		J K Q ⁺	$Q^+ = J \cdot Q' + K' \cdot Q$	Q Q ⁺ J K
		0 0 Q		0 0 0 X
		0 1 0		0 1 1 X
		1 0 1		1 0 X 1
		1 1 Q'		1 1 X 0
T Flip-flop		T Q ⁺	$Q^+ = T \oplus Q$	Q Q ⁺ T
		0 Q		0 0 0
		1 Q'		0 1 1
				1 0 1
				1 1 0

Note: Q⁺: Next state—that is, the value of Q after clock.

X: Don't care.

The truth table modeling the operation of this register is shown in Figure 3.5.

This table shows that manipulation of the selection inputs S_1 and S_0 can cause three distinct operations: parallel load, shift left, and shift right. If $S_1 S_0 = 11$, the external inputs X_3 through X_0 are selected as the D-inputs for the flip-flops, the output V_i will follow the input x_i after the clock. By choosing the correct values for serial shift inputs R and L, logical, arithmetic or rotate shifts can be achieved. Figure 3.6 shows how to realize three different right shifts by choosing the appropriate values for the right shift serial input R.

This register can be loaded with any desired data in a serial fashion. For example, after 4 successive right shift operations, data X will be loaded into the register if the register is set in the right shift mode and the required data X is applied serially to input R.

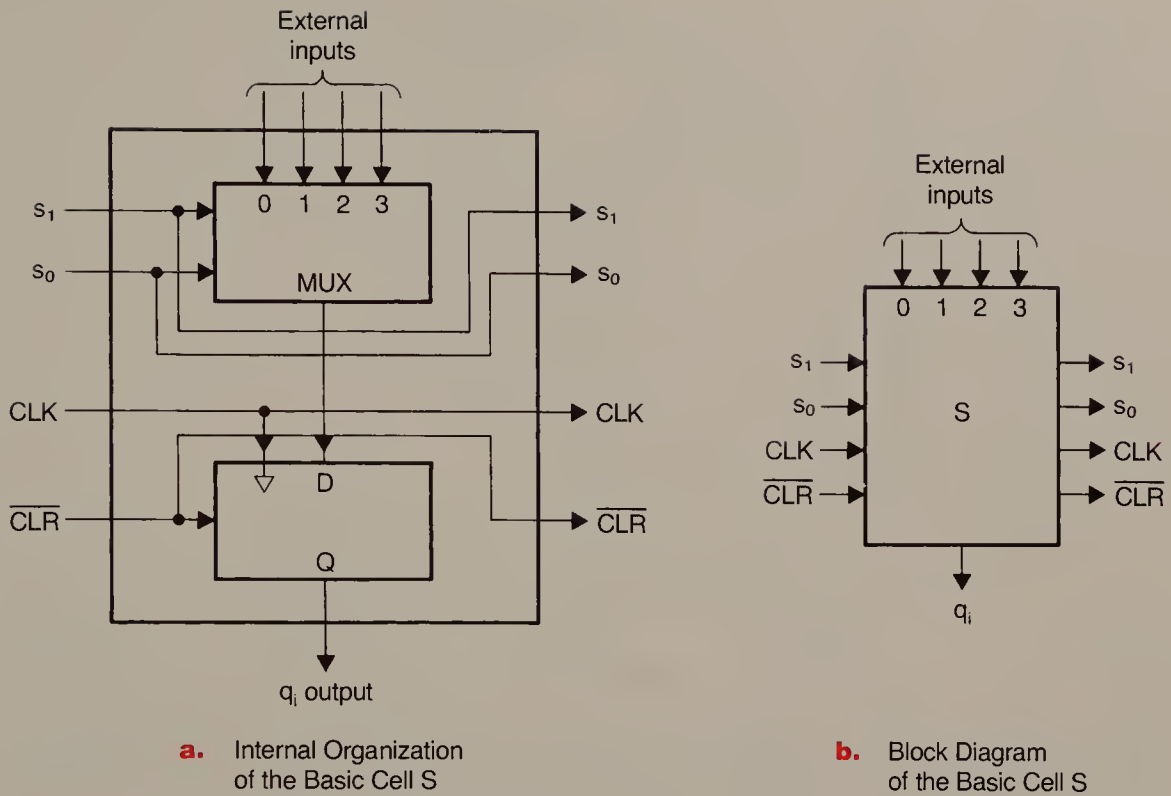


Figure 3.3 A Basic Cell for Designing a GPR

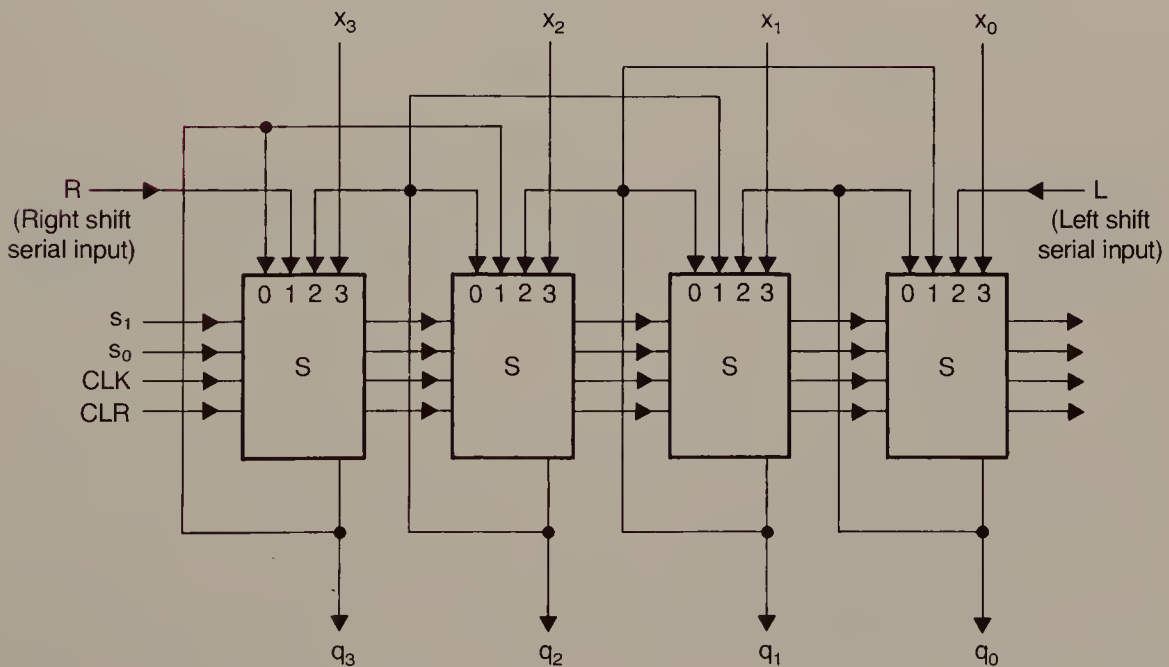


Figure 3.4 A 4-bit General Register

Selection Inputs		Clock Input	Clear Input	Operation
s_1	s_0	CLK	$\overline{\text{CLR}}$	
X	X	X	0	Clear
0	0		1	No operation
0	1		1	Shift right
1	0		1	Shift left
1	1		1	Parallel load

Note: X – don't care;  : rising edge of the clock

Figure 3.5 Truth Table Controlling the Operation of the General Register

3.2.2 Combinational Shifter Design

The optional speed of a flip-flop-based shifter is a function of the clock frequency. A high-speed shifter can be designed using combinational circuit components such as a multiplexer. The block diagram, internal organization, and the truth table of a typical combinational shifter are shown in Figure 3.7. From the truth table, the following equations can be obtained:

$$Y_3 = s_1's_0'i_3 + s_1's_0'i_2 + s_1s_0'i_1 + s_1s_0'i_0$$

$$Y_2 = s_1's_0'i_2 + s_1's_0'i_1 + s_1s_0'i_0 + s_1s_0'i_{-1}$$

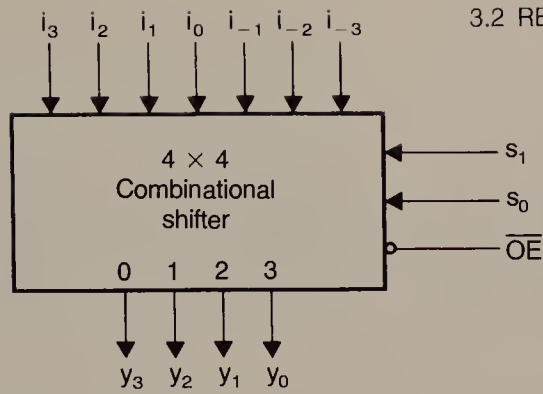
$$Y_1 = s_1's_0'i_1 + s_1's_0'i_0 + s_1s_0'i_{-1} + s_1s_0'i_{-2}$$

$$Y_0 = s_1's_0'i_0 + s_1's_0'i_{-1} + s_1s_0'i_{-2} + s_1s_0'i_{-3}$$

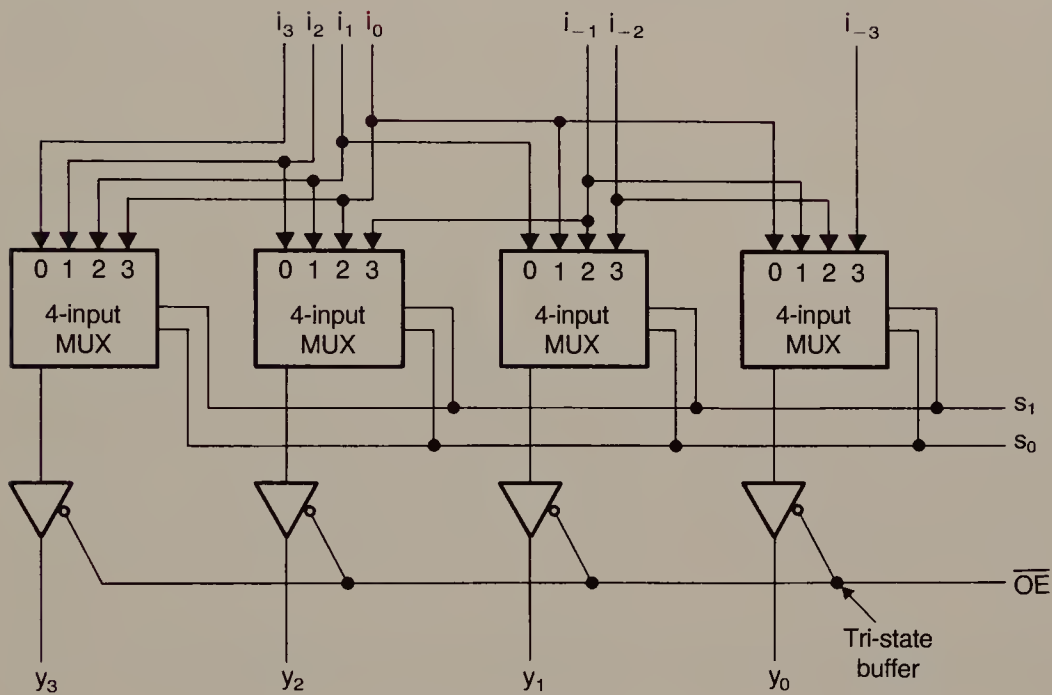
Combinational shifters of this type are produced by Texas Instruments (74350) and Advanced Micro-Devices (AM25S10). The 4×4 shifter of Figure 3.7 can be expanded to obtain a system capable of rotating 16-bit data to the left by 0, 1, 2, or 3 positions. (see Figure 3.8.)

$s_1s_0 = 01$	
Value of R	Shift realized
0	Logical right shift
q_3	Arithmetic right shift
q_0	Rotate right

Figure 3.6 Three Variations of the Right Shift Operations



a. Block Diagram



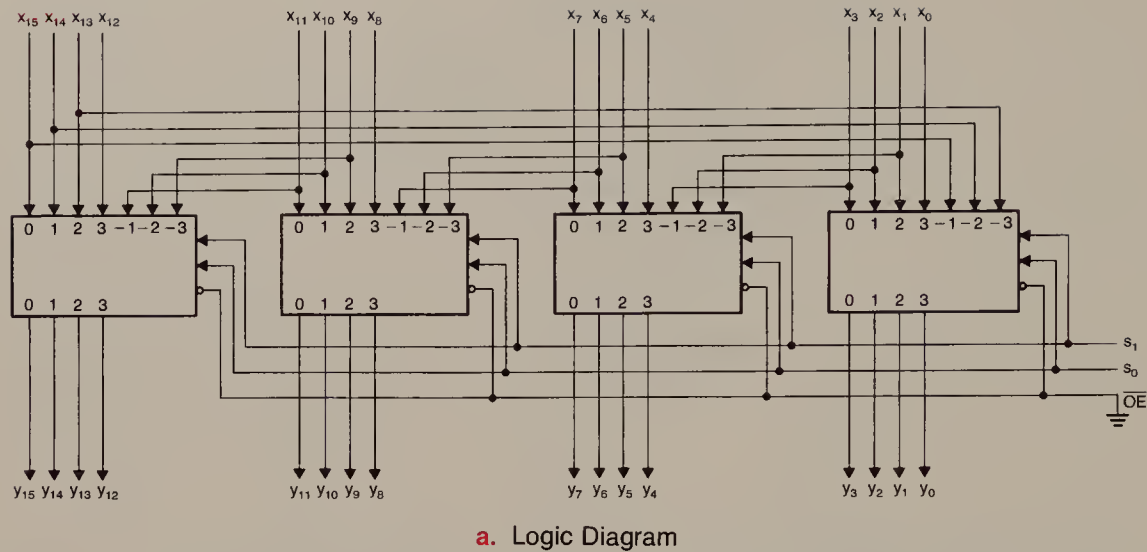
b. Internal Schematic

$\overline{\text{OE}}$	Shift Count		Output				Comment
	s_1	s_0	y_3	y_2	y_1	y_0	
1	X	X	Z	Z	Z	Z	Output lines float
0	0	0	i_3	i_2	i_1	i_0	Pass (no shift)
0	0	1	i_2	i_1	i_0	i_{-1}	Left shift once
0	1	0	i_1	i_0	i_{-1}	i_{-2}	Left shift twice
0	1	1	i_0	i_{-1}	i_{-2}	i_{-3}	Left shift three times

Note: Z—High-impedance state.
X—Don't care.

c. Truth Table

Figure 3.7 4×4 Combinational Shifter



Shift Count		Output															
s_1	s_0	y_{15}	y_{14}	y_{13}	y_{12}	y_{11}	y_{10}	y_9	y_8	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0
0	0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0
0	1	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}
1	0	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}
1	1	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}

b. Truth Table**Figure 3.8** Combinational Shifter Capable of Rotating 16-bit Data to the Left by 0, 1, 2, or 3 Positions

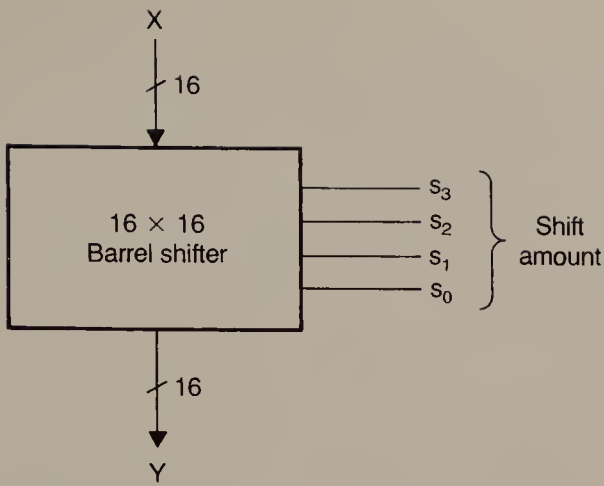
This design can be extended to obtain a more-powerful shifter called the *barrel shifter*.

The block-diagram representation of a 16×16 barrel shifter is shown in Figure 3.9(a). This shifter is capable of rotating the given 16-bit data to the left by n positions, where $0 \leq n \leq 15$. Figure 3.9(b) shows the truth table representing the operation of this shifter.

Functionally, the barrel shifter can be divided into two 16×16 combinational logic units (see Figure 3.9(c)). The level 1 shifter rotates the given 16-bit data (x) i -positions to the left, where $0 \leq i \leq 3$. The level 2 shifter can rotate the input data to the left by $4k$ positions where $0 \leq k \leq 3$. For example – if x is to be rotated by 11 positions, then set $i = 3$ ($S_1S_0 = 11$) and $k = 2$ ($S_3S_2 = 10$).

The complete logic diagram of the final hardware is shown in Figure 3-9(d). The level 1 shifter is identical to the hardware developed earlier (see Figure 3.8). The interconnections between the level 1 and level 2 shifters are marked to maintain the clarity of the diagram. The task of verifying the correctness of the design is left as an exercise.

The barrel shifter is an on-chip component for VLSI 32-bit microprocessors such

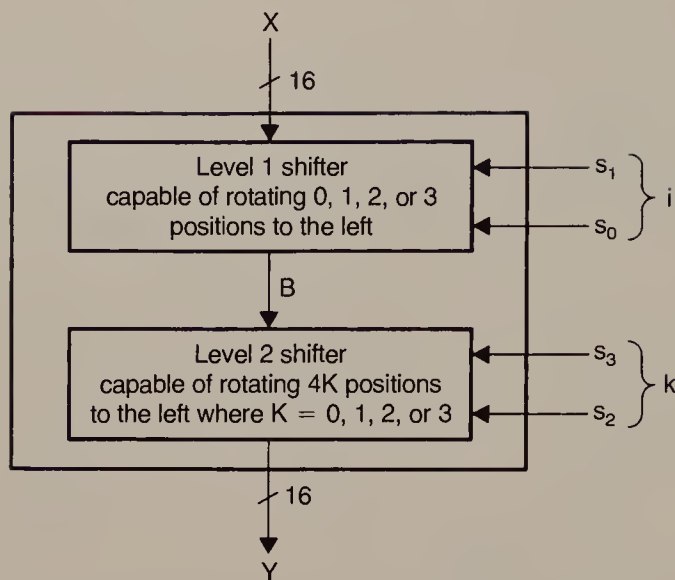


a. Block Diagram of a 16×16 Block Shifter

Shift Amount				Output															
s_3	s_2	s_1	s_0	y_{15}	y_{14}	y_{13}	y_{12}	y_{11}	y_{10}	y_9	y_8	y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0
0	0	0	0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0
0	0	0	1	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}
0	0	1	0	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}
0	0	1	1	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}
0	1	0	0	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}
0	1	0	1	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}
0	1	1	0	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}
0	1	1	1	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9
1	0	0	0	x_7	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8
1	0	0	1	x_6	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7
1	0	1	0	x_5	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6
1	0	1	1	x_4	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5
1	1	0	0	x_3	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4
1	1	0	1	x_2	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3
1	1	1	0	x_1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2
1	1	1	1	x_0	x_{15}	x_{14}	x_{13}	x_{12}	x_{11}	x_{10}	x_9	x_8	x_7	x_6	x_5	x_4	x_3	x_2	x_1

b. Truth Table of the 16×16 Barrel Shifter

Figure 3.9



c. Functional Block Diagram of a 16×16 Barrel Shifter

as the MC 68020 and the Intel 80386. The barrel shifter of the MC 68020 processor chip occupies 7.0% of the total chip area. The 80386 barrel shifter is capable of shifting a number from 0 through 64 positions in one clock period (clock rate is 16 MHz). Such high-speed shifting is desirable for manipulating floating-point numbers.

In scientific computation, numbers that are too large as well as too small need to be handled. In order to obtain such a wide range, numbers are represented by a special method called *floating-point notation*, similar to scientific notation. A complete discussion of this topic is beyond the scope of this chapter.

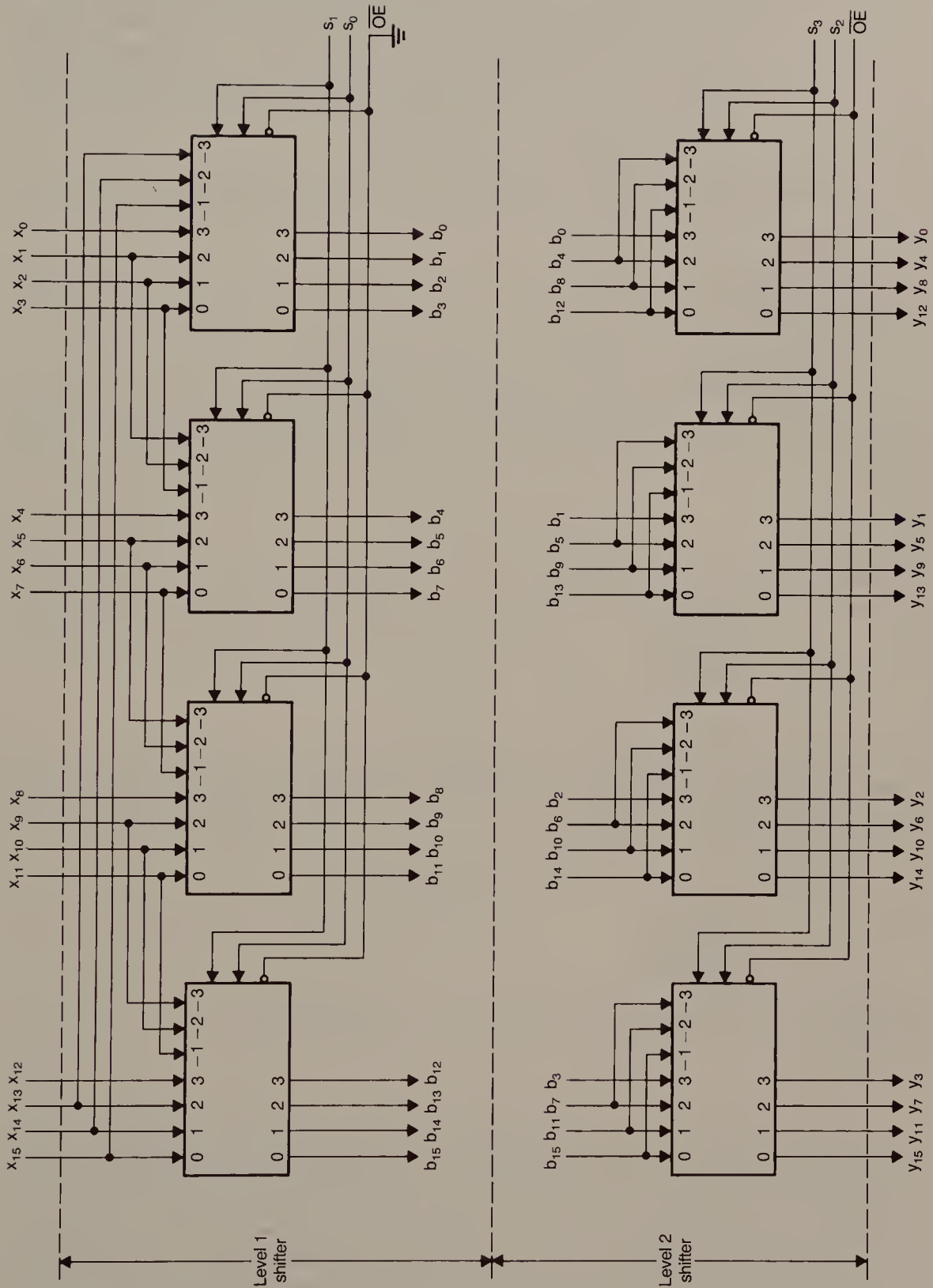
3.2.3 Flag Register

Previously, it was mentioned that the program status is held by a set of condition codes. In reality, condition codes are realized by designating one flip-flop for each condition. These flip-flops are typically set or reset depending on the result produced by the most recent arithmetic or a logical operation. In the following, it is assumed that the condition code register contains only the following flags:

Z (zero), N (negative), C (carry), and V (overflow)

The Z flag is set to 1 if the most-recent operation produced a zero result. The N flag is set to 1 if the most-recent arithmetic operation produced a negative result. In processors such as the PDP-11 and the MC68000, the C and V flags are manipulated in the following manner:

C Flag During the addition operation, the C flag is set to 1 if a carry is generated from the most-significant bit position; otherwise it is reset to 0. If the operation performed is a subtraction, the C flag is set to 1 when no carry is generated from the most-significant position.



d. Complete Logic Diagram Full 16-bit Barrel Shifter

V Flag During an addition operation, the V flag is set to 1 only when the operands added have the same sign and produce a sum of opposite sign. During a subtraction operation, the V-flag is set to 1 only when the operands A and B have different signs with $A - B$ and B having the same sign. It can be shown that for 8-bit signed operation $V = c_7 \oplus c_8$ where c_8 is final carry and c_7 is previous carry.

To illustrate how condition flags are affected, the following example is provided:



Example 3.1

Given that

$$A = 01100010$$

$$B = 01000101$$

determine the status of the condition flags if the operation performed were as follows.

a. $A + B$

b. $A \wedge B$

Solution

a.

A	01100010
B	<u>01000101</u>
$A + B$	10100111
	$\downarrow \quad \searrow$ $C_8 = 0 \quad C_7 = 1$

From this result, the sign of the result is negative: no overflow and no end-around carry; the result is nonzero. Therefore,

Therefore, $N = 1$, $V = 1$, $C = 0$, and $Z = 0$.

b.

A	10100010
B	<u>01000101</u>
$A \wedge B$	00000000

Following the same reasoning used in part (a), it is inferred that $N = 0$, $V = 0$, $C = 0$, and $Z = 1$.



Using condition codes, a programmer can detect whether or not an arithmetic operation produced a valid result. For instance, if the addition of two unsigned integers results in an overflow (carry), then C-flag is set to 1. Similarly, the V-flag serves as an overflow/underflow indicator when two signed integers represented in 2's complement form are added or subtracted. The condition codes are also useful in determining the relative magnitudes of two n -bit unsigned integers A and B. The previous chapter showed that by using compare or test, instructions decisions such as whether $A = B$ or $A \neq B$ can be made.

Note that the compare instruction of the form $\text{CMP } A, B$ subtracts B from A and

sets the flags based upon difference produced. After the compare operation, if the Z flag is set to 1, then $A = B$; otherwise $A \neq B$. From the fundamentals of digital arithmetic, it is known that

$$\begin{aligned} A - B &= A + 2\text{'s complement of } B \\ &= A + B' + 1 \\ &= A + 2^n - B \\ &= 2^n + A - B \end{aligned}$$

Observe that if B is an n -bit unsigned integer, then the following relationship holds: 2's complement of $B = 2^n - B$.

Suppose that $A > B$. This means that $A - B$ is positive, and thus $2^n + A - B > 2^n$. But, an unsigned n -bit integer can cover numbers in the range of 0 and $2^n - 1$. Therefore, when $A > B$, the subtraction operation produces a carry from the most-significant position, and thus the C-flag is reset to 0. $A > B$ also implies that $A \neq B$. Hence, we have the following interpretation:

$$C = 0 \quad \text{and} \quad Z = 0 \quad \text{implies that } A > B$$

$$C = 0 \quad \text{implies that } A \geq B$$

The negation of $A > B$ is true only when $A \leq B$. Similarly, the negation of $A \geq B$ is equivalent to asserting the condition $A < B$. This result suggests that the following interpretations in terms of C and Z flags are possible.

$$C = 1 \quad \text{or} \quad Z = 1 \quad \text{implies that } A \leq B$$

$$C = 1 \quad \text{implies that } A < B$$

Suppose that the operands A and B are represented in 2's complement form. Then the Z-, N-, and V-flags are used to determine their relative magnitudes. This interpretation is summarized in Figure 3.10.

The verification of this interpretation is left as an exercise.

FLAG STATUS	INTERPRETATION
$Z = 0$	$A \neq B$
$Z = 1$	$A = B$
$(Z = 0)$ and $(VN = 00 \text{ or } VN = 11)$	$A > B$
$(VN = 00 \text{ or } VN = 11)$	$A \geq B$
$(VN = 01 \text{ or } VN = 10)$	$A < B$
$Z = 1$ or $(VN = 01 \text{ or } VN = 10)$	$A \leq B$

Figure 3.10 Status Flag Interpretation
When A and B are in 2's Complement Form

3.3 ADDERS

Addition is the basic operation performed by an ALU. The operation is versatile, since $A - B$ is equivalent to $A + 2$'s complement of B , and $C * A$ may be obtained by adding A to itself for $C - 1$ times. Thus the speed of the adder unit is essential to the efficient operation of an execution unit.

A hardware unit that adds two single-bit numbers with a carry is called a full adder. For convenience, the block diagram, the truth table and the implementation of a full adder are shown in Figure 3.11.

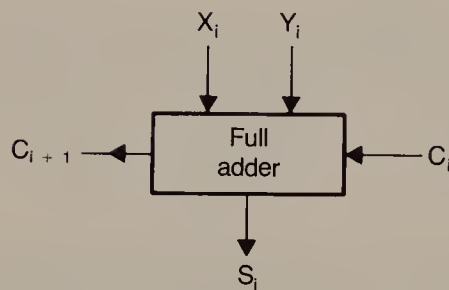
$$\text{From the truth table, } c_i = x_i' y_i' c_i + x_i' y_i c_i' + x_i y_i' c_i' + x_i y_i c_i$$

$$= c_i(x_i' y_i' + x_i y_i) + c_i'(x_i' y_i + x_i y_i')$$

$$\text{Let } z = x_i' y_i + x_i y_i' = x_i \oplus y_i$$

$$\text{then } c_i = c_i z' + c_i' z$$

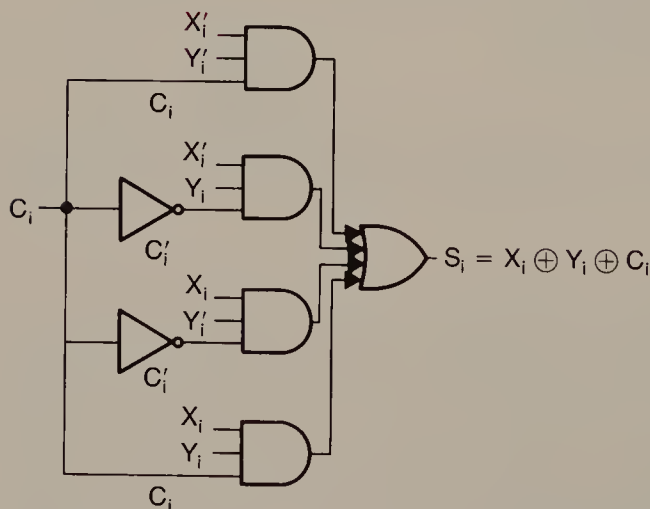
$$= z \oplus c_i = x_i \oplus y_i \oplus c_i$$



a. Block Diagram

X_i	Y_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

b. Truth Table



c. Implementation

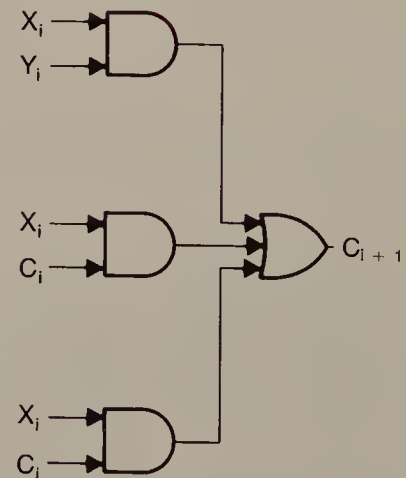


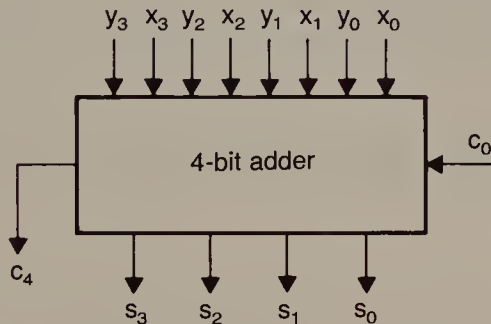
Figure 3.11 Full Adder

$$\begin{aligned}
 \text{Also, } c_{+1} &= x_i' y_i c_i + x_i y_i' c_i + x_i y_i c_i' + x_i y_i c_i \\
 &= x_i' y_i c_i + x_i y_i c_i + x_i y_i' c_i + x_i y_i c_i + x_i y_i c_i' + x_i y_i c_i \\
 &= y_i c_i + x_i c_i + x_i y_i
 \end{aligned}$$

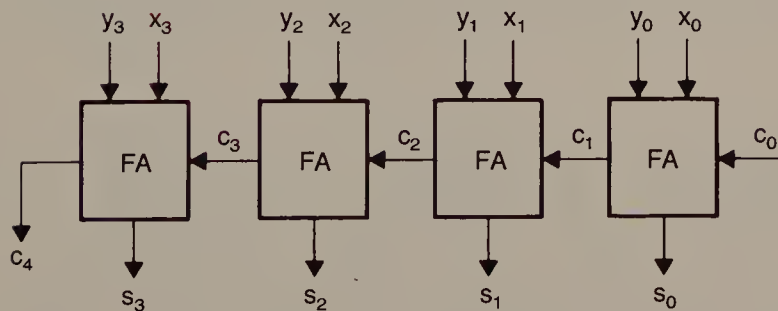
As has been made apparent by Figure 3.11, for generating C_{i+1} from C_i , two gate delays are required. In order to generate S_i from C_i , three gate delays are required, since C_i must be inverted to obtain C_i' . Note that no inverters are required to get X_i' or Y_i' from X_i or Y_i , respectively, since the numbers to be added are usually stored in a register that is a collection of flip-flops. The flip-flop often generates both normal and complemented outputs. For the purpose of discussion, assume that the gate delay is Δ time units, and the actual value of Δ is decided by the technology. For example, if Transistor Translator Logic (TTL) circuits are used, the value of Δ will be 10 ns.

By cascading n full adders, an n -bit parallel adder capable of handling two n -bit operands (X and Y) can be designed. The implementation of a 4-bit ripple-carry adder is shown in Figure 3.12.

When two unsigned integers are added, the input carry, C_0 , is always zero. Yet, when this system is to operate on 2's complement numbers, the value of C_0 may be zero or one. (This is explained later in the section). The 4-bit adder is also called a



a. Block Diagram of a 4-bit Ripple-Carry Adder



b. Four Full Adders are Cascaded to implement a 4-bit Ripple-Carry Adder

Figure 3.12 Implementation of a 4-bit Ripple-carry Adder

carry-propagate adder (CPA), since the carry is propagated serially through each full adder. This hardware can be cascaded to obtain a 16-bit CPA, as shown in Figure 3.13.

Although the design of an n -bit CPA is straight forward, the carry propagation time limits the speed of operation. For example, in the 16-bit CPA (see Figure 3.13), the addition operation is completed only when the sum bits S_0 through S_{15} are available.

To generate S_{15} , C_{15} must be available. The generation of C_{15} depends on the availability of C_{14} , which must wait for C_{13} to become available. In the worst case, the carry propagates through 15 full adders. Therefore, the worst-case add-time of the 16-bit CPA can be estimated as follows:

$$\begin{aligned} &\text{Time taken for carry to propagate} \\ &\quad \text{through 15 full adders (the delay} \\ &\quad \text{involved in the path from } C_0 \text{ to } C_{15}) = 15 * 2\Delta \\ &\text{Time taken to generate } S_{15} \text{ from } C_{15} = \frac{3\Delta}{33\Delta} \end{aligned}$$

If $\Delta = 10$ ns, then the worst-case add-time of a 16-bit CPA is 330 ns. This delay is prohibitive for high-speed systems in which the expected add-time is typically less than 100 ns. This creates the necessity to devise a new technique to increase the speed of operation by a factor of 3. One such technique is known as the carry look-ahead. In this approach the extra hardware is used to generate each carry (C_i , $i > 0$) directly from C_0 . To be more practical, consider the design of a 4-bit carry look-ahead adder (CLA). It will be shown how this may be used to obtain 16-bit adder that operates at a speed higher than the 16-bit CPA.

Recall that in a full adder, the output carry C_{i+1} is related to its carry input, C_i , as follows:

$$C_{i+1} = X_i Y_i + X_i C_i + Y_i C_i$$

This result can be rewritten:

$$C_{i+1} = G_i + P_i C_i \quad \text{where} \quad G_i = X_i Y_i \quad \text{and} \quad P_i = X_i + Y_i$$

The function G_i is called the *carry-generate function*, since a carry is generated when $X_i = Y_i = 1$. If X_i or Y_i is a 1, then the input carry C_i is propagated to the next

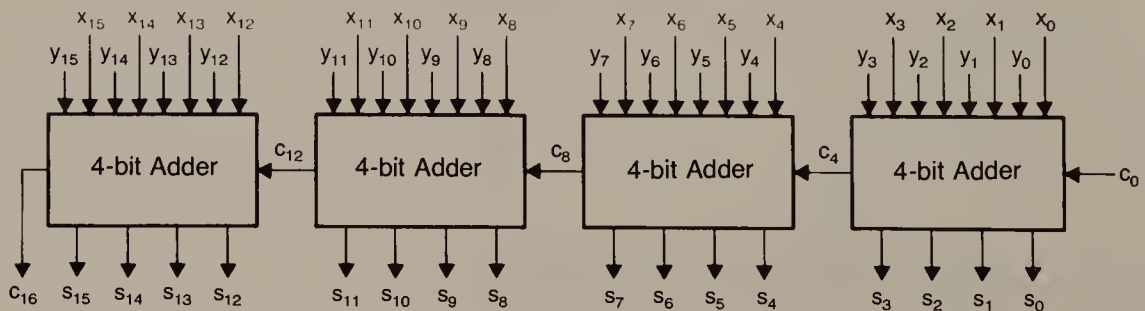


Figure 3.13 Implementation of a 16-bit Adder Using 4-bit Adders as Building Blocks

stage. For this reason, the function P_i is often referred to as *carry-propagate function*. Using G_i and P_i , C_1 , C_2 , C_3 , and C_4 can be expressed as follows:

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1C_1$$

$$C_3 = G_2 + P_2C_2$$

$$C_4 = G_3 + P_3C_3$$

These equations are recursive, and the recursion can be removed as indicated below:

$$C_1 = G_0 + P_0C_0$$

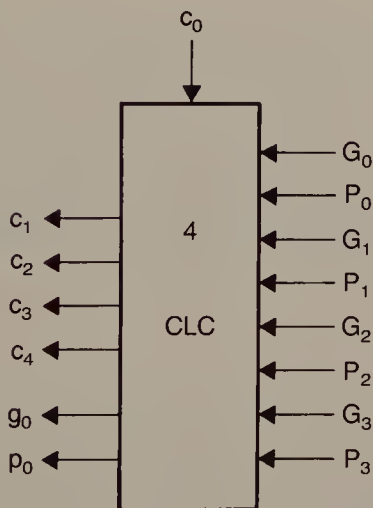
$$C_2 = G_1 + P_1C_1 = G_1 + P_1(G_0 + P_0C_0) = G_1 + P_1G_0 + P_1P_0C_0$$

$$\begin{aligned} C_3 &= G_2 + P_2C_2 = G_2 + P_2(G_1 + P_1G_0 + P_1P_0C_0) \\ &= G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0 \end{aligned}$$

$$\begin{aligned} C_4 &= G_3 + P_3C_3 = G_3 + P_3(G_2 + P_2G_1 + P_2P_1G_0 + P_1P_1P_0C_0) \\ &= G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0 \end{aligned}$$

This result suggests that C_1 , C_2 , C_3 , and C_4 can be generated directly from C_0 . For this reason, these equations are called *carry look-ahead equations*, and the hardware that implements these equations is called a *4-stage carry look-ahead circuit (4-CLC)*. The block diagram of such circuit is shown in Figure 3.14. Following are some important points about this system:

- A 4-CLC can be implemented as a two-level AND/OR logic circuit (The first level comprises AND gates, whereas the second level comprises OR gates). In



NOTE: $g_0 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$
 $p_0 = P_3P_2P_1P_0$

Figure 3.14 A Four-Stage Carry Look-ahead Circuit

particular, the fan-in requirement for this realization is 5. To generate $P_3 P_2 P_1 P_0 C_0$, a 5-input AND gate is required; to generate C_4 from C_0 a 5-input OR gate is needed.

- If $G_i P_i$ ($0 \leq i \leq 3$) is available, then all C_i 's with $1 \leq i \leq 4$ can be generated in 2 gate delays.
- The outputs g_0 and p_0 are useful to obtain a higher-order look-ahead system.

To construct a 4-bit CLA, assume the existence of the basic adder cell shown in Figure 3.15.

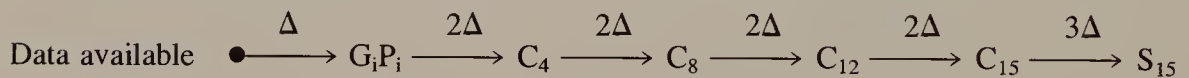
Using this basic cell and 4-bit CLC, the design of a 4-bit CLA can be completed as shown in Figure 3.16.

Using this 4-bit CLA as the building block, a 16-bit adder can be designed, as shown in Figure 3.17.

The worst-case add-time of this adder can be calculated as follows:

	Delay
For $G_i P_i$ generation from $X_i Y_i$ ($0 \leq i \leq 15$)	Δ
To generate C_4 from C_0	2Δ
To generate C_8 from C_4	2Δ
To generate C_{12} from C_8	2Δ
To generate C_{15} from C_{12}	2Δ
To generate S_{15} from C_{15}	3Δ
Total delay	12Δ

A graphical illustration of this calculation can be shown as follows:



From this calculation, it is apparent that the new 16-bit adder is faster than the 16-bit CPA by a factor of 3. In fact, this system can be speeded up further by employing

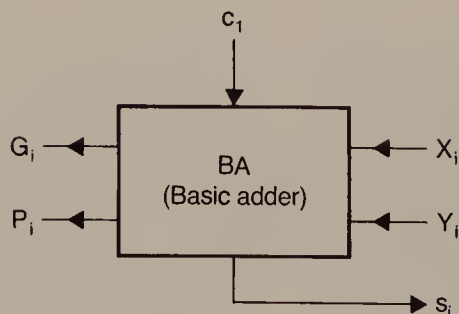


Figure 3.15 Basic CLA Cell

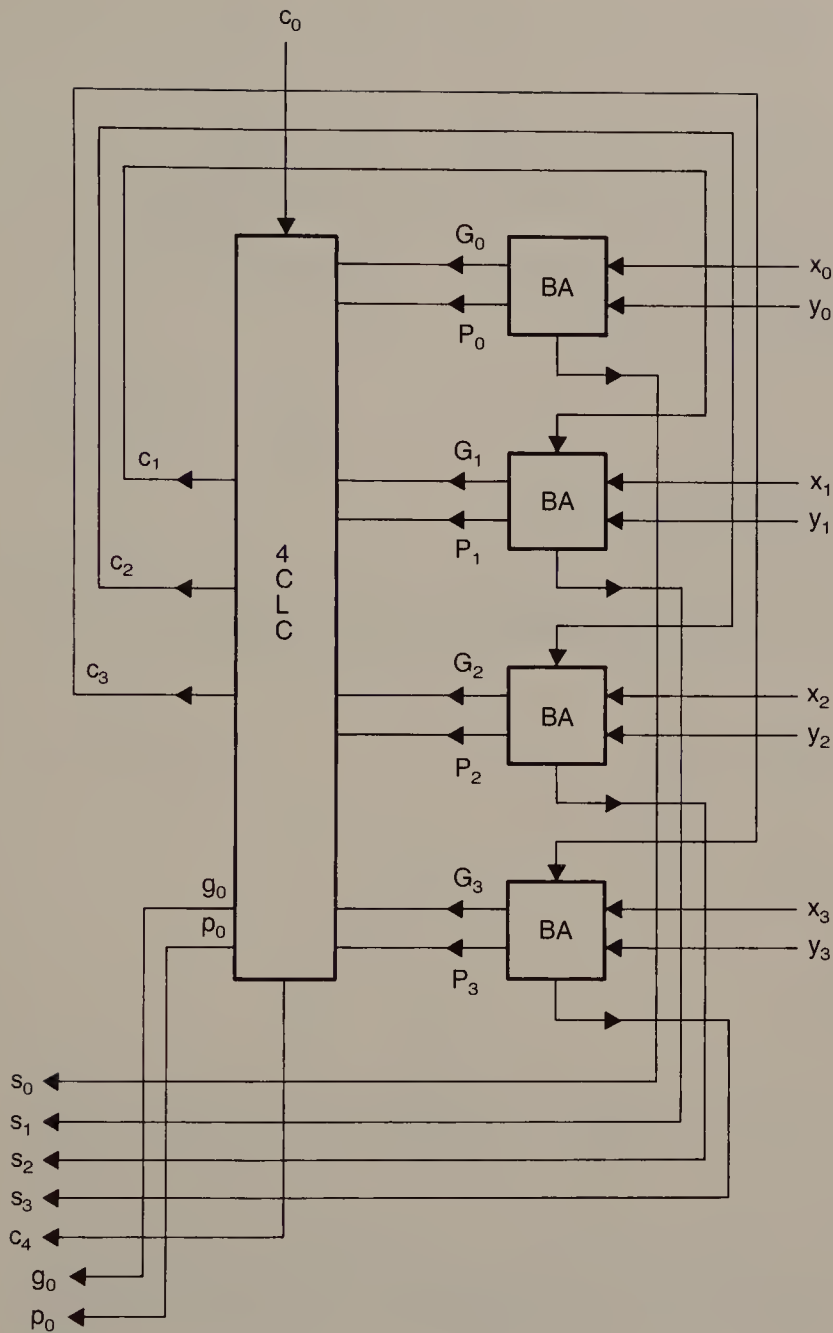


Figure 3.16 A 4-bit CLA

another 4-CLC and eliminating the carry propagation between the 4-bit CLA blocks. For this purpose, the g_i and p_i outputs generated by the 4-bit CLA are used. This design task is left as an exercise to the reader.

It should be emphasized that a two-level realization of a n -CLC requires an $n + 1$ input AND gate and an $n + 1$ input OR gate. Since the maximum fan-in offered by the existing technology is 8, it is not possible to build a two-level CLC when $n + 1 > 8$. For this condition, design of an n -bit adder in terms of 4-bit slice blocks offers an excellent compromise.

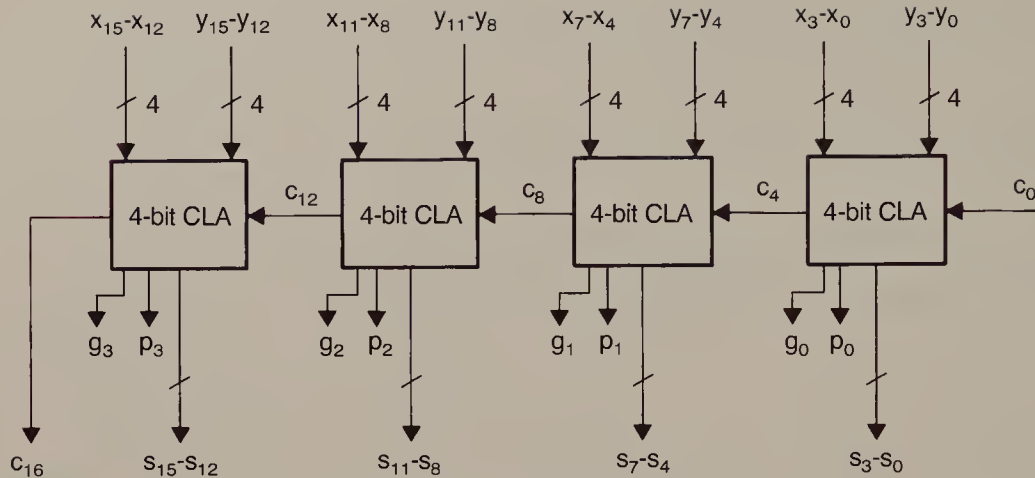


Figure 3.17 Design of 16-bit Adder Using 4-bit CLA's

If there is a need to add more than 3 operands, a technique known as *carry-save addition* is used. To see the effectiveness, consider the following example:

```

44
28
32
79
63 ← Sum vector
12 ← Carry vector
183 ← Final answer

```

In this example, four decimal numbers are added. First, the unit digits are added, and producing a sum of 3 and a carry digit of 2. Similarly, the tens digits are added, producing a sum digit of 6 and a carry digit of 1. Since there is no carry propagation from the unit digit to the tenth digit, these summations can be carried out in parallel to produce a sum vector of 63 and a carry vector of 12. When all operands are exhausted, the sum and the shifted carry vector are added in the conventional manner, which produces the final answer. Note that the carry is propagated only in the last step, which generates the final answer no matter how many operands are added. This concept is also referred to as *addition by deferred carry assimilation*.

The carry-save addition process can be implemented in fully parallel mode or in series-parallel mode. In the first method, all operands are to be added simultaneously. For example, consider the summation:

$$Z = A + B + C + D$$

where A, B, C, and D are 3-bit 2's complement numbers. The block diagram representation of this summation process is shown in Figure 3.18(a).

In this system, the first carry-save adder (CSA) adds A, B, and C and produces a sum vector (S^1) and a carry vector (C^1). The sum vector and the shifted carry vector,

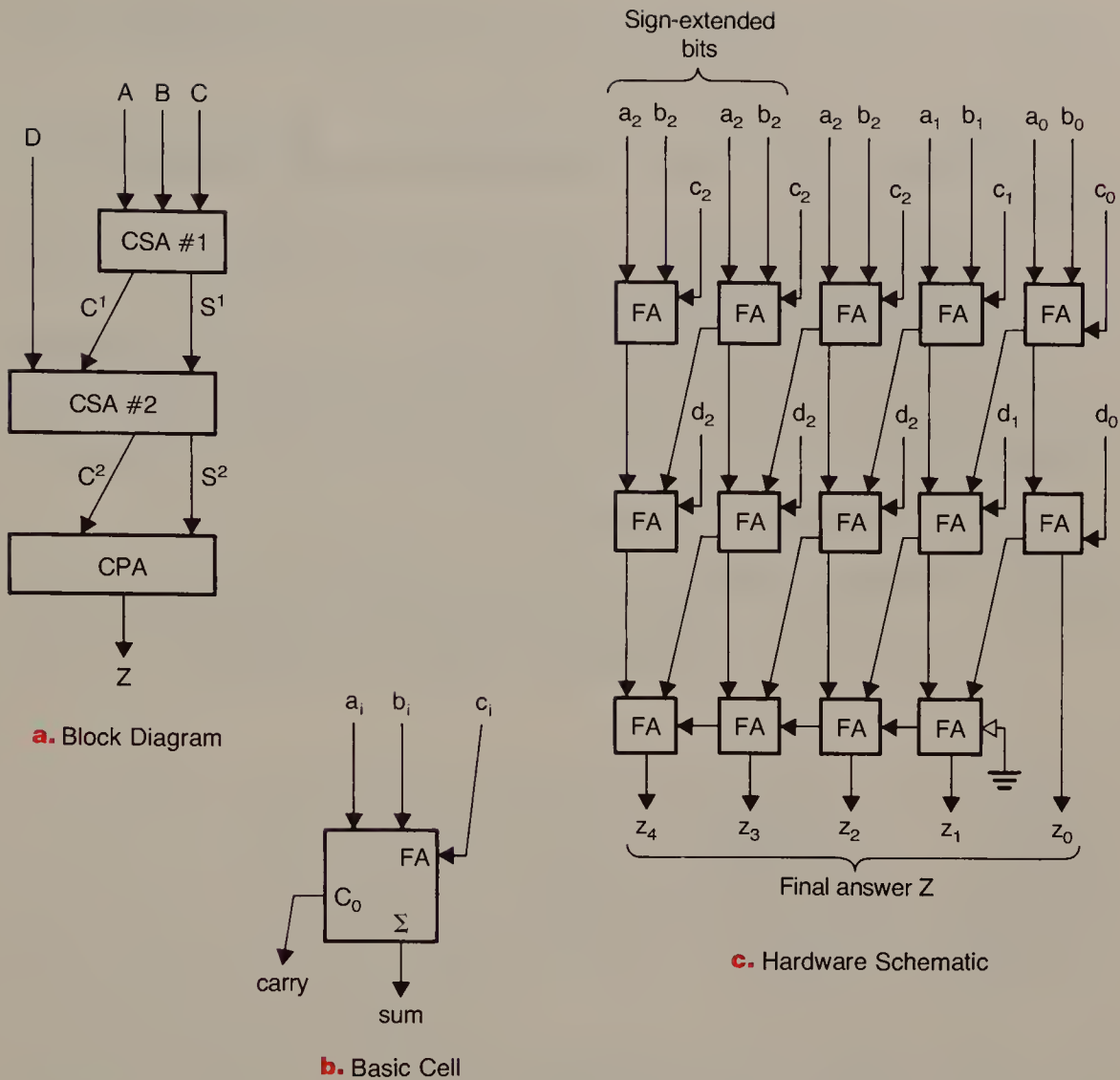


Figure 3.18 5-operand Summation

along with the fourth operand D, are applied as the input to the second CSA. The results produced by the CSA are finally handled by a CPA to generate Z. The carry is propagated only in the last step. So the total time required to add 4 operands is:

$$T(4) = 2 * [\text{CSA add time}] + [\text{CPA time}]$$

In general, n operands can be summed in:

$$T(n) = (n - 2) * [\text{CSA add time}] + [\text{CPA time}]$$

This result can further be improved by using a CLA in the last stage (as opposed to a CPA). Figure 3-18(b) and (c) shows the hardware implementation of the 4-operand summation. When four 3-bit operands are added, the sum, Z, lies in the range of -16 and 12 (with 3 bits we can cover numbers in the range of -4 and 3). Hence, 5 bits are required to represent Z. These extra bits (2 bits in this case) are called *guard bits*, and,

in general an n -operand summation requires $\lceil \log_2 n \rceil$ guard bits. The operator $\lceil \rceil$ is called the ceiling and is formally defined as follows:

$$\lceil X \rceil = Y \quad \text{where } Y \text{ is the least integer such that } Y \geq X$$

To include these guard bits, the given 3-bit operands are sign-extended to obtain a 5-bit data.

The number of CSA stages in a fully parallel multioperand summation can be reduced if the CSAs are organized in the form of a tree. Tree structure corresponding to a 6-operand summation is shown in Figure 3.19.

This structure is a Wallace tree. Basically, a Wallace tree is an interconnection of CSAs that reduces the column of height N to a height of 2. Note that a column of height 2 can be immediately handled by a CPA or CLA. In such a scheme, it can be proven that the total number of CSA levels can be approximated by the following formula:

$$\text{number of CSA levels} = \lceil \log_{1.5}(N/2) \rceil$$

By using a Wallace tree structure, n operands can be summed within $T(n)$ time units, where:

$$T(n) = \lceil \log_{1.5}(n/2) \rceil * (\text{CSA add time}) + (\text{CPA or CLA add time})$$

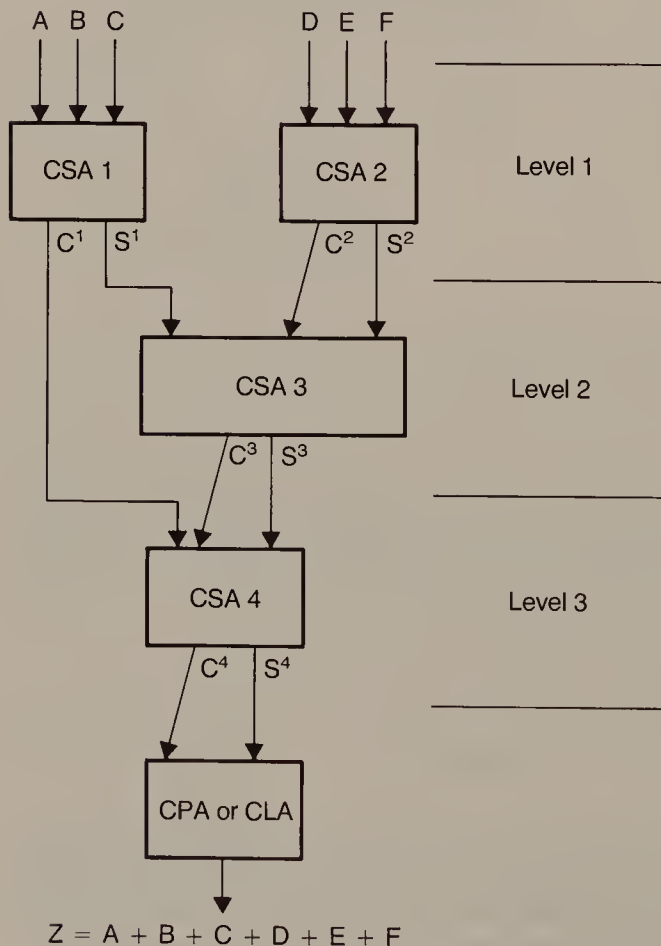


Figure 3.19 A Tree Structure for 6-operand Summation

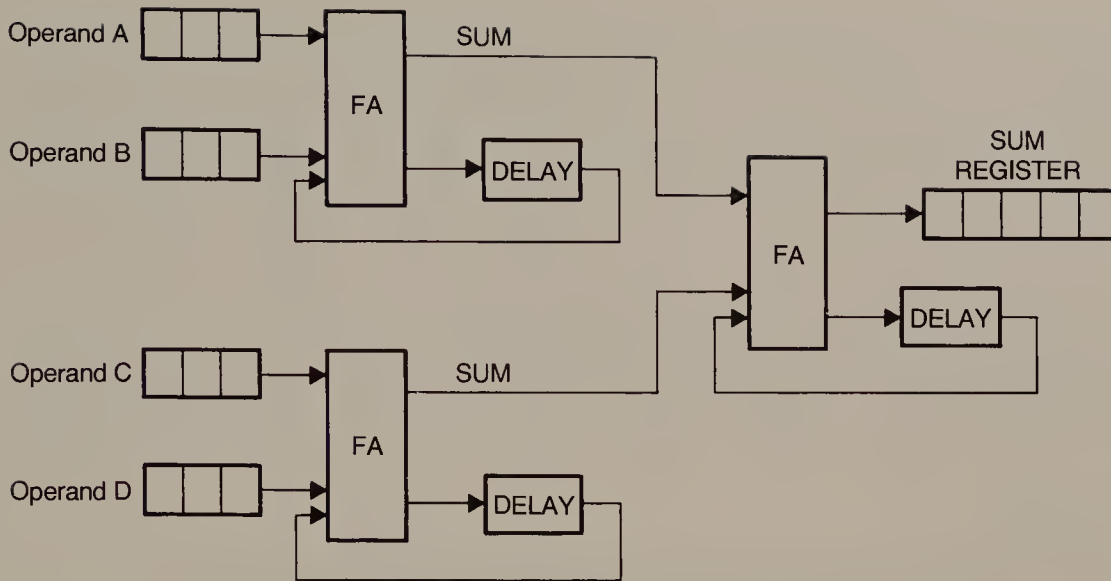


Figure 3.20 Series-parallel Approach to 4-operand Summation (From Norman R. Scott, *Computer Number Systems and Arithmetic*, Prentice-Hall, 1985, p. 66. Adapted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

The Wallace tree structure is an essential component in the design of high-speed multipliers. This concept is expanded upon later in this chapter.

Although the fully parallel approach offers great speed, it requires expensive hardware. To reduce the hardware cost, a series-parallel approach is used as a compromise. In this method, the operands are input column by column serially. However, all bits of the same columns are summed in parallel. For example, consider the summation of four 3-bit operands shown in Figure 3.20.

For this hardware, the 3-bit operands are held in shift registers. During the first clock, the first column of all operands (the least-significant bits) are summed by the full adders, and the final sum is shifted into the sum register of the next clock. The delay unit at the output of the FA delays the carry output by one clock period, causing the carry to be added to the next column sum. Although the hardware produces the final answer in a serial fashion, it is still well suited for pipelining. The pipeline concept is a method for obtaining a speedup via overlapped execution of two or more operations. This is discussed in a later chapter. Suppose that there are n operands. If $n = 2k$, for some $k > 1$, the series-parallel approach will require $(n - 1)$ adders. If there are m bits per operand, the entire operation can be completed within a time T , where:

$$T \equiv m * (\text{clock period}) + (\log_2 k) * (\text{full adder time})$$

The task of verifying these results is left as an exercise.

It was mentioned earlier that $X - Y$ is the same as $X + 2$'s complement of $Y = X + y^1 + 1$. The hardware that implements this concept is shown in Figure 3.21.

The select input controls the input carry (C_0) and the left input of each full adder are as follows for this configuration:

For the first step, a two-function arithmetic unit, as shown in Figure 3.22, is designed. The key element of this system is a 4-bit parallel adder. The multiplexers select either Y or Y' for the 3-input of the parallel adder. In particular, if $S_0 = 0$, then $B = Y$; otherwise $B = Y'$. Since the selection input (S_0) also controls the input carry (C_{in}), the following results:

$$\begin{aligned} \text{If } s_0 = 0 \quad & \text{then} \quad F = X \text{ plus } Y \\ & \text{else} \quad F = X \text{ plus } Y' \text{ plus } 1 \\ & \quad = X \text{ minus } Y \end{aligned}$$

The operation of this circuit is identical to the adder/subtractor circuit discussed in the previous section.

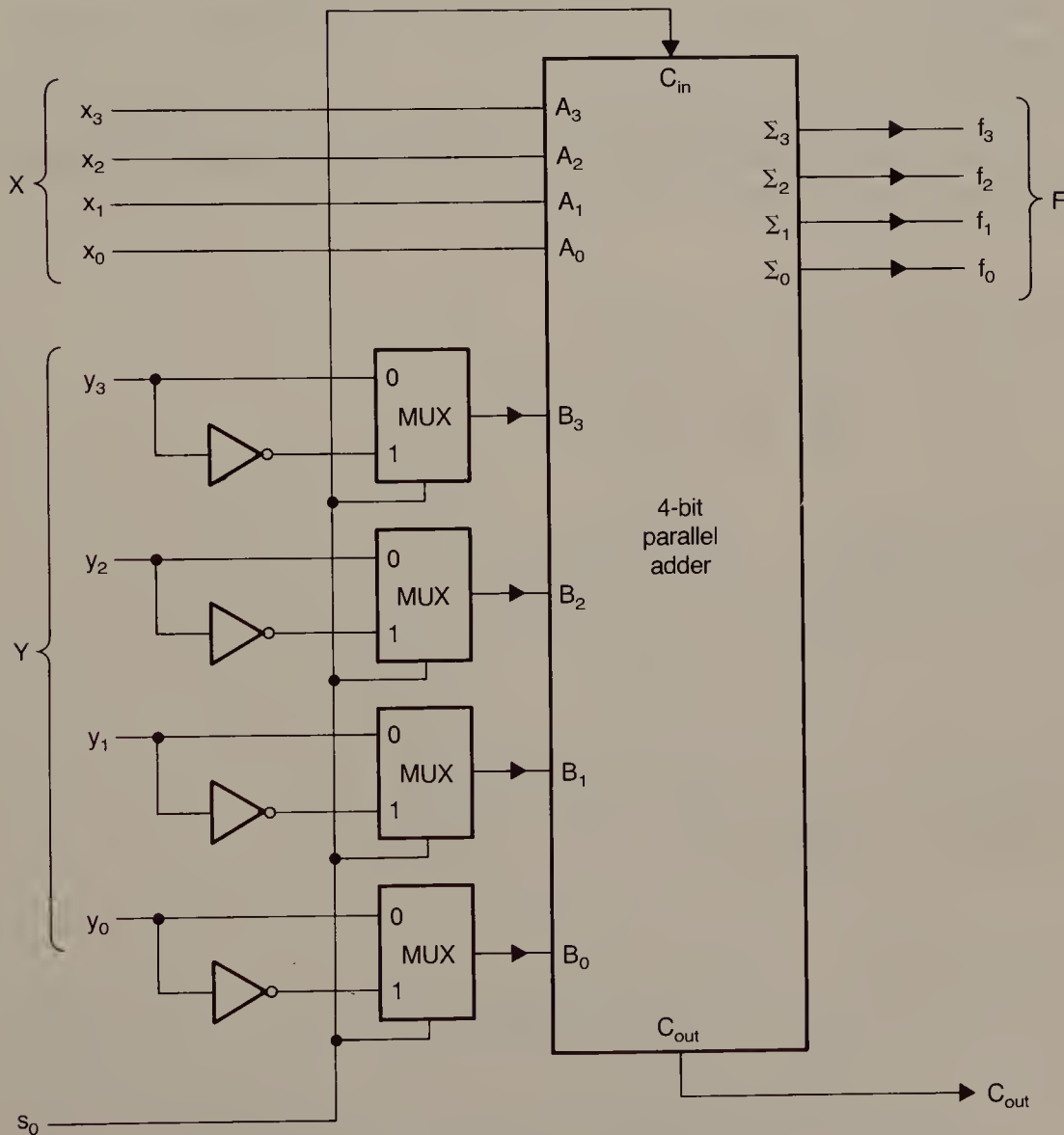


Figure 3.22 Organization of an Arithmetic Unit

For the second step, let us design a two-function logic unit; this is shown in Figure 3.23.

From Figure 3.23, it can be seen that when $s_0 = 0$, the output $G = X \text{ AND } Y$; otherwise the output $G = X \oplus Y$. Note that from these two Boolean operations, other operations such as NOT and OR can be derived by the following Boolean identities:

$$1 \oplus x = x'$$

$$x \text{ OR } y = x \oplus y \oplus xy$$

Therefore, NOT, and OR operations can be obtained by using additional hardware and the circuit of Figure 3.23. The outputs generated by the arithmetic and logic units can be combined by using a set of four multiplexers, as shown in Figure 3.24.

From this organization it can be seen that when the select line $s_1 = 1$, the multiplexers select outputs generated by the logic unit; otherwise, the outputs of arithmetic unit are selected.

More commonly, the select line, s_1 , is referred to as the *mode input* since it selects the desired mode of operation (arithmetic or logic). A complete block diagram schematic of this ALU is shown in Figure 3.25.

The truth table illustrating the operation of this ALU is shown in Figure 3.26.

This table shows that this ALU is capable of performing 2 arithmetic and 2 logic operations on the 4-bit operands X and Y.

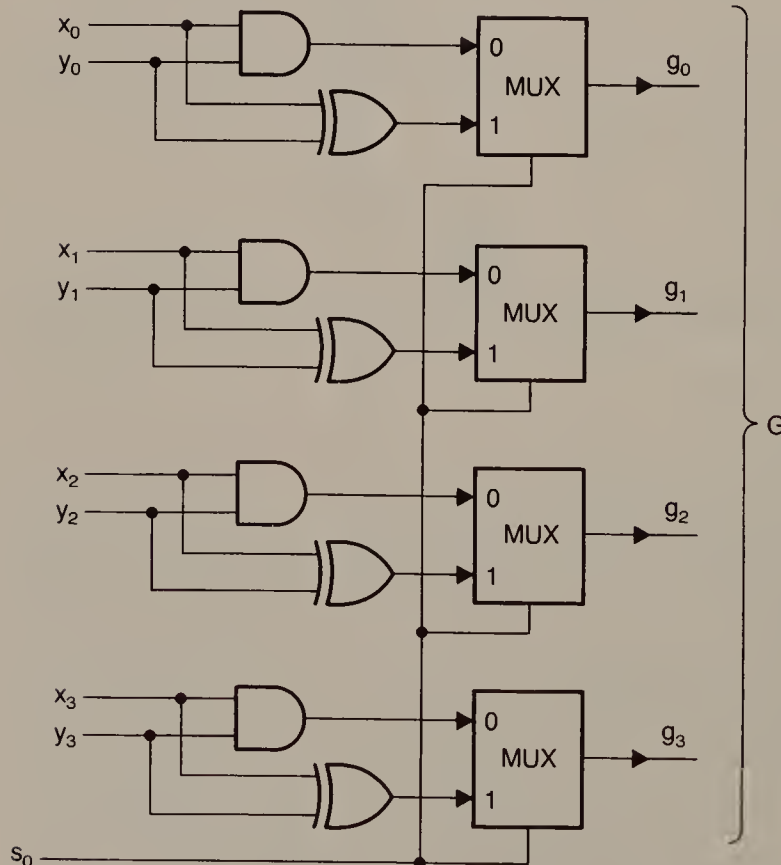


Figure 3.23 Organization of a 4-bit Two-function Logic Unit

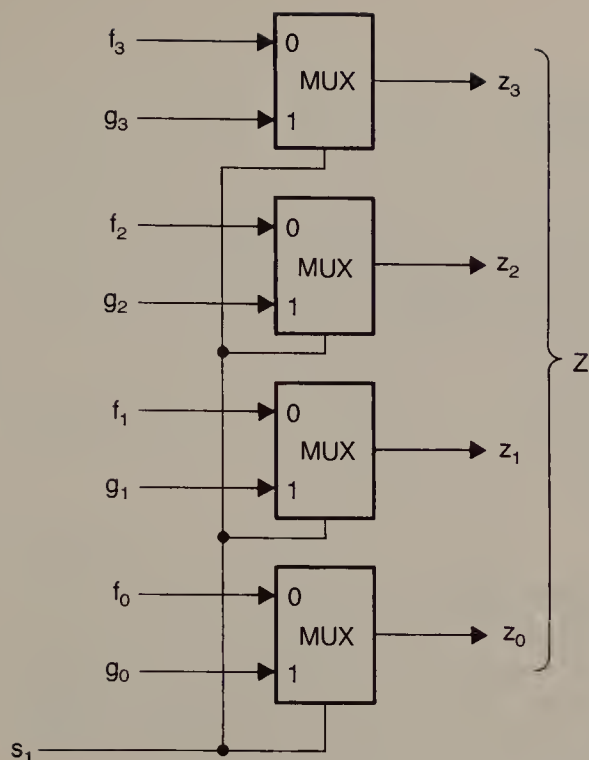


Figure 3.24 Combining the Outputs Generated by the Arithmetic and Logic Units

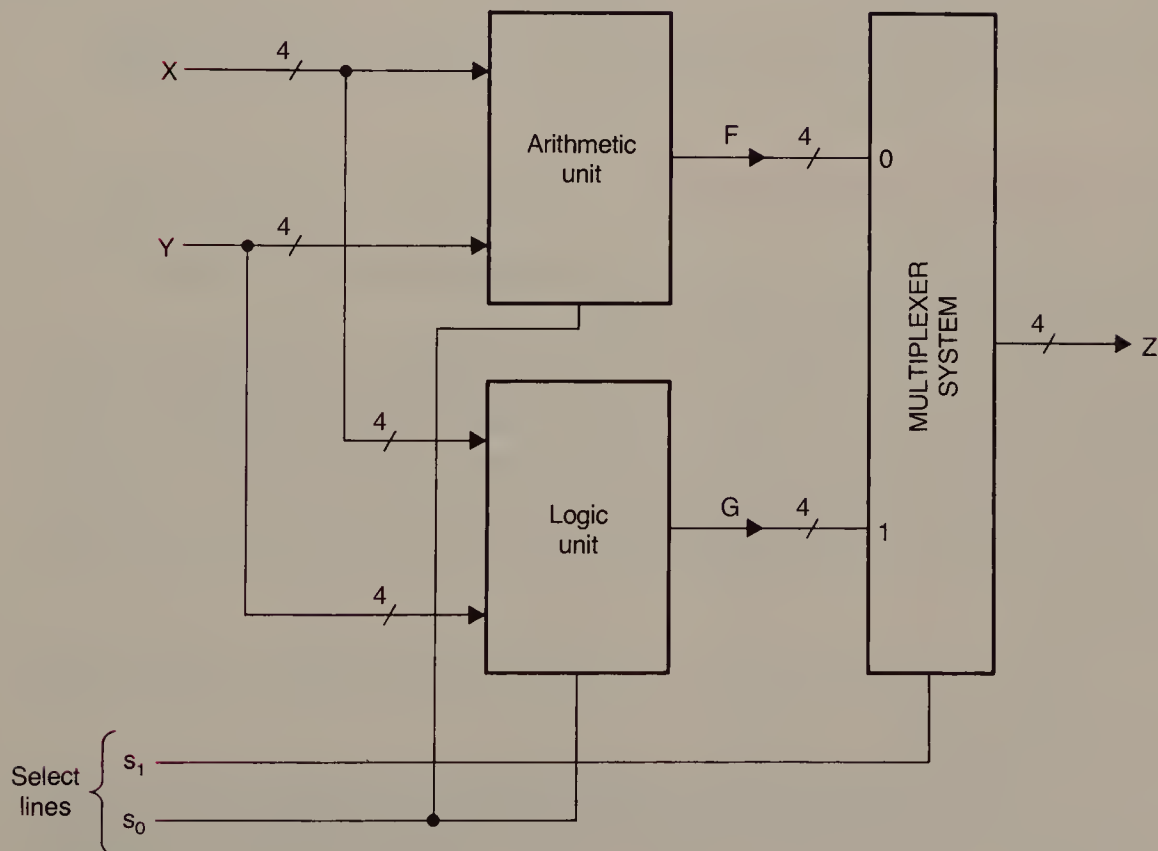


Figure 3.25 Schematic Representation of the Four-functions ALU (M. Morris Mano, *Digital Logic and Computer Design* © 1979 p. 377. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

Select Lines		Output Z	Comment
s_1	s_0		
0	0	X plus Y	Addition
0	1	X plus Y plus 1	2's complement subtraction
1	0	$X \wedge Y$	Boolean AND
1	1	$X \oplus Y$	Exclusive-OR

Figure 3.26 Truth Table Controlling the Operation of the ALU of Figure 3.25

The rapid growth in IC technology permitted the manufacturers to produce an ALU as an MSI block. Such systems implement many operations, and their use as a system component reduces the hardware cost, board space, debugging effort, and failure rate. Usually, each MSI ALU chip is designed as a 4-bit slice. However, a designer can easily interconnect n such chips to get a $4n$ -bit ALU. The most popular 4-bit ALU chips are the 74381 and 74181. The 74381 ALU performs 3 arithmetic, 3 Boolean, and 2 miscellaneous operations on 4-bit operands. The 74181 ALU performs 16 arithmetic and 16 Boolean operations on two 4-bit operands, using either active high or active low data.

A complete description and operational characteristics of these devices may be found in the original-equipment manufacturer's data books listed in the reference.

3.5 BIT-SLICE PROCESSOR

The chip count, which results in high power dissipation and poor reliability, are the main disadvantages of MSI functional blocks.

To eliminate these difficulties, a new approach called the *bit-slice technique* became feasible because of advances in IC technology. A bit-slice processor chip typically includes an ALU and a few registers. Using bit slices, a designer can build a processor of any word length. For example, a 4-bit slice processor includes a register file and an ALU for performing operations on 4-bit data, so that four such chips can be combined to build a 16-bit processor unit. In this section, we discuss the architecture and operational characteristics of a popular bit-slice processor—the Advanced Micro-Device's AM 2901.

The block diagram schematic of the AM 2901, 4-bit processor is shown in Figure 3.27. The key elements are the 16×4 two-port RAM, the high-speed ALU, and the accumulator register Q.

The RAM has two ports: Port A and Port B. Port A is a read-only port, and Port B may be used as a read/write port. Data from Ports A and B can be simultaneously read by specifying the appropriate addresses on the A and B address lines, though new data is always written into the RAM word whose address is specified by the B-address line.

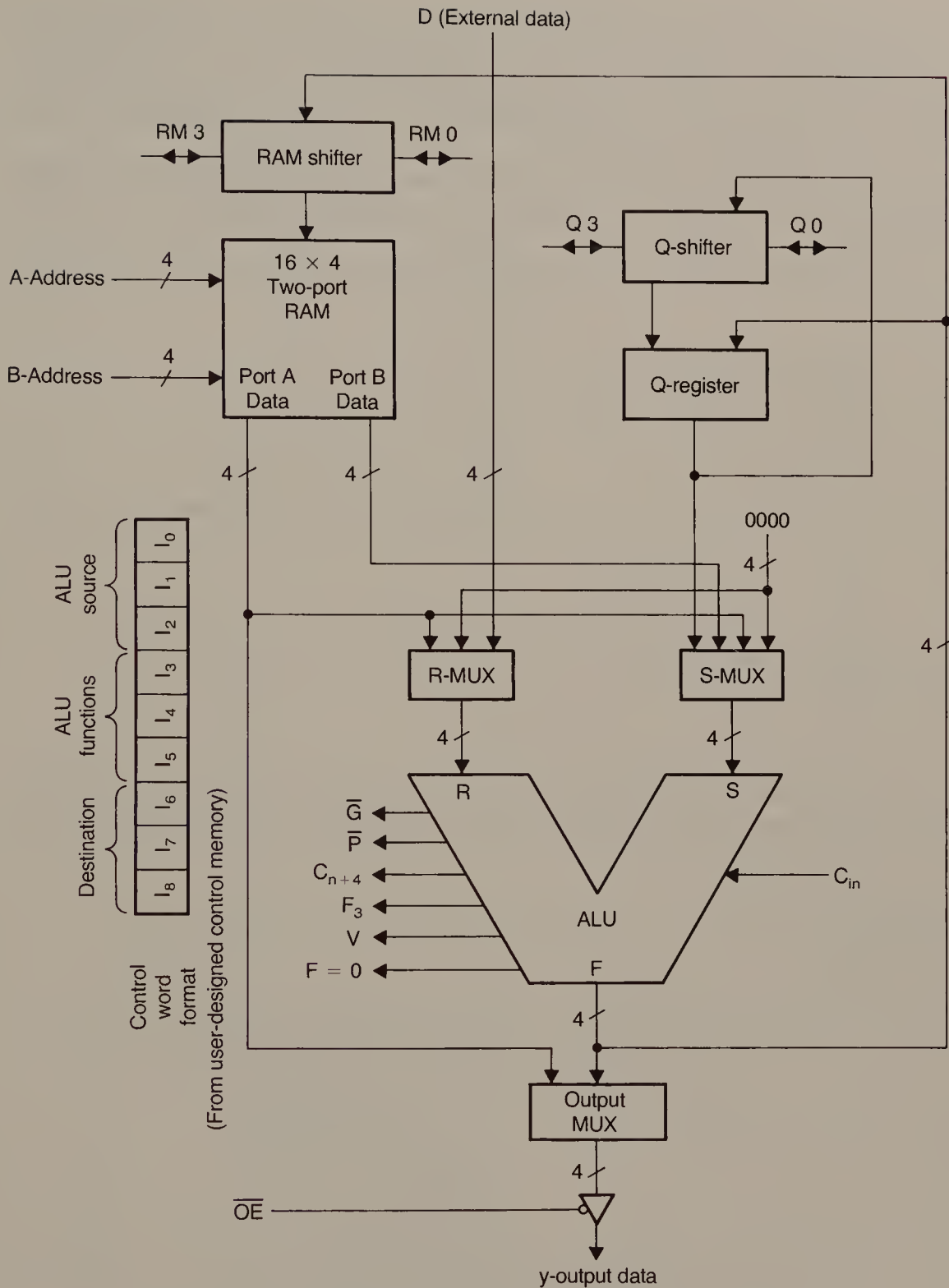


Figure 3.27 Block Diagram Schematic of the AM 2901 Bit-slice Processor (From J. P. Hayes, *Digital System Design and Microprocessors*, McGraw-Hill, 1984, p. 702. Reprinted by permission of McGraw-Hill)

The ALU performs 3 arithmetic and 5 Boolean operations. The entire operation of the bit slice is controlled by the 9-bit control word (I_8 through I_0). These control words are fetched from the control memory designed by the user (see Chapter 5). Three bits, I_5 , I_4 , and I_3 , are used to select the specific ALU operation. The interpretation of this 3-bit field is shown in Figure 3.28.

The ALU has two inputs—R and S. By examining Figure 3.27 it can be seen that the data for R and S inputs are selected by the multiplexers R-MUX and S-MUX, respectively.

The R-MUX selects one of the following three sources for the R input of the ALU:

- Output of the RAM-port A
- A zero value, which is 0000
- Four-bit external data D

The S-MUX selects one of the following four sources for the S input of the ALU:

- Output of the accumulator Q
- Output of the RAM Port B
- Output of the RAM Port A
- A zero value

This discussion suggests that there will be 12 distinct input pairs for the ALU. The 2901 designers have chosen only 8 of them, however. Each pair is selected using the bits I_2 , I_1 , and I_0 of the control word. The interpretation of this 3-bit field (source field) is summarized in Figure 3.29.

ALU Function Field			Operation Performed	Remarks
I_5	I_4	I_3		
0	0	0	R plus S plus C_{in}	Addition with a carry
0	0	1	S minus R minus C_{in}'	Subtraction with a borrow
0	1	0	R minus S minus C_{in}'	Subtraction with a borrow
0	1	1	$R + S$	Boolean OR
1	0	0	$R \cdot S$	Boolean AND
1	0	1	$R' \cdot S$	NOT (R) AND S
1	1	0	$R \oplus S$	Exclusive-OR
1	1	1	$(R \oplus S)'$	Exclusive-NOR

Figure 3.28 Interpretation of the ALU Function Field

ALU Source Field			Source Input Pair	
I_2	I_1	I_0	ALU input R	ALU input S
0	0	0	RAM port A	Q register
0	0	1	RAM port A	RAM port B
0	1	0	0000	Q register
0	1	1	0000	RAM port B
1	0	0	0000	RAM port A
1	0	1	External input D	RAM port A
1	1	0	External input D	Q register
1	1	1	External input D	0000

Figure 3.29 Interpretation of the ALU Source Field of the Control Word

In this system, there are three destination points, as follows:

- External output bus Y: This output, derived using output MUX, selects either ALU output F or RAM Port A. From Figure 4.27 it is shown that output y is a tristated output. This bus is in the high-impedance state when input $\overline{OE}$ is held high.
- Q register: This register obtains its input from either the ALU output or the Q-shifter. This is a combinational shifter that allows one to compute $Q/2$ or $2Q$ and has two bidirectional lines (Q3 and Q0) for the purpose of expansion.
- Two-port RAM: The output generated by the ALU is routed to the two-port RAM via the RAM shifter. This shifter is identical to the Q-shifter and it allows F, $F/2$, or $2F$ to be saved into the word address specified on the B-address line.

The destination points are selected using the bits I_8 , I_7 , and I_6 of the control word. A complete interpretation of this 3-bit field is summarized in Figure 3.30.

The AM 2901 chip generates the following status outputs:

C_{n+4}	:	End around carry
$\overline{G}$	:	Carry generate function
$\overline{P}$	:	Carry propagate function
F_3	:	Most-significant bit of the ALU (sign of the results)
Z	:	Zero status output (This output will be 1 if $F = 0$.)
V	:	Overflow status output

These outputs may be used to design the condition code register. The following examples demonstrate the versatility of the AM 2901 device.

Destination Field			Values Appear (or stored) in Different Destinations		
I ₈	I ₇	I ₆	Y	Two-port RAM	Q Register
0	0	0	F	None	F
0	0	1	F	None	None
0	1	0	RAM port-A	F	None
0	1	1	F	F	None
1	0	0	F	F/2	Q/2
1	0	1	F	F/2	None
1	1	0	F	2F	2Q
1	1	1	F	2F	None

Figure 3.30 Interpretation of the Destination-control Field of the AM 2901 Control Word

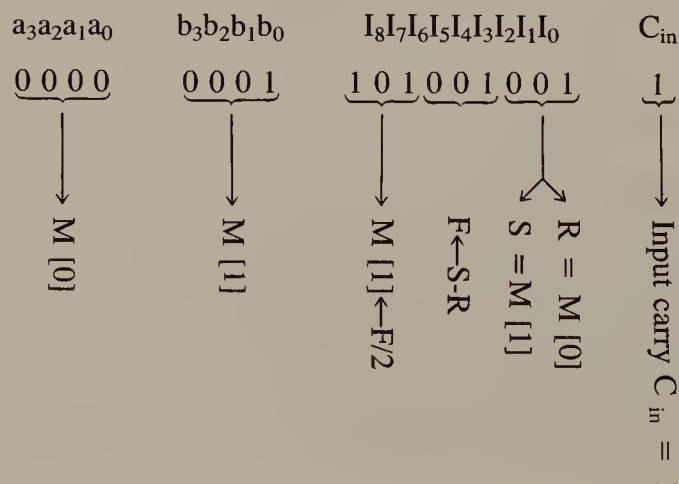
In this discussion, it is assumed that the complete control word is organized as follows:

A-ADDRESS	B-ADDRESS	DESTINATION FIELD	ALU FUNCTION	SOURCE FIELD	CARRY INPUT
a ₃ a ₂ a ₁ a ₀	b ₃ b ₂ b ₁ b ₀	I ₈ I ₇ I ₆	I ₅ I ₄ I ₃	I ₂ I ₁ I ₀	C _{in}

Using this format, the control words corresponding to some typical microoperations are written. For the first example, consider the following microoperation. (an operation completed during one clock cycle is called a *microoperation*):

$$M[1] \leftarrow (M[1] \text{ minus } M[0]) / 2;$$

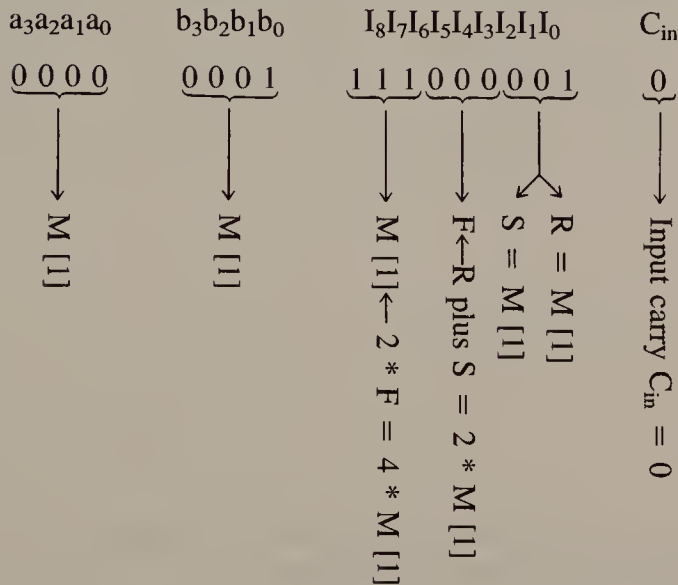
The symbol $M[i]$ denotes the RAM word whose address is i . The control word corresponding to this operation is shown next:



For a second example, here is another microoperation:

$$M[1] \leftarrow 4 * M[1]$$

The control word for this operation is shown next:



In both the examples, the ALU output F also appears at the external output bus Y , though this is discarded.

In general, one can obtain a 4K-bit processor by simply cascading K AM 2901s. In Figure 3.31 three AM 2901s are cascaded to form a 12-bit processing unit.

Some technical points relevant to this interconnection are summarized next:

- For all devices, except for the most significant, the bidirectional shift lines, Q_3 and RM_3 , are connected, respectively, to the bidirectional shift lines Q_0 and RM_0 of the more-significant devices.
- Since the $F = 0$ output is an open-collector output, the overall zero status output is obtained by wire ANDing the $F = 0$ output of each device.
- The final sign-status output is derived by using the F_3 output of the most significant device.
- The end-around carry status of the 12-bit system is the same as the C_{n+4} output of the most-significant device.

Finally, the cost of such a simple hardware configuration is the reduction in the operating speed, since a ripple-carry scheme is used to improve speed. The AM 2902, a 4-bit look-ahead-carry generator chip, may be used to increase the operating speed.

A complete discussion of this technique is beyond the scope of this chapter.

The AM 2901C and AM 2901C-1 are variations of the AM 2901 bit-slice RALU discussed in this section. The AM 2901C is a plug-in replacement for earlier versions of the AM 2901 but is considerably faster than its predecessors. The AM 2901C is a speed selected version of the AM 2901C, offering a 30% speed improvement on critical paths.

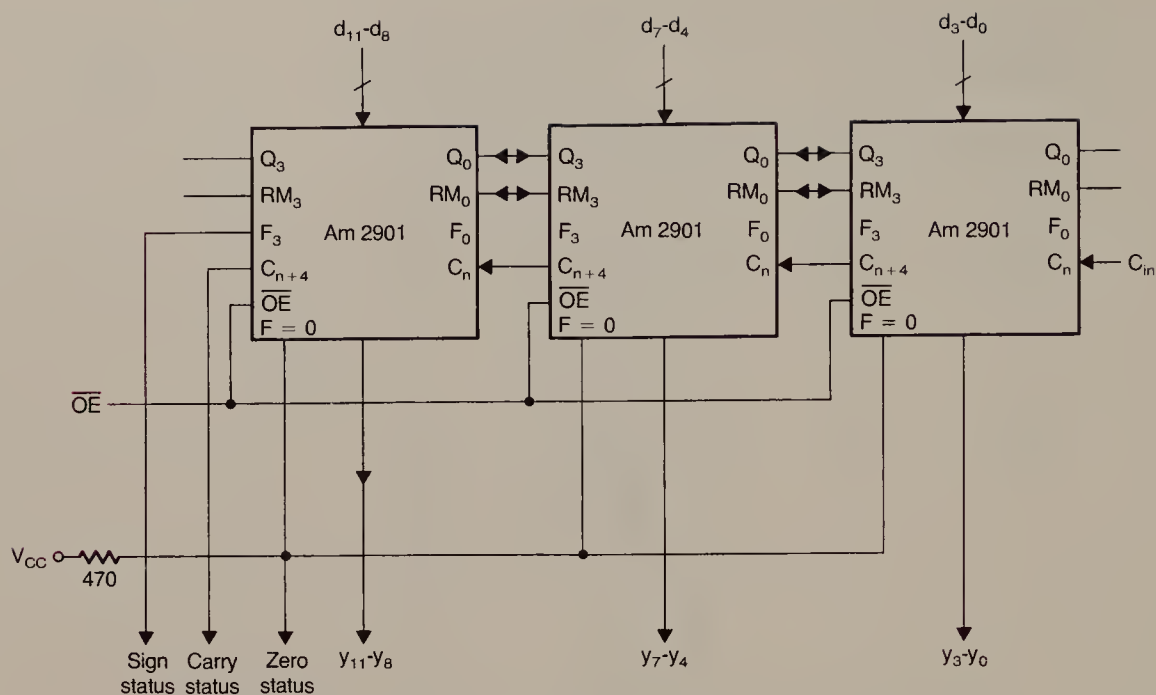


Figure 3.31 Cascaded Interconnection of Three AM 2901s (Copyright © Advanced Micro Devices, Inc., 1982. Reprinted by permission of copyright owner)

Another upgrade for the Am 2901 is the AM 2903 bit-slide device. The AM 2903 is a 4-bit expandable bipolar bit-slide ALU, which provides all functions performed by the AM 2901 and a number of enhancements useful in arithmetic-oriented processors, including a larger instruction set with nine special functions that use internal controls to facilitate operations such as 2's complement multiply, sign-extend, and normalization.

3.6 MULTIPLICATION OF UNSIGNED AND SIGNED INTEGERS

In a typical 8-bit processor such as the Intel 8085, the processor hardware does not include any provision to multiply two numbers, due to a limitation on the component density. Hence, the processor's instruction set does not include a multiplication instruction. In this situation, however, one can write a program that multiplies two numbers. Although this solution seems viable, the operational speed is unsatisfactory.

For application environments such as real-time digital filtering, in which the processor is expected to perform 32 to 64 eight-bit multiplication operations within 100 μ sec (sampling frequency = 10 KHZ), speed is an important factor.

New device technology such as high-density metal oxide semiconductors (HMOS) allows manufacturers to pack nearly a quarter of a million devices per chip. Consequently, state-of-the-art 32-bit microprocessors such as the MC68020 and I 80386 designed using this technology, have a larger instruction set than their predecessors, in-

cluding multiplication and division. In this section, multiplier design principles are discussed.

Two unsigned integers can be multiplied the same way as two decimal numbers are by hand. Consider the multiplication of two unsigned integers, where the multiplier $Q = 15$ and the multiplicand is $M = 14$, as illustrated:

$M \longrightarrow 1110 \longrightarrow$ Multiplicand (14_{10})

$Q \longrightarrow 1111 \longrightarrow$ Multiplier (15_{10})

```

      _____
      1110 ←
      1110 ←
      1110 ← Partial products
      1110 ←
      _____

```

$P \longrightarrow 11010010 \longleftarrow$ Final product

In the paper and pencil algorithm, shifted versions of multiplicands are added. This procedure can be implemented by using combinational circuit elements such as AND gates and FULL adders.

Generally, a 4-bit unsigned multiplier Q and a 4-bit unsigned multiplicand M can be written as

$M: m_3m_2m_1m_0$

$Q: q_3q_2q_1q_0$

The process of generating the partial products and the final product can also be generalized as shown in Figure 3.32.

Each cross-product term (m_iq_j) in this figure can be generated using an AND gate. This requires 16 AND gates to generate all cross-product terms that are summed by full adder arrays, as shown in Figure 3.33(b).

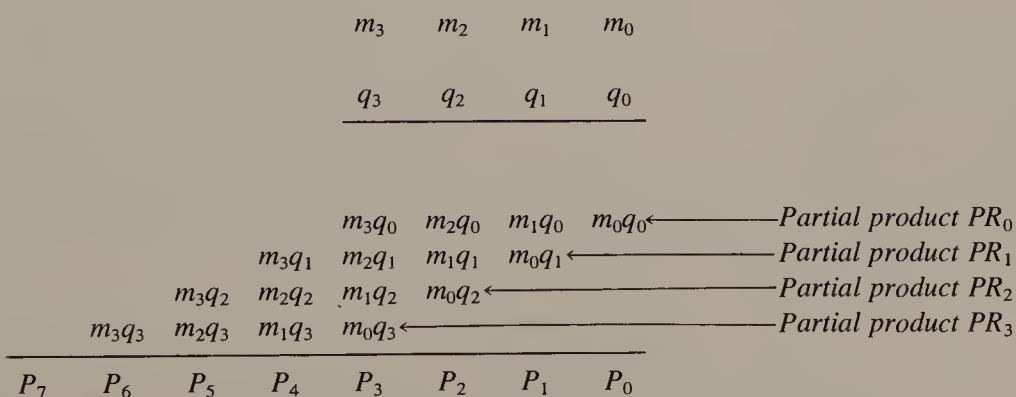


Figure 3.32 Generalized Version of the Multiplication of Two 4-bit Numbers Using the Paper and Pencil Algorithm

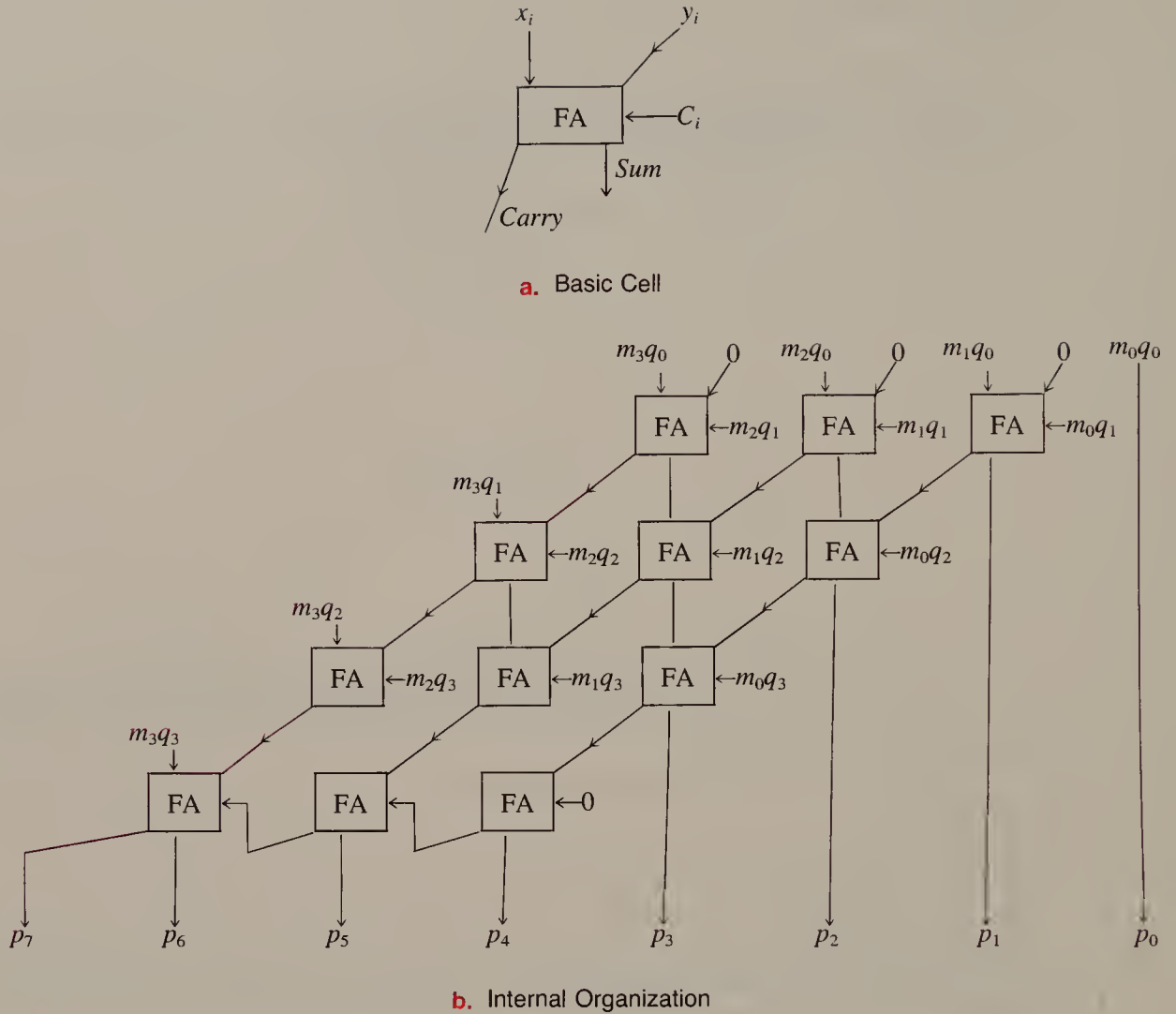


Figure 3.33 4×4 Array Multiplier

Consider the generation of p_2 in Figure 3.33(b). From Figure 3.32, p_2 is the sum of m_2q_0 , m_1q_1 and m_0q_2 . The sum of these three elements is obtained by using two full adders. (See column for p_2 in Figure 3.33(b)). The top full-adder in this column generates the sum $m_2q_0 + m_1q_1$. This sum is then added to m_0q_2 by the bottom full-adder along with any carry from the previous full-adder for p_1 .

The time required to complete the multiplication can be estimated by considering the longest carry propagation path comprising of the rightmost diagonal (which includes the full-adder for p_1 and the bottom full-adders for p_2 and p_5), and the last row (which includes the full-adder for p_6 and the bottom full-adders for p_4 and p_5). The time taken to multiply two n -bit numbers can be expressed as follows:

$$T(n) = \Delta_{\text{AND gate}} + (n - 1)\Delta_{\text{carry propagation}} + (n - 1)\Delta_{\text{carry propagation}}$$

In this equation, all cross-product terms m_iq_i can be generated simultaneously by an array of AND gates. Therefore, only one AND gate delay is included in the equation.

Also, the rightmost diagonal and the bottom row contain $(n - 1)$ full-adders each for the $n \times n$ multiplier.

Assuming that $\Delta_{\text{AND gate}} = \Delta_{\text{carry propagation}} = 2 \text{ gate delays} = 2\Delta$, the preceding expression can be simplified as shown:

$$\begin{aligned} T(n) &= 2\Delta + (2n - 2)2\Delta \\ &= (4n - 2)\Delta \end{aligned}$$

The array multiplier that has been considered so far is known as Braun's multiplier. The hardware is often called a *nonadditive multiplier* (NM), since it does not include any additive inputs. An *additive multiplier* (AM) includes an extra input R; it computes products of the form

$$P = M * Q + R$$

This type of multiplier is useful in computing the sum of products of the form $\sum X_i Y_i$.

Both an NM and an AM are available as standard IC blocks. Since these systems require more components, they are available only to handle 4- or 8-bit operands.

Alternatively, the same 4×4 NM discussed earlier can be obtained using a 256×8 ROM as shown in Figure 3.34.

It can be seen that a given MQ pair defines a ROM address, where the corresponding 8-bit product is held. The ROM approach is attractive for small-scale multipliers because:

- The technological advancements allow the manufacturers to produce low-cost ROMs.
- The design effort is minimum.

In case of large multipliers, ROM implementation is unfeasible, since large-size ROMs are required. For example—in order to implement an 8×8 multiplier, a $2^{16} \times 16$ ROM is required. If the required 8×8 product is decomposed into a linear combination of four 4×4 products, an 8×8 multiplier can be implemented using four 256×8 ROMs and a few 4-bit parallel adders. This construction is left as an exercise.

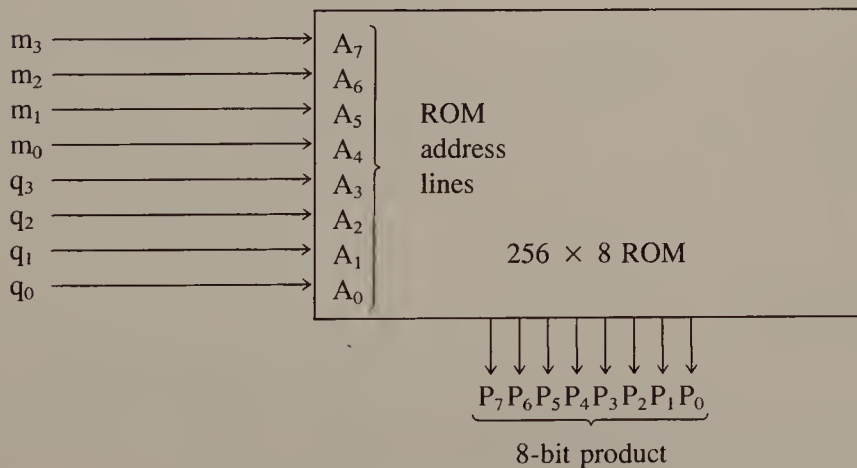
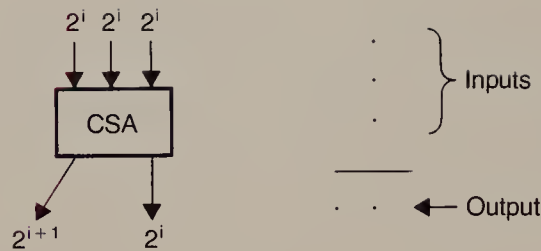


Figure 3.34 ROM-based 4×4 Multiplier

**a.** Block Diagram**b.** Dot Pattern**Figure 3.35** Three-input Wallace Tree

A fast 8×8 multiplier can be designed by using 4×4 NMs and Wallace tree. Recall that a Wallace tree is an interconnection of CSAs that reduces a column height N to less than 2. A 3-input Wallace tree and the associated dot pattern are shown in Figure 3.35.

The 3-input Wallace tree is implemented by using a CSA. The dot pattern shows

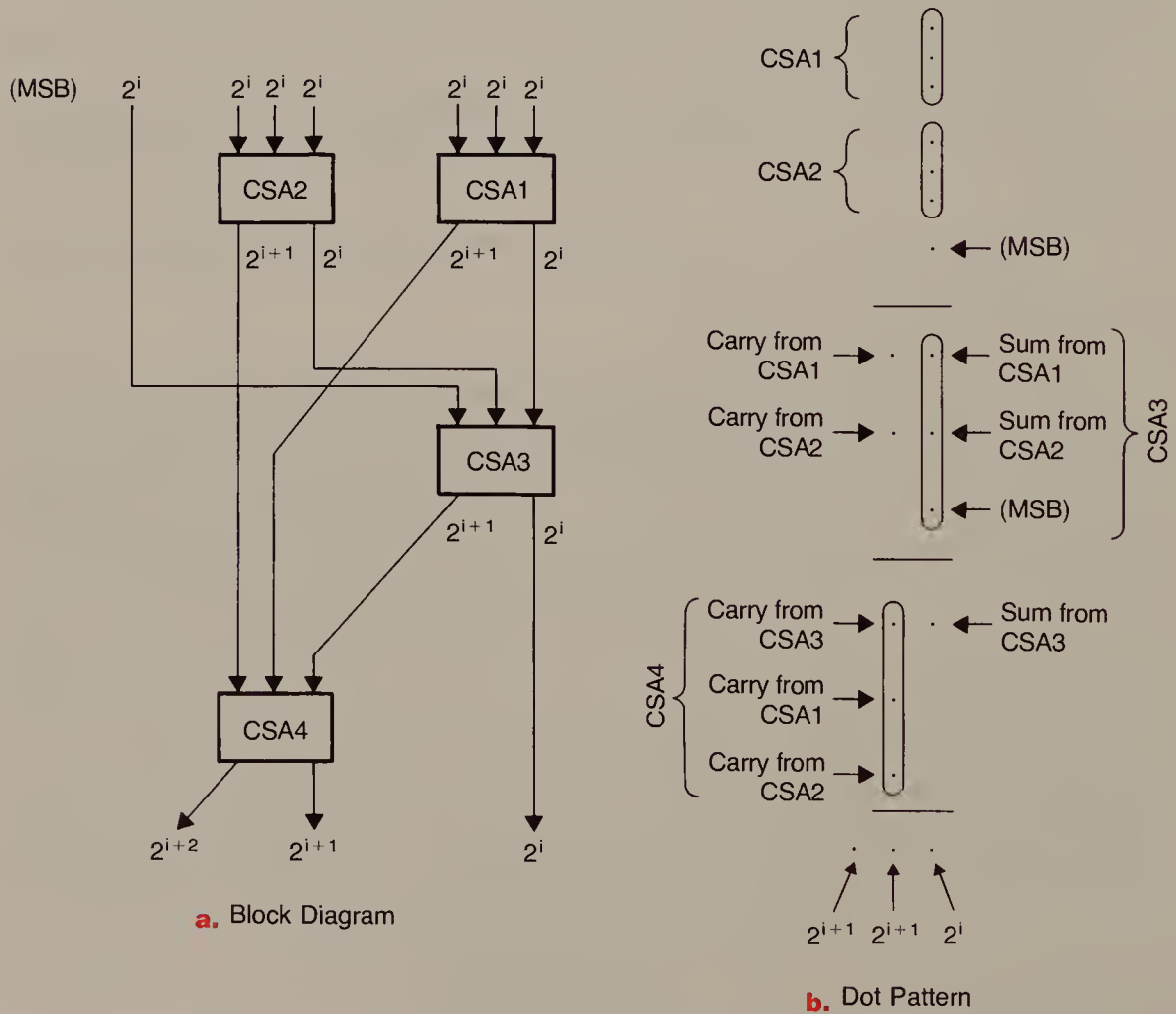
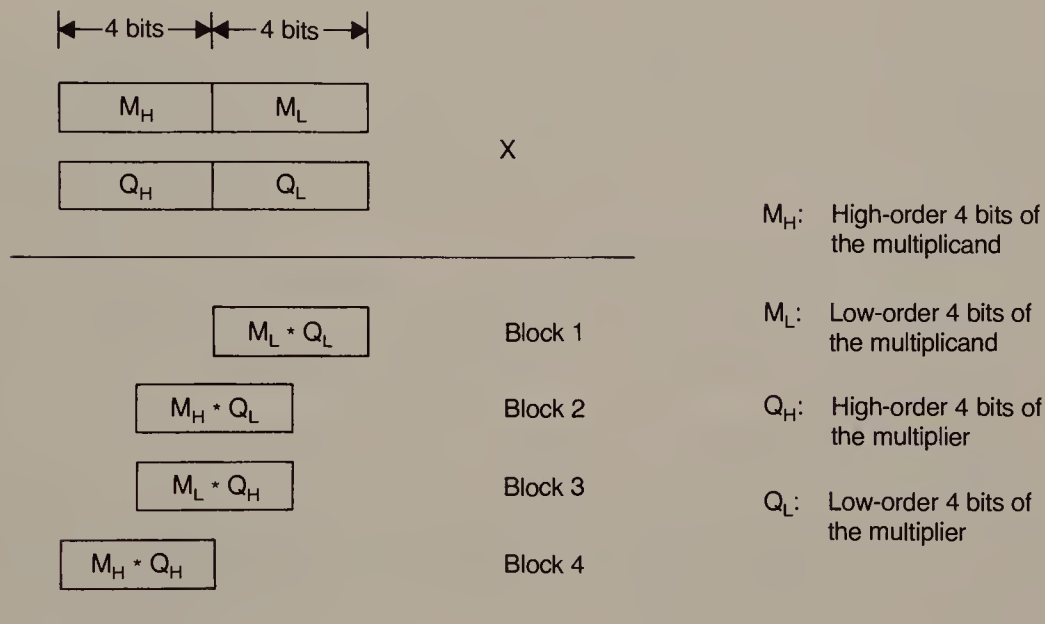
**a.** Block Diagram**b.** Dot Pattern

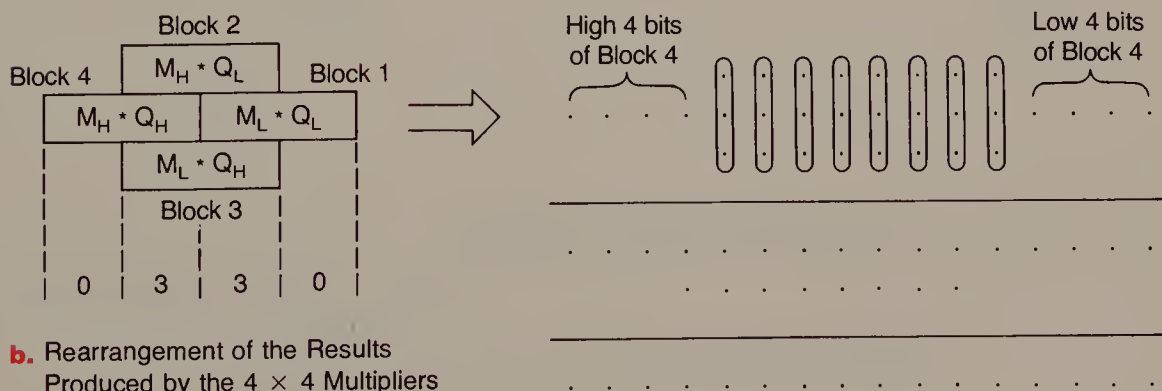
Figure 3.36 Seven-input Wallace Tree (From Norman R. Scott, *Computer Number System and Arithmetic*, Prentice-Hall, 1985, pp. 102–103. Adapted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

that this module actually reduces a column of height 3 to a height of 1. It is possible to design a 7-input Wallace tree using the 3-input Wallace tree as a building block. This process is shown in Figure 3.36.

With these Wallace tree adders and the 4×4 NMs, a fully combinational 8×8 or 16×16 multiplier can be constructed that works at great speed. Consider the design of an 8×8 multiplier. In this case, first divide the multiplicand M and multiplier Q into 4-bit groups (M_H , M_L , Q_H , Q_L) as indicated in Figure 3.37(a). Then use four 4×4 NMs to evaluate the products $M_L * Q_L$, $M_H * Q_L$ and $M_L * Q_H$. Each 4×4 multiplication produces an 8-bit product. These results can be rearranged as shown in Figure 3.37(b). The low-order 4 bits of block 1 and the high-order 4 bits of block 4 directly provide the final answer. The low-order 4 bits of block 4, high-order 4 bits of block 1, and all bits of blocks 2 and 3 must be added in the right order to obtain the



a. 8×8 Multiplication Using 4×4 Multipliers



b. Rearrangement of the Results
Produced by the 4×4 Multipliers

c. Dot Pattern

Figure 3.37 8×8 Multiplier Design

final answer. Therefore, blocks 1 and 4 are concatenated as a 16-bit number and placed in the middle of blocks 2 and 3.

The dot pattern corresponding to the rearranged products is shown Figure 3.38. The top row is block 2 (8 bits), the middle row contains the concatenation of blocks 1 and 4 (16 bits), and the bottom row is block 3 (8 bits).

It is apparent that the overlapping portions of these three rows provide a maximum column height of 3, and, therefore, eight 3-input Wallace tree adders are required. For the most-significant (high 4 bits of block 4) and the least-significant (low 4 bits of block 1) 4 columns, no Wallace tree adders are required, since the column height is 1. These observations are indicated by the number sequence 0 3 3 0 (0 means no Wallace tree adders are required, and 3 means a 3-input Wallace tree adder is required) underneath the rearranged pattern of Figure 3.37(b). Finally, a 12-bit CLA is used to obtain the 16-bit result. This process is shown by the dot pattern in Figure 3.38.

This concept can be extended to obtain 16×16 or 32×32 multipliers.

Even though a fully combinational Wallace tree multiplier operates at high speed, it is not practical for designing large multipliers, since it requires more hardware than a Braun multiplier. An alternative is the scheme proposed by L. Dadda—a modification of Wallace's scheme to achieve some hardware savings at the expense of reduction in the overall operational speed. Interested readers are suggested to consult the original work of L. Dadda [4] for more information.

The Wallace tree was used by the multiplier of the IBM 360/91. In order to reduce hardware, however, IBM designers have employed an iterative scheme through the use of a small Wallace tree multiplier (12×12 multiplier), which is used to perform 56×56 multiplications.

So far, unsigned combinational multipliers have been discussed. In small systems, where high-speed operation is unnecessary, two unsigned numbers can be multiplied, using an iterative approach (sequential multiplier). The key to this approach is that the product is the sum of appropriately shifted multiplicands. To illustrate this, consider the design of a 4×4 unsigned sequential multiplier. The hardware configuration of such a multiplier is shown in Figure 3.39.

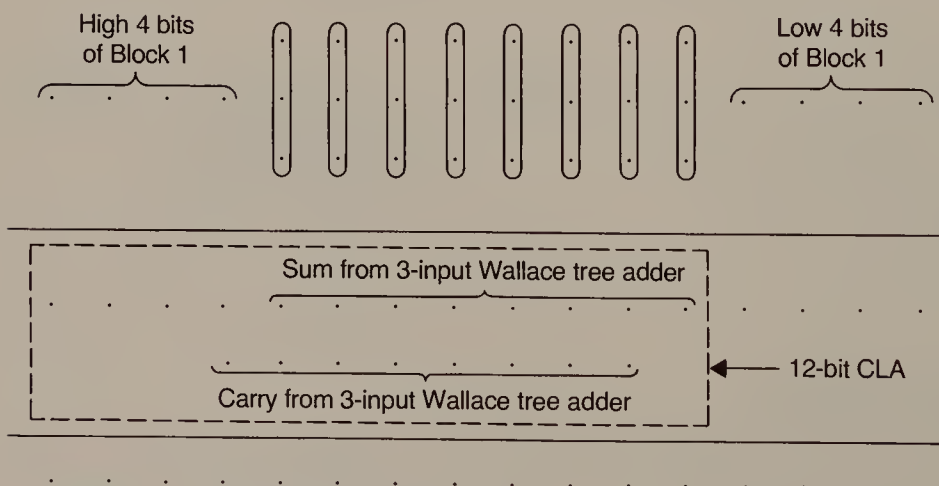
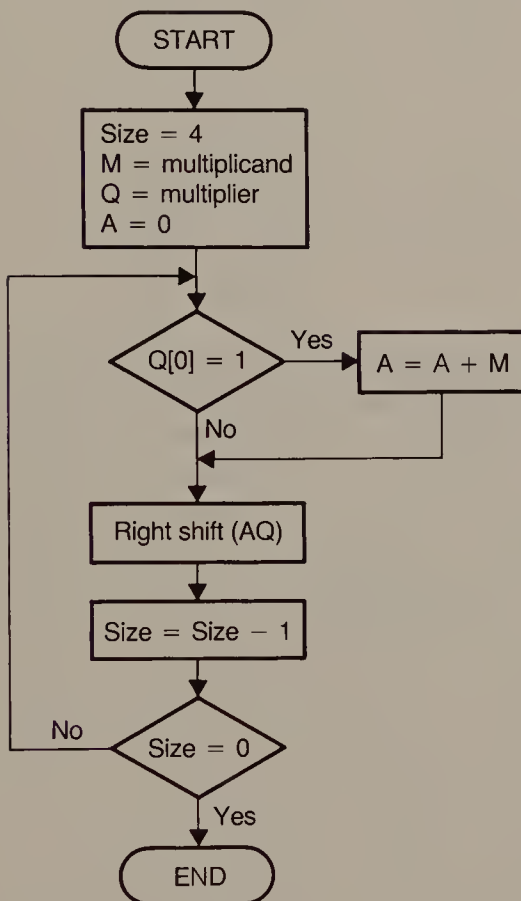


Figure 3.38 8×8 Multiplier Design

A 4×4 unsigned multiplier has three 4-bit registers, A, M, and Q. The A register, called the *accumulator*, is initially cleared to zero. When the algorithm is terminated, the A register holds the high-order 4 bits of the product. The Q register is initialized with the multiplier. When the algorithm is terminated, the Q register holds the low-order 4 bits of the product. The register pair AQ then holds the required 8-bit product. The M register always holds the multiplicand. The 4-bit parallel adder produces the sum of the contents of the A and M registers. The A register is then updated with the result produced by the parallel adder. The F flip-flop holds the end-around carry generated in the addition process. This carry is used as the serial input, while the register pair AQ is shifted one position to the right.

The multiplication algorithm can be described in the accompanying flowchart.



To illustrate this procedure, the following trace is provided, assuming that $M = 5_{10} = 0101_2$ and $Q = 12_{10} = 1100_2$:

	<i>M</i>	<i>F</i>	<i>A</i>	<i>Q</i>	<i>Size</i>
Initial configuration	0101	0	0000	1100	4

Iteration 1					
Shift AQ					
(because					
Q [0] = 0)	0101	0	0000	0110	3
Iteration 2					
Shift AQ					
(because					
Q [0] = 0)	0101	0	0000	0011	2
Iteration 3					
A := A + M					
(because					
Q [0] = 1)	0101	0	0101	0011	
Shift AQ	0101	0	0010	1001	1
Iteration 4					
A := A + M					
(because					
Q [0] = 1)	0101	0	0111	1001	
Shift AQ	0101	0	0011	1100	
					Product = 60 ₁₀

From the preceding trace, it can be seen that this procedure requires n iterations, where n is the number of bits in the multiplier. Observe that the operation $A := A + M$ is performed only when $Q[0] = 1$. Also, for $Q[0] = 0$ or 1 , the content of register

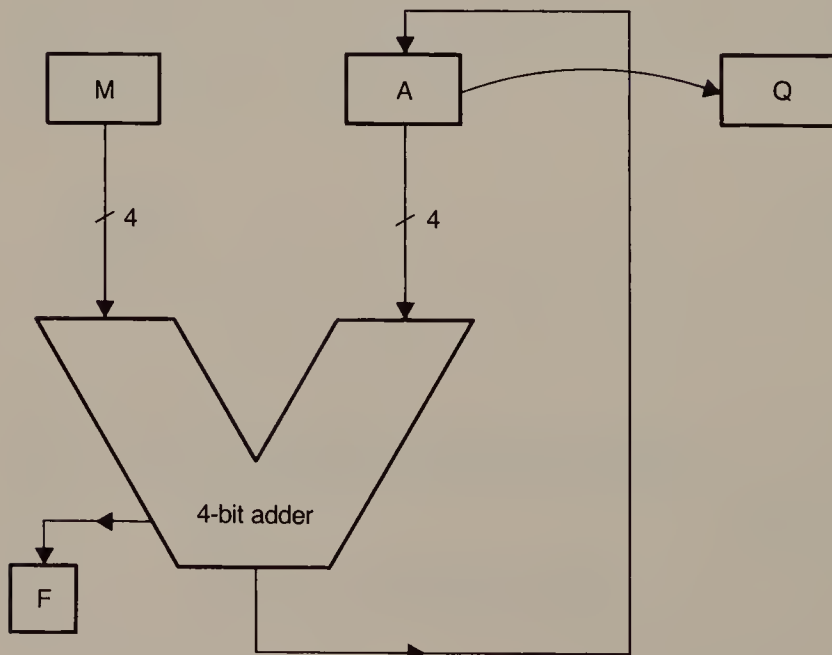


Figure 3.39 4 × 4 Unsigned Sequential Multiplier Hardware

pair AQ is shifted. In the worst case, (when all multiplier bits are 1), n additions plus n right shifts are needed. Therefore, whenever the multiplier contains fewer ones, more speed is achieved.

In the case of signed numbers, there are three possibilities:

1. M and Q are in sign-magnitude form.
2. M and Q are in 1's complement form.
3. M and Q are in 2's complement form.

For the first case, multiply the magnitudes either combinational or sequentially. The sign of the product is determined as $M_n \oplus Q_n$, where M_n and Q_n are the most-significant bits (sign bits) of the multiplicand (M) and the multiplier (Q), respectively. For the second case, proceed as follows:

Step 1: If $M_n = 1$, then compute the 1's complement of M.

Step 2: If $Q_n = 1$ then compute the 1's complement of Q.

Step 3: Multiply the $n - 1$ bits of the multiplier and the multiplicand either combinational or sequentially.

Step 4: $S_n = M_n \oplus Q_n$

Step 5: If $S_n = 1$, then compute the 1's complement of the result obtained in Step 3.

Whenever the 1's complement of a negative number (sign bit = 1) is taken, the sign is reversed. Hence, with respect to the multiplier, the inputs are always a positive quantity. When the sign of the result is negative, however, ($M_n \oplus Q_n = 1$), the result must be represented in 1's complement form. This is why the 1's complement is computed of the product found by the unsigned multiplier. When M and Q are in 2's complement form, the same procedure is repeated, with the exception that the 2's complement must be determined when $Q_n = 1$, $M_n = 1$, or $Q_n \oplus M_n = 1$. Consider M and Q as 2's complement numbers. Suppose $M = 1100_2$ and $Q = 0111_2$. Since $M_n = 1$, take the 2's complement of $M = 0100_2$; since $Q_n = 0$, do not change Q. Multiply 0111_2 and 0100_2 using the unsigned multiplier (sequential or combinations) discussed before. The product is 00011100_2 . The sign of the product is $S_n = M_n \oplus Q_n = 1 \oplus 0 = 1$. Hence, take 2's complement of the product 00011100_2 to obtain 11100100_2 , which is the final answer: -28_{10} .

The multiplication of signed numbers requires extra processing steps besides the core unsigned multiplication. This is an overhead when operands are denoted in 1's or 2's complement form. The overhead is at a maximum when M and Q are in 2's complement form, since computation of 2's complement requires an increment operation besides the bit-by-bit complement. This overhead can be eliminated by the recoded multiplication approach, in which the multiplier is mapped in accordance with the recoding procedure, a specific mapping rule.

The key to all recording procedures is the string property:

In a binary sequence, a block of consecutive k 1s may be replaced with a block of $k - 1$ consecutive 0s surrounded by the digits 1 and $\bar{1}$.

For example, consider the following sequence:

$$\begin{array}{c} 5 \text{ ones} \\ 0 \ 0 \ \overbrace{11111} \ 0 \end{array}$$

By the string property it may be rewritten as follows;

$$\begin{array}{c} 0 \ 0 \ 11111 \ 0 \\ \Downarrow \\ 0 \ 1 \ 0000\bar{1} \ 0 \end{array}$$

In the rewritten form, a block of four 0s surrounded by the digits 1 and $\bar{1}$ is present. If a weight of -1 is assigned to the digit $\bar{1}$, assuming that the original sequence represents a 2's complement number, both the original and the new patterns represent the same number. This is shown next:

$$\begin{array}{lcl} & -2^7 2^6 2^5 2^4 2^3 2^2 2^1 2^0 & \\ \text{Original pattern:} & 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 0 & = 2^5 + 2^4 + 2^3 + 2^2 + 2^1 = 62_{10} \\ \text{Recoded pattern:} & 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ \bar{1} \ 0 & = 1 \cdot 2^6 + (-1)2^1 = 62_{10} \end{array}$$

It must be emphasized that the string property also works if the sequence is interpreted as an unsigned integer. Since the string property increases the density of zeros in a given multiplier, the number of addition (or subtraction) operations to be performed is reduced when the recoded multiplier is used. This allows for quick multiplication.

Booth's recording procedure, a method that makes use of the string property, inspects two multiplier bits and decides which action to take, as shown in Figure 3.40. In this algorithm the multiplier is scanned from the right, 2 bits at a time. These decide the action to be performed. In the beginning, the index is $j = 0$; q_{j-1} is assumed to be 0. Following are two examples that illustrate the mechanics of Booth's recording procedure:

Given multiplier (implicit value = +7)

$$\begin{array}{ccccccc} 0 & 1 & 1 & 1 & 0 \leftarrow q_{j-1} & \text{(assumed to be zero)} \\ \downarrow & \downarrow & \downarrow & \downarrow & & \\ 1 & 0 & 0 & \bar{1} & & \end{array}$$

Booth's recoded multiplier (implicit value = $8 - 1 = 7$)

Given multiplier

$$\begin{array}{ccccccccccc} 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \leftarrow q_{j-1} & \text{(assumed to be 0)} \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & & \\ 0 & \bar{1} & 1 & 0 & 0 & \bar{1} & 0 & 0 & & \end{array}$$

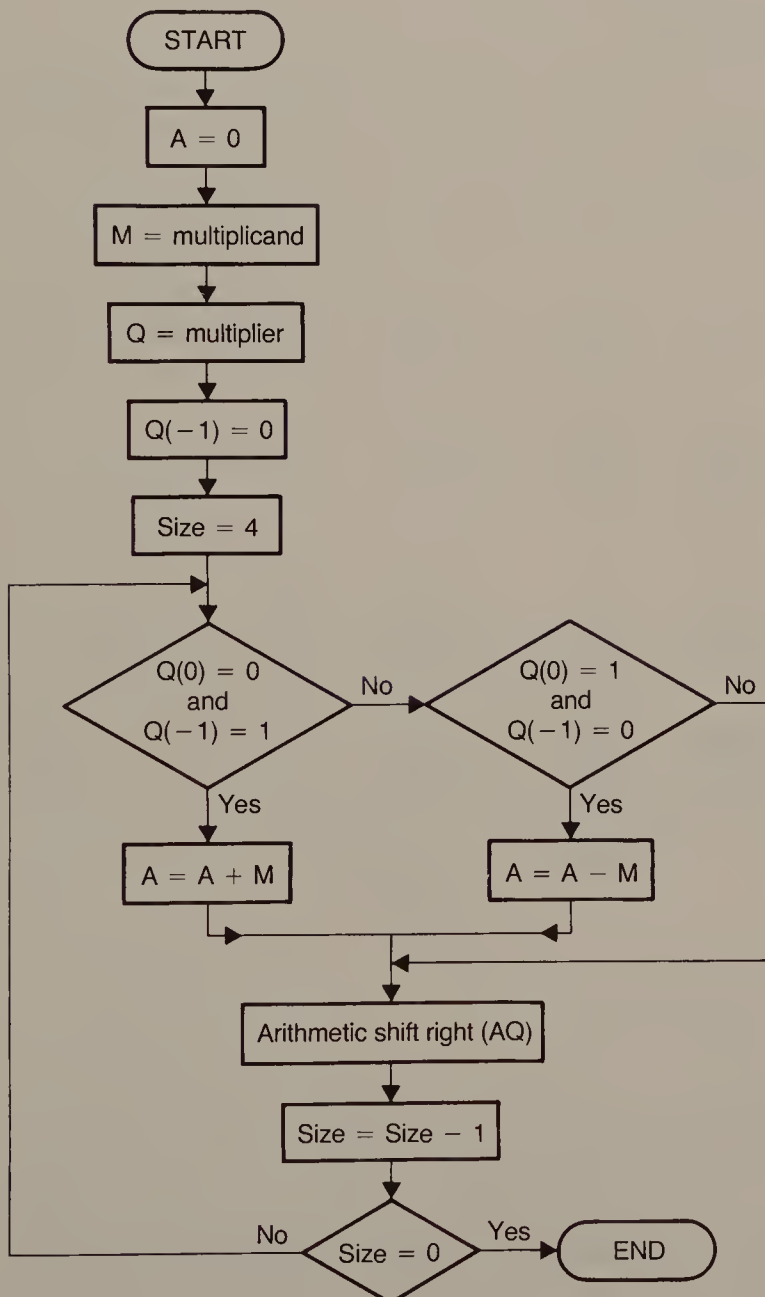
Booth's recoded multiplier (implicit value = $-64 + 32 - 4 = -36$)

(In these examples, it is assumed that the multipliers are represented in 2's complement form).

Booth's procedure can be implemented by modifying the sequential multiplier hardware (see Figure 3.39) as follows:

- Extend the Q register size from 4 to 5 bits so that the extra position will initially hold the fictitious zero.
- Replace the 4-bit parallel adder with a 4-bit adder/subtractor. (See Figure 3.21.)

The accompanying flowchart specified the mechanics of a 4×4 two's complement sequential Booth's multiplier:



Multiplier Bits Inspected $q_j q_{j-1}$	Recorded Digit Underneath Position j	Implied Action in Position j
0 0	0	None
0 1	1	Add M
1 0	$\bar{1}$	Sub M
1 1	0	None

Figure 3.40 Booth's Recording Procedure

To see how this procedure works, the following trace is provided. Assume that $M = -4 = 1100$ and $Q = 7 = 0111$.

	<i>M</i>	<i>A</i>	<i>Q</i>	
Initial configuration	1100	0000	01110	$q_{j-1} = \text{assumed } 0$ Bit pair inspected = 10
Iteration 1				
A: = A - M	1100	0100	01110	
Shift (AQ)	1100	0010	00111	Bit inspected = 11
Iteration 2				
Shift (AQ)	1100	0001	00011	Bit pair inspected = 11
Iteration 3				
Shift (AQ)	1100	0000	10001	Bit pair inspected = 01
Iteration 4				
A: = A + M	1100	1100	10001	
Shift (AQ)	1100	1110	01000	
				Product = 11100100
				= 100100
				= <u>-28</u>

From this trace, notice the following:

- Preprocessing steps are unnecessary, so Booth's algorithm treats signed numbers in a uniform manner.
- Only one addition and one subtraction are needed to carry out this multiplication. (Observe that $M * 7 = M * (8 - 1) = (8M - M)$).

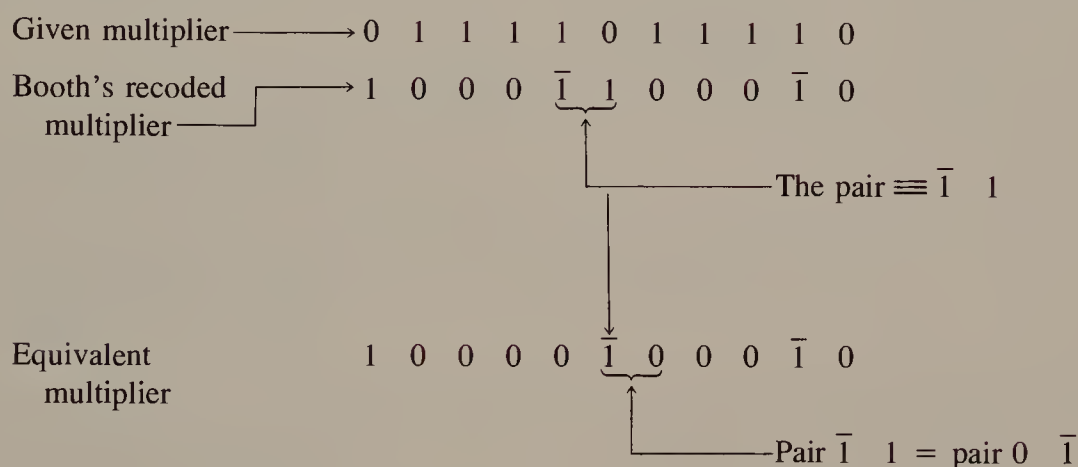
Booth's algorithm is called a *skipping over 1 technique*, since it skips over a block of consecutive ones. This process resembles the differentiation process; therefore,

Booth's recording procedure is referred to as a *differential recoding scheme*. (See N. Scott [15].)

Booth's procedure can be made more efficient if the following situations are considered:

	BOOTH'S PAIR	EQUIVALENT PAIR	REASON
Weight $\longrightarrow$	$2^n \quad 2^{n-1}$	$2^n \quad 2^{n-1}$	
	$\bar{1} \quad 1$	$0 \quad \bar{1}$	$-2^n + 2^{n-1} = -2^{n-1}$
	$1 \quad \bar{1}$	$0 \quad 1$	$2^n - 2^{n-1} = 2^{n-1}$

Booth's algorithm generates the pair $\bar{1} \ 1$ when there is a single zero and a continuous sequence of ones:



This single 0 is called an *isolated 0*. If the equivalence bit pair $\bar{1} \ 1 =$ bit pair $0 \ 1$ is applied, the final equivalent multiplier can be obtained. The final form is preferred, since it contains only three nonzero digits (whereas the Booth's multiplier contains four nonzero digits). Since the application of the equivalence rule increases the density of zeros, the multiplication can be performed quickly.

Likewise, Booth's algorithm generates the pair $1 \ \bar{1}$ when there is a single 1 in a continuous sequence of zeros:

Given multiplier	0 0 0 0 1 0 0 0
Booth's recoded multiplier	0 0 0 1 $\bar{1}$ 0 0 0

This single 1 is called an *isolated 1*. The equivalence pair $1 \ \bar{1} =$ pair $0 \ 1$ is applied to the Booth's recoded multiplier, and the original multiplier is returned. The original multiplier is preferred since it contains only one nonzero digit (whereas the Booth's recoded multiplier has two nonzero digits).

When the following modifications are introduced to the conventional Booth's recoding procedure, fewer nonzero digits result from the recoded multiplier:

- If an isolated zero is present, place a 1 into the corresponding position of the recoded multiplier, and continue the scanning process.
- Do not recode an isolated 1.

In order to implement these modifications, an additional bit m is introduced—the *mode bit*. Initially, the mode bit is reset to 0. It is set to 1 only when the beginning of a block of 1s is detected. Similarly, when the end of a 1s block is recognized, the m bit is reset to zero. When the extra bit is included, the modified Booth's recoding procedure is obtained, as shown in Figure 3.41.

When the modified Booth's recoding procedure is used, the following points must be considered:

m	q_{j+1}	q_j	Recoded Digit underneath position j	Action in Position j	Reason
0	0	0	0	None	The string of 1s has not yet started so continue scanning
0	0	1	1	Add M	This is an isolated 1, and therefore it is not recoded. This means that 1 is plugged into the recoded multiplier as well.
0	1	0	0	None	This may be an isolated 1, and thus nothing is done until the next pair is inspected.
0	1	1	$\bar{1}$	SUB M	The beginning of a string of 1s is recognized; A $\bar{1}$ is plugged in and m is set to 1.
1	0	0	1	Add M	Here, the string of 1s ends. A 1 is plugged in and m is reset to 0.
1	0	1	0	None	In this case, this 0 may be an isolated zero; nothing is done until the next pair is inspected
1	1	0	$\bar{1}$	SUB M	This 0 is an isolated 0. A $\bar{1}$ is plugged into the recoded multiplier.
1	1	1	0	None	The string of 1s continues. They are skipped without doing anything.

Figure 3.41 Modified Booth's Recoding procedure

- This procedure starts with the pair q_1q_0 . Therefore, no fictitious 0 is needed to the right of the least-significant bit of the multiplier.
- If the n -bit multiplier Q is expressed in 2's complement form, then sign-extending may be necessary
- If the n -bit multiplier Q is expressed as an unsigned integer, appending either one fictitious 0 or a pair of fictitious 0s (that is, 00) to the most-significant bit q_{n-1} may be required. In the case of unsigned integers, the most-significant bit is a positive quantity with a weighted SUM = $2^{n+1} \cdot q_{n-1}$. For 2's complement numbers, the most-significant bit indicates the sign of the number, so the weighted sum in this case becomes $(-2^{n-1}) \cdot q_{n-1}$.

Case 1.

Given multipliers	→ 1 0 1 1	Interpretation: It is in 2's complement form	Implicit value = -5
Value of m	→ 1 1 1 1 0		
Sign extension	→ 1 0 1 1		
Modified Booth's recoded multiplier	→ 0 1 0 1	Implicit value = $(-4) + (-1) = -5$	

Case 2.

Given Multiplier	→ 0 1 0 1	Interpretation: It is in 2's complement form	Implicit value = +5
Value of m	→ 0 0 0 0 0		
Sign Extension	→ 0 1 0 1		
Modified Booth's Recoded Multiplier	→ 0 1 0 1	Implicit value = +5	

Case 3.

Given multiplier	→ 1 0 1 1	Interpretation: unsigned integer	Implicit value = 11
Value of m	→ 0 1 1 1 1 0		
Pair of fictitious 0s	→ 1 0 1 1		
Modified Booth's recoded multiplier	→ 1 0 1 0 1	Implicit value = $16 - 4 - 1$ = $16 - 5 = 11$	

Case 4.

Given multiplier	→ 0 1 1 1	Interpretation: unsigned integer	Implicit value = 7
Value of m	→ 0 1 1 1 1 0		
Pair of fictitious zeros	→ $\overbrace{00}^{\text{fictitious zeros}}$ 1 0 1 1		
Modified Booth's recoded multiplier	→ 1 0 0 $\overline{1}$	Implicit value = $8 - 1 = 7$	

A multiplier recoding procedure is said to be *minimal* if it produces a minimum number of nonzero digits. A canonical multiplier recoding system is minimal and guarantees that all nonzero digits are separated by at least one 0. The modified Booth's recoding procedure is therefore a canonical recoding algorithm.

A recoding procedure can be devised that is capable of inspecting 3 bits simultaneously. This procedure, the *overlapping triplet technique*, is employed in the multiplier design of the IBM 360/91 CPU. The principal advantage of this approach is its guarantee of uniform shifting. A discussion of this topic may be found in the works of N. Scott [15] or K. Hwang [11]. Note that the method of overlapping triplets is also employed in the combinational 4×4 additive multipliers designed by manufacturers such as Texas Instruments and Advanced Micro-devices. Interested readers may consult the manufacturer's data book [17] for more information.

3.7 DIVISION OF UNSIGNED INTEGERS

This section describes how to divide two unsigned integers. Given a dividend (D) and a divisor (V), the quotient (Q) and remainder (R) are related as follows:

$$D = QV + R$$

In particular, a Q that will make the product VQ very close to D is needed. This implies that the remainder, R , of a division will always be in the range of $0 \leq R < V$. In many practical situations, only the quotient Q is desired. This means that the remainder, R , that is generated in the division process is ignored. Usually, the quotient, Q , is represented using a finite number of bits, and it may be necessary to approximate the quotient by truncation or rounding. It can then be represented by using a fixed number of allocated bits. When a high degree of accuracy is desired, however, the remainder (R) is further divided by the divisor (V) to produce a more-accurate quotient.

When examining the relation $D \approx Q \times V$, a great deal of similarity between multiplication and division can be recognized. The following are analogous pairs: the product and the dividend, the quotient and the multiplicand, and the multiplier and the divisor. Such a relationship implies that the same hardware unit used for multiplication can be used to perform division, after some modification. In a sequential multiplier, the product is obtained by adding shifted multiplicands. However, in the case of division, the quotient is obtained by repeatedly subtracting the shifted divisor from the dividend.

Subtractive division algorithms are derived from the paper and pencil approach. This process is illustrated in Figure 3.42 using $D = 22$ and $Q = 3$.

In this method, 1 quotient bit per iteration is generated. First, line up the divisor against the dividend and try to subtract it from the dividend. If the result of the subtraction is positive, then enter a 1 for the quotient, and shift the divisor one position to the right. Then, repeat the whole process. However, if the divisor cannot be subtracted from the dividend without yielding a negative result, enter a 0 for the quotient and shift the divisor to the right. Then, try to subtract the same from the dividend. This process continues until all bits of the dividend are covered.

This algorithm can be mechanized with the following modifications:

- Do not move the divisor. Instead shift the dividend to the left. When dividing two 4-bit numbers, use three 4-bit registers (M, A, and Q). The register M is used to hold the divisor. Register A and Q are initialized with the dividend. Initially, the A register is cleared to 0, and the Q register holds the 4-bit dividend. In each iteration, the contents of A and Q are shifted to the left first. The contents of M are then subtracted from the contents of A. If the result of subtraction is positive, a 1 is placed into the vacant position created by the left shift operation; otherwise a 0 is plugged into this position. The process is repeated until all the bits of the dividend originally stored in the Q register are moved into the A register. In this particular case, the entire operation must be repeated four times, since the size of the Q register is 4 bits. The A register is used as the accumulator. Therefore, the result of the subtraction is always saved in the A register. When the algorithm terminates, the A register contains the remainder and the Q register contains the quotient.
- When the result of subtraction yields a negative answer, restore the contents of A by adding the contents of A register with M before beginning the next iteration. This step explains why the algorithm is referred to as a *restoring division algorithm*.

To divide two n -bit numbers using the restoring division algorithm, repeat the steps shown in Figure 3.43 n times.

The hardware that implements the restoring division algorithm is given in Figure 3.44.

$$\begin{array}{r}
 111 \leftarrow \text{Quotient} \\
 011 \overline{) 10110} \\
 \underline{011} \\
 101 \\
 \underline{011} \\
 100 \\
 \underline{011} \\
 011 \leftarrow \text{Remainder}
 \end{array}$$

Figure 3.42 Paper and Pencil Division Technique

Step 1: Shift A and Q registers to the left.

Step 2: Subtract M from A.

Step 3: If the result of the subtraction is positive, set $Q[0] = 1$; otherwise set $Q[0] = 0$.

Figure 3.43 Basic Steps of the Restoring Division Algorithm

	<i>M</i>	<i>A</i>	<i>Q</i>	
Initial				
Configuration	00011	00000	1011	
<i>Iteration 1</i>				
Left shift (AQ)	00011	00001	011?	
$A = A - M$	00011	11110	011?	
Restore A and Reset Q [0] to 0 (since A [4] = 1)	00011	00001	0110	
<i>Iteration 2</i>				
Left shift (AQ)	00011	00010	110?	
$A = A - M$	00011	11111	110?	
Restore A and reset Q [0] to 0 (since A [4] = 1)	00011	00010	1100	
<i>Iteration 3</i>				
Left shift (AQ)	00011	00101	100?	
$A - A - M$	00011	00010	100?	
Set Q [0] to 1 (since A [4] = 0)	00011	00010	1001	
<i>Iteration 4</i>				
Left shift (AQ)	00011	00101	001?	
$A = A - M$	00011	00010	001?	
Set Q [0] to 1 (since A [4] = 0)	00011	00010	0011	Quotient = 3 Remainder = 2

From the preceding trace, notice that the quotient is 3 and the remainder is 2.

The restoring division algorithm discussed so far needs n subtractions and extra additions when dividing two n -bit numbers, due to the restoration process. The exact number of extra additions depends on the data, so it cannot be figured in deterministic manner. Assuming, however, that 50% of the subtraction operations yield a negative result, then $n/2$ extra additions are needed. By properly restructuring the restoring divi-

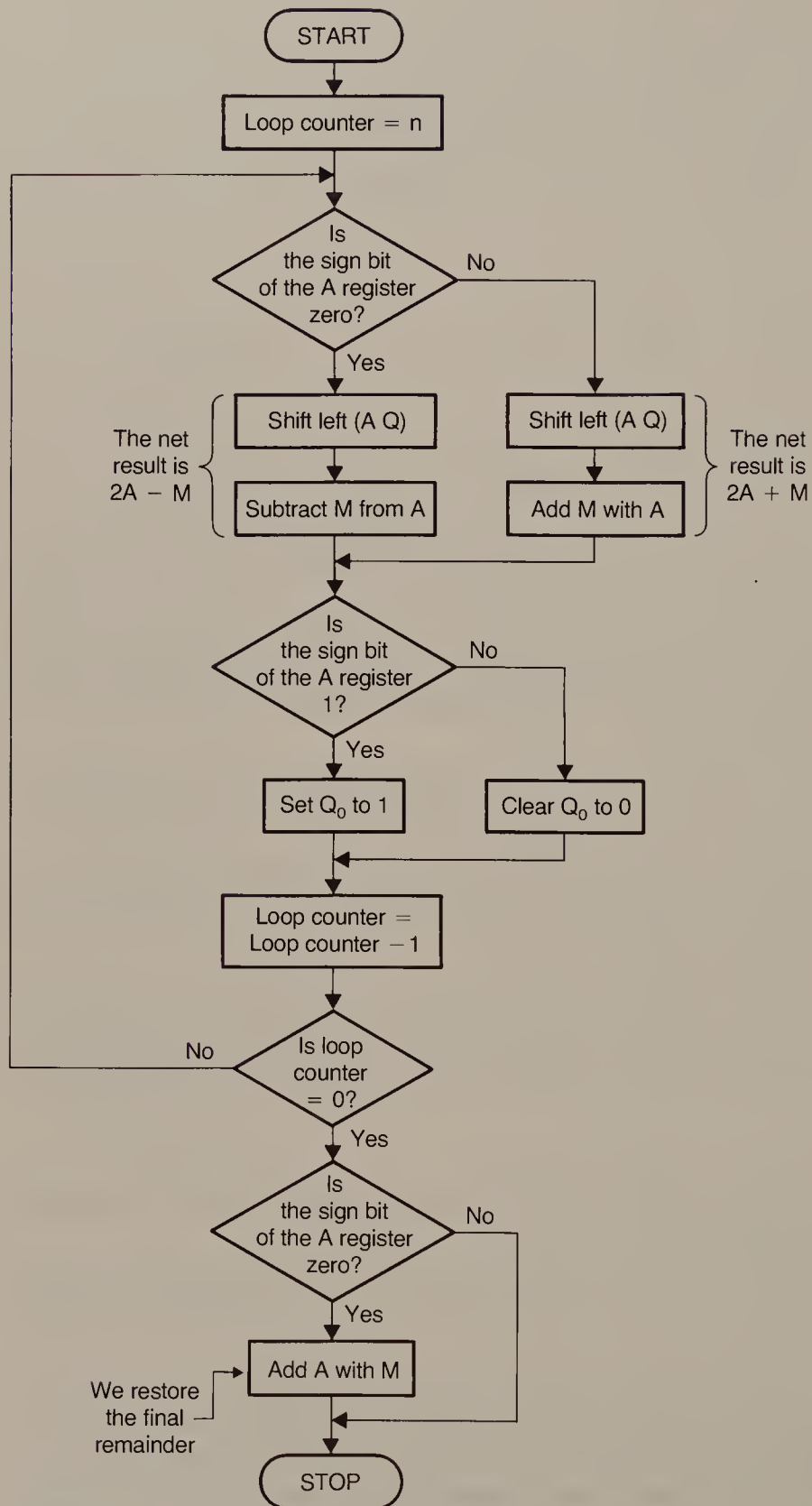


Figure 3.45 Salient Characteristics of the Nonrestoring Division Algorithm

To appreciate this concept, rework the previous division example using the nonrestoring division algorithm. Shown next is a complete trace of this process.

	<i>M</i>	<i>A</i>	<i>Q</i>
Initial configuration	00011	00000	1011
Iteration 1			
(Note that $A[4] = 0$)			
Shift left (AQ)	00011	00001	011-
$A: = A - M$	00011	11110	011-
Clear $Q[0]$			
(since $A[4] = 1$)	00011	11110	0110
Iteration 2			
(Note that $A[4] = 1$)			
Shift left (AQ)	00011	11100	110-
$A: = A + M$	00011	11111	110-
Clear $Q[0]$			
(Since $A[4] = 1$)	00011	11111	1100
Iteration 3			
(Note that $A[4] = 1$)			
Shift left (AQ)	00011	11111	100-
$A: = A + M$	00011	00010	100-
Set $Q[0]$ to 1			
(since $A[4] = 0$)	00011	00010	1001
Iteration 4			
(Note that $A[4] = 0$)			
Shift left (AQ)	00011	00101	001-
$A: = A - M$	00011	00010	001-
Set $Q[0]$ to 1			
(since $A[4] = 0$)	00011	00010	0011

Quotient = 3

Remainder = 2

In this example, there is no need to restore the remainder, since the A register already holds a positive number at the end of the fourth iteration. These algorithms may be extended to handle signed integers if both the dividend and the divisor are mapped to positive values. The sign of the result must be figured later, as in the case of multiplication. A discussion of this procedure may be found in the work of S. Waser and M. J. Flynn [19].

Another approach to carry-out division is through multiplication. Although this method appears unorthodox, it is the method employed by such mainframe computer

manufacturers as Honeywell and Amdahl in their high-speed models. The key to this concept is that D/V is equivalent to D times $1/V$. Therefore, by finding the reciprocal of the divisor V , the quotient can be computed by performing multiplication.

To compute the reciprocal of a number V , use an iterative algorithm, such as Newton-Raphson.

The Newton-Raphson algorithm is normally used to find the root of a equation of the form $f(x) = 0$. Here, the trial root x_i is found, so a better estimate is obtained by using the following recursive formula:

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

The mechanics of this process may be graphically illustrated, as shown in Figure 3.46.

Note that $f'(x_i)$ is the derivative of $f(x)$ when $x = x_i$.

Suppose that $f(x) = x - 1/V$. Then $f(x) = 0$ implies that $x = 1/V$. Therefore, the reciprocal-finding problem is nothing other than solving the equation $f(x) = 0$, where $x = 1/V$. In this situation, the following results are obtained:

$$f(x_i) = \frac{1}{x_i} - V$$

$$f'(x_i) = \frac{-1}{x_i^2}$$

Therefore,

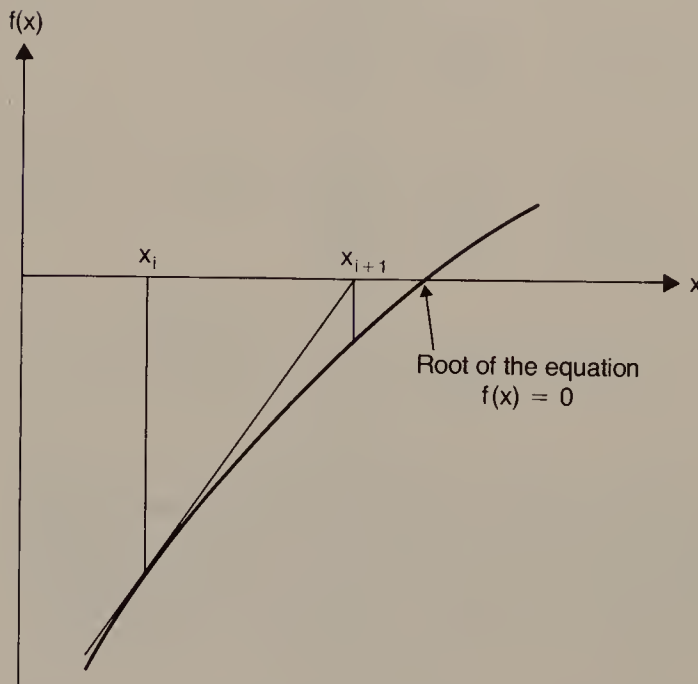


Figure 3.46 Solving the Equation $f(x) = 0$ Using the Newton-Raphson Algorithm

$$\begin{aligned}
x_{i+1} &= x_i - \frac{f(x_i)}{f'(x_i)} \\
&= x_i + \frac{1/x_i - V}{1/x_i^2} \\
&= x_i + (1 - Vx_i)x_i \\
&= 2x_i - Vx_i^2 \\
&= x_i(2 - Vx_i)
\end{aligned}$$

Thus, the general Newton-Raphson recursion formula becomes:

$$x_{i+1} = x_i (2 - Vx_i)$$

The following example illustrates the effectiveness of this approach.



EXAMPLE 3.2

Using Newton-Raphson, find $1/V$ when $V = 0.42$.

Solution

Assume that $x_0 = 1$, which provides the following results:

ITERATION NUMBER	NEW ESTIMATE	ERROR
1	$x_1 = X_0 (2 - VX_1) = 1.58000$	0.80095
2	$x_2 = X_1 (2 - VX_1) = 2.11151$	0.26944
3	$x_3 = X_2 (2 - VX_2) = 2.35046$	0.03049
4	$x_4 = X_3 (2 - VX_3) = 2.38056$	0.00039

Therefore, the approximate answer is 2.38056.

Example 3.2 shows that the error decreases quadratically. In fact, it can be proven that $E_{i+1} \leq E_i^2$, where E_i is the error in the i th iteration. The result shows that the Newton-Raphson approach is a quadratic convergence algorithm. Such a proof may be found in the work of S. Waser and M. J. Flynn [19].

In a practical implementation, a ROM is used to find the first estimate with a significance of 8 bits. The second iteration produces a 16-bit significant estimate, and the third iteration produces a 32-bit significant estimate.

In the IBM 360/91 CPU, the division is performed through iterative multiplications. This procedure is based on the binomial expansion of $1/V$. Readers may consult the original paper by M. J. Flynn [5] to find more information. In his research paper, Flynn illustrates the equivalence between binomial expansion and the Newton-Raphson techniques.

3.8 COPROCESSORS*

In typical 8-bit microprocessors such as the Intel 8085 and Z=80, technology places a limit on the chip area. In consequence, these CPUs include no hardware or firmware for performing scientific computations such as floating-point arithmetic, matrix manipulation, and graphic-data processing. Therefore, users of these systems must write these operation programs. Unfortunately, this approach is unacceptable in high-speed applications, since program execution takes a significant amount of time. To eliminate this problem, coprocessors are used.

In this approach, a single chip is built for performing scientific computations at high speed. However, the chip is regarded as a companion to the original or host CPU. Typically, each special operation is encoded as an instruction that can be interpreted only by the companion CPU. When the companion CPU encounters one of these special instructions, it assumes the processing functions independent of the host CPU. The companion CPU that operates in this manner is called the coprocessor. Therefore, this concept not only extends the capabilities of the host CPU but also increases the processing rate of the system. The coprocessor concept is widely used with typical 32-bit microprocessors such as the Motorola 68020 and Intel 80386.

Functionally, the coprocessor provides a logical extension of the programmer's model in the way of instructions, registers, and operand types. This extension is transparent to the programmer.

It is important to make the distinction between standard peripheral hardware and a coprocessor. A coprocessor is a device that has the capability of communicating with the main processor through the protocol defined as the coprocessor interface. As mentioned before, the coprocessor also adds additional instructions, registers, and data types that are not directly supported by the main processor. The coprocessor provides capabilities to the user without appearing to be hardware external to the main processor.

Standard peripheral hardware, on the other hand, is generally accessed through the use of interface registers mapped into the memory space of the main processor. The programmer uses standard processor instructions to access the peripheral interface registers and thus utilize the services provided by the peripheral. It should be pointed out that even though a peripheral can provide capabilities equivalent to a coprocessor for many applications, the programmer must implement the communication protocol between the main processor and the peripheral necessary to use the peripheral hardware. There are three general techniques that may be used to pass commands to a coprocessor. These are: intelligent monitor interface, coprocessors with special signals and instructions, and coprocessors using special signals.

3.8.1 Intelligent Monitor Interface

Figure 3.47 shows a block diagram of the *intelligent monitor interface*. The coprocessor must monitor the instruction stream by obtaining commands directly from the bus at the same time as the main processor. The Intel 8087 floating-point coprocessor is of this type, as it monitors the instruction stream simultaneously with a main processor such as

*Modified from ELECTRONICS, November 17, 1983 issue. Copyright © 1983, McGraw-Hill, Inc. All rights reserved. Reprinted with permission.

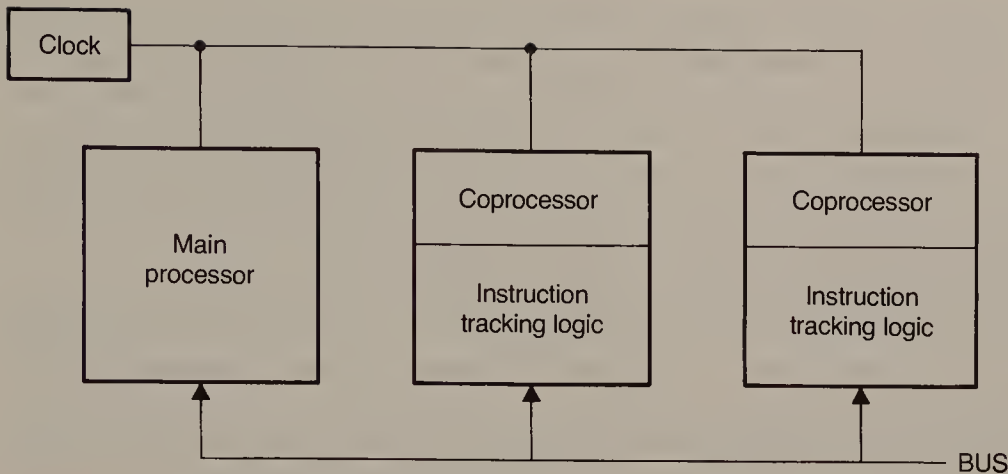


Figure 3.47 Intelligent Monitor Interface

the Intel 8086. Since the 8086 uses an instruction queue, the coprocessor must maintain an identical queue. This has the obvious advantage of requiring no additional bus cycles to pass the content of the instruction word to the coprocessor. There are several disadvantages, however:

- Each coprocessor in the system must duplicate the bus monitoring circuitry and instruction queue, tracking all branches, wait states, operand fetches, and instruction fetches.
- Simultaneous bus-monitoring techniques require that the processor/coprocessor pair operate no faster than the capabilities of the slowest member of the pair, even when no coprocessor instructions are executed.

3.8.2 Interface Using Special Bus Signals and Instructions

The coprocessor may be implicitly addressed by certain instructions that invoke a special protocol; a special bus line tells the coprocessor when to listen to the instruction stream. Figure 3.48 shows a block diagram of this type of coprocessor interface.

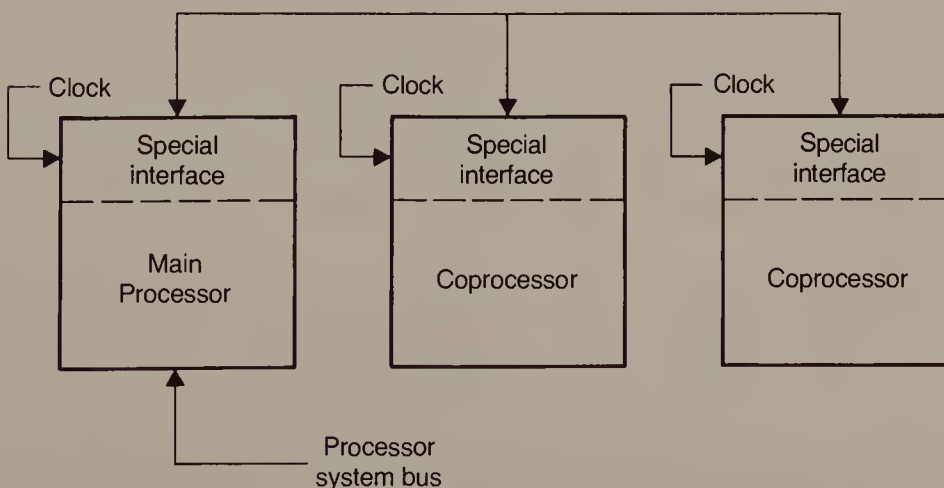


Figure 3.48 Coprocessor Using Special Signals and Instructions

An important attribute of this technique is that the main processor must have a prior knowledge of the coprocessor if the two are to work together effectively. For example, the main processor must know all data sizes of all coprocessors that are to be attached. An advantage of this method is that the coprocessor may run at different speeds since data transfer occurs asynchronously via handshaking. One of the main disadvantages is the extra CPU overhead. That is, the main processor has to recognize a coprocessor instruction and direct it to the appropriate coprocessor.

The National NS16081 is an example of this type. A special signal, an address translation/slave processor control, is used as a bidirectional handshake line between the main processor and coprocessor. In the National implementation, the coprocessor does not monitor the original instruction fetch by the processor, and the circuitry required to monitor and store the instruction queue is not required. However, the coprocessor identity is specific to a coprocessor type such that only one coprocessor of a given type is permitted in a system. As implemented by National, only sequential (nonconcurrent) operations are permitted. The main processor must always read status at the end of each command.

3.8.3 Coprocessor Interface Using Special Instructions

The coprocessor may be explicitly addressed by certain instructions, which initiate a special sequence of microinstructions in the main processor to effect command and operand transfer. Figure 3.49 is a block diagram of a system configured using this technique.

In this approach, when the main processor executes a coprocessor instruction, it decodes the instruction and writes a command in the command register (one of the interface registers) specifying the operation required by the coprocessor. In response, the coprocessor writes data back in a register called the *response register* (one of the interface registers). The main processor can read this data, and it tells the main processor certain information such as whether additional information is required by the coprocessor to carry out the operation. If such data is required, the main processor provides

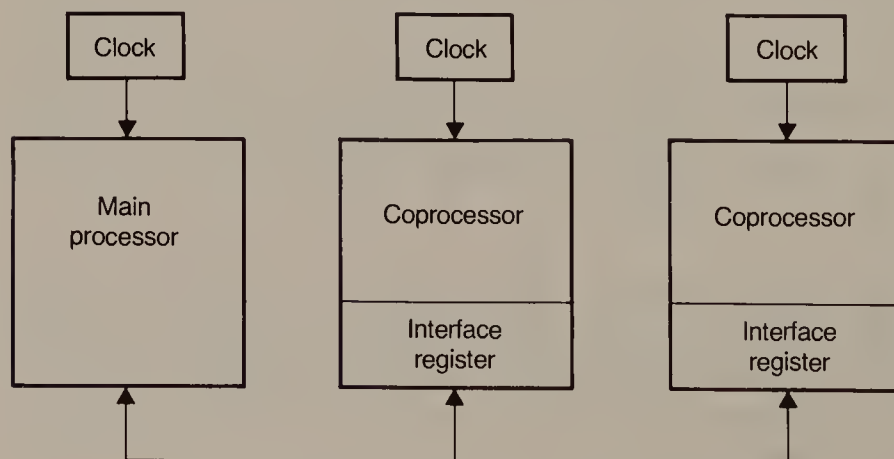


Figure 3.49 Coprocessor Interface Using Special Instructions

this; otherwise, the coprocessor carries out the operation concurrently with the main and provides the result.

Some of the advantages of this method are as follows:

- No special signals are required.
- No inherent limit to the number of coprocessors.

One of the main disadvantages of this method is that once the main processor detects a coprocessor instruction, the main has to use bus bandwidth and timing to transmit the command to the appropriate coprocessor. The Motorola 68881 (floating-point coprocessor) is of this type.

The MC68881 floating-point coprocessor is intended primarily for use as a coprocessor to the MC 68020 32-bit microprocessor for providing a logical extension to the 68020 arithmetic capabilities. The major features of the MC68881 are these:

- Eight general-purpose floating-point data registers, each supporting a full 80-bit extended precision real-data format (a 64-bit mantissa plus a sign bit and a 15-bit signed exponent).
- A 67-bit ALU to allow very fast calculations.
- A 67-bit barrel shifter for high-speed shifting operations.
- Forty-six instructions, including 35 arithmetic operations.
- Full set of trigonometric and logarithmic functions.
- Twenty-two constants available in the on-chip ROM including π , e , and powers of 10.

The MC68881 is internally divided into three processing elements: the bus interface unit (BIU), the execution control unit (ECU), and the microcode control unit (MCU). The BIU contains the coprocessor interface register and the 32-bit control, status, and instruction address registers. The eight 80-bit floating-point data registers are located in the ECU. In addition, the ECU contains the 67-bit ALU for both mantissa and exponent calculations, the 67-bit barrel shifter and the ROM constant. The MCU contains the clock generator, a two-level microcoded sequencer that controls the ECU, the microcode ROM, and self-test circuitry.

When the 68020 detects a typical 68881 instruction, the 68020 writes the instruction to the memory-mapped command register in the 68881 and then reads the response register in the 68881. In this response, the BIU encodes requests for any additional action required of the 68020 on behalf of the 68881. Once the 68020 fulfills the 68881 request, the 68020 is free to fetch and execute subsequent instructions.

QUESTIONS AND PROBLEMS

- 3.1** Explain the significance of CPU registers.
- 3.2** Consider the following statement: The number of CPU registers provides some measure of the power of a processor. Comment on the validity of this statement.

3.3 Design a 4-bit general purpose register as follows:

s_1s_0 FUNCTION

00 Load external data

01 Rotate left ($A_3 \leftarrow A_0$; $A_i \leftarrow A_{i-1}$ for $i = 1, 2, 3$)

10 Rotate right ($A_0 \leftarrow A_3$; $A_i \leftarrow A_{i+1}$ for $i = 0, 1, 2$)

11 Increment

3.4 Design a full 18×16 barrel shifter.

3.5 Explain the importance of the flag register in a computer.

3.6 Prove the validity of the results shown in Figure 3.10.

3.7 Show that the carry output of a full adder can be expressed as $c_{i+1} = y_i y_i + (x_i \oplus y_i) c_i$. What is the advantage of this representation?

3.8 Using a 4-bit CLA as a building block, design a 16-bit adder whose worst-case add-time is 10Δ .

3.9 Using a 4-bit CLA as the building block, design the fastest 64-bit adder. Estimate the worst-case add-time of your design.

3.10 Design a combinational circuit to compute the function $f(x) = (3/8)*x$, where x is a 4-bit 2's complement number.

3.11 Suppose there are n operands where $n = 2^k$ for some $k > 1$. Show that the series-parallel multioperand summation approach requires $n - 1$ adders. Also, show that when there are m -bits per operand, one add-time T will be

$$T = (m * \text{clock period}) + (\log_2 k) * (\text{full adder time})$$

3.12 Design a combinational circuit to compute the absolute value of the 4-bit, 2's complement number x . Use 4-bit adders and exclusive-Or gates.

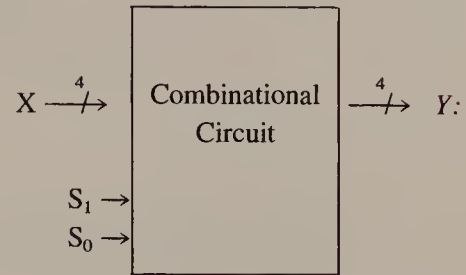
3.13 Design and implement a combinational circuit that will work as follows:

S_1	S_0	F
0	0	A plus B
0	1	Shift left (A)
1	0	A plus B plus 1
1	1	Shift left (A) + 1

Note that A and B are 4-bit operands

3.14 a. Design a combinational circuit that will satisfy the following specification.

S_1	S_0	Y
0	0	0
0	1	0
1	0	X
1	1	X



Using the results of part (a), design a 4-bit, 8-function arithmetic unit that
b. will function as described next:

S_2	S_1	S_0	F
0	0	0	A
0	0	1	A plus B
0	1	0	A plus B'
0	1	1	A minus 1
1	0	0	A plus 1
1	0	1	A plus B plus 1
1	1	0	A plus B' plus 1
1	1	1	A

3.15 Design a 4-bit, 8-function arithmetic unit that will meet the following specifications:

S_2	S_1	S_0	F
0	0	0	2A
0	0	1	A plus B'
0	1	0	A plus B
0	1	1	A minus 1
1	0	0	2A plus 1
1	0	1	A plus B' plus 1
1	1	0	A plus B plus 1
1	1	1	A

- 3.16 a.** Using a 4-bit parallel adder with inputs A, B, and C_{in} outputs F and C_{out} , and one selection bit S_0 , design an arithmetic circuit as follows:

S_0 FUNCTION TO BE PERFORMED

0 A plus B

1 B plus 1

- b.** Using another selection bit S_1 , modify the circuit of (a) to include the arithmetic and logic functions as follows:

S_1 S_0 FUNCTION TO BE PERFORMED

0 0 F = A plus B

0 1 F = B

1 0 F = shift left (logical) A

1 1 F = A'

- 3.17** Design a 4-bit logic unit that will function as follows:

S_1	S_0	F
0	0	$A \vee B$
0	1	$A \wedge B$
1	0	A'
1	1	$A \oplus B$

- 3.18** Point out four important features of the Am 2901 bit-slice processor.

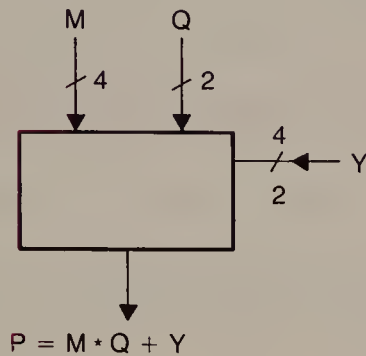
- 3.19** In the AM 2901, why are the following features provided?

- a.** Zero as an ALU input
- b.** Two-port RAM
- c.** Q and RAM shifters
- d.** $F = 0$ output
- e.** Q register

- 3.20** Write a suitable AM 2901 code sequence to perform each one of the following operations:

- a.** $M[0] \leftarrow (M[0] \text{ plus } M[1])/2$
- b.** $M[5] \leftarrow 0$
- c.** $M[2] \leftarrow -M[3]$
- d.** $M[1] \leftarrow \text{average of } M[1] \text{ through } M[4]$

- 3.21** Design and implement a 6×6 Braun's multiplier.
- 3.22** Design an unsigned 8×4 non-additive multiplier using additive-multiplier-module whose block diagrams representation is as follows:



Assume that M , Q , and Y are unsigned integers.

- 3.23** Using four 256×8 ROMs and 4-bit parallel adders, design a 8×8 unsigned, nonadditive multiplier. Draw a logic diagram of your implementation.
- 3.24** Comment on the hardware requirements for a fully combinational 32×32 Wallace tree multiplier.
- 3.25** Using add-shift method, perform the following 5×5 multiplication $10101 * 11001$ ($21 * 25$)
- 3.26** Perform the following two's complement multiplication $110011 * 110110$ ($-13 * -10$) using a) Booth's Algorithm and b) Modified Booth's Algorithm.
- 3.27** Perform the division $111 \overline{)101011}$ using each algorithm.
- a. Restoring division algorithm
 - b. Nonrestoring division algorithm
- 3.28** Explain how to extend the division procedures described in this chapter to handle signed numbers.
- 3.29** Using the Newton-Raphson method, find $1/V$, where $V = 0.60$.
- 3.30** How is the Newton-Raphson iteration method extended to find $\sqrt{N}$? Formulate $f(x)$ and derive the necessary recursive relation. Using this relation, find $\sqrt{5}$. Assume that $X_0 = 1$.
- 3.31** Explain the differences between a coprocessor and a peripheral device.
- 3.32** What are the various types of coprocessor interfaces? Discuss them briefly.

REFERENCES

1. Advanced Micro Devices, *Bipolar Microprocessor Logic and Interface Databook*. 1982.
2. Advanced Micro Devices. *AM 9500 Peripheral Products Interface Guide*. 1980.
3. Anderson, S. F., J. G. Earle, R. E. Goldschmit, and D. M. Powers. "The IBM System/360 Model 91: floating-point execution unit." *IBM Journal of Research and Development* 11, no. 1 (January 1967) 34–53.
4. Dadda, L. "Some Schemes for Parallel Multipliers." *Alta Frequenza* 34, no. 5 (May 1965) 349–356.
5. Flynn, M. J. "On Division by Functional Iteration." *IEEE Transactions on Computers* C-19, no. 8 (August 1970) 702–706.
6. Groves, Stan. "The Making of a Coprocessor." *Electronics* (April 29, 1983).
7. Hallin, T. G., and M. J. Flynn. "Pipelining of Arithmetic Functions." *IEEE Transactions on Computers* C-21, no. 8 (August 1972) 880–886.
8. Hamacher, V. C., Z. G. Vranesic and S. G. Zaky. *Computer Organization*. New York: McGraw-Hill, 1984.
9. Hayes, J. P., *Computer Architecture and Organization*. New York: McGraw-Hill, 1978.
10. Hayes, J. P., *Digital System Design and Microprocessors*. New York: McGraw-Hill, 1984.
11. Hwang, K. *Computer Arithmetic: Principles and Practice*. New York: John Wiley, 1979.
12. Mano, M. M., *Computer System Architecture*. Englewood Cliffs, N.J.: Prentice-Hall, 1982.
13. Palmer, J., R. Nave, C. Wymore, R. Koehler, and C. McMinn. "Making Mainframe Mathematics Accessible to Microprocessors." *Electronics* (May 1980).
14. Roth, C. *Fundamentals of Logic Design*. St. Paul: West Publishing, 1981.
15. Scott, N. *Computer Number Systems and Arithmetic*. Englewood Cliffs, N.J.: Prentice-Hall, 1985.
16. Singh, S., and R. Waxman. "Multioperand Addition and Multiplication." *IEEE Transactions on Computers* C-22, no. 2 (February 1973) 113–120.
17. Texas Instruments, "TTL Data Book for Design Engineers", 1972.
18. Wallace, C. S., "A Suggestion for A Fast Multiplier." *IEEE Transactions on Electronic Computers* EC-13, no. 1 (February 1964) 14–17.
19. Waser, S., and M. J. Flynn. *Introduction to Arithmetic for Digital Systems Designers*. New York: Holt, Reinhart & Winston, 1982.

CHAPTER 4

Control Unit

In this chapter, the process of designing a control unit is discussed. Topics covered include: hardwired control, microprogrammed control, and control store–minimization methods.

4.1 INTRODUCTION

A CPU can be viewed as a collection of two major components:

- Processing section
- Control unit

The processing section typically includes the hardware elements (ALU, shift registers, comparators, and multipliers) to operate on data such as integer or real numbers, character codes, operation codes, and offset values.

The purpose of the control unit is to control system operations by routing the selected data items to the selected processing hardware at the right time. A control unit's responsibility is to drive the associated processing hardware by generating a set of signals that are synchronized with a master clock. This synchronization establishes a time-reference for system analysis and design.

The signals generated by a control unit actually initiate an operation within the system. In order to carry out a task such as ADD, the control unit must generate a set of control signals in a predefined sequence governed by the hardware structure of the processing section. Sometimes, the control unit decides the operation sequence based on the status information provided by the status register (for example, Z and N condition codes) or the command signals generated by an external agent (such as RESET and ABORT signals issued from the operator's console). The inputs to the control unit are the master clock, status information from the processing section, and command signals from the external agent. The outputs produced by the typical control unit are the signals that drive the processing section and responses to an external environment (operation complete and operation aborted) due to exceptions (integer overflow or underflow).

A control unit undertakes the following responsibilities:

- Instruction interpretation
- Instruction sequencing

In the interpretation phase, the control unit reads instructions from the memory (using the PC as a pointer). It then recognizes the instruction type, gets the necessary operands, and routes them to the appropriate functional units of the execution unit. Necessary signals are then issued to the execution unit to perform the desired operation, and the results are routed to the specified destination.

In the sequencing phase, the control unit determines the address of the next instruction to be executed and loads it into the PC. To design a control unit, one must know the basic concepts addressed in the next section.

4.2 BASIC CONCEPTS

The fundamental concepts forming the basis for control unit design are the register transfer operations and their analytical descriptions. A digital processing system is a collection of registers where a processing activity is performed by a sequence of data-transfer operations among the registers either directly or with processing hardware (ALU). Consider the simple transfer shown in Figure 4.1

Here, 8 bits of information are to be moved from register A to register B. This means transferring a copy of A to B.

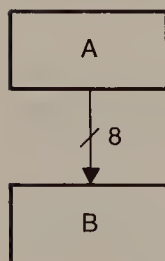


Figure 4.1 A Simple Register Transfer from A to B

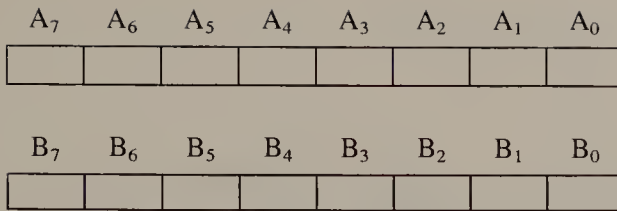


Figure 4.2 Convention Used to Mark Individual Bits of a Register

Such an operation can be described by the following notation:

$$B \leftarrow A$$

The symbol $\leftarrow$ is called the transfer operator. The statement $B \leftarrow A$ does not indicate how many bits are transferred. To indicate this, a declaration statement is used, which specifies the size of each register. For example, the registers shown in Figure 4.2 can be declared as

Declare registers $A [8], B [8]$

In this declaration A and B are 8-bit registers.

Similarly, a register can be defined as a portion of some other register. For example, if the high-order byte of a 16-bit program counter is considered as one 8-bit register, the following declarations are made:

Declare register $PC [16]$

Declare subregisters $PCHI [8] = PC [15-8]$

The convention used to mark individual bits of a register is shown in Figure 4.2.

In this figure, the bit positions are numbered as bits 0, 1, 2, . . . , 7. The binary weighting for bit 0 is 2^0 , bit 1 is 2^1 , and so on.

Consider the following statement:

$$B [0] \leftarrow A [7]$$

This means that the most-significant bit of the A register is transferred to the least-significant bit of the B register.

Normally, the information transfer between two registers is controlled by an enable signal E, as shown in Figure 4.3.

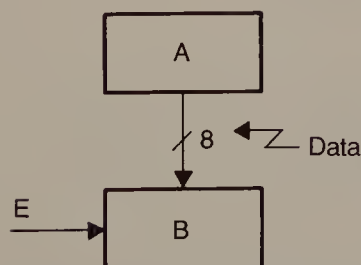


Figure 4.3 A Register Transfer Controlled by an Enable Input

Generally two inputs are associated with each register:

- Enable input or control input
- Data input

The enable input controls the data flow from register A to B. The B register of Figure 4.3 is loaded with the contents of the A register only when the enable input E is held high; otherwise the contents of the register remain the same. Such a conditional transfer can be expressed as

$$E: B \leftarrow A$$

A possible hardware implementation of a register with an enable input is shown in Figure 4.4.

The bit B_i in this figure is only loaded with bit A_i when the enable input E is high. The purpose of the enable input is to make sure that data transfer between the A and B registers takes place only under a predetermined condition and not after every clock pulse. For this reason, this input is called the *control input* and is driven by the control unit.

Sometimes, the control input may be a function of more than one variable. Consider the following register transfer involving three 8-bit registers A, B, and D:

$$\text{IF } A > B \text{ and } D[0] = 0 \text{ then } A \leftarrow B$$

The condition $A > B$ can be determined by using an 8-bit comparator. If we assume that the comparator output G goes high when $A > B$, then this conditional transfer can be described as follows:

$$C_0: A \leftarrow B \quad \text{where} \quad C_0 = G \wedge D[0]'$$

The hardware setup corresponding to this situation is shown in Figure 4.5.

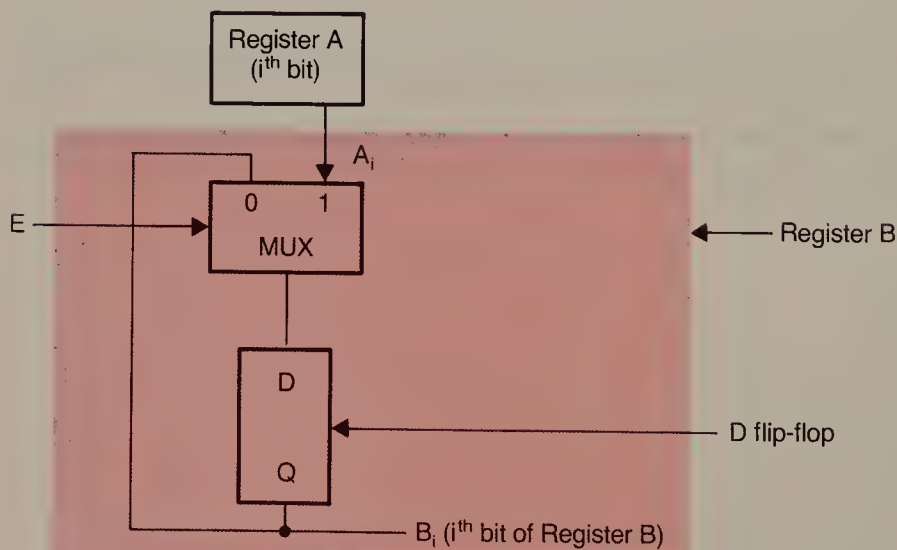


Figure 4.4 Hardware Implementation of a Register with Enable Input

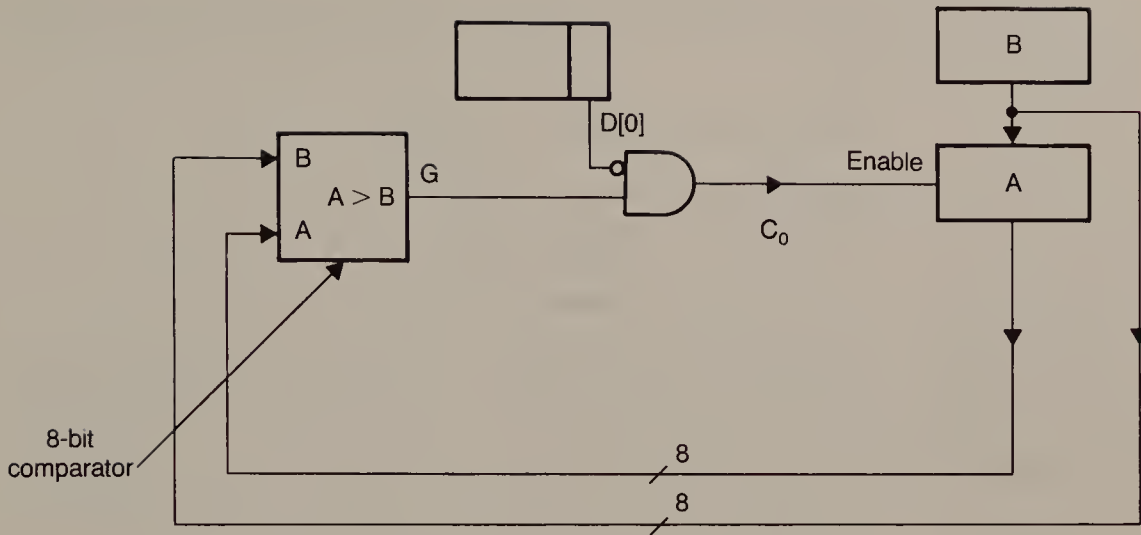


Figure 4.5 Hardware Implementation of $C_0: A \leftarrow B$ where $C_0 = G D [0]'$

It may often be necessary to perform some register transfer operation that involves selection. For example:

if $x = 0$ and $t = 1$ then $A \leftarrow B$
 else $A \leftarrow D$

Where A, B, and D are 8-bit registers, such a selective register transfer can be expressed as follows:

$C_0: A \leftarrow B$

$C_0': A \leftarrow D$

where $C_0 = x't$ and $C_0' = (x't)' = x + t'$.

A hardware implementation for the preceding situation is shown in Figure 4.6.

The MUX of Figure 4.6 selects register B if $C_0 = 1$; otherwise register D is selected.

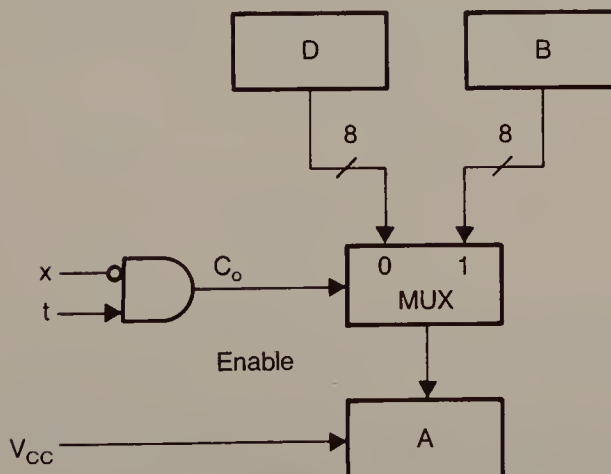


Figure 4.6 Hardware Implementation if $x = 0$ and $t = 1$, then $A \leftarrow B$ else $A \leftarrow D$

The register transfer description of some typical arithmetic and Boolean operations are summarized as follows:

$D \leftarrow A'$	Transfer the complement of A to D.
$A \leftarrow A + 1$	Increment the contents of A by 1.
$A \leftarrow A - 1$	Decrement the contents of A by 1.
$A' + 1$	Compute the 2's complement of A.
$D \leftarrow A \vee B$	Compute the logical OR of A and B and save the result in D.
$D \leftarrow A \wedge B$	Compute the logical AND of A and B and save the result in D.
LSR (A)	Logically shift the contents of the register A one position to the right.
ASR (A)	Arithmetically shift the contents of the register A one position to the right.

The mnemonics LSL, ASL, ROR, and ROL are used to indicate logical left shift, arithmetic left shift, rotate right and rotate left operations respectively.

For this notation the \$ symbol is used as the concatenation operator. For example, if A and Q are two 8-bit registers and ASR (A\$Q) is written, the contents of AQ are arithmetically shifted one position to the right. This operation is a 16-bit operation with high- and low-order bytes held in the A and Q registers, respectively.

Whenever a read/write memory (RWM) is a part of the processing section, the data transfers are described with respect to the RWM unit. As mentioned before, each word in the RWM is considered as an addressable register; thus, the whole memory unit can be viewed as a collection of such addressable registers.

Associated with each RWM unit are two registers:

- Memory address register (MAR)
- Memory buffer register (MBR)

The MAR is used as a pointer to a memory word. It holds the address of the desired memory word. The purpose of the MBR is to act as a buffer register in all data transfer operations. The general structure of a RWM is shown in Figure 4.7.

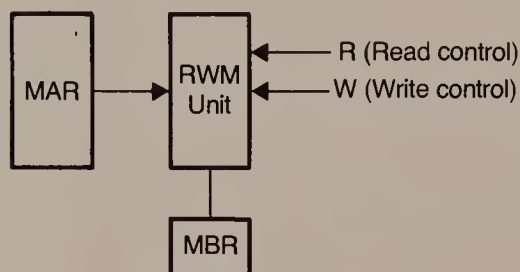


Figure 4.7 A Basic RWM Unit (M. Morris Mano, *Digital Logic and Computer Design*, © 1979, p. 314. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

The two basic operations performed by the RWM unit are read and write operations, expressed in register transfer notation as

R: $MBR \leftarrow M((MAR))$

W: $M((MAR)) \leftarrow MBR$

The letter M is used to indicate a memory. $M((MAR))$ indicates the memory word X addressed by the contents of the MAR.

In a typical read operation, the contents of the memory word, whose address is specified in MAR, are transferred into the MBR. Similarly, a write operation refers to the data transfer from MBR to the memory word whose address is specified in MAR. Since the data flows in and out of MBR, the set of data lines or the data bus between the RAM unit and MBR should be bidirectional. Such a bidirectional bus is easily implemented using tristate buffers as shown in Figure 4.8.

When $C = 1$ in Figure 4.8, data transfer takes place from X to Y, and vice versa when $C = 0$.

Buses are also used to route data in and out of a digital processing system. For example, in a typical digital system, there will be a pair of buses (“inbus” and “outbus”) to transfer data from the external environment into the processing section, and vice versa. As in the case of registers, the existence of such buses is shown in a register transfer description by using declaration statements. For example, declare Inbus [4] Outbus [4] indicates that the system under description includes two 4-bit-wide data buses (inbus and outbus). $A = \text{inbus}$ means the data on the inbus is transferred into the A register when the next clock arrives. The equate symbol specifies a transfer from a register into the bus. For example, $\text{outbus} = Q[7:4]$ means the high-order 4 bits of an 8-bit register Q are made available on the outbus for one clock period.

By sequencing a set of register transfer operations and including typical control structures such as if-then and go-to, an algorithm implemented by a digital system can be described. Consider the following description:

Declare registers A [8], M [8], Q [8]:

Declare buses inbus [8], outbus [8]:

Start: $A \leftarrow 0, M \leftarrow \text{inbus}$; Transfer the multiplicand and clear accumulator

$Q \leftarrow \text{inbus}$; Transfer the multiplier

Loop: $A \leftarrow A + M, Q \leftarrow Q - 1$; Add multiplicand
If $Q < > 0$ then go to Loop; repeat if $Q \neq 0$
 $\text{Outbus} = A$;

Halt: Go to Halt

The hardware for the preceding description includes an 8-bit inbus, an 8-bit outbus, an 8-bit parallel adder, and three 8-bit registers—A, M, and Q. This hardware performs unsigned multiplication by repeated addition, much like a program written using a combination of high-level and assembly languages.

Therefore, a distinguishing feature of this description is the ability to describe concurrent operations. For example, the operations $A \leftarrow 0$, and $M \leftarrow \text{Inbus}$ can be

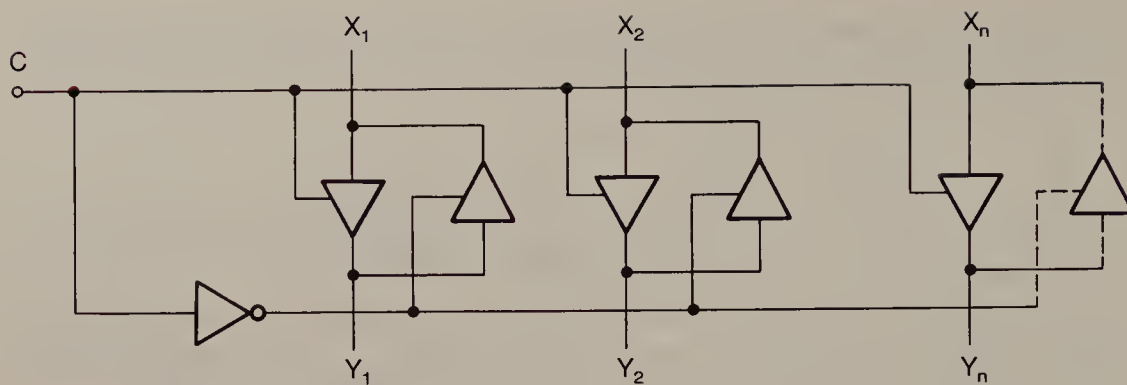


Figure 4.8 Bidirectional Data Bus

executed at the same time. As a general rule, a comma is inserted between operations that can be executed simultaneously.

Similarly, a semicolon between two register transfer operations dictates that they must be performed serially. This restriction is primarily due to the data paths provided in the hardware. For example, in the description just presented, there is only one input bus. The operations $M \leftarrow \text{inbus}$ and $Q \leftarrow \text{inbus}$ must be performed serially. However, either one may be overlapped with the operation $A \leftarrow 0$, since the operation does not use the input bus. The description also includes features such as labels and comments to increase the readability of the task description.

Operations such as $A \leftarrow 0$ and $A \leftarrow A + M$ are called *microoperations*, since they are completed in one clock cycle. In principle, any task can be expressed as a sequence of microoperations. This topic is addressed later in this chapter.

The rate which a computer processes operations such as $A \leftarrow A + B$, and $A \leftarrow A \wedge B$ is determined by its bus structure. Similarly, bus organization determines the cost of the system. Therefore, a less-complex bus organization is less expensive.

The simplest of all bus structures is the single-bus organization shown in Figure 4.9. All CPU registers in Figure 4.9 are connected to the same bus. At any given time, data may be transferred between any two CPU registers or between a CPU register and the ALU.

Whenever the ALU in Figure 4.9 requires the two operands (such as in the case of addition), the operands can be transferred only one at a time. Single-bus architecture should have the following features:

- The bus must be multiplexed among various operands.
- The ALU must have buffer registers to hold the transferred operand.

In Figure 4.9 an operation such as $R_2 \leftarrow R_0 + R_1$ is completed in three clock cycles because of the following:

- In the first clock cycle, the contents of register R_0 are transferred to buffer register A of the ALU.
- During the second clock cycle the contents of register R_1 are transferred to buffer register B of the ALU.

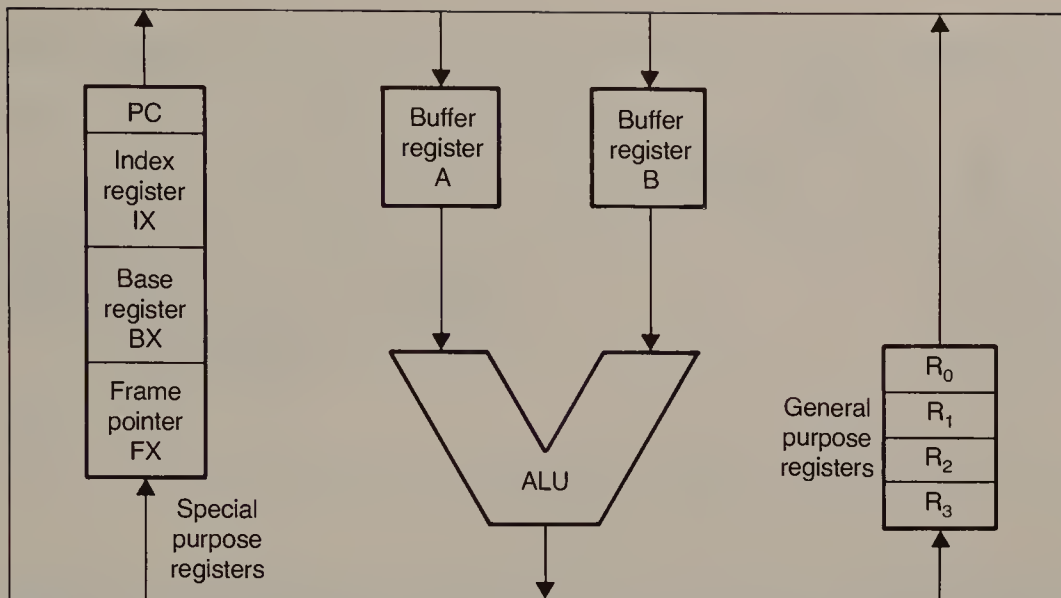


Figure 4.9 Organization of a Single Bus-oriented RALU

- The sum produced by the ALU is loaded into register R_2 when the third clock pulse arrives.

Single-bus organization reduces the speed of the addition operation when the operands are already in the CPU registers. If the operands are in memory, two clock cycles are required to retrieve the required operands. Therefore, this organization affects the speed of execution of a typical two-operand memory-reference instruction.

To carry out a binary operation (such as addition), the control logic must follow a three-step sequence. Each step represents a control state. Therefore, a single-bus architecture increases the number of states in the control logic. More hardware may be required to design the control unit. Since all data transfers take place through the same bus, one at a time, the design effort required to build the control logic is greatly reduced.

A single-bus organization's speed of operation is not as high as that of a two-bus organization. A typical two-bus scheme is shown in Figure 4.10.

All general-purpose registers are connected to both buses (bus A and bus B) to form a two-bus organization. The two operands required by the ALU are, therefore, routed in one clock cycle. Instruction execution is faster, since the ALU does not wait for the second operand, as in the case with a single-bus organization. The information flowing on a bus may be from a general-purpose register or a special-purpose register. In this arrangement, the special-purpose registers are often divided into two groups. Each group is connected to one of the buses. The data from two special-purpose registers of the same group cannot be transferred to the ALU at the same time.

In Figure 4.10, the contents of the PC are always transferred to the right input of the ALU, since it is connected to bus A. Similarly, the contents of the special-purpose register MBR are always transferred to the left input of the ALU, since it is connected to bus B. Whenever there is a need to process simultaneously the contents of two

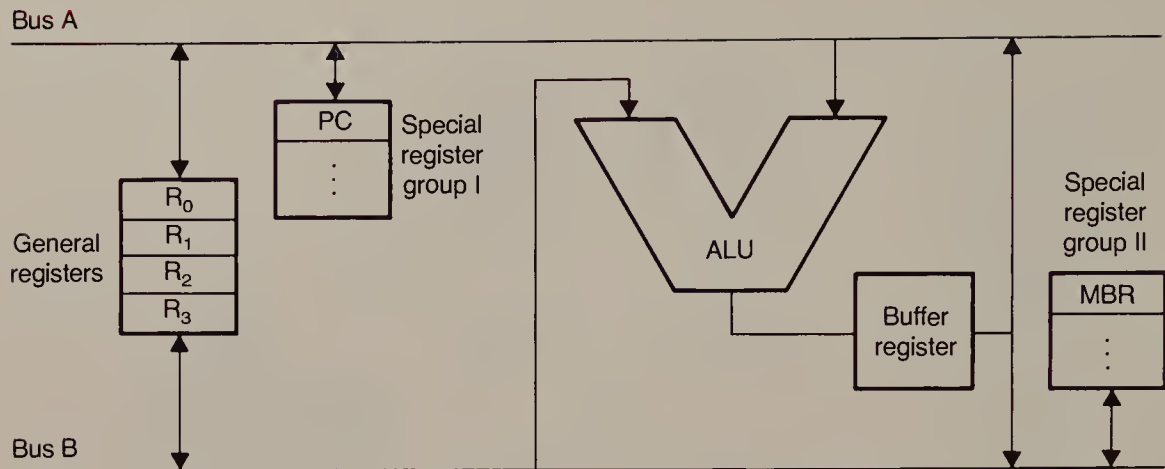


Figure 4.10 The Architecture of a Two-bus-oriented RALU

special-purpose registers of the same group, the contents of one of the registers must be transferred to one of the general-purpose registers prior to processing. The initialization step requires the execution of an additional instruction. The output of the ALU may be routed to either a special-purpose or a general-purpose register. Since the ALU does not have any input buffer registers, both buses carry operands and are busy during binary operations. Therefore, the output produced by the ALU will first be routed to the output register.

Transfer of the required operands and loading of the ALU output buffer register takes place in one cycle. The result held in the ALU output buffer register is then routed to the required destination in the second clock cycle. The contents of the ALU output buffer register can be gated to either bus A or bus B.

Besides the data paths shown in Figure 4.10, there may be additional dedicated data paths between special-purpose registers. For example, a dedicated data path is often provided between the MAR and PC because the register transfer operation

$$\text{MAR} \leftarrow \text{PC}$$

appears at the beginning of each instruction cycle.

The performance of a two-bus organization can be further improved by adding a third bus, bus C, at the output of the ALU. The resulting structure is known as a *three-bus organization* and is shown in Figure 4.11.

From this figure notice that a three-bus structure is reduced to the two-bus structure if bus C is replaced with the ALU output buffer register. The addition of bus C allows the system to perform all ALU operations, such as $R_2 \leftarrow R_0 + R_1$, in one cycle, since three separate data paths or buses are provided with the system. These data paths include routing for the two operands and the result. A finite delay will occur due to the transferring of operands to the ALU and the time taken by the ALU for producing the result. This delay must be taken care of when transferring the result to the third bus. The addition of the third bus will increase the system cost. The design of the control logic will be complicated because of the critical time delays involved.

Another important concept closely related to the control unit design is the generation of timing signals. One task of a control unit is properly to sequence a set of

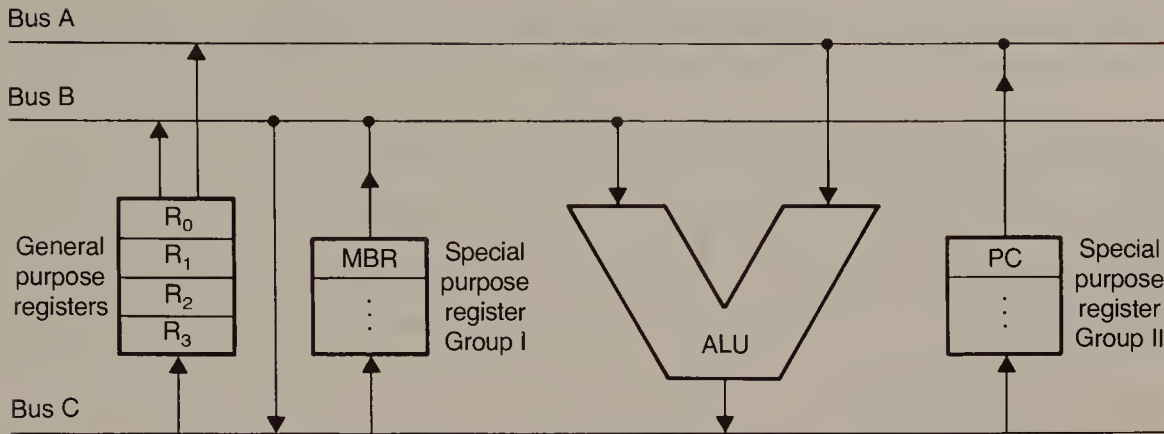


Figure 4.11 The Organization of a Three-bus-Oriented RALU

operations. Normally, a sequence of N consecutive operations will occur in response to N consecutive clock pulses. To carry out an operation, P_i at the i th clock pulse, a control unit must count the clock pulses and produce a timing signal T_i . T_i will assume a value of 1 during the duration of the i th clock pulse. For example, the input clock pulse and the four timing signals T_0 , T_1 , T_2 , and T_3 are shown in Figure 4.12.

In Figure 4.12, the timing signal T_i marks when the i th clock pulse has occurred and stays high until the next pulse. The frequency of the signal T_i is one-fourth the input clock pulse, since one clock period of T_i accommodates four clock periods of the input pulse.

An easy way to generate this type of timing signals is by designing a ring counter, as shown in Figure 4.13.

This system will sequence the following manner:

PRESENT STATE	NEXT STATE
A B C D	$A^+ B^+ C^+ D^+$
1 0 0 0	0 1 0 0
0 1 0 0	0 0 1 0
0 0 1 0	0 0 0 1
0 0 0 1	1 0 0 0

This circuit is also known as a *circular shift register*, since the least-significant bit shifted is not lost. The Boolean equations for each timing variable are derived by inspection as follows:

$$T_0 = A; \quad T_1 = B; \quad T_2 = C; \quad T_3 = D;$$

The chief advantages of this circuit are design simplicity and the ability to generate timing signals without a decoder. Nevertheless, N flip-flops are required to generate N timing signals. This approach is not economically feasible for large values of N .

To generate timing signals economically, a new approach is used. A modulo- 2^N counter is first designed using N flip-flops. The N outputs from this counter are then

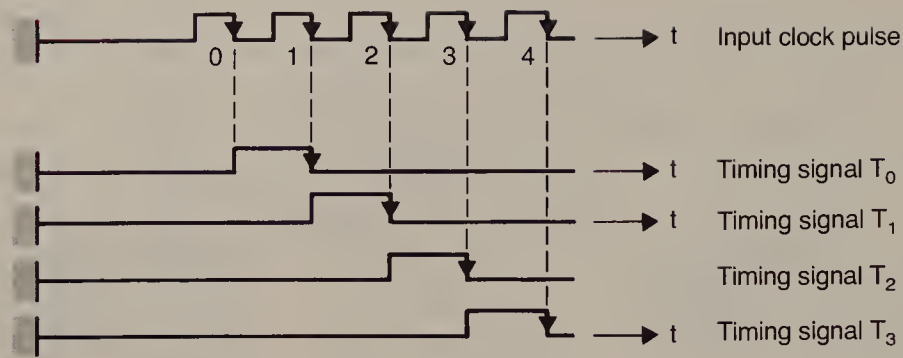


Figure 4.12 Timing Signals

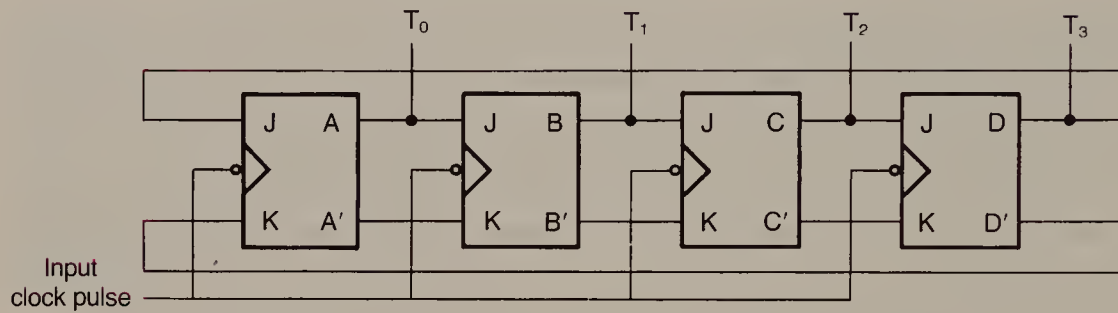


Figure 4.13 Ring Counter

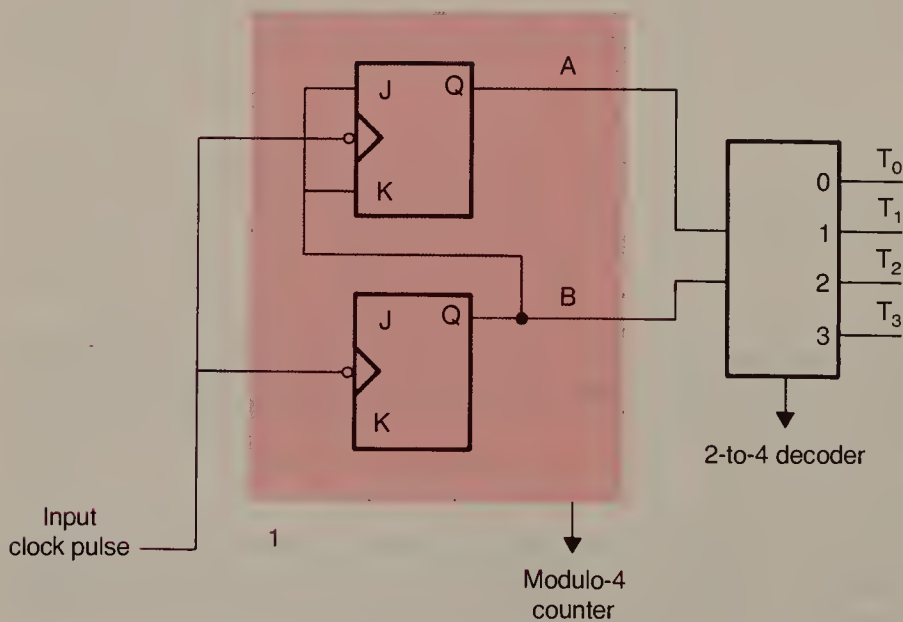


Figure 4.14 Modulo-4 Counter with a Decoder

given to a N -to- 2^N decoder as input to generate 2^N timing signals. The circuit depicted in Figure 4.14 shows how to generate four timing signals using a modulo-4 counter and a 2-to-4 decoder:

In the preceding circuit, the Boolean equation for each timing signal can be derived as:

$$T_0 = A'B'$$

$$T_1 = A'B$$

$$T_2 = AB'$$

$$T_3 = AB$$

These equations show that four 2-input AND gates are needed to derive the timing signals (assuming single-level decoding). The main advantage of this approach is that 2^n timing signals using only n flip-flops are generated. In this method, though, 2^n (n -input) AND gates are required to decode the n -bit output from the flip-flops into 2^n different timing signals. Yet the ring counter approach requires 2^n flip-flops to accomplish the same task.

4.3 DESIGN METHODS

Control units are designed in two different ways:

- Hardwired approach
- Microprogramming

The first approach exists because control logic is a clocked sequential circuit and, therefore, conventional sequential circuit design procedures can be applied to build a control unit. This technique earns the name *hardwired approach*, because the final circuit is obtained by physically connecting typical components such as gates and flip-flops. In the microprogrammed approach, all control functions that can be simultaneously activated are grouped to form control words stored in a separate memory called the control memory. The control words are fetched from the control memory, and the individual control fields are routed to various functional units to enable appropriate gates. When these gates are activated sequentially, the desired task is performed.

The design of a control unit with a control memory is more expensive than hardwired. This is typical when the control system to be designed does not require many microoperations or many control decisions. The inclusion of the control ROM carries overhead such as hardware production cost for masking the bits in the control ROM. A control memory may reduce the overall speed of the system, since the microinstruction retrieval process takes a significant amount of time.

The most important advantage of microprogramming is it provides a well-structured control organization. Control functions are systematically transformed into programming discipline, and a regular memory replaces most of the random combinational circuit elements.

During the design of a system, improvements are thought of and accidental deletions discovered. With microprogramming, many additions and changes are made by simply changing the microprogram in the control memory. A small change in the hardwired approach may lead to redesigning the entire system.

Although hardwired logic is an economical way of obtaining a simple control algorithm, the cost of the control logic increases with system complexity. In microprogrammed implementations, the cost of the simplest system is higher, though adding new features only requires additional control memory. Because of advances in IC technology, replacement of a random logic circuit with a ROM offers substantial hardware savings. CAD methods can be used to reduce the design cost of the system. Microprogramming also simplifies documentation and service training of the system, resulting in reduced total system cost.

Every computer system requires a package of diagnostic routines for checking, locating, and isolating hardware malfunctions. With microprogramming, these routines can be made available in the control memory. An EAROM (electrically alterable ROM) may load these microdiagnostic routines in small segments, with each checking a different portion of hardware.

By changing the control program, it is possible to interpret a new instruction set. The simulation of a processor's instruction set in another processor is called emulation. Emulation provides compatibility of instruction sets between smaller and larger machines of a series.

Implementation of commonly used routines, such as matrix inversion and table look-up procedures in microcode, can result in significant performance improvement. Savings in speed are achieved if a special microprogram is written to carry out repetitive system tasks such as program interpretation and job-queue manipulation (operating system routine).

In summary, microprogramming is accepted as a standard tool for designing the control unit of a computer. For example, processors such as the IBM 370, VAX-11, POP-11, MC68000, and Intel 8086 have a microprogrammed control unit. Nevertheless, Zilog's 16-bit microprocessor Z8000 still employs a hardwired control unit.

4.3.1 Hardwired Control Design

In this section, the hardwired approach to control logic design is discussed. The steps involved in this method are summarized as follows:

1. Define the task to be performed.
2. Propose a trial processing section.
3. Provide a register-transfer description of the algorithm based on the processing section outlined in the previous step.
4. Validate the algorithm by using trial data.
5. Describe the basic characteristics of the hardware elements to be used in the processing section.

6. Complete the design of the processing section by establishing necessary control points.
7. Propose a block diagram of the controller.
8. Specify the state diagram of the controller.
9. Specify the characteristics of the hardware elements to be used in the controller.
10. Complete the controller design and draw a logic diagram of the final circuit.

The following discussion explains this procedure by using an example. For the first step, the control task is described as:

Task definition: Design a Booth's multiplier to multiply two 4-bit 2's complement numbers.

In previous chapter, it was shown that Booth's procedure inspects a pair of multiplier bits and initiates one of the following actions:

MULTIPLIER BITS EXAMINED	ACTION
$q_j q_{j-1}$	(In position j)
0 0	None
0 1	Add M
1 0	Sub M
1 1	None

To implement Booth's procedure, the processing section (Step 2) shown in Figure 4.15 is proposed. This setup is the same as the one explained in the previous chapter.

As mentioned earlier, the 4-bit register M will hold the multiplicand. The Q register is 5 bits wide. Initially, the high-order 4-bits of this register will hold the 4-bit multiplier. The least-significant bit of this register is initialized with the fictitious zero discussed in the previous chapter. The 4-bit adder/subtractor is used to perform the operations $A + M$ or $A - M$. The result produced by this hardware is always routed to the 4-bit accumulator register A. In this implementation the multiplicand is not shifted to the left; rather the accumulated partial product is shifted right. When the computation terminates, the high- and low-order 4 bits of the final product are found in registers A and Q, respectively.

The L register is used to keep track of the iteration count. In this particular case, this register is initialized with 4_{10} (100_2) and is decremented by 1 at the completion of each iteration. Therefore, the algorithm terminates when 000 is in the L register. The 4-bit data buses, inbus and outbus, are used to transfer data into and out of the processing section, respectively.

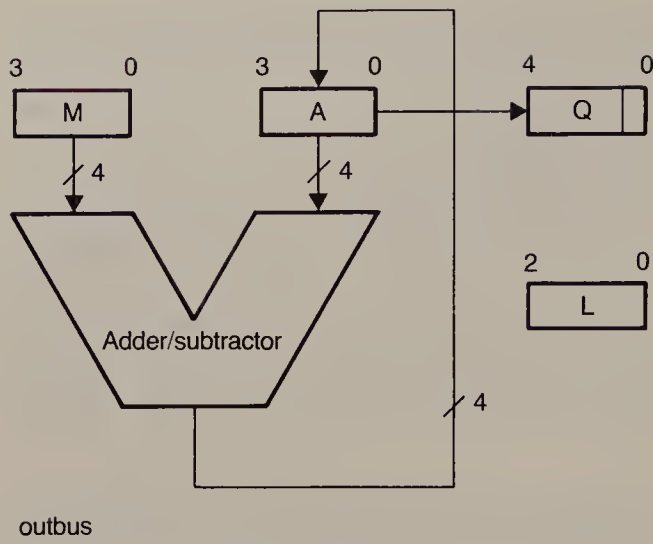


Figure 4.15 Organization of the Processing Section for Performing 4×4 Multiplication Using Booth's Algorithm

For Step 3, a register transfer description of Booth's multiplication algorithm is provided in Figure 4.16.

In Figure 4.16, $Q[1:0]$ is used to indicate the low-order 2 bits of the Q register ($Q[1]$ and $Q[0]$). Similarly, $Q[4:1]$ indicates the high-order 4 bits of the Q register. The last step, go to HALT, introduces an infinite loop after the computation is completed.

The correctness of this procedure is validated by a complete trace (Step 4) as shown next:

$$M = 1100_2 = -4_{10} \quad \text{and} \quad M = 0111_2 = 7_{10}$$

	MULTIPLIER BITS INSPECTED			
	M	A	Q	L
Initialization	1100	0000	011 ¹⁰	100
Iteration 1				
$A \leftarrow A - M$	1100	0100	01110	100
ASR (A\$Q), $L \leftarrow L - 1$	1100	0010	001 ¹¹	011
Iteration 2				
ASR (A\$Q), $L \leftarrow L - 1$	1100	0001	000 ¹¹	010
Iteration 3				
ASR (A\$Q), $L \leftarrow L - 1$	1100	0000	100 ⁰¹	001
Iteration 4				
$A \leftarrow A + M$	1100	1100	10001	001
ASR (A\$Q), $L \leftarrow L - 1$	1100	1110	01000	000
				→ Product = -28

```

Declare registers A [4], M [4], Q [5], L [3];
Declare buses    Inbus [4], Outbus [4];
Start:    A  $\leftarrow$  0, M  $\leftarrow$  Inbus, L  $\leftarrow$  4; clear A and transfer M

          Q [4:1]  $\leftarrow$  Inbus, Q [0]  $\leftarrow$  0; Transfer Q
Loop:    If Q [1:0] = 01 then go to Add;
          If Q [1:0] = 10 then go to Sub;
          Go to RShift;
Add:     A  $\leftarrow$  A + M;
          Go to RShift
Sub:     A  $\leftarrow$  A - M;
RShift:  ASR (AQ), L  $\leftarrow$  L - 1;
          If L < 0 then go to Loop
          Outbus = A;
          Outbus = Q [4:1];
Halt:    Go to Halt

```

Figure 4.16 Register Transfer Description of Booth's Multiplication Procedure

The processing section includes three main components:

- General-purpose storage registers
- 4-bit adder/subtractor
- Tristate buffers

The functional characteristics of these parts are summarized in Figure 4.17 (Step 5).

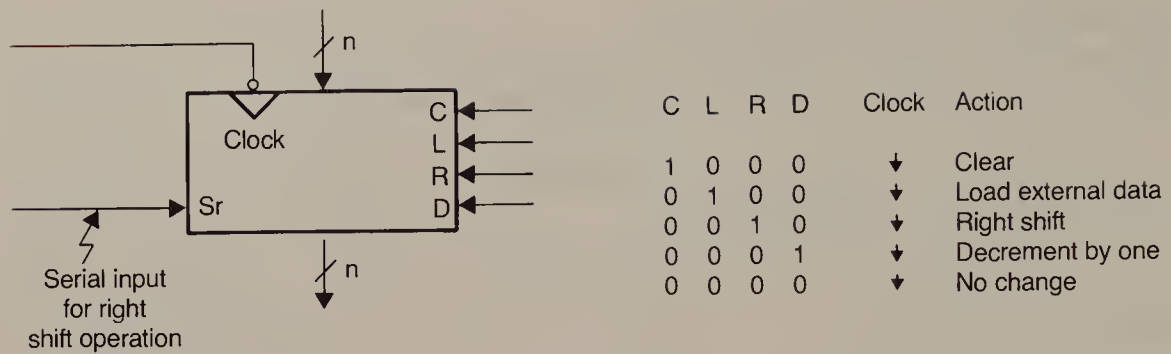
Notice that the general register is basically a trailing-edge-triggered device. Typically, four operations (clear, parallel load, right shift, and decrement) can be performed by introducing the proper values to control inputs C, L, R, and D. All these operations are synchronized with the trailing edge (high to low) of the clock pulse.

Such a general-purpose register can be built using standard MSI parts and gates. The 4-bit adder/subtractor is implemented by using a 4-bit parallel adder chip (7483) and four 2-input exclusive-OR gates. The tristate buffers are used to control the data transfer to the outbus.

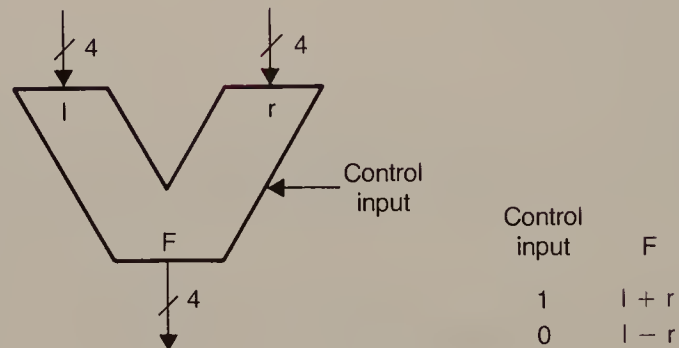
A detailed logic diagram of the processing section, along with interpretation of various control points, is shown in Figure 4.18 (Step 6).

From this figure, we observe that there are 10 control points, $C_0, C_1, C_2, \dots, C_9$.

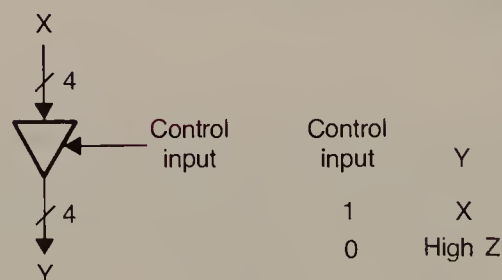
If C_0 is held high, the A register will be cleared with the trailing edge of the next clock. The other control points are interpreted similarly. One control point is introduced for each microoperation specified in the register-transfer description (see Figure 4.16). The processing section extends three outputs Q [1], Q [0], and Z; Z = 1 only when the contents of the L register become zero. These outputs are the status outputs and are used as inputs to the controller to allow the controller to decide the future course of action.



a. Storage Register



b. Adder/subtractor



c. Tri-state Buffer

Figure 4.17 Characteristics of the Component Parts Used in the Processing Section of the Booth's Multiplier

With this information a block-diagram representation for the controller can be formed, as shown in Figure 4.19 (Step 7).

From the block diagram, it can be seen that the controller has 5 inputs and 10 outputs. The RESET input is an asynchronous input used to reset the controller so a new computation can begin. The clock input is used to synchronize the controller's action. All activities are assumed to be synchronized with the trailing edge of the clock pulse.

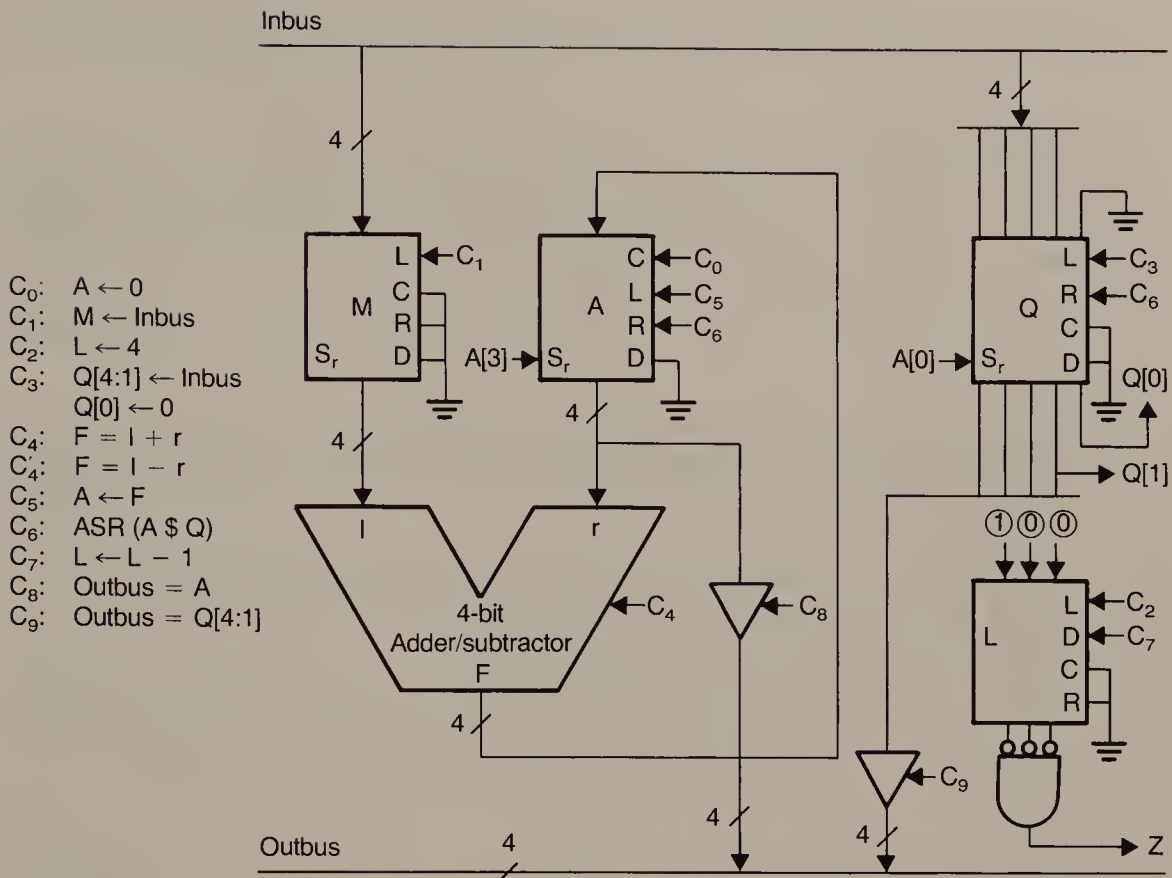


Figure 4.18 Processing Section of the Booth's Multiplier

We mentioned earlier a controller must initiate a set of operations in a specified sequence. Therefore, it is modeled as a sequential circuit. The state diagram for the Booth's Multiplier Controller is shown in Figure 4.20 (Step 8).

Initially, the controller is in the state T_0 . At this point the control signals C_0 , C_1 , and C_2 are high. Operations $A \leftarrow 0$, $L \leftarrow 4$, and $M \leftarrow \text{Inbus}$ are carried out with the

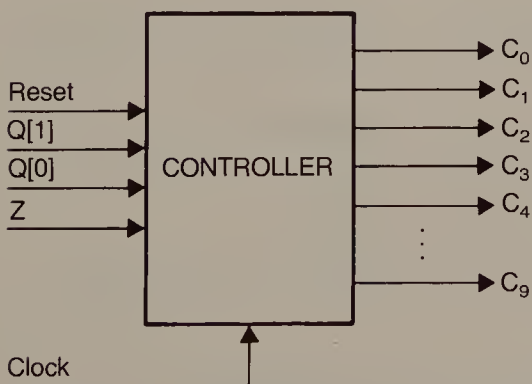


Figure 4.19 Block-diagram of the Booth's Multiplier Controller

trailing edge of the next clock pulse. The controller moves to the state T_1 with this clock. When the control is in the state T_3 , $A \leftarrow A + M$ is performed. If it is in T_4 , $A \leftarrow A - M$ is performed.

All these transfers take place with only the trailing edge of the next clock. If the controller is in the state T_3 or T_4 , it moves to the state T_5 with the next trailing edge of the clock. The controller moves to the state T_8 only when a computation is completed—so it stays in that state forever. A hardware RESET input causes the controller to switch to the state T_0 , and a new computation begins.

In this state diagram, selection of states are made according to the following guidelines:

- If the operations are independent of each other and can be completed within one clock cycle, they are grouped within one control state. For example, in Figure 4.19, operations $A \leftarrow 0$, $M \leftarrow \text{Inbus}$, and $L \leftarrow 4$ are independent of each other. With this hardware they are executed in one clock cycle. That is, they are microoperations. If these operations cannot be performed within the selected T_0 clock cycle, however, either clock duration is increased or the operations are divided into a sequence of microoperations.
- Conditional testing usually implies introduction of new states. For example, in Figure 4.20 the conditional testing of the bit pair $Q[1]Q[0]$ introduces the new state T_2 .

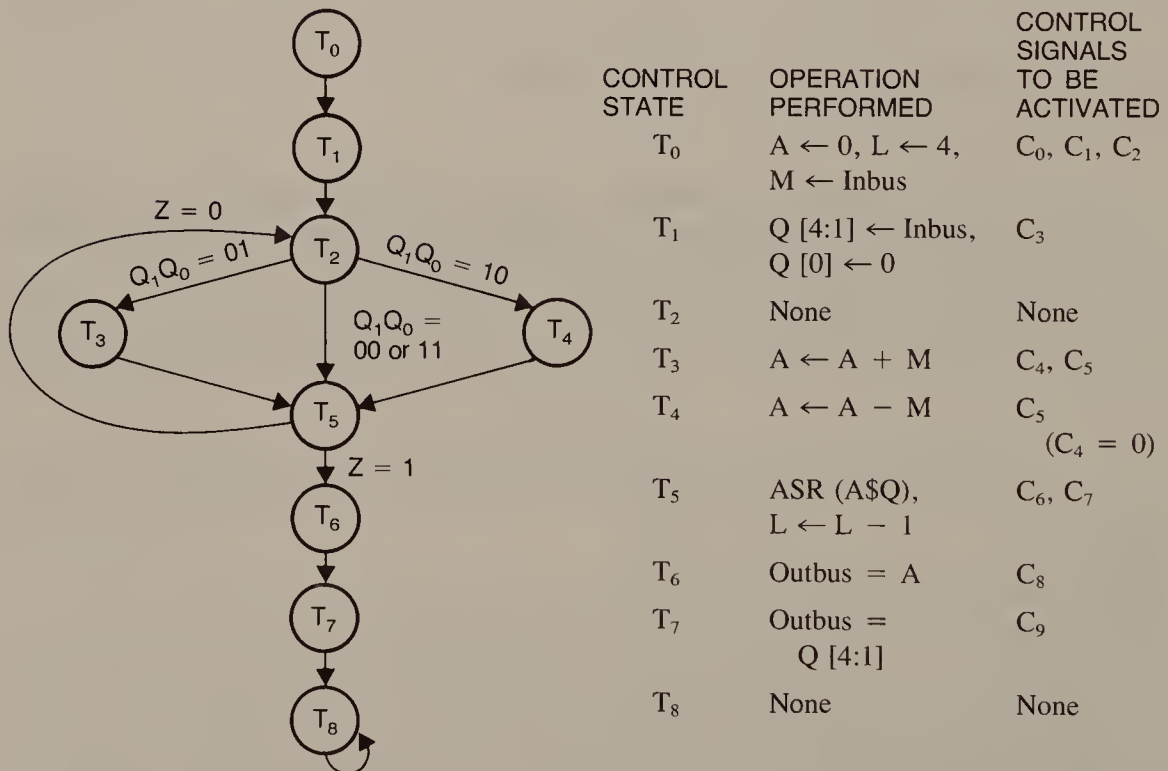


Figure 4.20 Controller Description

- One should not make an attempt to minimize the number of states. When in doubt, new states must be introduced. The correctness of the control logic is more important than the economics of the circuit.

There are 9 states in the controller state diagram. Nine nonoverlapping timing signals (T_0 through T_8) must be generated so only one will be high for a clock pulse. Figure 4.21 shows the first three timing signals T_0 , T_1 , and T_2 .

As mentioned earlier, a mod-16 counter and a 4-to-16 decoder can be used to accomplish this task. The characteristics of the mod-16 counter is discussed in Figure 4.22 (Step 9).

The controller and its logic diagram are shown in Figure 4.23 (Step 10). The key element of this implementation is the sequence controller (SC) hardware, which sequences the controller as indicated in the state diagram (see Figure 4.20). Therefore, the truth table for the SC must be derived from the controller's state diagram (Figure 4.24(a)).

For example, consider the logic involved in deriving the first entry of the SC truth table. Observe that the mod-16 counter is loaded (or initialized) with the specified external data if the counter control inputs C and L are 0 and 1, respectively. In this counter module, counter load control, input L, overrides the count enable control, input E. This situation is typical of 4-bit MSI counter chips, such as the 74161.

From the controller's state diagram, it can be seen that if the present control state is T_2 (counter output $0_30_20_10_0 = 0010$) and if the bit pair inspected is 00, (that is, $Q[1]Q[0] = 00$), then the next control state is T_5 . When these input conditions occur, the counter must be loaded with external value 0101 along with the trailing edge of the next clock pulse ($T_5 = 1$ only when $0_3, 0_2, 0_1, 0_0 = 0101$). Therefore, the SC generates $L = 1$ and $d_3, d_2, d_1, d_0 = 0101$.

Using the same reasoning, the fifth entry of the SC truth table is obtained as follows. From the controller's state diagram, it can be seen that if the present control state is T_5 and if $Z = 0$, the next control state is T_2 . The SC must generate the outputs $L = 1$ and $d_3d_2d_1d_0 = 0010$ to achieve the desired state sequence. Other entries of the SC truth table are derived in the same manner.

When the counter load control input $L = 0$, the counter will automatically count

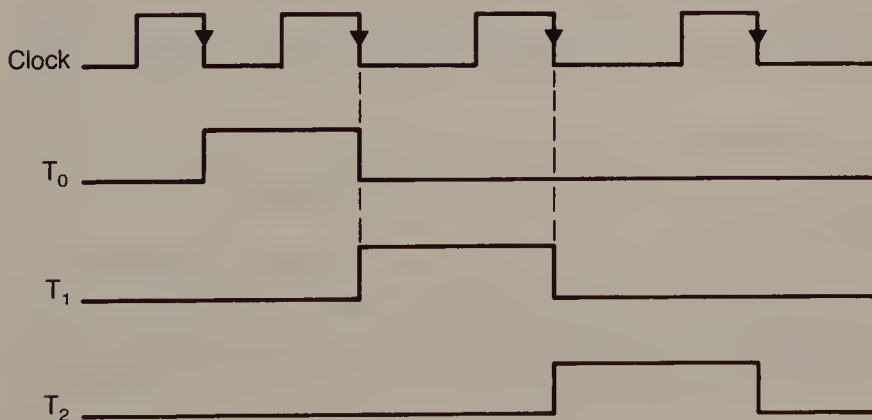
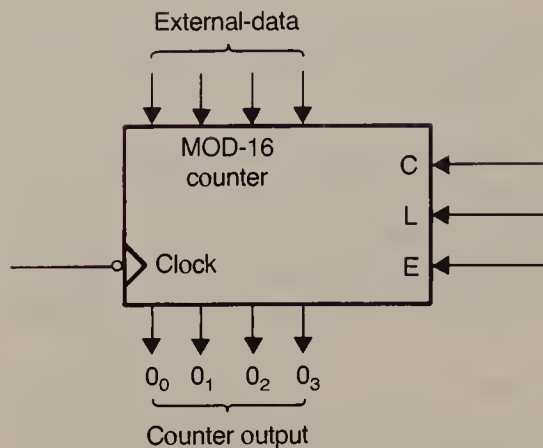


Figure 4.21 Timing Signals Generated by the Controller



a. Block Diagram

C	L	E	Clock	Action
1	X	X	X	Clear
0	1	X	↓	Load external data
0	0	1	↓	Count up
0	0	0	↓	No operation

b. Function Table

Figure 4.22 Characteristics of the Counter Used in the Controller Design

up in response to the next clock pulse (because the enable input E is tied to high). Such normal sequencing activity is desirable in the following situations:

- Present control state is T_0 , T_1 , T_4 , T_6 , or T_7 .
- Present control state is T_2 and $Q[1]Q[0] = 01$.
- Present control state is T_5 and $Z = 1$.

These results suggest that the SC should not affect the counter load control input L. Hence, these possibilities are excluded from SC truth table. The SC must exercise control only when there is a need for the counter to deviate from its normal counting sequence.

If the SC is removed and the counter load control input L is tied to low, the counter-decoder combination acts as a ring counter. By including the SC, a status-dependent ring counter is achieved.

Although the SC has to handle 8 inputs and generate 5 outputs, it must examine a few possibilities with 8 inputs. (*Note:* the total number of input combinations is $2^8 = 256$). When a designer encounters such a situation, the use of a programmed logic array (PLA) is a good choice for the system implementation. The PLA contents are shown in Figure 4.24(b).

This implementation follows the SC truth table. For each row of the SC truth

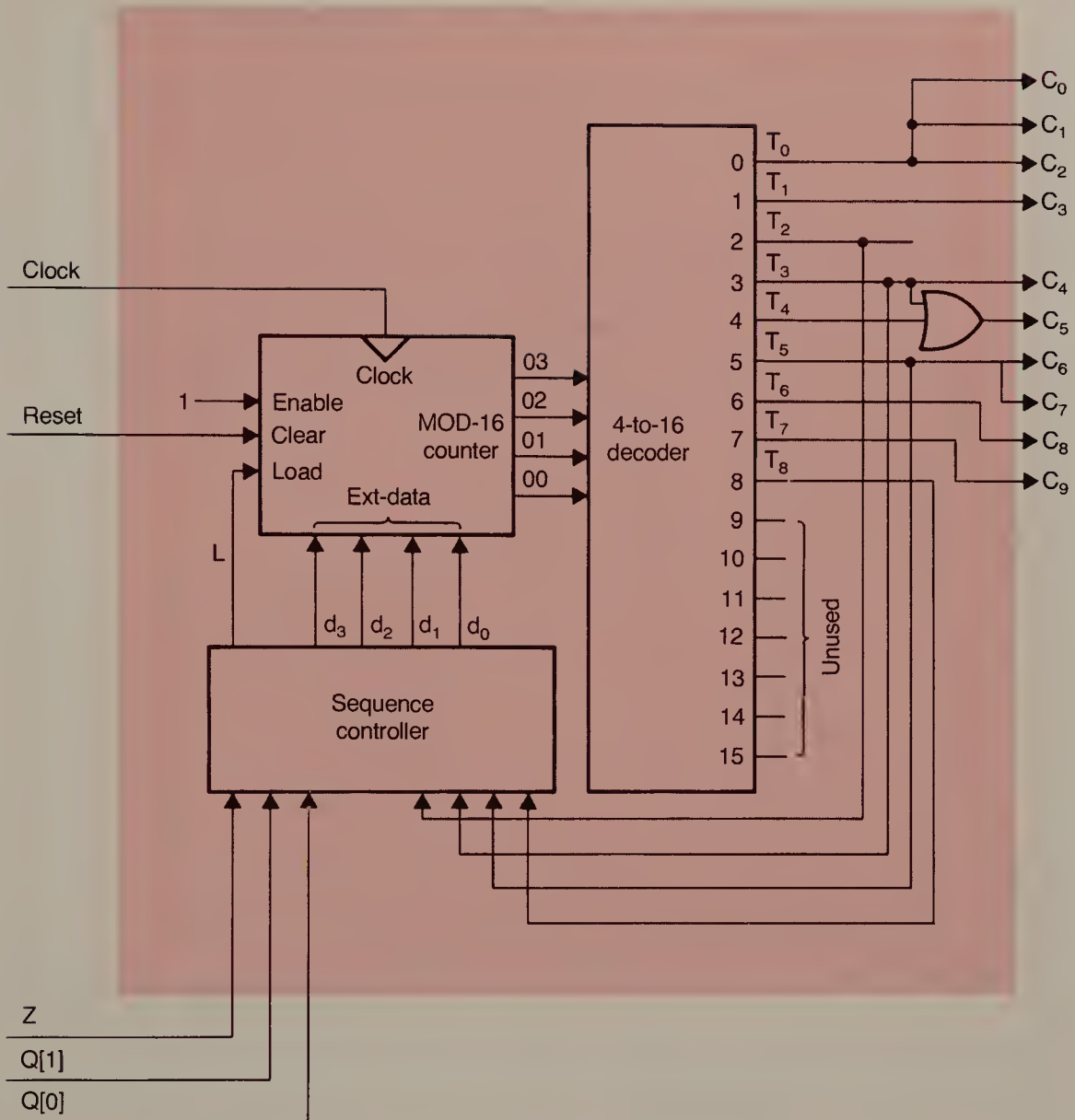


Figure 4.23 Logic Diagram of the Booth's Multiplier Controller

table, a product term is generated in the PLA. The product terms generated are summarized as follows:

$$P_0 = Q[1]^1 Q[0]^1 T_2$$

$$P_2 = Q[1] Q[0] T_2$$

$$P_2 = Q[1] Q[0]^1 T_2$$

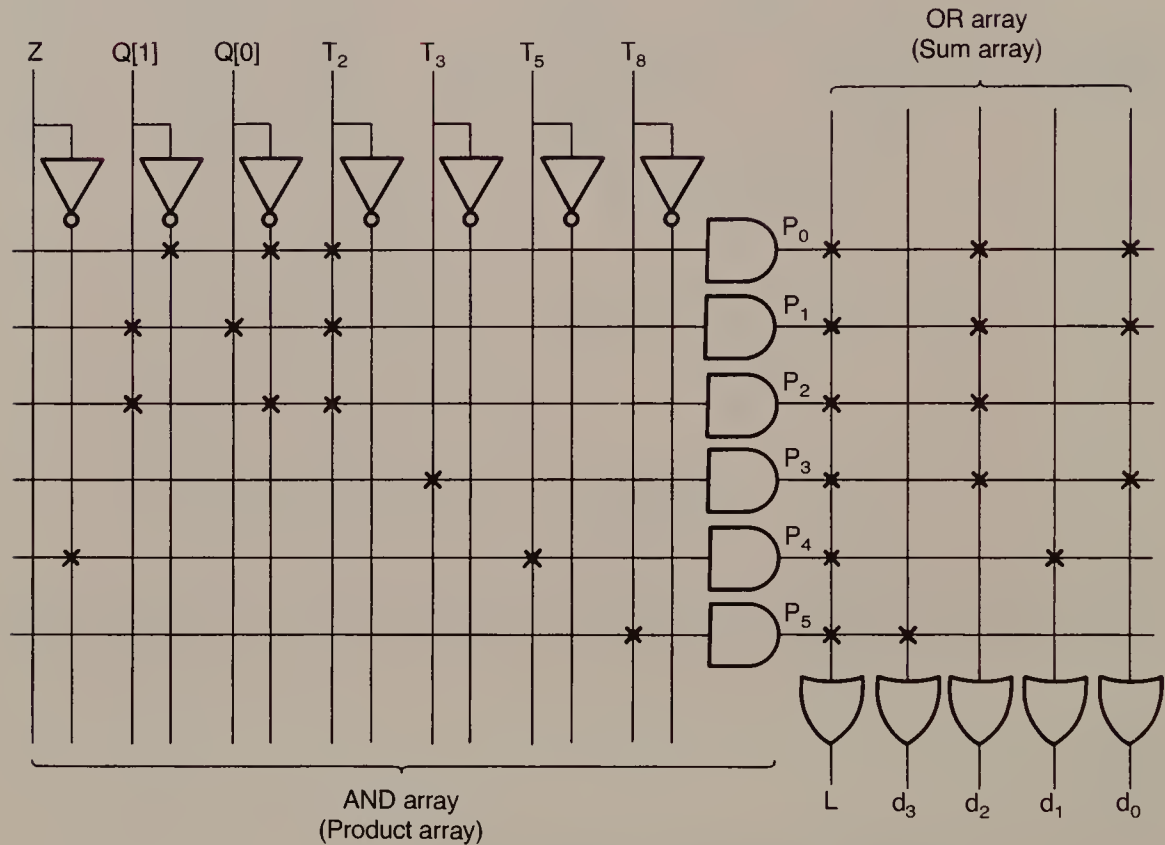
$$P_3 = T_3$$

$$P_4 = Z' T_5$$

$$P_5 = T_8$$

Z	Q [1]	Q [0]	T ₂	T ₃	T ₅	T ₈	L	External-data			
								d3	d2	d1	d0
X	0	0	1	X	X	X	1	0	1	0	1
X	1	1	1	X	X	X	1	0	1	0	1
X	1	0	1	X	X	X	1	0	1	0	0
X	X	X	X	1	X	X	1	0	1	0	1
0	X	X	X	X	1	X	1	0	0	1	0
X	X	X	X	X	X	1	1	1	0	0	0

a. Truth Table



b. PLA Implementation

Figure 4.24 Sequence Controller Design

The PLA generates five outputs: L, d₃, d₂, d₁, and d₀. Each output is directly generated by using the SC truth table and the product terms. The Boolean equations that govern the relationship between the product terms and the PLA outputs are summarized below.

$$L = P_0 + P_1 + P_2 + P_3 + P_4 + P_5$$

$$d_3 = P_5$$

$$d_2 = P_0 + P_1 + P_2 + P_3$$

$$d_1 = P_4$$

$$d_0 = P_0 + P_1 + P_3$$

From the above equations, notice that to generate the output L , all the product terms are selected and summed using the PLA-OR gate array. The output d_0 is produced by selecting three product terms— P_0 , P_1 and P_3 —and summing them. The use of a PLA has significantly minimized the design effort.

The controller design is completed by relating the control states (T_0 through T_8) with the control signals (C_0 through C_9) as follows:

$$C_0 = C_1 = C_2 = T_0$$

$$C_3 = T_1$$

$$C_4 = T_3$$

$$C_5 = T_3 + T_4$$

$$C_6 = C_7 = T_5$$

$$C_8 = T_6$$

$$C_9 = T_7$$

These results directly follow the controller action specified in Figure 4.20(b). When the control is in the state T_0 , three microoperations are needed: $A \leftarrow 0$, $L \leftarrow 4$, and $M \leftarrow \text{Inbus}$, implying that $C_0 = C_1 = C_2 = T_0$. $A \leftarrow F$ is performed when the control state is T_3 or T_4 . Therefore, $C_5 = T_3 + T_4$.

A control unit can be designed by using a single PLA and D flip-flops. The organization of a PLA control unit for the Booth's multiplier unit is shown in Figure 4.25.

In Figure 4.25, four D flip-flops and a PLA are used. The D flip-flops hold the present control state, and the PLA uses this information to generate the required control signals and the next control state. The flip-flops are updated with the next state information when the next clock arrives. The controller's state diagram (see Figure 4.20) contains 9 states. The binary encoding of these states calls for 4 bits and four D flip-flops are required to hold them.

The PLA of Figure 4.25 implements the state table shown in Figure 4.26. This state table is the tabular representation of the controller's state diagram. The table also includes the control functions to be activated at any particular state. This information is obtained from Figure 4.20(b). The PLA contents are shown in Figure 4.27. Verification of the correctness of this figure is left as an exercise.

The PLA, a programmable LSI component, is effective in replacing a random logic circuit that typically contains 6 to 8 MSI chips. Such replacement is desirable for the following reasons:

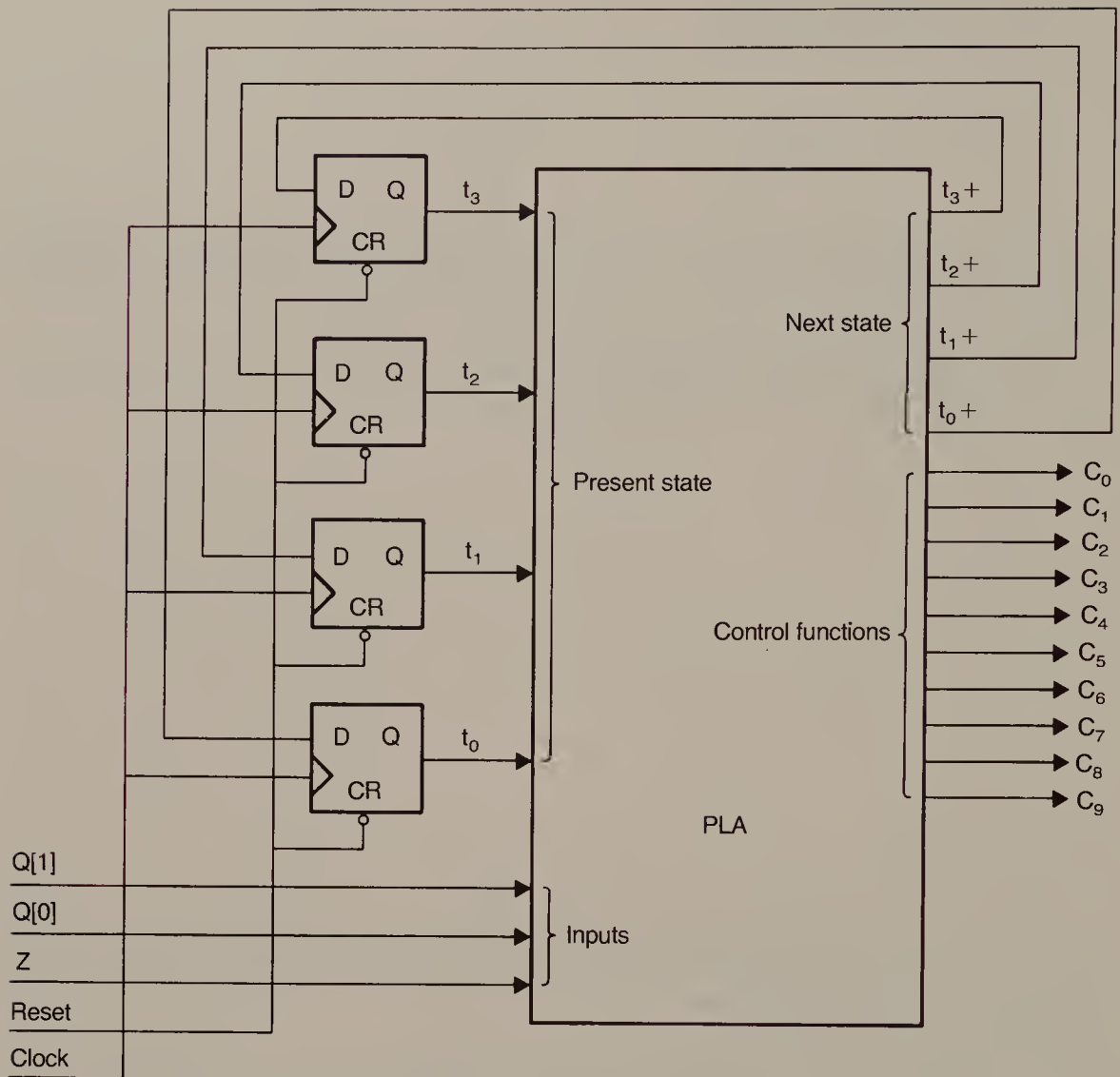


Figure 4.25 Organization of the PLA Control Unit for the Booth's Multiplier

1. Increased flexibility and reliability
2. Minimum design effort
3. Uniform component structure
4. Space saving

Uniform hardware component structures result in efficient mass production giving lower hardware cost. The size of the control memory (to be explained later) of the Intel 8086 processor is 504 words with 21 bits per word. To save chip area, this component is made by using a PLA, as opposed to a ROM. With the advent of integrated circuit technology, manufacturers are able to produce erasable programmable PLAs.

Such a product is produced by ALTERA Corporation. Such PLAs offer great flexibility since their contents can be altered on line. It is speculated that in the future the cost of the erasable PLA will drop to a level allowing wider usage.

PLA Input										PLA Output												
Present state is	Present state in binary				Inputs			Next state (in binary)				Control functions										
	t ₃	t ₂	t ₁	t ₀	Q [1]	Q [0]	Z	t ₃ ⁺	t ₂ ⁺	t ₁ ⁺	t ₀ ⁺	C ₀	C ₁	C ₂	C ₃	C ₄	C ₅	C ₆	C ₇	C ₈	C ₉	
T ₀	0	0	0	0	X	X	X	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0
T ₁	0	0	0	1	X	X	X	0	0	1	0	0	0	0	1	0	0	0	0	0	0	0
T ₂	0	0	1	0	0	1	X	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0
T ₂	0	0	1	0	0	0	X	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0
T ₂	0	0	1	0	1	1	X	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0
T ₂	0	0	1	0	1	0	X	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
T ₃	0	0	1	1	X	X	X	0	1	0	1	0	0	0	0	1	1	0	0	0	0	0
T ₄	0	1	0	0	X	X	X	0	1	0	1	0	0	0	0	0	1	0	0	0	0	0
T ₅	0	1	0	1	X	X	0	0	0	1	0	0	0	0	0	0	0	1	1	0	0	0
T ₅	0	1	0	1	X	X	1	0	1	1	0	0	0	0	0	0	0	1	1	0	0	0
T ₆	0	1	1	0	X	X	X	0	1	1	1	0	0	0	0	0	0	0	0	1	0	0
T ₇	0	1	1	1	X	X	X	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1
T ₈	1	0	0	0	X	X	X	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Controller's state table

Figure 4.26 PLA Table

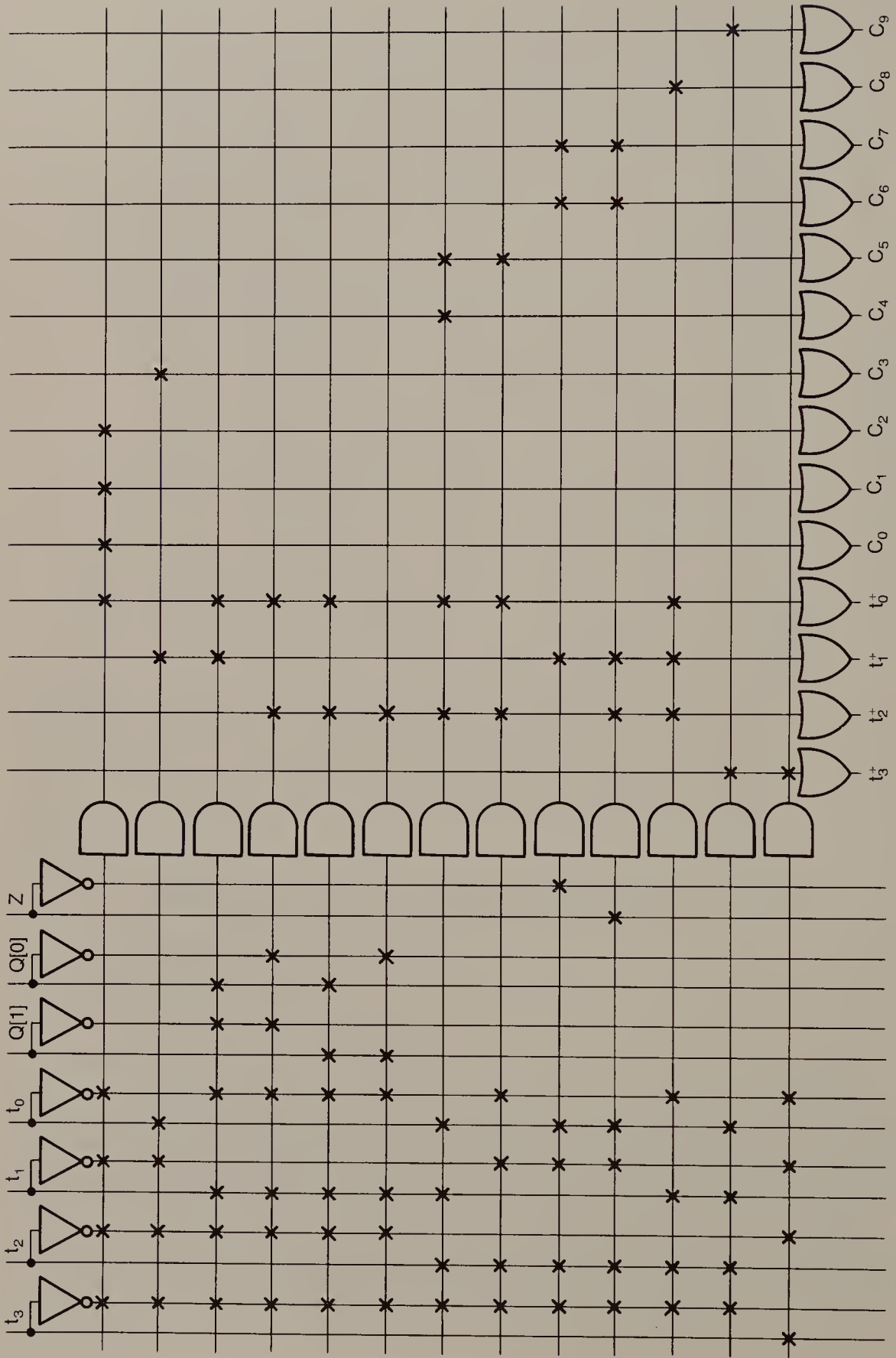


Figure 4.27 PLA Contents

4.3.2 Microprogrammed Control Unit

The design of a microprogrammed unit will be discussed in detail. As seen earlier, a microprogrammed control unit's control words are held in a separate memory called the *control memory* (CM). Each control word contains signals to activate one or more microoperations. When these words are retrieved in a sequence, a set of microoperations are activated that will complete the desired task.

Retrieval and interpretation of the control words are done the same as a conventional program. The instructions of a processor are held in the main store. They are fetched and executed in a sequence. By changing the instructions stored in the main memory, the processor can perform different functions. Similarly, by changing the contents of the CM, the control unit can execute a different control function. Therefore, the microprogrammed approach offers greater flexibility than its hardwired counterpart, since it provides an easy means for altering the contents of the CM.

Generally, all microinstructions have two important fields:

- Control field
- Next-address field

The purpose of the control field is to indicate which control lines are to be activated. The purpose of the next address field is to specify the address of the next microinstruction to be executed. When M. V. Wilkes proposed this idea in 1951, he used the organization close to the one shown in Figure 4.28.

In Figure 4.28 the decoder and diode matrix utilize an 8×8 ROM. Each ROM holds an 8-bit control word whose low-order three positions specify the next address, and high-order five positions indicate control signals. The contents of the ROM are retrieved by specifying the address in the control memory address register (CMAR). For example, if $X_2X_1X_0 = 010$, then control lines C_0 and C_3 are enabled, and the next ROM address, 011, is fed back to the CMAR at the same time.

It is possible to load the CMAR with an externally specified new starting address. Whenever the external load input E is 1, the CMAR is loaded with an externally specified address. It is then possible to keep several microprograms in the control memory and execute any desired microprogram by specifying the starting address as an external address.

Another important feature of this organization is the ability to implement conditional branching. The external condition sets or resets the condition flip-flop. For instance, if the external condition sets the condition flip-flop to 1, control is transferred to ROM address 1 ($X_2X_1X_0 = 001$) after execution of microinstruction at ROM address 5. If the external condition is zero, the condition flip-flop will be reset. Therefore, after execution of the microinstruction at ROM address 5, the control will be transferred to ROM address 6 ($X_2X_1X_0 = 110$).

The microprogramming approach involves the inclusion of a control memory in addition to the conventional main memory. Therefore, a major design effort has its emphasis on minimizing the length of the microinstruction. The length of the microinstruction decides the size of the control store, as well as the cost involved with this approach. The length of a microinstruction is directly related to the following factors:

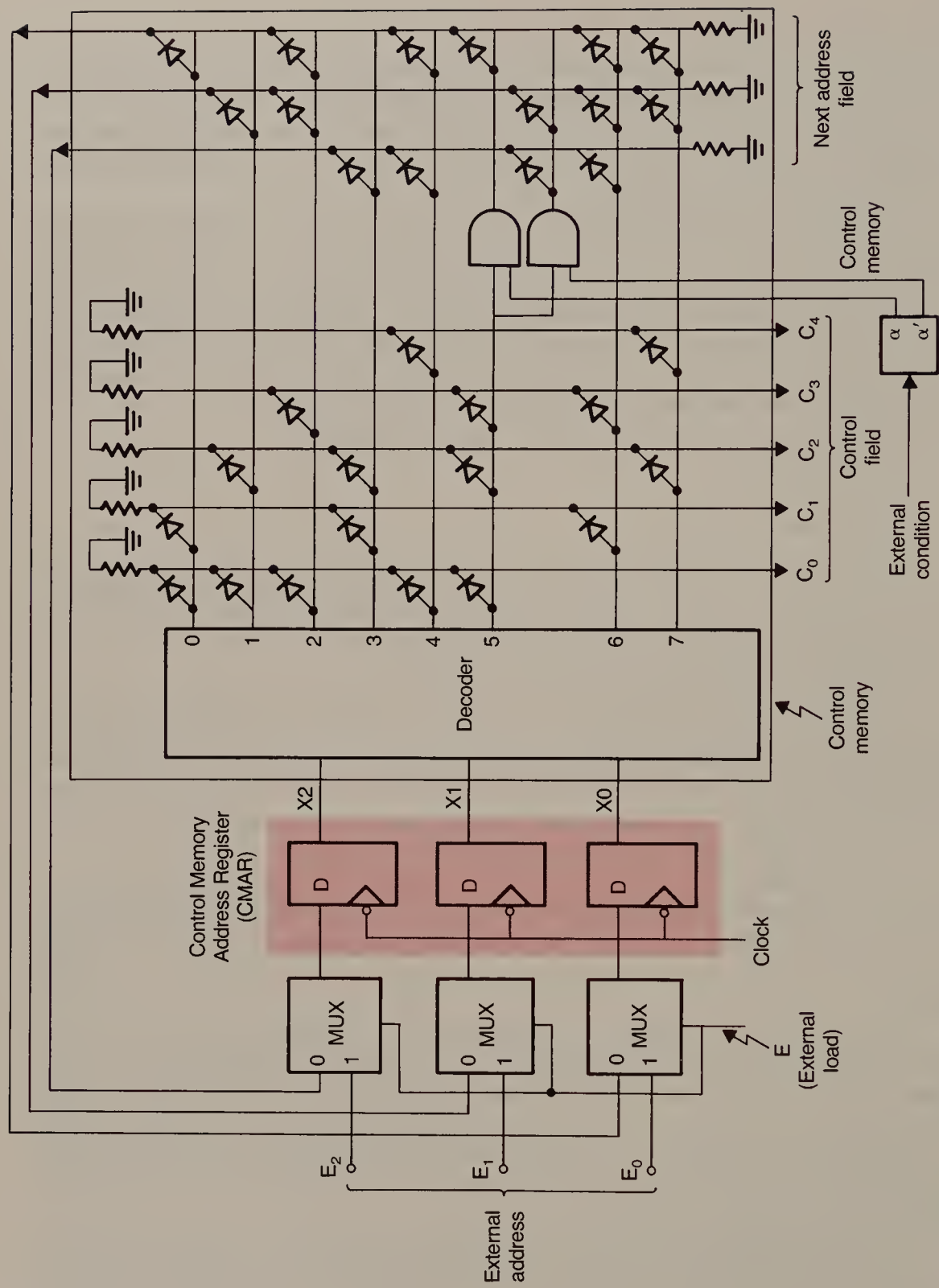


Figure 4.28 Wilke's Design

- This method is also known as *unencoded format*.

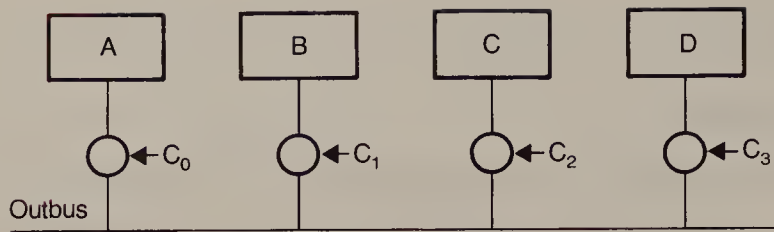


Figure 4.29 A Simple Register Transfer

There are only five valid binary patterns in the preceding format; but from the basic switching theory, five distinct binary patterns can be represented by using only 3 bits. Such an arrangement is shown in Figure 4.30.

In Figure 4.30, the control information is encoded into a 3-bit field, and a decoder is needed to extract the actual control information. The relationship between the encoded and the actual control information is specified as follows:

E_2	E_1	E_0	
0	0	0	No operation
0	0	1	Outbus = A
0	1	0	Outbus = B
0	1	1	Outbus = C
1	0	0	Outbus = D

This method is known as *encoded format*, which leads to a short control field and short microinstructions. The price paid for such a reduction is the need to have a decoder. Therefore, a compromise must be sought. Fifteen control lines can be specified in a fully unencoded form as shown next:

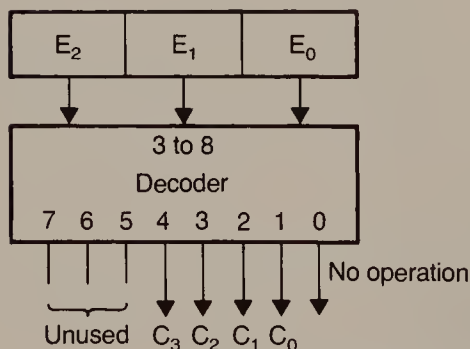
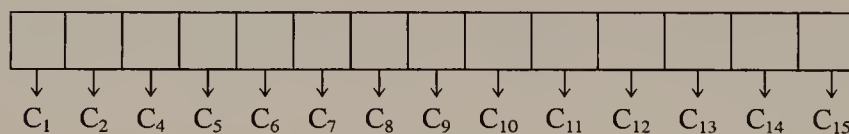
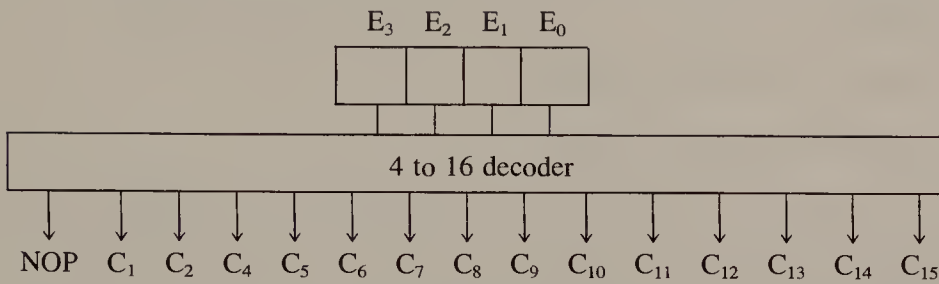
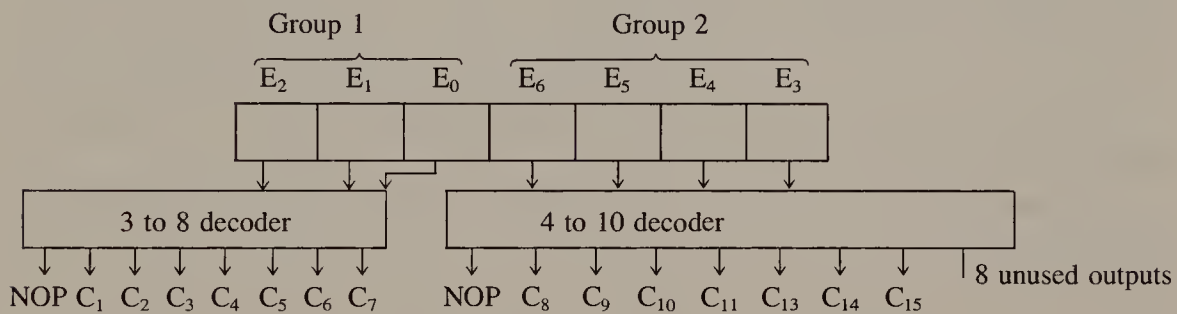


Figure 4.30 Encoded Control Arrangement

The same information can be specified in the encoded form as shown next:



In the first and second cases, the sizes of the control field are 16 and 4 bits, respectively. The latter approach, however, needs a 4-to-16 decoder to derive the actual control signals. As a measure of compromise, the control information is given by a partial encoding as shown next:



The control signals are partitioned into disjoint groups, so two signals of different groups can be activated in parallel. The control signals have been divided into three groups:

Group 1: C_1 , C_2 , C_3 , C_4 , C_5 , C_6 , C_7

Group 2: C_8 , C_9 , C_{10} , C_{11} , C_{12} , C_{13} , C_{14} , C_{15}

With the above grouping, C_8 and C_1 are activated simultaneously but not C_1 and C_2 . The actual control signals are derived by using one 3 to 8 and one 4 to 10 decoders. In the last case, the control field requires 7 bits which lie midway between the unencoded and fully encoded approaches. Microinstructions are classified into two groups, which express parallelism ability and the amount of encoding called:

- Horizontal
- Vertical

The horizontal microinstruction has the following features:

- Long microinstructions
- Capability of expressing a high degree of parallelism
- Very little encoding

The vertical instruction possesses the following basic attributes:

- Short instructions
- Limited scope for expressing parallelism
- Needs considerable amount of decoding

The preceding classification is done informally; no precise formal definition exists.

How to specify the next address of the microinstruction that follows the current instruction being executed will be discussed. In the original design proposed by M. V. Wilkes, the next address is specified in each microinstruction. A close examination reveals that except in the case of a branch instruction, the address of the next microinstruction to be executed is the address of the memory word that follows the current microinstruction word. Therefore, the next address field from the microinstruction can be eliminated by introducing a separate register used as a pointer to the next instruction to be executed. This register is, in principle, the same as the PC. For this reason, it is known as the *microprogram counter* (MPC). The MPC is usually incremented after each microinstruction fetch so it will always point to the next microinstruction to be executed.

The architecture of a typical modern microprogrammed control unit is shown in Figure 4.31.

A summary of the use of various components included in this organization is provided next:

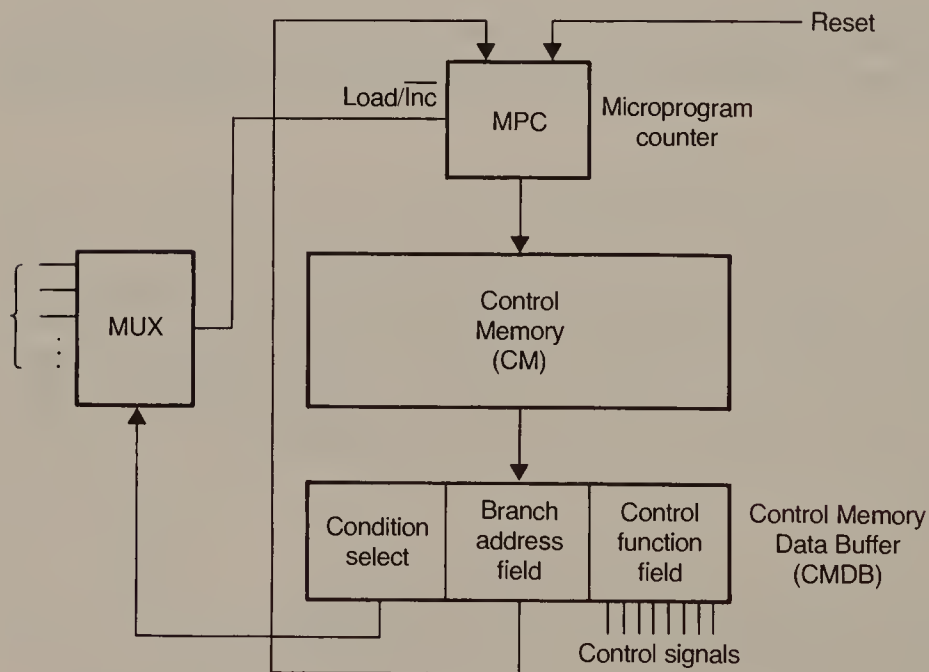


Figure 4.31 General-purpose Microprogrammed Control Organization (From J. P. Hayes, *Computer Architecture and Organization*, McGraw-Hill, 1978, p. 280. Adapted by permission of McGraw-Hill)

- *Control memory buffer register (CMBR)*: The CMBR functions the same as the MBR of the main memory. It is basically a latch, and acts as a buffer for the microinstructions retrieved from the CM. Typically, each microinstruction will have three distinct fields as diagrammed below:

Condition select	Branch address field	Control function field
---------------------	-------------------------	---------------------------

The condition *select field* selects the external condition to be tested. If the selected condition is true, the output of the MUX will be 1. Since the output of the MUX is connected to the load input of the MPC, the MPC will be loaded with the address specified in the branch address field of the microinstruction. However, if the selected external condition is false, the MPC will point to the next microinstruction to be executed. Therefore, this arrangement allows conditional branching. The control function field of the microinstruction may hold the control information in an encoded form.

- *The microprogram counter (MPC)*: The MPC holds the address of the next microinstruction to be executed. Initially, it is loaded from an external source to point to the starting address of the microprogram to be executed. From then on, the MPC is incremented after each microinstruction fetch, and the instruction fetched is transferred to the CMBR. When a branch instruction is encountered, the MPC will be loaded with the contents of the branch address field of the microinstruction that is held in the CMBR.
- *External condition select MUX*: This MUX selects one of the external conditions according to the contents of the condition select field of the microinstruction. Therefore, the condition to be selected must be specified in an encoded form. Any encoding leads to a short microinstruction, which implies a small control memory; hence, the cost is reduced. Suppose six external conditions, X_1 , X_2 , X_3 , X_4 , X_5 , X_6 , are to be tested; then the condition-select field and the MUX can be organized as shown in Figure 4.32.

In Figure 4.32, the contents of the condition-select field and actions taken are summarized next:

CONDITION SELECT	ACTION TAKEN
000	No branching
001	Branch if $X_1 = 1$
010	Branch if $X_2 = 1$
011	Branch if $X_3 = 1$

100	Branch if $X_4 = 1$
101	Branch if $X_5 = 1$
110	Branch if $X_6 = 1$
111	Branch always (unconditional branch)

If the condition-select field contains 000, the output of the MUX is 0. The MPC will then be incremented to point the next address. Therefore, no branching will take place. When the condition-select field is 111, the output of the MUX is 1, causing the MPC to download a branch address. Since this happens regardless of any external condition, an unconditional branch is formed. When the condition-select field is 010, the output of the MUX is the same as the value of the external condition variable X_2 . Therefore, the MPC will be loaded with a branch address only when $X_2 = 1$; otherwise it is incremented. Conditional branching is achieved. The structure of the control memory will now be addressed.

In the early days, the control memory was organized as a ROM. ROM's were constructed by using a diode matrix, since it accessed faster than ferrite-core read-write memories. Since the control program had little likelihood to change, it was economical to use a ROM rather than a read-write memory.

Present technology allows the use of control memories whose contents may be rewritten. As mentioned earlier, a CPU designed with such a writable control memory can be made to interpret different instruction set, by inserting the appropriate microprogram. In effect, it is possible to produce different virtual machines with the same hardware. When the microcode of CPU A interprets the instruction set of CPU B, machine A emulates machine B. This philosophy is adopted by many leading manufacturers, such as the Burroughs Corporation and IBM.

Microprogramming is the activity of writing microprograms for a microprogrammable processor. Writing microprograms is similar to writing programs in an assembly

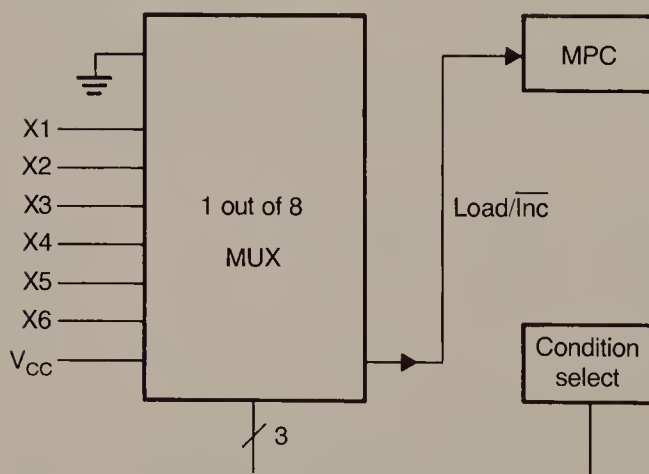


Figure 4.32 External Condition Select Multiplexer

language. However, a microprogrammer must have a more thorough knowledge about the system's architecture than an assembly language programmer. To speed up the microcode-development process, microprograms are written in a symbolic language called *microassembly language*. These are then translated by a microassembler to produce microcodes that can be held in the CM.

The design of a typical microprogrammed system is now discussed. Consider the design of a microprogrammed control unit for the 4 x 4 Booth's multiplier presented earlier. For the first step, write the microprogram in a symbolic form, as shown in Figure 4.33.

This program is identical to the register transfer description presented earlier. Thus, there is a one-to-one correspondence between the register transfer description of a task and its microprogram implementation. Each line of the above symbolic program is stored as a word in the CM whose address is specified as an integer number. For example, the word stored in the CM address 0 activates operations $A \leftarrow 0$, $M \leftarrow \text{Inbus}$, and $L \leftarrow 4$. Similarly, the word stored in the CM address 4 implements the unconditional branch instruction go to RSHIFT. The control memory is able to hold 13 words, requiring a 4-bit branch address field.

In this task, three conditions, $Q[1]Q[0] = 01$, $Q[1]Q[0] = 10$, and $Z = 0$, are checked. These conditions are applied as inputs to the condition select MUX. Additionally, a logic 0 and a logic 1 are applied as data inputs to this MUX to take care of no-branch and unconditional-branch situations, respectively. Therefore, the MUX is able to handle five data inputs and must be at least a 1-out-of-8 data selector. The size of the condition-select field must be 3 bits wide ($2^3 = 8$).

A 3-bit condition-select field guess gives eight distinct 3-bit patterns. However, only the first five 3-bit patterns are used to encode the five different conditions encountered in this problem. With this design, the condition select field may be interpreted as follows on the top of page 164:

Control Memory Address		Control Word
0	START	$A \leftarrow 0$, $M \leftarrow \text{Inbus}$, $L \leftarrow 4$
1		$Q[4:1] \leftarrow \text{Inbus}$, $Q[0] \leftarrow 0$;
2	LOOP	If $Q[1:0] = 01$ Then go to ADD
3		If $Q[1:0] = 10$ Then go to SUB
4		Go to RSHIFT;
5	ADD	$A \leftarrow A + M$;
6		Go to RSHIFT;
7	SUB	$A \leftarrow A - M$;
8	RSHIFT	ASR (A\$Q), $L \leftarrow L - 1$;
9		If $Z = 0$ then go to LOOP
10		Outbus = A;
11		Outbus = $Q[4:1]$;
12	HALT	Go to HALT

Figure 4.33 Symbolic Microprogram for 4 x 4 Booth's Multiplier

CONDITION-SELECT FIELD	INTERPRETATION
000	No branching
001	Branch if $Q[1] = 0$ and $Q[0] = 1$
010	Branch if $Q[1] = 1$ and $Q[0] = 0$
011	Branch if $Z = 0$
100	Unconditional branching

With these details, the size of the control word is found as follows:

$$\begin{aligned}
 \text{size of a control word} &= \text{size of the cond.-select field} + \text{size of the branch address field} + \text{number of control functions} \\
 &= 3 + 4 + 10 \\
 &= 17 \text{ bits}
 \end{aligned}$$

Hence, the sizes of the CMDB and CM are 17 and 221 (13 x 17) bits, respectively. The complete hardware organization of the control unit is shown in Figure 4.34.

Finally, the production of the binary microprogram stored in the CM will be discussed. As mentioned before, for each line of the symbolic program listing, there exists a control word. For example, consider the first line of the symbolic listing shown (see Figure 4.33). This instruction introduces no branching. Therefore, the condition-select

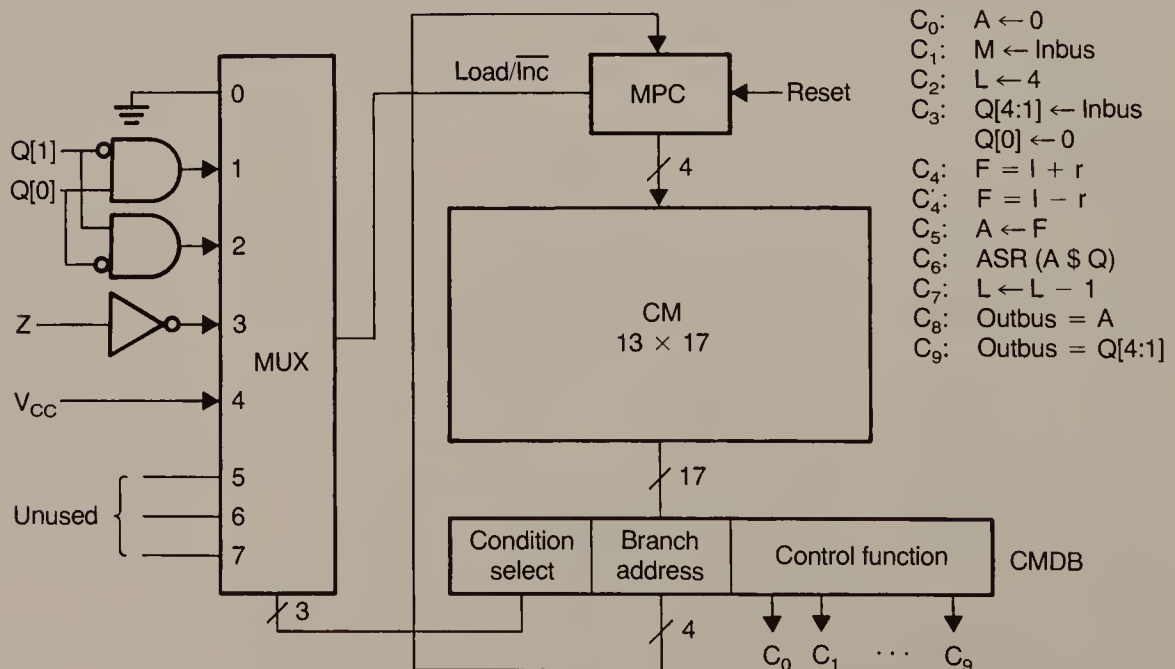


Figure 4.34 Microprogrammed Multiplier Control Unit

field should be 000. Thus the contents of the branch address field are irrelevant. However, the contents of this field can be reset to 0000 without any loss of generality. In this instruction, three microoperations C_0 , C_1 , and C_2 are activated. Therefore, only the corresponding bit positions in the control function field are set to 1. This results in the following binary microinstruction:

Condition	Branch	Control
<u>Select</u>	<u>Address</u>	<u>Function</u>
000	0000	1110000000

The binary microinstructions corresponding to the tenth line of the symbolic microprogram does not activate any microoperations. Therefore, all bits of the control function field must be reset to 0. Since the branching is based on the value of the external condition variable Z , the condition select field must be set to 011. If the condition is met, the program control must branch to execute the microinstruction whose symbolic address is LOOP. Since this label appears in line number 2 of the symbolic listing, the branch address is 2 (0010). The value of the branch address field must be 0010. The following binary microinstruction is obtained:

Condition	Branch	Control
<u>Select</u>	<u>Address</u>	<u>Function</u>
011	0010	0000 0000 0000

Continuing in this manner, the complete binary-microprogram can be produced (Figure 4.35).

The microprogrammed Booth's multiplier is timed by using a three-phase clock signal generated as shown in Figure 4.36.

The contents of the CM word whose address is specified in the MPC are transferred into the CMBR with the trailing edge of the timing pulse P_0 . The processing section responds to the trailing edge of the timing pulse P_1 . The MPC is then loaded with a branch address or incremented by 1 with the trailing edge of the timing pulse P_2 . This process continues until the last instruction of the microprogram is reached. Close examination reveals that the microinstruction execution and the next address generation may be overlapped by making both the processing section and the MPC respond to the same timing pulse. Branching and microoperation initiation can thus be combined in the same control word.

The control words stored in CM addresses 5 and 6 (see Figure 4.35) can be combined as one single word as shown next:

CONDITION SELECT	BRANCH ADDER	CONTROL FUNCTION
		$C_0 C_1 C_2 C_3 C_4 C_5 C_6 C_7 C_8 C_9 C_{10}$
100	1000	0 0 0 0 1 1 0 0 0 0 0;
		$A \leftarrow A + M$ and go to 8

ROM Address												
In decimal	In binary				Cond. Sel			Branch Addr.				
					c _{s2}	c _{s1}	c _{s0}	b _{r3}	b _{r2}	b _{r1}	b _{r0}	c ₀
0	0	0	0	0	0	0	0	0	0	0	0	1
1	0	0	0	1	0	0	0	0	0	0	0	0
2	0	0	1	0	0	0	1	0	1	0	1	0
3	0	0	1	1	0	1	0	0	1	1	1	0
4	0	1	0	0	1	0	0	1	0	0	0	0
5	0	1	0	1	0	0	0	0	0	0	0	0
6	0	1	1	0	1	0	0	1	0	0	0	0
7	0	1	1	1	0	0	0	0	0	0	0	0
8	1	0	0	0	0	0	0	0	0	0	0	0
9	1	0	0	1	0	1	1	0	0	1	0	0
10	1	0	1	0	0	0	0	0	0	0	0	0
11	1	0	1	1	0	0	0	0	0	0	0	0
12	1	1	0	0	1	0	0	1	1	0	0	0

Figure 4.35 Binary Listing of the Microprogram For the 4 x 4 Booth's Multiplier

When doing so, however, one must make sure that the system operation produces the correct result. By separately handling branch instructions (as in Figure 4.35), a new strategy for minimizing the size of the control store can be devised. This method is referred to as *multiple microinstruction format* and is covered later.

Next, the design of a microprogrammed processor is illustrated. The programming model of this processor is shown in Figure 4.37.

The CPU entails two registers:

1. An 8-bit accumulator register A
2. A 2-bit flag register F

The flag register holds only zero (Z) and carry (C) flags. All programs and data are stored in the 256 x 8 RAM. The detailed hardware schematic of the data-flow part of this processor is shown in Figure 4.38.

From Figure 4.38, it can be seen that the hardware organization includes four more 8-bit registers, PC, IR, MAR, and BUFFER. These registers are transparent to a programmer. The 8-bit register BUFFER is used to hold the data that is retrieved from memory. In this system, only a restricted number of data paths are available. These paths are controlled by the control inputs C₀ through C₉, as described on page 167 bottom.

Control Word									Comments
Control Function									
c ₁	c ₂	c ₃	c ₄	c ₅	c ₆	c ₇	c ₈	c ₉	
1	1	0	0	0	0	0	0	0	A ← 0, M ← Inbus, L ← 4
0	0	1	0	0	0	0	0	0	Q [4:1] ← Inbus, Q [0] ← 0
0	0	0	0	0	0	0	0	0	If Q [1:0] = 01 Then go to 5 (ADD)
0	0	0	0	0	0	0	0	0	If Q [1:0] = 10 Then go to 7 (SUB)
0	0	0	0	0	0	0	0	0	go to 8 (RSHIFT)
0	0	0	1	1	0	0	0	0	A ← A + M
0	0	0	0	0	0	0	0	0	go to 8 (RSHIFT)
0	0	0	0	1	0	0	0	0	A ← A - M
0	0	0	0	0	1	1	0	0	ASR (A\$Q), L ← L - 1
0	0	0	0	0	0	0	0	0	If Z = 0 Then go to 2
0	0	0	0	0	0	0	1	0	Outbus = A
0	0	0	0	0	0	0	0	1	Outbus = Q [4:1]
0	0	0	0	0	0	0	0	0	Go to 12 (HALT)

MICROOPERATION	COMMENT
C ₀ : PC ← 0	Clear PC to zero.
C ₁ : PC ← PC + 1	Advance the PC.
C ₂ C ₅ C ₆ ': PC ← M ((MAR))	Read the data from the memory and save it in the PC.
C ₃ 'C ₄ : MAR ← PC	Transfer the contents of the PC into MAR.
C ₅ C ₆ ' C ₇ : BUFFER ← M ((MAR))	Read the data from the memory and save the result in BUFFER.
C ₃ C ₄ : MAR ← BUFFER	Transfer the content of the BUFFER into MAR.
C ₅ C ₆ 'C ₈ : IR ← M ((MAR))	Read the data from memory and save the result into IR.
C ₉ : A ← F	Transfer the ALU output into the A register.
C ₅ 'C ₆ ': M ((MAR)) ← A	Save the accumulator contents in the memory.

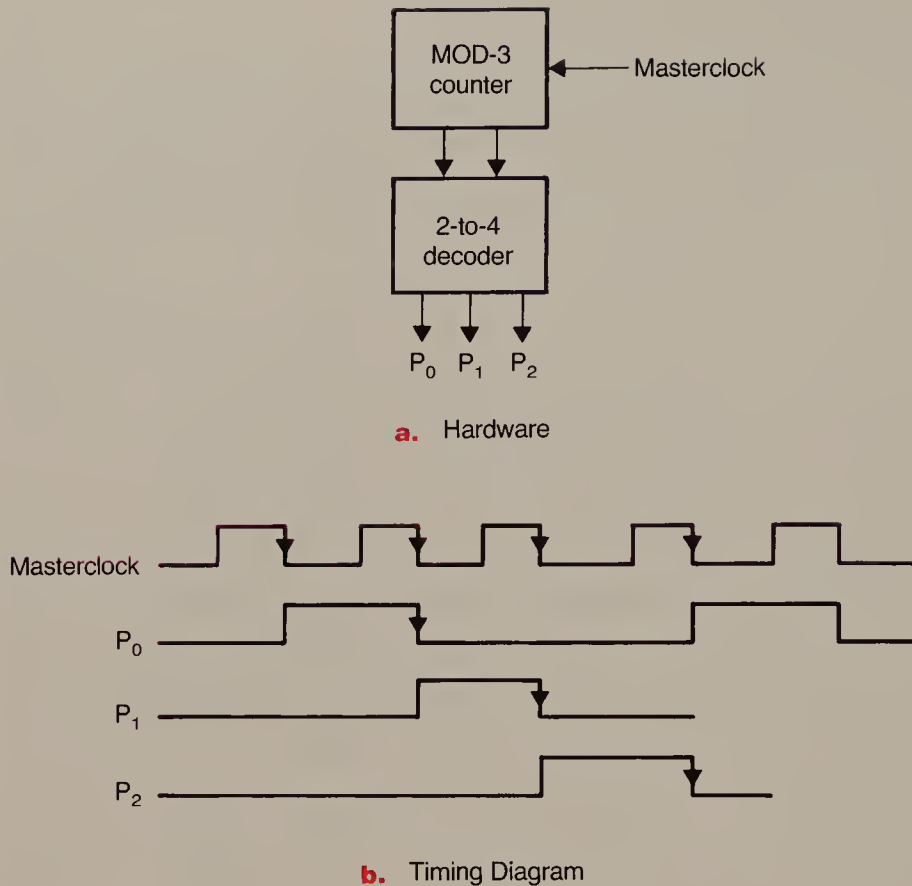


Figure 4.36 Microinstruction Timing

The ALU in this hardware can perform eight distinct operations. They are controlled by the select inputs C_{10} , C_{11} , and C_{12} as follows:

C_{10}	C_{11}	C_{12}	F
0	0	0	0
0	0	1	R
0	1	0	$L + R$
0	1	1	$L - R$
1	0	0	$L + 1$
1	0	1	$L - 1$
1	1	0	$L \wedge R$
1	1	1	L'

Implementation of the instruction set described in Figure 4.39 is now shown by writing a suitable microprogram.

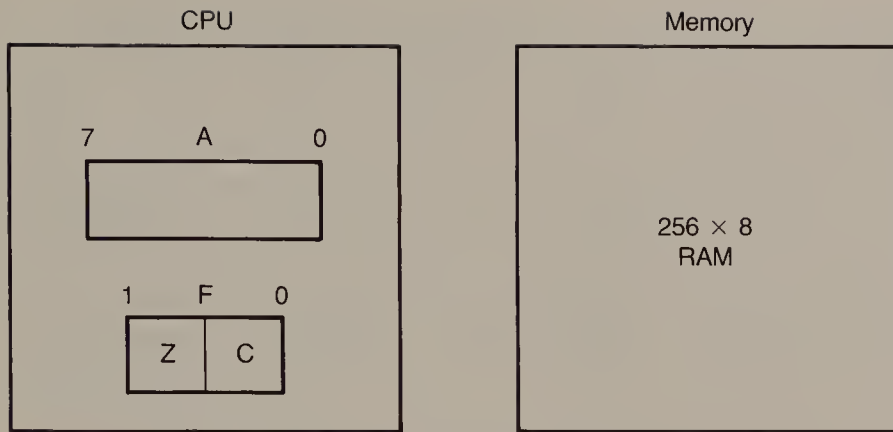


Figure 4.37 Programming Model of a Simple Processor

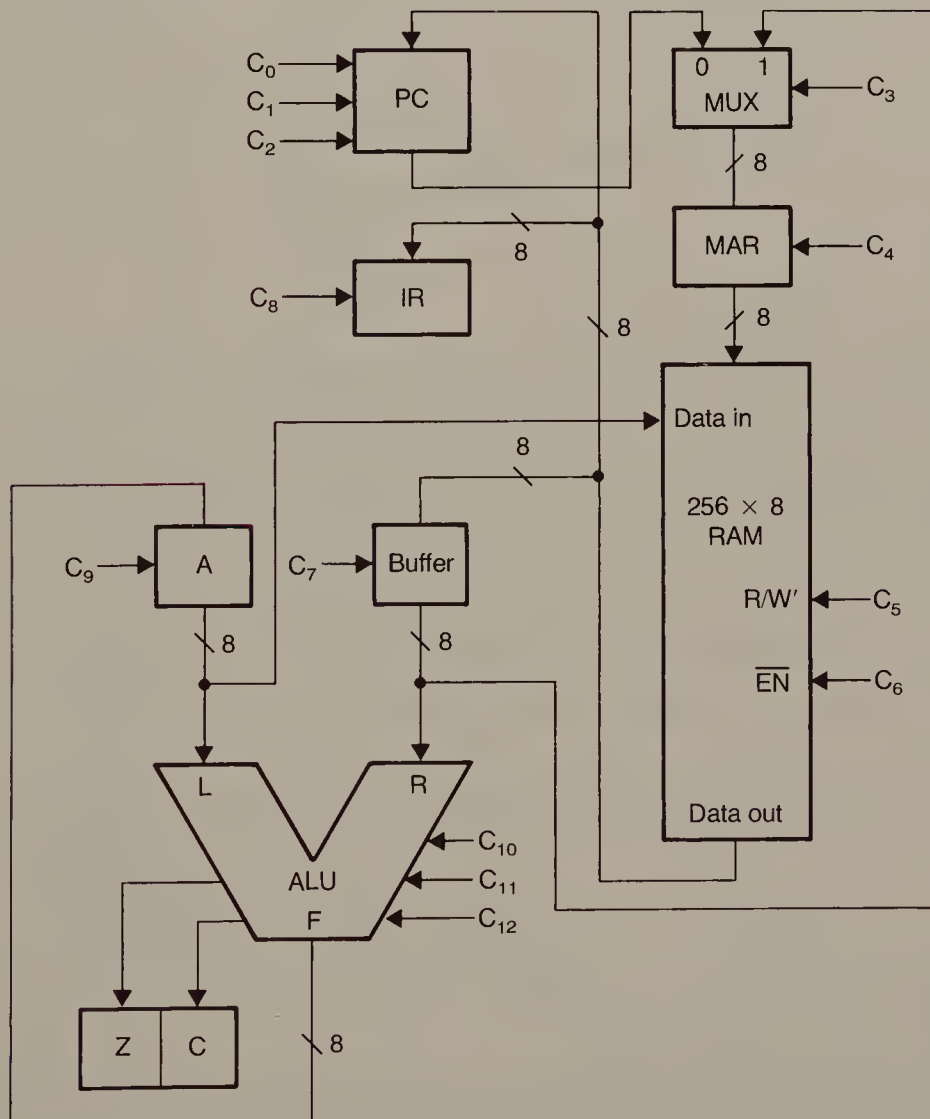


Figure 4.38 Hardware Schematic of the Simple Processor

General Format	Instruction Length in Bytes	Object Code		Instruction Type	Operation	Comment
		In binary	In hex			
LDA <addr>	2	0000 1000	08	MRI	$A \leftarrow M(\langle\text{addr}\rangle)$	Load accumulator direct
		<addr8>	<addrH>			
STA <addr>	2	0000 1001	09	MRI	$M(\langle\text{addr}\rangle) \leftarrow A$	Store accumulator direct
		<addr8>	<addrH>			
ADD <addr>	2	0000 1010	0A	MRI	$A \leftarrow A + M(\langle\text{addr}\rangle)$	Add accumulator direct
		<addr8>	<addrH>			
SUB <addr>	2	0000 1011	0B	MRI	$A \leftarrow A - M(\langle\text{addr}\rangle)$	Subtract accumulator direct
		<addr8>	<addrH>			
JZ <addr>	2	0000 1100	0C	MRI	If $Z = 1$ then $PC \leftarrow \langle\text{addr}\rangle$ else $PC \leftarrow PC + 1$	Jump on zero flag set
		<addr8>	<addrH>			
JC <addr>	2	0000 1101	0D	MRI	If $C = 1$ then $PC \leftarrow \langle\text{addr}\rangle$ else $PC \leftarrow PC + 1$	Jump on carry flag set
		<addr8>	<addrH>			
AND <addr>	2	0000 1110	0E	MRI	$A \leftarrow A \wedge M(\langle\text{addr}\rangle)$	And accumulator direct
			<addrH>			
CMA	1	0000 0000	00	NMRI	$A \leftarrow A'$	Complement accumulator
INCA	1	0000 0010	02	NMRI	$A \leftarrow A + 1$	Increment accumulator
DCRA	1	0000 0100	04	NMRI	$A \leftarrow A - 1$	Decrement accumulator
HLT	1	0000 0110	06	NMRI	Halt	Halt CPU.

Note:

<addr8>: 8-bit memory address in binary

<addrH>: 8-bit memory address in hex

MRI: memory reference instruction

NMRI: nonmemory reference instruction.

Figure 4.39 Instruction Set to be Implemented.

From Figure 4.39, notice that the proposed instruction set contains 11 instructions. The first 7 instructions are classified as memory reference instructions, since they all require a memory address (which is an 8-bit quantity in this case). The last 4 instructions do not require any memory address; they are called nonmemory reference instructions. Each memory reference instruction is assumed to occupy 2 consecutive bytes in the RAM. The first byte is reserved for the op-code, and the second byte indicates the 8-bit memory address. In contrast, a nonmemory reference instruction takes only one byte of storage. This instruction set supports only two addressing modes: implicit and direct. Both branch instructions are assumed to be absolute mode branch instructions. The op-code encoding for this instruction set is carried out in a logical manner, as explained in Figure 4.40.

The bit I_3 of Figure 4.40 decides the instruction type. If $I_3 = 1$, it is a memory reference instruction (MRI), otherwise it is a nonmemory reference instruction (NMRI).

Within the memory reference category, instructions are classified into four groups, as follows:

GROUP NO.	INSTRUCTIONS
0	Load and store
1	Add and subtract
2	Jumps
3	Logical

There are two instructions in the first three groups. Bit I_0 is used to determine the desired instruction of a particular group. If I_0 of group 0 equals zero, it is the load (LDA) instruction; otherwise it is the store (STA) instruction. Nevertheless, no such classification is required for group 3 and the nonmemory reference instructions.

As mentioned before, the instruction execution involves the following steps:

Step 1: Fetch the instruction.

Step 2: Decode the instruction to find out the required operation.

Step 3: If the required operation is a halt operation, then go to Step 6; otherwise continue.

Step 4: Retrieve the operands and perform the desired operation.

Step 5: Go to Step 1.

Step 6: Execute an infinite LOOP.

The first step is known as the fetch cycle, and the rest are collectively known as the execution cycle. To decode the instruction, the hardware shown in Figure 4.41 is used.

With this hardware and the status flags (Z and C), a microprogram to implement the instruction set can be written. The symbolic version of this microprogram is shown in Figure 4.42.

Mnemonic	Op-code Bit of Their Interpretations				TC	GN		SC
	I ₇	I ₆	I ₅	I ₄				
LDA	0	0	0	0	1	0	0	0
STA	0	0	0	0	1	0	0	1
ADD	0	0	0	0	1	0	1	0
SUB	0	0	0	0	1	0	1	1
JZ	0	0	0	0	1	1	0	0
JC	0	0	0	0	1	1	0	1
AND	0	0	0	0	1	1	1	0
CMA	0	0	0	0	0	0	0	0
INCA	0	0	0	0	0	0	1	0
DCRA	0	0	0	0	0	1	0	0
HLT	0	0	0	0	0	1	1	0

Note:

TC: Type classifier (if I₃ = 1, then it is a MRI; otherwise it is a NMRI)

GN: Group number within a type
 (I₂ I₁ Group no.
 0 0 0
 0 1 1
 1 0 2
 1 1 3)

SC: Subcategory within a group

Figure 4.40 Op-code Encoding Logic

The hardware organization of the microprogrammed control unit for this situation shown in Figure 4.43 directly follows the symbolic listing shown in Figure 4.42. No attempt has been made toward arriving at a minimal microprogram. Rather, the concept was presented. The task of translating the symbolic microprogram of Figure 4.42 into a binary microprogram is left as an exercise.

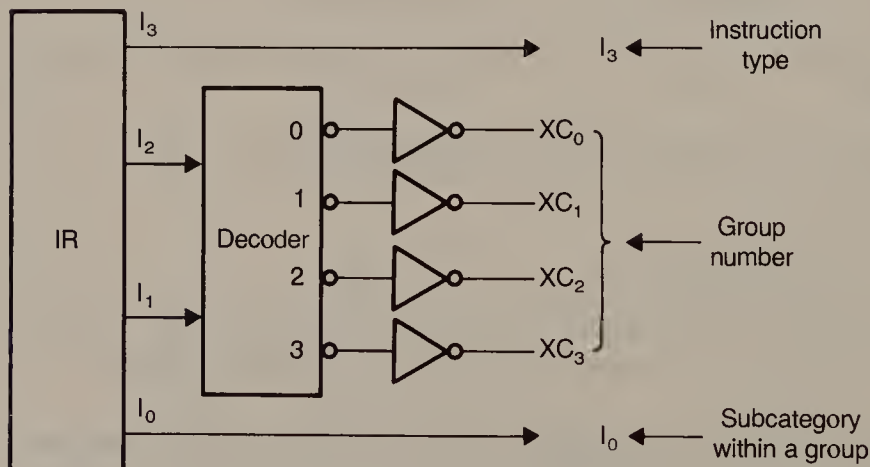


Figure 4.41 Instruction-decoding Hardware

Symbolic Microprogram:

ROM Address		
0	$PC \leftarrow 0;$	These operations constitute the fetch cycle.
1	FETCH $MAR \leftarrow PC;$	
2	$IR \leftarrow M((MAR)), PC \leftarrow PC + 1;$	
3	IF $I_3 = 1$ then go to MEMREF;	
4	IF $XC0 = 1$ then go to CMA;	Here we decode the instructions.
5	IF $XC1 = 1$ then go to INCA;	
6	IF $XC2 = 1$ then go to DCRA;	
7	Go to HALT;	
8	CMA $ACC \leftarrow ACC'$	Execute CMA instructions.
9	Go to FETCH;	
10	INCA $ACC \leftarrow ACC + 1$	Execute INCA instruction.
11	Go to FETCH;	
12	DCRA $ACC \leftarrow ACC - 1;$ then go to LDSTO;	Execute DCRA instruction.
13	Go to FETCH;	
14	MEMREF IF $XC0 = 1$ then go to LDSTO;	Here we branch to the various groups of the memory reference instruction.
15	IF $XC1 = 1$ then go to ADSUB;	
16	IF $XC2 = 1$ then go to JMPS;	
17	AND $MAR \leftarrow PC;$	
18	$BUFFER \leftarrow M((MAR)), PC \leftarrow PC + 1;$	Execute AND instruction.
19	$MAR \leftarrow BUFFER$	
20	$BUFFER \leftarrow M((MAR));$	
21	$ACC \leftarrow ACC \wedge BUFFER;$	
22	Go to FETCH;	
23	LDSTO $MAR \leftarrow PC;$	
24	$BUFFER \leftarrow M((MAR)), PC \leftarrow PC + 1;$	
25	$MAR \leftarrow BUFFER;$	
26	IF $I0 = 1$ then go to STO	
27	LOAD $BUFFER \leftarrow M((MAR));$	
28	$ACC \leftarrow BUFFER$	
29	Go to FETCH:	
30	STO $M((MAR)) \leftarrow ACC;$	
31	Go to FETCH;	

Figure 4.42 Symbolic Microprogram that Implements the Instruction Set Shown in Figure 4.39 on the Hardware Shown in Figure 4.38 (*Continues*)

32	ADSUB	MAR $\leftarrow$ PC;	
33		BUFFER $\leftarrow$ M ((MAR)), PC $\leftarrow$ PC + 1	
34		MAR $\leftarrow$ BUFFER	
35		BUFFER $\leftarrow$ M ((MAR));	
36		IF I0 = 1 then go to SUB;	
37	ADD	ACC $\leftarrow$ ACC + BUFFER;	Execute ADD instruction
38		Go to FETCH;	
39	SUB	ACC $\leftarrow$ ACC - BUFFER;	Execute SUB instruction
40		Go to FETCH;	
41	JMPS	MAR $\leftarrow$ PC;	
42			
43		IF I0 = 1 then go to JOC;	
44	JOZ	IF Z = 1 then go to LOADPC;	Execute JZ instruction
45		Go to FETCH;	
46	JOC	IF C = 1 then go to LOADPC;	Execute JC instruction
47		Go to FETCH:	
48		LOADPC	PC $\leftarrow$ M((MAR))
49		Go to FETCH;	
50	HALT	Go to HALT;	Execute HALT instruction

Figure 4.42 Continued

The microprogramming approach is systematic, flexible, and less error-prone, as is evident from these examples. Advances in IC technology have made LSI designers think of a general solution for implementing a microprogrammed CPU. A microprogrammed CPU has two major activities to be performed:

1. Fetching and interpreting microinstructions.
2. Generating the next address of the microinstruction to be retrieved.

The first task is assumed by the control memory and the associated circuit elements.

Designers have replaced the next address generation of a microprogrammed control unit with a single LSI component called a *microprogram sequencer*, which checks certain bits in the microinstruction and finds the next address for the control memory. The sequencer contains a microprogrammed counter (MPC) and circuit elements necessary to perform functions such as address incrementing, address sequencing for subroutine calls, returns, and conditional branching. At the present time, many microprogrammed control units manufactured use the following components:

- A control memory (ROM or RAM)
- A microprogram sequencer

The general organization of a microprogrammed control unit constructed using a microprogram sequencer is shown in Figure 4.44. The microinstruction in this figure is assumed to have the following format:

Condition select field	Branch-type field	Branch address field	Control function field
------------------------	-------------------	----------------------	------------------------

An additional field called *branch type field* (BT) has been included. The BT generates the next address, a 3-bit field (B2 B1 B0). Its interpretation is summarized as follows:

B2 B1 B0

- | | |
|-----|--|
| 000 | No branching and, therefore, the next address is $MPC + 1$. |
| 001 | Branch to the address specified in the branch address field if the condition selected by the condition select field is true. The MPC is loaded with the contents of the branch address field. |
| 010 | Branch to the subroutine if the condition is met. Here, the return address is saved in a subroutine-return address register (SRAR). MPC is loaded with the subroutine's address, or MPC is loaded with $MPC + 1$. The single register SRAR is replaced with a set of registers to form a stack, if it is desired to implement nested or recursive subroutine calls. |
| 011 | Unconditional branch to subroutine loads the MPC in the subroutine's address, and the return address is saved in SRAR. |
| 100 | Unconditional branch loads the MPC with the contents of the branch address field. |
| 101 | Conditional return from subroutine loads the MPC with SRAR, if the condition is met; otherwise, it is loaded with $MPC + 1$. |
| 110 | Unconditional return from subroutine. Always loads the MPC with the return address saved in SRAR. |

The MPC is loaded with one of these sources:

- $MPC + 1$ computed by the parallel adder
- Branch address field A_1A_0 of the microinstruction
- SRAR

As shown from the branch-type field B2 B1 B0 listed before, the SRAR is loaded only when

$$B'_2 B_1 B'_0 Z_1 + B'_2 B_1 B_0 \text{ is } 1$$

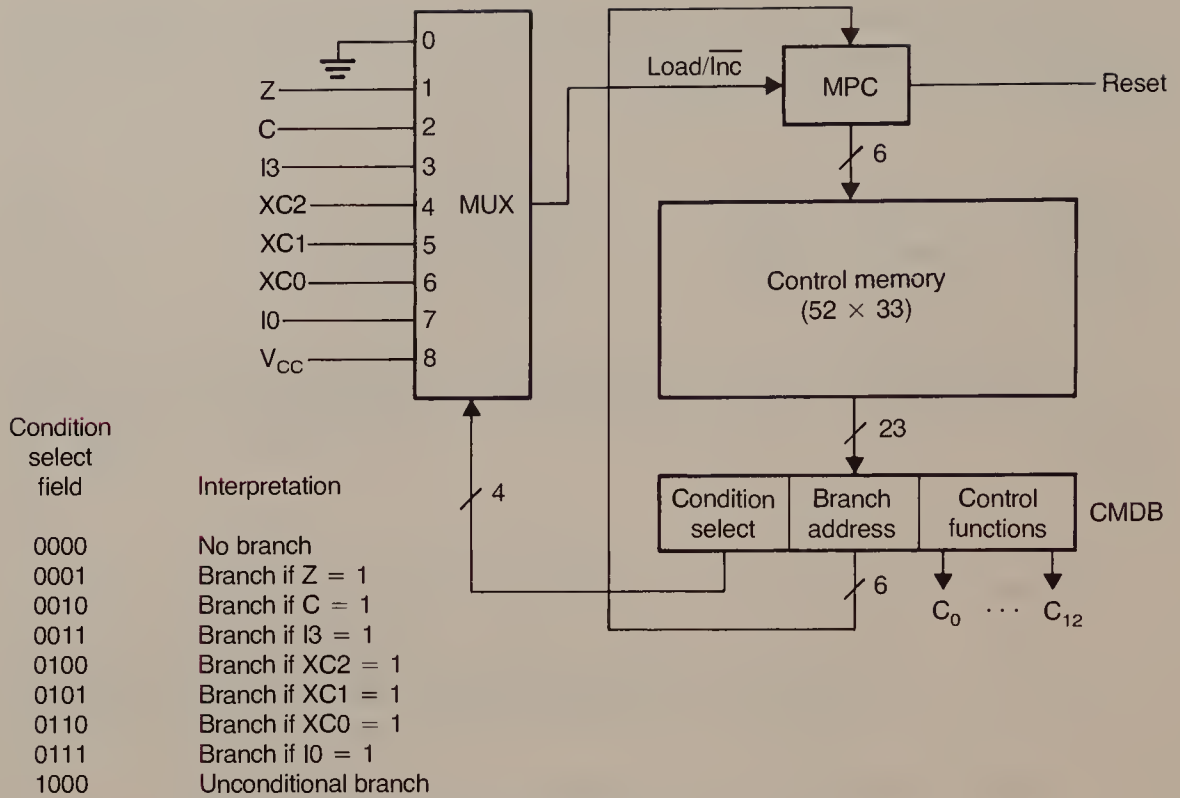


Figure 4.43 Microprogrammed Controller for the CPU

This Boolean equation is true when there is an unconditional branch to the subroutine or when a branch to a subroutine with the selected condition is true.

In the preceding equation, Z_1 refers to the output of the MUX1. This equation can also be simplified as follows:

$$\begin{aligned}
 & B_2' B_1 B_0' Z_1 + B_2' B_1 B_0 \\
 &= B_2' B_1 (B_0' Z_1 + B_0) \\
 &= B_2 B_1 [(B_0 + B_0') (B_0 + Z_1)] && \text{Since } yz + x = (x + y) (x + Z) \\
 &= B_2' B_1 (B_0 + Z_1) && \text{Since } B_0 + B_0' = 1
 \end{aligned}$$

This hardware implementation has a 2-bit microprogram counter and three multiplexers, MUX1, MUX2, and MUX3. The MUX1 selects the desired condition, and multiplexers MUX2 and MUX3 select the low- and high-order address bits, respectively. First, the control word is fetched to the CMDB. Depending on the value of the BT, the next address is computed and loaded into the MPC. Consider some specific numeric examples:

IF $B_2 B_1 B_0 = 000$, line 0 of MUX2 and MUX3 are selected. Since these lines are outputs of the 2-bit parallel adder, outputs of the multiplexer MUX2 and MUX3 transfer the value $MPC + 1$ into the MPC.

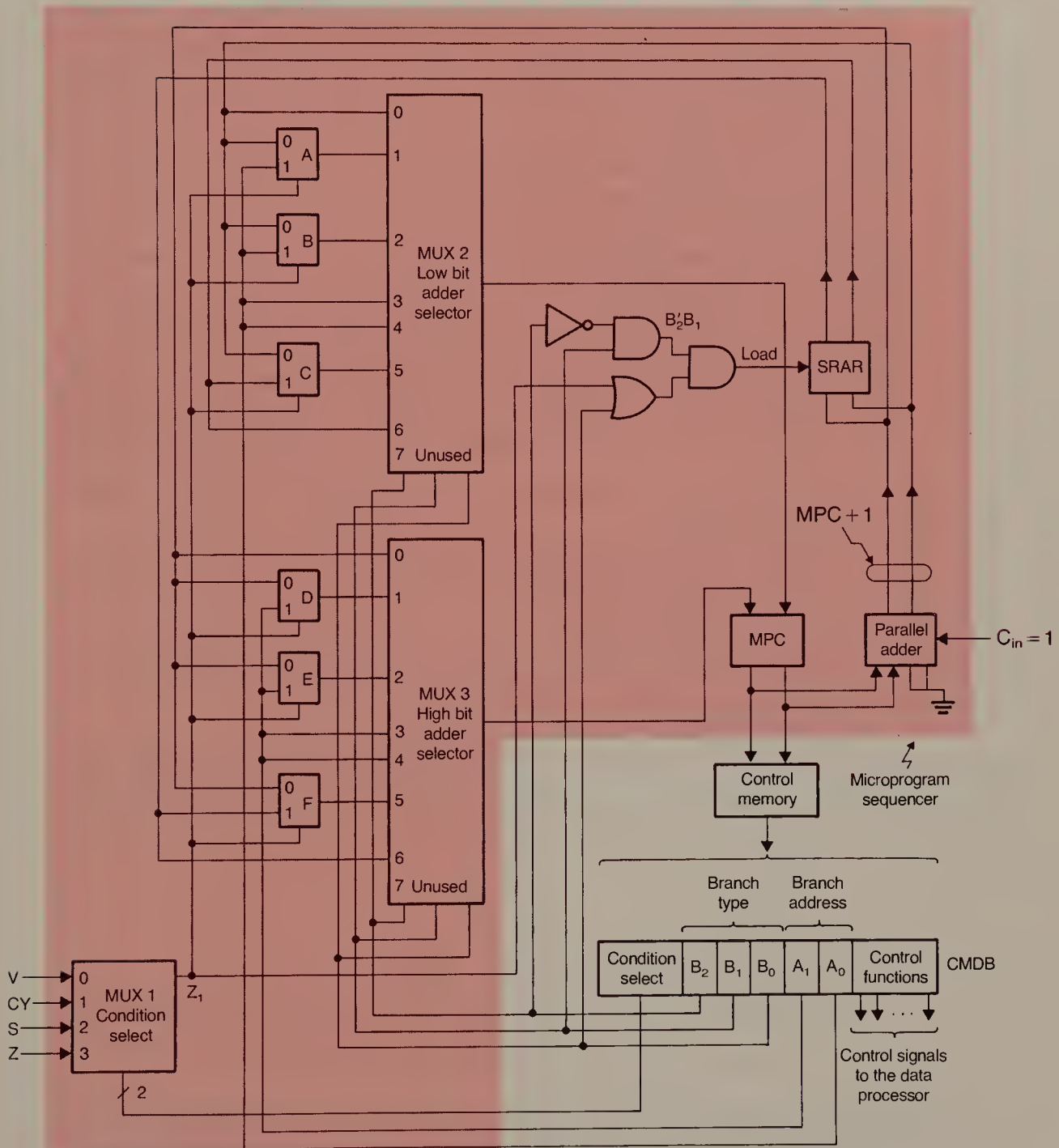


Figure 4.44 Structure of a Microprogrammed CPU Using Microprogram Sequencer

Assume $B_2B_1B_0 = 001$ and the contents of the condition select field are 01. The binary pattern of the BT selects line 1 of the multiplexers MUX2 and MUX3. These lines are outputs of multiplexers A and D. These two multiplexers are controlled by output Z_1 of the condition-select multiplexer (MUX1). Line 0 of the A and D multiplexers are connected to the output of the 2-bit parallel adder with a value of $MPC + 1$. Line 1 of the A and D multiplexers are connected to branch address A_1A_0 . The condition select field selects the CY flag. Therefore, if $Cy = 1$, then $Z_1 = 1$. Line 1 of the A and D multiplexers are selected, and branch address A_1A_0 is transferred to the MPC. However, if $Cy = 0$, the MPC is loaded with $MPC + 1$.

If the contents of the BT and the condition-select field are 010 and 10, respectively, line 2 of the multiplexers MUX2 and MUX3 are selected. These lines are outputs of multiplexers B and D; both of these multiplexers are controlled by output Z_1 of the MUX1. Therefore, if S-flag = 1, lines B and D are selected, and MPC is loaded with the starting address of the subroutine specified in A_1A_0 . The SRAR is simultaneously loaded with the return address $MPC + 1$ (since $B_2' B_1(B_0 + Z_1)$ is true). If S-flag = 0, however, the MPC is loaded with $MPC + 1$. The SRAR remains unaffected (because $B_0Z_1 = 00$).

The other operating modes of this circuit can be explained in a similar manner. This task is left as an exercise.

The microprogram sequencer described so far is simple in structure. A practical LSI sequencer offers more attractive features. To illustrate this, consider the popular microprogram sequencer, the AM 2909.

The AM 2909 is a 4-bit microprogram sequencer manufactured by Advanced Microdevices Incorporated. This LSI device can generate a 4-bit address. Its block diagram schematic is shown in Figure 4.45.

The 4-bit address generated by this sequencer appears at output bus y . The next address may originate from 4 different sources. The desired source is selected by the MUX. The MUX is controlled by the 2-bit selection inputs s_1 and s_0 . The truth table describing this selection process is specified as shown:

s_1	s_0	Selected source
0	0	MPC
0	1	External address stored in the register R
1	0	STACK
1	1	External address directly specified in the input bus D

The MPC is a 4-bit register that includes a combinational logic circuit called the *incrementer*. The purpose of the incrementer is to compute $y + C_{in}$. The MPC receives this quantity as its input after every clock pulse. The address of a subroutine is usually specified in the external 4-bit register R or on the 4-bit external input bus D. The STACK contains a 4×4 register file and an up/down counter (used as the SP). The purpose of the stack is to implement subroutine calls. Typically, when $\overline{SE} = 0$ and $POP/PUSH = 1$, the contents of the MPC are pushed into the stack. Similarly, when

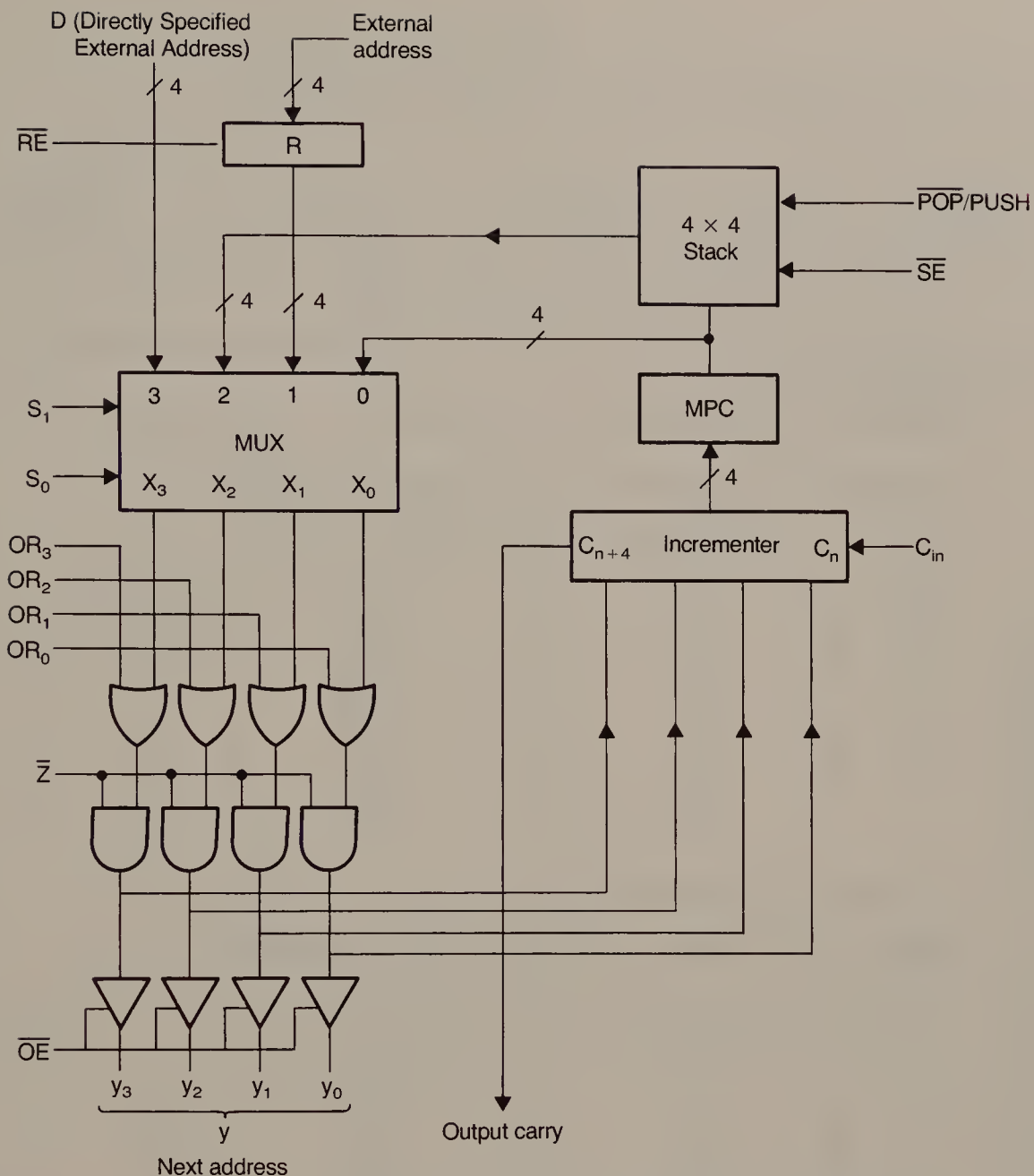


Figure 4.45 Block diagram of Am 2909 Microprogram Sequencer (From J. P. Hayes, *Digital System Design and Microcomputers*, McGraw-Hill, 1984, p. 709. Reprinted by permission of McGraw-Hill)

$\overline{\text{SE}} = 0$ and $\overline{\text{POP/PUSH}} = 0$, the contents of the stack word appear at MUX input 2. The MUX control inputs s_1 and s_0 are part of the computer's microinstruction.

The purpose of the $\bar{Z}$ input is to implement the external reset mechanism. When an external circuit drives this input low, the generated address is a 0. The four inputs (OR₃ through OR₀) along with the four 2-inputs are used to implement multiway conditional branching. Assume OR₂ = OR₁ = OR₀ = 0 and OR₃ = c is some external condition. Two-way branching is formed as shown next:

if $c = 0$ then $y_3y_2y_1y_0 = x_3x_2x_1x_0$
 else $y_3y_2y_1y_0 = 1x_2x_1x_0$

Assume the following microinstruction format in the sequencer:

Source Field		Stack Field		R-REG Field	OR Field				Zero Field
s_1	s_0	$\overline{SE}$	$\overline{POP/PUSH}$	$\overline{RE}$	OR_3	OR_2	OR_1	OR_0	Z

This format will be used to describe some typical microinstructions. Consider the following microinstruction:

Branch to the address specified in the R register

The control word corresponding to this situation is shown next:

s_1	s_0	$\overline{SE}$	$\overline{POP/PUSH}$	$\overline{RE}$	OR_3	OR_2	OR_1	OR_0	$\overline{Z}$
0	1	1	X	0	0	0	0	0	1
Select R register.		Disable the stack.	This is a don't care.	Enable the register R.	Set all OR-inputs to produce a null effect.				Set Z to produce a null effect.

Now consider the following instruction:

Branch to the subroutine whose address is specified on the input bus D.

The control word corresponding to this instruction is shown next:

s_1	s_0	$\overline{SE}$	$\overline{POP/PUSH}$	$\overline{RE}$	OR_3	OR_2	OR_1	OR_0	$\overline{Z}$
1	1	0	1	1	0	0	0	0	1
Select the input bus.		Enable the stack.	Push the return address.	Disable the register R.	Set the OR inputs to produce a null effect.				Set the Z input to produce a null effect.

Figure 4.46 shows the interconnection of three AM 2909s to form a 12-bit sequencer, which will expand the sequencer. The only interconnection between packages other than the control lines is the ripple carry between the MPC and the incrementers.

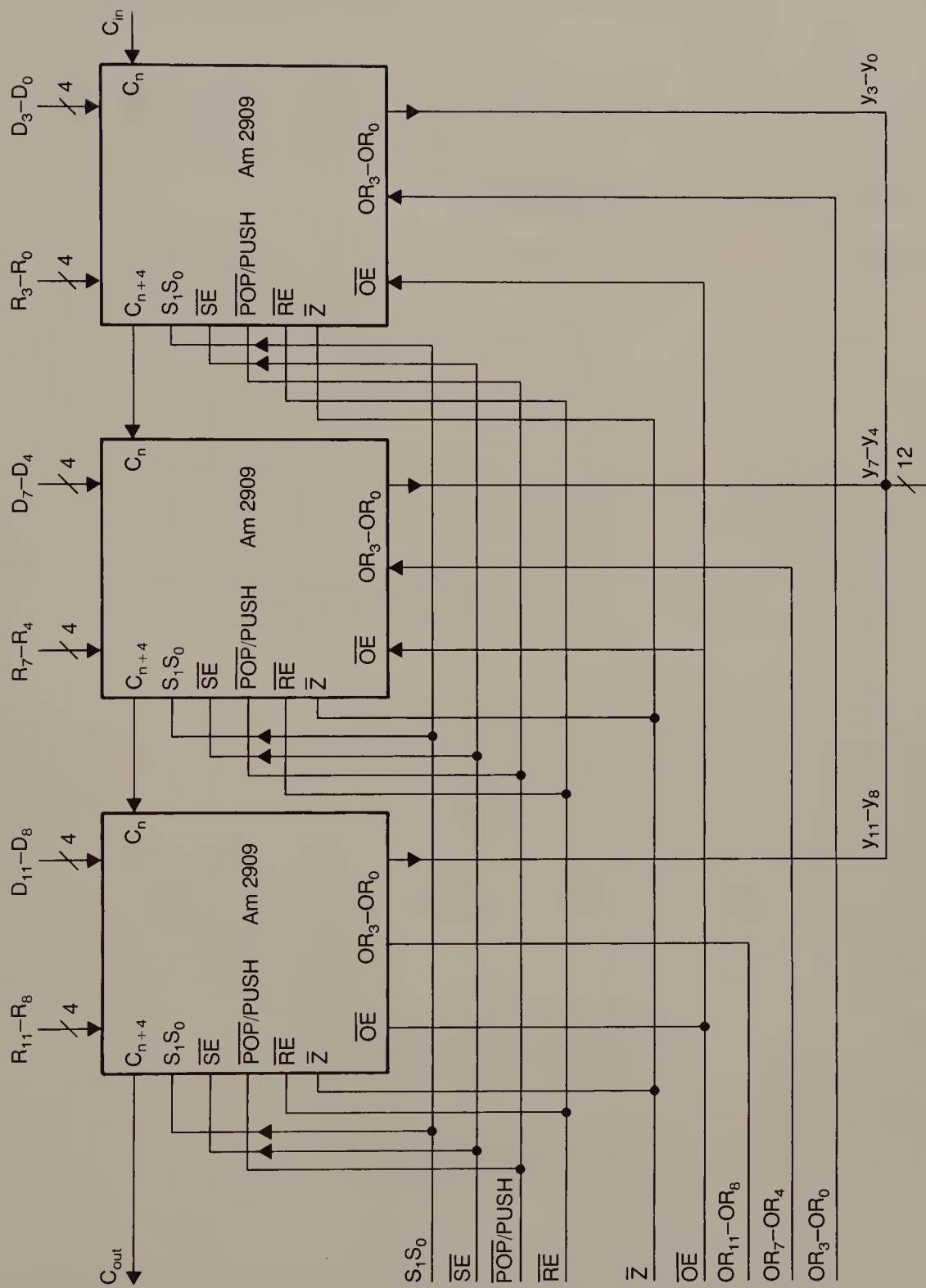


Figure 4.46 A 12-bit Microprogram Sequencer (From J. P. Hayes, *Digital System Design and Microprocessors*, McGraw-Hill, 1984, p. 711. Reprinted by permission of McGraw-Hill)

A complete computer system can be designed using bit-slice devices such as the AM 2901 and AM 2909. Bit-slice components offer flexibility for a microprogrammed implementation. The salient characteristics of bit-slice-device-based computer philosophy are summarized in Figure 4.47.

Most bit-slice systems use the mapping PROM concept to generate the address of the microinstructions. The contents of IR are applied as inputs to a mapping PROM. The PROM generates the address of the first microinstruction, which must be executed to perform the required function. Consider the following four microinstructions along with associated op-codes and starting addresses:

MICROINSTRUCTION	OP-CODE	STARTING ADDRESS OF THE FIRST MICROINSTRUCTION
	X1 X0	
MO	O O	8
M1	O 1	12
M2	1 O	24
M3	1 1	28

The PROM is essentially a combinational circuit that is governed by the following truth table:

Design Criterion	Remarks
Architecture	Somewhat flexible
Component count	50 (typical)
Word length	Multiples of 2 or 4
Operating speed (cycle time)	100–200 ns
Design pocess	Fast
Debugging effort	Relatively simple if a development system is available
Major design technique	Firmware
Expandability	Can be achieved very easily
Reliability	Higher than the MSI-chip-based computer
Total cost	Medium range

Figure 4.47 Characteristics of the Bit-slice-devices-based Computer Design Philosophy

X1	X0	MPC4	MPC3	MPC2	MPC1	MPC0
0	0	0	1	0	0	0
0	1	0	1	1	0	0
1	0	1	1	0	0	0
1	1	1	1	1	0	0

A typical block diagram of a system and a 256×40 control PROM is shown in Figure 4.48.

A smaller control store is more economically efficient. The following methods are employed to optimize size:

1. Reduce the length of the microinstruction.
2. Reduce the length of the microprogram.
3. Use a second-level control called *nanomemory*.

Reduction of Microinstruction Length

The length of a microinstruction can be reduced by encoding the control function field. In 1971, S. R. Das and others [5] proposed a formal approach to encode the

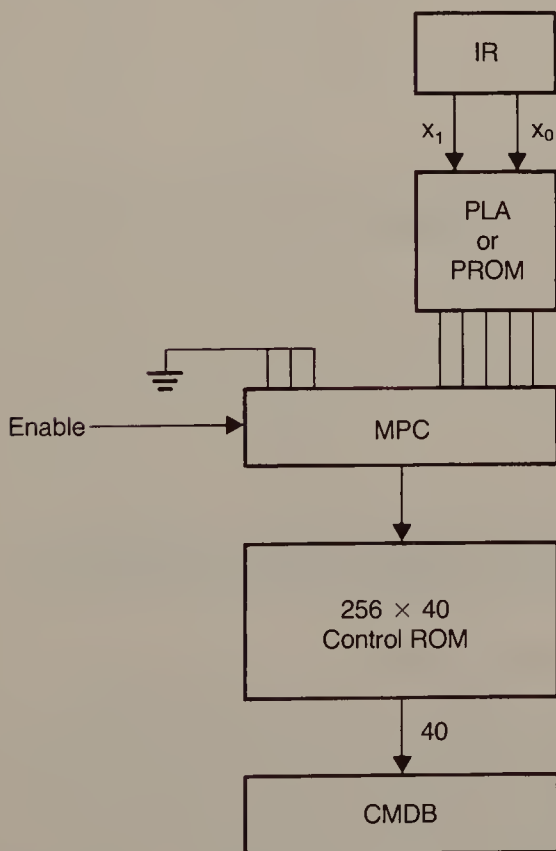


Figure 4.48 Control Organization Using a Mapping PROM

control function field so the storage requirements are minimized. The primary advantage of this method is that it can be computerized. Nevertheless, this algorithm arbitrarily groups the control functions to be encoded, which increases the effort required to write microprograms.

To overcome this difficulty, a new approach is presented to encode the control functions in a logical manner. The control functions confined to a particular functional unit are first grouped together. They are then encoded. This approach is referred to as the *encoding by function* technique. To illustrate this idea, the following example is provided. Consider the following control functions:

C0: $ACC \leftarrow ACC + R$

C1 $\wedge$ C2: $ACC \leftarrow ACC - R$

C3 $\wedge$ C4: $ACC \leftarrow 0$

C5: $R \leftarrow 0$

C6 $\wedge$ C7 $\wedge$ C8: $R \leftarrow R + 1$

C9 $\wedge$ C10: $R \leftarrow R - 1$

C11 $\wedge$ C12: $R \leftarrow INBUS$

C13 $\wedge$ C14: $ACC \leftarrow INBUS$

C15 $\wedge$ C16: $Outbus \leftarrow ACC$

C17 $\wedge$ C18: $Outbus \leftarrow R$

In the above description, assume that ACC is the accumulator and R is the CPU register. The following functional units need to be considered in this situation:

- Accumulator unit
- Count unit (register R)
- Input bus
- Output bus

Therefore, the control function field is divided into four subfields, as follows:

Control Function Field							
ACC Field		Count Field		Input Bus Field		Output Bus Field	
A ₁	A ₀	B ₁	B ₀	I ₁	I ₀	D ₁	D ₀

Each subfield is encoded as shown next:

$A_1 A_0$	ACTION TAKEN	CONTROL SIGNAL ACTIVATED
0 0	No operation	None
0 1	$ACC \leftarrow ACC + R$	C0
1 0	$ACC \leftarrow ACC - R$	C1, C2
1 1	$ACC \leftarrow 0$	C3, C4

$B_1 B_0$	ACTION TAKEN	CONTROL SIGNAL ACTIVATED
0 0	No operation	None
0 1	$R \leftarrow 0$	C5
1 0	$R \leftarrow R + 1$	C6, C7, C8
1 1	$R \leftarrow R - 1$	C9, C10

$I_1 I_0$	ACTION TAKEN	CONTROL SIGNAL ACTIVATED
0 0	No operation	None
0 1	$R \leftarrow \text{inbus}$	C11, C12
1 0	$\text{Acc} \leftarrow \text{inbus}$	C13, C14
1 1	Unused	None

$D_1 D_0$	ACTION TAKEN	CONTROL SIGNAL ACTIVATED
0 0	No operation	None
0 1	$\text{Outbus} \leftarrow \text{ACC}$	C15, C16
1 0	$\text{Outbus} \leftarrow R$	C17, C18
1 1	Unused	None

Note that the length of the control function field is only 8 bits. If a fully unencoded format is used, the length of the control function field is 19 bits.

Reduction of the Length of a Microprogram

By examining the microprograms so far, it can be seen that in the event of branch instructions, the control function field is merely filled with 0s.

Whenever there are more branch instructions, the microinstruction is formatted by using a new approach: the multiple microinstruction format.

The microinstructions are divided into two groups: operate and branch instructions.

An operate instruction initiates one or more microoperations. For example, after the execution of an operate instruction, the MPC will be incremented by 1. In the case of a branch instruction, no microoperation will usually be initiated, and the MPC may be loaded with a new value.

This suggests the removal of the branch address field from the microinstruction format, so the control function field is used to specify the branch address itself. Typically, each microinstruction will have two fields, as shown next:

CONDITION- SELECT FIELD		CONTROL FUNCTION FIELD					
S_1	S_0	C_5	C_4	C_3	C_2	C_1	C_0

If $S_1 S_0 = 00$, the microinstruction is considered as an operate instruction, and the contents of the control function field are treated as the control signals. Assume the Condition Select Field is encoded as follows:

$S_1 S_0$

0 0 No branch

0 1 Branch if cond-1 = 1

1 0 Branch if cond-2 = 1

1 1 Unconditional branch

If $S_1 S_0 = 01$, the instruction is regarded as a branch instruction, and the contents of the control field are assumed to be a 6-bit branch address. When $S_1 S_0 = 01$, the MPC will be loaded with $(C_5 C_4 C_3 C_2 C_1 C_0)$ if Cond-1 is met.

To illustrate this, Booth's multiplier example is reconsidered. The binary microprogram will be rewritten using the multiple microinstruction instruction format, as shown in Figure 4.49.

From Figure 4.49 it can be seen that the total size of the control store is 169 bits ($13 \times 13 = 169$). For the first two microinstructions, the contents of the condition select field are 0 and are considered as operation instructions. In this case, the contents of the control function field are directed to the processing hardware.

The third microinstruction is a branch instruction, since the contents of the condition select field are not 0s. Here, the 13-bit control function field directly specifies the desired branch address ($00000\ 0000\ 0101_2 = 5_{10}$).

The hardware organization that implements the microcode of Figure 4.49 is shown in Figure 4.50.

Nanomemory

Nanomemory is another approach for reducing the size of the control memory. This technique involves a two-level memory: control memory and nanomemory. At the

ROM address	Control Word												Comments	
	Condition Selection Field	Control Function/Branch Address Field										Instruction Type	Operation Performed	
		c ₉	c ₈	c ₇	c ₆	c ₅	c ₄	c ₃ / b _{r3}	c ₂ / b _{r2}	c ₁ / b _{r1}	c ₀ / b _{r0}			
0000	000	0	0	0	0	0	0	0	1	1	1	Operate	A ← 0, M ← Inbus L ← 4	
0001	000	0	0	0	0	0	0	1	0	0	0	Operate	Q [4:1] ← Inbus, Q [0] ← 0	
0010	001	0	0	0	0	0	0	0	1	0	1	Branch	If Q [1:0] = 0 Then go to 5 (ADD)	
0011	010	0	0	0	0	0	0	0	1	1	1	Branch	If Q [1:0] = 10 Then go to 7 (SUB)	
0100	100	0	0	0	0	0	0	1	0	0	0	Branch	Go to 8 (RSHIFT)	
0101	000	0	0	0	0	1	1	0	0	0	0	Operate	A ← A + M	
0110	100	0	0	0	0	0	0	1	0	0	0	Branch	Go to 8 (RSHIFT)	
0111	000	0	0	0	0	1	0	0	0	0	0	Operate	A ← A - M	
1000	000	0	0	1	1	0	0	0	0	0	0	Operate	ASR (A\$Q), L ← L - 1	
1001	011	0	0	0	0	0	0	0	0	1	0	Branch	If Z = 0 Then go to 2 (LOOP)	
1010	000	0	1	0	0	0	0	0	0	0	0	Operate	Outbus = A	
1011	000	1	0	0	0	0	0	0	0	0	0	Operate	Outbus = Q [4:1]	
1100	100	0	0	0	0	0	0	1	1	0	0	Branch	Go to 12 (HALT)	

Figure 4.49 Binary Listing of the 4 x 4 Booth's Multiplication Microprogram (in Multiple Microinstruction Format)

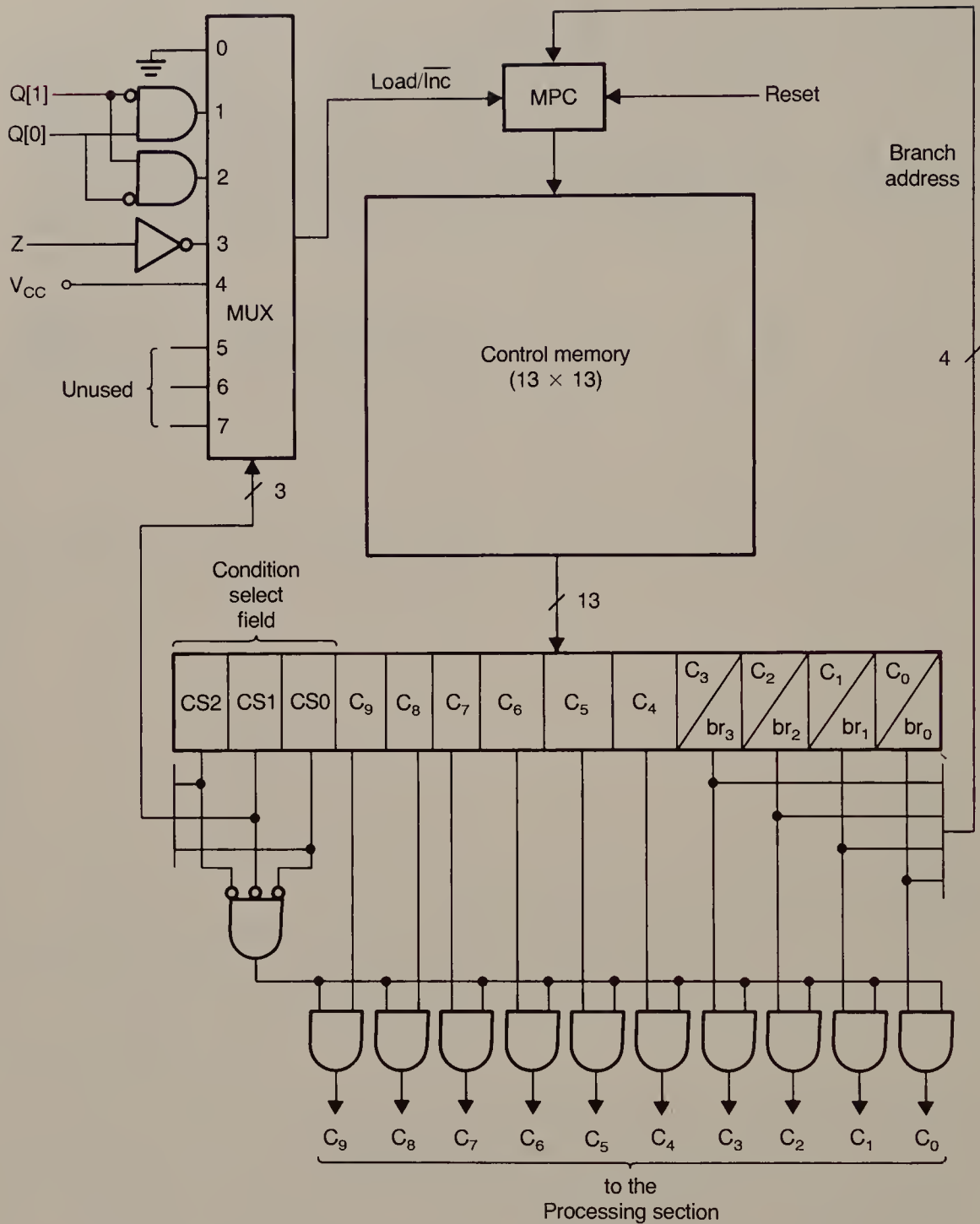


Figure 4.50 Organization of the Microprogrammed Controller for Implementing the Microcode Shown in Figure 5.49

outset, one may feel that two-level structure will increase the overall cost, but it actually improves the economy of the system by reducing the total space required.

The concept of nanomemory is derived from a combination of horizontal and vertical microinstructions. This method, in fact, provides useful trade-offs between these two disciplines. Nanomemory offers significant savings in space when a group of microoperations occur many times in a microprogram.

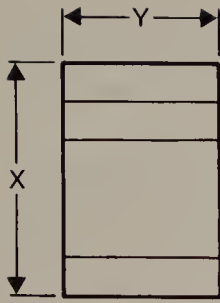


Figure 4.51 A Microprogram of Size $X \times Y$

Consider the microprogram of Figure 4.51. The microprogram in this figure consists of X microinstructions Y bits wide.

The size of the control memory to store this microprogram is XY bits. Assume this microprogram has p ($p < X$) unique microinstructions. These p microinstructions can be held in a separate memory called the *nanomemory* of size pY bits. These p instructions occur once with the nanomemory. Each microinstruction in the original microprogram is replaced with an address that specifies the location of the nanomemory in which the original Y -bit-wide microinstruction is held. Since the nanomemory has p addresses, only $\lceil \log_2 p \rceil$ (rounded up to the next integer) bits are needed to specify one nanomemory address. Therefore, the size of each microinstruction in a two-level control store is only $\lceil \log_2 p \rceil$ bits. This is illustrated in Figure 4.52.

The operation of a control unit employing a two-level store can be explained as follows. The first word from the microprogram is read. This word is actually the first address of the nanomemory. The contents of this location in the nanomemory is the control word, which is transferred to the CMDDB. The bits in this register are then used to control the appropriate gates for one cycle. Upon completion of the cycle, the second word is read from the control memory and the process continues. Note that a control unit using a nanomemory control store is slower than one using only a conventional control memory, since the nanomemory concept consists of two-level memory. Therefore, two memory fetches (one for the control memory and the other for the nanomemory) are required. The conventional control memory consists of a single memory; thus,

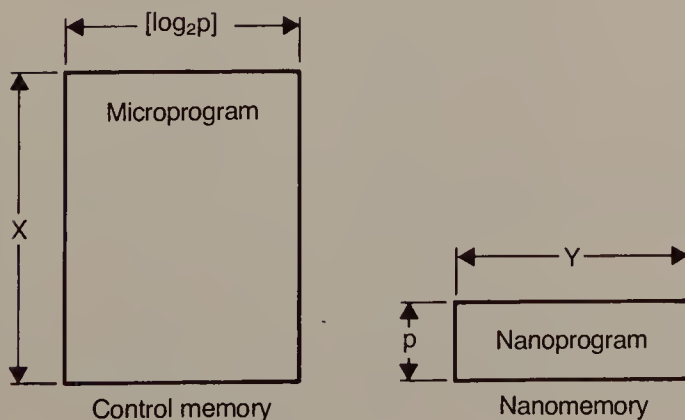


Figure 4.52 Nanomemory Concept

one memory fetch is necessary. This reduction in control unit speed is offset by the cost of the memory when the same microinstructions occur many times in the microprogram.

Consider the 14×8 bit microprogram stored in the single control memory of Figure 4.53.

This microprogram is implemented using a nanomemory, and an investigation is then carried out to determine which technique is appropriate to implement the microprogram. The program may not serve any practical purpose, but it is presented to show the rationale behind the nanomemory concept.

In this program, 5 out of 14 microinstructions are unique. Therefore, the size of the microcontrol store is $14 \times \lceil \log_2 5 \rceil$, or 14×3 , bits, and the size of the nanomemory is 5×8 bits. This is illustrated in Figure 4.54.

memory requirements for single-control store = $14 \times 8 = 112$ bits

memory requirements for nanomemory = $(14 \times 3 + 5 \times 8)$ bits
 $= 42 + 40 = 82$ bits

therefore, savings using nanomemory = $112 - 82 = 30$ bits

Hence, the use of two-level control store may be recommended.

0000	0000 0010
0001	0011 0111
0010	0010 0100
0011	0100 1010
0100	0000 0011
0101	0000 0010
0110	0011 0111
0111	0100 1010
1000	0100 1010
1001	0011 0111
1010	0100 1010
1011	0011 0111
1100	0000 0011
1101	0011 0111
1110	
1111	

Figure 4.53 14×8 -bit Single-control Store

0000	000	000	0000 0010
0001	001	001	0011 0111
0010	010	010	0010 0100
0011	011	011	0100 1010
0100	100	100	0000 0011
0101	000		
0110	001		
0111	011		
1000	011		
1001	001		
1010	011		
1011	001		
1100	100		
1101	001		

5×8 nanocontrol store

Figure 4.54 14×3 Microcontrol Store

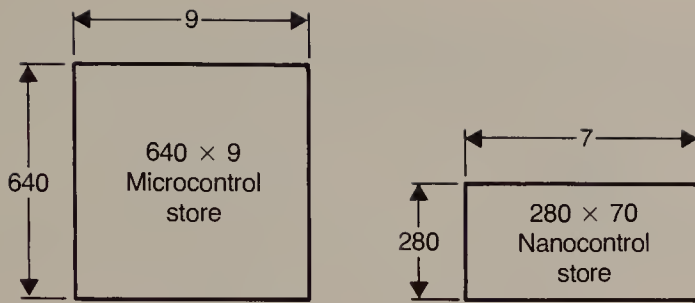


Figure 4.55 MC68000 Control Unit Structure

As a practical example of the nanomemory concept, consider the nanomemory structure of the Motorola MC68000 16-bit microprocessor in Figure 4.55.

From Figure 4.55 it can be seen that out of 640 microinstructions, 280 are unique. The contents of the microcontrol store are pointers to the nanocontrol store. Each word of the microcontrol store is $\lceil \log_2 280 \rceil = 9$ bits wide.

The MC 68000 offers control memory savings. In the MC68000, the microcontrol store is 640×9 bits, and the nanocontrol store is 280×70 bits, since there are 280 unique microinstructions. If the MC68000 is implemented by using a single CM, this memory will have 640×70 bits. Therefore,

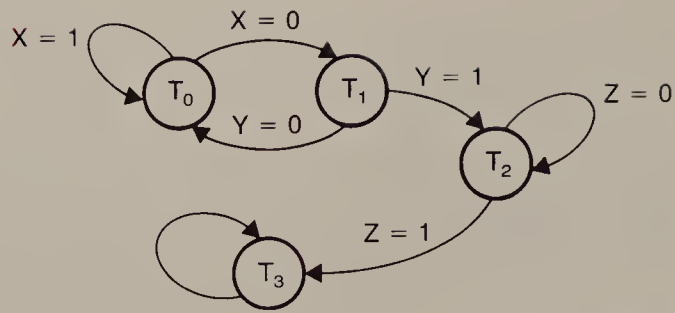
$$\begin{aligned}
 \text{memory savings} &= 640 \times 70 - (640 \times 9 + 280 \times 70) \\
 &= 44,800 - (5760 + 19,600) \\
 &= 19,440 \text{ bits}
 \end{aligned}$$

QUESTIONS AND PROBLEMS

- 4.1** Explain the functions of a control unit in a digital computer.
- 4.2** Build hardware to implement each of the following register transfers:
- a)** If X is even then $A \leftarrow B + C$
 else $A \leftarrow (B.C)'$
 - b)** If X is zero then $A \leftarrow A + 1$
 else $A \leftarrow A - 1$

Assume that A , B , C , and X are 4-bit registers.

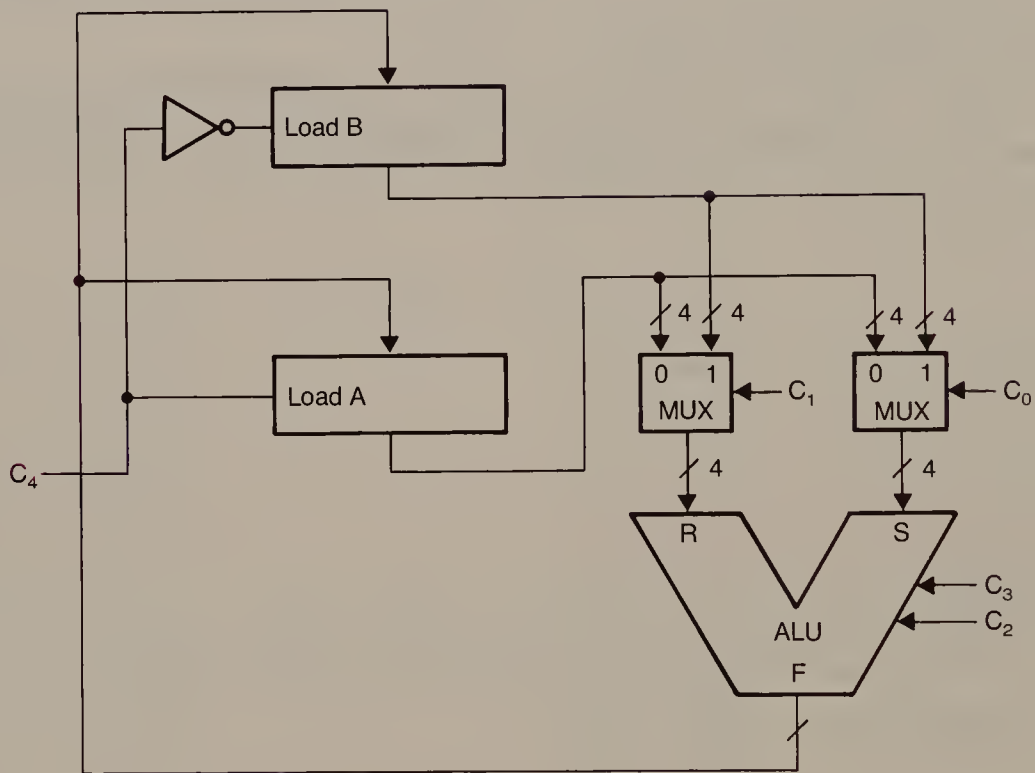
- 4.3** Design a circuit that will generate 19 timing signals.
- a)** Use a straight ring counter.
 - b)** Use a mod-32 counter and a 5-to-32 decoder.
- 4.4** A control circuit has three inputs X , Y , and Z , and its state diagram is shown next:



Design the control circuit using counter, decoder and a PLA.

4.5 Repeat problem 4.4 with D flip-flops and a PLA.

4.6 Consider the RALU shown next:



The interpretation of various control points are summarized as follows:

C_3C_2	F
0 0	R plus S
0 1	R minus S
1 0	R and S
1 1	R EX-OR S

C_1C_0	R-INPUT	S-INPUT
0 0	A	A
0 1	A	B
1 0	B	A
1 1	B	B

C_4	ACTION
0	$B \leftarrow F$
1	$A \leftarrow F$

Answer the following questions by writing suitable control word(s). Each control word must be specified according to the following format:

$C_4 C_3 C_2 C_1 C_0$

For example:

$C_4 C_3 C_2 C_1 C_0$

1 0 0 0 1 ; $A \leftarrow A \text{ plus } B$

- a) How will the A register be cleared? (Suggest at least two possible ways.)
DIRECT CLEAR input is not available.
- b) Suggest a sequence of control words that exchanges the contents of A and B registers (exchange means $A \leftarrow B$ and $B \leftarrow A$).

4.7 Consider the following algorithm:

Declare registers A [8], B [8], C [8];

START: $A \leftarrow 0$;
 $B \leftarrow 00001010$;

LOOP: $A \leftarrow A + B$;
 $B \leftarrow B - 1$;
 If $B < > 0$ then go to LOOP
 $C \leftarrow A$;

HALT: Go to HALT

Design a hardwired controller that will implement this algorithm.

4.8 It is desired to build an interface in order establish communication between a 32-bit host computer and a front end 8-bit micro computer (See figure shown below). The operation of this system is described as follows:

Step 1: First the host processor puts a high signal on the line “want” (saying that it needs a 32-bit data) for one clock period.

Step 2: The interface recognizes this by polling the want line.

Step 3: The interface unit puts a high signal on the line “fetch” for one clock period (that is it instructs the microcomputer to fetch an 8-bit data).

Step 4: In response to this, the microcomputer samples the speech signal, converts it into an 8-bit digital data and informs the interface that the data is ready by placing a high signal on the “ready” line for one clock period.

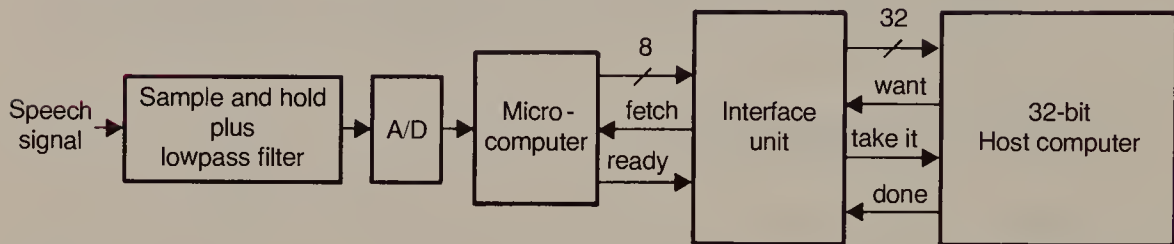
Step 5: The interface recognizes this (by polling the ready line), and it reads the 8-bit data into its internal register.

Step 6: The interface unit repeats the steps 3 through 5 for three more times (so that it acquires 32-bit data from the microcomputer).

Step 7: The interface informs the host computer that the latter can read the 32-bit data by placing a high signal on the line “takeit” for one clock period.

Step 8: The interface unit maintains a valid 32-bit data on the 32-bit output bus until the host processor says that it is done (the host puts a high signal on the line “done” for one clock period). In this case, the interface proceeds to step 1 and looks for a high on the “want” line.

- a) Provide a RTL description of the interface
- b) Design the processing section of the interface.
- c) Draw a block diagram of the interface controller.
- d) Draw state diagram of the interface controller.



4.9 Consider the following algorithm:

```

begin
    m: = 14
    q: = 5;
    i: = 1;
    s: = 0;
    While i <= m do
    begin
        j: = 1
        while j <= q do
        begin
            s: = s + 1;
            j: = j + 1
        end;
        i: = i + 1
    end
end

```

Explain the task accomplished by this algorithm.

- 4.10** Explain the significance of horizontal and vertical microinstructions.
- 4.11** Solve Problem 4.7 using the microprogrammed approach.
- 4.12** Obtain the binary listing of the symbolic microprogram shown in Figure 4.42.
- 4.13** Design a microprogrammed system to add numbers stored in the register pair AB

and CD. A, B, C, and D are 8-bit registers. The sum is to be saved in the register pair AB. Assume that only an 8-bit adder is available.

- 4.14** The goal of this problem is to design a microprogrammed 3rd order FIR (Finite impulse response) digital filter. In this system, there are 4 coefficients w_0 , w_1 , w_2 , and w_3 . The output y_k (at the k th clock period) is the discrete convolution product of the inputs (x_k s) and the filter coefficients. This is formally expressed as follows:

$$y_k = w_0 * x_k + w_1 * x_{k-1} + w_2 * x_{k-2} + w_3 * x_{k-3}$$

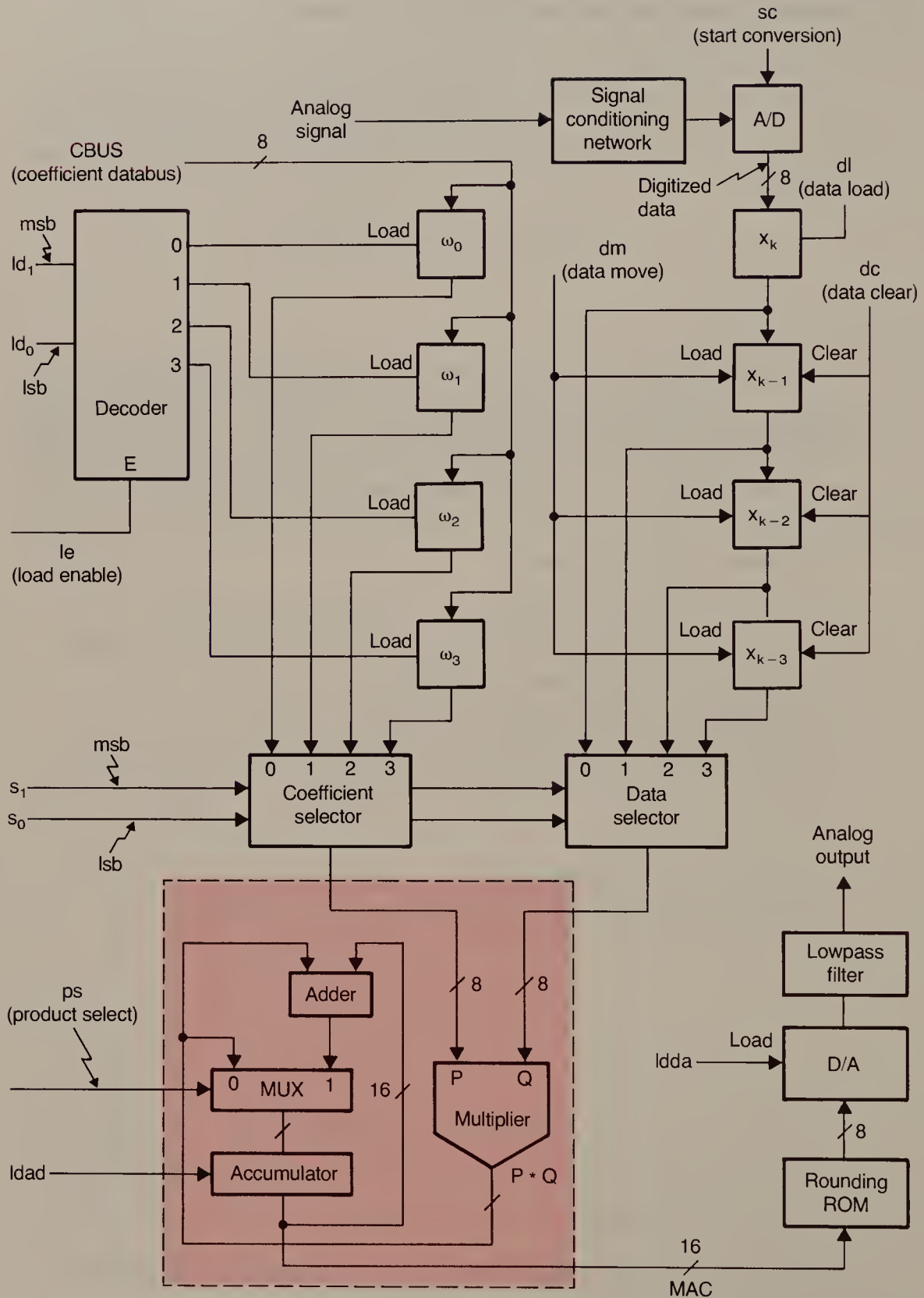
$$= \sum_{i=0}^3 w_i x_{k-i}$$

In the above summation, x_k represents the input at the k th clock period while x_{k-i} represents input at $(k-i)$ th sample period. For all practical purposes, we assume that our system is causal and so $x_i = 0$ for $i < 0$. The processing hardware is shown below. This unit includes 8 eight-bit registers (to hold data and coefficients), A/D (Analog digital converter), MAC (multiplier accumulator), and a D/A (Digital analog converter). The processing sequence is shown below:

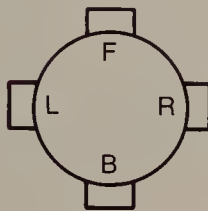
- 1 Initialize coefficient registers
- 2 Clear all data registers except x_k
- 3 Start A/D conversion (first make $sc = 1$ and then retract it to 0)
- 4 Wait for one control state (To make sure that the conversion is complete)
- 5 Read the digitized data into the register x_k
- 6 Iteratively calculate filter output y_k (use MAC for this)
- 7 Pass y_k to D/A (Pass Accumulator's output to D/A via Rounding ROM)
- 8 Move the data to reflect the time shift ($x_{k-3} := x_{k-2}$, $x_{k-2} := x_{k-1}$, $x_{k-1} := x_k$)
- 9 Go to 3

a) Specify the controller organization.

b) Produce a well documented listing of the binary microprogram



4.15 Your task is to design a microprogrammed controller for a simple robot with 4 sensors (see figure A). The sensor output will go high only if there is a wall or an obstruction within a certain distance. For example, if $F = 1$, there is an obstruction or wall in the forward direction. In particular, your controller is supposed to communicate with a motor controller unit shown in figure B. The flow chart that describes the control algorithm is shown in figure C. The outputs such as MFTS, MRT, MLT, MUT, and STP, and the status signals such as FMC, and TC will be high for one clock period. Assume that a power on reset causes the controller to go the WAIT STATE 0.



F: forward direction sensor
R: right direction sensor
L: left direction sensor
B: backward direction sensor

Figure A

a) Specify the controller organization.

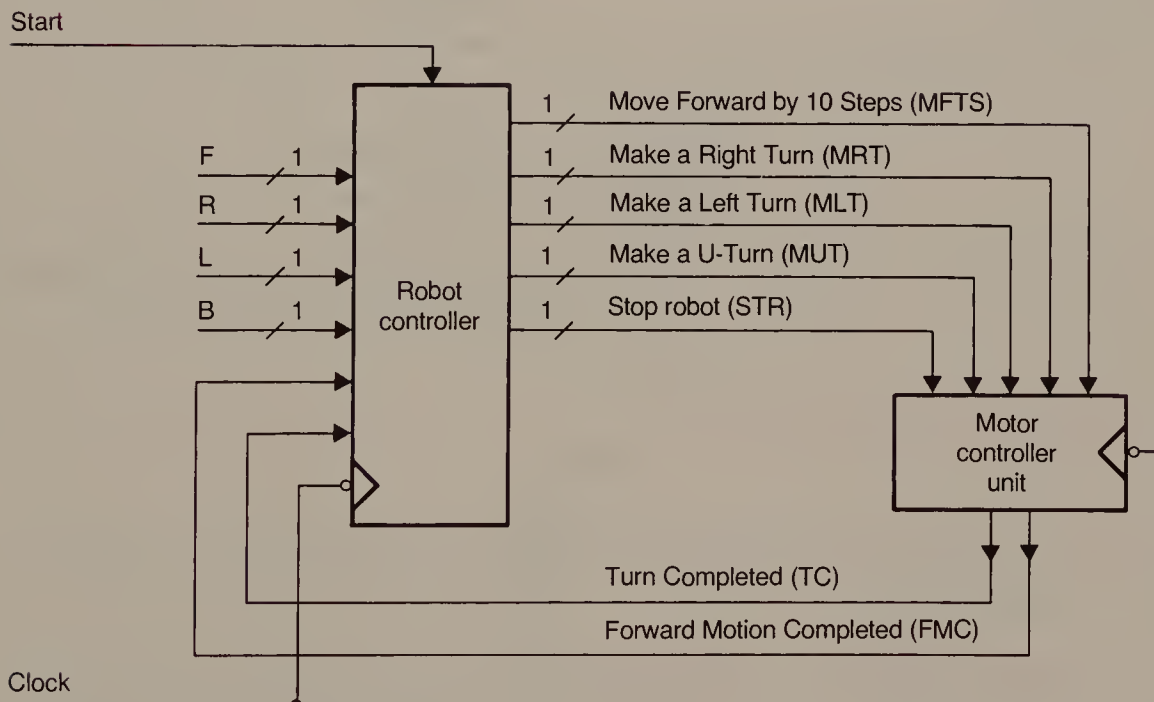


Figure B

b) Provide a well documented listing of the binary microprogram.

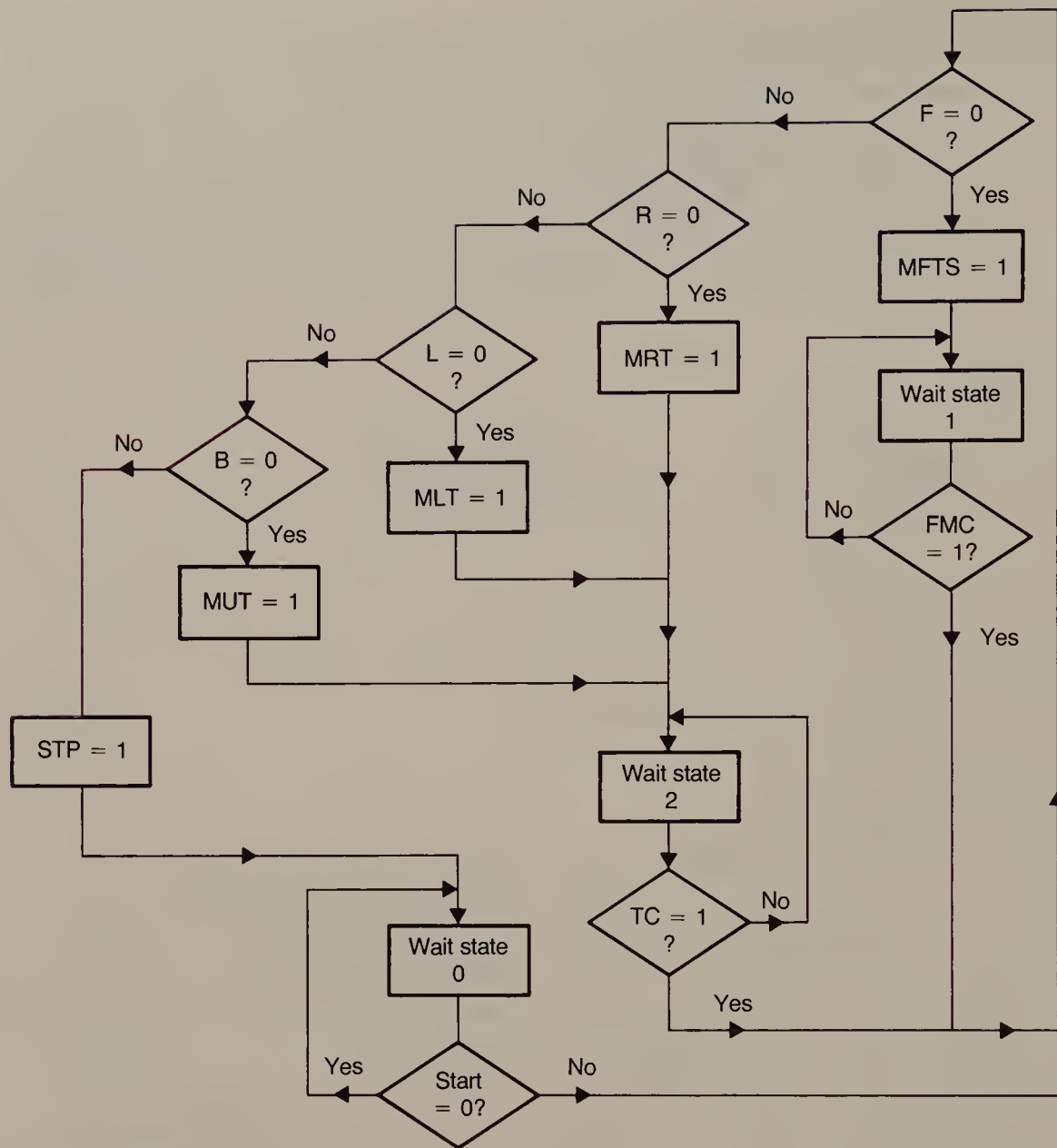


Figure C

4.16 It is desired to add the following instructions to the instruction set shown in Figure 4.39.

GENERAL FORMAT

a) MVIA <data8>

OPERATION

$A \leftarrow \langle \text{data8} \rangle$

This is an immediate mode move instruction.

The first byte contains the op-code while the second byte contains the 8-bit data.

- | | | |
|---------|-------------------------|--|
| b) CLRA | $A \leftarrow 0$ | This is a clear instruction. |
| c) PRSA | $A \leftarrow 11111111$ | This is a preset instruction |
| d) NEGA | $A \leftarrow -A$ | This instruction negates the contents of A |

Write a symbolic microprogram that describes the execution of each instruction.

- 4.17** Explain how the effect of an unconditional branch instruction of the following form is simulated:

JP ⟨addr⟩

Use the instruction set shown in Figure 4.39.

- 4.18** Using the instruction set shown in Figure 4.39, write a program to add the contents of the memory locations 64_{16} through $6D_{16}$ and save the result in the address $6E_{16}$.
- 4.19** Mention four salient features of the Am 2909 microprogram sequencer.
- 4.20** Make a 16-bit microprogram sequencer using four AM 2909s.
- 4.21** What is the significance of the OR output in the Am 2909 microprogram sequencer?
- 4.22** Obtain the necessary input configurations to perform the following functions with an Am 2909:
- a)** Branch to the specified external address (D).
 - b)** Return from a microsubroutine.
- 4.23** The op-code and starting address of the first microinstruction corresponding to eight microinstructions are specified next:

MACROINSTRUCTION	OP-CODE	STARTING ADDRESS OF THE FIRST MICROINSTRUCTION (IN DECIMAL)
M_0	000	8
M_1	001	12
M_2	010	24
M_3	011	28
M_4	100	40
M_5	101	44
M_6	110	56
M_7	111	60

- a)** Design a suitable mapping PROM that will achieve this mapping. Assume that the control memory has 4k words.

- b)** Is it possible to achieve the same result without using the mapping PROM? Assume that the control memory has 4K words.
- c)** Compare the advantages and disadvantages of the approaches used in parts (a) and (b) of this exercise.
- 4.24** What are the advantages and disadvantages of the microprogrammed approach over the hardwired approach?
- 4.25** What are the advantages and disadvantages of a two-level control structure?
- 4.26** A conventional microprogrammed control unit includes 2048 words by 117 bits. Each of 512 microinstructions is unique. Calculate the savings achieved by having a Nanomemory. Calculate the sizes of micro-control store and nanostore.
- 4.27** Repeat Question 4.8 using microprogramming.
- 4.28** Show that it is possible to specify 675 microoperations using a 10 bit control function field.
- 4.29** Consider the following 10×8 microprogram implemented in a conventional control memory:

0000	0001 0000
0001	0010 1001
0010	0001 1111
0011	0001 0000
0100	0001 1111
0101	0001 0000
0110	0001 1111
0111	0010 1001
1000	0010 1001
1001	0010 1001
1010	
1011	

Implement the above microprogram in nanomemory. Justify the use of either a single-control store or a two-level store for the program.

- 4.30** A microprogram occupies 100 words and each word typically emits 70 CONTROL SIGNALS. The architect claims that by using a $2^l \times 70$ nanomemory (for some $l > 0$), it is possible to save 4260 bits. IF this were true, DETERMINE THE NUMBER OF DISTINCT CONTROL STATES IN THE ORIGINAL MI-

CROPROGRAM (Note that here when we say a control state we refer only to the control function field).

HINT: You may have to employ a trial and error approach to solve this problem

REFERENCES

1. Advanced Micro Devices Inc. *The AM 2900 Family Data Book*. 1976.
2. Advanced Micro Devices Inc. *Microprogramming Handbook*. 1977.
3. Agarwala, A. K. and T. G. Rausher. *Foundations of Microprogramming: Architecture, Software, and Applications*. Orlando: Academic Press, 1976.
4. Chu, Y. *Computer Organization and Microprogramming*. Englewood Cliffs, N.J.: Prentice-Hall, 1972.
5. Das, S. R., D. K. Banerji, and A. Chattopadhyay. "On Control Memory Minimization in Microprogrammed Computers." *IEEE Transaction on Computers* C-23 (September 1973).
6. Frieder, G. and J. Miller. "An Analysis of Code Density for the Two-Level Programmable Control of the Nanodata QM-1": Tenth Annual Workshop on Microprogramming, October 1975, pp. 26–32.
7. Hamacher, V. C., Z. G. Vrasenic, and S. G. Taky. *Computer Organization*. New York: McGraw-Hill, 1978.
8. Hayes, J. P. *Computer Architecture and Organization*. New York: McGraw-Hill, 1978.
9. Husson, S. S. *Microprogramming: Principles and Practice*. Englewood Cliffs, N.J.: Prentice-Hall, 1970.
10. Katzan, H., Jr.. *Microprogramming Primer*. New York: McGraw-Hill, 1977.
11. Mano, M. M.. *Digital Logic and Computer Design*. Englewood Cliffs, N.J.: Prentice Hall, 1979.
12. ———. *Computer Systems Architecture*. Englewood Cliffs, N.J.: Prentice-Hall, 1982.
13. Mead, C., and L. Conway. *Introduction to VLSI Systems*. Reading, Mass.: Addison-Wesley, 1980.
14. Mick, J., J. Brick. *Bit-Slice Microprocessor Design*. New York: McGraw-Hill, 1980.
15. Rauscher, T. G., and P. M. Adams. Microprogramming: A Tutorial and Survey of Recent Development. *IEEE Transaction on Computers* C-29, no. 1, (January 1980): 2–49.
16. Stritter, S., and N. Tredernick. "Microprogrammed Implementation of a Single Chip Microprocessor." *Sigmicro Newsletter* 9, no. 4 (December 1978).
17. Tanenbaum, A. S. *Structured Computer Organization*. Englewood Cliffs, N.J.: Prentice-Hall, 1981.

CHAPTER 5

Memory Organization

This chapter addresses the principles of memory organization. Topics covered include characteristics of memory systems, hierarchical memory organization, cache, associative, and virtual memory systems.

5.1 INTRODUCTION

A memory unit is an integral part of a computer system. Its primary function is to store all information needed by the system. Typically, a memory unit holds programs and data. A computer designer has to pay attention to the memory unit design because the memory system cost is a significant fraction of the cost of the total system. The system performance is largely dependent on the organization, storage capacity, and speed of operation of the memory system.

In a broad sense, a computer memory system can be logically divided into three groups:

- Internal memory
- Main memory
- Secondary memory

Internal memory refers to the set of registers confined to the CPU. These registers hold temporary results when a computation is in progress. There is no speed disparity between these registers and the CPU because they are fabricated with the same technology. However, the cost involved in this approach forces a computer architect to include only a few registers (usually 8 or 16) in the CPU.

Primary memory is the storage area in which all programs are executed. The CPU can directly access only those items that are stored in primary memory. Therefore, all programs and data must be within the primary memory to speed up execution. Previously, the primary memory was designed using magnetic cores. In modern computers, MOS technology is employed in primary memory design. Usually the size of the primary memory is much larger than processor memory, and its operating speed is slower than the processor registers by a factor of 10.

Secondary memory refers to the storage medium comprised of slow devices such as magnetic tapes and drums. These devices are used to hold data files and programs, such as compilers and data base management systems, which are not frequently needed by the processor. Secondary memories are also referred to as auxiliary, or backup store.

Studies have shown that typical programs spend most of their execution times in a few main routines or tight loops. In this situation, in a short period of time, the addresses generated by a CPU have the tendency to get clustered around small regions in the main memory. This phenomenon is known as the locality of reference. Suppose a small but a fast memory (faster than the main memory by 5 or 6 times) is installed to keep the most frequently needed information, and the CPU is instructed to access this fast memory (as opposed to the main memory). The efficiency of program execution can then be significantly improved. This newly added memory is known as the *cache memory*. This concept was first implemented in the IBM 360/85 computer. Since this technique is proved to be very successful, it is even included in minicomputer systems such as the PDP-11/70.

With the rapid growth of IC technology, this idea is also gaining acceptance in the microprocessor world. For example, a small cache memory is implemented in Motorola's 32-bit microprocessor, the MC68020. Various cache organizations and their merits are discussed later in this chapter.

The principal design goals of a memory system are summarized as follows:

1. Adequate storage capacity must be provided.
2. Cost per bit must be low.
3. Programmers must be relieved from performing tedious memory-management functions (such as dividing a large program into a set of small segments or overlays such that each overlay can fit into the main memory).
4. All memory management algorithms must be devised in such a way that they normally result in efficient space utilization.

5.2 CHARACTERISTICS OF MEMORY SYSTEMS

To design an efficient memory system, a computer designer should be aware of some basic characteristics of memory systems. In the following, some of them are discussed.

A memory system may be centralized or distributed. The organization of a centralized memory is shown in Figure 5.1.

In this arrangement, three processors share one common memory unit. This configuration is also referred to as a *tightly coupled system*. The salient features of this approach can be summarized as follows:

- Inexpensive
- Easy to control
- High use of the available space.

Alternatively, a memory system may be distributed. Such an organization is shown in Figure 5.2.

In this case, three processors share a common memory. In addition, each processor has a private or local memory attached to it. This configuration is sometimes called a *loosely coupled system*. The advantages of a distributed memory system are summarized as follows:

- Several memory units can be kept busy at the same time.
- Since each processor has its own private memory, they can be simultaneously engaged to perform local processing.

However, this approach increases the system cost and the complexity of the controller.

A memory system is said to be *passive* if no processing activities take place within a memory unit. In such a memory unit, only two standard operations are permitted: read from and write into the memory. A passive storage system is less expensive and calls for a simple controller. A memory unit can be made intelligent by associating a processor with each storage cell. Typically, this processor can perform operations such as add and increment. Such an intelligent memory unit can improve the system performance because it can process several items simultaneously.

In a conventional memory system, a stored item is selected by specifying its address. With a position-addressable memory, the address of the data must first be known. Once the address is known, data can be accessed by presenting the address to the memory. If the address of the data to be accessed is not available, data must be searched for.

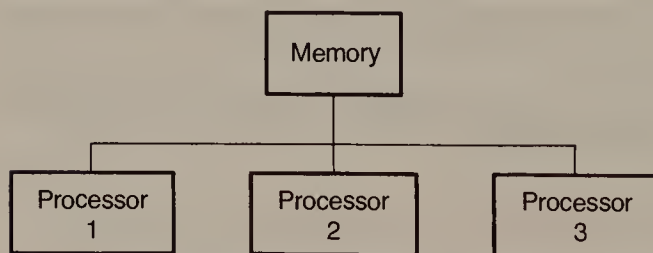


Figure 5.1 Centralized Memory Organization

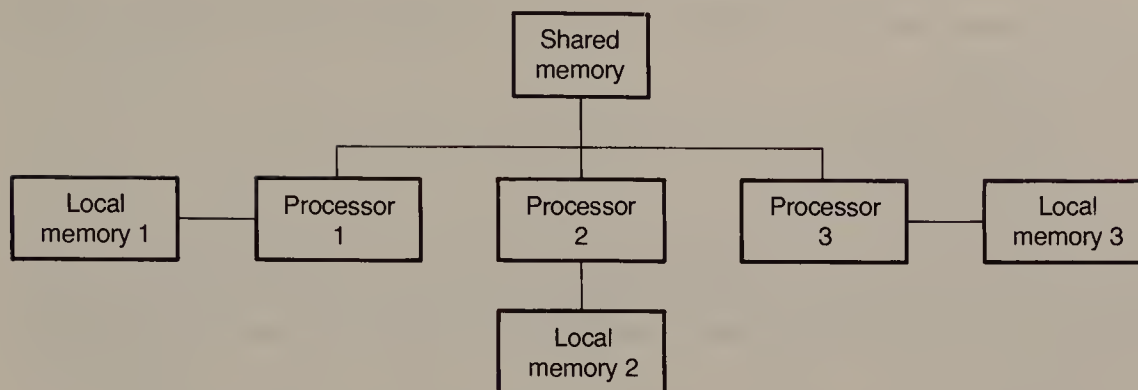


Figure 5.2 Distributed Memory Organization

In contrast, a content-addressable memory does not require an address to access data. When the memory unit is presented with a certain attribute of data (commonly referred to as a *key*) the memory responds with a presence flag and the data associated with the key, if the key is found in the memory. For this reason, a content-addressable memory is called an *associative memory*.

The key is simultaneously compared with all stored items for a match. Thus it operates at a great speed. However, this type of memory requires expensive hardware so it is usually limited to few storage locations. The design of an associative memory is discussed later in this chapter.

A linearly structured memory's storage cell addresses constitute a one-dimensional address space of addresses $0, 1, \dots, N - 1$, where $N - 1$ is the address of the last cell. Programs must be written to obtain useful data structures such as queues and stacks.

A memory unit may be organized as a stack or tree. These structured memory systems are efficient for application areas that use a stack or tree as the principal data structure.

In a conventional memory system, stored items are viewed as a sequence of bits, so no semantic details are provided. This type of memory unit is called an *untagged memory unit*. Conversely, a tagged memory system adds semantic specifications to each stored item by appending some extra bits (called tag bits) to each stored item.

Typically, a tag may be used to indicate whether a stored item is an instruction, a data item, or an address. A tagged memory offers the following advantages:

- More protection to stored items against misuse
- Elegant error-detection mechanisms
- Minimum programming effort

However, addition of tag bits increases the system cost and processing time.

The most important factor of a memory system is its cost. Usually it is expressed in dollars per bit. Suppose a memory unit has B bits and costs P dollars; the cost per bit C is then P/B dollars per bit. Therefore, a good design implies a very low cost per bit.

Two basic parameters indicate the speed of information being transferred in and out of a memory:

1. Access time t_A
2. Cycle time t_c

The access time, t_A , is defined as the average time taken to read a unit of information from the memory. Sometimes the access time is referred to as *read-access time*. *Write-access time* is similar to and usually equal to read-access time. The cycle time, t_c , of a memory unit is defined as the average time lapse between two successive read operations.

The reciprocal of access time is called the access rate r_A . It is expressed by

$$r_A = 1/t_A$$

Here, r_A is expressed in bits per second. The reciprocal of cycle time, t_c , is the data transfer rate or bandwidth, r_c . It is expressed by:

$$r_c = 1/t_c$$

Here, r_c is also expressed in bits per second. Bits per second is the same as baud rate (1 baud is equivalent to 1 bit per second).

The access mode is an important characteristic of a memory unit. The access mode refers to the method used to access information from the memory. There are two major access modes: random access and sequential access modes.

For random-access mode, any location of the memory can be accessed at random. The access time, t_A , is independent of the location from where the data is read.

The memory is accessed in a strictly sequential manner during sequential access. For example, if the third record stored in a sequential access memory is desired, the first two records have to be skipped. Therefore, the access time depends on the location of the data. Sequential access memories are also referred to as *serial-access memories*. Bipolar memory and magnetic tape are typical examples for random- and serial-access memories, respectively. Random-access memory is much faster and more expensive than sequential access.

To achieve a compromise, some memories combine both access modes. These memories are called *semirandom*, or *direct-access*, *memories*. A typical example of a semirandom access memory is a magnetic disk with one read/write head for each track. This arrangement permits any track to be accessed at random. However, access within a track must be made in a serial fashion.

There are some important characteristics of changing stored information. For instance, a read/write memory allows both read and write accesses. However, this flexibility results in high cost. Programs such as interpreters are less likely to get altered compared to programs in RAM. These programs can be stored in a relatively inexpensive ROM. As the name implies, a ROM permits only read access. Some ROMs are custom made, so their contents are programmed by the manufacturer. Such ROMs are called *mask programmable* ROMs. A typical example of a mask programmable ROM

is the character generator ROM 2513. Since these ROMs are mass-produced, they are inexpensive.

Sometimes a user may have to program a ROM in the working environment. For instance, in a fusible-link ROM, programming involves blowing off fuse links by passing electrical impulses. This type of ROM is called a programmable read-only memory (PROM). The major disadvantage of a PROM is it cannot be reprogrammed.

In practice, when programs are in the development stage, it is necessary to alter them before they become a marketable product. ROMs that allow this are called erasable programmable read-only memories (EPROMs) or electrically alterable read-only memories (EAROMs).

EPROM programs are entered using electrical impulses, and the stored information is erased by passing ultraviolet rays. Usually, an EPROM is programmed by inserting the EPROM chip into the socket of a PROM programmer and providing program addresses and voltage pulses (-25 to -40 V) at the appropriate pins of the chip. Typical erase times vary between 10 and 30 minutes.

With the advent of IC technology, it is possible to achieve an electrical means of erasure. These new ROMs are the EAROMs and can be programmed when in the circuit board. These memories are also called read mostly memories (RMMs), since they have much slower write times than read times. EAROMs are well suited for applications in which the read operations are more frequent than write operations.

Stored information will be lost if the power is turned off in semiconductor RAMs. This property is known as *volatility*, and hence, bipolar RAMs are usually called *volatile memories*. However, stored information in a magnetic tape or magnetic disk is not lost when the power is turned off. Therefore, these storage devices are called *nonvolatile memories*. A ROM is a nonvolatile memory.

A semiconductor memory constructed using bipolar transistors stores information in the form of flip-flop voltage levels. These voltage levels are not likely to be discharged. Such memories are called static memories because the stored information remains constant for a long period of time.

Semiconductor memories that are designed using MOS transistors hold information in electrically charged capacitors. The stored charge has the tendency to leak away. A stored 1 will become a 0 if no precautions are taken. These memories are called dynamic memories.

To prevent any information loss, dynamic memories have to be refreshed at regular intervals. Refreshing is boosting up the signal level and writing it back. This activity is performed by a hardware unit called a *refresh logic*. Figure 5.3 shows the various subcategories of ROMs, RAMs, and their associated technologies.

One of the best-known read/write memories is the ferrite core memory. The reading process destroys stored information. This property is called *destructive read-out* (DRO). Whenever data is read, it is transferred to a buffer. From this buffer, data is rewritten into the location where it was originally read. The inclusion of the memory buffer register (MBR) as a part of a RAM unit is the direct consequence of the phenomenon of DRO.

A semiconductor memory sometimes does not exhibit this property. Solid-state memories are referred to as nondestructive read-out (NDRO) memories.

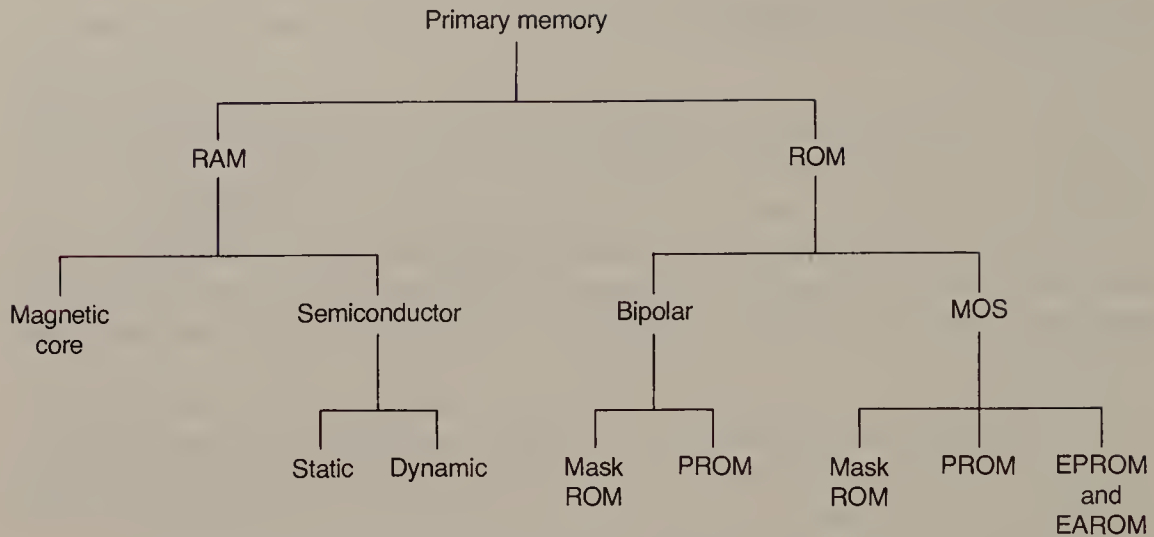


Figure 5.3 Subcategories of RAMs and ROMs

5.3 MAIN MEMORY DESIGN

The main memory of today's computer is formed using solid-state technology. Recent developments in memory technology allow manufacturers to provide up to 64 K bytes of storage on one chip. This figure is expected to increase by a factor of 4 within the next few years.

Although a single chip of 64K bytes is fairly adequate, there are applications that require storage in the order of 10^{10} bits. A designer is often required to obtain the needed memory capacity by inter connecting several available small size memory chips. This concept is known as *memory array design*.

Consider the block diagram of a typical static RAM chip shown in Figure 5.4.

This chip has 4096 bits, and these bits are organized as 1024 (2^{10}) words with 4 bits per word. Each word has a unique address that is specified on 10 address lines (A_9-A_0). The inputs and outputs are routed through 4-bit bidirectional data lines. (D_3-D_0). The operation of this chip is governed by the two control inputs: $\overline{WE}$ (write-enable) and $\overline{CS}$ (chip-select). The truth table of this chip is shown in Figure 5.5.

The table shows that when the $\overline{CS}$ input is high, the chip is not selected. Thus

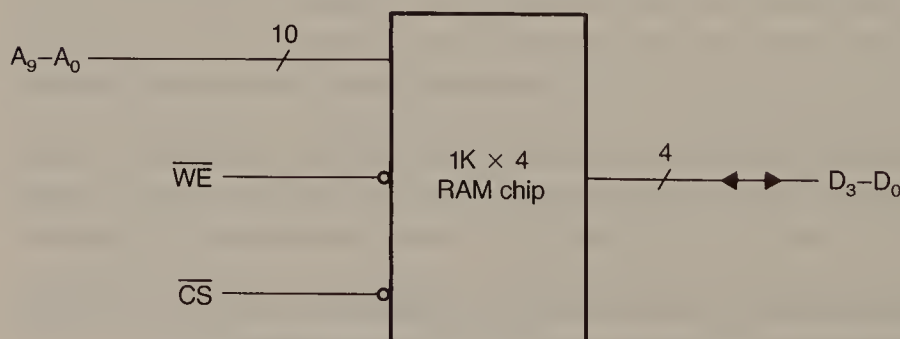


Figure 5.4 Block Diagram of a Typical $1K \times 4$ Static RAM Chip

$\overline{\text{CS}}$	$\overline{\text{WE}}$	MODE	STATUS OF THE BIDIRECTIONAL DATA LINES D3-D0	POWER
H	X	not selected	High impedance	Standby
L	L	Write	Acts as an input bus	Active
L	H	Read	Acts as an output bus	Active

Note: H-High, L-Low, X-don't care

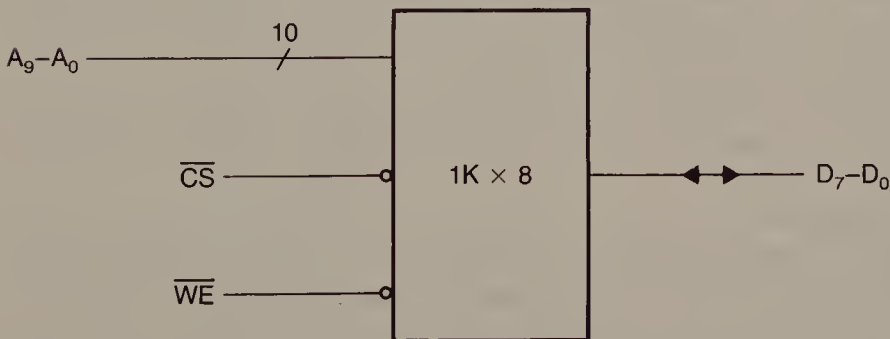
Figure 5.5 Truth Table for the $1\text{K} \times 4$ Static RAM Chip

lines D_3 – D_0 go to the high-impedance state. When $\overline{\text{CS}}$ and $\overline{\text{WE}}$ are low, data on lines D_3 – D_0 are written into the word addressed by A_9 – A_0 . When $\overline{\text{CS}}$ is low and $\overline{\text{WE}}$ is high, the contents of the memory word (whose address is specified on address lines A_9 – A_0) will appear on lines D_3 – D_0 . The device is disabled and automatically reduces its power requirements when the chip select input $\overline{\text{CS}}$ goes high. It remains in this low-power standby mode as long as $\overline{\text{CS}}$ remains high. This feature results in system power savings as high as 85% in larger systems where the majority of devices are disabled. The organization and characteristics of this chip are typical with respect to Intel's RAM chip, the 2148H.

Two RAMs can be horizontally expanded to obtain a memory system of $1\text{K} \times 8$ bits (See Figure 5.6). Four $1\text{K} \times 4$ RAM chips can be connected vertically to get a $4\text{K} \times 4$ RAM system. (See Figure 5.7.)

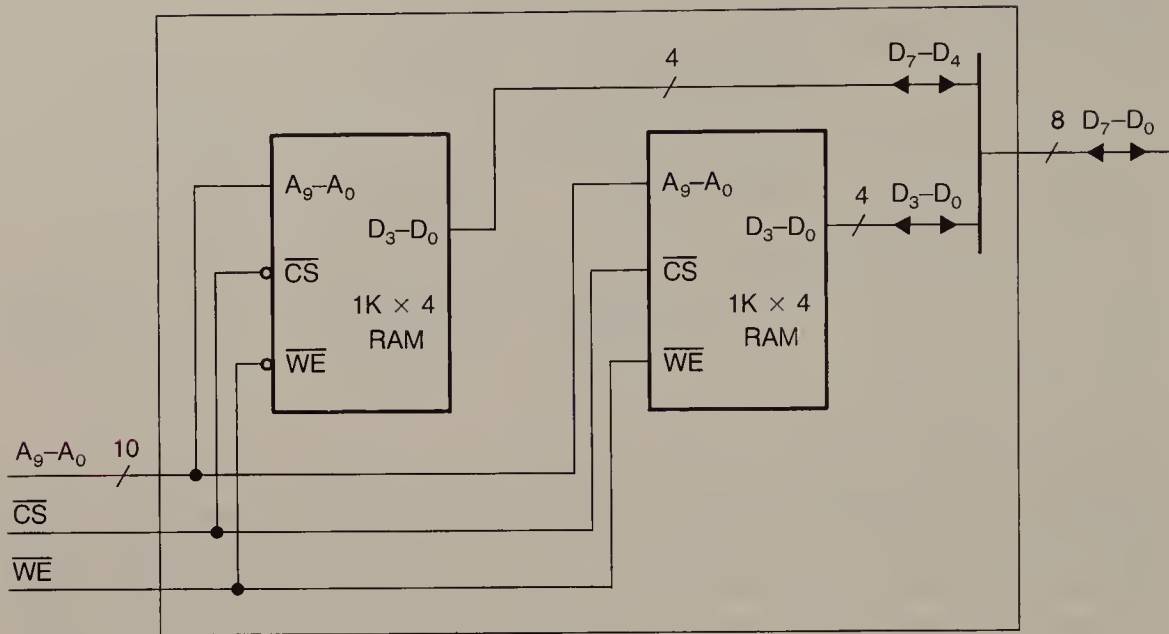
The size of the address bus is 12 bits wide. The first two high-order address lines, A_{11} and A_{10} are applied as inputs to a 2-to-4 decoder, such as the 74LS139. The decoder outputs select the appropriate RAM chips as follows:

A_{11}	A_{10}	Chip selected
0	0	RAM chip 0
0	1	RAM chip 1
1	0	RAM chip 2
1	1	RAM chip 3



a. Block Diagram

Figure 5.6 Realization of a $1\text{K} \times 8$ RAM using Two $1\text{K} \times 4$ RAM Chips



b. Internal Schematic

If the required RAM size is $M \times N$ and if the chip capacity is $m \times n$, then the number of chips can be calculated as follows:

$$\text{number of chips} = k = \frac{MN}{mn}$$

where k is rounded up to the nearest integer.

Suppose a processor has a 16-bit address bus. This means it can directly address 64k bytes ($2^{16} = 64K$) of main memory. However, typical operational environments such as multiuser and multitasking require more than 64K bytes of storage. In this case, the addressing capability of the processor can be extended by employing a software approach called *bank switching*. This method involves merely turning on one of the several memory banks under software control. A conceptual organization of this scheme is shown in Figure 5.8.

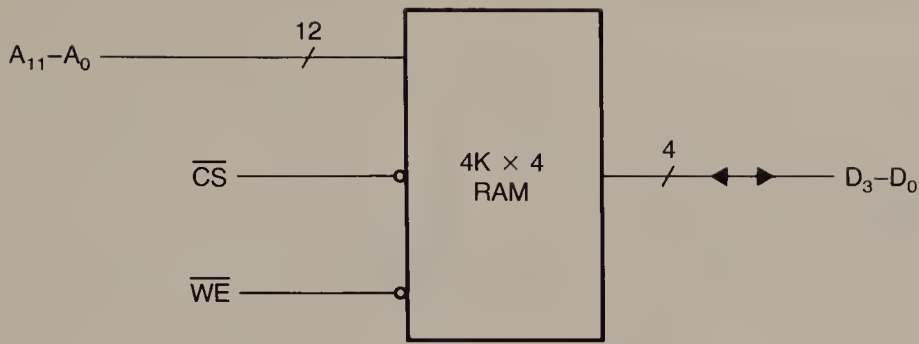
Here, several banks of memory are connected in parallel. Only one is activated at any one time. An output port specifies the bank to be switched on, and an off-board decoder guarantees that only one bank will be active.

Although the bank-switching concept reduces the operational speed (because of program execution), it is successfully used in the design of many multiuser microsystems.

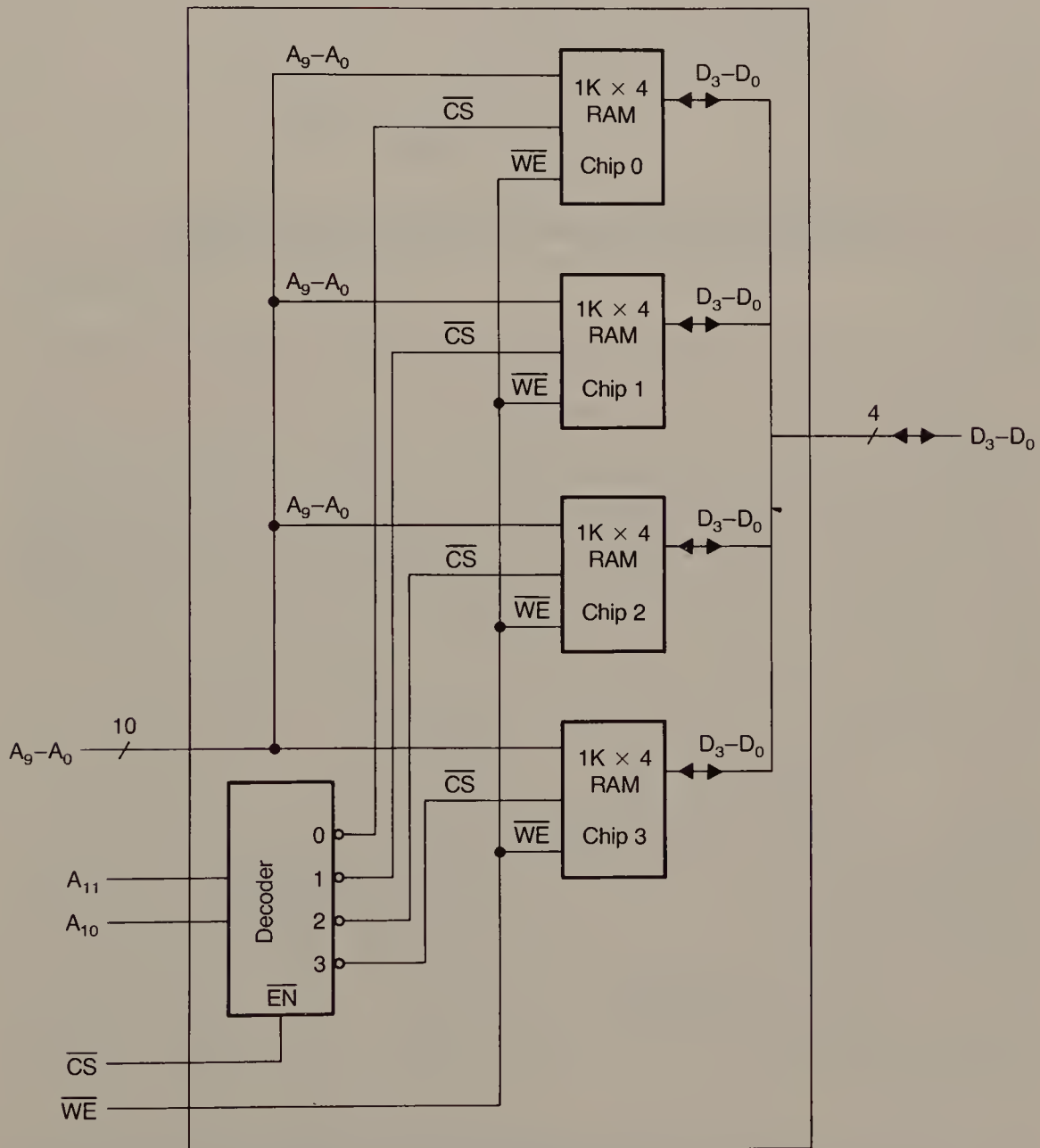
The addressing capability of the same processor discussed before can also be increased by expanding the generated address via a latch. This method is known as *address extension*. A conceptual view of this technique is illustrated in Figure 5.9.

The external latch can expand the available memory space if the data in the latch is used as a base address. The address coming from the processor is used as an offset and is appended to the base to form a wider address bus.

A natural extension of this scheme is to use some upper bits of the address gen-



a. Block Diagram



b. Internal Schematic

Figure 5.7 Realization of a 4K × 4 RAM using Four 1K × 4 RAM Chips

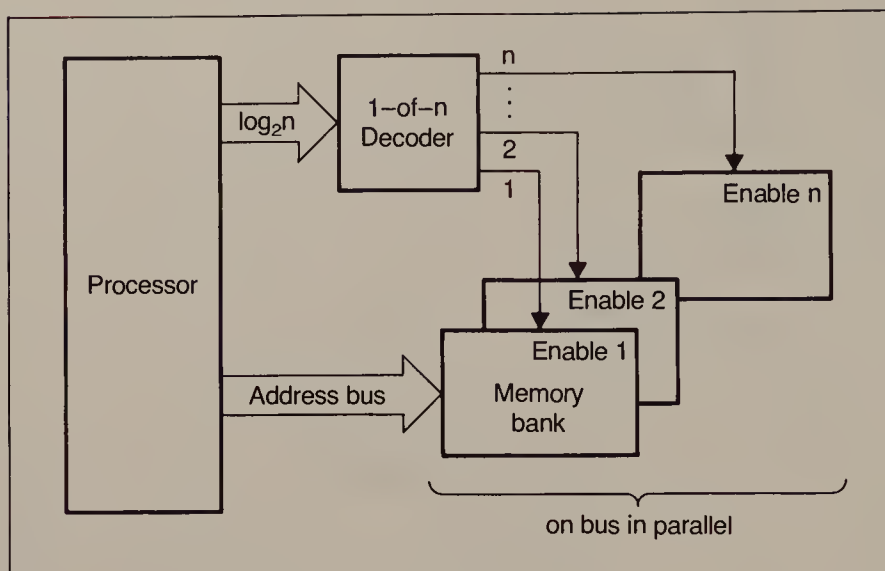


Figure 5.8 Conceptual Organization of the Bank-Switching Scheme (Reprinted from ELECTRONICS, August 11, 1987 issue. Copyright © 1987, McGraw-Hill, Inc. All rights reserved.)

erated by a processor for selecting one of the several base registers (as opposed to having a single latch).

This is used by the 74610 IC chip manufactured by Texas Instruments to expand the addressing capability of an 8-bit microprocessor. Figure 5.10 illustrates the use of this chip.

The 74610 chip includes sixteen 12-bit base registers and allows communication with 16M bytes of physical memory. The bank-switching and address-extension schemes discussed so far are actually rudimentary forms of automatic memory management. This topic is discussed later in this chapter.

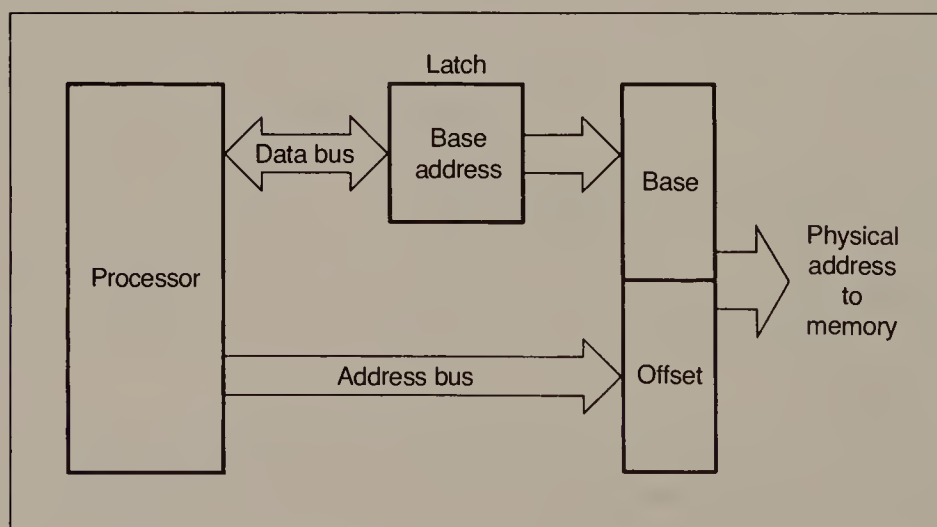


Figure 5.9 Conceptual Organization of the Address Extension Scheme (Reprinted from ELECTRONICS, August 11, 1987 issue. Copyright © 1987, McGraw-Hill, Inc. All rights reserved.)

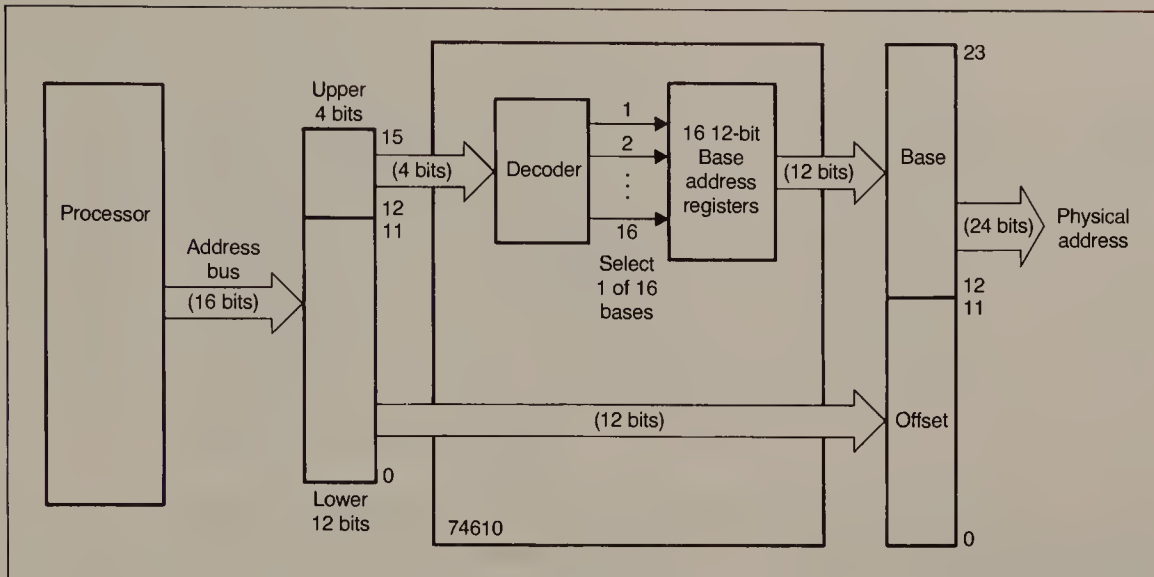


Figure 5.10 Address Extension using the 74610 Memory Mapping Circuit (Reprinted from ELECTRONICS, August 11, 1987 issue. Copyright © 1987, McGraw-Hill, Inc. All rights reserved.)

5.4 POPULAR ELECTROMECHANICAL MEMORY DEVICES

Electromechanical memory devices form a major segment of a computer's secondary memory and allow storage of large programs and data at a low cost. These components permit a serial access to stored data. Thus, they are significantly slower than the primary memory. In this section, some salient features of typical electromechanical devices such as tapes, disks, and drums are described.

Magnetic tapes were the first kind of secondary memory used in computer systems. A magnetic tape is similar to a home tape recorder. However, a magnetic tape holds digital information, whereas a tape recorder holds analog information.

A magnetic tape is divided vertically into frames and longitudinally into nine parallel tracks, as shown in Figure 5.11.

Each frame is capable of holding 9 bits of data. Out of these 9 bits, the first 8 bits constitute a 1-byte message and the ninth bit holds the parity. The parity information is used for error correction and detection. Information is stored along tracks using read-write heads. Read-write heads are designed so they can access all nine tracks contained in a frame simultaneously. Information is written on the tape by varying the current through the read write head.

The length of a magnetic tape is typically 2400 feet, and it is stored on a reel. This makes magnetic tapes not only compact but easily transportable. The tape are driven by a tape-drive unit.

In response to a read request, the tape is moved either forward or backward until the desired location is reached. The information is read or written only when the tape is moving at a constant velocity.

The tape velocity is defined as the rate of motion of the tape in the steady state.

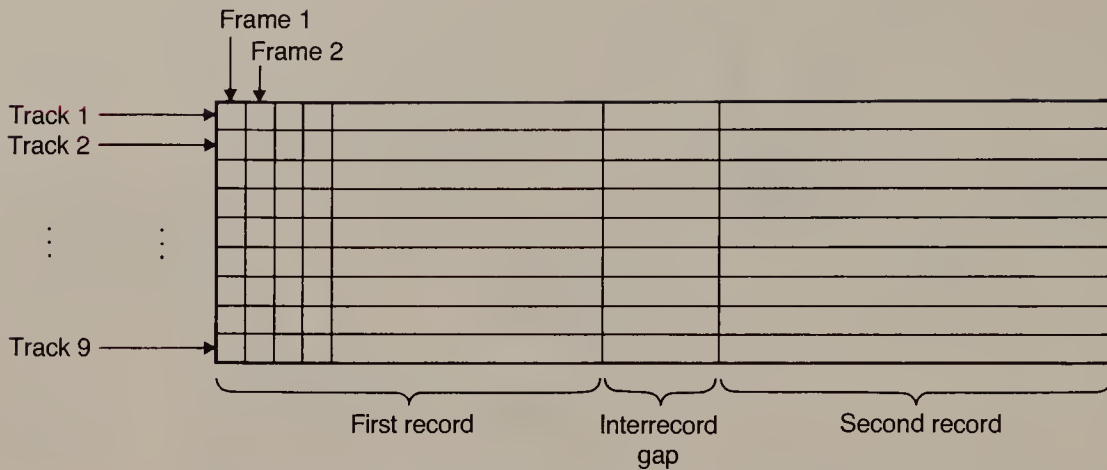


Figure 5.11 A Typical Nine-Track Magnetic Tape (Andrew S. Tanenbaum, *Structured Computer Organization*, 2e, © 1984, p. 31. Reprinted by permission of Prentice-Hall Inc., Englewood Cliffs, New Jersey.)

The tape is halted at the end of a data transfer. A storage density of 1600 bytes per inch and a tape velocity of 125 inches per second are typical figures for a standard magnetic tape. This implies that the data-transfer rate of a magnetic tape unit with these characteristics is $1600 \times 125 = 200,000$ bytes per second.

The growth in microcomputers popularized the use of a smaller magnetic tape unit enclosed in a plastic box. These units are called *cassettes*, and are low in cost. The major difficulty with these devices is the particle contamination caused by improper manual handling. With microcomputers, the use of audio cassettes is also permissible. An audio cassette costs \$1.00 to \$2.00 and can hold approximately 50 to 75 kilobytes of data. On an audio cassette, a binary 1 is stored as 2400-hertz square wave (8 cycles) and a binary 0 is stored as a 1200-hertz square wave (4 cycles). This gives a data transfer rate of 300 bits per second ($\frac{1}{4} \times 1200$).

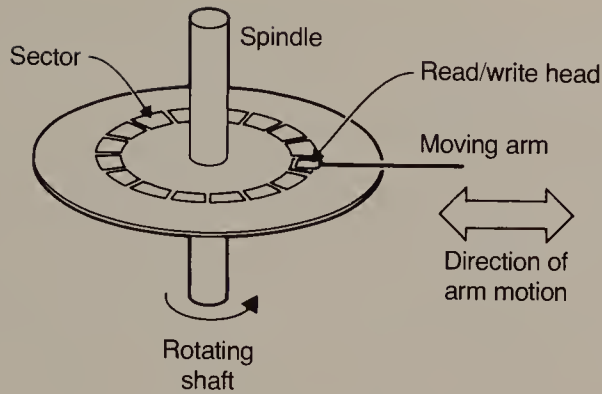
A magnetic disk looks like an LP phonograph record. The disk is made of either aluminum or plastic. The disk surface is coated with a magnetic material so information can be stored on it.

The recording surface is divided into a number of concentric circles called *tracks*. The information is written onto the tracks using a movable read/write head that moves from the innermost to the outermost tracks, and vice-versa. Usually, several identical disks are stacked over one another (with some separation between them) to form a disk pack. A typical disk pack organization is shown in Figure 5.12. In Figure 5.12 there is one read/write head per surface. Therefore, if there are n disks, there are $2n$ surfaces.

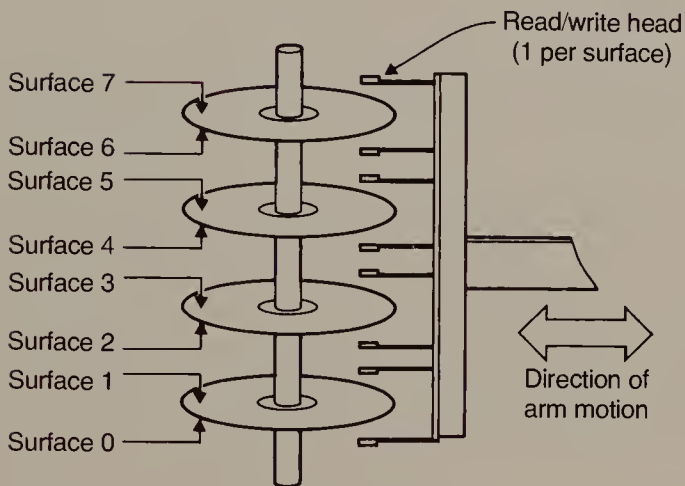
It is obviously possible to remove a disk from the disk-drive. During normal operation, disks are rotated continuously at a constant angular velocity. In a modern disk drive, all read-write heads can move simultaneously and independently. The radial position of the heads is called a *cylinder*. A disk pack with n disks has $2n$ tracks per cylinder.

Tracks are usually divided into sectors. A track may contain 32 to 256 sectors. Information transfer to and from a disk drive is achieved through a special hardware device called the *disk controller*.

To accomplish a data transfer, the read/write head must be positioned on the right



a. A Disk with One Platter



b. A Disk with Four Platters

Figure 5.12 Organization Diskpack (Andrew S. Tanenbaum, *Structured Computer Organization*, 2e, © 1984, p. 32. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey.)

track based on the given cylinder address. The time required to achieve this result is known as the *seek time*, t_s . Seeking the required track is the most time-consuming operation because it involves moving the read/write arm. It takes about 5 microseconds to move the arm from the innermost to the outermost cylinder.

After positioning the read/write head on the desired cylinder, the controller has to wait until the desired sector is under the read/write head. This waiting time is known as *latency*, or *rotational delay*, t_L . The access time is the sum of t_s and t_L . Here t_L depends on the rotation speed of the disk. There are two ways a w -bit word may be stored:

1. Consecutively store the entire word in one track of the same surface.
2. Store the same word in w different tracks of a cylinder.

For the second approach, it is possible to read or write all w bits at the same time provided all tracks are synchronized.

Both cases involve the same seek-time and latency overhead. Therefore, the over-

head is highest if only one word is read or written per access. For this reason, with respect to disk systems, information is transferred in terms of blocks where one block may span several kilo words.

A floppy-disk drive basically contains three elements. The first is a motor system that rotates the disk at a rate of 300 rpm. Second, the drive contains a read/write head that is used to read and write data to and from the diskettes. This read/write head is similar to the record/playback lead of a cassette tape recorder but is much more delicate and sensitive. The read/write head is positioned over various tracks on the diskettes. The number of tracks that a disk drive can handle varies from one manufacturer to another, but the currently available track handling capacities for 5¼-inch drives are 35, 40, 77, and 80 tracks, single- or double-sided. The third element of the drive is the electronics that send signals back to the floppy-disk controller to indicate the following typical situations:

- The disk cannot be written because of a special “write-protection tab” is placed on the “write-protect-slot.”
- The index-hole is detected. This hole actually indicates that the first data byte is about to pass by the read/write lead.

Floppy disks are inexpensive; in particular, a 3½-inch kind is capable of providing 800K bytes of storage.

The floppy disks are classified into two groups: single- and double-density diskettes. In IBM-compatible floppy single-density disks, each sector can hold 128 characters. In a double-density diskette, each sector can hold 256 characters. Although a double-density diskette provides more storage capacity, it increases the complexity of the disk-controller hardware. A floppy-disk controller can typically process the following commands:

- Move the head to a particular track.
- Read from or write into the specified sector.
- Read a track.
- Format the disk by performing a track-write operation.

A typical example of a widely used 5¼-inch floppy disk controller is Western Digital’s FD 1771 B floppy-disk controller IC chip. A complete discussion on the operation of this device is beyond the scope of this book. However, interested readers may consult the references listed at the end of this chapter for further information.

As with magnetic tape, the read/write head of a floppy disk is in direct contact with the disk surface. This may eventually damage the disk. Therefore, frequent replacement of disk and/or head is necessary.

5.5 MEMORY HIERARCHY

It is virtually impossible to build an entire memory unit using one single technology that offers both low cost and high operating speed. Therefore, a memory system is often

designed using different physical devices to achieve optimum performance. Based on the technology and/or devices used, a memory system is assumed to be composed of different levels. A typical four-level organization is shown in Figure 5.13.

This type of multilevel organization is also called a *hierarchical memory organization*. The CPU directly communicates with L_1 , L_1 communicates with L_2 , and so on. In general, all information accessed by the CPU is found in L_1 , and a copy of all information will be held in the lowest level (L_4 in this case). If the required word is missing from L_1 , then it is searched for in the lower levels. Suppose the missing word is found in L_j , where $j > 1$. It is then transferred to L_{j-1} , from L_{j-1} to L_{j-2} , and so on, until it reaches to L_1 . This transfer is essential because the CPU can access only L_1 .

Lower-level memories comprise slow devices such as disks and tapes. Therefore, to save time and to increase transfer efficiency, a block of words is transferred to a higher level rather than only the missing word. As previously stated, the locality of reference principle dictates that within a short period of time, a program typically generates only a set of addresses that are close to one another. Thus the block-transfer operation allows future references to be confined to the higher levels of the memory hierarchy. Typical parameters of a hierarchical memory organization are summarized in Figure 5.14.

Assume t_i , S_i , and C_i denote the access time, size, and cost per bit, respectively, of the i th level component in a hierarchical memory system. Then, the following relationships hold:

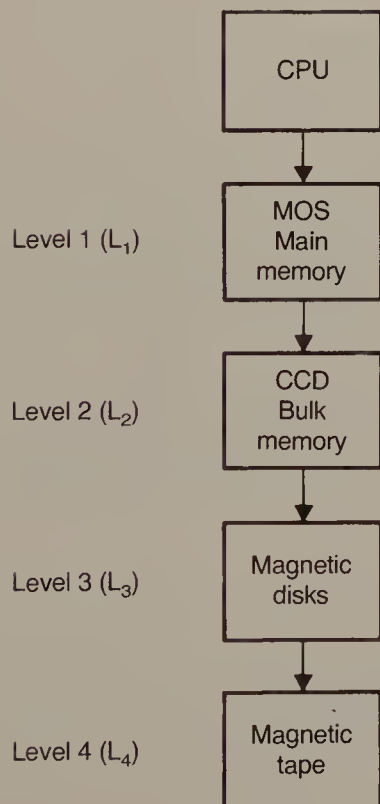


Figure 5.13 A Four-level Hierarchical Memory System

LEVEL	COMPONENT	SIZE	ACCESS TIME	GRANULARITY OF INFO TRANSFER
1	MOS main memory	4K–16M bytes	0.5–1 μ sec	2–32 words
2	CCD bulk memory	64K–64M bytes	5–10 μ sec	32–64 words
3	Magnetic disk	8M–500M bytes	50–100 μ sec	4K bytes
4	Magnetic tape	50M bytes	1–5 s	4K to 16 bytes

Figure 5.14 Parameters of a Typical Hierarchical Memory System

- $C_i > C_j$ for all i such that $i < j$
- $t_i < t_j$ for all i such that $i < j$
- $S_i < S_j$ for all i such that $i < j$

Some performance measures of an n -level hierarchical memory system will now be considered. The first performance measure is the average cost per bit $C(n)$. $C(n)$ is derived as follows:

$$C(n) = \frac{\sum_{i=1}^n S_i C_i}{\sum_{i=1}^n S_i} + C_0$$

In this equation, the summations $\sum_{i=1}^n S_i$ and $\sum_{i=1}^n S_i C_i$ provide the total size and cost of n -level memory system. C_0 reflects the cost of the interconnection network.

Let H denote the probability of finding information in a given level. This quantity is a function of the size, granularity of information and memory management algorithms. However, according to C. K. Chow[6], H is dependent on size; therefore, it is expressed as $H(S)$. The function $H(S)$ is also referred to by *success function*, or *hit ratio*. The quantity $1 - H(S) = F(S)$ is called the *miss ratio*, or *failure function*.

In a hierarchical memory system, the information contained in level L_i is also held by all levels L_j where $j > i$. Therefore, the probability for a hit to occur at level i and a miss in levels $i - 1$ through 1 can be expressed as:

$$h_i = H(S_i) - H(S_{i-1}) \quad (5.1)$$

The quantity h_i gives the access frequency to level i , so h_i is the relative number of successful accesses to level i (K. Hwang and F. A. Briggs[11]).

Earlier, it was observed that t_i is the access time for level i . Let T_i denote the effective access time from the CPU to level i . It can be derived as follows:

$$T_i = \sum_{j=1}^i t_j \quad (5.2)$$

In this equation it is assumed that the granularity of information transfer is just one word. From the basic principles of statistics, the average access time $\bar{T}$ of an n -level hierarchical memory system can be expressed as follows:

$$\bar{T} = \sum_{i=1}^n h_i T_i \quad (5.3)$$

Substituting the results of 5.1 and 5.2 into 5.3 gives

$$\bar{T} = \sum_{i=1}^n \{H(S_i) - H(S_{i-1})\} \sum_{j=1}^i t_j \quad (5.4)$$

By using induction on i , it can be shown that equation 5.4 can be simplified as follows:

$$\bar{T} = \sum_{i=1}^n \{H(S_n) - H(S_{i-1})\} t_i \quad (5.5)$$

The derivation of 5.5 is left as an exercise.

It is known that

$$H(S_n) = 1 \text{ (level } n \text{ contains everything)}$$

$$H(S_0) = 0 \text{ (level 0 does not exist)}$$

$$1 - H(S_i) = F(S_i)$$

$$\text{and } F(S_0) = 1 \text{ (because } H(S_0) = 0)$$

Therefore, equation 5.5 can be simplified as follows:

$$\bar{T} = \sum_{i=1}^n F(S_{i-1}) t_i \quad (5.6)$$

The following example illustrates how to calculate the average access time of a hierarchical memory system.



EXAMPLE 5-1

The details of a four-level hierarchical memory system can be summarized as follows:

LEVEL	COMPONENT TYPE	ACCESS TIME	HIT RATIO
1	ECL main memory	$t_1 = 100 \text{ ns}$	$H(S_1) = 0.6$
2	Core memory	$t_2 = 1 \text{ } \mu\text{s}$	$H(S_2) = 0.8$
3	Magnetic disk	$t_3 = 50 \text{ ms}$	$H(S_3) = 0.9$
4	Tape	$t_4 = 1 \text{ s}$	$H(S_4) = 1.0$

Determine the average access time of this memory system.

SOLUTION

$$F(S_0) = 1, F(S_1) = 1 - H(S_1) = 0.4,$$

$$F(S_2) = 1 - H(S_2) = 0.2, \text{ and } F(S_3) = 1 - H(S_3) = 0.1$$

$$\begin{aligned}\bar{T} &= F(S_0)t_1 + F(S_1)t_2 + F(S_2)t_3 + F(S_3)t_4 \\ &= (10^{-4} + 0.4 \times 10^{-3} + 0.2 \times 50 + 0.1 \times 1000) \text{ ms} \\ &= (0.0001 + 0.0004 + 10.0 + 100) \text{ ms} \\ &= 110.0005 \text{ ms}\end{aligned}$$

The concept of hierarchical memory systems is popular in large mainframe computers such as the IBM 370 and the CDC 7600. For these systems, the memory hierarchy includes bipolar cache memory, MOS main memory, ferrite core memory, magnetic disks, and tapes. However, such a multilevel organization is not common in microcomputer systems.

As stated before, the actual performance of a hierarchical memory system is dependent on several factors, some of which are as follows:

- Sequence of addresses generated by the CPU.
- Number of bits per level.
- Size of the block transferred when the required address is not found in level 1.
- Nature of policies that control the flow of information.

A rigorous mathematical analysis of the performance of a memory hierarchy is difficult because of the randomness associated with the hierarchical organization functions. However, satisfactory results may be obtained by employing simulation techniques. This topic is beyond the scope of this book.

5.6 CACHE MEMORIES

The performance of a computer system will be severely affected if the speed disparity between the processor and the main memory is significant. For example, the cycle time of the CDC STAR 100 processor is 40 nanosecond, while the cycle time of its main memory is 1280 nanosecond. In such a situation, the system performance can be significantly improved by adding a small, expensive, but fast buffer memory between the processor and the main memory. This buffer memory is called cache memory and was first implemented in the IBM 360/85 computer. Later, this concept became a part of minicomputers such as PDP-11 and Data General's Eclipse. With the advent of VLSI technology, the cache concept is gaining acceptance in the micro world. The fourth-generation 32-bit VLSI microprocessors such as Motorola's MC68020 and Intel's 80386 include an on-board cache in their CPU.

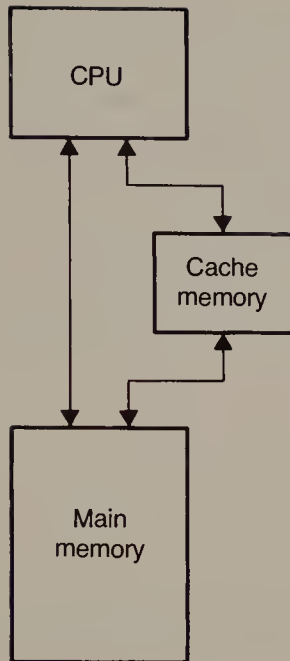


Figure 5.15 Memory Organization of a Computer System that Employs a Cache Memory

The block diagram representation of a computer system that employs a cache is shown in Figure 5.15. Cache assumes the top level in the computer's memory system. Usually, the cache-memory size is much smaller than the main memory and 5 to 10 times faster.

Assume the following mode of operation: An address generated by the CPU is first sent to the cache. If this reference is found in the cache, a *cache hit* is said to occur, and the data pertaining to the CPU reference is transferred to the CPU from the cache. However, if the reference is not found in the cache, a *cache miss* is said to have occurred. When there is a cache miss, first the required data is transferred to the cache from the main memory, and then it is read by the CPU from the cache. Usually the data is transferred from the main memory to cache in blocks.

Typical block size is 4 to 64 words. When the cache is full, one of the existing blocks will be evicted using standard replacement policies such as first-in first-out (FIFO) or least recently used (LRU). These policies are discussed later in this chapter. This block transfer is carried out in anticipation that the block is very likely to be referenced again and again in the near future. Such an intuitive guess is strongly supported by many practical situations such as tight loops and repeated use of a subroutine or a data structure such as a symbol table corresponding to a source program.

The performance of a system that employs a cache can be formally analyzed as follows: If t_c , h , and t_m specify the cache-access time, hit ratio, and the main memory access time, respectively; then the average access time can be determined as shown in equation 5.7:

$$\bar{t} = ht_c + (1 - h)(t_c + t_m) \quad (5.7)$$

The hit ratio h always lies in the closed interval 0 and 1, and it specifies the relative number of successful references to the cache. Equation 5.7 is derived using the fact that when there is a cache hit, the main memory will not be accessed; and in the event of a cache miss, both main memory and cache will be accessed. Suppose the ratio of main memory access time to cache access time is γ , then an expression for the efficiency of a system that employs a cache can be derived as follows:

$$\begin{aligned}
 \text{Efficiency} = \Lambda &= \frac{t_c}{\bar{t}} \\
 &= \frac{t_c}{ht_c + (1 - h)(t_c + t_m)} \\
 &= \frac{1}{h + (1 - h)\left(1 + \frac{t_m}{t_c}\right)} \\
 &= \frac{1}{h + (1 - h)(1 + \gamma)} \\
 &= \frac{1}{1 + \gamma(1 - h)}
 \end{aligned}$$

Note that Λ is maximum when $h = 1$ (when all references are confined to the cache). A hit ratio of 90% ($h = 0.90$) is not uncommon with many contemporary systems.

The following example provides a qualitative explanation:



EXAMPLE 5-2

Calculate $\bar{t}$, γ , and Λ of a memory system whose parameters are as indicated:

$$t_c = 160 \text{ ns}$$

$$t_m = 960 \text{ ns}$$

$$h = 0.90$$

SOLUTION

$$\begin{aligned}
 \bar{t} &= ht_c + (1 - h)(t_c + t_m) \\
 &= 0.9(160) + (0.1)(960 + 160) \\
 &= 144 + 112 \\
 &= 256 \text{ ns}
 \end{aligned}$$

$$r = \frac{t_m}{t_c} = \frac{960}{160} = 6$$

$$\Lambda = \frac{1}{1 + r(1 - h)} = \frac{1}{1 + 6(0.1)} = \frac{1}{1.6} = 0.625$$

This result indicates that by employing a cache, efficiency is improved by 62.5%.

Assume the unit of mapping is a block; then the relationship between the main and cache memory blocks can be established by using a specific mapping technique. Three mapping techniques are widely used:

- Fully associative mapping
- Direct mapping
- Set associative mapping

In a fully associative mapping scheme, a main memory block i can be mapped to any cache block j , where

$$0 \leq i \leq M - 1 \quad \text{and} \quad 0 \leq j \leq N - 1$$

This situation is shown in Figure 5.16.

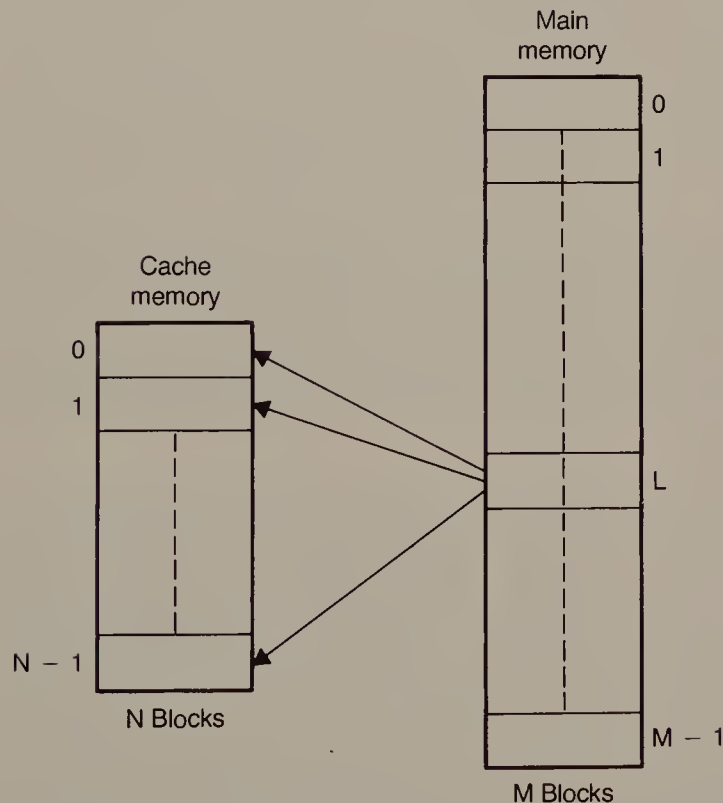


Figure 5.16 Fully Associative Mapping

From this figure, it is apparent that the main memory has M blocks and the cache is divided into N blocks. To determine which block of main memory is stored into the cache, a tag is required for each cache block. More formally:

Tag (j) = address of the main memory block stored in the cache block j

Suppose $M = 2^m$ and $N = 2^n$; then m and n bits are required to specify the address of a main and cache memory block, respectively. Since a main memory block can be mapped to any cache block, the entire m bits of a main memory block address has to be used as a tag. Since there are N cache blocks, N tags are needed. These tags can be either stored in the cache memory itself or separately stored in an associative memory called the tag directory.

In this scheme, when the CPU generates an address, the main memory block is extracted (usually the high-order m bits) and is then associatively compared with all N tags stored in the tag directory for a match. If a match occurs, the corresponding cache block number is retrieved, and the cache is accessed for the required data.

If the associative search fails, then the main memory is accessed for the required data. This is a cache miss. A block of data is transferred to the cache, and the tag directory is updated accordingly. If there is no free space in the cache, then the incoming main memory block will replace an existing cache block. Such a replacement is carried out using a replacement policy such as LRU or FIFO (to be explained later). The tag directory is also updated to reflect this action.

The principal advantages of this method are its great flexibility and that the address translation process can be performed quickly because of the high-speed tag directory. However, the high cost associated with a tag directory limits the liberal implementation of this idea. An increase in the number of main blocks linearly increases the size of the directory. Similarly, when the cache is expanded, the tag directory hardware also increases. The number of comparators required to conduct an associative search is equal to the number of cache blocks.

To reduce the hardware cost, the direct mapping technique is employed. A main memory block, i , is always mapped into the cache block $i \bmod N$. For this reason, this method is known as *congruent mapping* and is illustrated in Figure 5.17.

If $N = 2^n$ and $M = 2^m$, then $i \bmod N$ will be in the range of 0 to $2^n - 1$. This means that the low-order n bits of the binary representation corresponding to the main block i give the cache block number.

This is shown in Figure 5.18.

Figure 5.18 shows that the high-order $m - n$ bits can be used as a tag to determine if a main block is stored in the cache memory. When the CPU generates an address, the low-order n bits of the main memory block number field are used as the index to the tag directory, and the tag stored here is compared with the tag field of the specified main memory block number (high-order $m - n$ bits). If there is a match, the cache is accessed; otherwise the main memory is accessed. In the event of a cache miss, the incoming main memory block i always replaces the cache block $i \bmod N$ because it cannot be mapped into any other cache block. Thus no replacement policy is called for.

However, because of the mapping restriction, it causes some problems. For example, let i_1 and i_2 be two main memory blocks such that $i_2 = N + i_1$. In this case, both blocks are mapped to the same cache block because

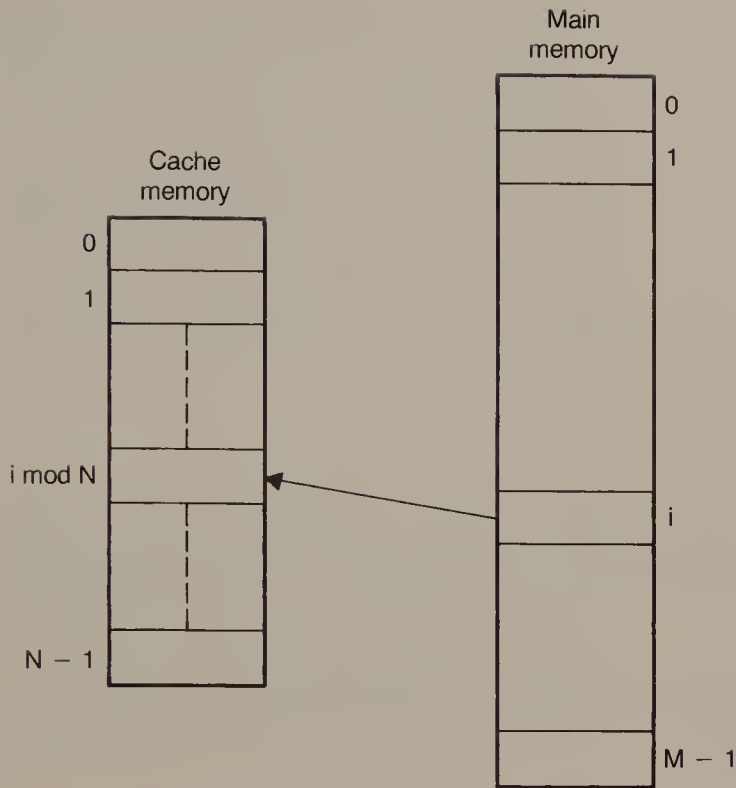


Figure 5.17 Direct Mapping

$$\begin{aligned}
 i_2 \bmod N &= (N + i_1) \bmod N \\
 &= N \bmod N + i_1 \bmod N \\
 &= 0 + i_1 \bmod N \quad (\because N \bmod N = 0) \\
 &= i_1 \bmod N
 \end{aligned}$$

Even though there may be other vacant cache blocks, i_2 needs to be evicted to provide room for i_1 , and vice versa. This situation leads to a drop in the hit ratio. This

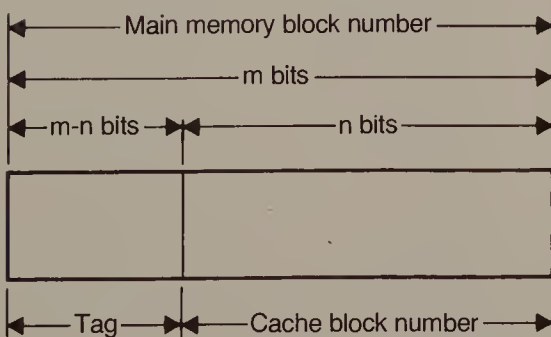


Figure 5.18 Address Format under Direct Mapping Scheme

reduction will be significant if the processor frequently requires information from two main memory blocks that are a multiple of N blocks away from one another. According to the locality of reference principle, such reference sequences are unlikely, and thus the direct mapping technique may not produce disastrous results all the time.

To achieve a compromise between fully associative and direct mapping methods, a new technique called *set associative mapping* is employed. In this case, the cache blocks are divided into N/S sets so there are s cache blocks per set. In this approach, a main memory block i will always be mapped to the cache set $i \bmod (N/S)$. However, within the set, the block can be placed anywhere. Therefore, direct mapping occurs at the set level, and fully associative mapping occurs within a set. A conceptual view of this idea is shown in Figure 5.19.

If $M = 2^m$, $N = 2^n$, and $S = 2^s$, then N/S will be in the range of 0 to $N/S - 1$, and $n - s$ bits are required.

Since $i \bmod (N/S)$ is also in the range of 0 to $N/S - 1$, the low-order $n - s$ bits of the main memory block address directly specify the cache set number. Therefore, the high-order $m - (n - s)$ bits of the main memory block address are treated as tag bits (see Figure 5.20).

If $s = 0$, this approach degenerates to direct mapping because each cache set contains only one block. When $s = n$, this method degenerates to fully associative mapping because the entire cache memory is treated as one set with $2^s = 2^n = N$.

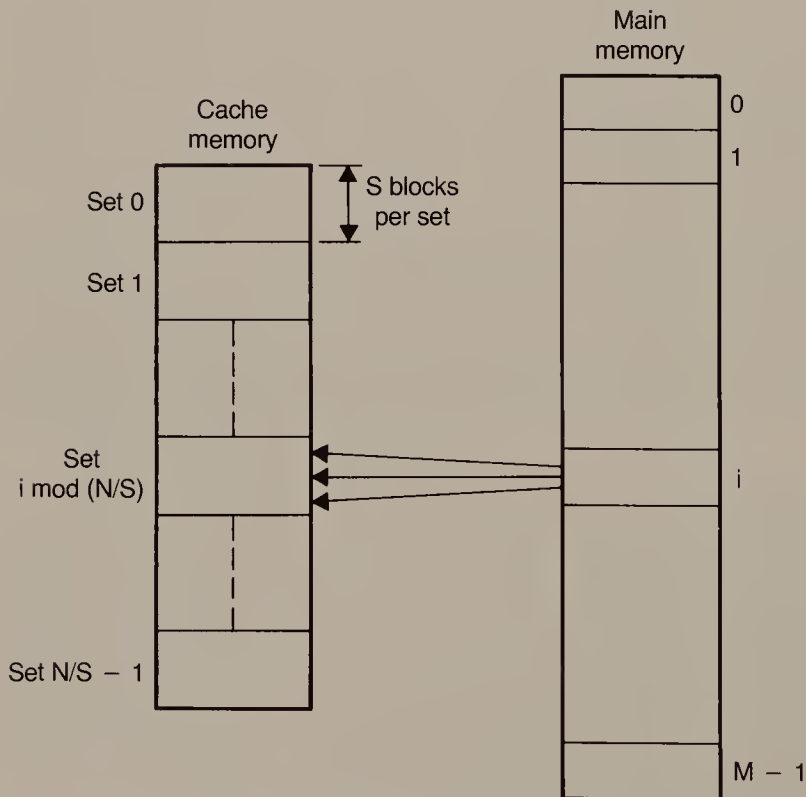


Figure 5.19 Set Associative Mapping

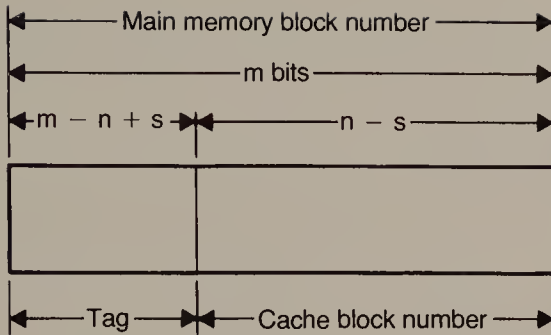


Figure 5.20 Address Format under Set Associative Mapping Scheme

blocks per set. This method is practical because it reduces the size of the tag field from m bits (fully associative) to $m - (n - s)$ bits. It is more flexible than direct mapping because there are N/S blocks within a set (as opposed to just one block per set). For this reason, it is widely employed in many contemporary computer systems such as VAX and Amdahl computers (see Figure 5.21).

The following example is included to explain the mapping techniques discussed so far.



EXAMPLE 5-3

The parameters of a computer memory system are specified as follows:

- Main memory size = 8K blocks
- Cache memory size = 512 blocks
- Block size = 8 words

Determine the size of the tag field of the main memory address under the following conditions:

COMPUTER SYSTEM	NUMBER OF CACHE BLOCKS	SIZE OF SETS
IBM 370/168	1024	8
Amdahl 470 V/8	2048	4
IBM 3033	1024	16
VAX - 11/780	1024	2

Figure 5.21 Set Associative Cache Parameters of Typical Computer Systems

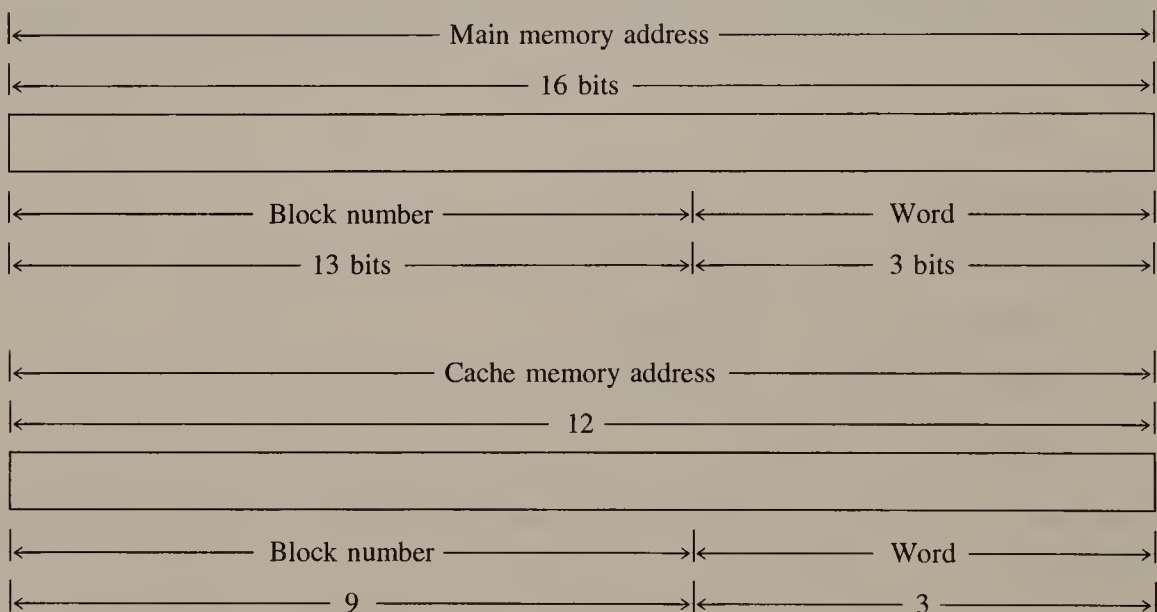
- a) Fully associative mapping
- b) Direct mapping
- c) Set associative mapping with 16 blocks/set

SOLUTION

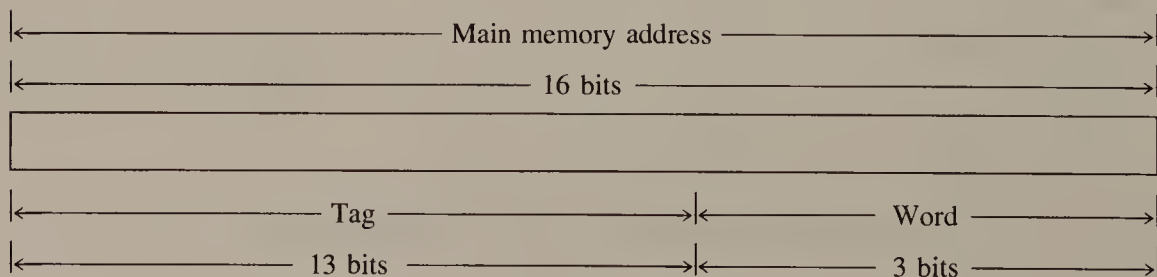
With the given data, compute the following:

- $M = 8K = 8192 = 2^{13}$, and thus $m = 13$.
- $N = 512 = 2^9$, and thus $n = 9$.
- Block size = 8 words = 2^3 words, and thus we require 3 bits to specify a word within a block.

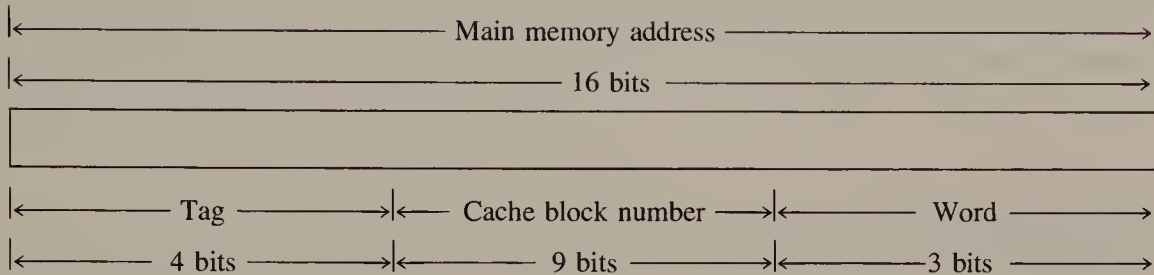
Using this information, we can determine the main and cache memory address formats as shown next:



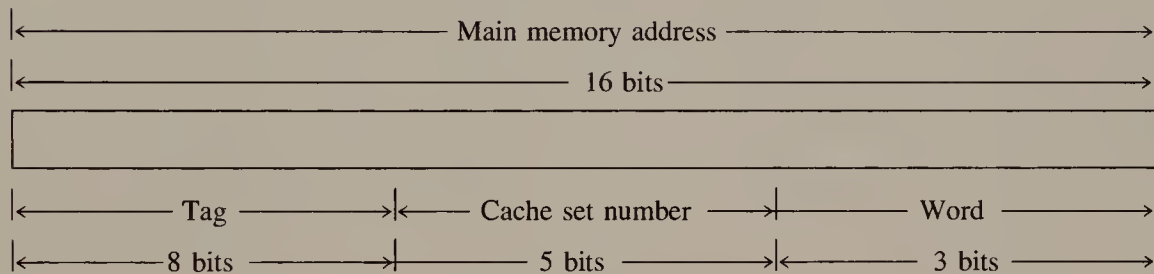
- a) In this case, the size of the tag field is $m = 13 =$ bits:



b) In this case, the size of the tag field is $m - n = 13 - 9 = 4$ bits:



c) $S = 16 = 2^4$, and thus $S = 4$. Therefore, the size of the tag field is $m - n + s = 13 - 9 + 4 = 8$ bits:



If the desired information is not found in the cache, data is retrieved from the main memory, and a block of data is transferred from the main memory to the cache. When the CPU alters the contents of the cache, it is necessary to update its main memory copy. There are two ways an update operation can be carried out.

In the first approach, whenever the CPU writes something into a cache block, that block is tagged as a *dirty block*. When a dirty block is to be replaced with a new block, the dirty block is copied into the main memory before it is overwritten by the incoming new block. This method is called *write-back*. The virtue of this policy is that it avoids unnecessary writing into main memory.

In the second method, whenever the CPU alters a cache address, the same alteration is made in the main memory copy of the altered cache address immediately. This technique is known as the *write-through* method. This policy can be easily implemented, and it ensures that the contents of the main memory are always valid. This feature is desirable in a multiprocessor system where the main memory is shared by several processors. However, this approach may lead to several unnecessary writes of a block to main memory before it is replaced in the cache.

One of the important aspects of cache-memory organization design is to devise a method that ensures proper use of the cache. Usually, the tag directory contains an extra bit for each entry. This additional bit is called a *valid bit*. When the power is turned on, the valid bit corresponding to each cache block entry of the tag directory is reset to zero. This is done to indicate that the cache block holds invalid data. When a block of data is first transferred from the main memory to a cache block, the valid bit corre-

sponding to this cache block is set to 1. In this arrangement, whenever the valid bit is a zero, it implies that whenever the valid bit is 0, the cache access should be disabled and such a situation is to be considered as a cache miss.

The following example illustrates the effect of cache management policies discussed so far.

EXAMPLE 5-4

The access time of a cache memory is 50 ns and that of the main memory is 500 ns. It is estimated that 80% of the main memory requests are for read and the remaining are for write. The hit ratio for read access only is 0.9 and a write-through policy is used.

- Determine the average access time considering only the read cycles.
- What is the average time if the write requests are also taken into consideration

SOLUTION

$$\begin{aligned}
 \text{a) } \bar{t} &= ht_c + (1 - h)(t_c + t_m) \\
 &= 0.9 \times 50 + (0.1)(550) \\
 &= 45 + 55 \text{ ns} \\
 &= 100 \text{ ns}
 \end{aligned}$$

$$\begin{aligned}
 \text{b) } t_{\text{read/write}} &= (\text{read request probability}) \times \bar{t}_{\text{read}} + (1 - \text{read request probability}) \times \bar{t}_{\text{write}} \\
 \text{Read-request probability} &= 0.8
 \end{aligned}$$

$$\text{Write-request probability} = 0.2$$

$$\bar{t}_{\text{read}} = \bar{t} = 100 \text{ ns (result of Part (a))}$$

$$\bar{t}_{\text{write}} = 500 \text{ ns (because both main and cache memories are updated at the same time)}$$

$$\begin{aligned}
 \bar{t}_{\text{read/write}} &= 0.8 \times 100 + 0.2 \times 500 \\
 &= 80 + 100 \text{ ns} \\
 &= 180 \text{ ns}
 \end{aligned}$$

The growth in IC technology has allowed manufacturers to fabricate a small cache on the CPU chip. The on-chip cache of Motorola's 32-bit microprocessor, the MC68020, is discussed next.

The MC68020 is an HMOS (high-density MOS) high-performance two-level microprogrammed microprocessor. Motorola claims it is the first 32-bit microprocessor to have evolved from a 16-bit machine to a full 32-bit machine that provides 32-bit address

and data buses as well as 32-bit internal structures. Many techniques were used to improve performance and maintain compatibility with other processors of the MC68000 family. Among the improvements are new addressing modes, which better support high-level language structures, an expanded instruction set, which provides 32-bit operations for the limited cases not supported by the MC68000 processor, and several new instructions, which support new data types and provide efficient solutions to a wide variety of special-purpose application problems.

The MC68020 on-chip cache is a direct mapped instruction cache. Only instructions are cached; data items are not. This cache is a collection of 64 entries, where each cache entry consists of a 26-bit tag field and 32-bit instruction data. The tag field includes the following components:

- High-order 24 bits of the memory address.
- The most-significant bit FC2 of the function code. In the MC68020 processor, the 3-bit function code combination FC2 FC1 FC0 is used to identify the status of the processor and the address space (to be explained later) of the bus cycle. For example, FC2 = 1 means the processor operates in the supervisory or privileged mode. Otherwise, it operates in the user mode. Similarly, when FC1 FC0 = 01, the bus cycle is made to access data. When FC1 FC0 = 10, the bus cycle is made to access code.
- Valid bit.

A block diagram of the MC68020 on chip cache is shown in Figure 5.22.

If an instruction fetch occurs when the cache is enabled, the cache is first checked to determine if the word requested is in the cache. This is achieved by first using 6 bits of the memory address (A_7 – A_2) to select one of the 64 entries of the cache. Next, address bits A_{31} – A_8 and function bit FC2 are compared to the corresponding values of the selected cache entry. If there is a match and the valid bit is set, a cache hit is said to occur. In this case, the address bit A_1 is used to select the proper instruction word stored in the cache and the cycle ends.

If there is no match or the valid bit is cleared, a cache miss is said to occur. In this case, the instruction is fetched from external memory. This new instruction is automatically written into the cache and the valid bit is set. Since the processor always prefetches instructions from the external memory in the form of long words, both instruction data words of the cache will be updated regardless of which word caused the miss.

The MC68020 on-chip instruction cache obtains a significant increase in performance by reducing the number of fetches required to external memory.

Typically, this cache reduces the instruction execution time in two ways. First, it provides a two-clock-cycle access time for an instruction that hits in the cache (see Figure 5.23); second, if the access hits in the cache, it allows simultaneous instruction and data access to occur. Of these two benefits, simultaneous access is more significant, since it allows 100% reduction in the time required to access the instruction rather than the 33% reduction afforded by going from three to two clocks.

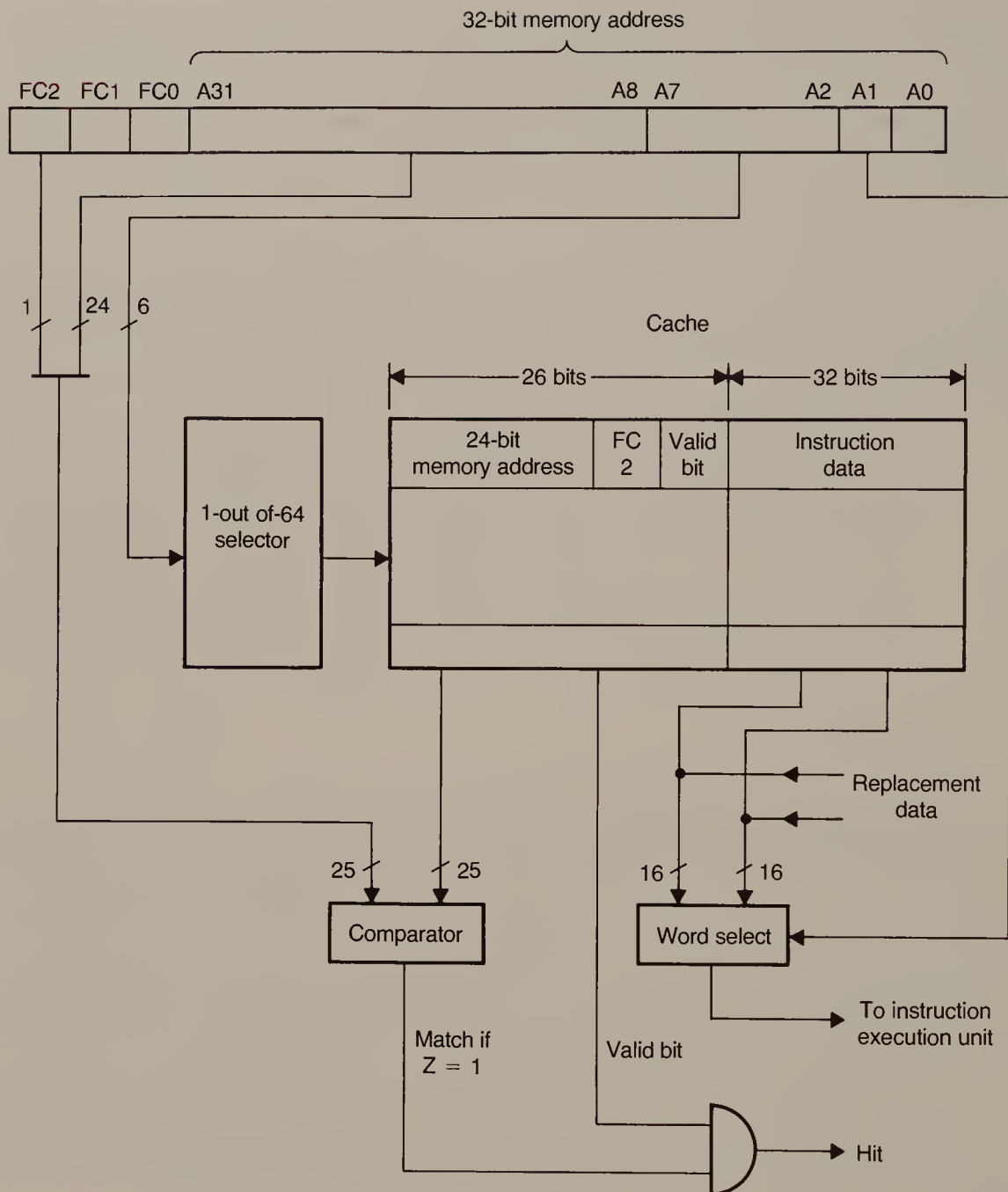


Figure 5.22 MC68020 On-chip Cache Organization

5.7 ASSOCIATIVE MEMORY

Earlier, it was shown that an associative memory is a hardware structure that allows the retrieval operations to be performed at a great speed. In this section, the hardware implementation of a word-organized associative memory and its salient characteristics are discussed. Consider the conceptual organization of an associative memory shown in Figure 5.24.

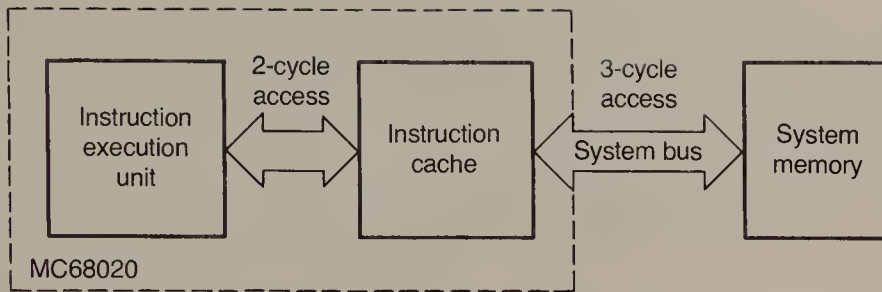


Figure 5.23 Mechanics of the MC68020 Instruction Cache (Courtesy of Motorola, Inc.)

In this arrangement items are stored in the form:

Key value	Attributes
-----------	------------

The given key value K is simultaneously compared with all the stored key values for a match. If a match occurs, the corresponding attributes are retrieved. The search time is greatly reduced because simultaneous comparison is performed by the hardware. The associative search is carried out solely on the basis of contents of the key and not on the basis of key addresses. Hence, the information has to be held in a special type of memory that allows content-based access to stored items. For this reason, associative memories are also referred to as *content-addressable memories*.

The general structure of an associative memory system is shown in Figure 5.25. In this organization, there are seven typical components and their functions are described as follows:

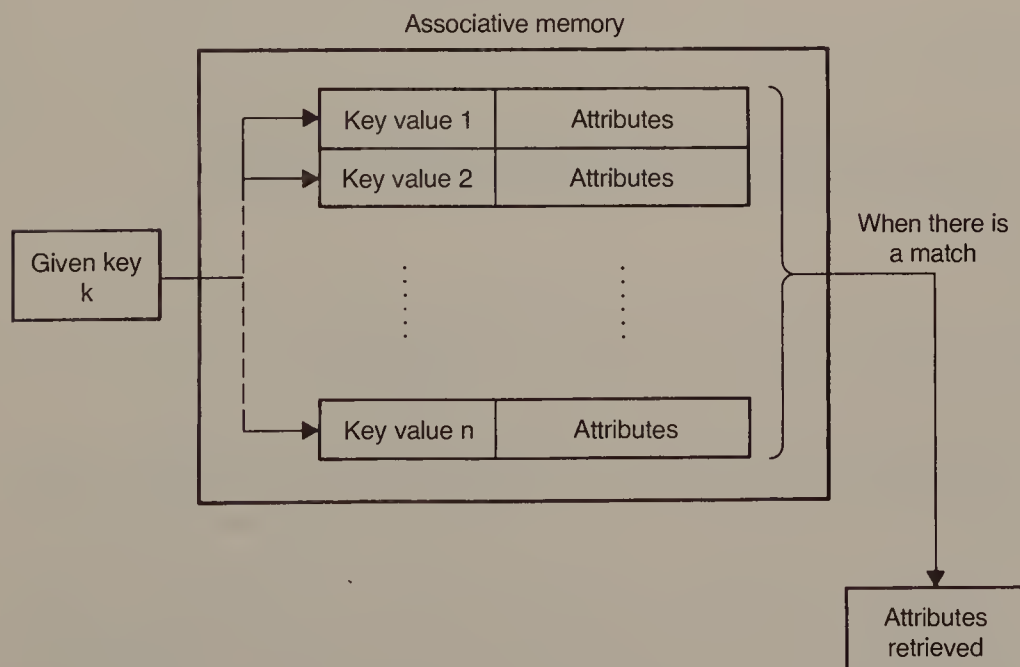


Figure 5.24 Conceptual Representation of an Associative Search

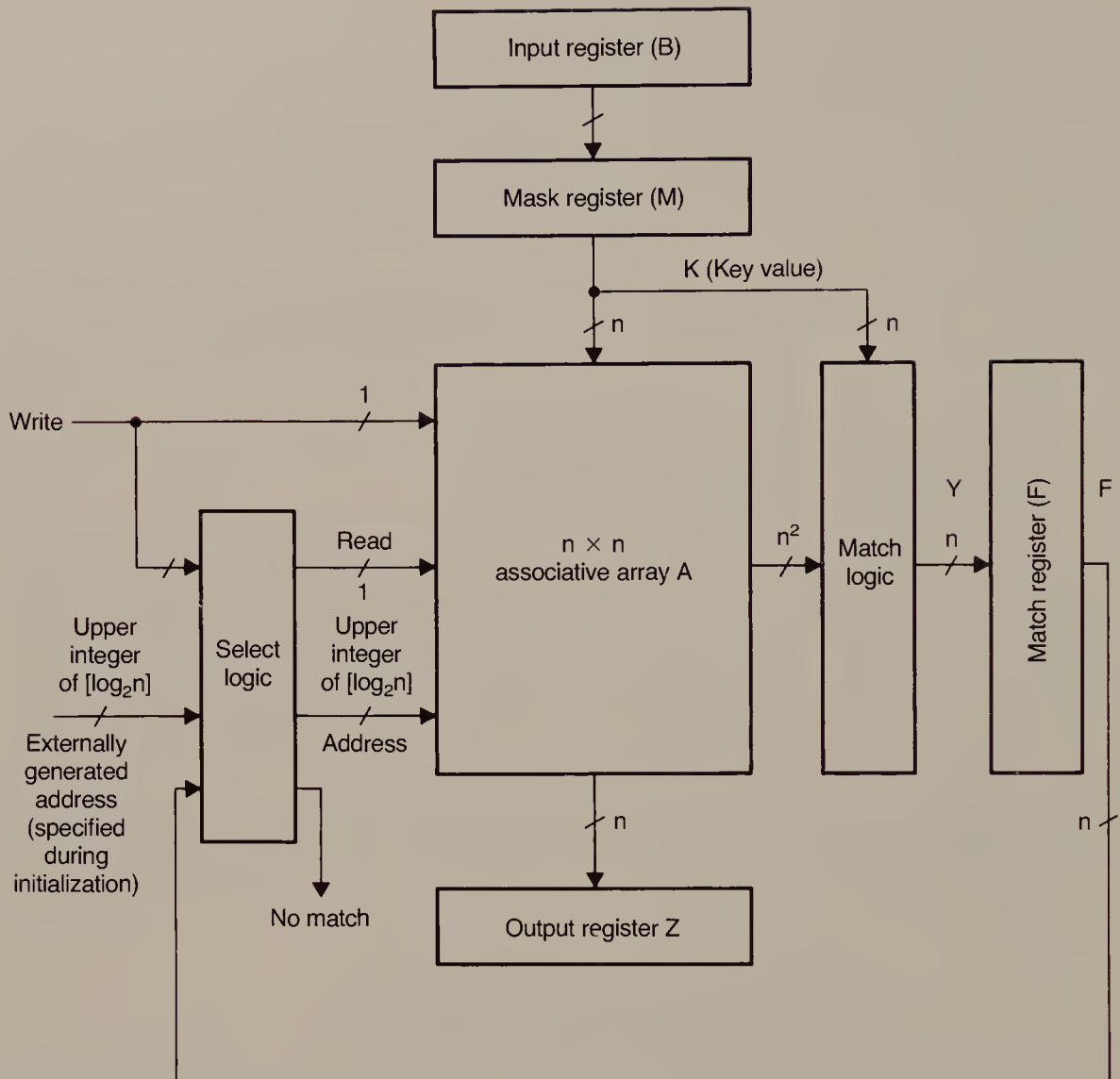


Figure 5.25 Associative Memory Organization (M. Morris Mano, *Computer System Architecture*, © 1982, p. 490. Adapted by permission of Prentice-Hall Inc., Englewood Cliffs, New Jersey.)

1. **Input register:** This is a buffer register. It holds the data to be written into the associate memory or the information to be searched for.
2. **Mask register:** The principal use of this register is to mask off some portions of the input register to form a key. Typically, the input information might contain several fields such as last name, account number, annual salary, and so on.

However, in a typical search only one field or a combination of some selected fields is needed. Therefore, to eliminate the unwanted fields from participating in the search process, the mask register is used. In general, the size of the mask register is the same as the input register. For each bit, b_i , of the input register B, there exists a corresponding bit, m_i , in the mask register M.

In this arrangement, the bit b_k of the input register is included in the search process only if the corresponding mask register bit m_k is set to 1. The desired result is obtained by performing a logical AND operation on the corresponding bits of the input and mask registers.

To illustrate the masking concept, consider the following example. For the sake of simplicity, assume both B and M registers are 8 bits wide. Assume the contents of the B and M registers are as follows:

	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0	
B =	1	0	1	1	1	1	0	1	Bits that are masked off

	m_7	m_6	m_5	m_4	m_3	m_2	m_1	m_0
M =	1	1	1	1	0	0	0	0

Only bits b_4 through b_7 will participate in the search. Thus the desired pattern we look for in the associative memory is 1 0 1 1 x x x (x is a don't care).

From the preceding example, it follows that masking off more bits will yield a lesser search time. Those bits that are masked off must not affect the final result of the associative search procedure. Since the flow of data from the input register to the associative cells is routed through the mask register, all the mask register bits are set to 1 when the associative memory cells are loaded from an external source of data.

3. *Output register*: This register functions as the destination for data being retrieved from the associative store.
4. *Associative array*: This is a collection of basic memory cells organized in the form of an array. Figure 5.26 shows the organization of an associative array of n words with n bits per word.

Each basic cell A_{ij} of Figure 5.26 is capable of holding 1 bit of information. This basic cell can be designed using a J-K flip-flop shown in Figure 5.27.

5. *Match logic*: This is a combinational circuit whose purpose is to detect if there is a match between the given key k and the associated value stored in the cellular array. Figure 5.28 shows the hardware structure of one stage of the match logic for a word-organized associative memory. In Figure 5.28, the effective key bit k_j (the key bit for which the corresponding mask register bit m_j is set to 1) is compared with the j th bit of the i th word A_{ij} stored in the associative array. The digital comparator is obtained by the exclusive NOR gate. Its output is 1 only when $k_j = A_{ij}$.

Recall that a key is formed by masking off some portions of the input register using the mask held in the mask register M. If the mask bit m_j is 1, the corresponding input bit b_j becomes part of the effective key, and, hence, this bit has to be tested for a match against the bit A_{ij} of the associative array. If $m_j = 0$, bit b_j should not be compared against bit A_{ij} , and this should not

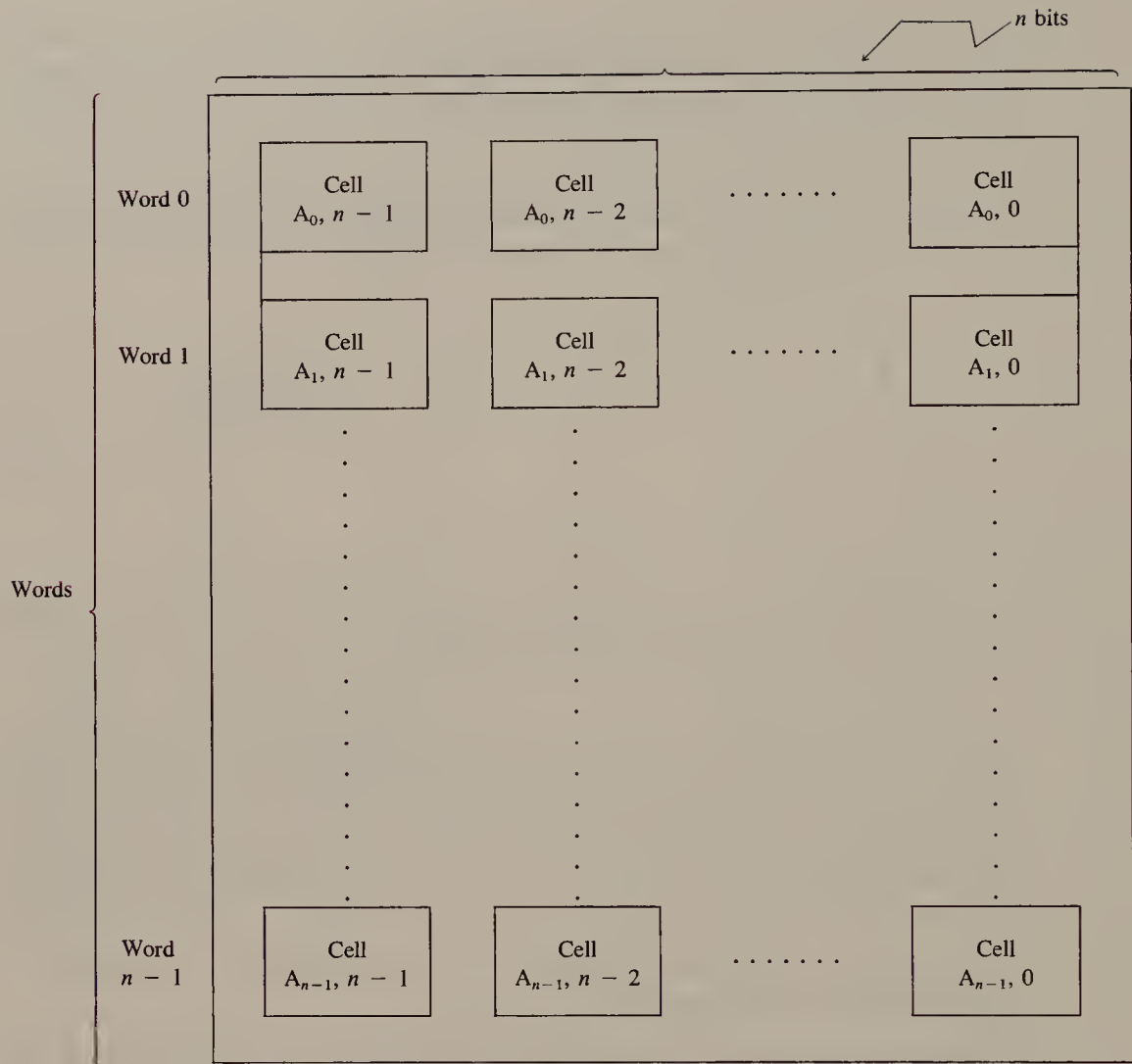


Figure 5.26 Organization of Associative Memory Cells (M. Morris Mano, *Computer System Architecture*, © 1982, p. 491. Adapted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey.)

affect the end result of the associative search. To realize these conditions, the match logic portion includes a two-input multiplexer. This multiplexer is controlled by the mask register bit m_j . Thus, when $m_j = 1$, the output c_{ij} is the output of the exclusive NOR gate; otherwise, the output of c_{ij} is 1. Thus the masked portion of the key does not affect the end result. The 1-bit match logic considered so far can be extended to take care of an n -bit word, as shown in Figure 5-29.

In the system of Figure 5.29., the outputs $c_{i, 0}$ through $c_{i, n-1}$ generated

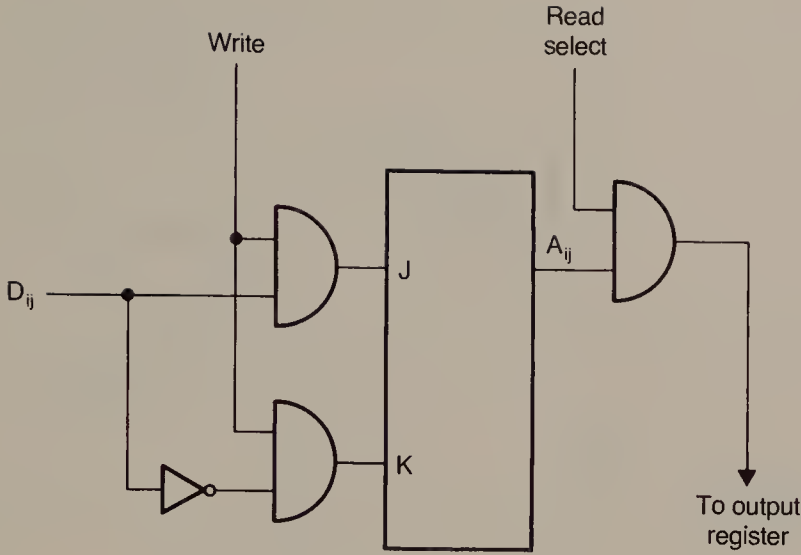


Figure 5.27 Basic Memory Cell

by the array of n 1-bit match logic units are ANDed to generate the output bit y_i . More formally, the Boolean equation formed by this hardware can be written as:

$$Y_i = \prod_{k=0}^{n-1} c_{ik}$$

where

$$c_{ik} = 1 \text{ if } m_k = 0$$

$$c_{ik} = b_k \odot A_{ik} \text{ if } m_k = 1$$

(The symbol $\odot$ indicates the exclusive NOR operation.)

From these equations, the bit y_i is 1 only when the given key k matches with the key value stored in the i th word of the associative array.

In general, if there are r words in the associative memory, the match logic system will generate an n -bit output $Y = Y_{r-1}Y_{r-2} \cdots y_1y_0$. If bit y_j ($0 \leq j < n - 1$) is 1, there is a match between the given key k and the key of the j th word of the associative store.

6. *Match register*: The purpose of this register is to latch the result generated by the match logic.

In Figure 5.30, the interconnection between the match logic and the match register F is shown. The register F consists of an array of D flip-flops, and each is set or reset depending on the results generated by the match logic. The flip-flop output f_i is set to 1 if and only if

$$\left. \begin{array}{l} k_j = A_{ij} \\ \text{or} \\ m_j = 0 \end{array} \right\} \text{ for } j = 0, \cdots, n - 1$$

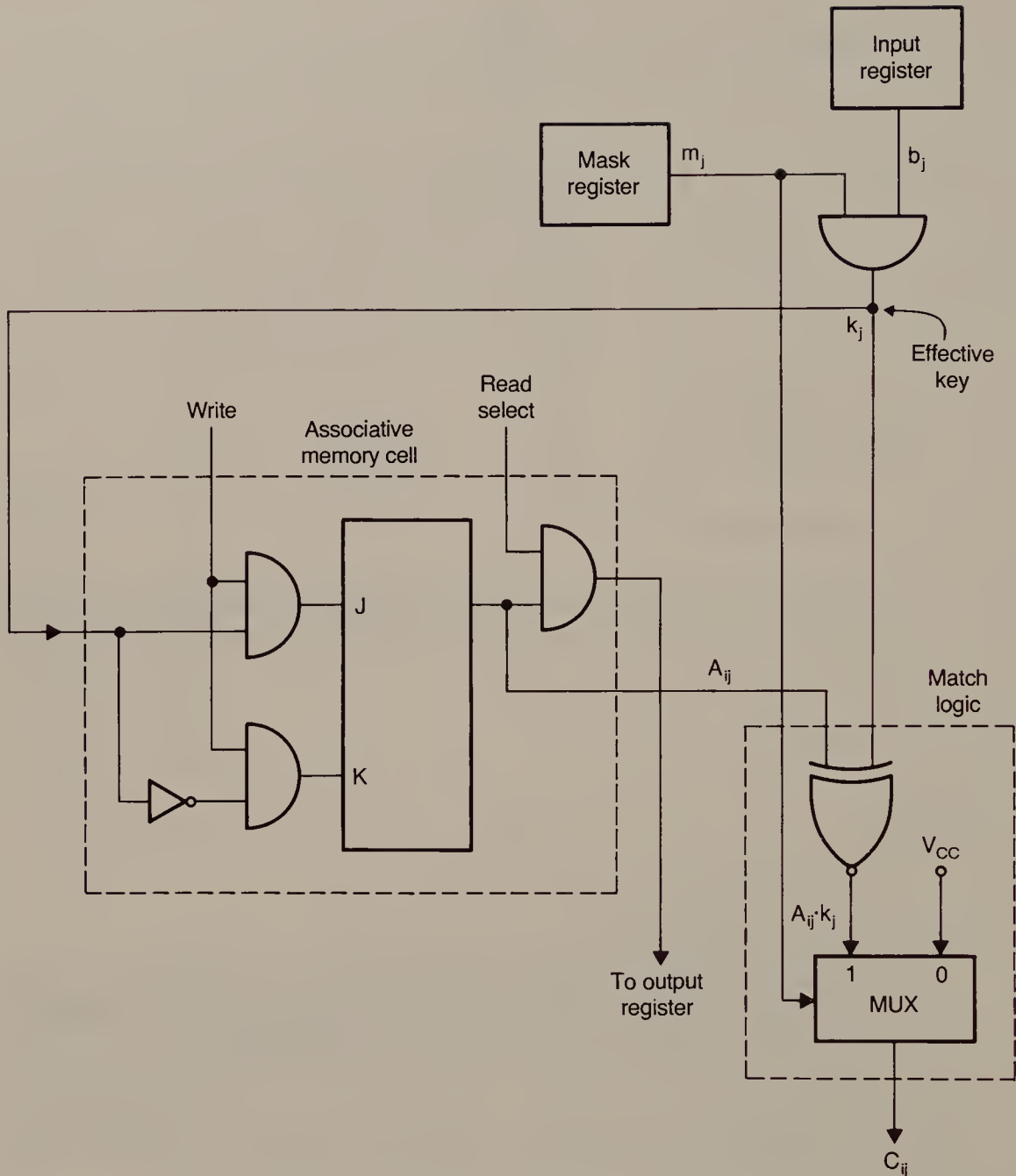


Figure 5.28 Hardware Structure of 1-bit Match Logic (M. Morris Mano, *Computer System Architecture*, © 1982, p. 492. Adapted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey.)

If duplication of key values in the associative array occurs, more than 1 bit of the match register can be set to 1. In general, associative memories are used for primary key retrieval. All keys held in the associative array are unique. Therefore, when there is no match, the contents of the match register will be reset to 0.

In Figure 5.30 the outputs of the match register are connected to an n -input OR gate, and the output of this OR gate is 1 only when there is a match.

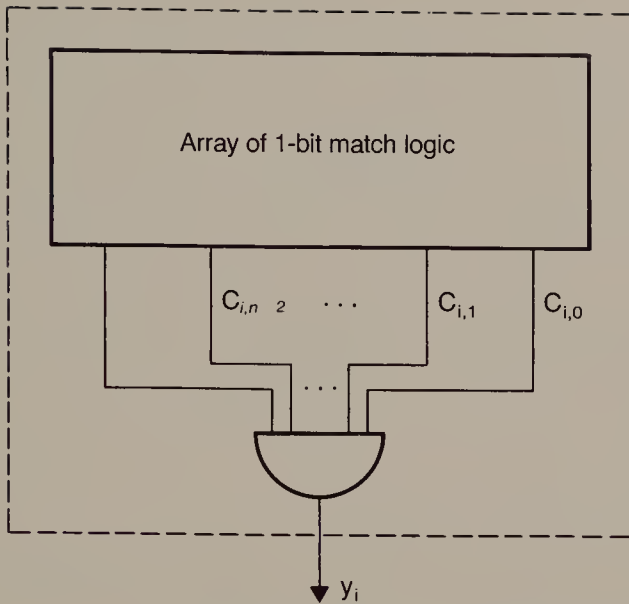


Figure 5.29 Hardware Structure of a Match Logic for an n -bit Word

7. *Selection logic:* Typically, this hardware computes the address of the matching word, retrieves it from the associative array, and routes it to the output register.

This circuit may be designed using an encoder, a decoder, a buffer register, and a multiplexer. The function of the encoder is to convert the match register output to a word address of the associative array. If the associative array has n words, the encoder will have n inputs, and thus it can generate one of the upper integer values of $\log_2 n$. The output of the encoder will be latched in a buffer register that acts as the MAR.

The output of this buffer register is given to the address decoder, which selects the desired cells from the associative array. It can be readily seen that the selection logic also initiates the read operation by enabling the read line. Thus, data read from the associative array is saved in the output register Z . For this associative memory organization, the address buffer register of the selection logic receives the address information from two different destinations:

- When the associative array is initialized via the normal write operation, the address needs to be specified. This is called the *external address*.
- In the event of a match, the data from the address generated by the match register-encoder combination needs to be retrieved.

Therefore, a multiplexer is included to distinguish these two situations. The memory “write” control is applied as the control input to this multiplexer, and the output of this multiplexer is in turn applied as the input to the address buffer register. When write control is 1, the multiplexer selects the external address as the input to the address buffer register. However, when the write control is 0, the multiplexer selects the encoder output as the input to the address buffer register.

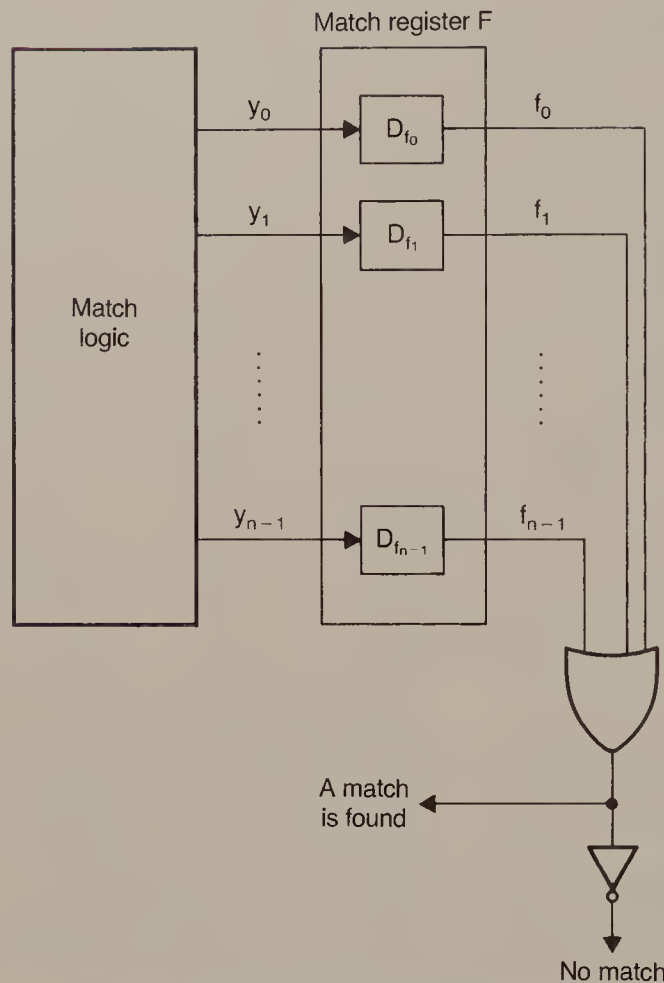


Figure 5.30 Interconnection between the Match Logic and Match Register

All technical points discussed so far are included in the complete design of a 2×2 word-organized associative memory depicted in Figure 5.31. The task of verifying the correctness of this design is left as an exercise.

Techniques used to initialize and update the contents of an associative memory are summarized next. There are two ways to initialize the associative memory with a completely new data set. In the first approach, there is one dedicated address line for each address. The information is written into the desired address, x , via a normal write operation after enabling the line.

The second approach involves the use of a decoder. If the associative memory has 2^k addresses, a k -to- 2^k decoder can be used to select the required address using only k bits. The latter approach is cost-effective because it needs only k address lines, whereas the former one needs 2^k dedicated address lines for an associative memory whose capacity is 2^k words.

Since an associative memory is referred to as content-addressable memory, the associative memory must be capable of being written into without actually specifying the address. To accomplish this, a tag register is included, which holds a 1-bit tag for each word in the associative memory. When the tag bit is a zero, it implies that a new word may be inserted into this address. If the tag bit is a 1, then this address contains

some data, and hence it is not possible to insert new data in this address. To insert a new word, first scan the tag register 1 bit at a time, and then the new word is inserted into the address with a 0 tag bit. In the preceding approach, a new word is inserted into the first available address.

The tag register offers an additional advantage. Any word can be made inactive by resetting the corresponding tag bit to 0. The status of the word can be determined by inspecting the tag bit. By examining the tag bit, the inactive words can be eliminated from the associative search.

Associative memories are desirable in high-speed applications. This concept is used to speed up dynamic address-translation schemes. This is explained in a later section of this chapter.

5.8 VIRTUAL MEMORY AND MEMORY-MANAGEMENT CONCEPTS

This section describes basic memory-management concepts. Topics include virtual memory, paging, segmentation, and address mapping schemes.

Virtual memory is the most fundamental concept implemented by a system that performs memory-management functions such as space allocation, program relocation, code sharing and protection.

The key idea behind this concept is to allow a user program to address more locations than those available in a physical memory. An address generated by a user program is called a *virtual address*. The set of virtual addresses constitutes the virtual address space. Similarly, the main memory of a computer contains a fixed number of addressable locations and a set of these locations forms the physical address space.

In the early days, when a programmer used to write a large program that could not fit into the main memory, it was necessary to divide the program into small portions so each one could fit into the primary memory. These small portions are called *overlays*. A programmer has to design overlays so they are independent of each other. Under these circumstances, one can successively bring each overlay into the main memory and execute them in a sequence. Although this idea appears to be simple, it increases the program-development time considerably.

However, in a system that uses a virtual memory, the size of the virtual address space is usually much larger than the available physical address space. In such a system, a programmer does not have to worry about overlay design, and thus a program can be written assuming a huge address space is available. In a virtual memory system, the programming effort can be greatly simplified. However, in reality, the actual number of physical addresses available is considerably less than the number of virtual addresses provided by the system. There should be some mechanism for dividing a large program into small overlays automatically. A virtual memory system is one that mechanizes the process of overlay generation by performing a series of mapping operations.

Sometimes the size of a virtual address space may be even less than the available physical address space. For example, in the PDP-11/45 minicomputer, the size of an address generated by a user program is 16 bits wide, allowing a user program to address only 64K different locations. However, the actual physical memory size can be up to

256K. To address 256K addresses, the length of the address field must be 18 bits. In this case, the mechanism that implements the virtual memory actually maps a 16-bit virtual address into an 18-bit physical address. This concept is known as *address extension*.

A virtual memory system may be configured in one of the following ways:

- Paging systems
- Segmentation systems

In a paging system, the virtual address space is divided into equal-size blocks called *pages*. Similarly, the physical memory is also divided into equal-size blocks called *frames*. The size of a page is the same as the size of a frame. The size of a page may be 512, 1024 or 2048 words.

In a paging system, each virtual address may be regarded as an ordered pair $\langle p, n \rangle$, where p is the page number and n is the word number within the page p . Sometimes the quantity n is referred to as the *displacement*, or *offset*. A user program may be regarded as a sequence of pages, and a complete copy of the program is always held in a backup store such as a drum or a disk. A page p of the user program can be placed in any available page frame p' of the main memory. A program may access a page if the page is in the main memory. In a paging scheme, pages are brought from secondary memory and are stored in main memory in a dynamic manner. All virtual addresses generated by a user program must be translated into physical memory addresses. This process is known as *dynamic address translation* and is shown in Figure 5.32.

When a running program accesses a virtual memory location $v = \langle p, n \rangle$, the mapping algorithm finds that the virtual page p is mapped to the physical frame p' . The physical address is then determined by appending p' to n .

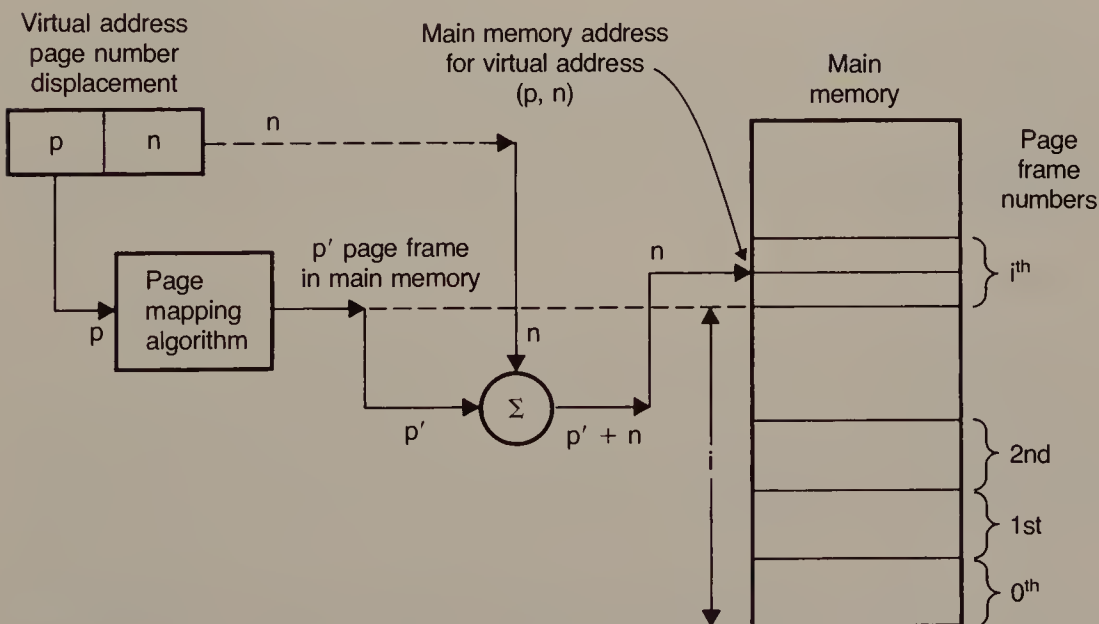


Figure 5.32 Paging Systems—Virtual versus Main Memory Mapping

This dynamic address translator can be implemented using a page table. In most systems, this table is maintained in the main memory. It will have one entry for each virtual page of the virtual address space. This is illustrated in the following example.



EXAMPLE 5.5

Design a mapping scheme with the following specifications:

- Virtual address space = 32K words
- Virtual memory size = 8K words
- Page size = 2K words
- Secondary memory address = 24 bits

SOLUTION

32K words can be divided into 16 virtual pages with 2K words per page, as follows:

VIRTUAL ADDRESS	PAGE NUMBER
0–2047	0
2048–4095	1
4096–6143	2
6144–8191	3
8192–10239	4
10240–12287	5
12288–14335	6
14336–16383	7
16384–18431	8
18432–20479	9
20480–22527	10
22528–24575	11
24576–26623	12
26624–28671	13
28672–30719	14
30720–32767	15

Since there are 8K words in the main memory, 4 frames with 2K words per frame are available:

PHYSICAL ADDRESS	FRAME NUMBER
0–2047	0
2048–4095	1
4096–6143	2
6144–8191	3

Since there are 32K addresses in the virtual space, 15 bits are required for the virtual address. Because there are 16 virtual pages, the page map table will contain 16 entries. The 4 most-significant bits of the virtual address can be used as an index to the page map table, and the remaining 11 bits of the virtual address can be used as the displacement to locate a word within the page frame. Each entry of the page table is 32 bits long. This obtained as follows:

1 bit for determining whether the page table is in main memory or not (residence bit).

2 bits for main memory page frame number.

24 bits for secondary memory address

5 bits for future use. (Unused at the moment)

32 bits total

The complete layout of the page table is shown in Figure 5.33.

Assume the virtual address generated is 0111 000 0010 1101. From this, compute the following:

Virtual page number = 7_{10}

Displacement = 43_{10}

From the page-map table entry corresponding to the address 0111, the page can be found in the main memory (since the page resident bit is 1).

The required virtual page is mapped to main memory page frame number 2. Therefore, the actual physical word is the 43rd word in the second page frame of the main memory.

So far, a page referenced by a program is assumed always to be found in the main memory. In practice, this is not necessarily true. When a page needed by a program is not assigned to the main memory, a page fault has occurred. A page fault is actually an interrupt, and when this interrupt occurs, control is transferred to a service routine of the operating system called the page-fault handler. The sequence of activities performed by the page-fault handler are summarized as follows:

- The secondary memory address of the required page p is located from the page table.

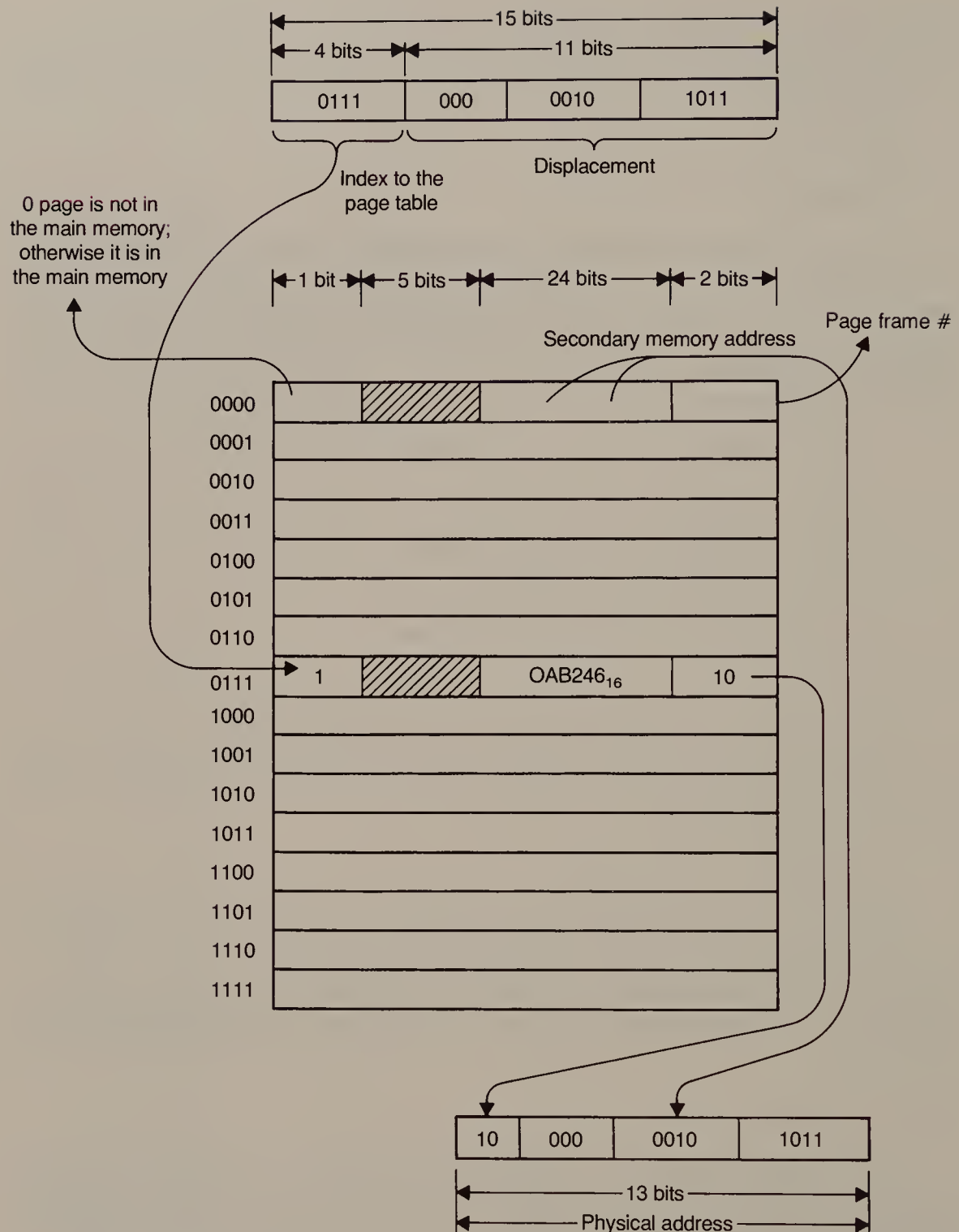


Figure 5.33 Mapping Scheme for the Pagiang System of Example 6.4

- Page p from the secondary memory is transferred into one of the available main memory frames by performing a block-move operation.
- The page table is updated by entering the frame number where page p is loaded and by setting the residence bit to 1 and the change bit to 0.

When a page-fault handler completes its task, control is transferred to the user program, and the main memory is accessed again for the required data or instruction. All these activities are kept hidden from a user. Pages are transferred to main memory only at specified times. The policy that governs this decision is known as the *fetch policy*. Similarly, when a page is to be transferred from the secondary memory to main memory, all frames may be full. At this point, one of the frames has to be removed from the main memory to provide room for an incoming page. The frame to be removed is selected using a replacement policy. The performance of a virtual memory system is dependent upon the fetch and replacement strategies. These issues are elaborated upon later.

The paging concept discussed so far is viewed as a one-dimensional technique because the virtual addresses generated by a program may linearly increase from 0 to some maximum value M . There are many situations where it is desirable to have a multidimensional virtual address space. This is the key idea behind segmentation systems.

Each logical entity such as a stack, an array, or a subroutine has a separate virtual address space in segmentation systems. Each virtual address space is called a *segment*, and each segment can grow from zero to some maximum value. Since each segment refers to a separate virtual address space, it can grow or shrink independently without affecting other segments.

In a segmentation system, the details about segments are held in a table called a *segment table*. Each entry in the segment table is called a *segment descriptor*, and it typically includes the following information:

- Segment base address b (starting address of the segment in the main memory)
- Segment length l (size of a segment)
- Segment presence bit
- Protection bits

From the structure of a segment descriptor, it is possible to create two or more segments whose sizes are different from one another. In a sense, a segmentation system degenerates to a paging system if all segments are of equal length. Because of this similarity, there is a close relationship between the paging and segmentation systems from the viewpoint of address translation.

A virtual address, V , in a segmentation system is regarded as an ordered pair $\langle s, d \rangle$, where s is the segment number and d is the displacement within segment s . The address translator for a segmentation system can be implemented using a segment table, and its organization is shown in Figure 5.34.

The mechanics of the address translation process is briefly discussed next.

Let V be the virtual address generated by the user program. First, the segment number field, s , of the virtual address V is used as an index to the segment table. The base address and length of this segment are b_s and l_s , respectively. Then, compare the displacement d of the virtual address V to the length of the segment l_s to make sure that the required address lies within the segment. Then, d must be less than or equal to l_s , and the comparator output z will be high. When $d \leq l_s$, the physical address is formed by adding b_s and d . From this physical address, data is retrieved and transferred to the

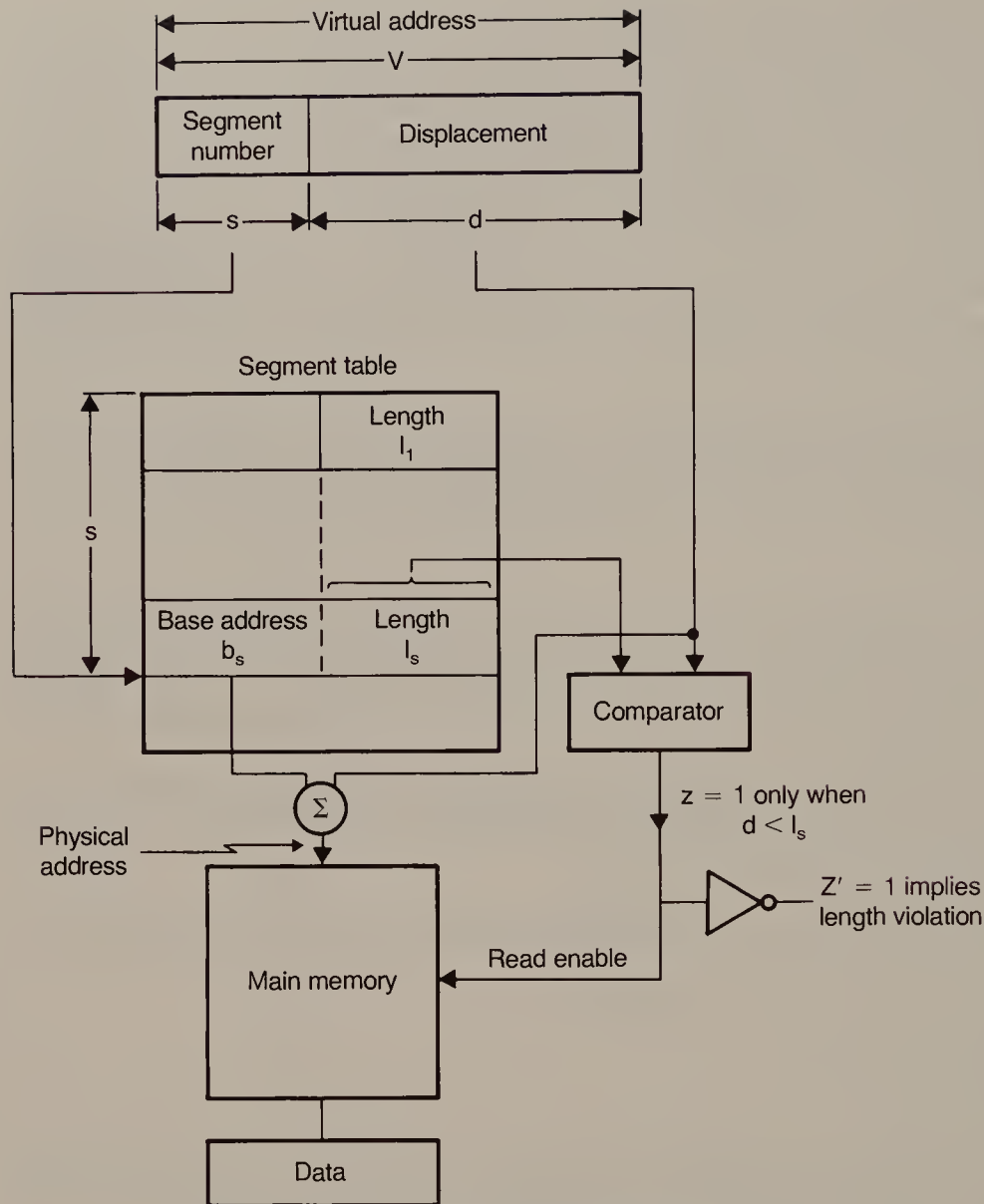


Figure 5.34 Address Translation in a Segmentation System

CPU. However, when $d > l_s$, the required address lies out of the segment range, and thus an *address out of range* trap will be generated. A *trap* is a nonmaskable interrupt with highest priority.

In a segmentation system, a segment needed by a program may not reside in main memory. This situation is indicated by a bit called a *valid bit*. A valid bit serves the same purpose as that of a page resident bit, and thus it is regarded as a component of the segment descriptor. When the valid bit is reset to 0, it may be concluded that the required segment is not in main memory.

This means that its secondary memory address must be included in the segment descriptor. Recall that each segment represents a logical entity. This implies that we can protect segments with different protection protocols based on the logical contents of

the segment. The following are the common protection protocols used in a segmentation system:

- Read only
- Execute only
- Read and execute only
- Unlimited access
- No access

Thus it follows that these protection protocols have to be encoded into some protection codes and these codes have to be included in a segment descriptor.

In a segmented memory system, when a virtual address is translated into a physical address, one of the following traps may be generated:

- *Segment fault trap* is generated when the required segment is not in the main memory.
- *Address violation trap* occurs when $d > l_s$.
- *Protection violation trap* is generated when there is a protection violation.

When a segment fault occurs, control will be transferred to the operating system. In response, the operating system has to perform the following activities:

- First, it finds the secondary memory address of the required segment from its segment descriptor.
- Next, it transfers the required segment from the secondary to primary memory.
- Finally, it updates the segment descriptor to indicate that the required segment is in the main memory.

After performing the preceding activities, the operating system transfers control to the user program and the data or instruction retrieval or write operation is repeated.

A comparison of the paging and segmentation systems is provided next. The primary idea behind a paging system is to provide a huge virtual space to a programmer, allowing a programmer to be relieved from performing tedious memory-management tasks such as overlay design. The main goal of a segmentation system is to provide several virtual address spaces, so the programmer can efficiently manage different logical entities such as a program, data, or a stack.

The operation of a paging system can be kept hidden in the user level. However, a programmer is aware of the existence of a segmented memory system.

To run a program in a paging system, only its current page is needed in the main memory. Several programs can be held in the main memory and can be multiplexed. The paging concept improves the performance of a multiprogramming system. In contrast, a segmented memory system can be operated only if the entire program segment is held in the main memory.

In a paging system, a programmer cannot efficiently handle typical data structures such as stacks or symbol tables because their sizes vary in a dynamic fashion during

program execution. Typically, large pages for a symbol table or small pages for a stack cannot be created. In a segmentation system, a programmer can treat these two structures as two logical entities and define the two segments with different sizes.

The concept of segmentation encourages people to share programs efficiently. For example, assume a copy of a matrix multiplication subroutine is held in the main memory. Two or more users can use this routine if their segment tables contain copies of the segment descriptor corresponding to this routine. In a paging system, this task cannot be accomplished so elegantly because the system operation is hidden from the user. This result also implies that in a segmentation system, the user can apply protection features to each segment in any desired manner. However, a paging system does not provide such a versatile protection feature.

Since page size is a fixed parameter in a paging system, a new page can always be loaded in the space used by a page being swapped out. However, in a segmentation system with uneven segment sizes, there is no guarantee that an incoming segment can fit into the free space created by a segment being swapped out.

In a dynamic situation, several programs may request more space, whereas some other programs may be in the process of releasing the spaces used by them. When this happens in a segmented memory system, there is a possibility that uneven-sized free spaces may be sparsely distributed in the physical address space. These free spaces are so irregular in size that they cannot normally be used to satisfy any new request. This is called an *external fragmentation*, and an operating system has to merge all free spaces to form a single large useful segment by moving all active segments to one end of the memory. This activity is known as *memory compaction*. This is a time-consuming operation and is a pure overhead. Since pages are of equal size, no external fragmentation can occur in a paging system.

In a segmented memory system, a programmer defines a segment, and all segments are completely filled.

The page size is decided by the operating system, and the last page of a program may not be filled completely when a program is stored in a sequence of pages. The space not filled in the last page cannot be used for any other program. This difficulty is known as *internal fragmentation*—a potential disadvantage of a paging system.

In summary, the paging concept simplifies the memory-management tasks to be performed by an operating system and is an elegant idea from the standpoint of an operating system. The segmentation concept offers a great appeal to programmers because it allows both protection and sharing of logical entities among a group of programmers.

To take advantage of both paging and segmentation, some systems use a different approach, in which these concepts are merged. In this technique, a segment is viewed as a collection of pages. The number of pages per segment may vary. However, the number of words per page still remains fixed. In this situation, a virtual address V is an ordered triple $\langle s, p, d \rangle$, where s is the segment number and p and d are the page number and the displacement within a page, respectively.

The following tables are used to translate a virtual address into a physical address:

- *Page table*: This table holds pointers to the physical frames.

- *Segment table*: Each entry in the segment table contains the base address of the page table that holds the details about the pages that belong to the given segment.

The address-translation scheme of such a paged-segmentation system is shown in Figure 5.35:

- First, the segment number s of the virtual address is used as an index to the segment table, which leads to the base address b_p of the page table.
- Then, the page number p of the virtual address is used as an index to the page table, and the base address of the frame number p' (to which the page p is mapped) can be found.
- Finally, the physical memory address is computed by adding the displacement d of the virtual address to the base address p' obtained before.

To illustrate this concept, the following numerical example is provided.



EXAMPLE 5–6

Assume the following values for the system of Figure 5.35:

- Length of the virtual address field = 32 bits
- Length of the segment number field = 12 bits
- Length of the page number field = 8 bits
- Length of the displacement field = 12 bits

Now, determine the value of the physical address using the following information:

- Value of the virtual address field = $000FA0BA_{16}$
- Contents of the segment table address $(000)_{16} = OFF_{16}$
- Contents of the page table address $(1F9_{16}) = AC_{16}$

SOLUTION

From the given virtual address, the segment table address is 000_{16} (three high-order hexadecimal digits of the virtual address). It is given that the contents of this segmentable address is OFF_{16} . Therefore, by adding the page number p (fourth and fifth hexadecimal digits of the virtual address) with OFF_{16} , the base address of the page table can be determined as:

$$OFF_{16} + FA_{16} = 1F9_{16}$$

From the hypothesis, the contents of the page table address $1F9_{16}$ is AC_{16} . Hence, the physical address can be obtained by adding the displacement (low-order three hexadecimal digits of the virtual address) with AC_{16} as follows:

$$AC000_{16}$$

$$\underline{000BA} \quad +$$

$$\underline{AC0BA}$$

In this addition, the displacement value OBA is sign-extended to obtain a 20-bit number that can be directly added to the base value p' . The same final answer can be obtained if p' and d are first concatenated. Thus, the value of the physical address is $AC0BA_{16}$.

The figures quoted in Example 5.6 are typical of IBM 360/67 computers. Since the virtual space of these machines use both paging and segmentation, it is called a *linear segmented virtual memory system*. In this system, the main memory is accessed three times to retrieve data (one for accessing the page table; one for accessing the segment table; and one for accessing the data itself).

Accessing the main memory is a time-consuming operation. To speed up the retrieval operation, a new approach is used. A small associative memory is used, called the translation lookaside buffer (TLB). The TLB stores the translation information for the 8 or 16 most recent virtual addresses. The organization of a address translation scheme that includes a TLB is shown in Figure 5.36.

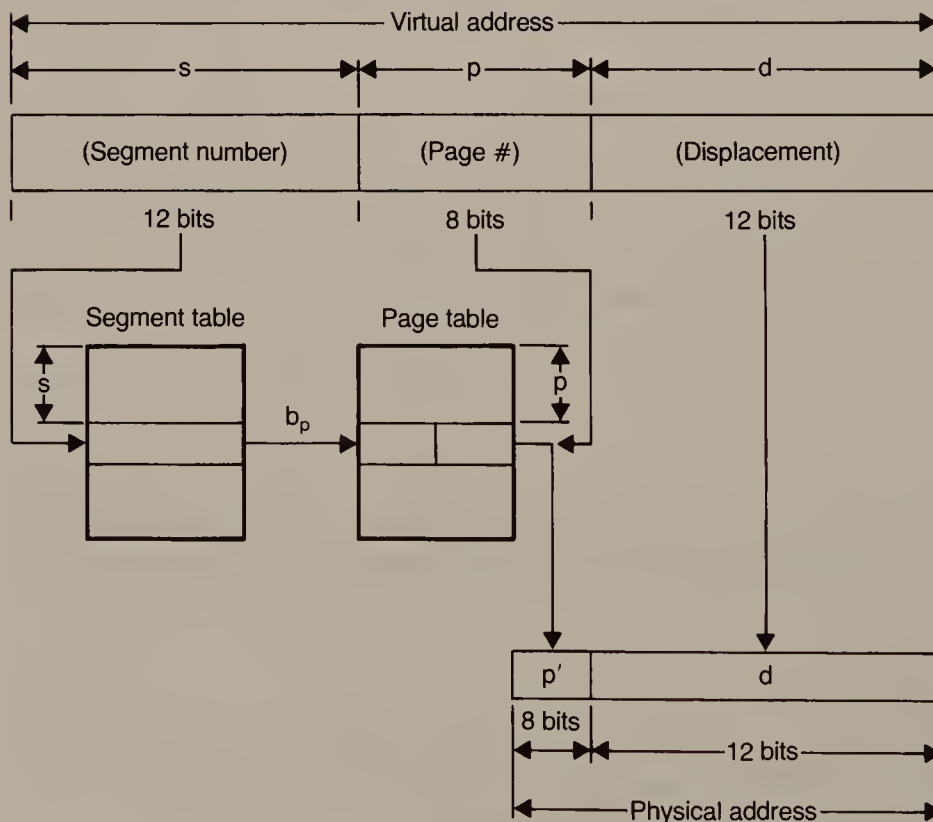


Figure 5.35 Address-translation Scheme for a Paged-segmentation System

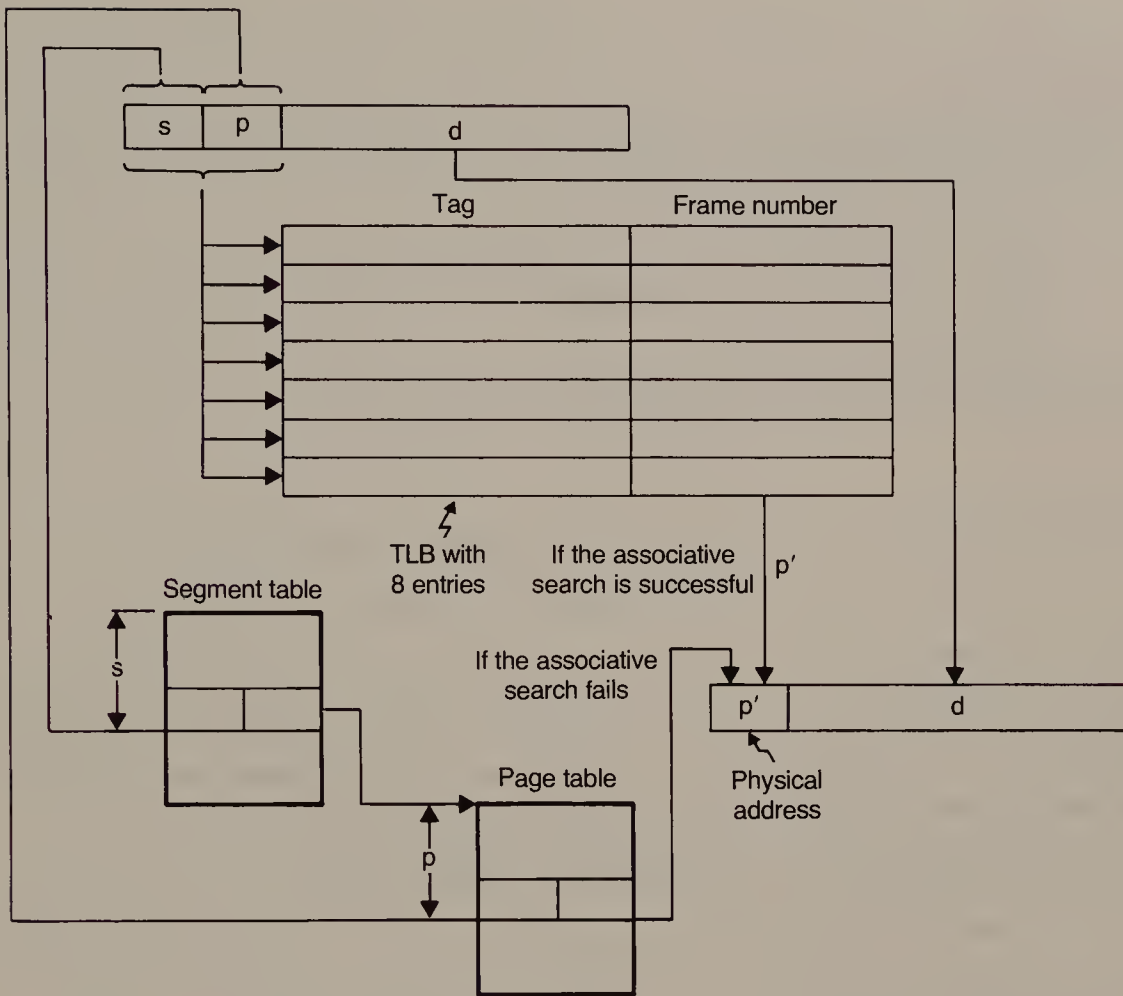


Figure 5.36 Address Translation Using a TLB

In this scheme, assume the TLB is capable of holding the translation information about the 8 most recent virtual addresses.

The pair $\langle s, p \rangle$ of the virtual address is known as a *tag*, and each entry in the TLB is of the form:

$\langle s, p \rangle$ or tag	Base address of the frame p'
----------------------------------	-----------------------------------

When a user program generates a virtual address, the $\langle s, p \rangle$ pair is associatively compared with all tags held in the TLB for a match. If there is a match, the physical address is formed by retrieving the base address of the frame p' from the TLB and concatenating this with the displacement **d**. However, in the event of a TLB miss, the physical address is generated after accessing the segment and page tables, and this information will also be loaded in the TLB. This ensures that translation information pertaining to a future reference is confined to the TLB. To illustrate the effectiveness of the TLB, the following numerical example is provided.

EXAMPLE 5-7

The following measurements are obtained from a computer system that uses a linear segmented memory system with a TLB:

- Number of entries in the TLB = 16
- Time taken to conduct an associative search in the TLB = 160 ns
- Main memory access time = 1 μ s

Determine the average access time assuming a TLB hit ratio of 0.75.

SOLUTION

In the event of a TLB hit, the time needed to retrieve the data is:

$$\begin{aligned} t_1 &= \text{TLB search time} + \text{time for one memory access} \\ &= 160 \text{ ns} + 1 \mu\text{s} \\ &= 1.160 \mu\text{s} \end{aligned}$$

However, when a TLB miss occurs, the main memory is accessed three times to retrieve the data. Therefore, the retrieval time t_2 in this case is

$$\begin{aligned} t_2 &= \text{TLB search time} + 3 (\text{time for one memory access}) \\ &= 160 \text{ ns} + 3 \mu\text{s} \\ &= 3.160 \mu\text{s} \end{aligned}$$

The average access time $\hat{t}$ is

$$\hat{t} = ht_1 + (1 - h)t_2$$

where h is the TLB hit ratio.

The average access time $\hat{t}$ is:

$$\begin{aligned} \hat{t} &= 0.75 (1.6) + 0.25 (3.160) \mu\text{sec} \\ &= 1.2 + 0.79 \mu\text{sec} \\ &= 1.99 \mu\text{sec} \end{aligned}$$

This example shows that the use of a small TLB significantly improves the efficiency of the retrieval operation (by 33%). There are two main reasons for this improvement. First, the TLB is designed using the associated memory. Second, the TLB hit ratio may be attributed to the locality of reference. Simulation studies indicate that it is possible to achieve a hit ratio in the range of 0.8 to 0.9 by having a TLB with 8 to 16 entries.

This concept is used in many mainframes, including the IBM 370/168 computer. In the IBM 370/168, the TLB can hold 128 entries. With the advent of IC technology, this technique is also gaining ground in the microprocessor world.

In a computer based on a linear segmented virtual memory system, the performance parameters such as storage use are significantly influenced by the page size p . For instance, when p is very large, excessive internal fragmentation will occur. If p is small, the size of the page table becomes large. This results in poor use of valuable memory space. The selection of the page size p is often a compromise. Different computer systems use different page sizes; Figure 5.37 summarizes the page sizes used in some popular large systems.

In the following, important memory-management strategies are described. There are three major strategies associated with the management:

- Fetch strategies
- Placement strategies
- Replacement strategies

All these strategies are governed by a set of policies conceived intuitively. Then they are validated using rigorous mathematical methods or by conducting a series of simulation experiments. A policy is implemented using some mechanism such as hardware, software, or firmware.

Fetch strategies deal with when to move the next page to main memory. Recall that when a page needed by a program is not in the main memory, a page fault occurs. In the event of a page fault, the page-fault handler will read the required page from the secondary memory and enter its new physical memory location in the page table, and the instruction execution continues as though nothing has happened.

In a virtual memory system, it is possible to run a program without having any page in the primary memory. In this case, when the first instruction is attempted, there is a page fault. As a consequence, the required page is brought into the main memory, where the instruction execution process is repeated again. Similarly, the next instruction may also cause a page fault. This situation is handled exactly in the same manner as described before. This strategy is referred to as *demand paging* because a page is brought in only when it is needed. This idea is useful in a multiprogramming environment because several programs can be kept in the main memory and executed concurrently.

Model	Page Size	Word Length
IBM 360/67	4096	32 bits
IBM 370/168	1024 or 512	32 bits
DEC 10	512	36 bits
VAX 11/780	512	32 bits

Figure 5.37 Page Sizes in Some Popular Computer Systems

However, this concept does not give best results if the page fault occurs repeatedly. For instance, after a page fault, the page-fault handler has to spend a considerable amount of time to bring the required page from the secondary memory. Typically, in a demand paging system, the effective access time $\hat{t}$ is the sum of the main memory access time t and μ , where μ is the time taken to service a page fault. Example 5.8 illustrates the concept.



EXAMPLE 5–8

- a) Assuming that the probability of a page fault occurring is p , derive an expression for $\hat{t}$ in terms of t , μ , and p .
- b) Suppose that $t = 500$ ns and $\mu = 30$ ms, calculate the effective access time a if it is given that on the average, one out of 200 references results in a page fault.

SOLUTION

- a) If a page fault does not occur, then the desired data can be accessed within a time t . (From the hypothesis the probability for a page fault not to occur is $1 - p$). If the page fault occurs, then μ time units are required to access the data. The effective access time is

$$\hat{t} = (1 - p)t + p\mu$$

- b) Since it is given that one out of every 200 references generates a page fault, $p = 1/200$. Using the result derived in part (a):

$$\begin{aligned}\hat{t} &= [(1 - 0.005) \times 0.5 + 0.005 \times 30,000] \mu s \\ &= [0.995 \times 0.5 + 150] \mu s \\ &= [0.4975 + 150] \mu s \\ &= 150.4975 \mu s\end{aligned}$$



In modern computer systems, the typical values of t and μ are:

$$500 \text{ ns} \leq t \leq 2 \mu s$$

$$5 \text{ ms} \leq \mu \leq 50 \text{ ms}$$

These parameters have a significant impact on the performance of a time-sharing system.

As an alternative approach, anticipatory fetching can be adapted. This conclusion is based on the fact that in a short period of time addresses referenced by a program are clustered around a particular region of the address space. This property is known as *locality of reference*. P. J. Denning's [8] working set model is one of the useful results related to this area.

The working set of a program $W(m, t)$ is defined as the set of m most recently needed pages by the program at some instant of time t . The parameter m is called the *window* of the working set. For example, consider the stream of references shown in Figure 5.38:

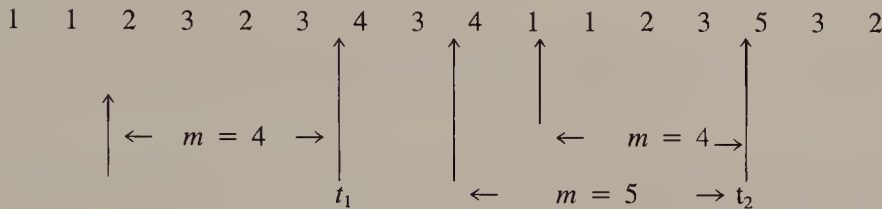


Figure 5.38 Stream of Page References

From this figure, determine that:

$$W(4, t_1) = \{2, 3\}$$

$$W(4, t_2) = \{1, 2, 3\}$$

$$W(5, t_2) = \{1, 2, 3, 4\}$$

In general, the cardinality of the set $W(0, t)$ is zero, and the cardinality of the set $W(\infty, t)$ is equal to the total number of distinct pages in the program. Since $m + 1$ most-recent page references include m most-recent page references:

$$\#[W(m + 1, t)] \subseteq \#[W(m, t)]$$

In this equation, the symbol $\#$ is used to indicate the cardinality of the set $W(m, t)$. When m is varied from 0 to ∞ , the quantity $\#W(m, t)$ continuously increases. $\#W(m, t)$ is a monotonically increasing function of m . The relationship between m and $\#W(m, t)$ is shown in Figure 5.39.

In practice, the working set of a program varies slowly with respect to time. Therefore, the working set of a program can be predicted ahead of time. For example, in a

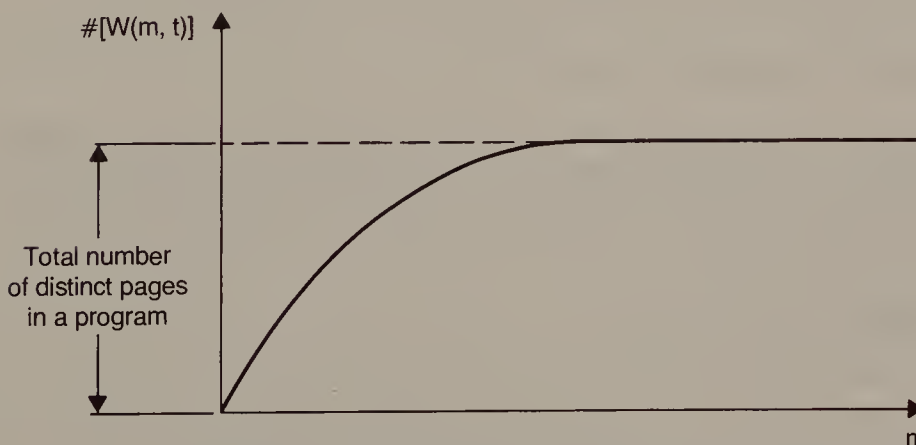


Figure 5.39 Relationship between One Cardinality of the Working Set and the Window Size m

multiprogramming system, when the execution of a suspended program is resumed, its present working set can be reasonably estimated based on the value of its working set at the time it was suspended. If this estimated working set is loaded, page faults are less likely to occur. This anticipatory fetching further improves the system performance because the working set of a program can be loaded while another program is being executed by the CPU.

However, the accuracy of a working set model depends on the value of m . Larger values of m result in more-accurate predictions. Typical values of m lie in the range of 5000 to 10,000.

To keep track of the working set of a program, the operating system has to perform time-consuming housekeeping operations. This activity is pure overhead, and thus the system performance may be degraded.

Placement strategies are significant with segmentation systems, and they are concerned with where to place an incoming program or data in the main memory. The following are the three widely used placement strategies:

- First-fit technique
- Best-fit technique
- Worst-fit technique

The first-fit technique places the program in the first available free block or hole that is adequate to store it. The best-fit technique stores the program in the smallest free hole of all the available holes able to store it. The worst-fit technique stores the program in the largest free hole.

The first-fit technique is easy to implement and does not have to scan the entire space to place a program. The best-fit technique appears to be efficient because it finds an optimal hole size. However, it has the following drawbacks:

- It is very difficult to implement.
- It may have to scan the entire free space to find the smallest free hole that can hold the incoming program. Therefore, it may be time-consuming.
- It has the tendency continuously to divide the holes into smaller sizes. These smaller holes may eventually become useless.

Worst-fit strategy is sometimes used when the design goal is to avoid creating small holes. In general, the operating system maintains a list known as the available space list (ASL) to indicate the free memory space. Typically, each entry in this list includes the following information:

- Starting address of the free block
- Size of the free block

After each allocation or release, the operating system updates the ASL.

In the following example, the mechanics of the various placement strategies presented earlier are explained.

EXAMPLE 5.9

The available space list of a computer memory system is specified as follows:

STARTING ADDRESS	BLOCK SIZE (IN WORDS)
100	50
200	150
450	600
1200	400

Determine the available space list after allocating the space for the stream of requests consisting of the following block sizes:

25, 100, 250, 200, 100, 150

- a) Use the first-fit method.
- b) Use the best-fit method.
- c) Use the worst-fit method.

SOLUTION

a) *First-fit method.* Consider the first request with a block size of 25. Examination of the block sizes of the available space list reveals that this request can be satisfied by allocating from the first available block. The block size (50) is the first of the available space list and is adequate to hold the request (25 blocks). Therefore, the first request with 25 blocks will be allocated from the available space list starting at address 100 with a block size of 50. Request 1 will be allocated starting at an address of 100 ending at an address $100 + 24 = 124$ (25 locations including 100). Therefore, the first block of the available space list will start at 125 with a block size of 25.

The starting address and block size of each request can be calculated similarly.

- b) *Best-fit method.* Consider request 1. Examination of the available block size reveals that this request can be satisfied by allocating from the first smallest available block capable of holding it. Request 1 will be allocated starting at address 100 and ending at 124. Therefore, the available space list will start at 125 with a block size of 25.
- c) *Worst-fit method.* Consider request 1. Examination of the available block sizes reveals that this request can be satisfied by allocating from the third block (largest) starting at 450. After this allocation the starting address of the available list will be 500 instead of 450 with a block size of $600 - 25 = 575$. Various results for all the other requests are shown in Figure 5.40.

Figure 5.40 Memory Map after Allocating Space for All Requests Given Example Using Different Placement Strategies

	Request 1 (25)		Request 2 (100)		Request 3 (250)		Request 4 (200)		Request 5 (100)		Request 6 (150)	
	Start address	Block size	Start address	Block size	Start address	Block size	Start address	Block size	Start address	Block size	Start address	Block size
First fit	125 200 450 1200	25 150 600 400	125 300 450 1200	25 50 600 400	125 300 700 1200	25 50 350 400	125 300 900 1200	25 50 150 400	125 300 1000 1200	25 50 50 400	125 300 1000 1350	25 50 50 250
Best fit	125 200 450 1200	25 150 600 400	125 300 450 1200	25 50 600 400	125 300 450 1450	25 50 600 150	125 300 650 1450	25 50 400 150	125 300 650 1550	25 50 400 50	125 300 800 1550	25 50 250 50
Worst fit	100 200 500 1200	50 150 575 400	100 200 600 1200	50 150 475 400	100 200 850 1200	50 150 225 400	100 200 850 1400	50 150 225 200	100 200 950 1400	50 150 125 200	100 200 850 1550	50 150 125 50

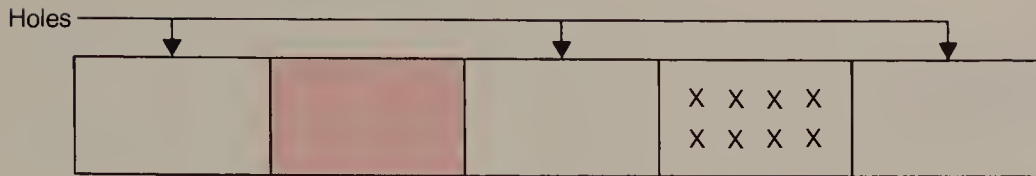
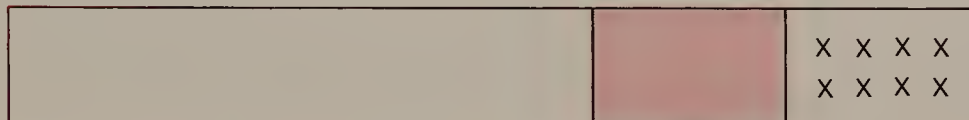


Figure 5.41a. Memory Status before Compaction



b. Memory Status after Compaction

In a multiprogramming system, programs of different sizes may reside in the main memory. As these programs are completed, the allocated memory space becomes free. It may happen that these unused free spaces, or holes, become available between two allocated blocks, or partitions. Some of these holes may not be large enough to satisfy the memory request of a program waiting to run. Thus valuable memory space may be wasted. One way to get around this problem is to combine adjacent free holes to make the hole size larger and usable by other jobs. This technique is known as *coalescing of holes*.

It is possible that the memory request made by a program may be larger than any free hole but smaller than the combined total of all available holes. If the free holes are combined into one single hole, the request can be satisfied. This technique is known as *memory compaction*. For example, the status of a computer memory before and after memory compaction is shown in Figure 5.41(a) and (b), respectively.

Placement strategies such as first-fit and best-fit are usually implemented as software procedures. These procedures are included in the operating system's software. The advent of high-level languages such as Pascal and C greatly simplify the programming effort because they support abstract data objects such as pointers. The available space list discussed in this section can easily be implemented using pointers.

The memory compaction task is performed by a special software routine of the operating system called a *garbage collector*. Normally, an operating system runs the garbage collector routine at regular intervals.

In a paged virtual memory system, when no frames are vacant, it is necessary to replace a current main memory page to provide room for a newly fetched page. The page for replacement is selected using some replacement policy. An operating system implements the chosen replacement policy. In general, a replacement policy is considered efficient if it guarantees a high hit ratio. The hit ratio h is defined as the ratio of the number of page references that did not cause a page fault to the total number of page references.

The simplest of all page replacement policies is the FIFO policy. This algorithm selects the oldest page (or the page that arrived first) in the main memory for replacement. The hit ratio h for this algorithm can be analytically determined using some arbitrary stream of page references as illustrated in the following example.

EXAMPLE 5.10

Consider the following stream of page requests.

2, 3, 2, 4, 6, 2, 5, 6, 1, 4, 6

Determine the hit ratio h for this stream using the FIFO replacement policy. Assume the main memory can hold 3 page frames and initially all of them are vacant.

SOLUTION

The hit ratio computation for this situation is illustrated in Figure 5.42.

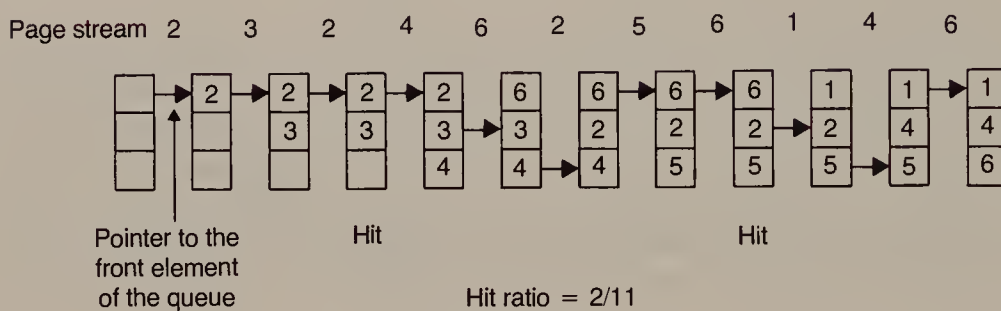


Figure 5.42 Hit Ratio Computation for Example 5.10

From Figure 5.42, it can be seen that the first two page references cause page faults. However, there is a hit with the third reference because the required page (page 2) is already in the main memory. After the first four references, all main memory frames are completely used. In the fifth reference, page 6 is required. Since this page is not in the main memory, a page fault occurs. Therefore, page 6 is fetched from the secondary memory. Since there are no vacant frames in the main memory, the oldest of the current main memory pages is selected for replacement. Page 6 is loaded in this position. All other data tabulated in this figure are obtained in the same manner. Since 9 out of 11 references generate a page fault, the hit ratio is $\frac{2}{11}$.

The primary advantage of the FIFO algorithm is its simplicity. This algorithm can be implemented by using a FIFO queue. FIFO policy gives the best result when page references are made in a strictly sequential order. However, this algorithm fails if a program loop needs a variable introduced at the beginning. Another difficulty with the FIFO algorithm is it may give anomalous results.

Intuitively, one may feel that an increase in the number of page frames will also increase the hit ratio. However, with FIFO, it is possible that when the page frames are increased, there is a drop in the hit ratio. Consider the following stream of requests:

1, 2, 3, 4, 5, 1, 2, 5, 1, 2, 3, 4, 5, 6, 5

Assume the main memory has 4 page frames; then using the FIFO policy there is a hit ratio of $\frac{4}{15}$. However, if the entire computation is repeated using 5 page frames, there is a hit ratio of $\frac{3}{15}$. This computation is left as an exercise. This surprising result was first observed by L. A. Belady in 1969[3], and for this reason is often referred to as Belady's anomaly.

Another replacement algorithm of theoretical interest is the optimal replacement policy. This idea is due to L. A. Belady:

When there is a need to replace a page, choose that page which may not be needed again for the longest period of time in the future.

The following numerical example explains this concept.

EXAMPLE 5.11

Using the optimal replacement policy, calculate the hit ratio for the stream of page references specified in Example 5.10. Assume the main memory has three frames and initially all of them are vacant.

SOLUTION

The hit ratio computation for this problem is shown in Figure 5.43.

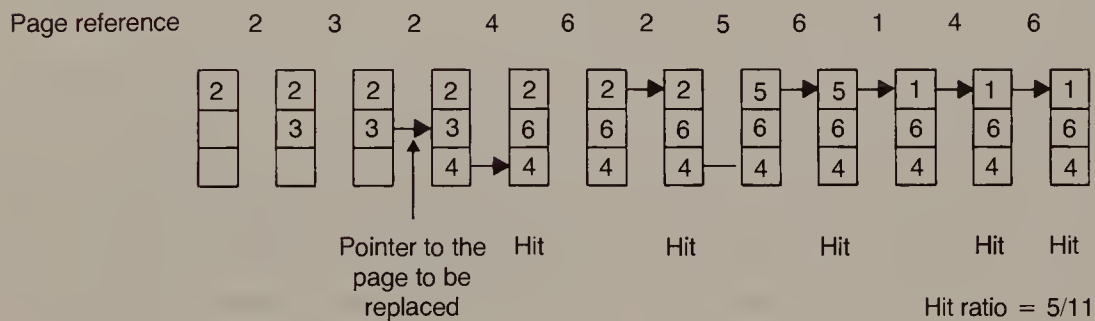


Figure 5.43 Hit Ratio Computation for Example 5.11

From Figure 5.43 it can be seen that the first two page references generate page faults. There is a hit with the sixth page reference, because the required page (page 2) is found in the main memory. Consider the fifth page reference. In this case, page 6 is required. Since this page is not in the main memory, it is fetched from the secondary memory. Now, there are no vacant page frames. This means that one of the current pages in the main memory has to be selected for replacement. Choose page 3 for replacement because this page is not used for the longest period of time. Page 6 is loaded into this position. Following the same procedure, other entries of this figure can be determined. Since 6 out of 11 page references generate a page fault, the hit ratio is $\frac{5}{11}$.

Now, let us summarize some important features of the LRU algorithm.

- In principle, the LRU algorithm is similar to the optimal replacement policy except that it looks backward on the time axis. Note that the optimal replacement policy works forward on the time axis.
- If the request stream is first reversed and then the LRU policy is applied to it, the result obtained is equivalent to the one that is obtained by the direct application of the optimal replacement policy to the original request stream.
- It has been proven that the LRU algorithm does not exhibit Belady's anomaly. This is because the LRU algorithm is a stack algorithm. A page-replacement algorithm is said to be a *stack algorithm* if the following condition holds:

$$P_t(n) \subset P_t(n + 1)$$

In the preceding relation the quantity $P_t(i)$ refers to the set of pages in the main memory whose total capacity is i frames at some time t . This relation is called the *inclusion property*. One can easily demonstrate that FIFO replacement policy is not a stack algorithm. This task is left as an exercise.

- The LRU policy can be easily implemented using a stack. Typically, the page numbers of the request stream are stored in this stack. Suppose that p is the page number being referenced. If p is not in the stack, then p is pushed into the stack. However, if p is in the stack, p is removed from the stack and placed on the top of the stack. The top of the stack always holds the most recently referenced page number, and the bottom of the stack always holds the least-recent page number. To see this clearly, consider Figure 5.45, in which a stream of page references and the corresponding stack instants are shown. The principal advantage of this approach is that there is no need to search for the page to be replaced because it is always the bottom most element of the stack. This approach can be implemented using either software or microcodes. However, this method takes more time when a page number is moved from the middle of the stack.
- Alternatively, the LRU policy can be implemented by adding an age register to each entry of the page table and a virtual clock to the CPU. The virtual clock is organized so that it is incremented after each memory reference. When a page is referenced, its age register is loaded with the contents of the virtual clock. The age register of a page holds the time at which that page was most recently referenced. The least-recent page is that page whose age register value is mini-

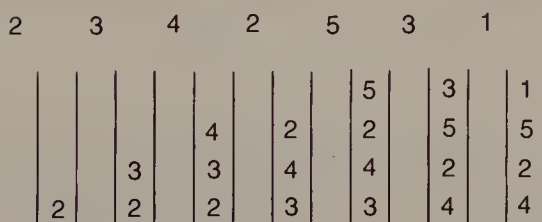


Figure 5.45 Implementation of the LRU Algorithm Using a Stack

mum. This approach requires an operating system to perform time-consuming housekeeping operations. Thus the performance of the system may be degraded.

- To implement these methods, the computer system must provide adequate hardware support. Incrementing the virtual clock using software takes more time. Thus the operating speed of the entire system is reduced. The LRU policy can not be implemented in systems that do not provide enough hardware support. To get around this problem, some replacement policy is employed that will approximate the LRU policy.
- The LRU policy can be approximated by adding an extra bit called an *activity bit* to each entry of the page table. Initially all activity bits are cleared to 0. When a page is referenced, its activity bit is set to 1. Thus this bit tells whether or not the page is used. Any page whose activity bit is 0 may be a candidate for replacement. However, the activity bit cannot determine how many times a page has been referenced.
- More information can be obtained by adding a register to each page table entry. To illustrate this concept, assume a 16-bit register has been added to each entry of the page table. Assume that the operating system is allowed to shift the contents of all the registers 1 bit to the right at regular intervals. With one right shift, the most-significant bit position becomes vacant. If it is assumed that the activity bit is used to fill this vacant position, some meaningful conclusions can be derived. For example, if the content of a page register is 0000_{16} , then it can be concluded that this page was not in use during the last 16 time-interval periods. Similarly, a value $FFFF_{16}$ for page register indicates that the page should have been referenced at least once in the last 16 time-interval periods. If the content of a page register is $FF00_{16}$ and the content of another one is $00FO_{16}$, the former was used more recently.
- If the content of a page register is interpreted as an integer number, then the least-recent page has a minimum page register value and can be replaced. If two page registers hold the minimum value, then either of the pages can be evicted, or one of them can be chosen on a FIFO basis.
- The larger the size of the page register, the more time is spent by the operating system in the update operations. When the size of the page register is 0, the history of the system can only be obtained via the activity bits. If the proposed replacement procedure is applied on the activity bits alone, the result is known as the *second-chance replacement* policy.
- Another bit called a *dirty bit* may be appended to each entry of the page table. This bit is initially cleared to 0 and set to 1 when a page is modified.
- This bit can be used in two different ways:
 - The idea of dirty bit reduces the swapping overhead because when the dirty bit of a page to be replaced is zero, there is no need to copy this page into the secondary memory, and it can be overwritten by an incoming page. A dirty bit can be used in conjunction with any replacement algorithm.

- A priority scheme can be set up for replacement using the values of the dirty and activity bits, as described next.

PRIORITY LEVEL	ACTIVITY BIT	DIRTY BIT	MEANING
0	0	0	Neither used nor modified.
1	0	1	Not recently used but modified.
2	1	0	Used but not modified.
3	1	1	Used as well as dirty.

Using the priority levels just described, the following replacement policy can be formulated: When it is necessary to replace a page, choose that page whose priority level is minimum. In the event of a tie, select the victim on a FIFO basis.

In some systems, the LRU policy is approximated using the least frequently used (LFU) and most frequently used (MFU) algorithms. A thorough discussion of these procedures is beyond the scope of this book.

- One of the major goals in a replacement policy is to minimize the page-fault rate. A program is said to be in a *thrashing* state if it generates excessive numbers of page faults. Replacement policy may not have a complete control on thrashing. For example, suppose a program generates the following stream of page references:

1, 2, 3, 4, 1, 2, 3, 4, 1, 2, 3, 4, . . .

If it runs on a system with three frames it will definitely enter into thrashing state even if the optimal replacement policy is implemented.

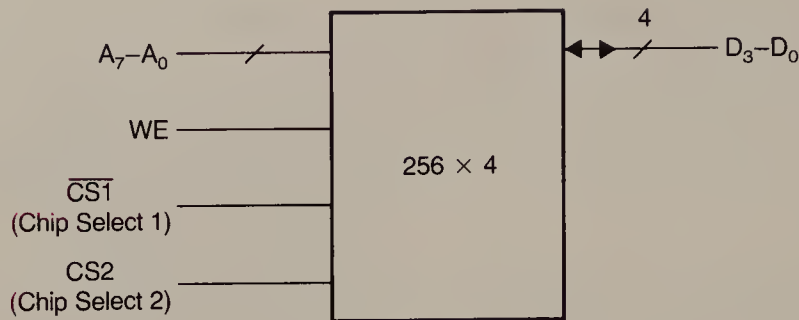
- There is a close relationship between the degree of multiprogramming and thrashing. In general, the degree of multiprogramming is increased to improve the CPU use. However, in this case more thrashing occurs. Therefore, to reduce thrashing, the degree of multiprogramming is reduced. Now the CPU utilization drops. CPU utilization and thrashing are conflicting performance issues.

QUESTIONS AND PROBLEMS

5.1 Match the devices shown in column 1 with the characteristics listed in column 2.

DEVICE	CHARACTERISTICS
a. Magnetic disk with multiple read/write heads	In-place update is not possible
b. MOS memory	DRO
c. IBM format magnetic tape	Volatile
d. Ferrite core	Direct access

5.2 The block diagram of a 256×4 RAM chip is shown next:



The memory chip is enabled only when $\overline{\text{CS1}} = 0$ and $\text{CS2} = 1$. Design a $2\text{K} \times 8$ RAM system using this chip as the building block. Draw a neat logic diagram of your implementation.

- 5.3** What is the basic difference between bank-switching and address-extension techniques?
- 5.4** Explain why floppy disks are popular with microprocessor systems.
- 5.5** Assume a disk has 7 tracks and each track has 12 sectors. To allow an operating system to allocate disk space, the details about the disk status are held in two different forms. In the first approach, all free sectors are indicated by the following table.

TRACK NUMBER	STARTING SECTOR NUMBER	NUMBER OF FREE SECTORS
0	2	3
1	3	2
1	8	3
2	4	3
3	5	4
4	3	5
5	3	7
6	4	2

For example, the first entry in this table says track 0, sector 2, and the next 2 sectors are free. In the second approach, called the bit-map method, a 12-bit binary pattern is used for each track (because there are 12 sectors per track). A 1 is entered if the sector is not free and a 0 is entered otherwise. For example, the bit map corresponding to track 0 of the table can be written as:

	0	1	2	3	4	5	6	7	8	9	10	11
Track 0	1	1	0	0	0	1	1	1	1	1	1	1

- a) Translate the tabular representation into bit-map representation.
- b) Discuss the advantages and disadvantages of these approaches.

5.6 Show that for an n -level hierarchical memory system the average access time

$$T(n) = \sum_{i=1}^n [H(S_n) - H(S_{i-1})]t_i$$

5.7 The details of a four-level hierarchical memory system are summarized as follows:

LEVEL	COMPONENT TYPE	ACCESS TIME	HIT RATIO
1	ECL main memory	$t_1 = 100 \text{ ns}$	$H(S_1) = 0.6$
2	Core memory	$t_2 = 1 \text{ ms}$	$H(S_2) = ?$
3	Disk	$t_3 = 0.5 \text{ ms}$	$H(S_3) = 0.9$
4	Tape	$t_4 = 50 \text{ ms}$	$H(S_4) = 1$

Determine $H(S_2)$ if the average access time of this memory hierarchy is found to be 5.1005 ms.

- 5.8** Explain the technical reasons that support using a fast cache between the CPU and the main memory.
- 5.9** Mention two advantages and disadvantages of the direct mapping technique.
- 5.10** A typical computer system has a 32K main memory and a 4K fully associative cache memory. The cache block size is 8 words. The access time for the main memory is 10 times that of the cache memory.
 - a) How many hardware comparators are needed?
 - b) What is the size of the tag field?
 - c) If a direct mapping scheme were used instead, what would be the size of the tag field?
 - d) Suppose the access efficiency is defined as the ratio of the average access time with a cache to the average access time without a cache, determine the access efficiency assuming a cache hit ratio h of 0.9.
 - e) If the cache access time is 200 nanoseconds, what hit ratio would be required to achieve an average access time equal to 500 nanoseconds?
- 5.11** A set associative cache has a total of 64 blocks divided into sets of 4 blocks each.
 - a) Main memory has 1024 blocks with 16 words per block. How many bits are needed in each of the tag, set, and word fields of the main memory address?

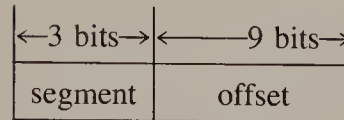
- b)** A computer system has 32K words of MM and a set associative cache. The block size is 16 words and the TAG field of the main memory address is 5-bit wide. If the same cache were direct mapped, the main memory will have a 3-bit TAG field. How many words are there in the cache? How many blocks are there in a cache set?
- 5.12** In Example 5.4, determine the hit ratio if write cycles are also taken into consideration.
- 5.13** Under what condition does the set associative mapping method reduce to each of the following?
- a)** Direct mapping
 - b)** Fully associative mapping
- 5.14** What are salient features of write-through and write-back procedures?
- 5.15** What is the importance of the valid bit in a cache memory organization?
- 5.16** Discuss the main features of 68020 on-chip cache.
- 5.17** What are the principal advantages and disadvantages of an associative memory?
- 5.18** Explain clearly the difference between segmentation and paging.
- 5.19** Can you think of a situation where it would be advantageous to define a virtual memory that is smaller than available physical memory?
- *5.20** Consider a logical address space of 8 pages of 1024 words mapped onto a physical memory of 32 frames
- a)** How many bits are in a logical address?
 - b)** How many bits are in a physical address?
- **5.21** Consider a typical segment table as shown.

SEGMENT NUMBER	SEGMENT BASE ADDRESS	SEGMENT LENGTH
0	0D1 ₁₆	258 ₁₆
1	8FC ₁₆	00E ₁₆
2	05A ₁₆	064 ₁₆
3	052F ₁₆	244 ₁₆

* J. Peterson and A. Silberschatz, *Operating System Concepts* © 1983, Addison-Wesley Publishing Co., Reading, Massachusetts, p. 183. Reprinted with permission.

** J. Peterson and A. Silberschatz, *Operating System Concepts* © 1983, Addison-Wesley Publishing Co., Reading, Massachusetts, p. 185. Reprinted with permission.

The virtual address format is:



What are the physical addresses for the following logical addresses?

- a)** $1FE_{16}$ **b)** $20A_{16}$ **c)** $20B_{16}$ **d)** 590_{16} **e)** 790_{16}

5.22 Design a direct mapped virtual memory system with the following specifications:

- Size of the virtual address space = 64K
- Size of the physical address space = 8K
- Page size = 512 words
- Total length of a page table entry = 24 bits

5.23 A virtual memory system has the following specifications:

- Size of the virtual address space = 64K
- Size of the physical address space = 4K
- Page size = 512

From the page table the following mapping is recognized:

VIRTUAL PAGE NUMBER	PHYSICAL PAGE FRAME NUMBER
0	0
3	1
7	2
4	3
10	4
12	5
24	6
30	7

- a)** Find all virtual addresses that will generate a page fault.
- b)** Compute the main memory address for the following virtual addresses:

24, 3784, 10250, 30780

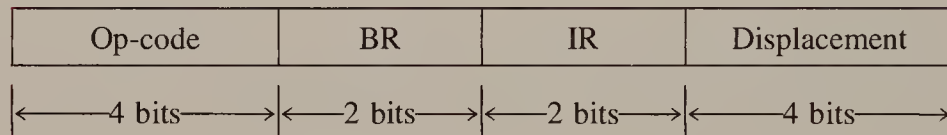
***5.24** Consider a demand paging system with a paging drum with an average latency and transfer time of 25 microseconds and 10 microseconds, respectively. Ad-

* J. Peterson and A. Silberschatz, *Operating System Concepts* © 1983, Addison-Wesley Publishing Co., Reading, Massachusetts, p. 238. Adapted by permission.

addresses are translated through a page table in main memory with an access time of 1 microseconds per memory access. Each reference through the page table takes two accesses. To improve this time, an associative memory has been added to hold 16 most recent page table entries. This associative memory is known as translation look aside buffer (TLB), and finding a page table entry in the TLB takes 160 nanoseconds, if it is there. If the page table entry is in the TLB, only one memory reference is needed.

Assuming that 80% of the access are in the associative memory and that of the remaining, 10% cause page fault, what is the effective access time?

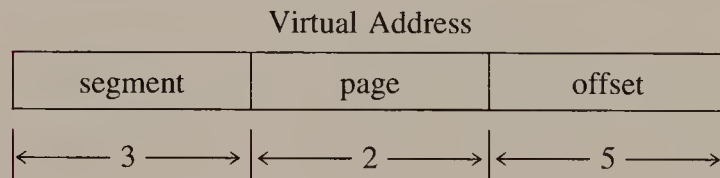
- 5.25** Assume a computer has a segmented memory with paged segments. The instruction format of this machine is as shown:



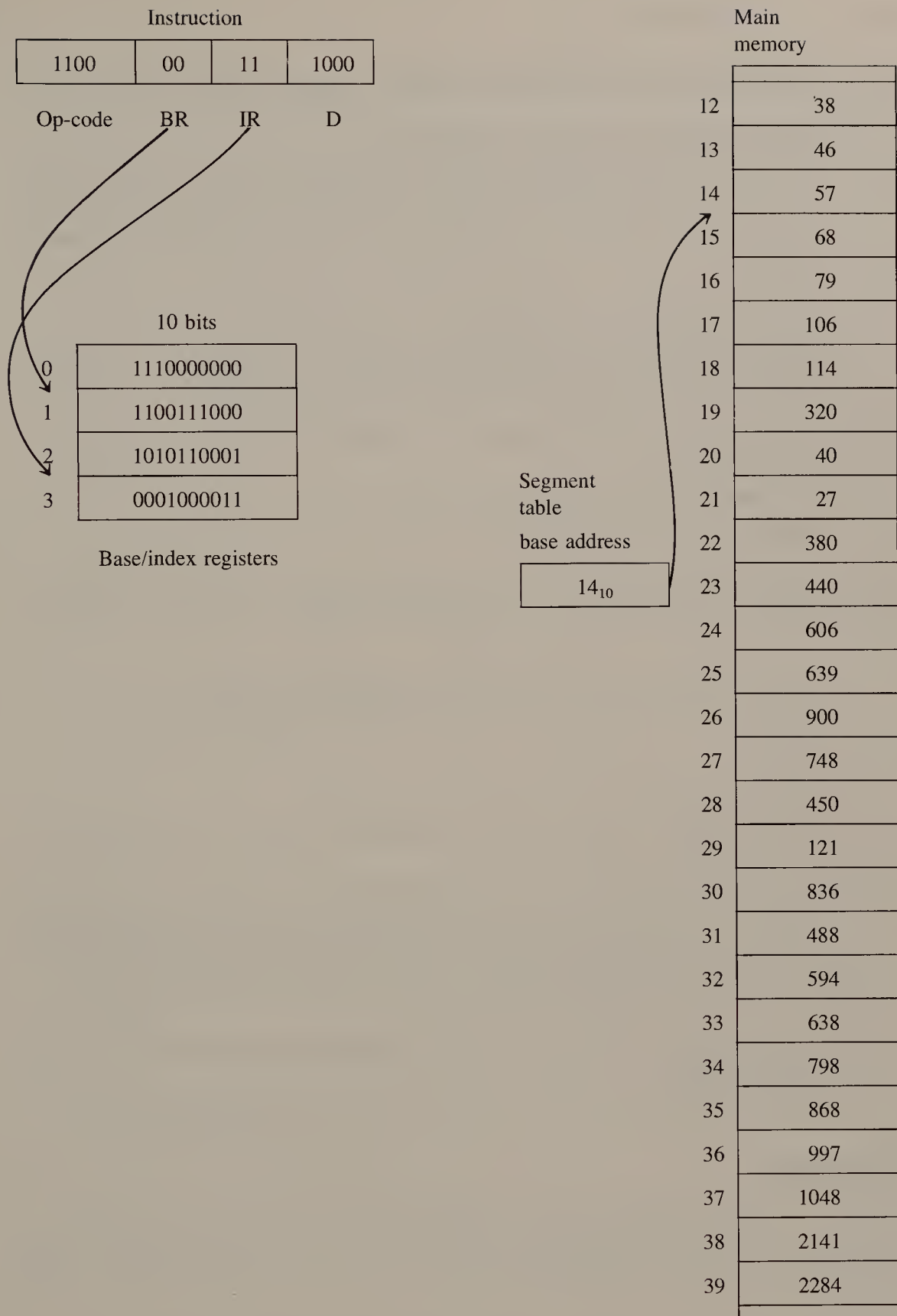
This format has the following fields:

- Op-code field
- 2-bit base register field BR
- 2-bit index register field IR
- 4-bit displacement field

The contents of the specified base and index registers are added with the displacement to produce a virtual address whose format is shown next:



The virtual address is translated into a physical address by means of segment and page tables, which are stored in the main memory. The segment table entry contains the starting address of its page table and the page table entry contains the address of the location which holds the page frame number. The segment table base address register contains the start address of the segment table. The final physical address is the sum of the page table entry and the offset from the virtual address. Consider the following situation:



- Compute the physical address needed by the given situation
- How many two-operand summations are required to compute one physical address?

***5.26** The following are the measurements taken in a demand paging system

- Main memory access time = 700 nanoseconds.
- Time taken to service a page fault if an empty page is available or the page to be replaced is clean (i.e., the page is not modified while it was in the main memory and so there is no need to write it in the back up store) = 15 microseconds.
- Time taken to replace a dirty page = 50 microseconds.

Assume the probability for a page to be dirty is 0.70. Calculate the effective access time when every thousand references causes a page fault.

5.27 Come up with an available space list and a stream of requests such that each of the following holds.

- a)** First-fit works but not best-fit.
- b)** Best-fit works but not first-fit.

5.28 Assume a main memory has 4 page frames and initially all page frames are empty. Consider the following stream of references;

1, 2, 3, 4, 5, 1, 2, 6, 1, 2, 3, 4, 5, 6, 5

Calculate the hit ratio if the replacement policy used is as follows.

- a)** FIFO
- b)** LRU

5.29 Repeat Problem 5.28 when the main memory has 5 page frames instead of 4. Comment on your results.

5.30 Consider the stream of references given in Problem 5.28. Plot a graph between the hit ratio and the number frames (f) in the main memory after computing the hit ratio for all values f in the range of 1 to 8. Assume LRU policy is used. (*Hint: Use the stack algorithm.*)

5.31 Using a counter example, show that FIFO replacement policy does not satisfy inclusion properly.

REFERENCES

1. Baer, J. *Computer Systems Architecture*. Rockville, Md.: Computer Science Press, 1978.
2. Belady, L. A. "A Study of Replacement Algorithms for a Virtual Storage Computer." *IBM Systems Journal* 5, no. 2 (1966) 78–101.

* J. Peterson, and A. Silberschatz, *Operating System Concepts* © 1983, Addison-Wesley Publishing Co., Reading, Massachusetts, p. 239. Adapted by permission.

3. Belady, L. A., and C. J. Kuehner. "Dynamic Space Sharing in Computer Systems", *CACM* 12, no. 5 (May 1969) 282–288.
4. Bell, J. D., D. Casanet, and C. G. Bell. "An Investigation of Alternative Cache Organization." *IEEE Transactions on Computers*, C-23 (April 1974) 346–351.
5. Calingaert, P. *Operating System Elements—A user perspective*. Englewood Cliffs, N.J.: Prentice-Hall, 1982.
6. Chow, C. K. "An Optimization of Storage Hierarchies." *IBM Journal of Research and Development* (May 1974) 194–203.
7. Denning, P. J. "Virtual Memory." *Computing Surveys* 2, no. 3 (September 1970) 153–189.
8. Denning, P. J. "Working Sets Past and Present." *IEEE Transactions on Software Engineering* SE-6, no. 1 (January 1980) 64–84.
9. Hamacher, V. C., Z. G. Vranesic, and S. G. Zaky. *Computer Organization*. New York: McGraw-Hill, 1984.
10. Hayes, J. F. *Computer Architecture and Organization*. New York: McGraw-Hill, 1978.
11. Hwang, K., and F. A. Briggs. *Computer Architecture and Parallel Processing*. New York: McGraw-Hill, 1984.
12. Knuth, D. E. *The Art of Computer Programming*. Vol. 1, *Fundamental Algorithms*. Reading, Mass.: Addison-Wesley, 1973.
13. Levy, H. M. and R. H. Eckhouse, Jr. *Computer Programming and Architecture—The VAX-11*, Boston, MA: Digital Press, 1980.
14. Macgregor, D., D. Mothersole, and B. Moyer; "The Motorola 68020." *IEEE Micro* 4, no. 4 (August 1984) 101–118.
15. Madnick, S. E., and J. J. Donovan. *Operating Systems*. New York: McGraw-Hill, 1974.
16. Peterson, J., and A. Silberschatz. *Operating System Concepts*. Reading, Mass: Addison-Wesley, 1983.
17. Shaw, A. C. *The Logical Design of Operating Systems*. Englewood Cliffs, N.J.: Prentice-Hall, 1974.
18. Stone, H. S. ed. *Introduction to Computer Architecture*. SRA (1980).
19. Tanenbaum, A. S. *Structured Computer Organization*. Englewood Cliffs, N.J.: Prentice-Hall, 1984.
20. Watson, R. W. *Timesharing System Design Concepts*. New York: McGraw-Hill, 1970.
21. Wolman, E. "A Fixed Optimum Cell-Size for Records of Various Lengths." *CACM* 12, no. 1 (January 1965) 53–70.

CHAPTER 6

Input/Output (I/O)

This chapter describes the basic input and output techniques used by a computer. Data transfer between the computer and an external device takes place through the I/O mechanism. The general characteristics of I/O are described first. These features are then related by the I/O techniques used by the popular 16-bit Motorola MC68000 microprocessor.

6.1 BASIC CONCEPTS*

One communicates with a computer system via the I/O devices interfaced to it. The user can enter programs and data using the keyboard on a terminal and execute the programs to obtain results. Therefore, the I/O devices connected to a computer system provide an efficient means of communication between the computer and the outside world. These I/O devices are commonly called *peripherals* and include keyboards, CRT displays, printers, disks, and teletypewriters.

The characteristics of the I/O devices are normally different from those of the computer. For example, the speed of operation of the peripherals is usually slower

* *Microcomputer Theory and Applications with the Intel SDK-85*. Rafiquzzaman, Second Edition, Wiley, 1987. Reprinted with permission.

compared to the computer, and the word length of the computer may be different from the data format of the peripheral devices. To make the characteristics of the I/O devices compatible with those of the computer, interface hardware circuitry between the computer and I/O devices is necessary. Interfaces provide all input and output transfers between the computer and peripherals by using an I/O bus. An I/O bus carries three types of signals: device address, data, and command.

For large computers, a separate intelligent I/O processor (IOP) or data channel is provided to route all I/O transfers. To make small computer systems inexpensive, a separate interface rather than a smart I/O processor is provided with each I/O device. I/O processors control all major I/O functions and relieve the computer of these tasks.

The CPU uses the I/O bus when it executes an I/O instruction. A typical I/O instruction has three fields. When the computer executes an I/O instruction, the control unit decodes the op-code field and identifies it as an I/O instruction. The CPU then places the device address and command from respective fields of the I/O instruction on the I/O bus. The interfaces for various devices connected to the I/O bus decode this address, and appropriate interface is selected. The identified interface decodes the command lines and determines the function to be performed. Typical functions include receiving data from an input device into the CPU or sending data to an output device from the CPU.

In a typical computer system, the user gets involved with two types of I/O devices: physical I/O and virtual I/O. When the computer has no operating system, the user must work directly with physical I/O devices and perform detailed I/O design.

There are three ways of transferring data between the computer and a physical I/O device:

- Programmed I/O
- Interrupt I/O
- Direct memory access (DMA)

The computer executes a program to communicate with an external device via a register called the I/O port for programmed I/O.

An external device requests the computer to transfer data by activating a signal on the computer's interrupt line during interrupt I/O. In response, the computer executes a program called the *interrupt-service routine* to carry out the function desired by the external device.

Data transfer between the computer's memory and an external device occurs without CPU involvement with direct memory access.

For a computer with an operating system, the user works with virtual I/O devices. The user does not have to be familiar with the characteristics of the physical I/O devices. Instead, the user performs data transfers between the computer and the physical I/O devices indirectly by calling the I/O routines provided by the operating system using virtual I/O instructions.

Basically, an operating system serves as an interface between the user programs and actual hardware. The operating system facilitates the creation of many logical or virtual I/O devices, and allows a user program to communicate directly with these logical devices. For example, a user program may write its output to a virtual printer. In reality, a virtual printer may refer to a block of disk space. When the user program

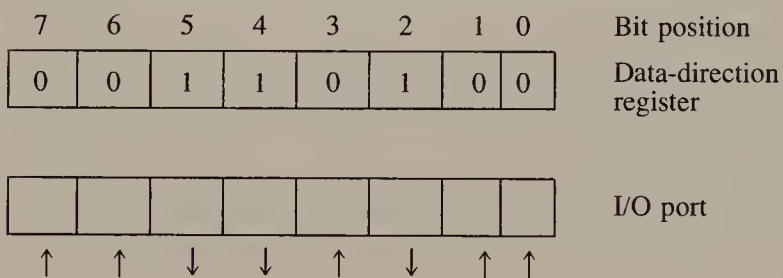
terminates, the operating system may assign one of the available physical printers to this virtual printer and monitor the entire printing operation. This concept is known as *spooling* and improves the system throughout by isolating the fast processor from direct contact with a slow printing device. A user program is totally unaware of the logical-to-physical device-mapping process. There is no need to modify a user program if a logical device is assigned to some other available physical device. This approach offers greater flexibility over the conventional hardware-oriented techniques associated with physical I/O.

6.2 PROGRAMMED I/O

As described earlier, the computer communicates with an external device via one or more registers called I/O ports using programmed I/O. These I/O ports are occasionally fabricated by the manufacturer in the same chip as the memory chip to achieve minimum chip count for small system applications. For example, the Intel 8355/8755 contains a 2K bytes of ROM/EPROM with two I/O ports. The Motorola 6846 has 2K bytes of ROM and an 8-bit I/O port.

I/O ports are usually of two types. For one type, each bit in the port can be individually configured as either input or output. For the other type, all bits in the port can be set up as either all parallel input or output bits. Each port can be configured as an input or output port by another register called the *command*, or *data-direction* register. The port contains the actual input or output data. The data-direction register is an output register and can be used to configure the bits in the port as inputs or outputs.

Each bit in the port can be set up as an input or output by writing a 0 or a 1 in the corresponding bit of the data-direction register. As an example, if an 8-bit data-direction register contains 34_{16} , then the corresponding port is defined as follows:



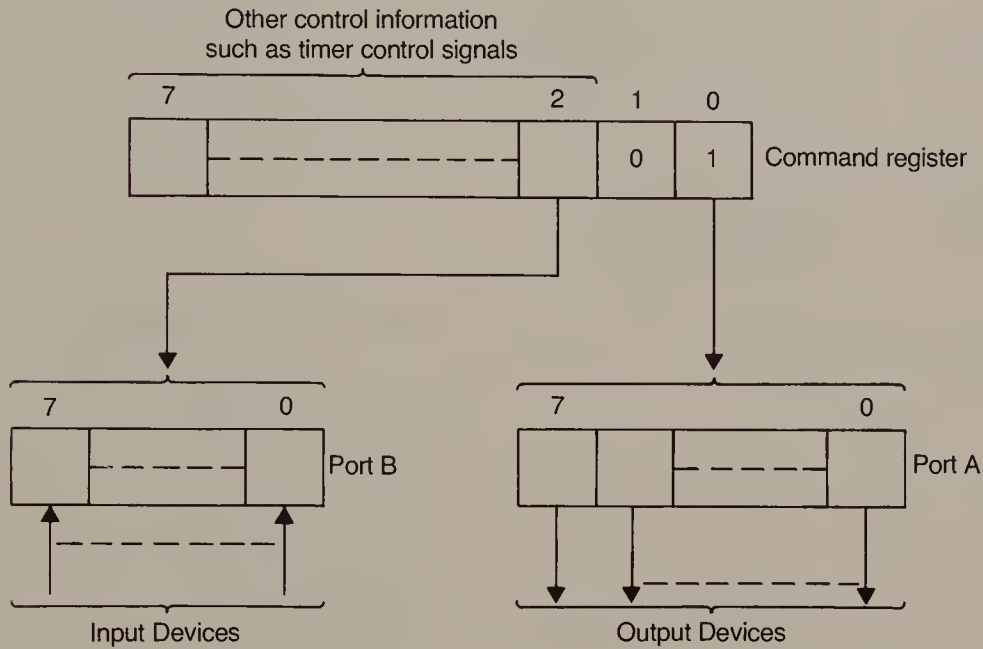
In the preceding example, since 34_{16} ($0011\ 0100_2$) is sent as an output into the data-direction register, bits 0, 1, 3, 6 and 7, of the port are set up as inputs, and bits 2, 4, and 5 of the port are defined as outputs. The computer can then send outputs to external devices, such as LEDs, connected to bits 2, 4, and 5 through a proper interface. Similarly, the computer can input the status of external devices, such as switches, through bits 0, 1, 3, 6, and 7. To input data from the input switches, the 8-bit computer assumed here inputs the complete byte, including the bits to which LED's are connected. While receiving input data from an I/O port, however, the computer places a value, probably 0, at the bits configured as outputs, and the program must interpret them as "don't cares." At the same time, the computer's outputs to bits configured as inputs are disabled.

For parallel I/O, there is only one data-direction register, known as the *command*

register for all ports. A particular bit in the command register configures all bits in the port as either inputs or outputs.

Consider two I/O ports in an I/O chip along with one command register. Assume a 0 or a 1 in a particular bit portion defines all bits of ports A or B as inputs or outputs.

For example,



Some I/O ports are called *handshake ports*. Data transfer occurs via these ports through exchanging of control signals between the I/O controller and an external device.

6.3 STANDARD I/O VERSUS MEMORY-MAPPED I/O*

I/O ports are addressed using either standard I/O or memory-mapped I/O techniques. The standard I/O, also called isolated I/O, uses the $\text{I/O}/\overline{\text{M}}$ control pin on the CPU chip. The processor outputs a HIGH on this pin to indicate to memory and the I/O chips that an I/O operation is taking place. A LOW output from the processor to this pin indicates a memory operation. Execution of IN or OUT instructions makes the $\text{I/O}/\overline{\text{M}}$ HIGH, whereas, memory-oriented instructions, such as LDA and STA, drive the $\text{I/O}/\overline{\text{M}}$ to LOW. In standard I/O, the processor uses the $\text{I/O}/\overline{\text{M}}$ pin to distinguish between I/O memory. For 8-bit processors, an 8-bit address is used for each I/O port. This is because 8 bits are the basic data unit for these processors. Eight-bit processors are usually capable of directly addressing 64K bytes of memory using 16 address lines. With an 8-bit I/O port address, these processors are capable of addressing 256 ports. However, in a typical application, there are usually four or five I/O ports required. Some of the address bits of the CPU are normally decoded to obtain the I/O port addresses. With memory-mapped I/O, the processor does not differentiate between I/O and memory and, therefore, does not use the $\text{I/O}/\overline{\text{M}}$ control pin. The processor uses a portion of the memory

* *Microcomputer Theory and Applications With the Intel SDK-85*. Rafiquzzaman, Second Edition, 1987, Wiley. Reprinted with permission.

addresses to represent I/O ports. The I/O ports are mapped into the processor's main memory and, hence, are called *memory-mapped I/O*. Each method has its advantages and disadvantages. For example, when standard I/O is used, an 8-bit processor normally uses the 2-byte IN or OUT instruction as follows:

IN	{	2-byte instruction for inputting data from the specified I/O port into the processor's accumulator
port number		
OUT	{	2-byte instruction for outputting data from the accumulator into the specified I/O port
port number		

With memory-mapped I/O, an 8-bit processor normally uses 3-byte instructions, namely, LDA and STA, as follows:

LDA	{	3-byte instruction for inputting a byte into accumulator
xx}		
xx} Port address mapped into memory		
STA	{	3-byte instruction for outputting data into the specified port
xx}		
xx} Port address mapped into memory		

In memory-mapped I/O, the most significant bit (MSB) of the address is typically used to distinguish between I/O and memory. If the MSB of address is 1, an I/O port is selected. If the MSB of address is 0, a memory location is accessed. Some microprocessors such as the Motorola MC68000 provide a special control signal for performing memory-mapped I/O. Thus, these processors do not use MSB of the address lines.

The processor can perform operations on port data without moving it into a CPU register through memory-mapped I/O. The port data must be moved into the accumulator for further processing while using standard I/O.

6.4 UNCONDITIONAL AND CONDITIONAL PROGRAMMED I/O*

The processor can send data to an external device at any time during unconditional I/O. The external device must always be ready for data transfer. A typical example is when the processor outputs a 7-bit code through an I/O port to drive a seven-segment display connected to this port.

In conditional I/O, the processor outputs data to an external device via handshaking. Data transfer occurs by the exchanging of control signals between the processor and an external device. The processor inputs the status of the external device to determine whether the device is ready for data transfer. Data transfer takes place when the

* *Microcomputer Theory and Applications With the Intel SDK-85*. Rafiquzzaman, Second Edition, 1987, Wiley. Reprinted with permission.

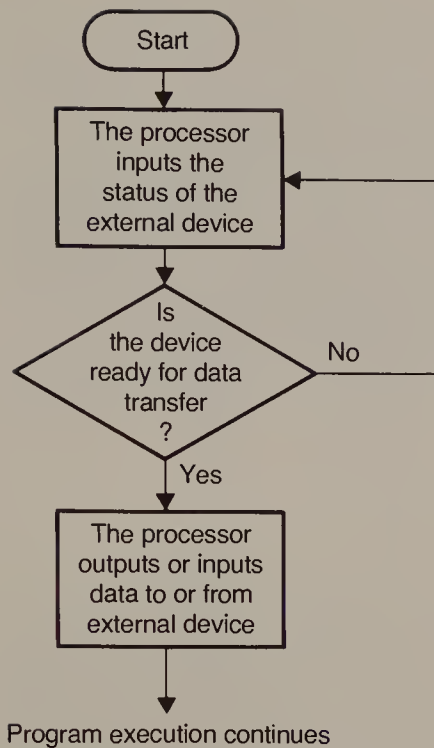
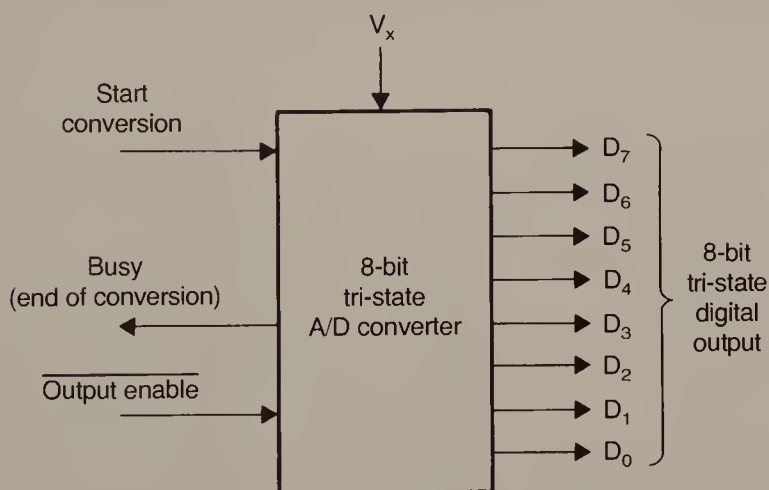


Figure 6.1 Flowchart for Conditional Programmed I/O

device is ready. The flowchart in Figure 6.1 illustrates the concept of conditional programmed I/O.

The concept of conditional I/O will now be demonstrated by means of data transfer between a processor and an analog-to-digital (A/D) converter. Consider, for example, the A/D converter shown in the accompanying figure.



The A/D converter just shown transforms an analog voltage V_x into an 8-bit binary output at pins D_7 – D_0 . A pulse at the START conversion pin initiates the conversion. This drives the BUSY signal LOW. The signal stays LOW during the conversion process. The BUSY signal goes HIGH as soon as the conversion ends. Since the A/D converter's output is tristated, a LOW on the OUTPUT ENABLE transfers the conver-

ter's outputs. A HIGH on the $\overline{\text{OUTPUT ENABLE}}$ drives the converter's outputs to a high impedance state.

The concept of conditinal I/O can be demonstrated by interfacing the A/D converter to an 8-bit processor. Figure 6.2 shows such an interfacing example.

The user writes a program to carry out the conversion process. When this program is executed, the processor sends a pulse to the START pin of the converter via bit 2 of port A. The processor then checks the BUSY signal by bit 1 of port A to determine if the conversion is completed. If the BUSY signal is HIGH (indicating the end of conversion), the processor sends a LOW to the $\overline{\text{OUTPUT ENABLE}}$ pin of the A/D converter. The processor then inputs the converter's D_0 – D_7 outputs via port B. If the conversion is not completed, the processor waits in a loop checking for the BUSY signal to go HIGH.

6.5 INTERRUPT I/O

A disadvantage of conditional programmed I/O is that the computer needs to check the status bit (BUSY signal for the A/D converter) by waiting in a loop. This type of I/O transfer is dependent on the speed of the external device. For a slow device, this waiting may slow down the computer's capability of processing other data. The interrupt I/O technique is efficient in this type of situation.

6.5.1 Basic Concepts

Interrupt I/O is a device-initiated I/O transfer. The external device is connected to a pin called the *interrupt* (INT) pin on the processor chip. When the device needs an I/O transfer with the computer, it activates the interrupt pin of the processor chip. The computer usually completes the current instruction and saves at least the contents of the current program counter in the stack.

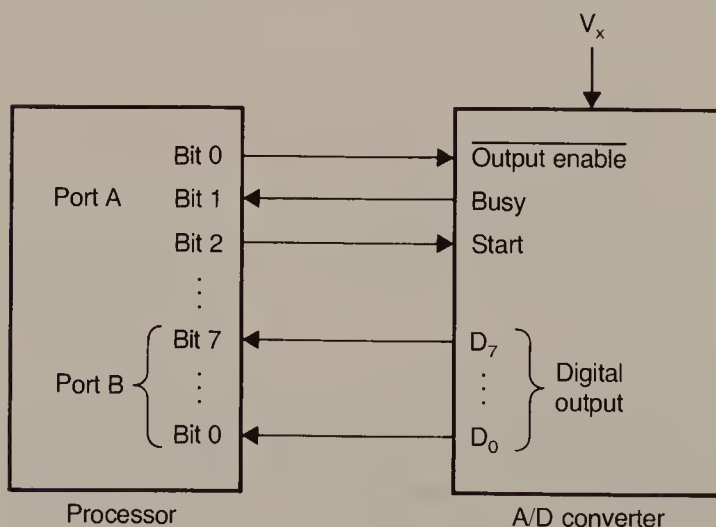


Figure 6.2 Interfacing an A/D Converter to an 8-bit Processor

The computer then automatically loads an address into the program counter to branch to a subroutine-like program called the *interrupt-service routine*. This program is written by the user. The external device wants the computer to execute this program to transfer data. The last instruction of the service routine is a RETURN, which is typically the same instruction used at the end of a subroutine. This instruction normally loads the address (saved in the stack before going to the service routine) in the program counter. Then, the computer continues executing the main program.

An example of interrupt I/O is shown in Figure 6.3.

Assume the computer is MC68000 based and executing the following program:

```

BEGIN    EQU    $3000
          ORG    $2000
          MOVE . B # $81, DDRA; configure bits 0 and
                                7 of port A as outputs
          MOVE . B # $00, DDRB; configure Port B as input
          MOVE . B # $81, PORTA } send start pulse to
                                A/D and HIGH TO OUTPUT ENABLE
          MOVE . B # $01, PORTA }
          CLR . W D0
BEGIN    MOVE . W D1, D2
          -
          -
          -
          -
          -
  
```

The preceding program is arbitrarily written. The program logic can be explained using the MC68000 instruction set. Ports DDRA and DDRB are assumed to be the data-direction registers for ports A and B, respectively. The first four MOVE instructions configure bits 0 and 7 of port A as outputs and port B as an input port and then send the

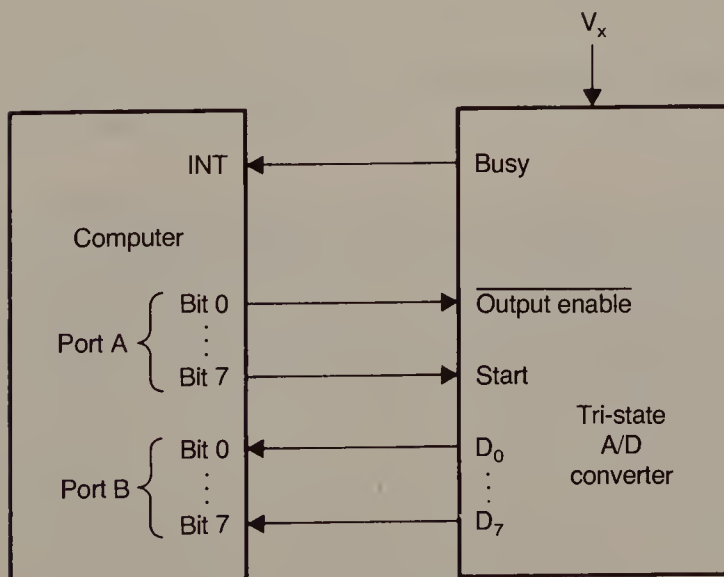


Figure 6.3 Computer A/D Converter Interface via Interrupt I/O

START pulse to the A/D converter along with a HIGH to the OUTPUT ENABLE. This HIGH OUTPUT ENABLE is required to disable the A/D's output. The computer continues with execution of the CLR. W D0 instruction until the busy signal becomes HIGH, indicating the end of conversion. This drives the INT signal to HIGH, interrupting the computer. The computer completes execution of the current instruction, CLR. W D0. It then saves the current contents of the program counter (address \$3000) and status register automatically and executes a subroutine-like program called the *service routine*. This program is usually written by the user. The computer manufacturer may define the starting address of the service routine, or it may be provided by the user via external hardware. Assume this address is \$4000. To service the A/D converter, the computer automatically branches to \$4000, where the user writes a service routine to input the A/D converter's output as follows:

```

MOVE·B#$00, PORT A;    Activate OUTPUT ENABLE.
MOVE·B PORT B, D1  ;    Input A/D.
RTE                    ;    Return and restore PC and SR.

```

In this service routine, the computer inputs the A/D converter's output. The return instruction RTE, at the end of the service routine, pops \$3000 and the previous status register contents from the stack and loads the program counter and status register with them. The computer executes the MOVE.W D1,D2 at BEGIN (address \$3000) and continues with the main program.

6.5.2 Main Features of Interrupt I/O*

The basic characteristics of interrupt I/O are discussed in the previous section. The main features of interrupt I/O provided with a typical computer are discussed next.

Interrupt Types

There are typically three types of interrupts: external interrupts, traps or internal interrupts, and software interrupts.

External interrupts are initiated through the computer's interrupt pins by external devices such as A/D converters. A simple example of an external interrupt was given in the previous section.

External interrupts can further be divided into two types: *maskable and nonmaskable*. A maskable interrupt is enabled or disabled by executing instructions such as EI or DI. If the computer's interrupt is disabled, the computer ignores the maskable interrupt. Some processors, such as the Intel 8086, have an interrupt-flag bit in the processor status register. When the interrupt is disabled, the interrupt-flag bit is 1, so no maskable interrupts are recognized by the processor. The interrupt-flag bit is reset to zero when the interrupt is enabled.

The nonmaskable interrupt has higher priority than the maskable interrupt. If both maskable and nonmaskable interrupts are activated at the same time, the processor will service the nonmaskable interrupt first. The nonmaskable interrupt is typically used as

* *Microcomputer Theory and Applications With the Intel SDK-85*. Rafiquzzaman, Second Edition, 1987, Wiley. Reprinted with permission.

a power failure interrupt. Processors normally use +5 volts DC, which is transformed from 110V AC. If the power falls below 90V AC, the DC voltage of +5 volts cannot be maintained. However, it will take a few milliseconds before the AC power can drop this low (below 90 volts AC). In these few milliseconds, the power failure-sensing circuitry can interrupt the processor. The interrupt service routine can be written to store critical data in nonvolatile memory such as battery-backed CMOS RAM. The interrupted program can continue without any loss of data when the power returns.

Some processors are provided with a maskable handshake interrupt. This interrupt is usually implemented by using two pins—INTR and INTA. When the INTR pin is activated by an external device, the processor completes the current instruction, saves at least the current program counter onto stack, and generates an interrupt acknowledge (INTA). In response to the INTA, the external device provides an instruction, such as CALL, using external hardware on the data bus of the computer. This instruction is then read and executed by the computer to branch to the desired service routine.

Internal interrupts, or traps, are activated internally by exceptional conditions such as overflow, division by zero, or execution of an illegal op-code. Traps are handled the same way as external interrupts. The user writes a service routine to take corrective measures and provide an indication to inform the user that an exceptional condition has occurred.

Many processors include software interrupts, or system calls. When one of these instructions is executed, the processor is interrupted and serviced similarly to external or internal interrupts. Software interrupt instructions are normally used to call the operating system. These instructions are shorter than subroutine calls, and no calling program is needed to know the operating system's address in memory. Software interrupt instructions allow the user to switch from user to supervisor mode. For some processors, a software interrupt is the only way to call the operating system, since a subroutine call to an address in the operating system is not allowed.

Interrupt Address Vector

The technique used to find the starting address of the service routine (commonly known as the *interrupt address vector*) varies from one processor to another. With some processors, the manufacturers define the fixed starting address for each interrupt. Other manufacturers use an indirect approach by defining fixed locations where the interrupt address vector is stored.

Saving the CPU Registers

When a processor is interrupted, it saves at least the program counter on the stack so the processor can return to the main program after executing the service routine. Some processors save only one or two registers, such as the program counter and status register. Other processors save all CPU registers before going to the service routine. The user should know the specific registers the processor saves prior to executing the service routine. This will enable the user to use the appropriate return instruction at the end of the service routine to restore the original conditions upon return to the main program.

Interrupt Priorities

A processor is typically provided with one or more interrupt pins on the chip. Therefore, a special mechanism is necessary to handle interrupts from several devices

that share one of these interrupt lines. There are two ways of servicing multiple interrupts: polled and daisy chain techniques.

Polled Interrupts

Polled interrupts are handled by software, and therefore are slower, compared with daisy chaining. The processor responds to an interrupt by executing one general-service routine for all devices. The priorities of devices are determined by the order in which the routine polls each device. The processor checks the status of each device in the general-service routine, starting with the highest-priority device, to service an interrupt. Once the processor determines the source of the interrupt, it branches to the service routine for the device. Figure 6.4 shows a typical configuration of the polled-interrupt system.

In Figure 6.4, several external devices (device 1, device 2, . . . , device N) are connected to a single interrupt line of the processor via an OR gate (not shown in the figure). When one or more devices activate the INT line HIGH, the processor pushes the program counter and possibly some other registers onto the stack. It then branches to an address defined by the manufacturer of the processor. The user can write a program at this address to poll each device, starting with the highest-priority device, to find the source of the interrupt. Suppose the devices in Figure 6.4 are A/D converters. Each converter, along with the associated logic for polling, is shown in Figure 6.5.

Assume that in Figure 6.4 two A/D converters (device 1 and device 2) are provided with the START pulse by the processor at nearly the same time. Suppose the user assigns

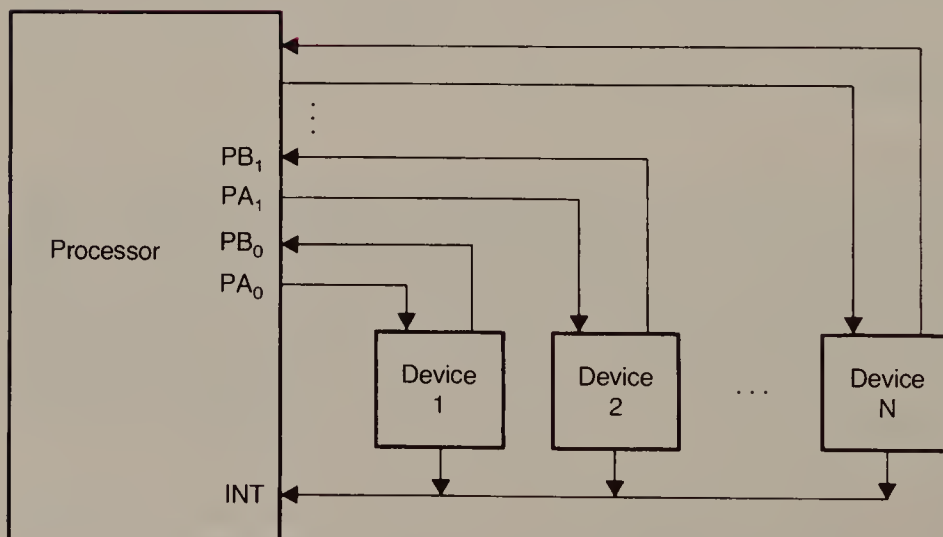


Figure 6.4 Polled Interrupt

device 2 with the higher priority. The user then sets up this priority mechanism in the general-service routine. For example, when the BUSY signals from device 1 and/or 2 become HIGH, indicating the end of conversion, the processor is interrupted. In response, the processor pushes at least the program counter on the stack, and loads the PC with the interrupt-address vector defined by the manufacturer.

The general interrupt-service routine written at this address determines the source of the interrupt as follows: A 1 is sent to PA1 for device 2, since this device has higher priority. If this device has generated an interrupt, the output (PB1) of the AND gate in Figure 6.5 becomes HIGH, indicating to the processor that device 2 generated the interrupt. If the output of the AND gate is 0, the processor sends a HIGH to PA0 and checks the output (PB0) for HIGH. Once the source of the interrupt is determined, the processor can be programmed to jump to the service routine for that device. The service routine enables the A/D converter, and inputs the converter's outputs to the processor.

Polled interrupts are slow and for a large number of devices, the time required to poll each device may exceed the time to service the device. In such a case, a faster mechanism, such as the daisy chain approach, can be used.

Daisy Chain Interrupts Devices are connected in a daisy chain fashion, as shown in Figure 6.6, to set up priority systems.

Suppose one or more devices interrupt the processor. In response, the processor pushes at least the PC and generates

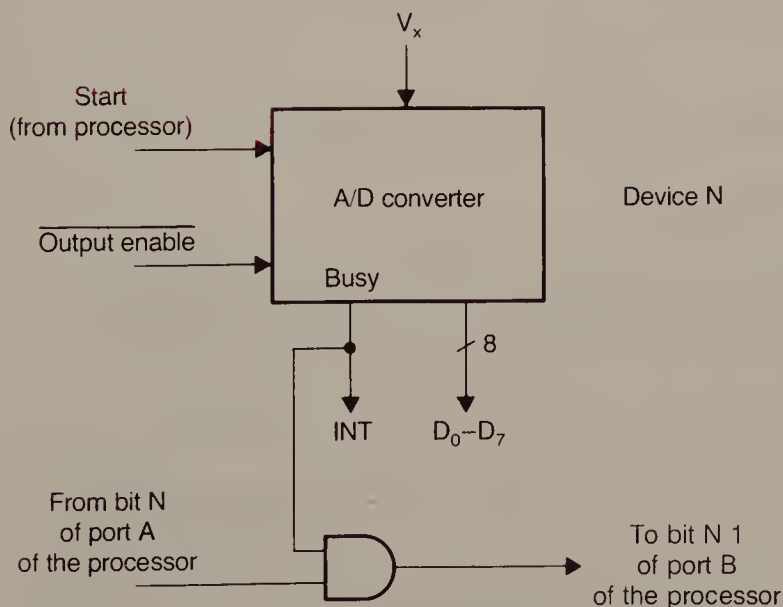


Figure 6.5 Device N and Associated Logic for Polled Interrupt

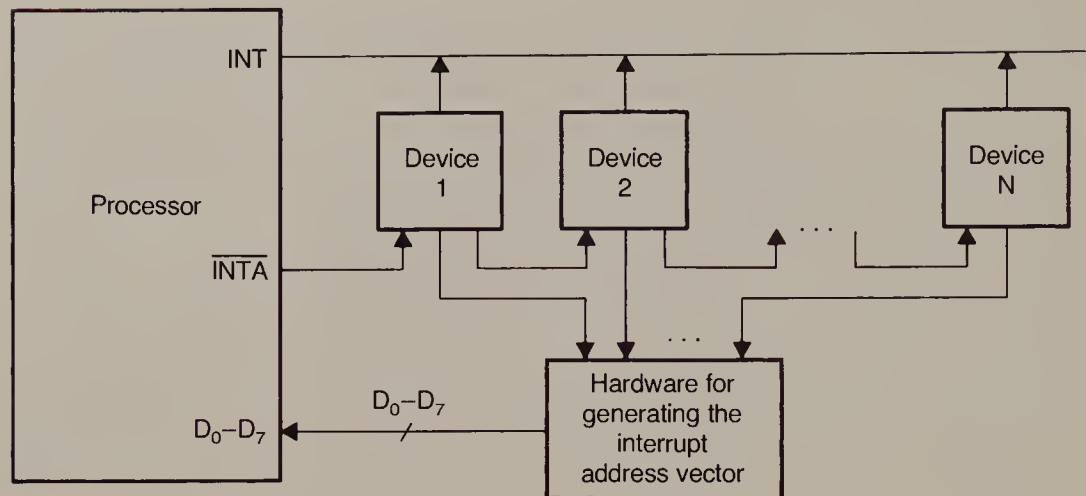


Figure 6.6 Daisy Chain Interrupt

an interrupt acknowledge ($\overline{\text{INTA}}$) signal to the highest-priority device (device 1 in this case). If this device has generated the interrupt, it will accept the $\overline{\text{INTA}}$; otherwise, it will pass the $\overline{\text{INTA}}$ onto the next device until the $\overline{\text{INTA}}$ is accepted. Once accepted, the device provides a means for the processor to find the interrupt-address vector by using external hardware. Assume the devices in Figure 6.6 are A/D converters. Figure 6.7 provides a schematic for each device and the associated logic.

Suppose the processor in Figure 6.6 sends a pulse to

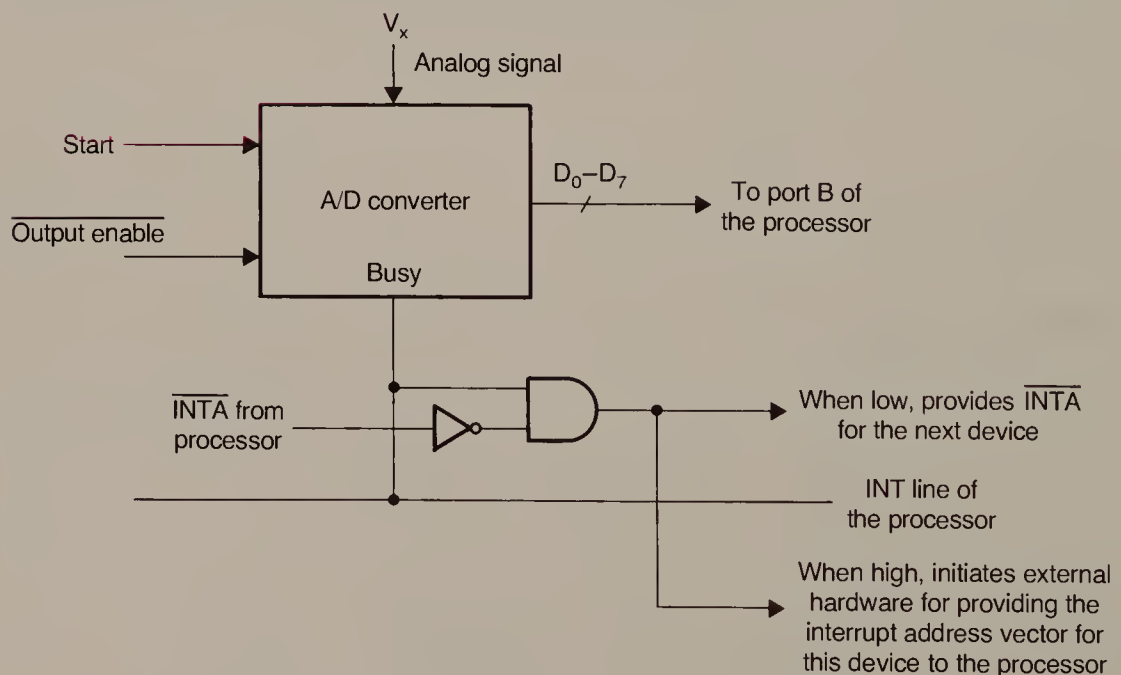


Figure 6.7 Each device and the Associated Logic in a Daisy Chain

start the conversions of A/D converter of devices 1 and 2 at nearly the same time. When the BUSY signal goes to HIGH, the processor is interrupted through the INT line. The processor pushes the PC and possibly some other registers. It then generates a LOW at the interrupt-acknowledge ($\overline{\text{INTA}}$) for the highest-priority device (device 1 in Figure 6.6). Device 1 has the highest priority—it is the first device in the daisy chain configuration to receive $\overline{\text{INTA}}$. If A/D converter 1 has generated the BUSY HIGH, the output of the AND gate becomes HIGH. This signal can be used to enable external hardware to provide the interrupt-address vector on the processor's data lines. The processor then branches to the service routine. This program enables the converter, and inputs the A/D output to the processor via Port B. If A/D converter #1 does not generate the BUSY HIGH, however, the output of the AND gate in Figure 6.7 becomes LOW (an input to DEVICE 2's logic) and the same sequence of operations takes place. In the daisy chain, each device has the same logic, with the exception of the last device which must accept the $\overline{\text{INTA}}$. Note that the outputs of all the devices are connected to the INT line via an OR gate (not shown in Figure 6.6).

6.6 DIRECT MEMORY ACCESS (DMA)*

Direct Memory Access (DMA) is a technique that transfers data between a computer's memory and I/O device without involving the CPU. DMA is widely used in transferring large blocks of data between a peripheral device and the computer's memory. The DMA technique uses a DMA controller chip for the data-transfer operation. The main functions of a typical DMA controller are summarized as follows:

- The I/O devices request DMA operation via the DMA request line of the controller chip.
- The controller chip activates the CPU HOLD pin, requesting the CPU to release the bus.
- The processor sends HLDA (hold acknowledge) back to the DMA controller, indicating the bus is disabled. The DMA controller places the current value of its internal registers, such as the address register and counter, on the system bus and sends a DMA acknowledge to the peripheral device. The DMA controller completes the DMA transfer.

There are three basic types of DMA: block-transfer, cycle-stealing, and interleaved DMA.

For block-transfer DMA, the DMA controller chip takes the bus from the com-

* *Microcomputer Theory and Applications With the Intel SDK-85*. Rafiquzzaman, Second Edition, 1987, Wiley. Reprinted with permission.

puter to transfer data between the computer memory and I/O device. The CPU has no access to the bus until the transfer is completed. During this time, the CPU can perform internal operations that do not need the bus. This method is popular with microprocessors. Using this technique, blocks of data can be transferred.

Data transfer between the computer memory and an I/O device occurs on a word-by-word basis with cycle stealing. Typically, the CPU is generated by ANDing an $\overline{\text{INHIBIT}}$ signal with the system clock. The system clock has the same frequency as the CPU clock.

THE DMA controller controls the $\overline{\text{INHIBIT}}$ line. During normal operation, the $\overline{\text{INHIBIT}}$ line is HIGH, providing the CPU clock. When DMA operation is desired, the controller makes the $\overline{\text{INHIBIT}}$ line LOW for one clock cycle. The CPU is then stopped completely for one cycle. Data transfer between the memory and I/O takes place during this cycle. This method is called cycle stealing because the DMA controller takes away or steals a cycle without CPU recognition. Data transfer takes place over a period of time:

With interleaved DMA, the DMA controller chip takes over the system bus when the CPU is not using it. For example, the CPU does not use the bus while incrementing the program counter or performing an ALU operation. The DMA controller chip identifies these cycles and allows transfer of data between the memory and I/O device. Data transfer takes place over a period of time for this method.

Since block-transfer DMA is common with modern microprocessors, a detailed description is provided.

Figure 6.8 shows a typical diagram of the block-transfer DMA.

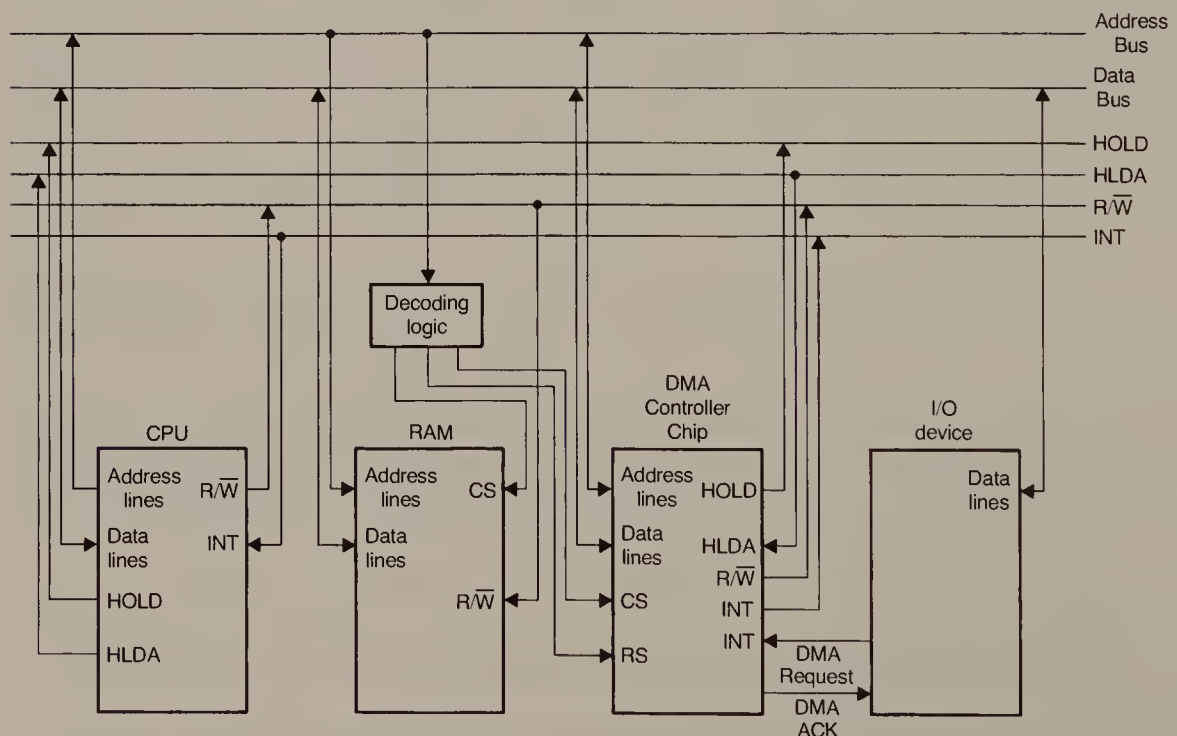


Figure 6.8 A Typical DMA System

In Figure 6.8, the I/O device requests for the DMA transfer via the DMA request line connected to the controller chip. The DMA controller chip then sends a HOLD signal to the CPU. It then waits for the HOLD acknowledge (HLDA) signal from the CPU. On receipt of HLDA, the controller chip sends a DMA ACK signal to the I/O device. The controller takes over the bus and controls data transfer between the RAM and I/O device. On completion of the data transfer, the controller interrupts the CPU by the INT line and returns the bus to the CPU by disabling the HOLD and DMA ACK signals.

The DMA controller chip usually has at least three registers normally selected by the controller's register select (RS) line: an address register, a terminal count register and a status register. Both the address and terminal count registers are initialized by the CPU. The address register contains the starting address of the data to be transferred, and the terminal count register contains the desired block to be transferred. The status register contains information such as completion of DMA transfer.

6.7 MOTOROLA MC68000 I/O

To illustrate the I/O concepts described in the previous sections, the Motorola MC68000 is considered.

6.7.1 MC68000 Programmed I/O

As mentioned before, the MC6800 uses memory-mapped I/O. Programmed I/O can be achieved in the MC68000 in one of the following ways:

1. Interfacing the MC68000 asynchronously with its own family of peripheral devices such as the MC68230 parallel interface/timer chip.
2. Interfacing the MC68000 synchronously with MC68000 peripherals such as the MC6821. Synchronous operation means every read or write operation must be synchronized with the clock. MC68000 peripherals are expensive. Therefore, it is both sufficient and economical to use 6800 peripherals to perform MC68000 programmed I/O.

MC68230's interface to the MC68000 will be considered in the following.

The MC68230 contains 23 internal registers which can be selected using RS1-RS5 pins. Each register has an assigned register select code defined by RS1-RS5 pins. For example, the code 01000₂ when applied to RS5 thru RS1 selects register R₈.

In addition to general I/O capabilities, the MC68230 also offers interrupt control and timer operations, the discussion of which is beyond the scope of this section. This discussion will be restricted to basic I/O operation.

The registers R₀ (Port General Control Register, PGCR), R₆ (Port A Control Register, PACR) and R₇ (Port B Control Register, PBCR) are the primary control registers for basic I/O operation.

Ports A and B of the MC68230 can be configured in one of four modes using bits 6 and 7 of PGCR as follows:

BIT 7	BIT 6	
0	0	mode 0 (Unidirectional 8-bit mode)
0	1	mode 1 (Unidirectional 16-bit mode)
1	0	mode 2 (Bidirectional 8-bit mode)
1	1	mode 3 (Bidirectional 16-bit mode)

The other bits in PGCR are used for handshake operations.

Ports set up for unidirectional operation must further be configured using the submode operation. In the submode operation, the ports can be configured as all input bits, or all output bits or bit addressable inputs or outputs. Bits 6 and 7 of PACR and PBCR respectively control submode operation for ports A and B. For example, for either port A or B, bits 7 and 6 of the associated control register (PACR or PBCR) define the use of the ports as follows:

FOR UNIDIRECTIONAL 8-BIT MODE

BIT 7 OF PACR OR PBCR	BIT 6 OF PACR OR PBCR	
0	0	Double Buffered Input
0	1	Double Buffered Output
1	X	Bit I/O

Note: X means don't cares. Double Buffered means that the ports have dual latches. Double buffering allows simultaneous reading of data from a port by the microprocessor and placing of new data into the same port by an external device.

FOR UNIDIRECTIONAL 16-BIT MODE

BIT 7 OF PACR OR PBCR	BIT 6 OF PACR OR PBCR	
X	X	Not used
X	0	Unidirectional 16-bit input
X	1	Unidirectional 16-bit output

The Port direction registers are R_2 (Port A Data Direction Registers—PADDDR), R_3 (Port B Data Direction Register—PBDDR) and R_4 (Port C Data Direction Register—PCDDR). A '1' in a particular bit position in these registers configures the corresponding bit in the port as an output and a '0' in a specific bit position in one of the DDR'S configures the corresponding bit in the port as an input when the ports are configured for unidirectional mode of operation.

In the bidirectional mode, the Data Direction registers are ignored and the direction is determined dynamically by the state of the four handshake pins.

EXAMPLE 6-1

- i) Interface one MC68230 to the 68000.
- ii) In the above 68000/68230 configuration, assume that Port A has three switches connected to bits 0, 1 and 2, and Port B has an LED connected to bit 2. Write 68000 assembly language program to turn the LED on if Port A has odd number of HIGH switch inputs; otherwise turn the LED OFF.

SOLUTION

- i) Figure 6.9 shows MC68230's interface to the MC68000.

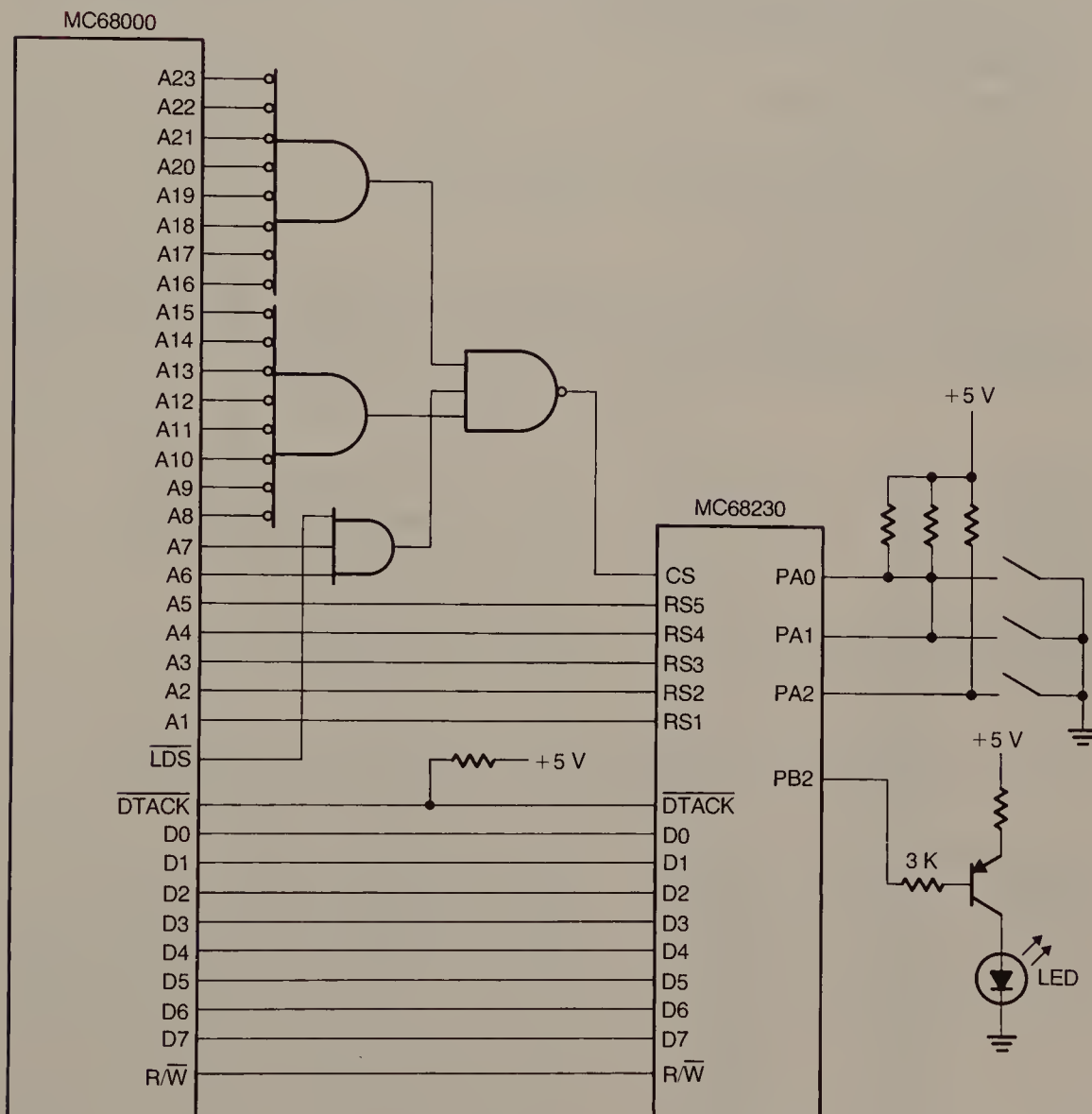


Figure 6.9 MC68000 I/O Port Block Diagram

From the figure, the register addresses are obtained as follows:

PGCR	000001 ₁₆
PADDR	000003 ₁₆
PBDDR	000007 ₁₆
PACR	00000D ₁₆
PBCR	00000F ₁₆
PADR	000011 ₁₆
PBDR	000013 ₁₆

ii) To accomplish the assigned task (light LED if odd no. of switch inputs are high), the following steps must be performed:

- (1) Initialization, configure ports.
- (2) Read Port A.
- (3) Mask off high 5 bits.
- (4) Test low 3 bits and see if odd condition is met.
- (5) If odd then write data to Port B to turn LED on.
- (6) If even then turn LED off.
- (7) Repeat steps 2 thru 6

The best port configuration to use is Mode 00 and Submode 1X which is the Bit I/O mode. This requires that the DDRs be programmed properly, PA0-PA2 as inputs and PB2 as an output. The assembly program listing is shown below:

```

                                ASSEMBLY LISTING
PGCR      EQU      $000001      ;Port General Control Reg.
PADDR     EQU      $000003      ;Port A Data Direction Reg.
PBDDR     EQU      $000007      ;Port B Data Direction Reg.
PACR      EQU      $00000D      ;Port A Control Reg.
PBCR      EQU      $00000F      ;Port B Control Reg.
PADR      EQU      $000011      ;Port A Data Reg.
PBDR      EQU      $000013      ;Port B Data Reg.
;*****
INIT:      MOVE.B    #$00,PGCR    ;Select mode 00
           MOVE.B    #$F8,PADDR   ;Bits 0-2 of Port A are input
           MOVE.B    #$FF,PBDDR   ;All bits Port B are output
           MOVE.B    #$FF,PACR    ;Bit I/O mode for Port A
           MOVE.B    #$FF,PBCR    ;Bit I/O mode for Port B
;*****
START:     MOVE.B    PADR,D1      ;Get switch settings
           ANDI.B    #$07,D1      ;Mask high five bits
           CMPI.B    #$07,D1      ;Are all three inputs high?
           BEQ       ODD          ;If so, then turn on LED
           CMPI.B    #$01,D1      ;Is input 0 high?
           BEQ       ODD          ;If so, then turn on LED
           CMPI.B    #$02,D1      ;Is input 1 high?
           BEQ       ODD          ;If so, then turn on LED
           CMPI.B    #$04,D1      ;Is input 2 high?
           BEQ       ODD          ;If so, then turn on LED

```

```

EVEN:    MOVE.B    #$FF,PBDR    ;Else turn off LED
          JMP      START        ; and repeat.
ODD:     MOVE.B    #$00,PADR    ;Turns on LED
          JMP      START        ; and repeat.

```

6.7.2 MC68000 Interrupt System*

The MC68000 interrupt I/O can be divided into two types: external Interrupts and internal interrupts.

External Interrupts

The MC68000 provides seven levels of external interrupts, 0 through 7. The external hardware provides an interrupt level using pins $\overline{\text{IPL0}}$, $\overline{\text{IPL1}}$, $\overline{\text{IPL2}}$. Like other CPUs, the MC68000 checks for and accepts interrupts only between instructions. It compares the value of inverted $\overline{\text{IPL0-IPL2}}$ with the current interrupt mask contained in bits 8, 9, and 10 of the status register.

If the value of the inverted $\overline{\text{IPL0}}$ through $\overline{\text{IPL2}}$ is greater than the value of the current interrupt mask, the CPU acknowledges the interrupt and initiates interrupt processing. Otherwise, the CPU continues with the current interrupt. Interrupt-request level 0 ($\overline{\text{IPL0-IPL2}}$ all high), indicates no interrupt service is requested. An inverted $\overline{\text{IPL2-IPL1-IPL0}}$ of interrupt level 7 is always acknowledged. Therefore, interrupt level 7 is nonmaskable. The interrupt level is indicated by the interrupt mask bits (inverted $\overline{\text{IPL2}}$, $\overline{\text{IPL1}}$, $\overline{\text{IPL0}}$) of the status register. To ensure an interrupt will be recognized, the following interrupting rules should be considered:

1. The incoming interrupt request level must be at a higher priority level than the mask level set in the interrupt mask bits (except for level 7, which is always recognized).
2. The $\overline{\text{IPL2-IPL0}}$ pins must be held at the interrupt request level until the 68000 acknowledges the interrupt by initiating an interrupt acknowledge (IACK) bus cycle.

Interrupt Level 7 is edge-triggered, whereas interrupts levels 1 to 6 are level sensitive. When one is acknowledged, the processor updates its interrupt mask at the same level.

When the 68000 is ready to service an interrupt it pushes PC and SR onto the stacks, enters supervisor state by setting S-bit to 1, clears the trace bit, updates the priority mask bits and address lines A3–A1 with the interrupt level and then sets FC2 FC1 FC0 to 111 to run an IACK cycle for 8-bit vector number acquisition. The 68000 multiplies the 8-bit vector by 4 to determine pointer to the locations containing the starting address of the service routine. The 68000 then branches to the service routine.

* *Microcomputer Theory and Applications With the Intel SDK-85*. Rafiquzzaman, Second Edition, 1987, Wiley. Reprinted with permission.

The last instruction of the service routine should be RTE which pops PC and SR back from the stack.

External logic can respond to the interrupt acknowledge in one of three ways: placing a vector number on the data bus (nonautovector), requesting automatic vectoring, or indicating that no device is responding (spurious interrupt).

Autovector If the hardware asserts $\overline{\text{VPA}}$ to terminate the IACK bus cycle, the MC68000 directs itself to the interrupt vector corresponding to the current interrupt level. No external hardware is required for providing interrupt address vector.

	Inverted values of		
	$\overline{\text{IPL2}}$	$\overline{\text{IPL1}}$	$\overline{\text{IPL0}}$
Level 1 $\leftarrow$ Interrupt vector 19H for	0	0	1
Level 2 $\leftarrow$ Interrupt vector 1AH for	0	1	0
Level 3 $\leftarrow$ Interrupt vector 1BH for	0	1	1
Level 4 $\leftarrow$ Interrupt vector 1CH for	1	0	0
Level 5 $\leftarrow$ Interrupt vector 1DH for	1	0	1
Level 6 $\leftarrow$ Interrupt vector 1EH for	1	1	0
Level 7 $\leftarrow$ Interrupt vector 1FH for	1	1	1

Nonautovector The interrupting device uses external hardware to place a vector number on data lines D0-D7 and performs a $\overline{\text{DTACK}}$ handshake to terminate the IACK bus cycle. The vector numbers allowed are 40H to FFH, but Motorola has not implemented a protection on the first 64 entries, so user-interrupt vectors may overlap at the discretion of the system designer.

Spurious Interrupt Another way to terminate an interrupt-acknowledge bus cycle is with the $\overline{\text{BERR}}$ (bus error) signal. Even though the interrupt control pins are synchronized to enhance noise immunity, it is possible external system interrupt circuitry may initiate an IACK bus cycle as a result of noise. Since no device is requesting interrupt service, neither DTACK nor VPA will be asserted to signal the end of the nonexistent IACK bus cycle. When there is no response to an IACK bus cycle after a specified period of time (monitored by the user using an external timer), the $\overline{\text{BERR}}$ can be asserted by the external timer. This tells the processor it has recognized a spurious interrupt. The 68000 provides 18H as the vector to fetch the starting address of this exception handling routine. Then, from the vector n , the CPU reads the long word located at memory $4(n)$. This long word is the address of the

service routine. The address is found using indirect addressing. The spurious interrupt and bus error interrupt, due to a troubled instruction cycle, are different interrupts.

Internal Interrupts

Internal interrupts are software interrupts. These interrupts are generated when the 68000 executes a software interrupt instruction called TRAP or by some undesirable event such as division by zero or execution of an illegal instruction.

MC68000 Interrupt Map

Figure 6.10 shows the interrupt map of the MC68000. Addresses 00H through 2EH (not shown in the figure) contain vector addresses for reset, bus error, trace, divide by 0, and the like.

MC68000 Interrupt-address Vector

Suppose the user decides to write a service routine starting at location 123456H using autovector 1. Since autovector 1 is at 000064H and 000066H, 0012H and 3456H must be stored in locations 000064H and 000066H, respectively.

Vector Address	Vector Number	
30H to 5CH	Unassigned	0CH to 17H
60H, 62H	Spurious interrupt	18H
64H, 66H	Autovector 1	19H
68H, 6AH	Autovector 2	1AH
6CH, 6EH	Autovector 3	1BH
70H, 72H	Autovector 4	1CH
74H, 76H	Autovector 5	1DH
78H, 7AH	Autovector 6	1EH
7CH, 7EH	Autovector 7	1FH
80H to BCH	TRAP instructions	20H to 2FH
C0H		30H
⋮	Unassigned	⋮
FCH		3FH
100H	User interrupts	40H
⋮	(Nonautovector)	⋮
3FCH		FFH

Figure 6.10 MC68000 Interrupt Map

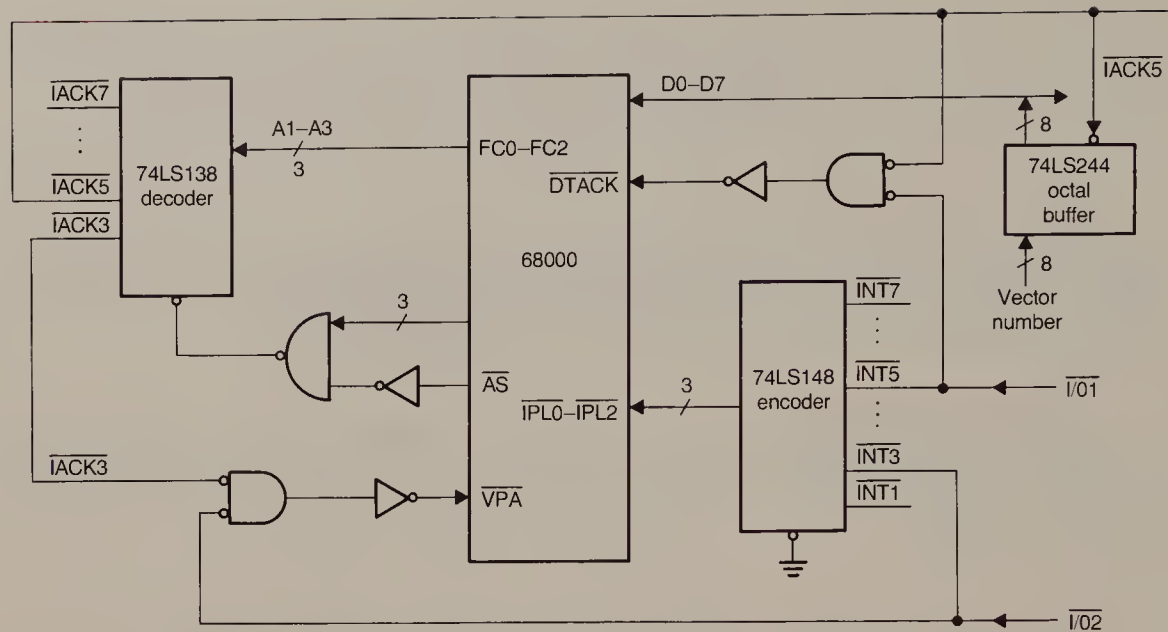


Figure 6.11 Autovector and Nonautovector Interrupts (Courtesy of Motorola)

An Example of Autovector and Nonautovector Interrupts to Illustrate the Concept of Autovector and Nonautovector Interrupts

Consider Figure 6.11. In this figure, I/O device 1 uses nonautovector interrupts, and I/O device 2 uses autovector interrupts. The system is capable of handling interrupts from 8 devices, since an 8-to-3 priority encoder, such as the 74LS148, is used. Suppose I/O device 2 drives the $\overline{I/O2}$ low to activate line 3 of this encoder. This interrupts the processor. When the MC68000 decides to acknowledge the interrupt, it drives FC0–FC2 HIGH. The interrupt level is reflected on A1–A3 when AS is activated by the MC68000. $\overline{IACK3}$ and $\overline{I/O2}$ signals are used to generate $\overline{VPA}$. Once $\overline{VPA}$ is asserted, the MC68000 obtains the interrupt-vector address using autovectoring.

In the case of $\overline{I/O1}$, line 5 of the priority encoder is activated to initiate the interrupt. By using appropriate logic, $\overline{DTACK}$ is asserted through $\overline{IACK5}$ and $\overline{I/O1}$. The vector number is placed on D0–D7 by enabling an octal buffer, such as the 74LS244, through $\overline{IACK5}$. The 68000 inputs this vector number, and multiplies it by 4 to obtain the interrupt-address vector.

6.7.3 MC68000 DMA*

Three DMA control lines are provided with the MC68000: Br (bus request), BG (bus grant), and BGACK (bus grant acknowledge). The BR line is an input to the MC68000. The external device activates this line to tell the MC68000 to release the system bus.

At least one clock period after receiving the BR, the MC68000 enables its BG output line to acknowledge the DMA request. The MC68000 will not relinquish the bus until it has completed the current instruction cycle. The external device must check the

* *Microcomputer Theory and Applications With the Intel SDK-85*. Rafiquzzaman, Second Edition, 1987, Wiley. Reprinted with permission.

AS (address strobe) line to determine the completion of instruction cycle by the MC68000. When the AS becomes HIGH the MC68000 tristates its address and data lines and gives up the bus to the external device.

After taking over the bus, the external device must enable the BGACK line. The BGACK line tells the MC68000 and other devices connected to the bus that the bus is being used. The MC68000 stays in a tristate condition until the BGACK becomes HIGH.

6.7.4 MC68000 Exception Handling*

A computer is usually capable of handling unusual or exceptional conditions such as execution of illegal instruction or division by zero. In this section, exception handling capabilities of a typical microprocessor such as the MC68000 are described.

When an exception occurs, the MC68000 saves the contents of the program counter and status register on the stack and executes a new program whose address is provided by the exception vectors. Once this program is executed, the 68000 returns to the main program by using the stored values of the program counter and status register.

Exceptions can be of two types: internal or external. The internal exceptions are initiated by situations such as division by zero, execution of illegal or unimplemented instructions, and address error. As mentioned before, internal interrupts are called traps.

The external exceptions are generated by bus error, reset, or interrupts. The basic concepts associated with interrupts that relate them to the 68000 have already been described. In this section, other exceptions are discussed.

In response to an exceptional condition, the processor executes a user-written program. In some computers, one common program is provided for all exceptions. The beginning section of the program determines the cause of the exception and branches to the appropriate routine. The 68000 uses a more-general approach. Each exception can be handled by a separate program.

As mentioned before, the MC68000 has two modes of operation: user state and supervisor state. The operating system runs in supervisor mode and all other programs are executed in user mode. The supervisor state is more privileged. Several privileged instructions such as MOVE to SR can only be executed in supervisor mode. Any attempt to execute them otherwise causes a trap. The MC68000 exceptions can be caused by external reset, instructions causing traps, bus and address errors, tracing, execution of privileged instructions in user mode, and execution of illegal/unimplemented instructions. The reset exception is generated externally. In response, the MC68000 automatically loads the initial starting address in the processor.

The MC68000 has a TRAP instruction that always causes an exception. The operand for this instruction varies from 0 to 15, so there are 16 TRAP instructions. Each trap instruction has an exception vector. TRAP instructions are normally used to call a subroutine in an operating system. This automatically places the MC68000 in the supervisor state. TRAPs are also used for inserting breakpoints in a program.

Two other MC68000 instructions cause traps if a particular condition is true:

* *Microcomputer Theory and Applications With the Intel SDK-85*. Rafiquzzaman, Second Edition, 1987, Wiley. Reprinted with permission.

TRAPV and CHK. TRAPV generates an exception if the overflow flag is set. The TRAPV instruction is inserted after every arithmetic operation in a program for causing a trap whenever there is overflow. A routine can be written at the vector address for the TRAPV to tell the user an overflow has occurred.

The CHK instruction is designed to ensure that access to an array in memory is within the range specified by the user. If there is a violation of this range, an exception is generated.

Consider bus and address errors. A bus error occurs when the MC68000 tries to access an address that does not belong to the devices connected to the bus. When this error is detected, the MC68000 can execute a user-written routine located at an address obtained from the exception vectors. An address error occurs when the MC68000 tries to read or write a word (16 bits) or long word (32 bits) in an odd address. This address error has a different exception vector than the bus error.

The trace exception in the MC68000 can be generated by setting the trace bit in the status register. This causes the MC68000 to indicate an internal exception after execution of every instruction. The user can write a routine at the exception vectors for the trace instruction to display registers and memory. The trace exception provides the MC68000 with a single-stepping debugging feature.

6.8 I/O PROCESSOR*

Large computers use auxiliary processors called I/O processors (IOPs) to communicate with I/O devices. An IOP contains its own processor and uses the DMA technique to communicate with I/O devices. A typical large computer system may consist of the CPU, a main memory unit, and several IOPs. These IOPs perform most of the I/O functions for the CPU and free the CPU of I/O activities. IOPs are also called *data channels*.

A typical system utilizing IOPs is given in Figure 6.12.

Each IOP talks to the corresponding I/O devices using its I/O bus. Since all data transfers between the primary memory and I/O devices take place via the IOP, the term *channel* is used interchangeably with IOP by some manufacturers. Since many I/O devices can communicate through a single I/O bus, the following items should be considered while designing the system:

1. A priority system is necessary for several IOPs sharing the single system bus.
2. A priority system is required for several I/O devices trying to access the single I/O bus.
3. The complex task of managing several I/O devices by a single IOP should be performed by either software, hardware, or a combination of both.

A computer system using IOPs may have to recognize two types of I/O instructions, including ones provided by the main CPU and others associated with the IOP.

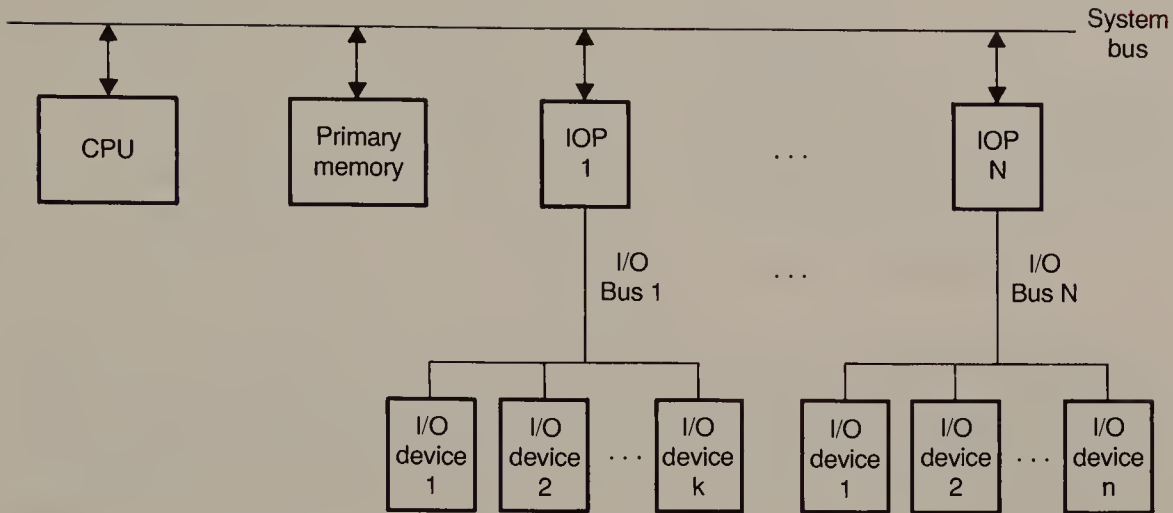


Figure 6.12 A Typical Computer System with IOP's

The main CPU requires a few basic I/O instructions. Yet the IOP performs detailed I/O operations based on commands from the main CPU. The main CPU has basic I/O commands such as START I/O, CHECK I/O and STOP I/O.

A typical format for an I/O instruction consists of three fields: op-code field, device address field, and memory address field. The main CPU decodes the instruction and translates the op-code field into a command to START, CHECK, or STOP I/O. The main CPU places the device address on the system bus. The IOPs decode this to select the appropriate IOP.

The memory address field is used only by the START I/O instruction. In response to the START I/O command, the selected IOP uses the memory address field to determine the starting address of the I/O program in IOP. The memory address field is not necessary if each IOP is provided with a register containing the starting address of the I/O program.

The complexity of IOP's depends on the types of devices connected to them. For devices capable of performing large volumes of data transfers, it may be necessary for an IOP to allow only one device to communicate with it at a time. Slower devices, such as printers and keyboards, may be allowed to communicate with the IOP under a priority scheme. Each device may have its own buffer register for data, a status bit indicating read/busy, and address register for identifying it. The IOP communicates with each device via these registers.

The basic features of an IOP can be illustrated by using the Intel 8089. The Intel 8089 is a general purpose I/O system built on a single 40-pin chip. Two independent I/O channels with DMA capabilities are provided. The 8089 has 50 different types of instructions for performing efficient I/O operations. The 8089 is directly compatible with the Intel 8086 and 8088 16-bit microprocessors. A typical 8086/8089 configuration block diagram is shown in Figure 6.13.

The 8086 activates the channel attention line to talk to the 8089. The 8089 can interrupt the 8086 by the interrupt-request line. The 8086 performs an I/O operation by forming a message for the 8089 in the main memory. The 8086 activates the channel

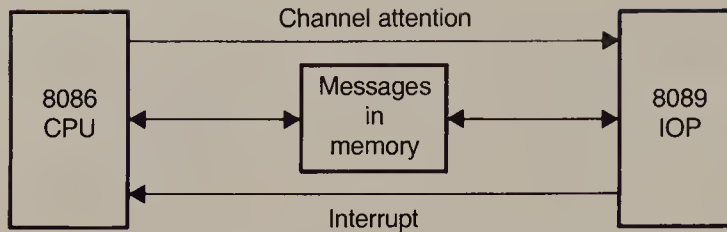


Figure 6.13 8086/8089 Block Diagram (Courtesy of Intel)

attention line to tell the IOP that an I/O operation is to be performed. The message provided by the 8086 specifies the function to be performed by the 8089. The 8089 reads the message and executes the specified operation by activating the required channel program. The 8089 informs the 8086 of completion of the channel program by activating the interrupt-request line. Typical I/O commands include starting or stopping an I/O program.

QUESTIONS AND PROBLEMS

- 6.1** Define programmed I/O, interrupt I/O, and DMA.
- 6.2** What is the difference between standard I/O and memory-mapped I/O?
- 6.3** Distinguish the difference between maskable and nonmaskable interrupts.
- 6.4** Explain briefly the basic difference between polled and daisy chain interrupts.
- 6.5** Interface two MC68230's to a Motorola 68000 chip to provide two 16-bit I/O ports.
- 6.6** In a MC68000-based microcomputer, port A has three switches connected to bits 0, 1, and 2. An LED is connected to bit 2 of Port B. Write an MC68000 assembly language program to turn the LED ON if port A has all low switch inputs; otherwise turn the LED OFF. Use the following port addresses: FFFFF8_{16} for port A; FFFFFC_{16} for DDRA; FFFFFA_{16} for Port B; and FFFFFE for DDRB. Assume that all ports are bit configurable and 8-bit wide.
- 6.7** It is desired to interface a pump to an MC68000-based microcomputer. The pump can be started by the microcomputer at the trailing edge of a START pulse via bit 2 of Port A. When the pump runs, a status signal from the pump called PUMP RUNNING goes to HIGH. The microcomputer polls this status through bit 0 of Port A. If the pump runs, the microcomputer turns an LED ON that is connected to bit 4 of Port A; otherwise, the LED is turned OFF. Write an MC68000 assembly language program to accomplish this. Use address FFFFF8_{16} for port A, and FFFFFC_{16} for DDRA. Assume port A is bit configurable and 8-bit wide.
- 6.8** Discuss MC68000 interrupt I/O.
- 6.9** Repeat Problem 6.7 using both autovector and nonautovector interrupts on the MC68000.

- 6.10** Suppose the four pumps (P1, P2, P3, P4) and four LED's (L1, L2, L3, L4) of Problem 6.7 are to be connected via the interrupt line 3 of the MC68000.
- a)** Assume an MC68000-based microcomputer with the priorities of the pumps as P3, P2, P1, P4. The microcomputer starts pumps P3, P2, P1, P4, respectively through bits 0, 1, 2, 3, of port A. Using the polled interrupt technique, implement these priorities for the pumps, and service each device as described in Problem 6.7.
 - b)** Repeat the problem using a daisy chain.
- 6.11** Discuss the importance of exception handling and error recovery in MC68000.
- 6.12** Discuss the main features of a typical IOP. Choose a popular IOP such as the Intel 8089 and explain how some of these features are implemented.

REFERENCES

1. Intel. *MCS-85 User' Manual*, 1982.
2. Intel. *Intel Data Manual*, 1982.
3. King, T. and Knight B. *Programming the MC68000*. Reading Mass.: Addison-Wesley, 1983.
4. Lewin, M., *Logic Design and Computer Organization*. Reading Mass.: Addison-Wesley, 1983.
5. Mano, M., *Computer System Architecture*. Englewood Cliffs, N.J.: Prentice-Hall, 1982.
6. Motorola. *MC 68000 Users Manual*, 1982.
7. Motorola, *Microprocessor Data Manual*, 1981.
8. Motorola. *MC 68000 Supplement Material* (Technical Training), 1982.
9. Peterson, J. and A. Silberschatz. *Operating System Concepts*. Reading, Mass.: Addison-Wesley, 1983.
10. Rafiquzzaman, M., *Microcomputer Theory and Applications with the Intel SDK-85*. New York: John Wiley, 1982.
11. Rafiquzzaman, M.. *Microprocessor-Based Systems*. New York, Wiley, 1984.
12. Tanenbaum, A. S.. *Structured Computer Organization*. Englewood Cliffs, N.J.: Prentice-Hall, 1984.
13. Wakerly, John P. *Microcomputer Architecture and Programming*. New York: John Wiley, 1981.

CHAPTER 7

Fundamentals of Parallel Processing

The hardware and software concepts covered earlier can be employed to design more-sophisticated systems called *parallel computers*. The purpose of this chapter is to provide an introduction to this growing area. The key topics addressed here are pipelining and multiprocessing.

7.1 INTRODUCTION

The term *parallel processing* means improving the performance of a computer system by carrying out several tasks simultaneously. A high volume of computation is often required in many application areas, including real-time signal processing. For example, a real-time 1024 point fast-fourier transform (FFT) processor is expected to perform 10240 floating-point multiplication in 1 microsecond (1 megahertz bandwidth). Therefore, to meet such demands, it is mandatory to design a new architecture that facilitates concurrent computations.

At present, IC processes such as the 1.5-micron process, allow the integration of several thousand functional elements within a given real-estate area for a reasonable cost. This trend indicates the feasibility of producing high-speed computers by designing

the CPU with several execution units. Hence computations can be performed in parallel to save execution time.

7.2 PARALLELISM IN CONVENTIONAL COMPUTERS

A conventional single CPU computer comprises three functional elements: CPU, memory, and I/O devices. In such a uniprocessor system, a reasonable degree of parallelism is achieved in the following manner:

1. The IBM 370/168 and CDC6600 computers include a dedicated I/O processor. This additional unit is capable of performing all I/O operations by employing the DMA technique explained earlier. In these systems, parallelism is achieved by keeping the CPU and I/O processor busy (as much as possible) with program execution and I/O operations, respectively.
2. In the CDC6600 CPU, there are 24 registers and 10 execution units. Each execution unit is specialized for a specific operation, such as addition, multiplication, and shifting. Since all units are independent of each other, several operations can be performed simultaneously. Execution unit scheduling is carried out by using hardware called a *score board*, which indicates whether or not a functional unit is busy.
3. In many uniprocessor systems such as the IBM 360 series computers, parallelism is achieved by using high-speed hardware elements such as carry-save adders, carry-look-ahead adders, and Wallace tree multipliers discussed previously.
4. In several conventional computers, parallelism is incorporated at the instruction-execution level. Recall that an instruction cycle typically includes activities such as op-code fetch, instruction decode, operand fetch, operation execution, and result saving. All these operations can be carried out by overlapping the instruction fetch phase with the instruction execution phase. This implementation is known as an *instruction pipeline*.
5. In many uniprocessor systems high throughput is achieved by employing high-speed memories, such as cache and associative memories discussed earlier. The use of virtual memory concepts such as paging and segmentation also allows one to achieve a high processing rate because they reduce the speed imbalance between a fast CPU and a slow peripheral device such as a disk unit.
6. It is common practice to achieve parallelism by employing software methods such as multiprogramming and time sharing in uniprocessors. In both approaches, the CPU is multiplexed among several jobs. This results in concurrent processing, which improves the overall system throughput.

7.3 GENERAL CLASSIFICATIONS OF COMPUTER ARCHITECTURES

Over the last two decades, parallel processing has drawn the attention of many research workers, and several high-speed architectures have been proposed. To present these results in a concise manner, different architectures must be classified in well defined groups.

All computers may be categorized into different groups using one of three classification methods:

1. Flynn
2. Feng
3. Handler

The two principal elements of a computer are the processor and the memory. A processor manipulates data stored in the memory as dictated by the instruction. Instructions are stored in the memory unit and always flow from memory to processor. Data movement is bidirectional, meaning data may be read from or written into the memory. Figure 7.1 shows the processor-memory interaction.

The number of instructions read and data items manipulated simultaneously by the processor form the basis for Flynn's classification.

Figure 7.2 shows the four types of computer architectures that are defined using Flynn's method.

The SISD computers are capable of manipulating a single data item by executing one instruction at a time. The SISD classification covers the conventional uniprocessor systems such as the VAX-11 and IBM 370 computers. The processor unit of a SISD machine may have one or many functional units. For example, the VAX-11/780 is a SISD machine with a single functional unit. CDC 6600 and IBM 370/168 computers are typical examples of SISD systems with multiple functional units. In a SISD machine, instructions are executed in a strictly sequential fashion.

The SIMD system allows a single instruction to manipulate several data elements. These machines are also called vector machines or array processors. Examples of this type of computer are the ILLIAC-IV and Burroughs Scientific Processor (BSP). The

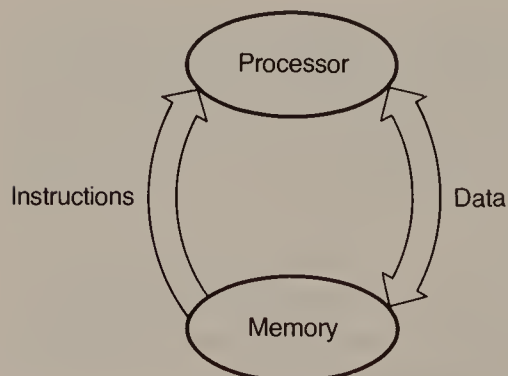


Figure 7.1 Processor-Memory Interaction

NAME OF THE ARCHITECTURE	NAME OF THE ARCHITECTURE IN ABBREVIATED FORM
Single-instruction stream–single-data stream	SISD
Single-instruction stream–multiple-data stream	SIMD
Multiple-instruction stream–single-data stream	MISD
Multiple-instruction stream–multiple-data stream	MIMD

Figure 7.2 Classification of Computers Using Flynn's Method

ILLIAC-IV was an experimental parallel computer proposed by the University of Illinois and built by the Burroughs Corporation. In this system, there are 64 processing elements. Each processing element has its own small local memory unit. The operation of all the processing elements is under the control of a central control unit (CCU). Typically, the CCU reads an instruction from the common memory and broadcasts the same to all processing units so the processing units can all operate on their own data at the same time. This configuration is very useful for carrying out a high volume of computations that are encountered in application areas such as finite-element analysis, logic simulation, and spectral analysis.

By definition, MISD refers to a computer in which several instructions manipulate the same data stream concurrently. This concept is only of theoretical interest, as no system has been built using this idea. However, the notion of pipelining is very close to the MISD definition.

A set of instructions constitute a program, and a program operates on several data elements. MIMD organization refers to a computer that is capable of processing several programs simultaneously. MIMD systems include all multiprocessing systems. With the advent of low-cost microprocessors, the idea of building multiprocessor systems is practically feasible. Typical examples of popular multiprocessor systems include Carnegie Mellon University's Cm* and Cray Research's Cray-2 computers. Based on the degree of processor interaction, multiprocessor systems may be further divided into two groups: loosely coupled and tightly coupled. A tightly coupled system has high interaction between processors. Multiprocessor systems with low interprocessor communications are referred to as *loosely coupled* systems.

In Feng's approach, computers are classified according to the number of bits processed within a unit time. However, Händler's classification scheme categorizes computers on the basis of the amount of parallelism found at the following levels:

- CPU
- ALU
- Bit

A thorough discussion of these schemes is beyond the scope of this book. However, interested readers may consult the references listed at the end of this chapter.

7.4 ARRAY PROCESSORS

In the previous section, Flynn's scheme was found to give rise to two important families of parallel computers: SIMD and MIMD machines. The SIMD computers are also called *array processors*.

A synchronous array processor may be defined as a computer in which a set of identical processing elements act under the control of a master controller (MC). A command given by the MC is simultaneously executed by all processing elements, and a SIMD system is formed. Since all processors execute the same instruction, this organization offers a great attraction for vector processing applications such as matrix manipulation.

A conceptual organization of a typical array processor is shown in Figure 7.3. The MC controls the operation of the processor array. This array consists of N identical processing elements (P_0 through P_{N-1}). Each processing element P_i is assumed to have its own memory, PM_i , to store its data. The MC of Figure 7.3 contains two major components:

- The master control unit (MCU)
- The master control memory (MCM)

The MCU is the CPU of the master controller and includes an ALU and a set of registers. The purpose of the MCM is to hold the instructions and common data.

Each instruction of a program is executed under the supervision of the MCU in a sequential fashion. The MCU fetches the next instruction, and the execution of this instruction will take place in one of the following ways:

- If the instruction fetched is a scalar or a branch instruction, it is executed by the MC itself.
- If the instruction fetched is a vector instruction, such as vector add or vector multiply, then the MCU broadcasts the same instruction to each P_i of the processor array, allowing all P_i 's to execute this instruction simultaneously.

Assume the required data is already within the processing element's private memory. Before the execution of a vector instruction, make sure appropriate data values are routed to each processing element's private memory. Such an operation can be performed in two ways:

- All data values can be transferred to the private memories from an external source via the system data bus.
- The MCU can broadcast the data values to the private memories via the control bus.

7.4.1 Systolic Arrays

A systolic system comprises a set of interconnected cells, each capable of performing a simple operation. Since each cell is simple, it is possible to design simple control circuitry. Normally, cells in a systolic system are interconnected as an array or tree. Information in a systolic system flows between cells in a pipelined fashion.

A basic systolic system is shown in Figure 7.4. It differs from a conventional system in two aspects:

1. A single processing element (PE) is replaced with an array of PEs. Thus it is possible to increase system throughput without increasing the memory bandwidths.
2. The function of the memory in a systolic system is analogous to the heart. It pulses data through the array of PEs.

The essence of this approach is to ensure that a data item fetched from the memory can be effectively used at each cell it passes. Many compute-bound problems (i.e. convolution, matrix multiplication, and LU decomposition) strongly favor this idea because in all these, several operations are performed on a single data item in a repetitive fashion.

Consider the systolic cell shown in Figure 7.5.

This cell has two inputs: upper input X and left input Y . This module multiplies the input X by a fixed value W and then adds the product to Y . Thus, the output is $Z = W * X + Y$.

Consider the configuration shown in Figure 7.6. In this case, three basic cells are interconnected as a one-dimensional array:

After the first clock, the outputs computed by various cells are shown in Figure 7.7(a). The output of the rightmost cell is W_0X_0 . After the second clock, the output generated at various points are shown in Figure 7.7(b). The rightmost cell generates an output after the third clock, $W_0X_2 + W_1X_1 + W_0X_0$.

In general, the output of the right most cell after the $(k + 1)$ st clock will be:

$$Y = W_0X_k + W_1X_{k-1} + W_2X_{k-2}$$

$$= \sum_{i=0}^2 W_iX_{k-i}$$

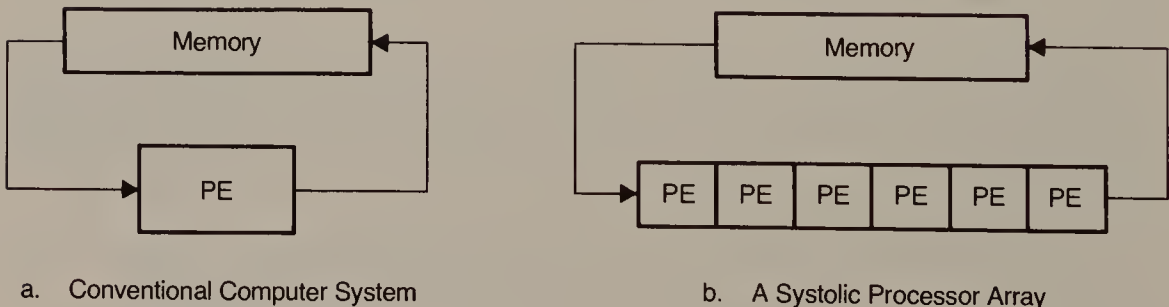


Figure 7.4 Differences Between a Conventional Computer and a Systolic System (Kung, H.T., *Why Systolic Architecture*, IEEE Comp., Jan. 1982, pp. 37–46.)

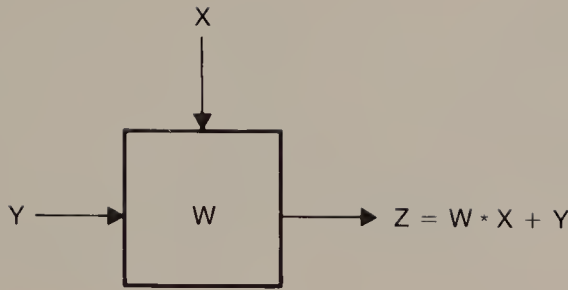


Figure 7.5 Systolic Array Cell

This summation is the time domain convolution of two discrete sequences. Close examination reveals that the systolic system shown in Figure 7.6 implements a second-order finite-impulse response (FIR) digital filter.

The key advantages of a systolic array include structured data flow, simple control structure, ease of expandability, and elimination of global broadcasting. The drawback of this concept is the bottleneck developed by the I/O barrier. The throughput of a systolic system is largely dependent on the rate at which the I/O operations are carried out. This barrier limits systolic arrays to being constructed only in small sizes.

7.4.2 Wavefront Array Processors

A wavefront processor array is similar to a systolic array by its modular structure, simple connection, and localized communications. However, unlike systolic processors, there is no external clock, and wavefront array processors perform their task in an asynchronous manner. Information is transmitted whenever it has been computed, and computing begins once all of the operands are present.

A wavefront processor can be used to perform a 2×2 matrix multiplication, as explained next. Consider the wavefront array cell shown in Figure 7.8. This cell has two input channels, a left and a top.

When both inputs are present, the processor multiplies the inputs together and adds the result onto the contents of an internal memory within the cell. The top input is transferred out to the bottom output, and the left input is transmitted to the right output.

If wavefront array is used to perform the matrix multiplication

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

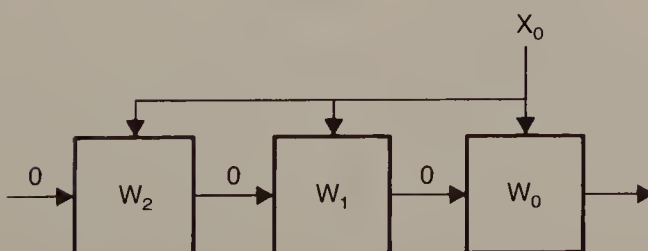


Figure 7.6 A One-dimensional Systolic Array

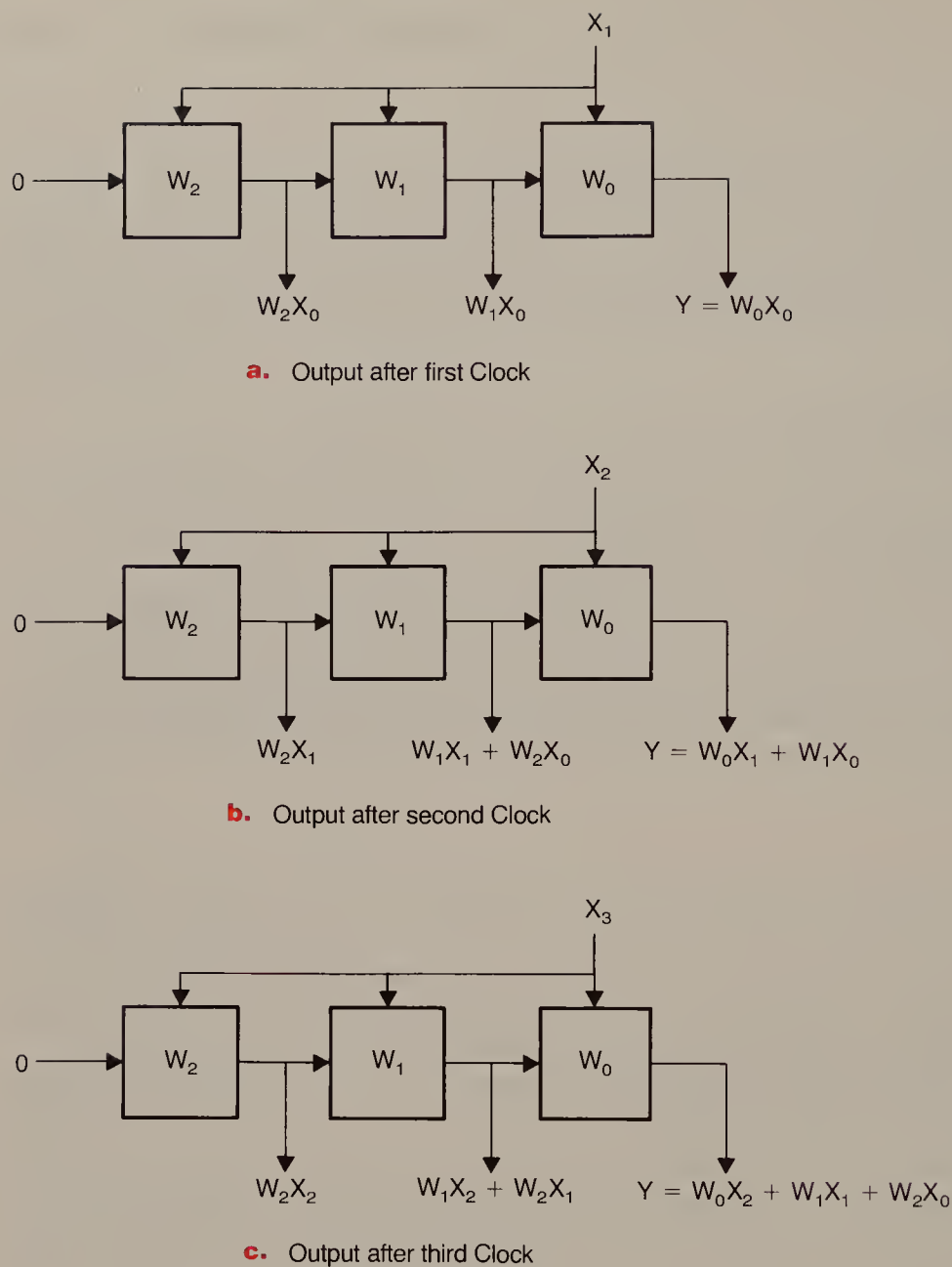


Figure 7.7

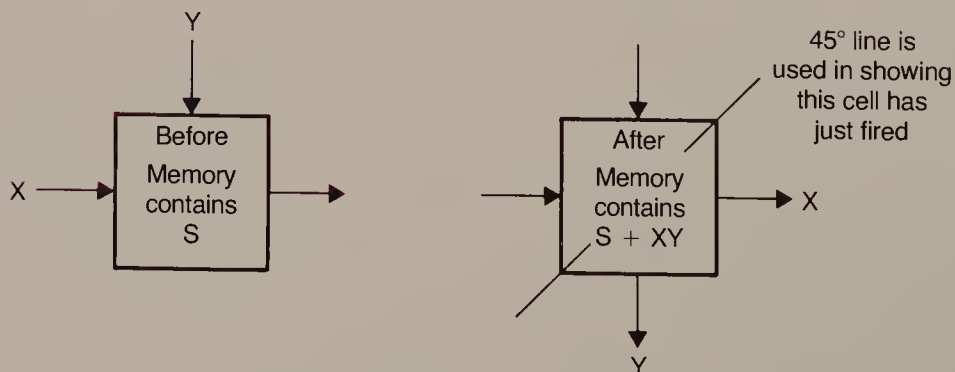


Figure 7.8 Wavefront Array Cell (Dougherty/Giardina, *Matrix Structured Image Processing* © 1987 p. 236. Reprinted by permission of Prentice-Hall, Inc. Englewood Cliffs, New Jersey.)

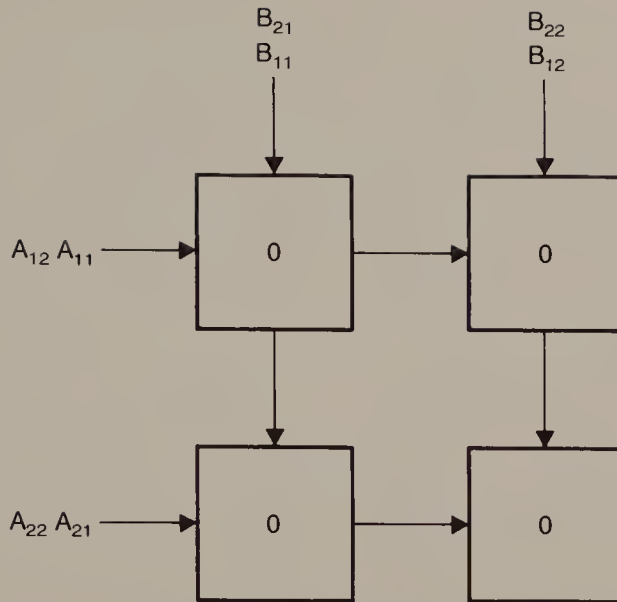


Figure 7.9 Initial Wave Front Array for 2×2 Matrix Multiplication (Dougherty/Giardina, *Matrix Structured Image Processing*, © 1987, pp. 236–237. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

to obtain the matrix

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{21} \end{bmatrix}$$

the input scheme shown in Figure 7.9 is used.

The initial contents of each cell's internal memory is initialized to 0. The desired output for this processor appears inside each cell after all the inputs are used up (after

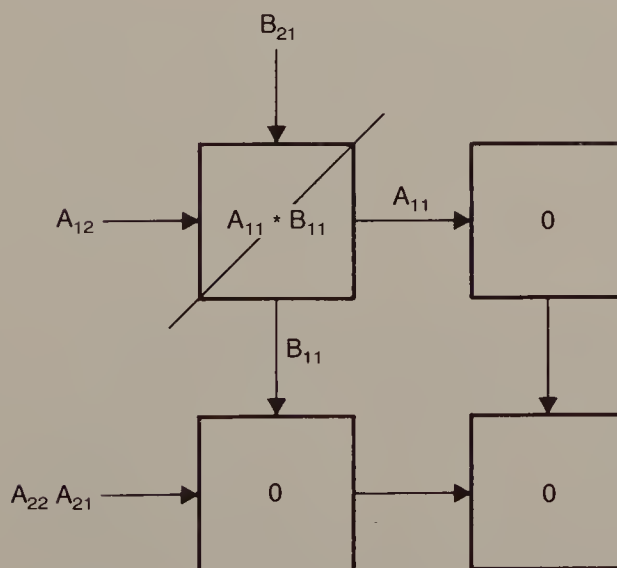


Figure 7.10 Upper-left Cell Fired (Dougherty/Giardina, *Matrix Structured Image Processing*, © 1987, pp. 236–237. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

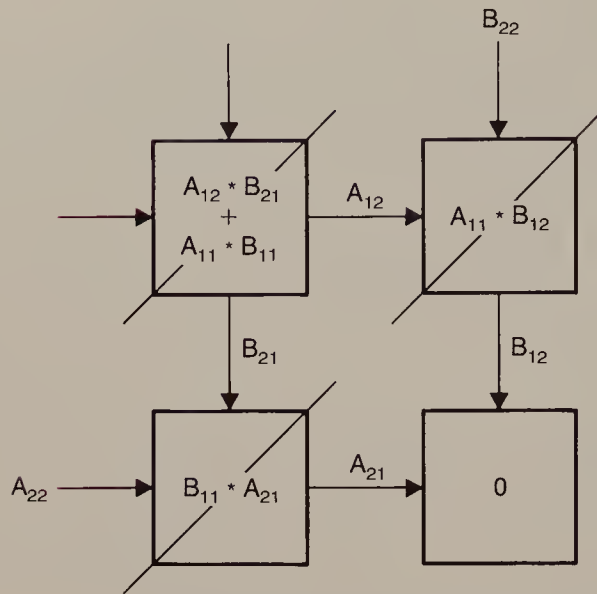


Figure 7.11 Left and Upper Cells Fired (Dougherty/Giardina, *Matrix Structured Image Processing*, © 1987, p. 237. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

each cell has executed twice). Cells execute at different times. There is no external clock. At the outset, only the upper left cell executes, since only it has both operands available. (See Figure 7.10.)

Then all cells except the bottom-right cell execute, as illustrated in Figure 7.11. (This shows two waves of information.)

Then all but the upper-left cell executes, as depicted in Figure 7.12. Finally, the bottom-right cell executes for the second time. Now all cells can be read and the results obtained. (See Figure 7.13.)

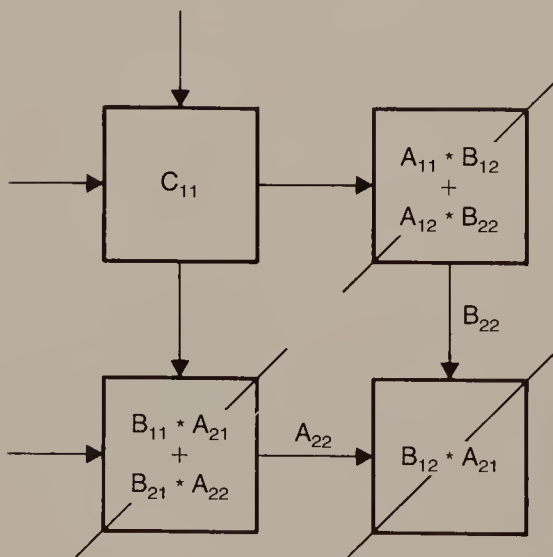


Figure 7.12 Bottom And Right Cells Fired (Dougherty/Giardina, *Matrix Structured Image Processing*, © 1987, p. 237. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

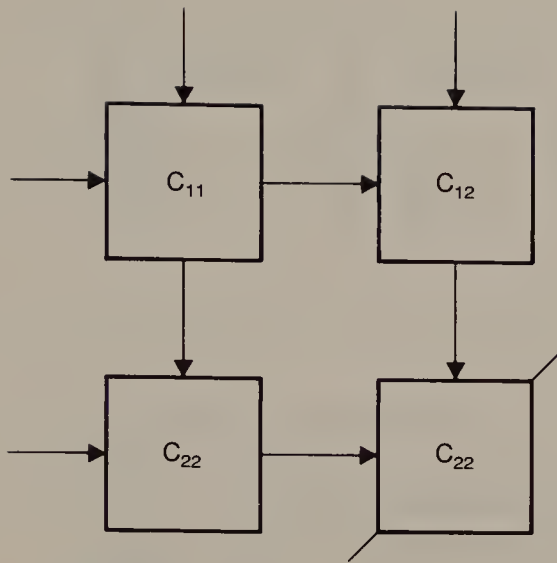


Figure 7.13 Bottom-right Cell Fired (Dougherty/Giardina, *Matrix Structured Image Processing*, © 1987, p. 237. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey)

7.5 PIPELINE PROCESSING

The second major concept that received considerable attention in the design of high-speed computers is pieplining. The purpose of this section is to provide a brief overview of this theory.

7.5.1 Basic Concepts

Assume a task T is carried out by performing four activities: A_1 , A_2 , A_3 , and A_4 , in that order. Hardware H_i is designed to perform the activity A_i . H_i is referred to as a segment, and it essentially comprises combinational circuit elements. Consider the arrangement shown in Figure 7.14.

In this configuration, a latch is placed between two segments so the result computed by one segment can serve as input to the following segment during the next clock period.

The execution of four tasks T_1 , T_2 , T_3 , and T_4 using the hardware of Figure 7.14 is described using a space-time chart shown in Figure 7.15.

Initially, task T_1 is handled by segment 1. After the first clock, segment 2 is busy with T_1 while segment S_1 is busy with T_2 . Continuing in this manner, the task T_1 is completed at the end of the fourth clock. However, following this point, one task is shipped out per clock. This is the essence of the pipeline concept. A pipeline gains efficiency for the same reason as an assembly line does: Several activities are performed but not on the same material.

Suppose T_i and L denote the propagation delays of segment i and the latch, respectively. Then the pipeline clock period T can be expressed as follows:

$$T = \max (T_1, T_2, \dots, T_n) + L$$

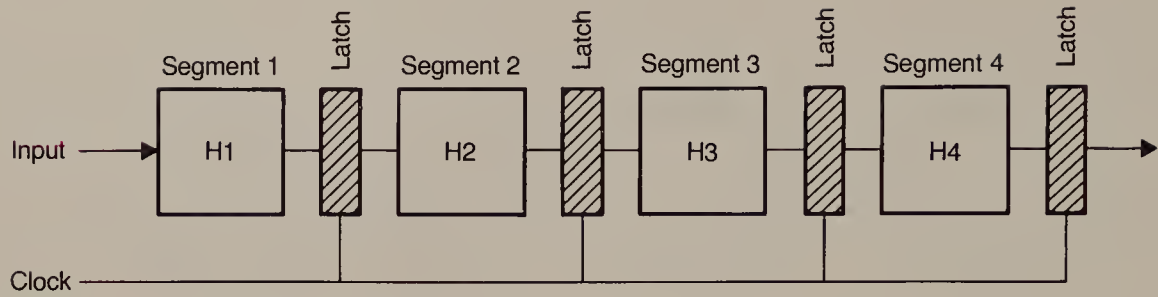


Figure 7.14 A Four-segment Pipeline

The segment with the maximum delay is known as the bottleneck, and it decides the pipeline clock period T . The reciprocal of T is referred to as the pipeline frequency.

Consider the execution of m tasks using an n -segment pipeline. In this case, the first task will be completed after n clocks (because there are n segments) and the remaining $m - 1$ tasks are shipped out at the rate of one task per pipeline clock.

Therefore, $n + (m - 1)$ clock periods are required to complete m tasks using an n -segment pipeline. If all m tasks are executed without any overlap, mn clock periods are needed because each task has to pass through all n segments. Thus speed gained by an n segment pipeline can be shown as follows:

$$\text{speedup } P(n) = \frac{\begin{array}{l} \text{number of clocks} \\ \text{required when there} \\ \text{is no overlap} \end{array}}{\begin{array}{l} \text{number of clocks} \\ \text{required when tasks} \\ \text{are overlapped in} \\ \text{time} \end{array}} = \frac{mn}{n + m - 1}$$

$P(n)$ approaches n when m approaches infinity. This implies that when a large number of tasks are carried out using an n -segment pipeline, an n -fold increase in speed can be expected.

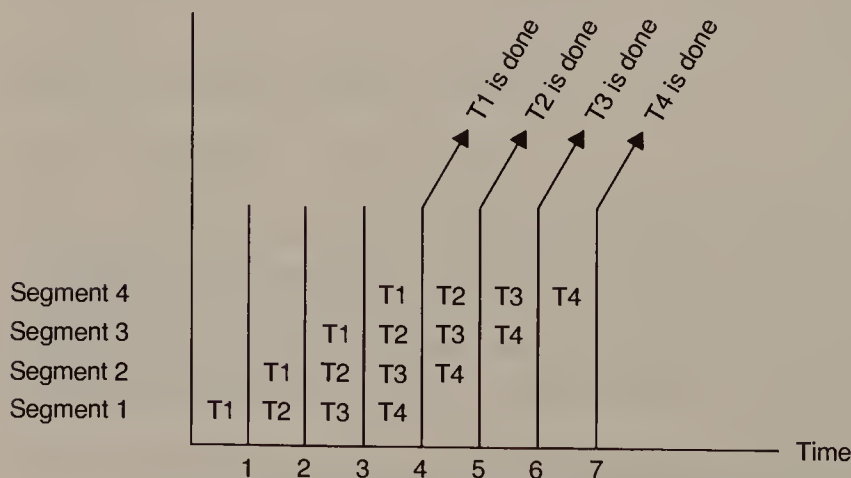


Figure 7.15 Overlapped Execution of Four Tasks Using A Pipeline

The previous result shows that the pipeline completes m tasks in the $m + n - 1$ clock periods. Therefore, its throughput can be defined as follows:

$$\begin{array}{l} \text{throughput} \\ \text{of an } n\text{-} \\ \text{segment} \\ \text{pipeline} \end{array} = U(n) = \begin{array}{l} \text{number of} \\ \text{tasks} \\ \text{computed} \\ \text{per unit} \\ \text{time} \end{array} = \frac{m}{(n + m - 1)T}$$

For a large value of m , $U(n)$ approaches $1/T$, which is the pipeline frequency. Thus the throughput of an ideal pipeline is equal to the reciprocal of its clock period.

The efficiency of an n -segment pipeline is defined as the ratio of the actual speed-up to the maximum speedup realized more formally.

$$\begin{array}{l} \text{efficiency} \\ \text{of an } n\text{-} \\ \text{segment} \\ \text{pipeline} \end{array} = E(n) = \frac{\text{actual speedup}}{\text{maximum speedup}} = \frac{P(n)}{n}$$

This illustrates that when m is very large, $E(n)$ approaches 1 as expected.

In many modern computers, the pipeline concept is used in carrying out two tasks: arithmetic operations and instruction execution.

7.5.2 Arithmetic Pipelines

The pipeline concept can be used to build high-speed multipliers. Consider the multiplication $P = M * Q$, where M and Q are 8-bit numbers. The 16-bit product P can be expressed as:

$$\begin{aligned} P &= M(q_7 2^7 + q_6 2^6 + q_5 2^5 + q_4 2^4 + q_3 2^3 + \\ &\quad q_2 2^2 + q_1 2^1 + q_0 2^0) \\ &= \sum_{i=0}^7 M q_i 2^i \end{aligned}$$

This result can also be rewritten as:

$$P = \sum_{i=0}^7 S_i$$

where, $S_i = M q_i 2^i$ and each S_i represents a 16-bit partial product. Each partial product is the shifted multiplicand. All 8 partial products can be summed up using the Wallace tree adder, which is a network of carry-save adders. By placing a latch between two successive levels, a 6-segment pipelined multiplier can be obtained, as shown in Figure 7.16.

This concept can be extended to design an $n \times n$ pipelined multiplier. Here n partial products must be summed with $2n$ bits per partial product. So, as n increases, the hardware cost associated with a fully combinational Wallace tree multiplier increases in an exponential fashion. To reduce the hardware cost, large multipliers are designed

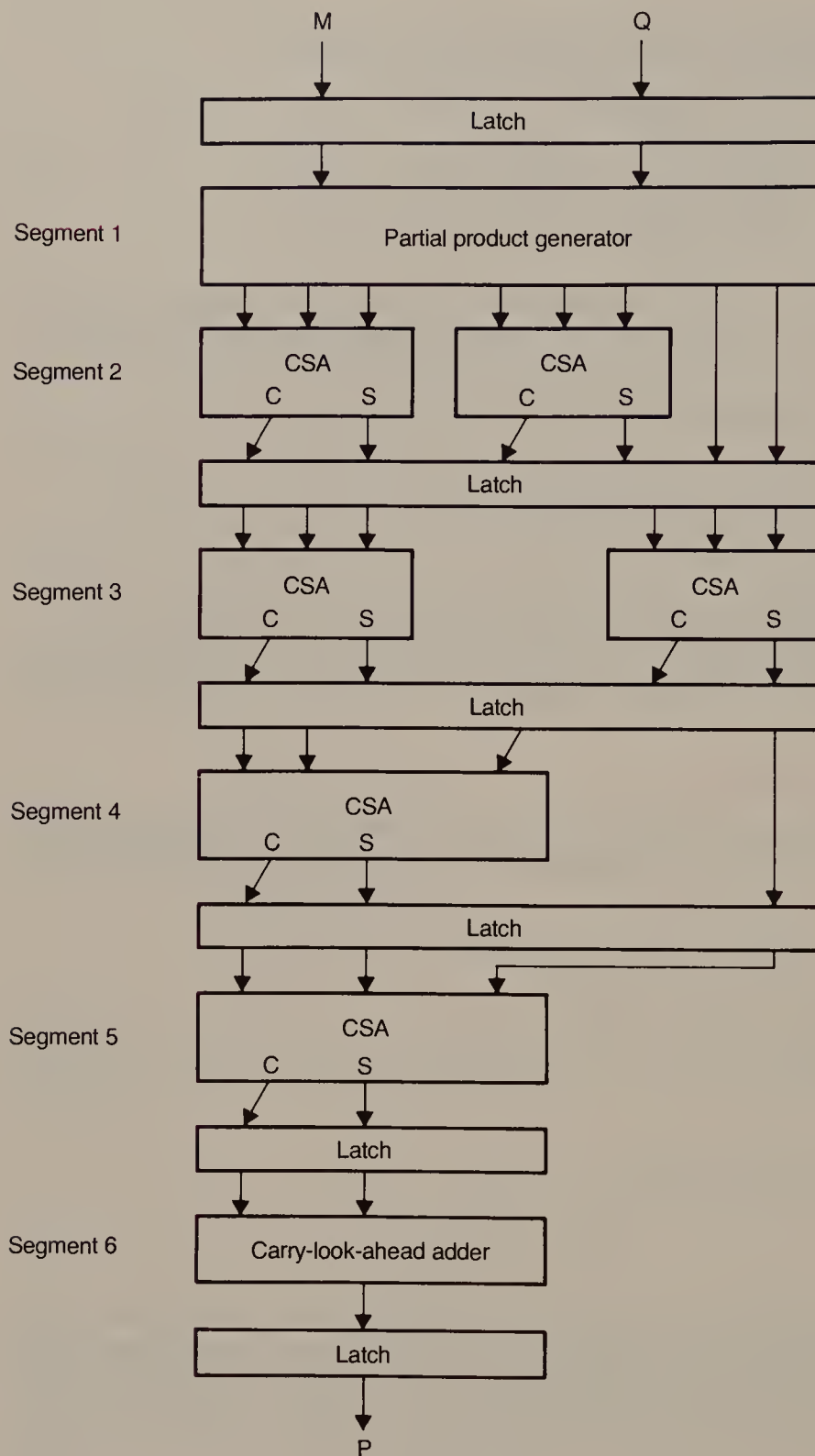


Figure 7.16 A 6-segment 8×8 Pipelined Wallace Tree Multiplier

using pipelines with feedback inputs. A thorough discussion of this topic is found in the work of K. Hwang and F. A. Briggs [8].

The pipeline concept is widely used in designing floating-point arithmetic units. Consider the process of adding two floating point numbers $A = 0.9234 * 10^4$ and $B = 0.48 * 10^2$. First, notice that the exponents of A and B are unequal. This suggests a modification of the smaller number so that its exponent is equal to the exponent of the greater number. For this example, modify B to $0.0048 * 10^4$. This modification step is known as *exponent alignment*. Here the decimal point of the significand 0.48 is shifted to the right to obtain the desired result.

After the exponent alignment, the significands 0.9234 and 0.0048 are added to obtain the final solution of $0.9282 * 10^4$.

For a second example, consider the operation $A - B$, where $A = 0.9234 * 10^4$ and $B = 0.9230 * 10^4$. In this case, no exponent alignment is necessary because the exponent of A equals the exponent of B. Therefore, subtract the significand of B from the significand of A to obtain $0.9234 - 0.9230 = 0.0004$. However, $0.0004 * 10^4$ cannot be the final answer because the significand, 0.0004, is not normalized. A floating-point number with base b is said to be *normalized* if the magnitude of its significand satisfies the following inequality:

$$\frac{1}{b} \leq |\text{significand}| < 1$$

In this example, since $b = 10$, a normalized floating-point number must satisfy the condition.

$$0.1 \leq |\text{significand}| < 1$$

(Note that normalized floating-point numbers are always considered because for each real-world number there exists one and only one floating-point representation. This uniqueness property allows processors to make correct decisions while performing compare operations).

The final answer is modified to $0.4 * 10^3$. This modification step is known as postnormalization, and here the significand is shifted to the left to obtain the correct result.

In summary, addition or subtraction of two floating-point numbers calls for four activities:

1. Exponent comparison
2. Exponent alignment
3. Significand addition or subtraction
4. Postnormalization

Based on this result, a four-segment floating-point adder/subtractor pipeline can be built, as shown in Figure 7.17.

It is important to realize that each segment in this pipeline is primarily composed of combinational components such as multiplexers. The shifter used in this system is the barrel shifter discussed earlier.

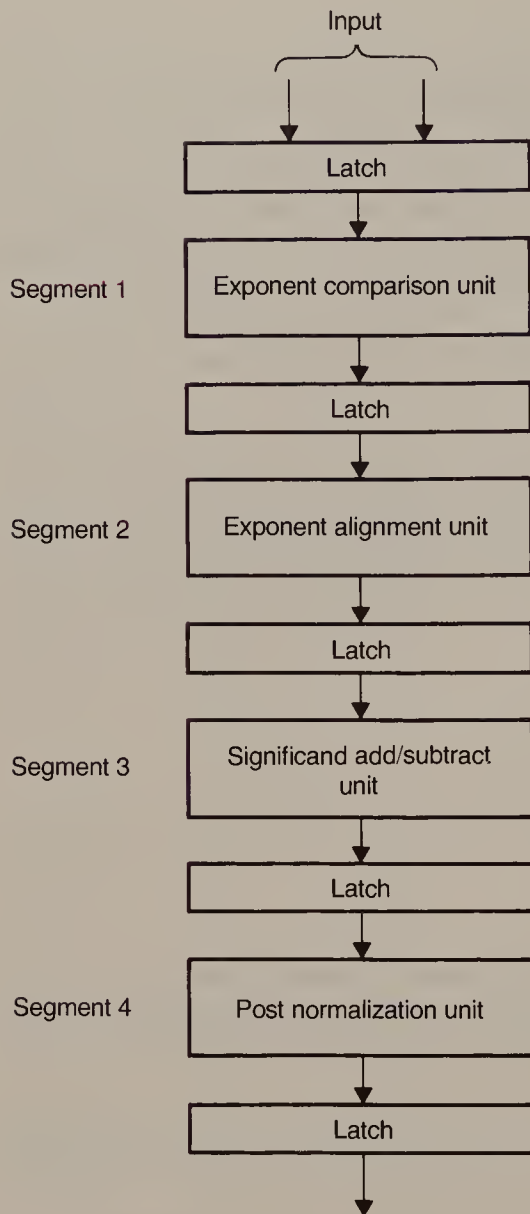


Figure 7.17 A Pipelined Floating-point Add/Subtract Unit

Readers may refer to the work of S. Waser and M. J. Flynn [18] for a detailed discussion on the hardware design of a floating-point add/subtract unit.

7.5.3 Instruction Pipelines

Recall that an instruction cycle typically involves the following activities:

1. Instruction fetch
2. Instruction decode
3. Operand fetch

4. Operation execution

5. Result routing

This process can be effectively carried out by using the pipeline shown in Figure 7.18.

As mentioned earlier, in such a pipelined scheme the first instruction requires five clocks to complete its execution. However, the remaining instructions are completed at

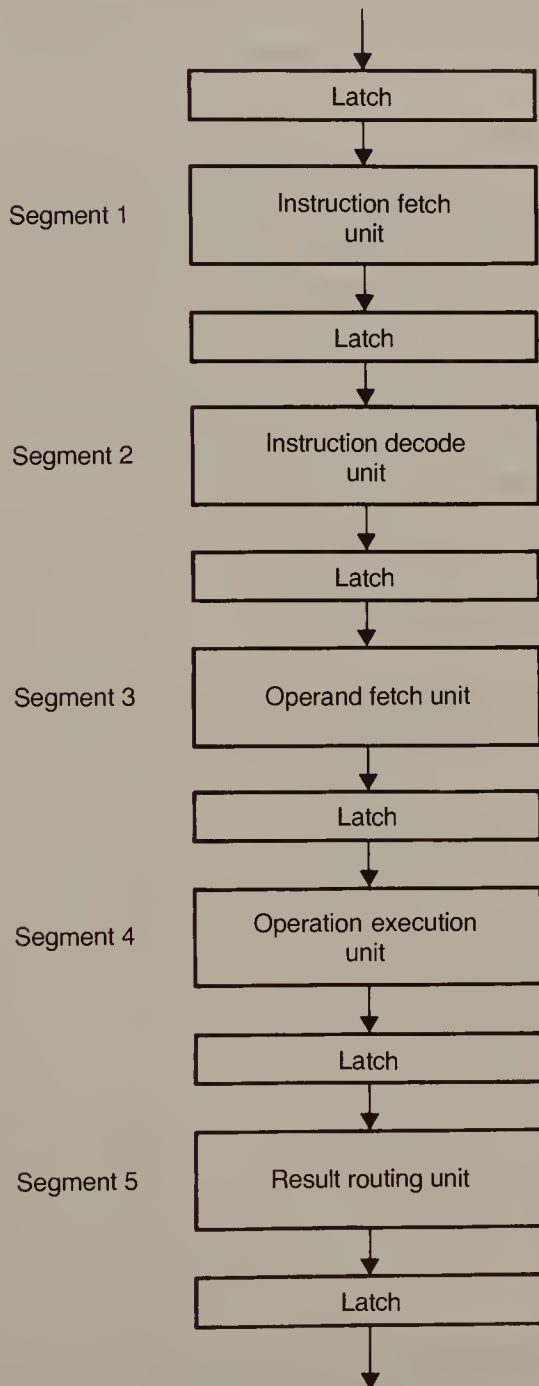


Figure 7.18 A Five-segment Instruction Pipeline

a rate of one per pipeline clock. Such a situation prevails as long as all the segments are busy.

In practice, the presence of branch instructions and conflicts in memory accesses poses a great problem to the efficient operation of an instruction pipeline.

For example, consider the execution of a stream of five instructions: I1, I2, I3, I4, and I5. I3 is a conditional branch instruction. This stream is processed by the instruction pipeline (of Figure 7.18) as depicted in Figure 7.19.

When a conditional branch instruction is fetched, the next instruction cannot be fetched because the exact target is not known until the conditional branch instruction has been executed. The next fetch can occur once the branch is resolved. Four additional clocks are required due to I3.

Suppose a stream of s instructions is to be executed using an n -segment pipeline. If c is the probability for an instruction to be a conditional branch instruction, there will be sc conditional branch instructions in a stream of s instructions. Since each branch instruction requires $n - 1$ additional clocks, the total number of clocks required to process a stream of s instructions is

$$(n + s - 1) + sc(n - 1)$$

An instruction cycle constitutes n pipeline clocks. Therefore, the total number of instruction cycles required to execute an instruction is

$$I = \frac{(n + s - 1) + sc(n - 1)}{n}$$

The average number of instructions executed per instruction cycle is

$$\frac{s}{I} = \frac{sn}{(n + s - 1) + sc(n - 1)} = \frac{n}{\frac{n}{s} + \frac{(s - 1)}{s} + c(n - 1)}$$

For a large value of s , the preceding result can be simplified to:

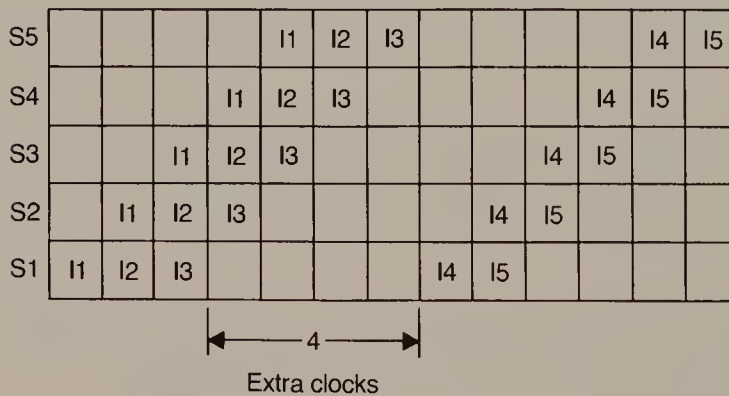


Figure 7.19 Pipelined Execution Of A Stream of Five Instructions that Includes a Branch Instruction

$$\lim_{s \rightarrow \infty} \frac{s/I}{1 + c(n-1)} = \frac{n}{1 + c(n-1)}$$

For $n = 5$, the equation becomes:

$$\frac{5}{1 + 4c}$$

For no conditional branch instructions ($c = 0$), 5 instructions per instruction cycle are executed. This is the best result produced by a five-segment pipeline. If 25% of the instructions are branch instructions only,

$$\frac{5}{1 + 4 \times 0.25} = 2.5 \text{ instructions}$$

per instruction cycle can be executed. This shows how pipeline efficiency is significantly decreased even with a small percentage of branch instructions.

In many contemporary systems, branch instructions are handled using a strategy called *Target Prefetch*. When a conditional branch instruction is recognized, the immediate successor of the branch instructions and the target of the branch are prefetched. The latter is saved in a buffer until the branch is executed. If the branch condition is successful, one pipeline is still busy because the branch target is in the buffer.

Another approach to handle branch instructions is the use of the delayed branch concept. In this case, the branch is assumed to not take place until after the following instruction. To illustrate this, consider the instruction sequence shown in Figure 7.20.

Suppose the compiler inserts a NOP instruction and changes the branch instruction to JMP 2051. The program semantics remain unchanged. This is shown in Figure 7.21.

MEMORY ADDRESS	INSTRUCTION
2000	LDA X
2001	INC Y
2002	JMP 2050
2003	SUB Z
.	
.	
.	
.	
.	
2050	STA W
.	
.	
.	
.	
.	

Figure 7.20 A Hypothetical Program

MEMORY ADDRESS	INSTRUCTION
2000	LDA X
2001	INC Y
2002	JMP 2051
2003	NOP
2004	SUB Z
.	.
.	.
.	.
2051	STA W

Figure 7.21 Modified Sequence

This modified sequence will be executed by a two-segment pipeline, as shown in Figure 7.22:

Instruction fetch	LDA X	INC Y	JMP 2051	NOP	STA W
Instruction execute		LDA X	INC Y	JMP 2051	NOP

Figure 7.22 Pipelined Execution of a Hypothetical Instruction Sequence

Because of the delayed branch concept, the pipeline still functions correctly without damage.

The efficiency of this pipeline can be further improved if the compiler produces a new sequence as shown in Figure 7.23.

In this case, the compiler has reversed the instruction sequence. The JMP instruction is placed in the location 2001, and the INC instruction is moved to memory location 2002. This reversed sequence is executed by the same 2-segment pipeline, as shown in Figure 7.24.

MEMORY ADDRESS	INSTRUCTION
2000	LDA X
2001	JMP 2050
2002	INC Y
2003	SUB Z
.	.
.	.
.	.
2050	STA W

Figure 7.23 Instruction Sequence with Branch Instruction Reversed

Instruction fetch	LDA X	JMP 2050	INC Y	STA W	
Instruction execute		LDA X	JMP 2050	INC Y	

Figure 7.24 Execution of the Reversed-instruction Sequence using a 3-segment Pipeline

It is important to understand that due to the delayed branch rule, the INCY instruction is fetched before the execution of JMP 2050 instruction; therefore, there is no change in the order of instruction execution. This implies that the program will still produce the same result. Since the NOP instruction was eliminated, the program is executed more efficiently.

The concept of *delayed branch* is one of the key characteristics of RISC philosophy as it makes concurrency visible to a programmer.

A vigorous discussion on RISC ideas can be found in the research paper of J. Patteson [12]. For a textbook discussion of this topic, consult the work of W. Stallings [15].

As does the presence of branch instructions, memory-access conflicts cause damage to pipeline performance. For example, if the instructions in the operand fetch and result-saving units refer to the same memory address, these operations cannot be overlapped.

To reduce such memory conflicts, a new approach called *memory interleaving* is often employed. For this case, the memory addresses are distributed among a set of memory modules, as shown in Figure 7.25.

In this arrangement, memory is distributed among 4 modules. Since consecutive addresses are placed into different modules, the CPU can access up to a maximum of 4 words in one memory access.

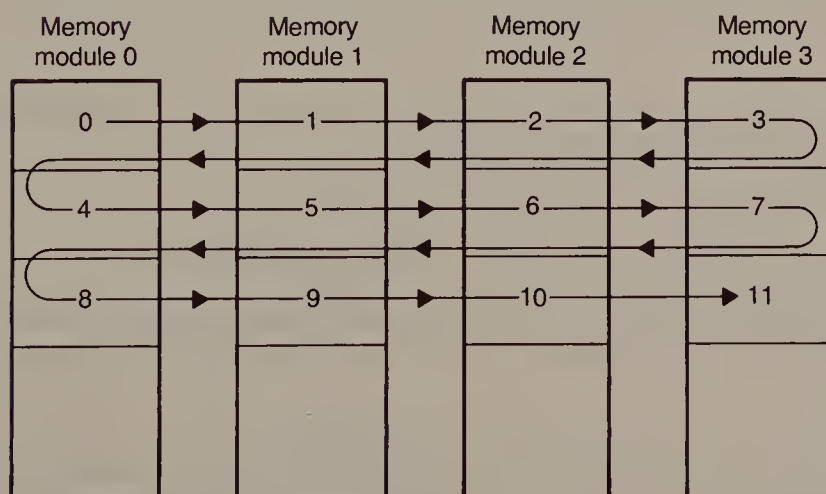


Figure 7.25 Memory Interleaving

Again, due to data dependence, the performance of such an interleaved memory organization can be severely degraded. A detailed discussion of this topic is beyond the scope of this text.

Interested readers may consult the works of K. Hwang and F. A. Briggs [8] or P. M. Kogge [9] for a quantitative treatment of this concept.

7.6 MULTIPROCESSORS*

The concept of multiprocessing is effective in the design of high-speed computer systems. A multiprocessor system may be defined as a computer system in which several processing elements are operating under the supervision of a centralized control unit. Here, a processing element refers to a CPU or an I/O processor. Normally, a computer system with at least two CPUs is called a multiprocessor. However, a computer with one CPU and one or more I/O processors is called a multiprocessor system when the capabilities of the I/O processors are close to that of the CPU.

Two predominant features of a multiprocessor system are sharing and interaction. First, a brief description of these features are given, and then the salient characteristics of a multiprocessor system are summarized.

The hardware structure of a multiprocessor is configured so all CPUs can share the main memory and so the CPU and main memory share all I/O devices. Although this type of sharing is restricted in multiprocessor systems with private memories, it can be assumed that sharing is a valid concept.

Processing elements of a multiprocessor must interact with each other to complete a function. The important point to be considered here is the unit of interaction. The basic unit of interaction may be a data element, a record, or a file of records. A multiprocessor is well designed only if it allows different levels of interaction. From the standpoint of efficient control, a true multiprocessor system must provide interaction between complete programs as well as interaction between instructions of a particular program.

It is possible to determine the hardware and software requirements of a multiprocessor only if the sharing and interaction levels are precisely defined. Following the lines of P. H. Enslow[6], the salient characteristics of a true multiprocessor system are summarized as follows:

- The system must have more than one processing element, and the capabilities of all processing elements must be nearly identical.
- All processing elements share one common memory unit.
- All I/O devices and I/O channels are shared by all processing elements.
- The operation of the entire system is controlled by one operating system, which provides necessary interactions between processors and their program in different levels.

*The materials presented in this section are based on Enslow, P.H., *Multiprocessor Organization: A Survey*, ACM Computing Surveys 9, no. 1, 1977, 103–109 © 1977 ACM Inc. Adapted with permission.

In a multiprocessor system with n processors, each processor is often specialized for handling a particular task efficiently. Therefore, the time required to perform a computation is drastically reduced, and the throughput of the system is significantly increased.

The multiprocessing concept increases the overall reliability of the system. For example, a multiprocessor system can be designed so when one or more processors fail, the remaining processors in the system can still continue without disrupting the overall operation. Other advantages offered by a multiprocessor system are more availability and flexibility. The result of all these advantages is the significant improvement in the system performance.

The four principal elements of a multiprocessor system are:

- A set of processors
- A set of memories
- A set of I/O devices
- An interconnection system that links the preceding three elements.

A conceptual view of a typical multiprocessor system is shown in Figure 7.26.

One of the main considerations in designing an efficient multiprocessor system is to find an optimum interconnection scheme for various processors and memories in the system. Both the system operation and topology are governed by the chosen interconnection scheme. The following interconnection schemes are very popular in multiprocessors:

- Single bus
- Multiple bus
- Crossbar switching
- Multistage interconnection network
- Multiport memory

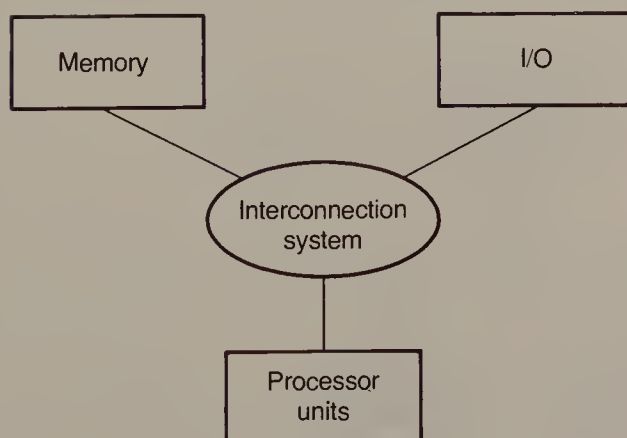


Figure 7.26 A Conceptual View of a Typical Multiprocessor Organization

7.6.1 Single Bus

The organization of a multiprocessor system using a single bus is shown in Figure 7.27.

In this approach, a common bus links all processors, memory, and other functional units in the system. The internal structure of the bus is simple because it entails no active components such as switches and amplifiers. All transfer operations, such as of the sending and receiving units, are controlled by the bus interface. In a single-bus organization, a sending unit, processor, or I/O unit must first determine the availability status of the bus. If the bus is available, the sending unit acquires the control of the bus. After this, the sending unit addresses the destination unit and determines its availability and capability to receive the transfer.

The sending unit tells the destination unit what to do with data being transferred. The function of a receiving unit is to recognize its address and respond to the control signals generated by the sender. Only one processor can gain the control of the bus. Therefore, the other processors of the system have to wait until the bus becomes available or perform some operations not requiring the bus. It is quite possible that a bus conflict may result when two or more processors try to gain control of the bus at the same time. Such a contention is often resolved by using a bus controller/arbiter.

Single-bus organization is typical with uniprocessor systems such as the PDP-11. For this system, the common bus is called UNIBUS and has 56 lines. These lines include necessary control lines and data paths to transfer words of only 16 bits. A processor may function as a master or a slave in this system. Any processor that has acquired control of the bus is referred to as the master. As a master, a processor can select any I/O device as a slave device and control its operation. However, when the processor relinquishes the control of the bus, it is no longer a master and may later on act as a slave for a new element that has gained control of the bus.

The advantages and disadvantages of a single-bus organization are summarized next. The *advantages* are:

- Overall hardware cost is minimal
- Structure of the interconnection scheme is simple because it contains only passive conductors.

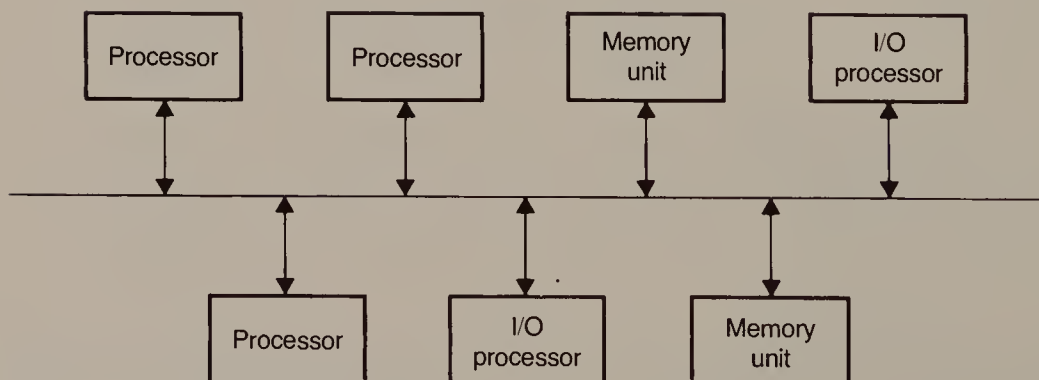


Figure 7.27 Single-bus Organization

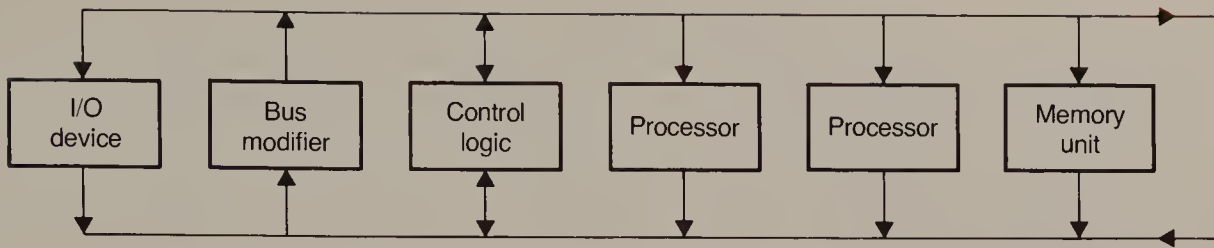


Figure 7.28 A Common Bus Organization with Two One-way Paths

- It is easy to modify the hardware configuration by adding more functional units or by removing some existing functional units.

The *disadvantages* are:

- Overall system capability is limited by the bus transfer rate.
- It has a poor fault-tolerant ability. Therefore, failure of the system bus causes total system failure.
- System throughput is reduced when the system is expanded with more functional units.

7.6.2 Multibus

The performance of a single-bus organization can be improved by providing two one-way paths, as shown in Figure 7.28.

In this organization, the addition of an extra bus neither increases the system complexity nor diminishes the reliability. However, a single transfer requires the use of both buses, and not much room is available for improvement in performance.

In a single bus organization, only one data transfer can be in progress at any given time. This restriction can be removed by providing multiple two-way buses, as shown in Figure 7.29.

From the figure, it is easy to see that this organization allows multiple simultaneous transfers. However, this considerably increases the complexity of the system. The

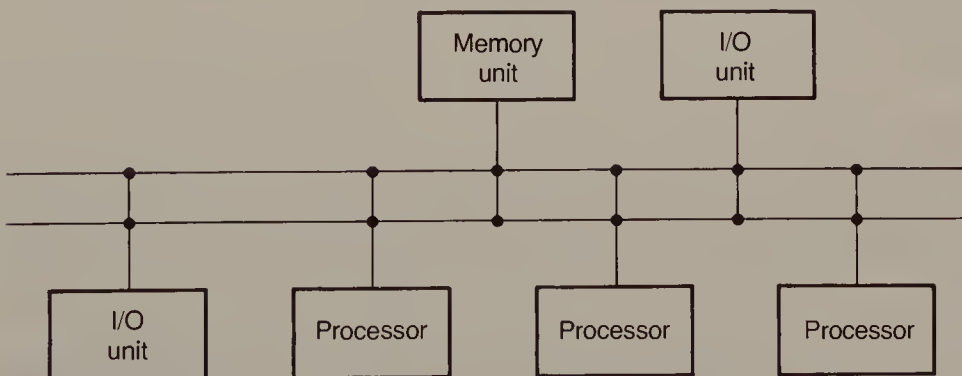


Figure 7.29 Common Bus Organization with Multiple Two-way Buses

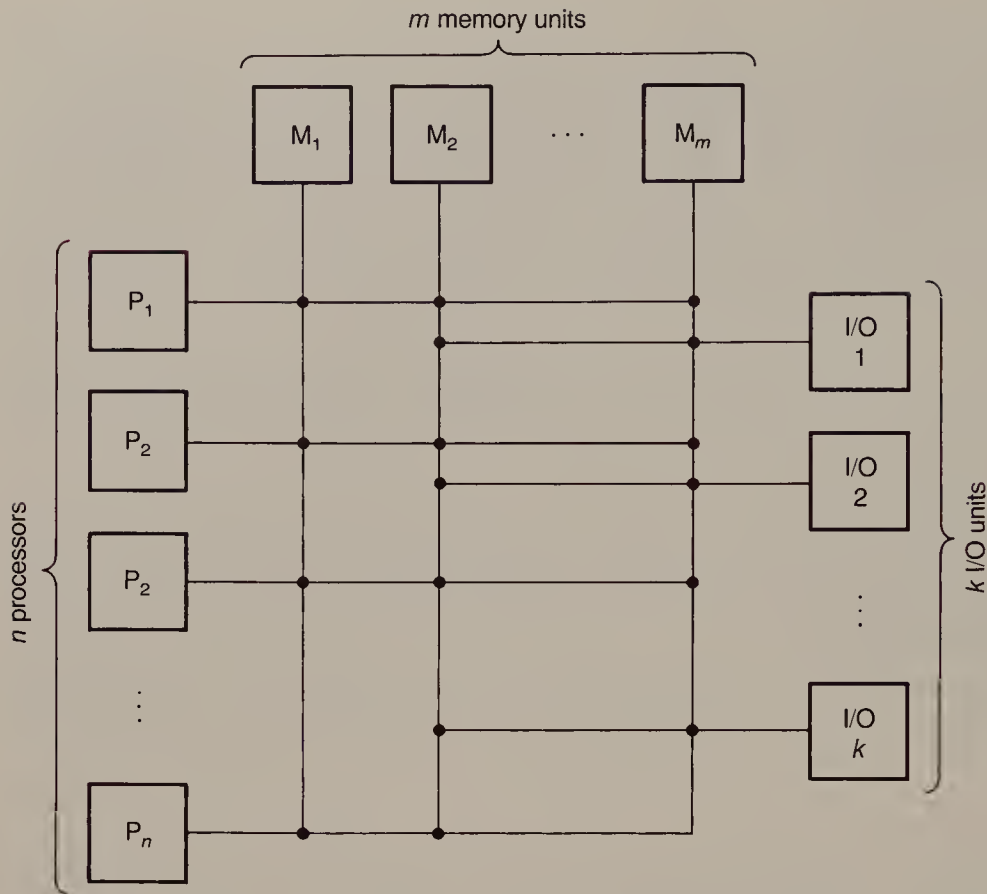


Figure 7.30 The Organization of a Crossbar Switch System

internal structure of the interconnection subsystem is no longer a set of passive conductors. Rather, appropriate logic, switching, and other control functions must be included at each point where the functional units are attached to the transfer bases. This organization is popular with many systems such as the IBM Stretch, UNIVAC LARC, and the CDC 6600.

If the number of buses is continuously increased in a time-shared system, a separate path will be available for each memory unit. The resulting organization is called a *crossbar* system, and its structure is shown in Figure 7.30.

7.6.3 Crossbar

The crossbar switches used in a multiprocessor system are complete with respect to the memory units. There is a separate bus associated with each memory unit, and the maximum number of transfers that can take place simultaneously is limited by the number of memory units and the bandwidth-speed product of the buses rather than the number of available paths.

A switch in a typical crossbar interconnection matrix has to perform the following operations:

- It must be capable of switching parallel transmissions.
- Whenever two or more processors try to access the same memory module, the switch must contain the necessary hardware to resolve such a conflict. Usually, all conflicting requests are handled on a predetermined priority basis. For example, an I/O unit may be given the top priority in all conflicts, and the priority of a processor unit may be higher than the priority of the memory unit.

A crossbar organization significantly reduces the cost of the functional unit because all data transfers and the bus arbitration tasks are handled by the switch. However, this increases the complexity of the interconnection scheme.

The following lists describe the advantages and disadvantages of crossbar organization. The *advantages* are:

- The functional units are the simplest and cheapest because the control and switching logic is in the switch.
- There is potential for the highest attainable total transfer rate.
- System expansion usually improves the overall performance.
- It is possible to expand the system without reprogramming the operating system.
- It is easy to partition the system to remove malfunctioning units or to establish separate systems.

The *disadvantages* are:

- This is the most-complex interconnection system.
- Since a basic switching matrix is required to assemble any functional units into a working configuration, this organization is cost-effective only for small multiprocessor systems.
- Theoretically, the expansion of the system is limited by the size of the switching matrix.
- The reliability of the switch decides the reliability of the system. However, the overall fault-tolerating ability of the system can be improved by segmentation and/or redundancy within the switch.

The flexibility of a crossbar organization can be increased by providing another crossbar matrix on the device side, as illustrated in Figure 7.31.

This organization provides extra flexibility when accessing I/O devices. The hardware required for this implementation is different and not as complex because controllers and devices are normally designed to recognize their own unique addresses. The effect is the same as if a primary bus were associated with each I/O channel and a crossbus to each controller device.

A variation of this concept is found in many systems such as the Burroughs multi-interpretor, RCS 215, and Carnegie Mellon University's C.mmp computers. H. Enslow's paper [6] thoroughly investigates all these systems.

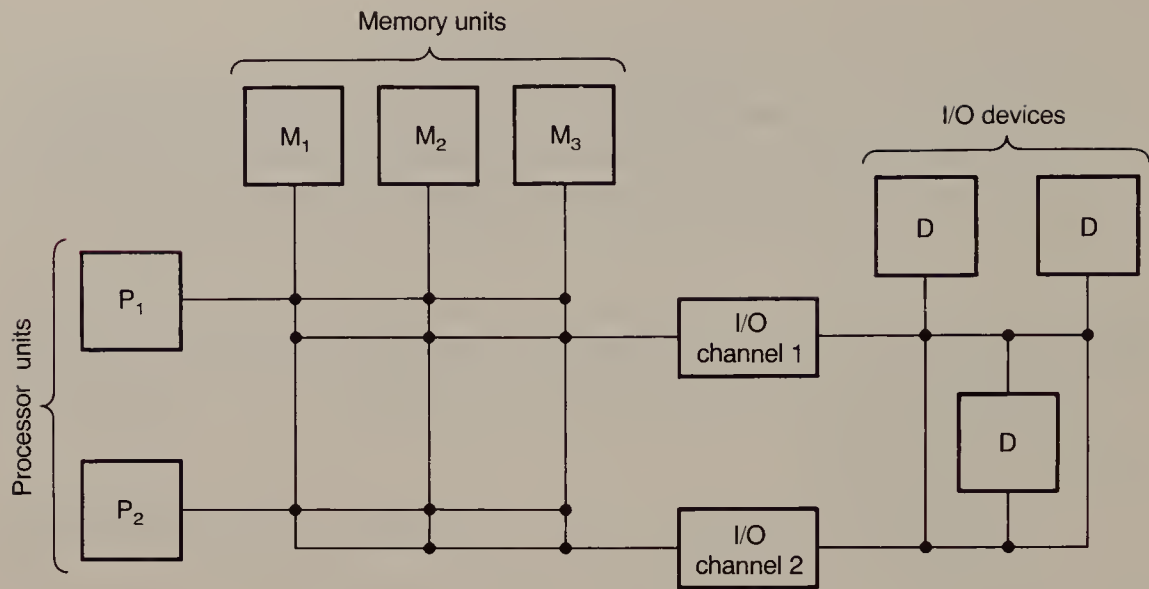


Figure 7.31 The Organization of a Crossbar System with an I/O Crossbar Switch Matrix

7.6.4 Multiport Memory

Multiport memory organization is when the control, switching, and priority arbitration logic is distributed throughout the crossbar switch matrix and concentrated at the interface to the memory units. Its structure is illustrated in Figure 7.25. This organization is well suited to both uni- and multiprocessor systems. In Figure 7.32, all memory conflicts are resolved by assigning a priority to each functional unit. In some cases, a fixed priority value is assigned to each memory port so the system can be configured at the

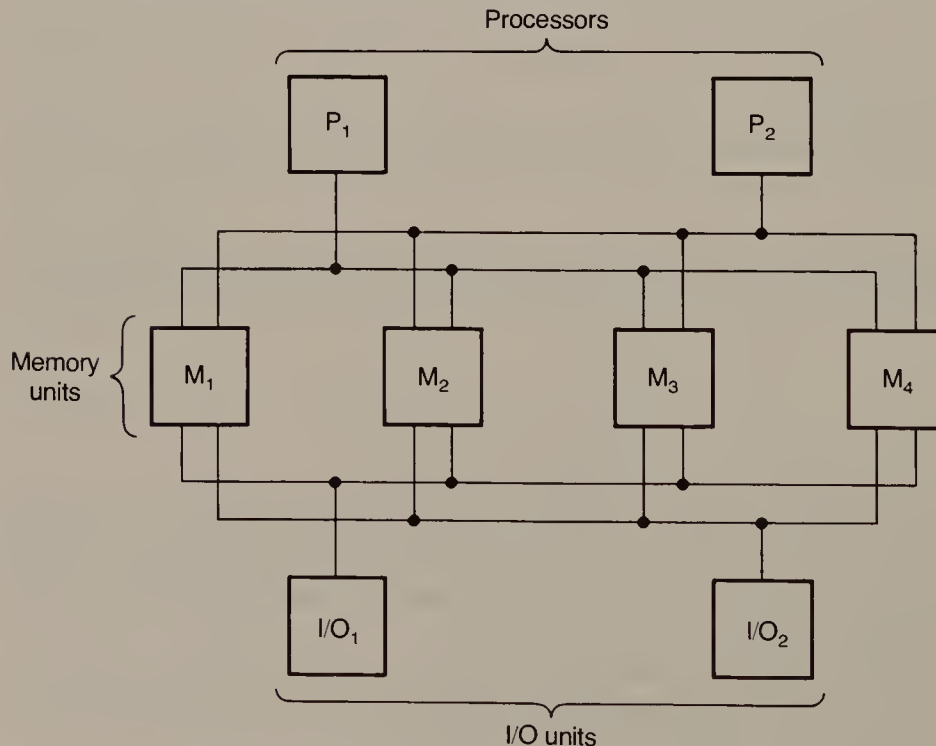


Figure 7.32 The Basic Structure of a Multiport Memory Organization

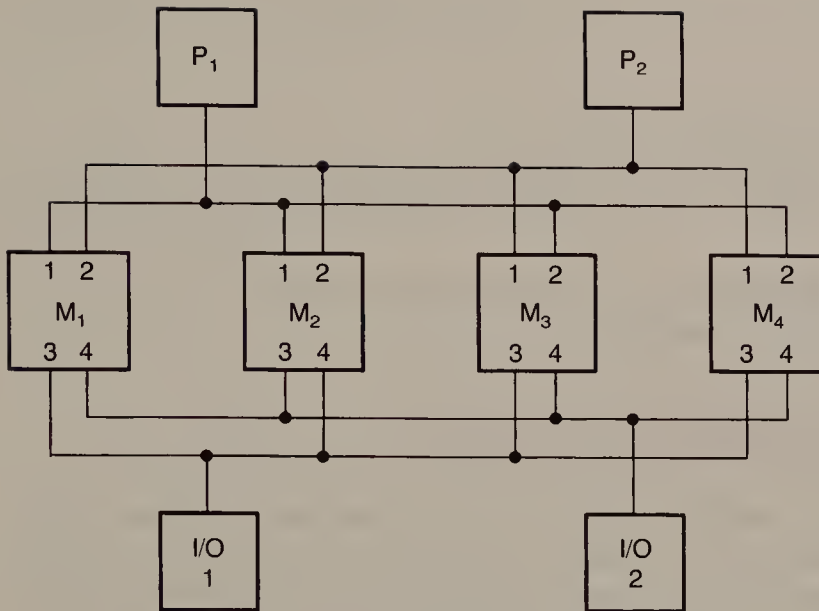


Figure 7.33 Multiport Memory Organization with Fixed Port Priorities

installation, allowing desired priorities among functional units to be provided. For example, consider the multiport organization shown in Figure 7.33.

In this structure, each memory module has four ports. The priorities for each port of a memory unit are all predefined, as illustrated in the figure.

Some important salient features of a multiport memory organization are as follows:

- It requires the most-expensive memory unit because the major portion of the control and switching circuitry is included in the memory unit.

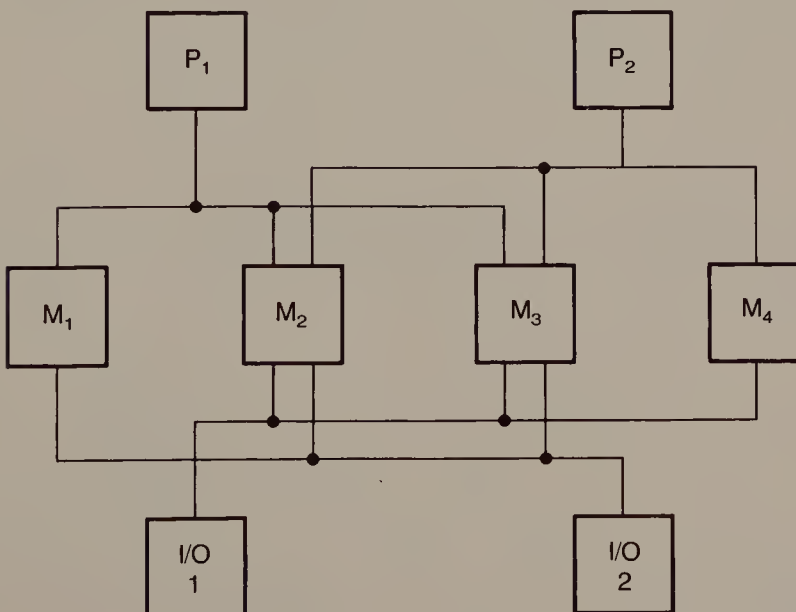


Figure 7.34 Multiport Memory Organization with Private Memory Units

- Several simple functional units can be combined to obtain a low-cost uniprocessor system.
- It is possible to achieve high transfer rates.

The integrity of computation can be increased by using a multiport memory organization with private memories. The architecture of this arrangement is shown in Figure 7.34.

Multiport memory organization is used in many popular systems, including UNIVAC 1108 and IBM 360/167 computers.

7.7 DATA-FLOW COMPUTER SYSTEMS

In traditional Van Neumann computer systems, a set of instructions is executed sequentially. For example, instruction 25 is always executed after the execution of instruction 24 (assuming no branches). Even multiprocessor systems, which can execute tasks in parallel, have a sequential control unit. In any computer, an instruction is executed only when directed to do so by the central control unit. This central control unit may not be located at one physical point and can be distributed over the entire system.

A data-flow computer system does not have sequential instruction control. Instructions on a data-flow computer are executed when all the operands needed for execution of the instructions are available.

Consider the computation $E = (A + B) * (C - D)$, where A , B , C , D , and E are scalars. The sum $A + B$ can be computed in parallel with the difference $C - D$. The product of $(A + B)$ and $(C - D)$ can be executed only after the previous two operations are completed.

Figure 7.35 shows the sequence as a function of time. At $t = 0$, tokens containing A , B , C , and D are present at input lines.

Tokens are abstract pieces of information containing data and the information indicating that the data is available to process. At $t = 1$, instructions $(A + B)$ and

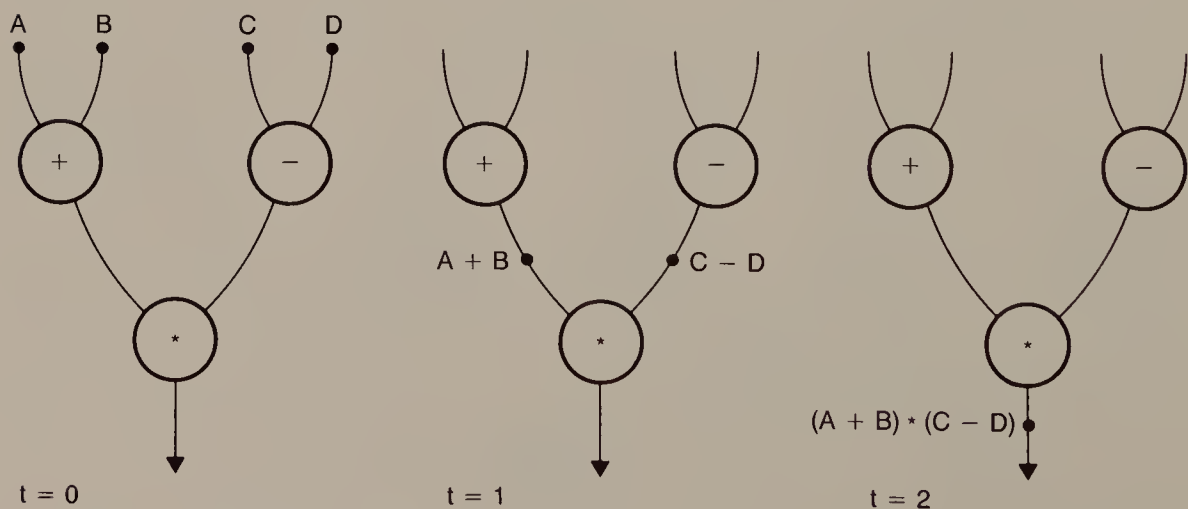


Figure 7.35 Data-flow Sequence

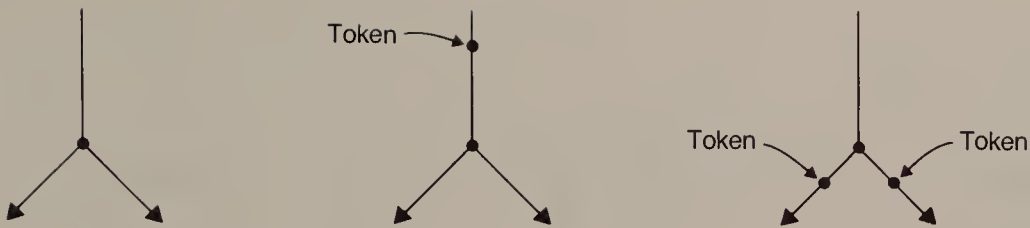


Figure 7.36 Data Link

$(C - D)$ are executed, and tokens containing results are present at the inputs of the multiplier. As soon as the operands are available to carry out the multiplication, the product $(A + B) * (C - D)$ is calculated, and the results are passed on in the token.

Most programs to be executed on a data-flow machine can be characterized using data-flow graphs. Operators are represented on the graph as nodes. Arcs represent the data or control flow between two nodes. Tokens propagate along the arcs of the graphs. Once a token is present at the inputs of a node, certain rules are used to decide what kind of output token, if any, is produced by the node. There are essentially five different kinds of nodes that are encountered in data-flow graphs. Other nodes can be constructed using these basic nodes:

1. Links
2. Operators
3. Deciders
4. T-gates
5. Merge gates

Other nodes such as identity operators, logical and control links, and inverters are all subsets of these fundamental nodes. In the following, these nodes are described in detail.

1. *Data links*: Data links are represented as shown in Figure 7.36. When a token appears at the input, it is replicated into N tokens. Each token then flows down each link arc. Since data link performs a replication and pass, it is considered a replicate link.
2. *Control link*: Control link (as shown in Figure 7.37) performs a function similar to that of the data link. When a control token appears at the input, it is replicated into N tokens, which flow down N control arcs.

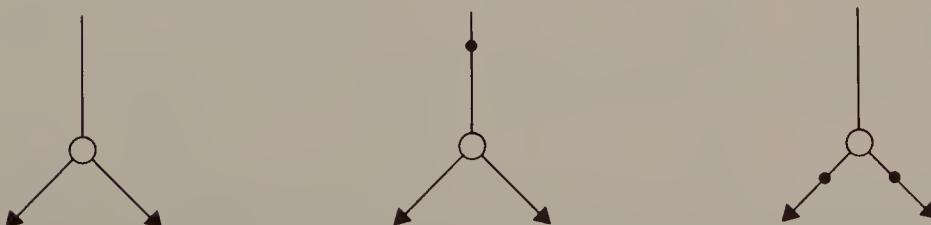


Figure 7.37 Control Link

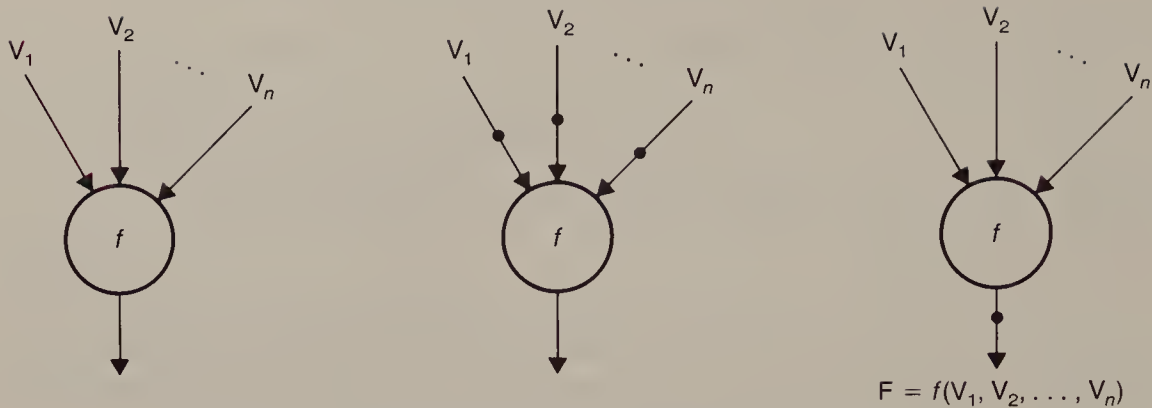


Figure 7.38 Operators (K. Hwang and F.A. Briggs, *Computer Architecture and Parallel Processing* © 1984, p. 742, McGraw-Hill. Reprinted by permission.)

3. *Operators*: The operators, as shown in Figure 7.38, have N inputs and one output. When all tokens are present at the input, they are absorbed into the node and operated upon, and the result is presented on the output arc. The input token may contain Boolean, integer, real, strings, or any other abstract data types.
4. *Deciders*: As shown in Figure 7.39, deciders perform operations similar to operators. When all N tokens are present at the input, they are absorbed by the node, and the output token is placed on the output arc. The operation P performed on the input is the predicate, which results in either true or false value.
5. *T-gates*: T-gates, as shown in Figure 7.40, implement a decision function. If the control token present at the control input is true, the input token is absorbed into the node and passed to the output arc without any modification.
If the control token is false, data token is absorbed into the node, but no output is produced.
6. *F-Gates*: The F-gate is similar to a T-gate, except the polarity is reversed. In this case, if the control token is false, the input data is passed on to the output arc. If the control token is true, the data token is absorbed and no output is produced. This can be seen in Figure 7.41.

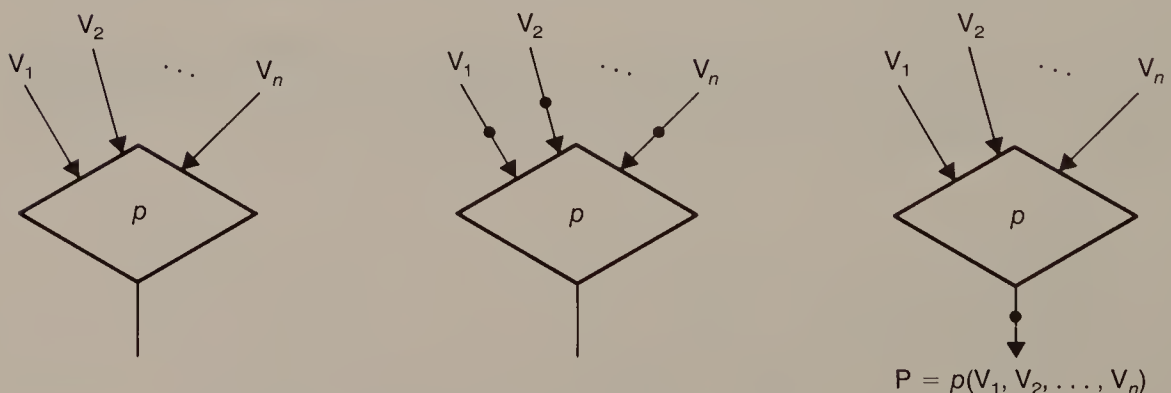


Figure 7.39 Deciders (K. Hwang and F.A. Briggs, *Computer Architecture and Parallel Processing* © 1984, p. 742, McGraw-Hill. Reprinted by permission.)

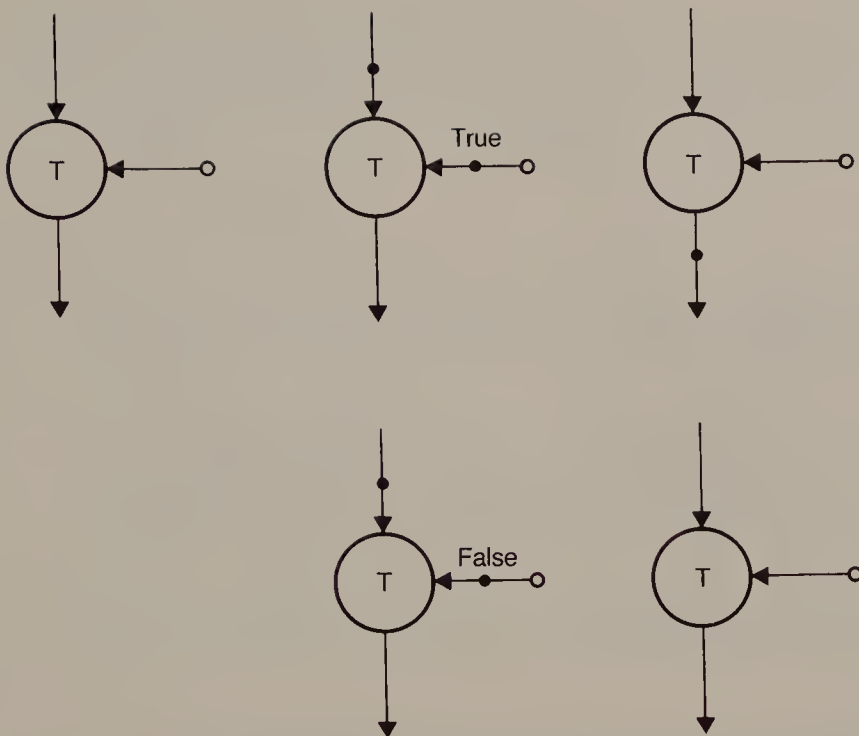


Figure 7.40 T-gates

7. *Merge Gates*: This gate can be seen in Figure 7.42. It allows merging of two data tokens onto another arc depending upon the control token. If the control token is true, the data token present at the T-input is passed to the output arc. If the control token is false, data present at the F-point passed to the output arc.

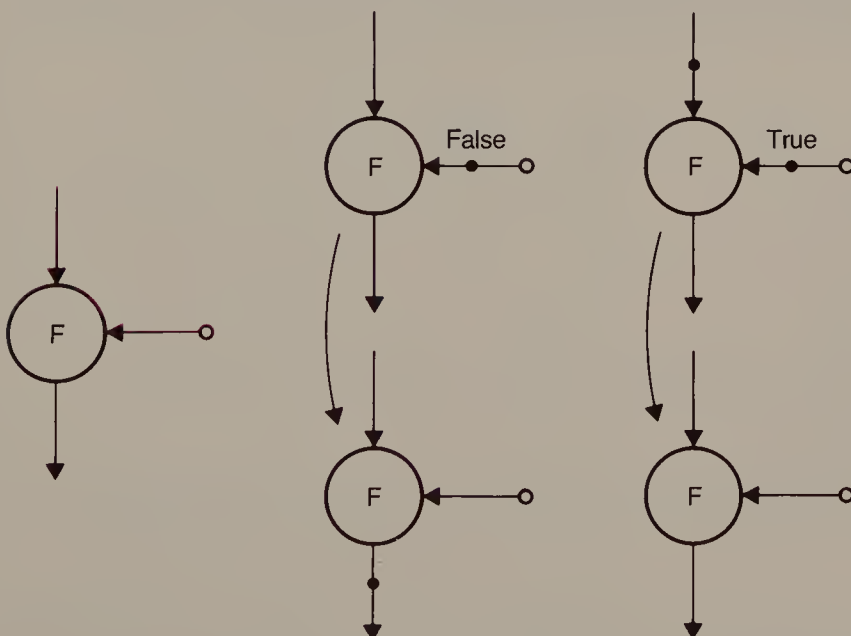


Figure 7.41 F-Gates

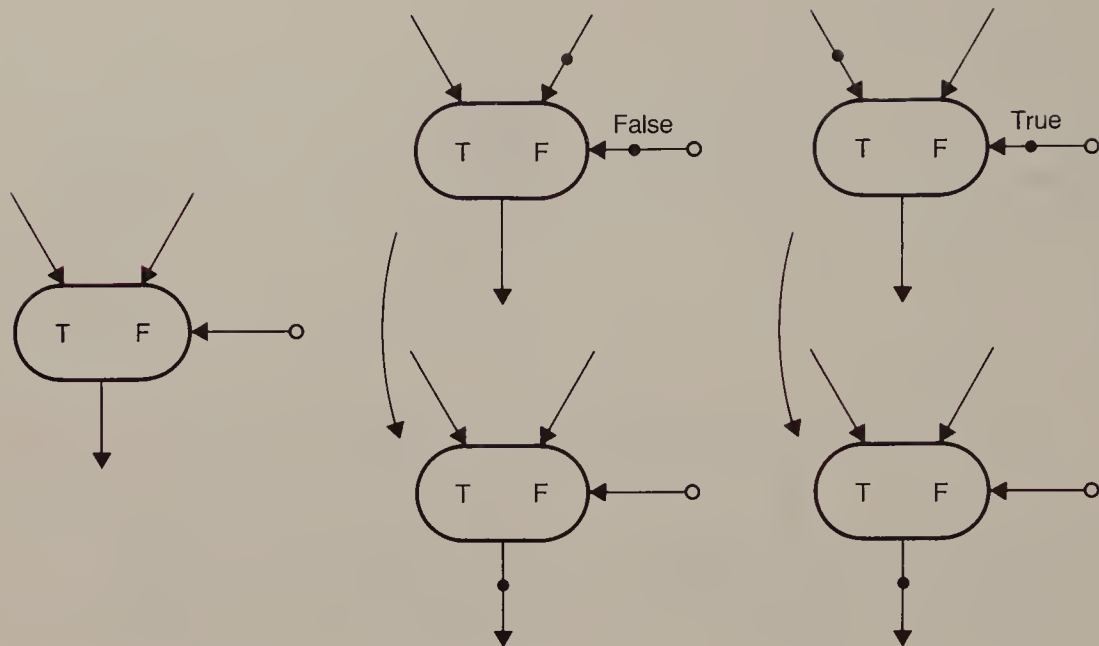


Figure 7.42 Merge Gate

When all these elements are combined, the resulting graph implements a specific task. The graph shown in Figure 7.43 is equivalent to the Pascal program.

```

if a = b      then d: = e + 1
               else d: = e - 1
  
```

The main reason for using a data-flow computer is to exploit the inherent parallelism in a computation. For example, addition of two matrices can be carried out by a data-flow machine in a highly concurrent fashion. Experimental data flow computers are already proposed at MIT, University of California, Irvine, and University of Manches-

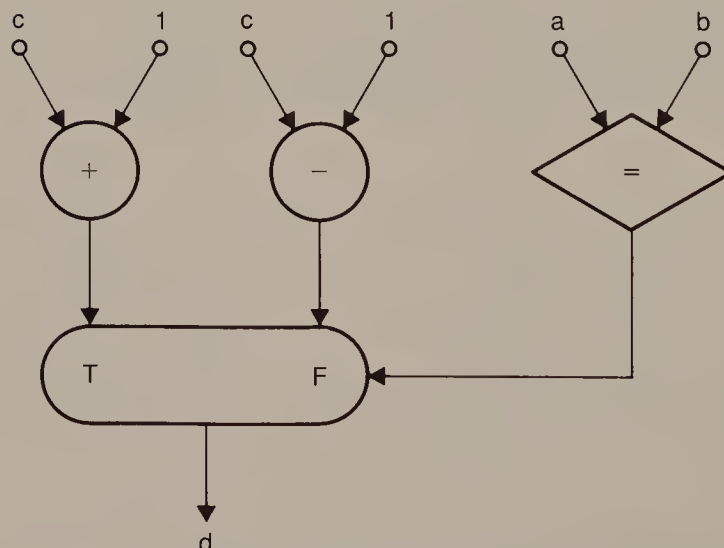


Figure 7.43 A Typical Data-flow Graph

ter, England. However, data-flow programs tend to waste memory space and cause extensive pipeline overhead per instruction. Some studies indicate that these shortcomings may make building large-scale data-flow computers improbable.

QUESTIONS AND PROBLEMS

7.1 Assume that a CPU is capable of performing 1 million floating-point operations (MFLOPS) per second. Suppose that a typical weather prediction computation requires 10^{15} floating-point operations, how long will this CPU take to solve this problem?

7.2 The factor that limits the speed of a computer is the velocity of light. What is the maximum distance between a processor and memory if transaction time is 50 pico seconds?

7.3 Select any 2 hardware strategies used to achieve parallelism in a uniprocessor system. Compare the advantages and disadvantages of the chosen strategies.

7.4 Let

T_i = time taken to solve a problem on a system with i processors

P_i = number of unit time operations performed by a system with i processors

Now, let us define the following performance measures:

S_i = Speedup realized by an i -processor system = T_1/T_i

(Note: T_1 is the time taken by a uniprocessor system to solve a problem)

E_i = Efficiency of a system with i processors = $S_i/i = T_1/(i * T_i)$

R_i = Operational redundancy = P_i/P_1

(Note: $T_1 = P_1$)

U_i = Processor Utilization = $P_i/(i * T_i)$

a) Prove the following results

i) $1/k \leq E_k \leq 1$

ii) $U_k = R_k * E_k$

iii) $E_k \leq U_k$

b) Calculate S_i , E_i , and R_i for the following computation

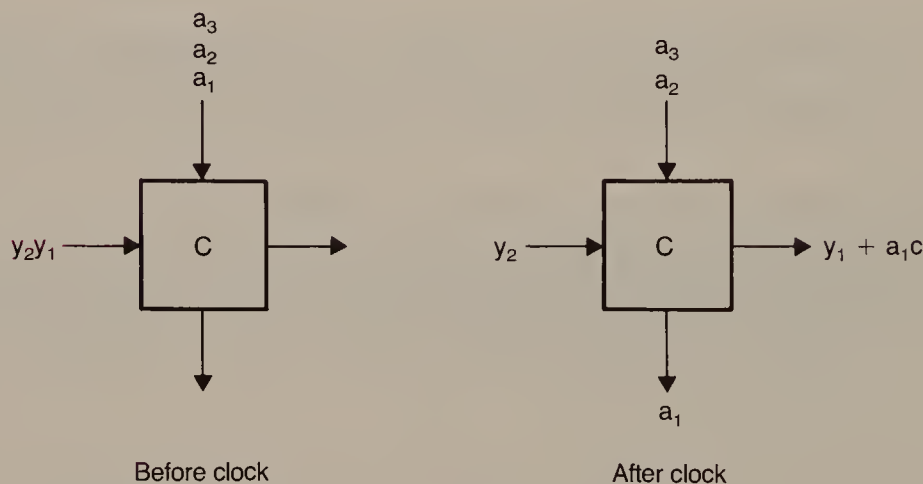
$$a * (b + c * d * e)$$

7.5 Let β denote the fraction of operations that must be performed in serial fashion (for example I/O operation). Then

$$S_k = T_1/T_k \leq 1/(\beta + (1 - \beta)/k)$$

This result is known as the Amdahl's Law. Determine the maximum value β that would allow a 5 processor system to operate with 75% efficiency.

- 7.6** The Cray — I computer has one CPU, and 12 functional units. Up to a maximum of 8 functional units can be cascaded to form a chain. Each functional unit is pipelined and the number of pipeline segments vary from 1 to 14. Each functional unit is capable of manipulating 64-bit data. Is it possible to describe this machine using Flynn's approach? Explain.
- 7.7** Consider a processor array with 4 floating-point processors (FPP). Suppose that each FPP takes 4 time units to produce one result, how long it would take to carry out 100 floating point operations? Is there any performance improvement if the same 100 floating-point operations are carried out using a 4-segment pipelined processor in which each segment takes 1 time unit to produce the result (Ignore latch delay)?
- 7.8** Explain the significance of masking in array processors.
- 7.9** What is the key principle behind systolic array processors?
- *7.10** Consider the systolic array element shown next:



How will the following matrix multiplication be performed using the set of these elements?

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

Show data organization and a complete trace.

- 7.11** Provide a step-by-step block-diagram approach to a 3×3 matrix multiplication using one wavefront architecture approach.

*Dougherty/Giardina *Matrix Structured Image Processing*, © 1987, p. 230. Reprinted by permission of Prentice-Hall, Inc., Englewood Cliffs, New Jersey.

7.12 Consider the floating-point pipeline discussed in this chapter. Assume:

$$T_1 = 40 \text{ ns} \quad T_2 = 100 \text{ ns}$$

$$T_3 = 180 \text{ ns} \quad T_4 = 60 \text{ ns}$$

$$T_1 = 20 \text{ ns}$$

- a)** Determine the pipeline clock rate.
- b)** Find the time taken to add 1000 pairs of floating-point numbers using this pipeline.
- c)** What is the efficiency of the pipeline when 2000 pairs of floating-point numbers are added?

7.13 Design a pipeline multiplier using carry/save adders (CSA) and carry-look-ahead adders to multiply a stream of input numbers X_0, X_1, X_2 , by a fixed number Y . Assume all X s and Y s are 6-bit numbers. The output should be a stream of 12-bit products YX_0, YX_1, YX_2, YX_3 . Draw a neat schematic diagram of your design.

7.14 Consider the execution of 1000 instructions using a 6-segment pipeline.

- a)** What is the average number of instructions executed per instruction cycle when $C = 0.2$?
- b)** What must be the value of C so execution of at least 4 instructions per instruction cycle is always allowed.

7.15 Describe the methods used to handle branches in a pipeline instruction execution unit.

7.16 Modify each of the following programs so the data flow in the 2-segment pipeline (Figure 7.22) is properly regularized:

a) MEMORY ADDRESS	INSTRUCTION
2000	LDA X
2001	DCR Y
2002	JMP 2040
2003	SUB Z
.	.
.	.
.	.
2040	STA W

b) MEMORY ADDRESS INSTRUCTION

2000	LDA X
2001	DCR Y
2002	JNZ 2040
2003	SUB Z
2004	.
.	.
.	.
.	STA W
2040	

7.17 Explain the significance of interleaved memory organization in pipelined computers.

7.18 What are the essential components of a multiprocessing system.

7.19 Compare the common bus organization with the crossbar organization.

7.20 What are the advantages of a multiport memory organization that has private memory units?

7.21 What are the basic differences between a data flow computer and each of the following?

a) Pipelined computer

b) Systolic array processor

c) Wavefront array processor

7.22 Draw a data-flow graph corresponding to each one of the following program segments:

a) Case i of

b) 0 : $A := X + 1$;

1 : $A := Y + 2$;

2 : $A := Z + 3$;

3 : $A := P - 4$;

Others : $A := R + 5$;

Sum := 0;

$i := 10$;

While $i > 0$ do

begin

 sum := sum + i ;

$i := i - 1$

end

REFERENCES

1. Austin, K. "Computer Architecture." Master's thesis, School of Engineering, Cal-Poly Pomona, 1986.
2. Baer, J. L. *Computer Systems Architecture*. Rockville, Md.: Computer Science Press, 1980.
3. Baer, J. L. "Multiprocessing Systems." *IEEE Transaction on Computers* C-25 (December 1976).
4. Dennis, J. B. "Data Flow Supercomputers." *IEEE Transaction on Computers* (November 1980): 48–56.
5. Dougherty, E. R., and C. R. Giardina. *Matrix Structured Image Processing*. Englewood Cliffs, N.J.: Prentice-Hall, 1987.
6. Enslow, P. H. "Multiprocessor Organization: A Survey." *ACM Computing Surveys* 9, no. 1 (1977) 103–129.
7. Hayes, J. P. *Computer Architecture and Organization*. New York: McGraw-Hill, 1978.
8. Hwang, K., and F. A. Briggs. *Computer Architecture and Parallel Processing*. New York: McGraw-Hill, 1984.
9. Kogge, P. M., "The Architecture of Pipelined Computers", Hemisphere, 1982.
10. Kung, S. Y., Whitehouse, H. J., and T. Kaitath, eds. *VLSI and Modern Signal Processing*. Englewood Cliffs, N.J.: Prentice-Hall, 1985.
11. Mano, M. M. *Computer System Architecture*. Englewood Cliffs, N.J.: Prentice-Hall, 1983.
12. Moldovan, D. "Parallel Computers." EE657 Class Notes, EE Systems Department, USC, Los Angeles, California, 1984.
13. Patterson, D., "Reduced Instruction Set Computers." *CACM* (January 1985).
14. Ramamoorthy, C. V., and H. F. Li. "Pipeline Architecture." *ACM Computing Surveys* 9 (March 1977) 61–102.
15. Siegel, H. J. "A Model of SIMD Machines and Comparison of Various Interconnection Networks." *IEEE Transaction on Computers* C-28 (December 1979) 907–917.
16. Stallings, W. *Computer Organization and Architecture*. New York: Macmillan, 1987.
17. Stone, H. S. "Parallel Processing with Perfect Shuffle." *IEEE Transaction on Computers* C-20 (February 1971) 153–161.
18. Stone, H. S., ed. *Computer Architecture*. Chicago: Science Research Associates, 1980.
19. Waser, S., and M. J. Flynn. *Introduction to Computer Arithmetic for Digital System Designers*. New York: Holt, Rinehart & Winston, 1982.

APPENDIX A

Introduction to Logic Design

The purpose of this section is to provide a tutorial introduction to logic design. Topics covered include Boolean algebra, combinational circuit design, ROMs, PLAs, and sequential circuit design.

A.1 BOOLEAN ALGEBRA

Boolean algebra is a mathematical structure that is extensively used in many areas such as digital system design, automata theory, and database systems. Formally, a boolean algebra (BA) may be characterized by a triple:

BA: $\langle V, O, P \rangle$
where

- V is a set of possible values assumed by a Boolean variable, say x
- O is a set of operators. (Note that the set V is closed with respect to these operators. That is, if x and y are the Boolean variables and if θ is an operator, then the following result holds:

$$x, y \in V \rightarrow (x \theta y) \in V$$

- P is a set of properties that are satisfied by the boolean operators.

x	x'
0	1
1	0

Figure A.1 Truth Table for the NOT Operation

It can be shown that an algebraic structure is qualified to be called a BA only when the number of elements in the set V can be expressed as 2^k where k is any positive integer greater than zero. In particular, when BA is applied to the design of digital systems, the set V will contain only two elements, 0 and 1. That is, $V = \{0, 1\}$. Under these circumstances, this special case is called as either two-valued Boolean algebra or switching algebra. In this discussion, the word BA is used to mean only the two-valued Boolean algebra. It should be pointed out that the elements 0 and 1 do not represent integer numbers. Rather they represent some physical conditions of a system. For example, in integrated circuits the element 0 is represented by 0 volt, whereas a 1 is represented by 5 volts. On the other hand, with respect to relay-switching networks, a closed switch represents a 1, whereas an opened switch represents a 0.

There are three basic operators in BA:

- NOT
- AND
- OR

The NOT operator is a unary operator. That is, it can operate on a single boolean variable. Formally, NOT of a Boolean variable x , written as x' , is 1 if and only if x is 0. Similarly, x' is 0 if and only if x is 1. This definition may also be specified in the form of a truth table, as described in Figure A.1.

The NOT operation is also called *complement operation*. The schematic representation of an electronic circuit that implements the NOT operation is shown in Figure A.2.

Any electronic circuit that implements a logic operation is often called a *gate*. In particular, a NOT gate is also referred to as an inverter because it inverts the voltage levels. For example, a 0-volt input causes a 5-volt output to be generated at the output, and vice versa.

The AND operation is a binary operation, as it needs two operands. Formally, x AND y , written $x \cdot y$, is 1 if only if both x and y are identically equal to 1. A truth description of this operation is shown in Figure A.3.

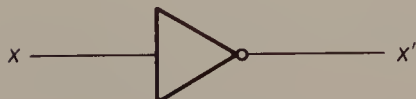


Figure A.2 Schematic Representation of a NOT gate

x	y	$Z = x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1

Figure A.3 Truth Table Description of the “AND” Operation

The schematic representation of an AND gate is diagrammed in Figure A.4.

The third basic operator of the Boolean algebra is the OR operator. This is again a binary operation, and its truth table description is provided in Figure A.5.

The schematic diagram of an OR gate is shown in Figure A.6.

Some important properties of these three operators are summarized as described next:

- AND and OR are commutative. That is,

$$xy = yx$$

$$x + y = y + x$$

- AND and OR are associative. That is,

$$(xy)z = x(yz)$$

- AND distributes over OR, and vice versa. That is,

$$x(y + z) = xy + xz$$

$$x + yz = (x + y)(x + z)$$

- AND and OR are idempotent. That is,

$$xxx \cdots x = x$$

$$x + x + x \cdots + x = x$$

- The law of double negation, or involution principle holds:

$$(x')' = x$$

- When a Boolean variable is ANDed with its complement, the result is 0. However when a Boolean variable is ORed with its complement, the result is 1.



Figure A.4 Schematic Representation of an AND Gate

x	y	$z = x + y$
0	0	0
0	1	1
1	0	1
1	1	1

Figure A.5 Truth Table Description of the OR Operation

More formally

$$x \cdot x' = 0$$

$$x + x' = 1$$

- When a Boolean variable is ANDed with 0, the result is also 0. However when it is ORed with 1, it remains invariant. That is,

$$x \cdot 0 = 0$$

$$x + 0 = x$$

- Similarly, ANDing a Boolean variable with 1 will produce a null effect. On the other hand, whenever a Boolean variable is ORed with 1, the result is always a 1. More formally,

$$x \cdot 1 = x$$

$$x + 1 = 1$$

- The complement of a Boolean product is the sum of the individual complements. Similarly, the complement of a Boolean sum is the product of the individual complements. That is,

$$(xyz)' = x' + y' + z'$$

$$(x + y + z) = x'y'z'$$

The above result is customarily referred to as *DeMorgan's laws*.

Another important characteristic of the BA is that it obeys the principle of duality. Now let us consider this aspect in brief. For any Boolean expression f , its dual f^D can be constructed by replacing every $+$, $\cdot$, and 1 by $\cdot$, $+$, and 0, respectively. For example, if



Figure A.6 Schematic Representation of an OR Gate

x	y	$x \text{ NAND } y$
0	0	1
0	1	1
1	0	1
1	1	0

Figure A.7 Truth Table for the NAND Operation

$$\begin{array}{rcccl}
 & f = (x + y) \cdot (x + z) & & & \\
 \text{then} & \downarrow & \downarrow & \downarrow & \\
 f^D = & xy & + & xz &
 \end{array}$$

The two other important Boolean operators that deserve a discussion are the NAND and NOR operators. The truth table that describes the NAND operation is shown in Figure A.7. The truth table shows that the result produced by the NAND operation is equivalent to $(x \cdot y)'$; that is, the result of x ANDed with y is complemented. The schematic representation of a NAND gate is shown in Figure A.8.

In this figure, the small circle at the output of the NAND gate is called the inversion bubble. This inversion bubble indicates the output of the AND gate and is actually complemented.

The truth table for the NOR operation is shown in Figure A.9. From the truth table, it is easy to see that the result achieved by the NOR operation is equivalent to the Boolean expression $(x + y)'$; that is, we complement the result produced by the operation x OR y . Figure A.10 illustrates the schematic representation of a NOR gate.

One of the important properties of the NAND operation is that it is functionally complete. That is, the basic operations NOT, AND, and OR can be expressed in terms of NAND. For this reason the NAND gate is said to be a universal gate. This is illustrated in the following example.

Another important logic operation is the exclusive-OR operation. This operation is a binary operation and is written as $x \oplus y$. The truth table corresponding to this operation is shown in Figure A.11.

From the truth table it is clear that $x \oplus y$ is 1 only when $x = 0$ and $y = 1$ or $x = 1$ and $y = 0$. More formally,

$$x \oplus y = x'y + xy'$$

The schematic representation of the exclusive OR gate is shown in Figure A.12.

The exclusive OR gate can be implemented using AND and OR gates; this is shown in Figure A.13.

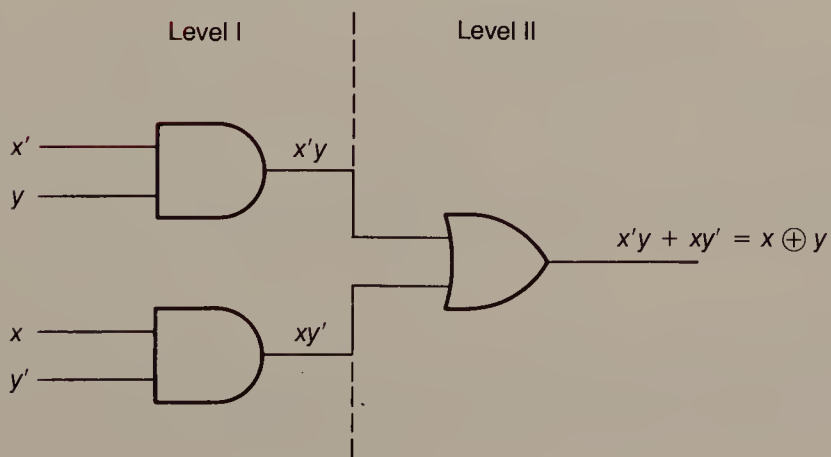


Figure A.8 The Schematic Representation of a NAND Gate

x	y	$x \text{ NOR } y$
0	0	1
0	1	0
1	0	0
1	1	0

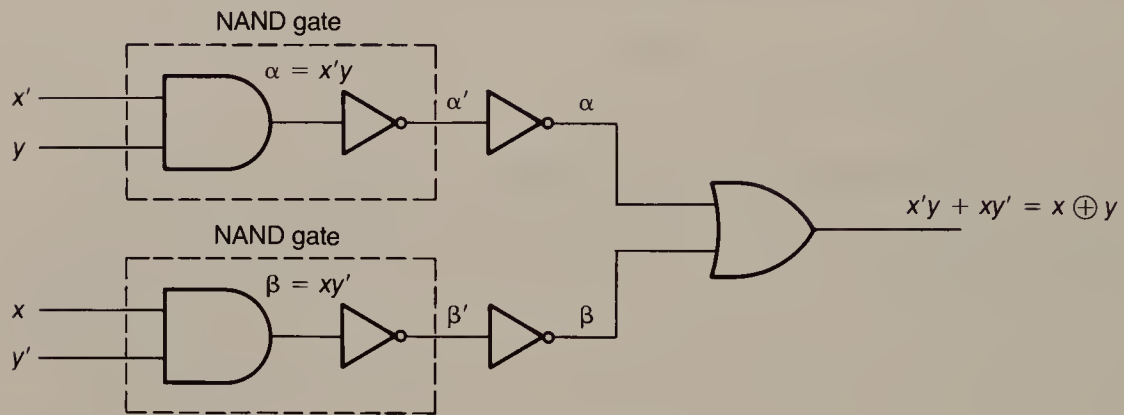
Figure A.9 Truth Table Description of the NOR Operation**Figure A.10** Schematic Representation of a NOR Gate

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

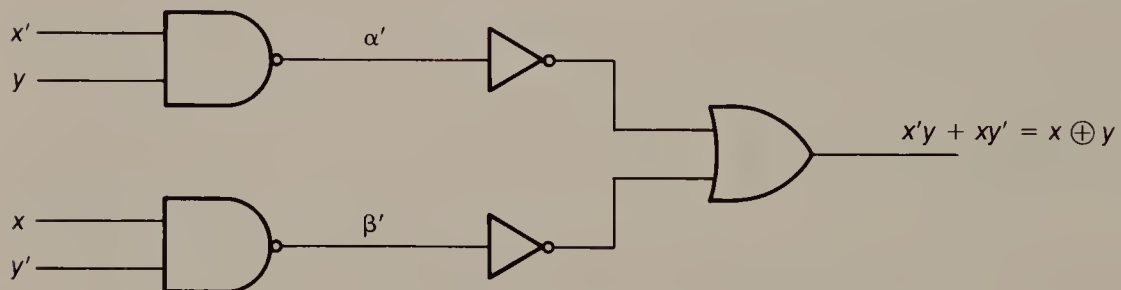
Figure A.11 Truth-table Specification of the Exclusive-OR Operation**Figure A.12** Schematic Representation of an Exclusive-OR Gate**Figure A.13** AND/OR Implementation of the Exclusive-OR Gate

Similarly, the exclusive-OR gate can also be implemented using only NAND gates. First, the circuit of Figure A.14 is modified by adding four inverters, as shown in Figure A.14(a). Note that the inclusion of these inverters does not change the output. However, the AND gate and the inverter combination can be replaced with a NAND gate. The result of this modification is shown in Figure A.14(b). Figure A.14(a) shows that the final answer is $\alpha + \beta$. But by DeMorgan's laws,

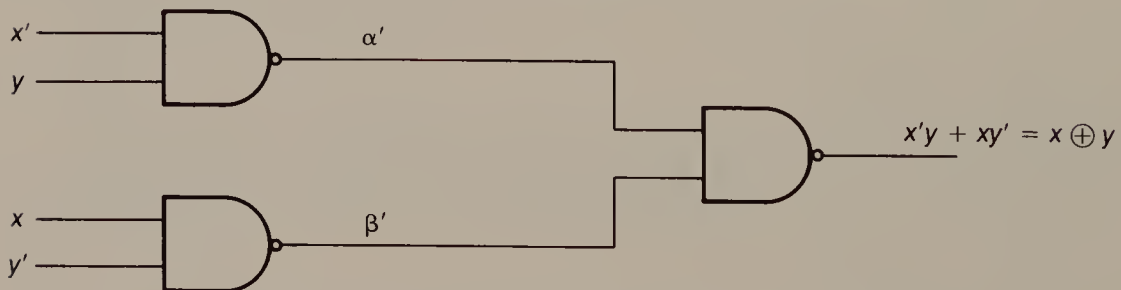
$$(\alpha' \cdot \beta')' = \alpha + \beta$$



a. Figure A.14 Redrawn



b. Figure A.14(a) redrawn



c. Implementation of the Exclusive OR Gate Using Only Nand Gates

Figure A.14

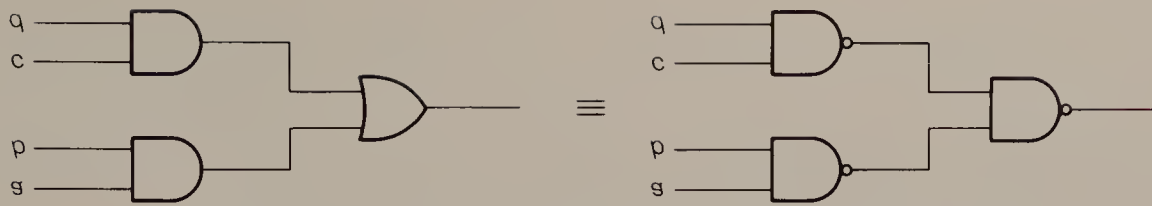


Figure A.15 Relationship Between a Two-level AND/OR and a Two-level NAND/NAND Implementations

Therefore, the final implementation of the exclusive-OR gate is obtained by applying the outputs of the NAND gates of Figure A.14(b) as inputs to another NAND gate, as shown in Figure A.14(c).

The circuit of Figure A.13 is called a two-level circuit. In particular, this implementation is referred to as a two-level AND/OR implementation because in the first level we have a set of AND gates in order to generate the required product terms. In the second level, an OR gate is needed to sum the products generated by the AND gate. In general, it is true that any Boolean expression that is in the sum of products form can be immediately implemented using a two-level AND/OR structure such as the one shown in Figure A.14. Also, it can be shown that in any two-level AND/OR implementation, if we replace each gate by a NAND gate, the output will remain invariant. This idea is illustrated in the Figure A.15. NAND/NAND implementation is preferable because it requires only one type of gate.

In a similar manner it can be said that any Boolean expression that is expressed in product of sum form can be immediately implemented by using a two-level OR/AND structure. Also, it can be proven that in a two-level OR/AND implementation, if each gate is replaced by a NOR gate, the output still remains invariant. These ideas are illustrated in Figure A.16(a) and (b).

The exclusive-OR operator has several useful properties, some of them are summarized next:

- Exclusive-OR is a commutative operation:

$$x \oplus y = y \oplus x$$

- Exclusive-OR is an associative operation:

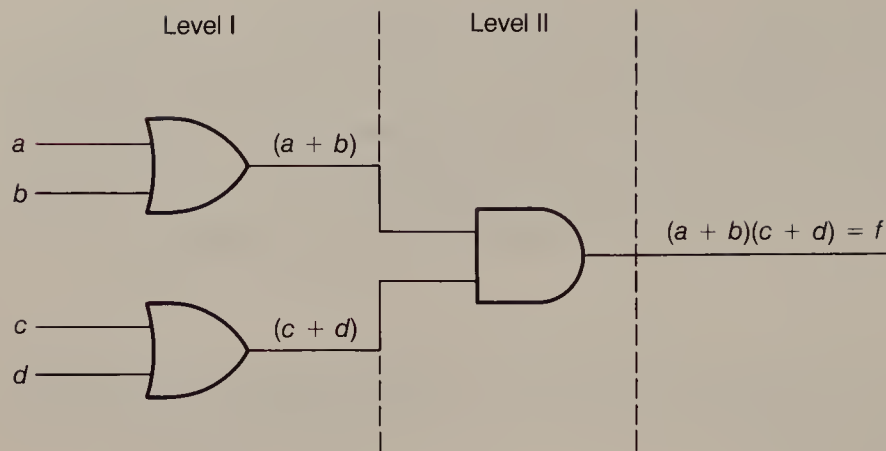
$$(x \oplus y) \oplus z = x \oplus (y \oplus z)$$

- An AND operation distributes over the exclusive-OR operation:

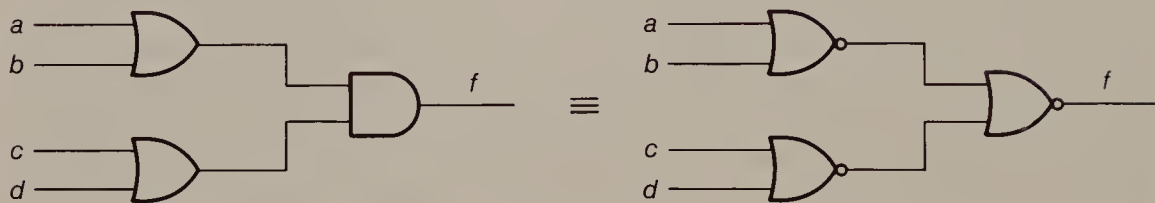
$$x(y \oplus z) = xy \oplus xz$$

- The OR operation can be expressed in terms of exclusive OR and AND as described next:

$$x + y = x \oplus y \oplus xy$$



a. A Two-level OR/AND Implementation of the Product of Sum Boolean Expression $(a + b)(c + d)$



b. The Relationship Between a Two-level OR/AND and NOR/NOR Implementations

Figure A.16

- Some additional useful properties of exclusive-OR are as follows:

$$x \oplus x = 0$$

$$x \oplus 0 = x$$

$$x \oplus 1 = x'$$

$$x \oplus (x \oplus y) = y$$

All the preceding properties can be proven by perfect induction or by algebraic manipulation. This task is left as an exercise.

The complement of the exclusive-OR operation is known as the exclusive-NOR operation; its truth table is described in Figure A.17.

x	y	$x \odot y$
0	0	1
0	1	0
1	0	0
1	1	1

Figure A.17 Truth Table Specification of Exclusive-NOR Operation



Figure A.18 Schematic Representation of an Exclusive-NOR Gate

Note that in the truth table we use the symbol $\odot$ in order to indicate the exclusive-NOR operation. Also, observe that the result of the exclusive-NOR operation is only 1 when both x and y assume identical values. This is why the operation is sometimes referred to as the *equivalence operation*. More formally,

$$x \odot y = x'y' + xy$$

The schematic representation of the exclusive-NOR gate is shown in Figure A.18.

A.2 COMBINATIONAL CIRCUIT DESIGN

In this section, the design of typical combinational circuits is discussed.

A.2.1. Adders

It can be shown that when bits x and y are added, a sum and a carry are generated. A circuit must be designed to perform this task. A block diagram representation of this circuit is shown in Figure A.19.

This system is also referred to as a half-adder because it takes only half of the bits to be added. A truth table for this hardware can be directly derived from the basic rules of the binary addition; it is shown in Figure A.20.

From the truth table, the Boolean equations for s and c outputs can be written as described next:

$$s = x'y + xy' = x \oplus y$$

$$c = xy$$

A hardware that implements these Boolean equations is diagrammed in Figure A.21.

More often an adder must be capable of taking care of three inputs: the bits to be added and a carry input from the previous stage. This type of adder is known as a *full-adder*, and its block diagram representation is specified in Figure A.22.

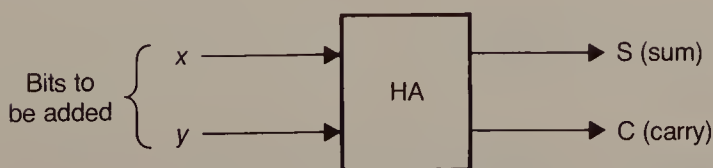


Figure A.19 Block Diagram Representation of a Half-adder

Inputs		Outputs	
x	y	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Figure A.20 Truth Table of a Half-adder

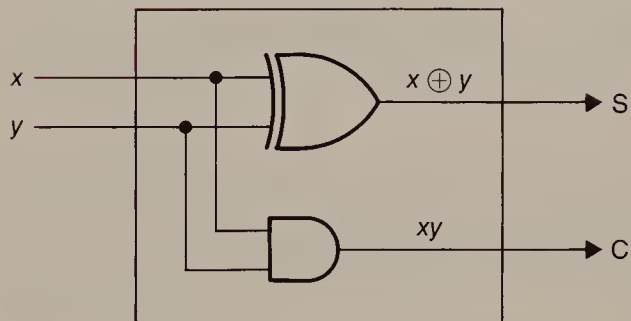


Figure A.21 Implementation of the Half-adder

The truth table that governs the operation of the full-adder is shown in Figure A.23.

$$s = x'y'C_{in} + x'yC_{in}' + xy'C_{in}' + xyC_{in}$$

$$= (x'y + xy'C_{in}') + (xy + x'y')C_{in}$$

If $w = x'y + xy' = x \oplus y$, then $w' = xy + x'y'$. Therefore,

$$s = wC_{in}' + w'C_{in}$$

$$= w \oplus C_{in} \quad (\text{by definition})$$

$$= x \oplus y \oplus C_{in} \quad (\text{therefore, } w = w \oplus y) \quad (\text{A.1})$$

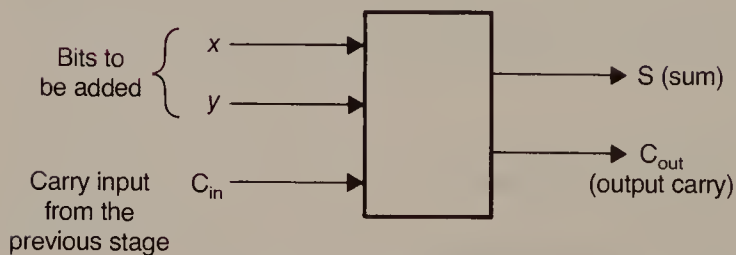


Figure A.22 Block Diagram Representation of a Full Adder

x	y	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Figure A.23 Truth Table Description of a Full-adder

$$\begin{aligned}
 C_{out} &= x'yC_{in} + xy'C_{in} + xyC_{in}' + xyC_{in} \\
 &= (x'y + xy') C_{in} + xy (C_{in} + C_{in}') \\
 &= w C_{in} + xy
 \end{aligned} \tag{A.2}$$

A logic circuit that implements equations A.1 and A.2 is shown in Figure A.24.

A.2.2 Comparators

Another widely used combinational system is the digital comparator. A block diagram of a two-bit digital comparator is shown in Figure A.25.

The input to this comparator are the two unsigned 2-bit integers A and B. The output G is 1 only if $A > B$. Similarly, the outputs E and L are 1 only when $A = B$ and $A < B$, respectively. It should be noted that for any given values of A and B only

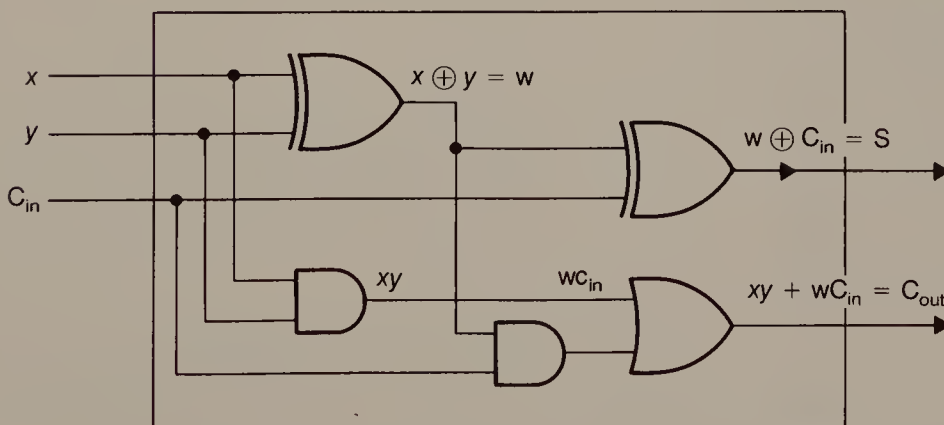


Figure A.24 Implementation of a Full-adder

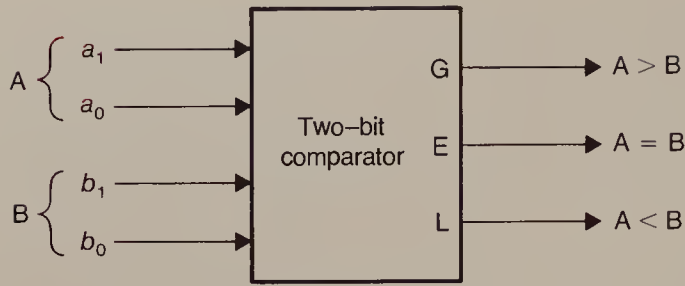


Figure A.25 Block-diagram Representation of a 2-bit Comparator

one of the three outputs will be 1. The necessary Boolean equations for implementing this comparator can be derived as follows.

When A and B are 2-bit numbers, $a_1 = 1$ and $b_1 = 0$ implies $A > B$. Thus

$$G = a_1 b_1' \quad (\text{A.3})$$

Similarly, $a_1 = 0$ and $b_1 = 1$ implies $A < B$. Thus it follows that

$$L = a_1' b_1 \quad (\text{A.4})$$

Thus $A = B$ only when corresponding bits are identical. That is,

$$A = B \rightarrow (a_1 = b_1) \text{ and } (a_0 = b_0)$$

This implication can be translated into the following Boolean equation

$$\begin{aligned} E &= (a_1 b_1 + a_1' b_1') (a_0 b_0 + a_0' b_0') \\ &= (a_1 \odot b_1) (a_0 \odot b_0) \end{aligned} \quad (\text{A.5})$$

A circuit that implements the equations A.3 through A.5 is shown in Figure A.26.

The same idea can also be extended to the design of an n -bit comparator where $n > 2$, interested readers may consult the references listed at the end of this appendix.

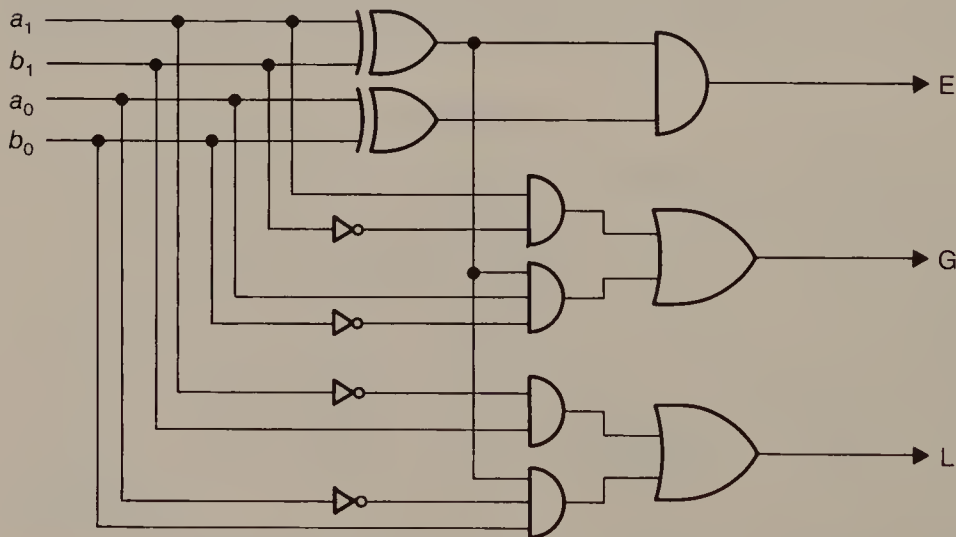
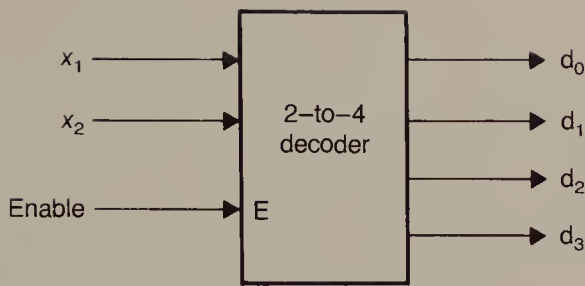


Figure A.26 Hardware Implementation of a 2-bit Comparator



A.27 Block-diagram Representation of a 2-to-4 decoder

A.2.3 Decoders and Encoders

A decoder is a very popular combinational circuit that is widely used in digital systems. The block diagram representation of a 2-to-4 decoder is shown in Figure A.27.

The truth table that governs the operation of this decoder is specified in Figure A.28.

From the truth table, an inspection of the subscript of each output line suggests that the decoder actually performs binary-to-decimal conversion operation. A complete hardware implementation of a 2-to-4 decoder is shown in Figure A.29.

In general, an n -to- 2^n decoder has n inputs and 2^n outputs. We can design large decoders using small decoders as the building blocks. For example, consider the design of a 4-to-16 decoder. This design can be accomplished by using five 2-to-4 decoders, as shown in Figure A.30.

It is possible to design a multiple-output combinational circuit using decoders and some extra gates. This idea is illustrated in the following example.

Let us design a full-adder using a 3-to-8 decoder and two 4 input or gates. From the truth table of the full-adder (See Figure A.22), we can directly obtain the following Boolean equations:

$$s = x'y'C_{in} + x'yC_{in} + xy'c_{in}' + xyC_{in}$$

$$C_{out} = x'yC_{in} + xy'C_{in} + xyC_{in}' + xyC_{in}$$

E	x_1	x_0	d_0	d_1	d_2	d_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

Figure A.28 Truth Table of the 2-to-4 Decoder

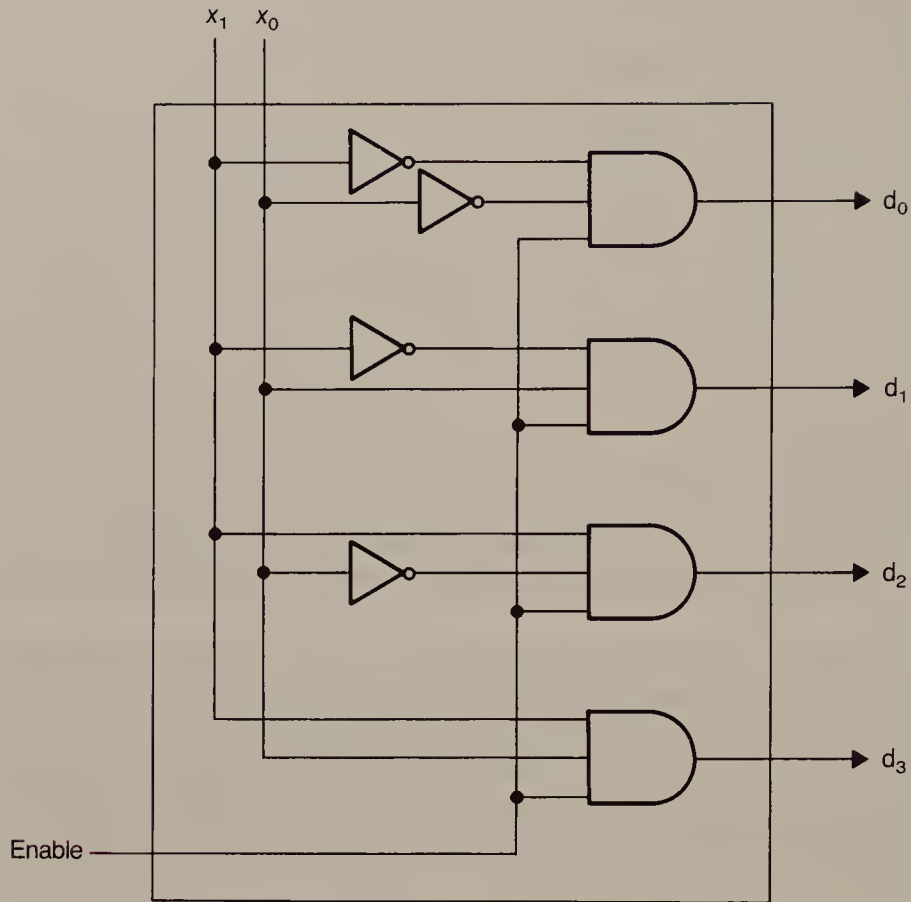


Figure A.29 Logic Diagram of a 2-to-4 Decoder

These equations can also be written in minterm form as follows:

$$s = m_1 + m_2 + m_4 + m_7 \quad (\text{A.6})$$

$$C_{\text{out}} = m_3 + m_5 + m_6 + m_7 \quad (\text{A.7})$$

where

$$m_1 = x'y'C_{\text{in}}$$

$$m_2 = x'yC_{\text{in}}'$$

$$m_3 = x'yC_{\text{in}}$$

$$m_4 = xy'C_{\text{in}}'$$

$$m_5 = xy'C_{\text{in}}$$

$$m_6 = xyC_{\text{in}}'$$

$$m_7 = xyC_{\text{in}}$$

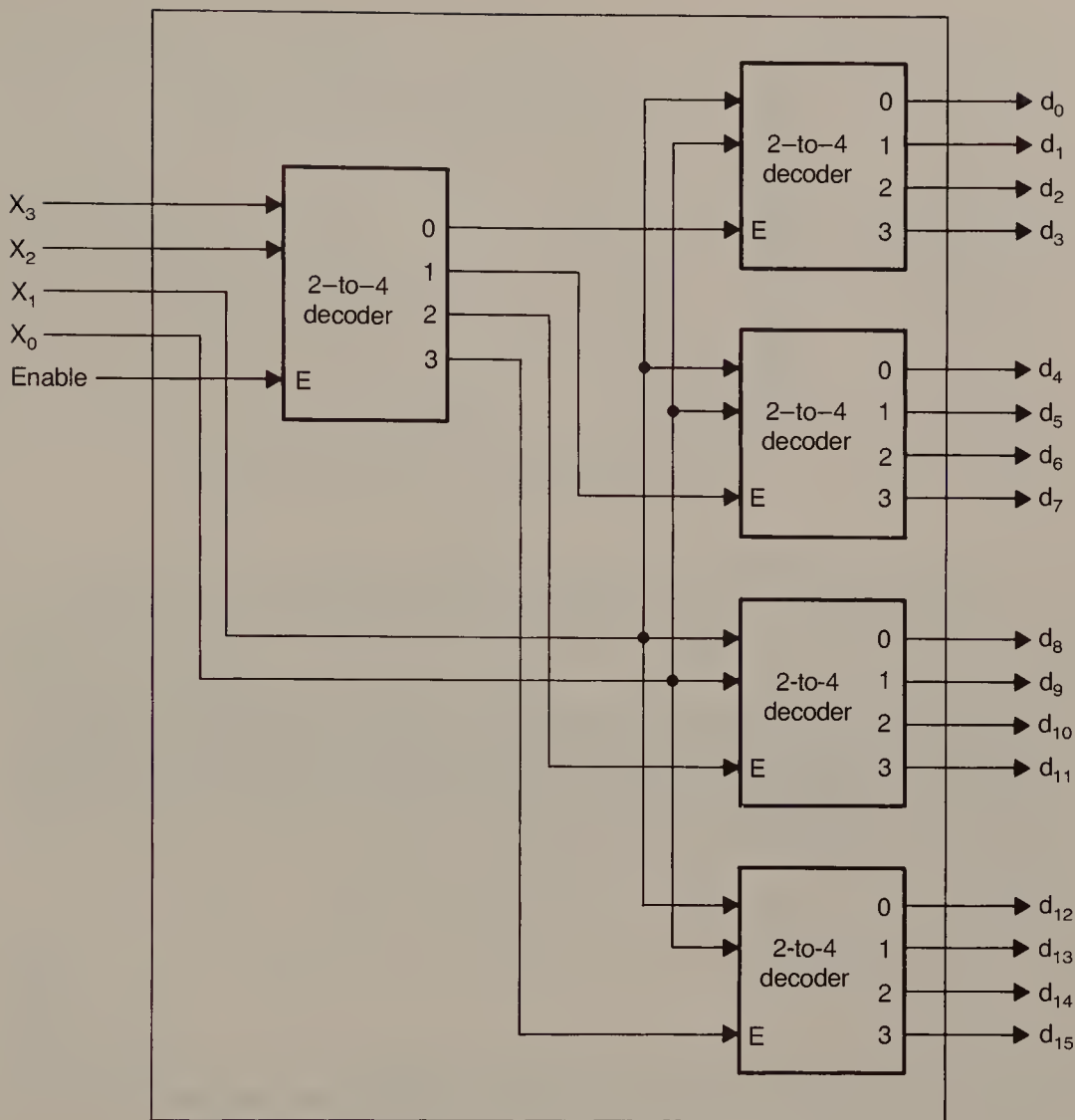


Figure A.30 Implementation of a 4-to-16 Decoder Using 2-to-4 Decoder

Note that the minterm representation allows one to specify a sum of product-type Boolean expression in a compact manner. The following are some important properties of the minterm expansion of Boolean function.

- Every Boolean function has a unique minterm representation. This means that two Boolean functions are equivalent if they have identical minterm representation.
- For a Boolean function of n variables, the subscript i of a particular minterm m_i can lie in the range 0 to $2^n - 1$.
- For any given input combination, only one of the 2^n possible minterms will assume a value 1. This implies that $m_i \cdot m_j = 0$ if $i \neq j$.

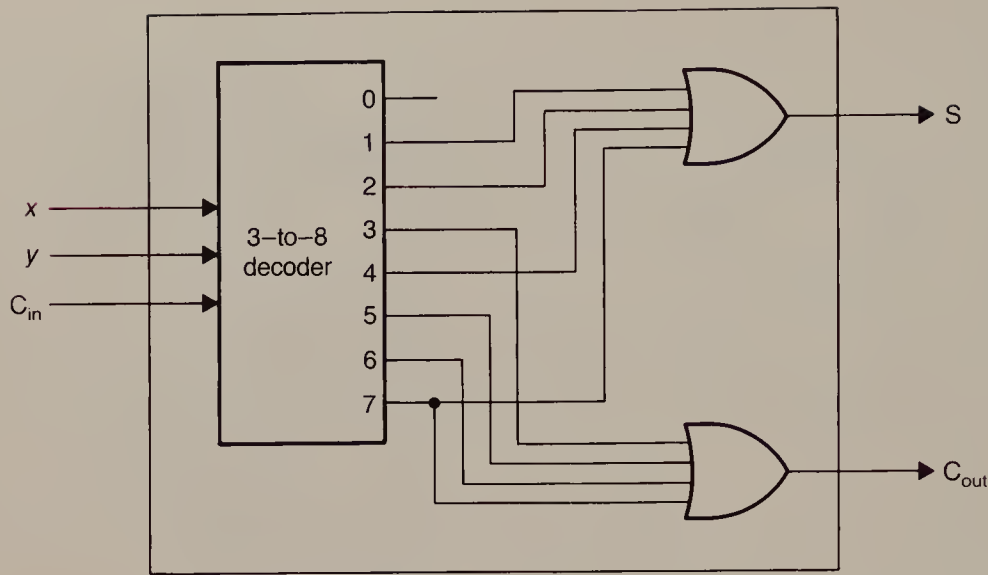


Figure A.31 Implementation of a Full-adder Using a 3-to-8 Decoder and Two 4-input OR Gates

- The complement of the minterm expansion is called the maxterm expansion, and it is used to take care of Boolean functions that are in product of sum form. For further discussions on this topic, we suggest that the readers consult the references listed at the end of this appendix.

Equations A.6 and A.7 can be further simplified as follows:

$$S = \Sigma_m (1, 2, 4, 7) \quad (\text{A.8})$$

$$C_{\text{out}} = \Sigma_m (3, 5, 6, 7) \quad (\text{A.9})$$

Note that in the preceding result, only the decimal subscripts of the corresponding min-terms that are included in the approximate Boolean equations are specified. Using the result of Equations A.8 and A.9, a decoder-based circuit can immediately be designed; this is shown in Figure A.31. The task performed by an encoder is exactly opposite to that of a decoder. The block diagram and the truth table corresponding to a 4-to-2 encoder are shown in Figures A.32 and A.33, respectively.

The necessary Boolean equations can be derived from the truth table.

$$x_0 = d_0'd_1d_2'd_3' + d_0'd_1'd_2d_3 \quad (\text{A.10})$$

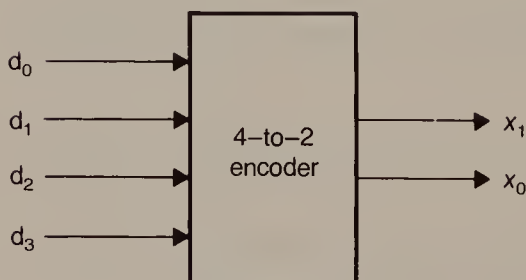


Figure A.32 Block Diagram of a 4-to-2 Encoder

d_0	d_1	d_2	d_3	x_1	x_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	1	1	1	1

Figure A.33 Truth Table Description of a 4-to-2 Encoder

$$\begin{aligned}
 x_1 &= d_0'd_1'd_2d_3' + d_0'd_1'd_2d_3 \\
 &= d_0'd_1'd_2(d_3 + d_3') \\
 &= d_0'd_1d_2
 \end{aligned} \tag{A.11}$$

A logic circuit that implements these equations is shown in Figure A.34.

In general, an encoder will have 2^n inputs and n outputs. Also, from the truth table of Figure A.33, it is easy to notice that an encoder actually performs decimal-to-binary conversion.

In this discussion, it was assumed that at any given time, only one of the 2^n outputs will assume a value 1. If this restriction is relaxed, the resulting circuit will be called a priority encoder. In this system if we regard each input as a decimal number, then the output will be the binary equivalent of the maximum decimal number that has assumed a value 1.

For example, in a 4-to-2 priority encoder, if the inputs d_0 , d_2 , and d_3 are 1, the output will be 11 because 3 is the maximum of the three decimal numbers 0, 2, and 3. ($3_{10} = 11_2$). A complete truth table description of the priority encoder is shown in Figure A.35.

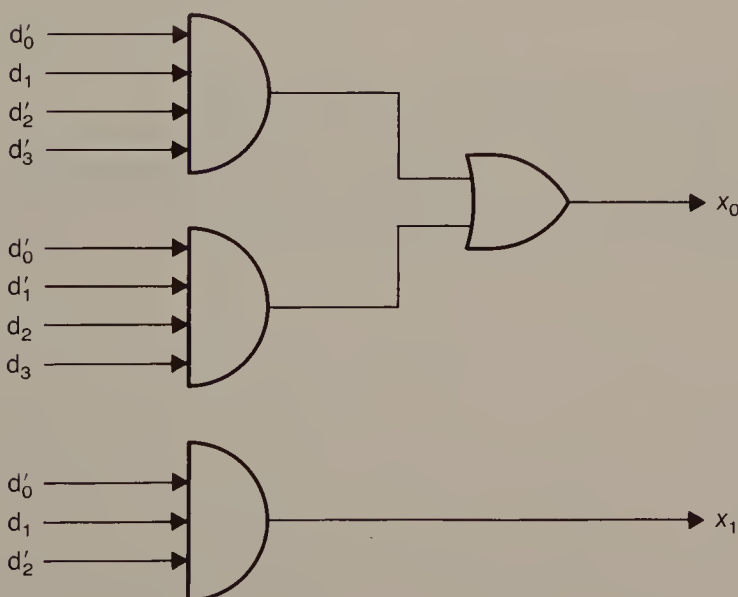


Figure A.34 Implementation of a 4-to-2 Encoder

d_3	d_2	d_1	d_0	x_1	x_0
1	x	x	x	1	1
0	1	x	x	1	0
0	0	1	x	0	1
0	0	0	1	0	0

$x = \text{don't care}$

Figure A.35 Truth Table of a Typical 4-to-2 Priority Encoder

This hardware is very useful in the design of interrupt systems and bus arbitration schemes.

A.2.4 Multiplexers

Another useful combinational logic element is the multiplexer, or data selector. The block diagram and a truth table of a 2-to-1 multiplexer are shown in Figures A.36 and A.37, respectively.

From the truth table, it is clear that a two-input multiplexer selects one of the two data inputs d_0 or d_1 , depending on the value of the selection input S_0 . In particular, if $S_0 = 0$, the system selects the input d_0 ; otherwise, the data input d_1 is selected. More formally,

if $S_0 = 0$
 then $Z \leftarrow d_0$
 else $Z \leftarrow d_1$

From this specification, the Boolean equation governing the operation of the multiplexer can be written:

$$Z = S_0'd_0 + S_0d_1 \quad (\text{A.12})$$

A logic circuit that implements equation A.12 is shown in Figure A.38.

In general, a multiplexer with n select lines is capable of selecting one of the 2^n data inputs $d_0, d_1, \dots, d_k$, where $k = 2^n - 1$. For example, the block diagram and the truth table of a four-input multiplexer are shown in Figures A.39 and A.40, respectively.

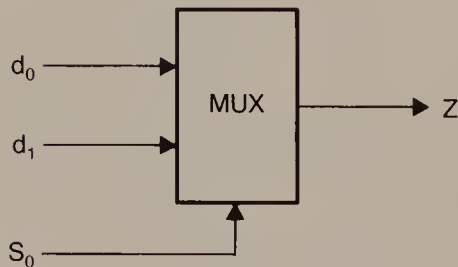


Figure A.36 Block Diagram of a Two-input Multiplexer

S_0	Z
0	d_0
1	d_1

Figure A.37 Truth Table of the Two-input Multiplexer of Figure A.36

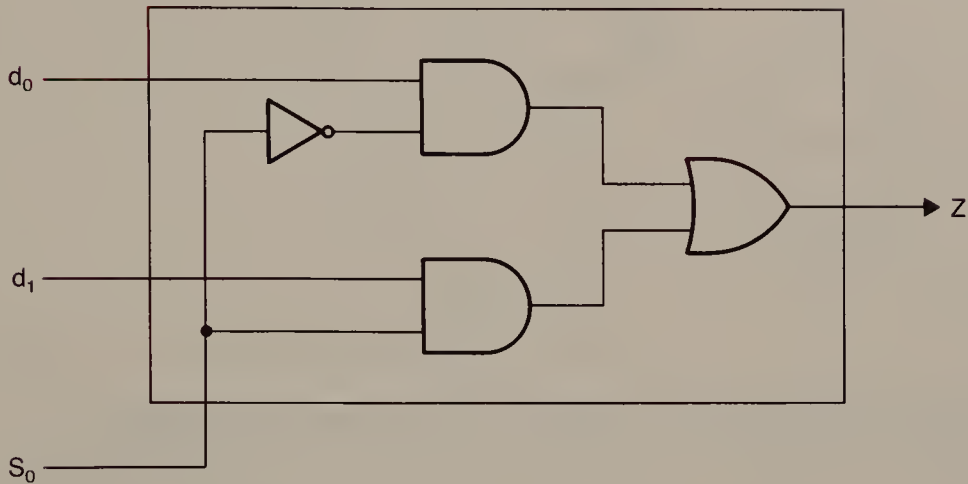


Figure A.38 Implementation of a Two-input Multiplexer

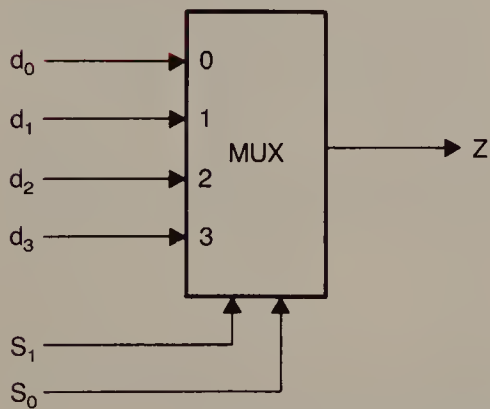


Figure A.39 Block-diagram Representation of a Four-input Multiplexer

S_1	S_0	z
0	0	d_0
0	1	d_1
1	0	d_2
1	1	d_3

Figure A.40 Truth Table Specification of the Four-input Multiplexer of Figure A.39

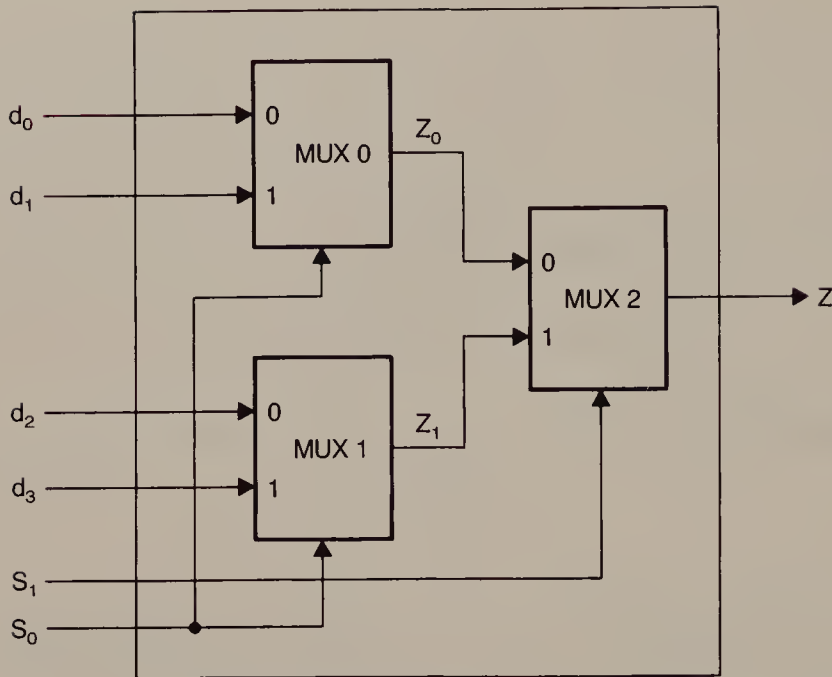


Figure A.41 Implementation of a Four-input Multiplexer Using Only Two-input Multiplexers

This multiplexer can be implemented using only two-input multiplexers; this is shown in Figure A.41.

In the figure, the select line S_0 is applied as input to the multiplexers Mux_0 and Mux_1 . This means that $z_0 = d_0$ or d_1 and $z_1 = d_2$ or d_3 , depending on whether $S_0 = 0$ or 1. The select line S_1 is given as input to the multiplexer Mux_2 . This implies that $z = z_0$ if $S_1 = 0$; otherwise $z = z_1$. In this arrangement if $S_1 S_0 = 11$ then $z = d_3$. This is because $S_0 = 1$ implies $z_0 = d_1$ and $z_1 = d_3$. Since $S_1 = 1$, the Mux_2 selects the data input z_1 , and thus $z = d_3$. The other entries of the truth table of Figure A.41 can be verified in a similar manner.

A.2.5 ROMs and PLAs

In this section, the structure and applications of ROMs and PLAs are described.

A ROM is a nonvolatile memory that provides read-only access to the stored data. A block-diagram representation of a ROM is shown in Figure A.42. Thus the total capacity of this ROM is $2^n \times m$ bits. Whenever an n -bit address is placed on the address line, the m -bit information stored in this address will appear on the data lines. The m -bit output generated by the ROM is also called a word. Now let us discuss the internal structure of a ROM.

Internally, a ROM is composed of a decoder and an array of OR gates; this is illustrated in Figure A.43.

The OR gate of the ROM may be built using diodes; a typical 3-input diode OR gate is shown in Figure A.44.

In order to see how a ROM works, let us consider the 2×4 ROM of Figure A.45.

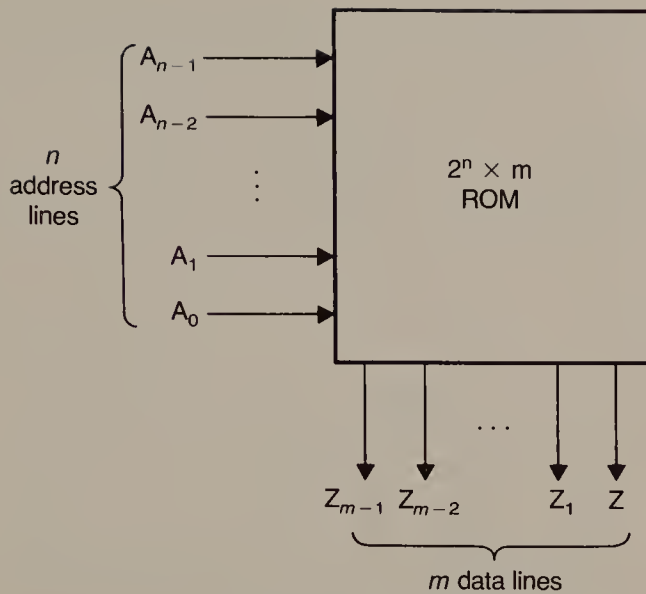


Figure A.42 Block-diagram Representation of a ROM

In this system when $A_1A_0 = 00$, the decoder output line 0 will be high. This causes the diodes D_{00} and D_{01} to conduct and thus the output $Z = Z_3Z_2Z_1Z_0 = 0011$. Similarly, when $A_1A_0 = 01$, the decoder output line 1 goes high. Under this situation, the diode D_{10} conducts, and it is easy to see that generated output will be $Z = Z_3Z_2Z_1Z_0 = 0100$. Continuing in the same manner, it may be concluded that this ROM actually implements the truth table shown in Figure A.46.

Note that this truth table is the same as that of Example A.5. Thus it follows that

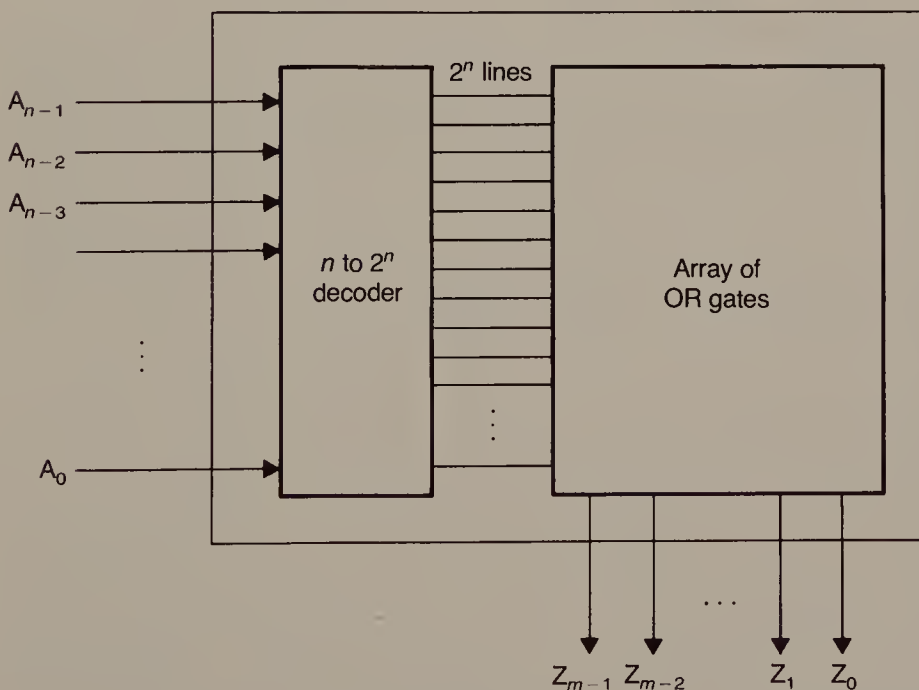


Figure A.43 Internal Structure of a ROM

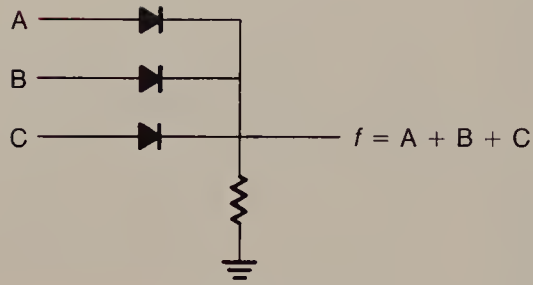


Figure A.44 Diode-OR Gate

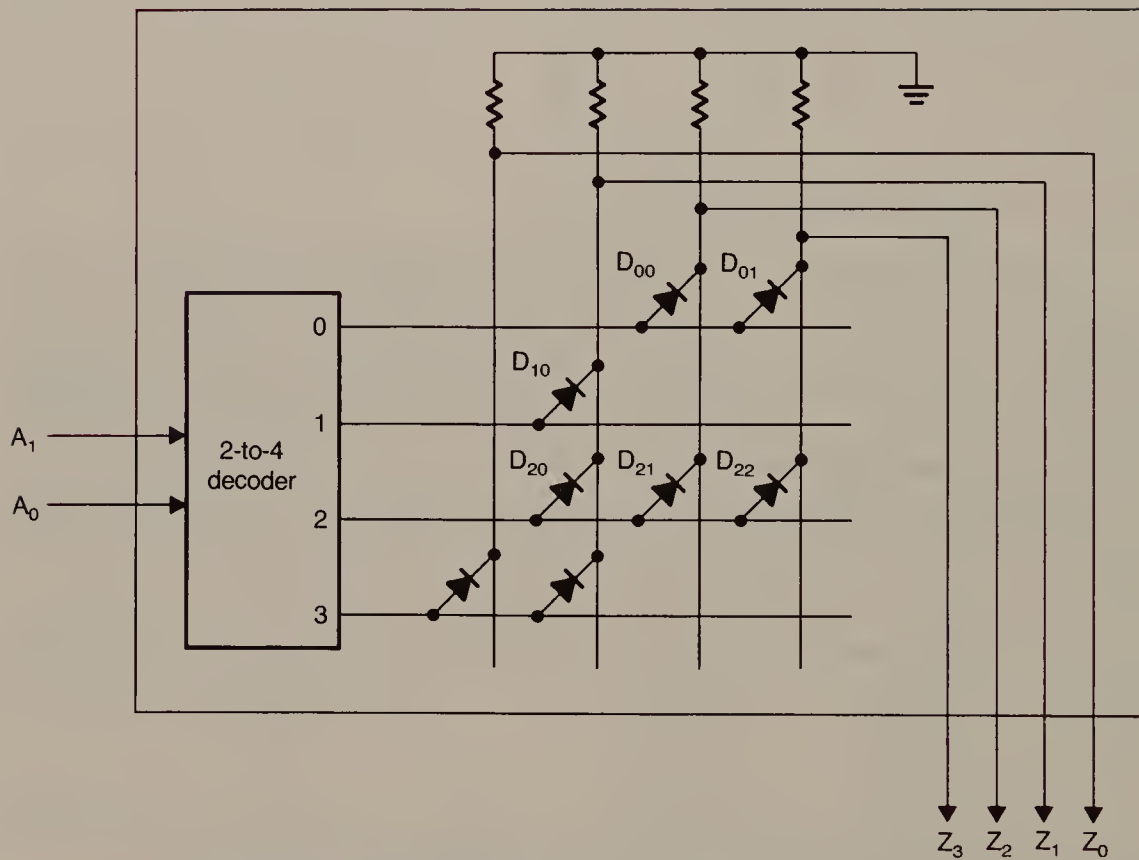


Figure A.45 Hardware Organization of a Typical 2×4 ROM

A_1	A_0	Z_3	Z_2	Z_1	Z_0
0	0	0	0	1	1
0	1	0	1	0	1
1	0	0	1	1	1
1	1	1		0	0

Figure A.46 Truth Table implemented by the ROM of Figure A.45

any combinational circuit can be implemented using a ROM. A ROM implementation of a truth table involves the following steps:

- If there are k inputs, a k -to- 2^k decoder is selected. In this case there will be 2^k output lines 0 through $2^k - 1$. Each such output line is called a *row line*.
- If there are l outputs, the ROM will have l output lines labeled z_0 through z_{l-1} . Each such line is called a *column line*.

A	B	C_{in}	Σ (sum)	C_{out} (carry)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

ROM implementation offers a cost-effective solution in order to build circuits for performing useful tasks such as square root and transcendental function computations.

The diode OR gate array of the ROM is also customarily referred to as the *diode matrix*. It should be pointed out that in this approach the decoder circuit has to supply the currents required by the load connected to the output lines. This requirement limits the use of diode matrix in some applications where the load demands substantial amounts of current. In order to rectify this difficulty, the diode matrix may be replaced with an array of transistors. For example, the ROM of Figure A.45 may be redesigned using transistors; it is shown in Figure A.47.

In this arrangement, when $A_1A_0 = 00$, the transistors T_{00} and T_{01} are turned on, and thus the output generated is $z_3z_2z_1z_0 = 0011$. Note that in this situation, the current required by the load is supplied by the ON transistor and not by the decoder.

The ROMs shown in Figures A.45 and A.47 are called *bipolar* ROMs because they use bipolar devices. It should be pointed out that in the IC technology a diode is derived from a transistor by short-circuiting the collector and the base of the latter. The principal advantage of a bipolar ROM is its high operating speed. The access time of a bipolar ROM typically lies in the range 16 to 50 nanoseconds. However, large-size ROMs cannot be constructed using bipolar devices because they occupy more real-estate area. Therefore, large-scale ROMs are often implemented using MOS transistors. A MOS ROM implementation of the truth table of Figure A.46 is shown in Figure A.48.

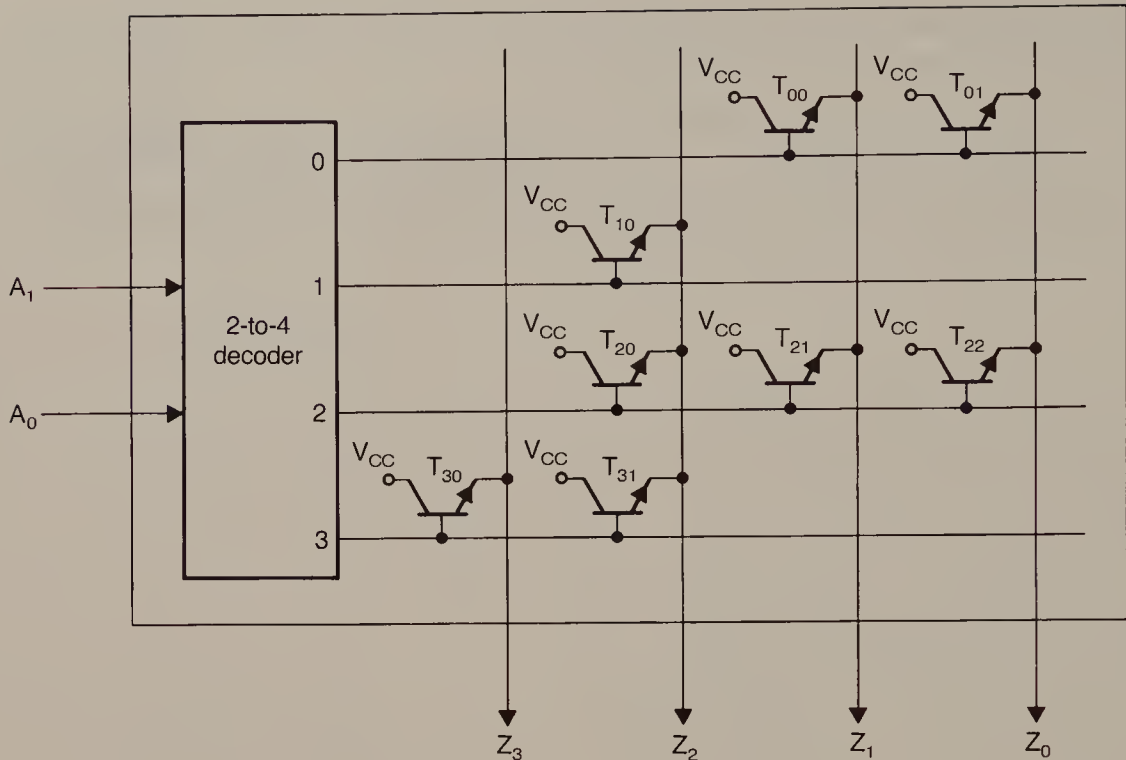


Figure A.47 Alternative Implementation of the ROM of Figure A.45

In this implementation the MOS transistors Q_0 , Q_1 , Q_2 , and Q_3 act as active loads to the MOS NAND gate realized by each column line. Under this arrangement, if $A_1A_0 = 00$, then the transistors Q_{00} and Q_{01} will conduct, and thus $z_3z_2 = 00$. Since the transistors Q_2 and Q_3 are always on, it follows that $z_1z_0 = 11$.

Thus the application of the input $A_1A_0 = 00$ results in an output 0011. It is interesting to note that in an MOS ROM implementation, we place a device in the intersection of the row line p and column line z_j if the output z_j corresponding to the binary input pattern is to be 0. The access time of a MOS is typically of the order of 450 nanoseconds. This figure indicates that the operating speed of this hardware is considerably less than its bipolar counterpart.

From this discussion it is clear that before a ROM can be used, its content must be initialized. This activity involves placing the switching devices at the appropriate intersection points of the row and column lines. Depending on the time and the manner in which this initialization takes place, a ROM is called by different names. For example, in a mask ROM the contents of the ROM are initialized by the manufacturer at the time of its production. This means that this approach is well suited for producing standard circuits (such as a bar-code generator or a 2×2 multiplier) that are widely used by many digital designers. Since these types of ROMs are mass-produced, their costs are also very low. However, a mask ROM cannot be reconfigured by a user. That is, a user cannot alter its contents.

There are occasions in which a user may wish to develop a specific ROM-based circuit as demanded by the application area. Under this circumstance, a ROM that allows a user to initialize its contents is required. A ROM with such a flexibility is known

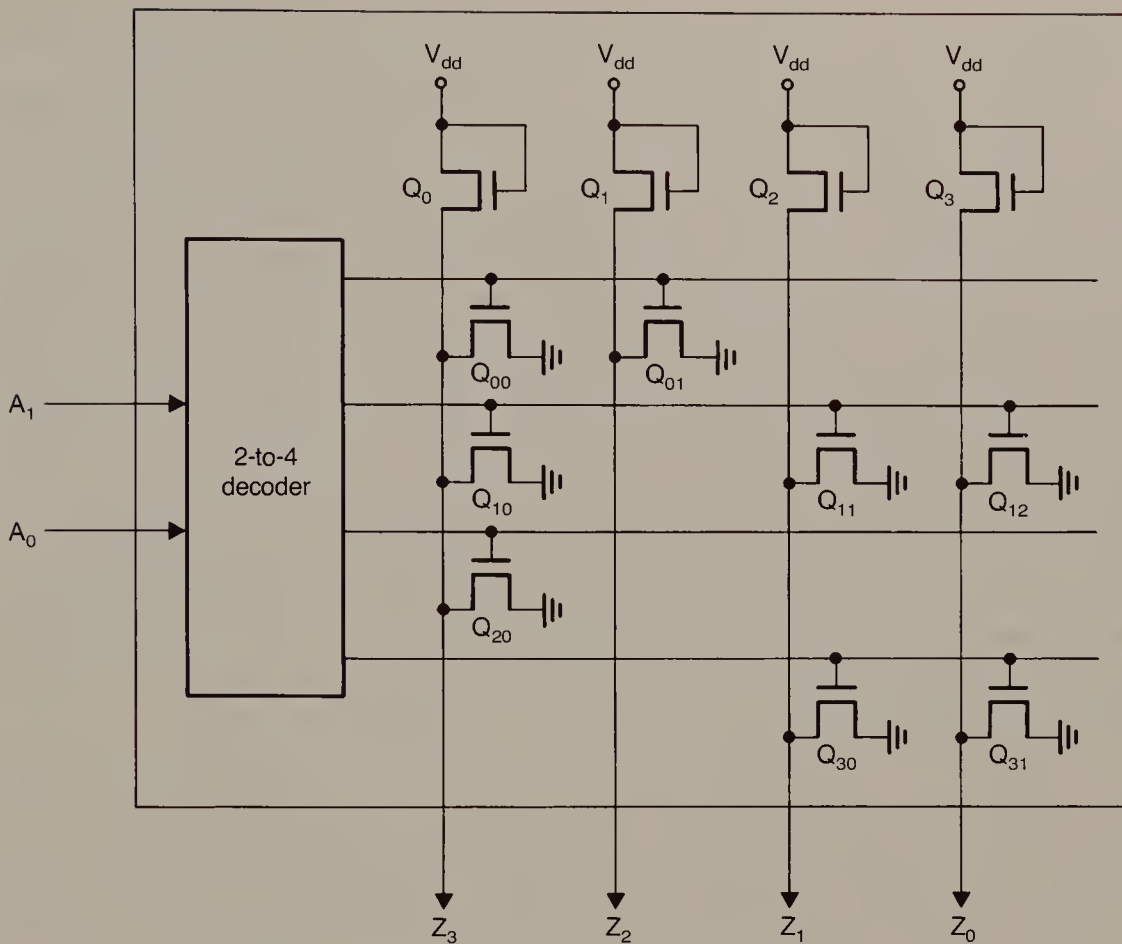


Figure A.48 MOS ROM Implementation of the Truth Table of Figure A.46

as a *PROM* (*programmable ROM*). PROMs may be manufactured using fusible links. A typical bipolar ROM cell with fusible link is illustrated in Figure A.49.

In this device, the manufacturer places a switching element along with a fusible link at each intersection. This implies that all ROM cells are initialized with a 1. If a user desires to see a zero in a particular cell, he or she needs to blow off the fuse at that point. This activity is called *programming*, and it may be accomplished by passing electrical impulses. It should be pointed out that in such a ROM a user can program the

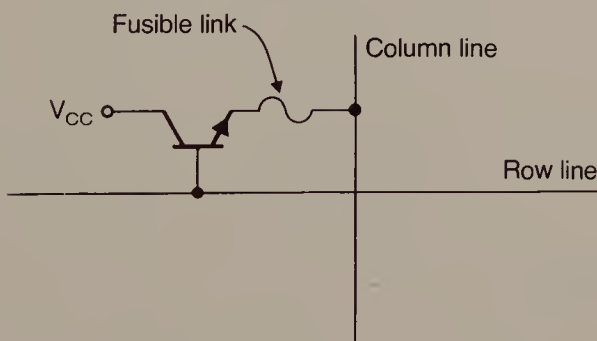


Figure A.49 A ROM Cell with Fusible Link

ROM only once. That is, if the fuse is blown off, then it may not be possible to restore it again.

When a new product is developed, it is likely that a designer may need to modify the contents of the ROM frequently. This means that it should be possible to erase the original ROM contents and initialize the device with new data. A ROM with this capability is referred to as an *EPROM* (*erasable programmable ROM*). Usually the contents of this memory are erased by exposing the ROM chip to ultraviolet light. Typical erase times vary between 10 and 30 minutes. After erasure the ROM may be reprogrammed by passing voltage pulses at the required points. The growth in the IC technology allows the production of a new type of ROM whose contents may be erased using electrical impulses. This memory device is customarily referred to as an *EAROM* (*electrically alterable ROM*). The principal advantage of an EAROM is that its contents may be changed without removing the chip from the circuit board. Readers may consult the references listed at the end of this appendix in order to obtain more information.

The internal structure of a typical PLA (programmable logic array) is shown in Figure A.50. The AND array of this system generates the required product terms and the OR array is used to sum the product terms generated by the AND array. As in the case of a ROM, these gate arrays can be realized using diodes, transistors, or MOS devices. The significance of a PLA is explained in the following example.

Consider the PLA shown in Figure A.51. This PLA has three inputs, A, B, and C. The AND array generates from product terms $A'B'$, $A'C'$, BC , and AC . These product terms are summed up in the OR array, and the outputs z_0 , z_1 , and z_2 are generated. Note that the dot in the figure indicates the presence of a switching element such as diode or transistor. It should be pointed out that the PLA does not need a decoder. The use of PLAs is very cost-effective when the number of inputs in a combinational circuit realized by a ROM is very high and all input combinations are not used. For example, consider the following multiple-output functions:

$$W = AE + BC$$

$$X = CD + FE$$

$$Y = FG + HI$$

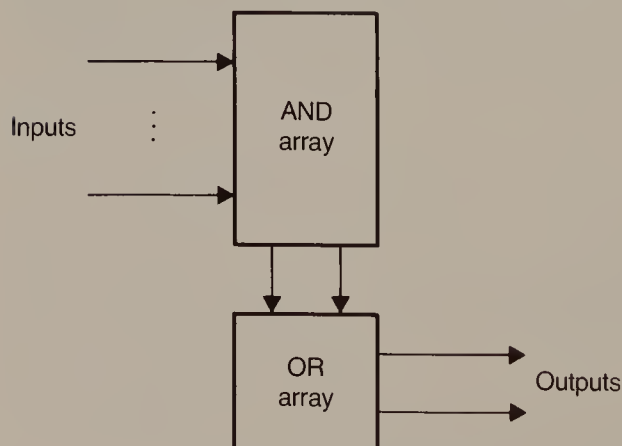


Figure A.50 Internal Structure of a PLA

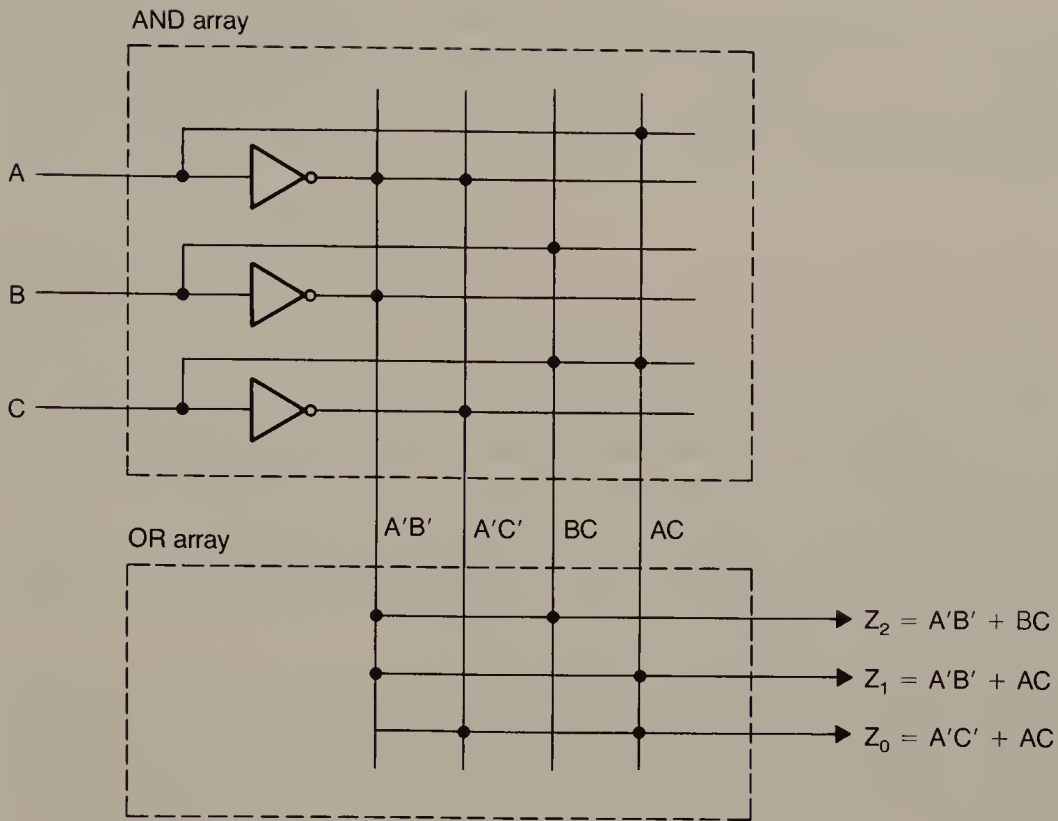


Figure A.51 A PLA with Three Inputs, Four Product Terms, and Three Outputs

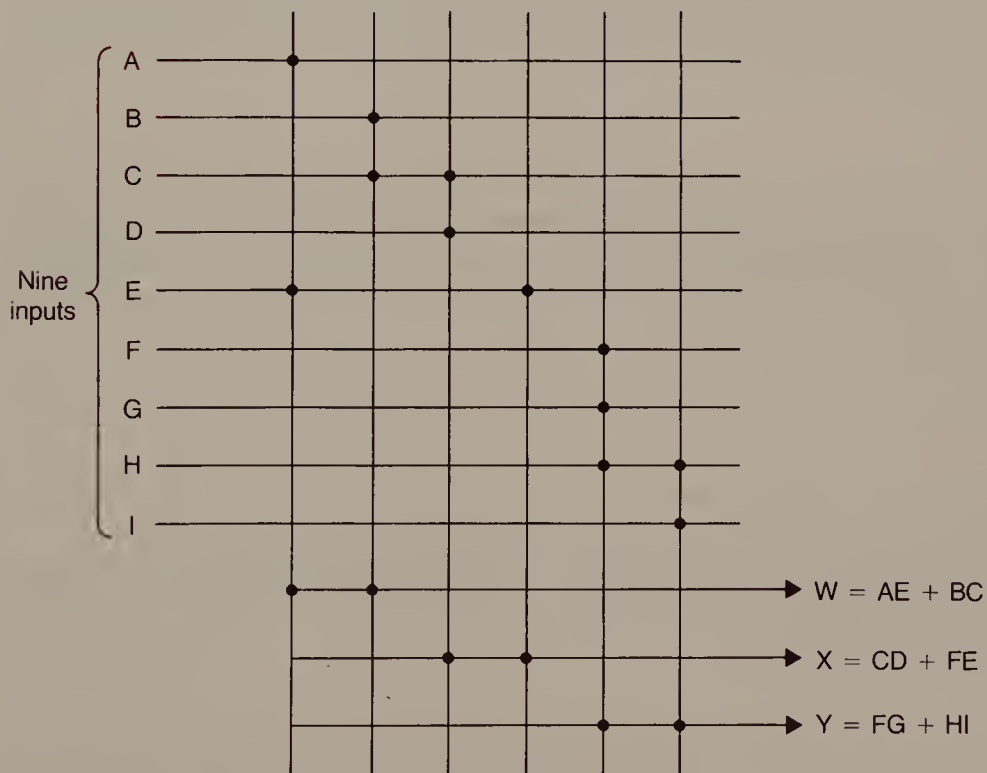


Figure A.52 PLA with Nine Inputs, Six Product Terms, and Three Outputs

In order to implement the preceding Boolean functions in a ROM, a 512×3 array is needed because in the preceding there are nine inputs (A thru I) and three outputs (w, x, y). But the same function can be realized in a PLA using six product terms, nine inputs, and three outputs, as shown in Figure A.52. Therefore, a considerable savings in hardware can be achieved with PLAs.

A.3 SEQUENTIAL CIRCUIT DESIGN

The design of combinational circuits is considered so far. The main characteristic of these circuits is that the outputs at a particular time t are determined by the inputs at the same time, t . This means that combinational circuits are memoryless. However, in reality there are a number of circuits that include memory along with combinational circuitry. These circuits are called *sequential*.

In sequential circuits, the present outputs depend on the present inputs and the previous states stored in the memory elements. Note that these states must be fed back to the inputs in order to generate the present outputs.

There are two types of sequential circuits—*synchronous* and *asynchronous*. In a synchronous sequential circuit, a clock signal is used in specific places to make sure that all desired operations are initiated only by the synchronizing clock pulse. On the other hand, in an asynchronous sequential circuit, the synchronizing clock pulse is not necessary. In this case, completion of one operation starts the next operation in sequence. Therefore, time for each operation is normally different from others. Most of the sequential circuits are synchronous in nature, since it is very easy to design and analyze such circuits. The following discussion is, therefore, restricted to synchronous sequential circuits.

A.3.1 Flip-Flops

A flip-flop is a 1-bit memory. As long as the power is available, the flip-flop retains the bit. However, its output (stored information) can be changed by the clock input. Since flip-flops play a vital role in synchronous sequential circuit, they are discussed in the following paragraphs.

There are four basic types of flip-flops. There are R-S, J-K, D, and T.

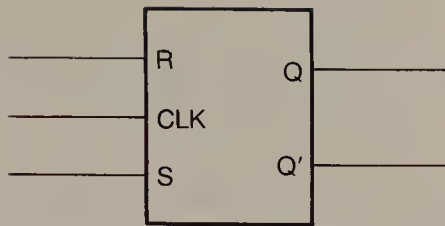
R-S Flip-Flop

A symbolic representation and characteristic table of R-S flip-flop is shown in Figure A.53.

Note that in case of the R-S flip-flop, when both R and S inputs are 1, the output is indeterminate. Therefore, the set (S) and reset (R) inputs of the R-S flip-flop can never be high at the same time.

J-K Flip-Flop

The indeterminate output of the R-S flip flop when $R = S = 1$ is eliminated in the J-K flip-flop. In this type, the J and K inputs are similar to R and S inputs. Figure

**a.** Symbolic Representation of R-S flip-flop

Q(t)	S	R	Q (t + Δt)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	?
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	?

b. Characteristic Table**Figure A.53** R-S Flip-flop

A.54 shows the symbolic representation and characteristic table of the J-K flip-flop. Note that in the figure, when $J = K = 1$, the output is complemented.

D Flip-Flop

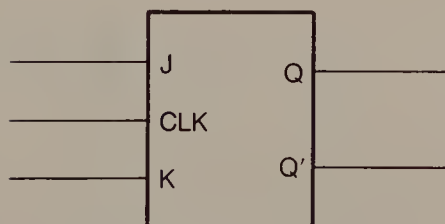
The D flip-flop has only two inputs, the D input and the clock. This type of flip-flop is usually used to transfer data to the flip-flop output. The symbolic representation and characteristic table of the D flip-flop are shown in Figure A.55.

T Flip-Flop

The T flip-flop also has two inputs. These are the T input and the clock. The T flip-flop toggles or complements the output when the clock input occurs. The T flip-flop can be obtained from the J-K flip-flop by connecting the J-K inputs to HIGH. Figure A.56 shows the symbolic representation and the characteristic table of the T flip-flop.

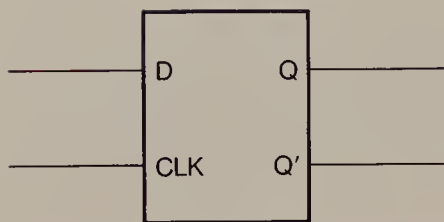
A.3.2 Systematic Design of Typical Sequential Circuits

In this section the design of typical sequential circuits is described. For the first example consider the following design problem. Design a sequential circuit with one input X and

**a.** J-K flip-flop Symbolic Representation

Q(t)	J	K	Q (t + Δt)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

b. Characteristic Table**Figure A.54** J-K flip-flop



a. Symbolic Representation of D Flip-flop

$Q(t)$	D	$Q(t + \Delta t)$
0	0	0
0	1	1
1	0	0
1	1	1

Figure A.55 D Flip-flop

b. Characteristic Table

one output Z. The input X is a serial message and the system reads X one bit at a time. The output $Z = 1$ whenever the pattern 101 is encountered in the serial message. For example:

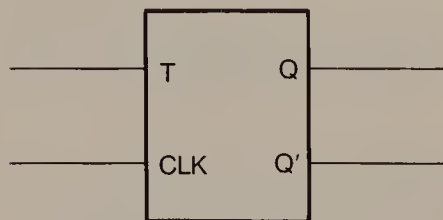
Input X: 00101011101000101 . . .

Output Z: 00001010001000001

For the first step the block diagram of this system is drawn; this is shown in Figure A.57.

For the second step a state diagram of the system is drawn; it is shown in Figure A.58.

In this diagram each node represents a state. The labeled arcs (lines joining two nodes) represent state transitions. For example, when the system is in the state C and if it receives an input 1, it produces an output 1 and makes a transition to the state D after the clock. Similarly, when the system is in state C and if it receives a 0 input, it generates a 0 output and moves to the state A after the clock. Note that the arcs are labeled in the format p/q . In this format, the symbols p and q denote the input read and



a. Symbolic Representation

$Q(t)$	T	$Q(t + \Delta t)$
0	0	0
0	1	1
1	0	1
1	1	0

Figure A.56 T Flip-flop

b. Characteristic Table

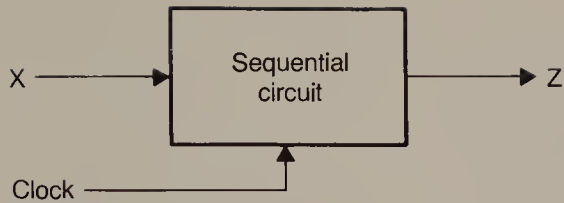


Figure A.57 Block Diagram of the Sequential Circuit

the output produced, respectively. This type of sequential circuits is called a *Mealy machine* because the output generated depends on both input X and the present state of the system. It should be emphasized that each state in the state diagram actually performs a bookkeeping operation; these operations are summarized as follows:

STATE	INTERPRETATION
A	Looking for a new pattern
B	Received the first 1
C	Received a 1 followed by a 0
D	Recognized the pattern 101

For the third step, the state diagram can be translated into a tabular representation called a *state table*, as shown in Figure A.59.

For the fourth step the state assignment is done, which means that a binary pattern is associated with each symbolic state of the state table. One possible state assignment for this problem is specified next:

SYMBOLIC STATE	BINARY STATE	
	y_1	y_0
A	0	0
B	0	1
C	1	1
D	1	0

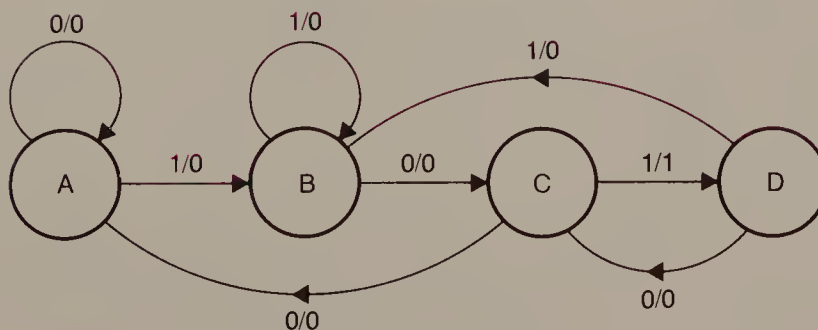


Figure A.58 State Diagram of the Sequential Circuit

Present State	Next State		Output Z	
	Input $x = 0$	Input $x = 1$	Input $x = 0$	Input $x = 1$
A	A	B	0	0
B	C	B	0	0
C	A	D	0	1
D	C	B	0	0

Figure A.59 State Table Corresponding to the State Diagram of Figure A.58

The original state table can be modified in order to reflect this state assignment, as illustrated in Figure A.60.

The excitation table for a T flip-flop is shown in Figure A.61.

Note that the excitation table actually describes the required excitation for a particular state transition to occur. For example, with respect to a T flip-flop, for the transition $0 \rightarrow 1$ or $1 \rightarrow 0$, a 1 must be applied to the T input. Similarly, a transition $0 \rightarrow 0$ or $1 \rightarrow 1$ (that is, no change of state), the T input must be made 0. Using this excitation table the flip-flop input equations can be derived. This step is shown in the form of a tabulation, as illustrated in Figure A.62.

In this figure the entries corresponding to the flip-flop inputs T_{y_1} and T_{y_0} are directly derived using the T flip-flop excitation table of Figure A.61. For example, consider the present state $y_1y_0 = 00$. When the input is $x = 1$, the next state is 01. This means that the flip-flop y_1 should not change its state and the flip-flop y_0 must change its state to 1. It follows that $T_{y_1} = 0$ (because a $0 \rightarrow 0$ transition is required) and $T_{y_0} = 1$ (because a $0 \rightarrow 1$ transition is required). The other entries for T_{y_1} and T_{y_0} may be obtained in a similar manner. Also notice that Figure A.62 is organized in such

Present State $y_1 \ y_0$	Next State $y_1(t + \Delta t) \ y_0(t + \Delta t)$		Output Z	
	Input $x = 0$	Input $x = 1$	Input $x = 0$	Input $x = 1$
0 0	0 0	0 1	0	0
0 1	1 1	0 1	0	0
1 1	0 0	1 0	0	1
1 0	1 1	0 1	0	0

Figure A.60 Modified State Table

Possible State Transitions		Required Excitation
Q	Q(t + Δt)	T
0	0	0
0	1	1
1	0	1
1	1	0

Figure A.61 Excitation Table for a T Flip-flop

a way that we see a three-variable (y_1 , y_0 , and x) k-map under output Z , T_{y_1} , and T_{y_0} . Thus, using this k-map, we can derive the required equations as follows:

$$\begin{aligned} Z &= xy_1y_0 \\ T_{y_1} &= x'y_0 + xy_1y_0' \\ T_{y_0} &= y_1 + xy_0' \end{aligned}$$

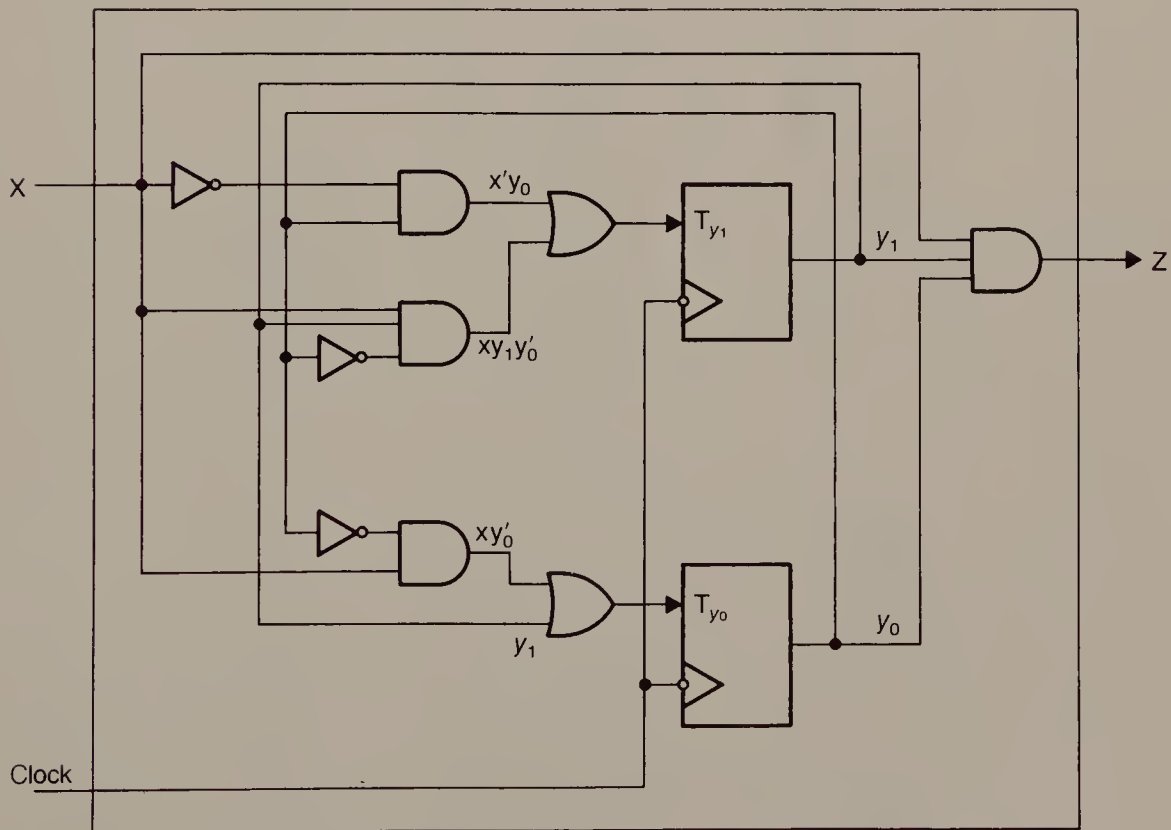


Figure A.63 Final Hardware Implementation

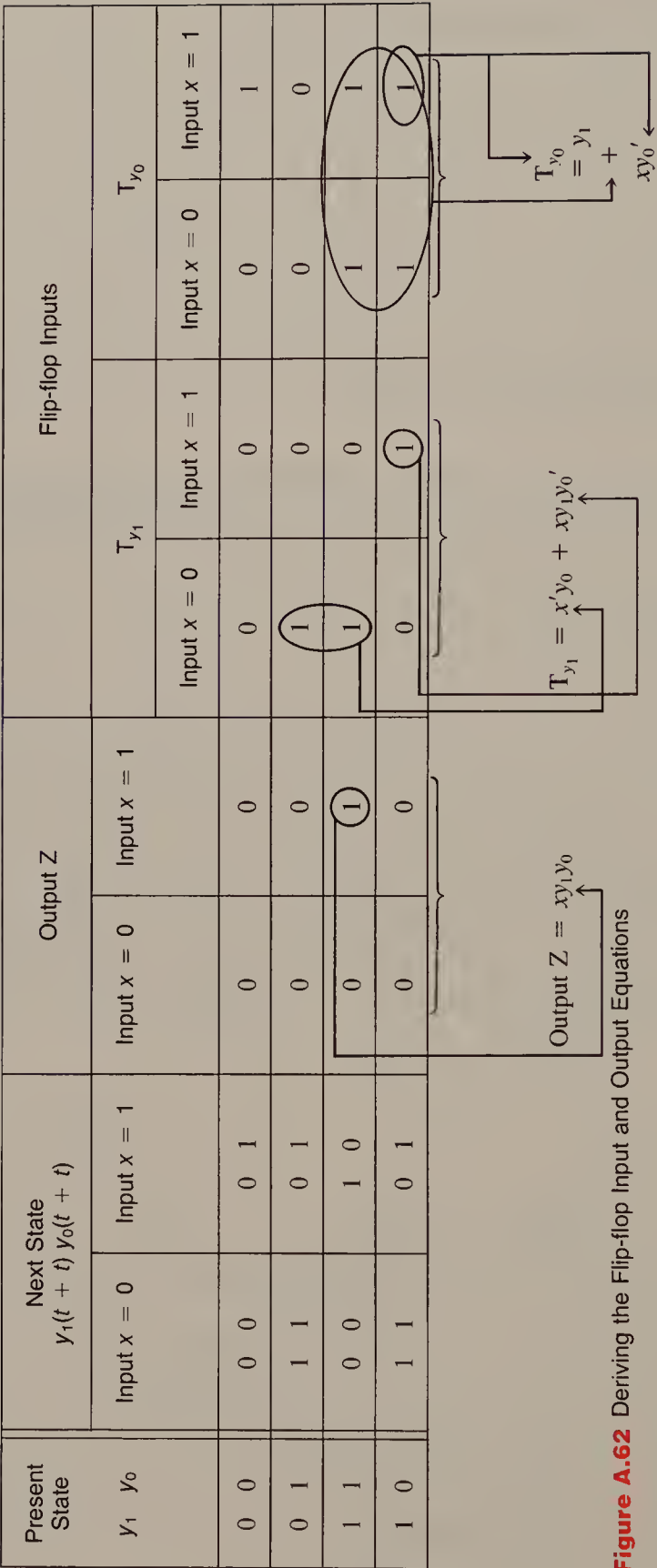


Figure A.62 Deriving the Flip-flop Input and Output Equations

The last step in the design problem is to draw the final circuit; it is shown in Figure A.63.

REFERENCES

1. Hayes, J. P. *Computer Architecture and Organization*. New York: McGraw-Hill, 1978.
2. Mitra, S. K. *An Introduction to Digital and Analog Integrated Circuits and Applications*. New York: Harper & Row, 1980.
3. Mano, M. M. *Digital Logic and Computer Design*. Englewood Cliffs, N.J.: Prentice-Hall, 1979.

APPENDIX **B**

Motorola's MC68000 and Its Instruction Set

MC68000 is Motorola's 16-bit nanoprogrammed microprocessor. This chip contains approximately 68000 transistors, and the entire system is housed in a 64-pin chip. This unit is operated by a single 5-volt power supply. The internal frequency of operation of this processor can be as high as 12.5 megahertz. The programming model of this device is shown in Figure B.1.

This CPU includes eight 32-bit data registers (D0–D7) and nine 32-bit address registers A0–A7 plus A₇'. Data registers normally hold data items such as 8-bit bytes, 16-bit words, 32-bit long words, and 4-bit BCD numbers. On the other hand, an address register usually holds the memory address of an operand. Note that address registers A7 and A₇' can function only as stack pointers. The MC68000 processor can operate in two different modes: user and supervisory modes. In the supervisory mode, the MC68000 can execute all instructions. However, when the processor is in the user mode, the processor cannot execute some instructions; these instructions are called *privileged instructions*. The system hardware automatically selects register A7 or A₇' as the user SP or supervisory SP, respectively. Note that the supervisory mode allows one to design MC68000-based multiprogramming and multitasking systems.

MC68000 is a general-purpose register processor because any data register can be configured as an accumulator or as a scratch-pad register. Also, any data or address

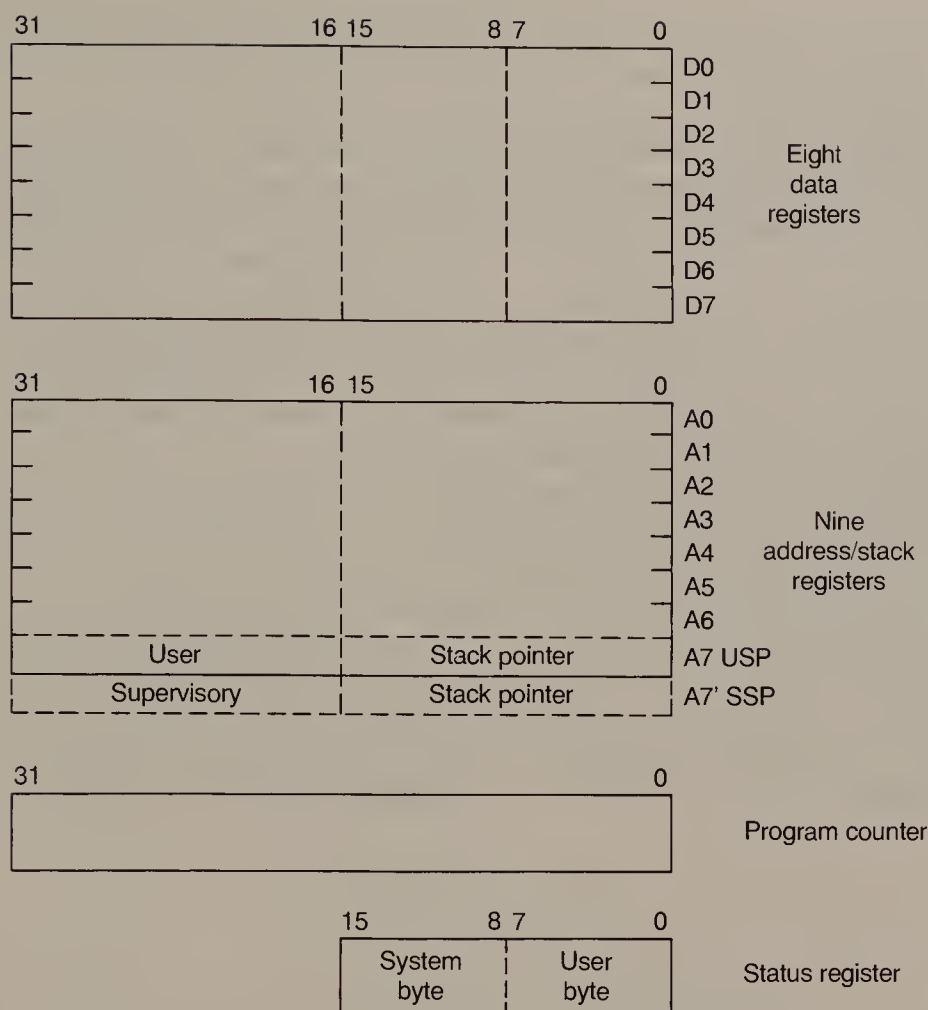


Figure B.1 MC68000 Programming Model (Courtesy of Motorola)

register can be used as an index register. Although the MC68000 PC is 32 bits wide, only the low-order 24 bits are used at present. Since this is a byte-addressable machine, it follows that the MC68000 CPU can directly address 16 megabytes.

The MC68000 status register is composed of two bytes: a user byte and a system byte. The user byte includes typical condition codes such as C, V, N, Z, AND X. Note that the MC68000 does not have any ADDC or SUBC instructions, but it has ADDX and SUBX instructions. Since the carry flag C and extend flag X are usually affected in an identical manner, the MC68000 uses ADDX or SUBX to reflect the carries in multiprecision arithmetic. The contents of the system byte include a 3-bit interrupt mask (I2, I1, I0), supervisory flag (S), and a trace flag (T). When the supervisory flag is 1, then the system operates in the supervisory mode; otherwise the user mode operation is assumed. When the trace flag is set to 1, the processor generates a trap (internal interrupt) after executing each instruction. A debugging routine can be written at the interrupt address vector to display registers and/or memory after execution of each instruction. Thus, this provides single-stepping facility.

The interrupt mask bits provide the status of the MC68000 interrupt pins IPL2,

IPL1, and IPL0. In the normal mode, I2 I1 I0 = 000 indicates no interrupt. I2 I1 I0 = 111 indicates a nonmaskable interrupt (level 7), and the other combinations of I2 I1 I0 provide the maskable interrupt levels.

MC68000 supports bytes (8 bits), words (16 bits), and long words (32 bits), as shown in Figure B.2. Byte addressing includes both odd and even addresses (0, 1, 2, 3, . . .). Word addressing includes only even addresses in increments of 2 (0, 2, 4, . . .) and long-word addressing contains even addresses in increments of 4 (0, 4, 8, . . .).

All MC68000 instructions occupy one to five words, and they always start on a word boundary. The first word is called the *operation word*, and it specifies the operation types, operand sizes, and their addressing modes. The remaining words include other information such as immediate operands or extensions to the effective address mode specified in the operation word (see Figure B.3).

In the MC68000, there are more than 18 different instruction formats; a detailed description of some of them is provided in Figure B.4.

Consider instruction format 1. Here the 2-bit operand type field specifies the operand size in the following manner: 01, byte; 11, word; and 10, long word. The 6-bit operand field is further divided into a 3-bit mode field and a 3-bit register field. In the MC68000, all registers cannot be used in all modes. In other words, a particular mode forces a programmer to use only a set of registers in that mode. For example, we cannot use an address register in as a destination in an 8-bit MOVE instruction. Because of

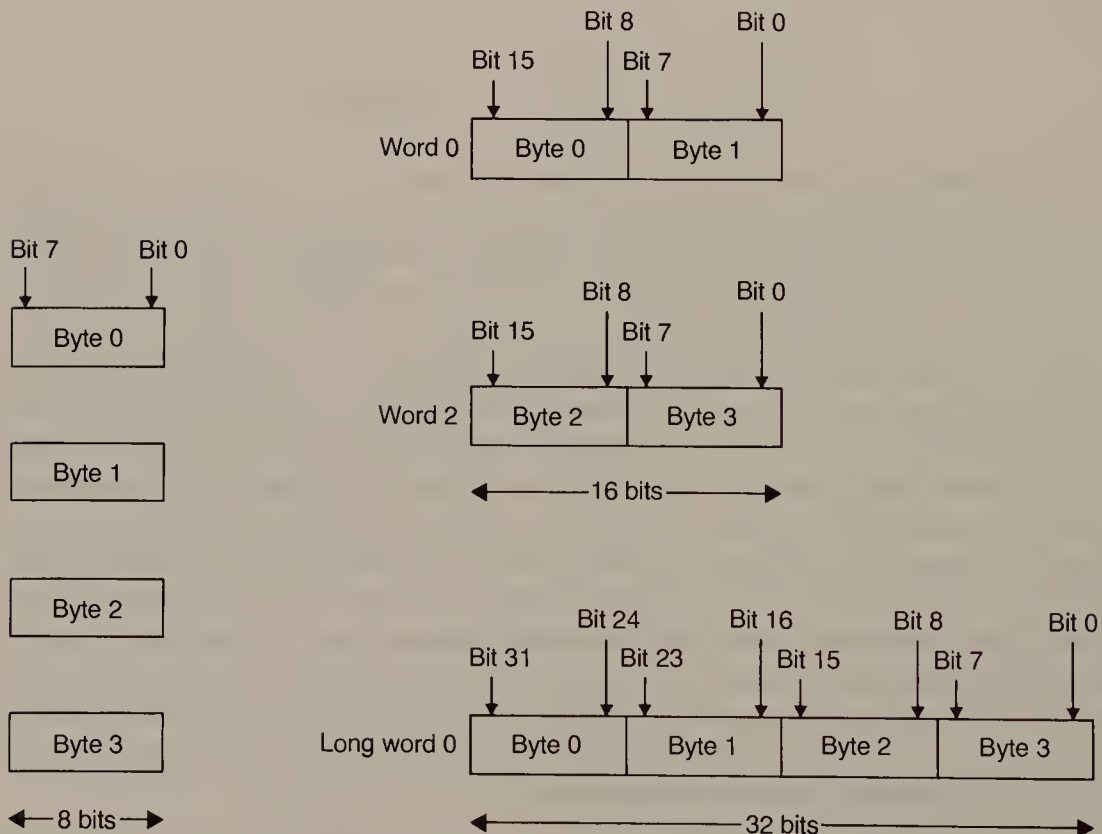


Figure B.2 MC68000 Addressing Structure

this restriction, a 3-bit register field is adequate to take care of 16 registers (D0 through D7 plus A0 through A7).

Also, many two address instructions expect one of the operands to be in the register mode. For example, consider the MC68000 ADD instruction (see format 2 of Figure B.4). In this case, we have only three possible formats, namely, register to register, memory to register, and vice versa. This means that we cannot perform memory-to-memory addition.

Some MC68000 instructions have multiple formats. For example, the ROR (rotate right) instruction has two different formats: one for register rotate and the other for memory rotate.

Small constants are represented using the formats 4 and 5. Typically, constants in the range from 1 to 8 may be specified using format 5. On the other hand, format 4 allows one to specify an 8-bit 2's complement number. In this case the destination must be a data register and the 8-bit data is sign extended to obtain a 32-bit long word; this result is transferred to the specified data register.

The following symbols will be used to discuss the MC68000 addressing modes:

SYMBOL	INTERPRETATION
EA	Effective address
Dn	Data register
An	Address register
Xi	Data or address register used as index register
PC	Program counter
d ₈	8-bit offset
d ₁₆	16-bit offset
addr16	16-bit address
addr32	32-bit address
data16	16-bit data
data32	32-bit data
k	Increment factor (1 for byte, 2 for word, and 4 for long-word operands)

The MC68000 processor supports 14 different addressing modes; 12 of them are described in Figure B.5. A close inspection of this table reveals the following:

- Most of the PDP-11 addressing modes are also available in the 68000.
- The table does not include two addressing modes: inherent and quick modes. Note that the quick mode is the same as the immediate mode, except that it allows only 8-bit constants. In the inherent mode, we specify an implied register such as USP, SSP, or CCR (condition code register)

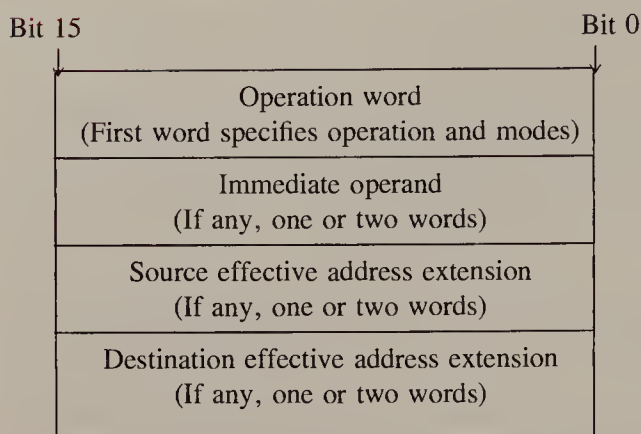


Figure B.3 MC68000 Instruction Format (Courtesy of Motorola)

- As far as indirection is concerned, only address-register-based one-level indirection is available.
- The indexed mode is very versatile. This allows a programmer to set up and access a table.

Let us now explain some of the addressing modes in Figure B.5.

- *Data register direct addressing mode:* Consider `ADD $ 003000, D5`. Assume $[00300] = 0004$, $[D5] = 002_{16}$. After execution of `ADD $ 003000, D5`, the contents of $D5 = 0004 + 0002 = 0006$. Note that in the preceding instruction, the symbol `$` is used to represent hexadecimal numbers by Motorola.
- *Address register indirect mode:* Consider `ADD (A2) +, D2`. This instruction uses postincrement mode. If $[A2] = 002000_{16}$, $[002000]_{16} = 0003_{16}$ $[D2] = 0005_{16}$, then after execution of this `ADD`, $[D2] = 0003 + 0005 = 0008_{16}$ and $[A2] = 002002_{16}$. Note that `A2` is incremented by 2, since this is a word instruction.
- *Immediate addressing mode:* Consider `ADD # $ 0003, D3`. If $[D3] = 0004_{16}$, then after this `ADD`, $[D3] = 0004 + 0003 = 0007_{16}$. Note that the `#` symbol is used by Motorola to indicate the immediate mode.

The MC68000 instruction set contains the 56 basic instructions summarized in Figure B.6. The repertoire is very versatile and offers efficient means to handle high-level language constructs such as arrays and linked lists. Note that in order to identify the operand size of an instruction, the following is placed after a 68000 mnemonic: B for byte, W or none for word, and .L for long word. For example,

```
ADD.B D2, D3;    [D3]8 ← [D2]8 + [D3]8
ADD.W   )
      or   ) D2, D3;    [D3]16 ← [D2]16 + [D3]16
ADD     )
```

```
ADD.L   D2, D3;    [D3]32 ← [D2]32 + [D3]32
```

All 68000 instructions may be classified into eight different groups, as follows.

FORMAT NUMBER	NUMBER OF FIELDS	NAME OF THE FIELD	FIELD SIZE (in bits)	INSTRUCTIONS THAT USE THIS FORMAT
1	4	Op-code	2	MOVE
		Operand type	2	
			6	
		Operand 1	6	
		Operand 2		
2	4	Op-code	4	ADD, AND, CMP, SUB
		Register	3	
		Mode	3	
		Operand	6	
3	5	Op-code 1	4	MOVEP
		Register 1	3	
		Mode	3	
		Op-code 1	3	
		Register	3	
4	4	Op-code 1	4	MOVEQ
		Register	3	
		Op-code 2	1	
		Data	8	
5	5	Op-code 1	4	ADDQ, SUBQ
		Data (small constant)	3	
		Op-code 2	1	
		Operand type	2	
			6	
		Operand		

Figure B.4 Some MC68000 Instruction Formats

NAME OF THE ADDRESSING MODE	MODE FIELD	REG. FIELD	REGISTER IMPLIED BY THE REGISTER FIELD	ASSEMBLER SYNTAX	NUMBER OF EXTRA WORDS REGISTERED	EA CALCULATION
Data register direct	000	Reg. NO.	Data register	Dn	None	$EA = Dn$
Address register direct	001	Reg. NO.	Address register	An	None	$EA = An$
Address register indirect	010	Reg. NO.	Address register	(An)	None	$EA = (An)$
Address register indirect with postincrement	011	Reg. NO.	Address register	(An) +	None	$EA = (An)$ $An = (An) + k$
Address register indirect with predecrement	100	Reg. NO.	Address register	-(An)	None	$An = (An) - k$ $EA = (An)$
Address register indirect with displacement	101	Reg. NO.	Address register	$d_{16}(An)$	1	$EA = (An) + d_{16}$

Address register indirect with index	110	Reg. NO.	Address register	$d_8(An, Xi)$	1	$EA = (An) + (Xi) + d_8$
Absolute short	111	000	None	$addr16 \cdot W$	1	$EA = \text{next memory word sign-extended}$
Absolute long	111	001	None	$addr32 \cdot L$	2	$Ea = \text{next two memory words}$
Program counter relative with displacement	111	010	None	$d_{16}(PC)$	1	$EA = (PC) + d_{16}$
Program counter relative with index	111	011	None	$d_8(PC, Xi)$	1	$EA = (PC) + (Xi) + d_8$
Immediate	111	100	None	#data16 #data32	1 or 2	Operand value follows the instruction word

Figure B.5 MC68000 Addressing Modes (Courtesy of Motorola)

MNEMONIC	DESCRIPTION
ABCD ADD AND ASL ASR	Add decimal with extend Add Logical AND Arithmetic shift left Arithmetic shift right
Bcc BCHG BCLR BRA BSET BSR BTST	Branch conditionally Bit test and change Bit test and clear Branch always Bit test and set Branch to subroutine Bit test
CHK CLR CMP	Check register against bounds Clear operand Compare
DBcc DIVS DIVU	Test condition, decrement, and branch Signed divide Unsigned divide
EOR EXG EXT	Exclusive-OR Exchange registers Sign extend
JMP JSR	Jump Jump to subroutine
LEA LINK LSL LSR	Load effective address Link stack Logical shift left Logical shift right
MOVE MOVEM MOVEP MULS MULU	Move Move multiple registers Move peripheral data Signed multiply Unsigned multiply
NBCD NEG NOP NO	Negate decimal with extend Negate No operation Ones complement
OR	Logical or
PEA	Push effective address

Figure B.6 MC68000 Instruction Set (Courtesy of Motorola)

MNEMONIC	DESCRIPTION
RESET	Reset external devices
ROL	Rotate left without extend
ROR	Rotate right without extend
ROXL	Rotate left with extend
ROXR	Rotate right with extend
RTE	Return from exception
RTR	Return and restore
RTS	Return from subroutine
SBCD	Subtract decimal with extend
Scc	Set conditional
STOP	Stop
SUB	Subtract
SWAP	Swap data register halves
TAS	Test and set operand
TRAP	Trap
TRAPV	Trap on overflow
TST	Test
UNLK	Unlink

Figure B.6 continued (Courtesy of Motorola)

Data-Movement Instructions

These instructions allow a data transfer from register to register, register to memory, memory to register, and memory to memory. Typically, we can transfer a byte, word, or long word. This is a truly orthogonal instruction because the source and destination operands can be in any mode. There are special data-movement instructions: move multiple registers (MOVEM); exchange registers (EXG); load effective address (LEA); push effective address (PEA); link stack (LINK); unlink stack (UNLK); and move quick (MOVEQ). In order to explain these instructions clearly, we provide the following illustrations:

INSTRUCTION

COMMENT

MOVEM.L
D0-D7/A0-A6,-(SP)

Save the contents of all the data registers and the first seven address registers in the stack. Stores address registers first A-7-A0 followed by data register D7-D0. Usually used for pushing multiple registers onto stack.

MOVEM.L
(SP)+, D0-D7/A0-A6

Restore the contents of the CPU registers from the stack. Registers are transferred in the order D0-D7, A0-A7 regardless of the order in list. Normally used for popping multiple registers from stack.

EXG	A0,A1	Exchange the contents of the address registers A0 and A1.
LEA	X, A0	Initialize the address register A0 with the address of the variable X.
PEA	Y	Push the address of the variable Y into the stack.
MOVEQ	#0, D2	Clear the data register D2.

The LINK and UNLK instructions are used to implement procedure calls. Typically, the LINK instruction creates space in the stack for the local variables of a procedure. On the other hand, the UNLK instruction releases the space that has been created for the local variable. This aspect is very important for implementing block-structured languages such as Pascal or ALGOL60. A detailed discussion of this topic is beyond the scope of this book.

A typical data-movement instruction is as follows. For example, consider MOVE (A2), D1. If $[A2] = 00000040_{16}$ and $[000040_{16}] = 0002$, then after this MOVE, $[D1]_{\text{lowword}} = 0002_{16}$.

Binary Arithmetic Instructions

The MC68000 instruction set allows 8-, 16-, or 32-bit quantities to be added and subtracted. A complete summary of these instructions is included in Figure B.7. Note that the 68000 instruction set does not include any explicit increment or decrement instructions; however, they are realized in the quick mode ADDQ or SUBQ instructions, respectively. For example, the one-word instruction ADDQ.B #1, D0 increments the low-order byte of the data register D0 by 1. Multiprecision arithmetic operations can be performed by using the instructions shown in Figure B.7.

Other arithmetic instructions include NEG, EXT, and CLR instructions. In order to negate an operand, the NEG instruction can be used. The general form of this instruction is

NEG.t EA ; EA $\leftarrow$ $-$ (EA)

In this case EA must be a data-addressing mode and the data type $t = b, W, \text{ or } L$. For example, the instruction

NEG.L D0

negates the 32-bit 2's complement number stored in the data register D0. Using the EXT instruction, byte or word operands stored in a data register can be sign-extended to a word or long word, respectively. For example, the instruction

EXT.W D2

sign-extends the low byte of this data register D2 to obtain a 16-bit word. Similarly the instruction

EXT.L D4

sign-extends the low-order 16 bits of the data register D4 to obtain a 32-bit long word.

S#	INSTRUCTION	DATA TYPE t	OPERATION	EXAMPLE	COMMENTS
1	ADD.t EA, Dn	t = B, W, or L	$Dn \leftarrow (EA) + (Dn)$	ADD.B D0, D1	When EA = An, t ≠ B.
2	ADD.t Dn, EA	t = B, W, or L	$EA \leftarrow (Dn) + (EA)$	ADD.W D0, (A0)	EA should not be an alterable addressing mode.
3	ADDI.t # d8, EA or ADDI.t #d16, EA or ADDI.t #d32, EA	t = B, W, or L	$EA \leftarrow d8 + (EA)$ $EA \leftarrow d16 + (EA)$ $EA \leftarrow d32 + (EA)$	ADD.L \$00FF, D0	The immediate data will be matched with the operand size.
4	ADDQ.t #d, EA	t = B, W, or L	$EA \leftarrow d + (EA)$	ADDQ.W #2, D0	$1 \leq d \leq 8$.
5	SUB.t EA, Dn	t = B, W, or L	$Dn \leftarrow (Dn) - (EA)$	SUB.B D0, D1	Same as 1.
6	SUB.t Dn, EA	t = B, W, or L	$EA \leftarrow (EA) - (Dn)$	SUB.L D0, (A0)	Same as 2.
7	SUB.t #d8, EA or SUBI.t #d16, EA or SUBI.t #d32, EA	t = B, W, or L	$EA \leftarrow (EA) - d8$ $EA \leftarrow (EA) - d16$ $EA \leftarrow (EA) - d32$	Sub.W \$00ff, D0	Same as 3.
8	SUBQ.t # d, EA	t = B, W, or L	$EA \leftarrow (EA) - d$	SUBQ.W #2, D1	Same as 4.

Figure B.7 MC68000 Add and Subtract Instructions (Courtesy of Motorola)

S#	INSTRUCTION	DATA TYPE T	OPERATION	COMMENTS
1	ADDX.t Di Dj	t = B, W, or L	$Dj \leftarrow (Di) + (Dj) + X$	Di and Dj are data registers.
2	ADDX.t - (Ai), - (Aj)	t = B, W, or L	$(Aj) \leftarrow ((Ai)) + ((Aj)) + X$	Address registers Ai and Aj are decremented first by 1 for B, 2 for W and 4 for L before the operation.
3	SUBX.t Di, Dj	t = B, W, or L	$Dj \leftarrow (Dj) - (Di) - X$	Same as 1.
4	SUBX.t - (Ai), - (Aj)	t = B, W, or L	$(Aj) \leftarrow ((Aj)) - ((Ai)) - X$	Same as 2.

Figure B.7 continued (Courtesy of Motorola)

It should be emphasized that in this instruction the destination operand must always be in data direct mode.

CLR reg or mem clears a register or memory location to zero. For example,

CLR.L D0

clears the 32-bit register D0 to zero.

The MC68000 instruction set includes instructions in order to perform 16×16 multiplication and $32 \div 16$ division.

The numbers must be integers; however, they may be unsigned or signed. A summary of these instructions is provided in Figure B.8:

As an example, consider:

MULS #-10, D0

This instruction multiplies the low-order 16 bits of the data register D0 by -10 and saves the 32-bit product in the data register D0.

Logical Instructions

The basic instructions in this group are the Boolean AND, OR, exclusive-OR (EOR), and NOT. These instructions can operate on byte, word, and long-word operands; they are summarized in Figure B.9.

For example, consider the following instruction:

AND \$000020,D1

If $[000020] = 0000_{16}$ and $[D1]_{\text{low word}} = \text{FFFF}_{16}$, then after execution of this instruction, the low-order 16 bits of the data register D1 will be cleared.

The compare instruction (CMP) subtracts the source operand from the destination operand and affects the condition flags according to the result of the subtraction. Note that the difference computed is not saved in the destination. It is interesting to note that with the memory-compare instruction

CMPM (A0)+, (A1)+

INSTRUCTION	DATA TYPE	OPERATION
MULU source, Dn	Unsigned integers	Dn.L $\leftarrow$ (Source.W)*(Dn.W)
MULS source, Dn	Signed integers	
DIVU source, Dn	Unsigned integers	Dn.L (Dn.L)/(Source.W)
DIVS source, Dn	Signed integers	After execution, the low- and high-order 16 bits of the data register will contain the quotient and remainder, respectively

Figure B.8 MC68000 Multiplication and Division Instructions (Courtesy of Motorola)

S#	Instruction	Data Type t	Operation	Comments
1	AND.t EA, Dn	t = B, W, or L	$Dn \leftarrow (EA) \wedge (Dn)$	EA should not be in address register direct mode.
2	AND.t Dn, EA	t = B, W, or L	$EA \leftarrow (Dn) \wedge (EA)$	EA should be an alterable addressing mode.
3	AND.t #d8, EA or AND.t #d16, EA or AND.t #d32, EA	t = B, W, or L	$EA \leftarrow d8 \wedge (EA)$ $EA \leftarrow d16 \wedge (EA)$ $EA \leftarrow d32 \wedge (EA)$	EA should be an alterable addressing mode.
4	OR.t EA, Dn	t = B, W, or L	$Dn \leftarrow (EA) \vee (Dn)$	Same as 1.
5	OR.t Dn, EA	t = B, W, or L	$EA \leftarrow (Dn) \vee (EA)$	Same as 2.
6	ORI.t #d8, EA or ORI.t #d 16, EA or ORI.t #d32, EA	E = B, W, or L	$EA \leftarrow d8 \vee (EA)$ $EA \leftarrow d16 \vee (EA)$ $EA \leftarrow d32 \vee (EA)$	Same as 3.
7	EOR.5 Dn, EA	t = B, W, or L	$EA \leftarrow (Dn) \oplus (EA)$	Same as 1.
8	EORI.t #d8, EA or EORI.t #d16, EA or EORI.t #d32, EA	t = B, W, or L	$EA \leftarrow d8 \oplus EA$ $EA \leftarrow d16 \oplus EA$ $EA \leftarrow d32 \oplus EA$	Same as 3.
9	NOT.t EA	t = B, W, or L	$EA \leftarrow (EA)'$	EA must be an alterable addressing mode.

Figure B.9 Typical MC68000 Boolean Instructions (Courtesy of Motorola)

two memory operands can be compared. In this instruction it is assumed that the address registers A0 and A1 are used as pointers to memory locations. It is important to point out that this instruction cannot be used in the predecrement mode.

The TEST MEM or REG instruction sets the condition codes after comparing the contents of mem or reg with zero. The test and set instruction (TAS) is a synchronization instruction useful in multiprocessor systems.

Shift and Rotate Instructions

The MC68000 has a group of shift and rotate instructions that can shift or rotate an operand in a register or memory to the right or to the left. These instructions are summarized in Figure B.10. For example, consider the instruction:

ROL #2, D5

If $(D_5)_{\text{low word}} = 0001_{16}$, then after execution of this instruction, (D_5) will be rotated twice to the left. That is, $(D_5) = 0004_{16}$ and $Cy = 0$.

INSTRUCTION	OPERAND SIZE	OPERATION
ASL RMD, RM }	8, 16, 32	
ASL (EA) }		
ASR RMD, RM }	8, 16, 32	
ASR (EA) }		
LSL D _n , D _y }	8, 16, 32	
LSL #d, D _y }		
LSR D _n , D _y }	8, 16, 32	
LSR #d, D _y }		
ROL D _n , D _y }	8, 16, 32	
ROL #d, D _y }		
ROR D _n , D _y }	8, 16, 32	
ROR #d, D _y }		
ROXL D _n , D _y }	8, 16, 32	
ROXL #d, D _y }		
ROXR D _n , D _y }	8, 16, 32	
ROXR #d, D _y }		

Figure B.10 Shift and Rotate Instructions (Courtesy of Motorola)

Bit-Manipulation Instructions

The MC68000 has four instructions that test the state of a specified bit in a memory location or register. These instructions are:

```
BTST    )
        )
BSET    )    Dn,        Dy
        )    or        or
BCLR    )
        )
BCHG    )    #data,    Mem
```

RMD contains data indicating number of times to be shifter. RMD indicates register, mem, or data. RM means REG or MEM.

For example, the instruction `BCHG #3, D4` tests bit 3 of the data register D4, transfers its value to the Z-flag, and finally, complements the bit tested. Suppose that the contents of the low-order 16 bits of the data register D4 are 0002_{16} . Then after the execution of the instruction `BCHG #3, D4`, the Z-flag will be cleared (because the bit 3 is a 0), and the contents of the low-order 16 bits of the data register D4 will become $000A_{16}$ (because bit 3 is complemented).

Binary-coded Decimal Instructions

The MC68000 provides the following three instructions in order to manipulate binary-coded decimal (BCD) data: add decimal with extend (ABCD), subtract decimal with extend (SBCD), and negate decimal with extend (NBCD). Note that these instructions always utilize the extend flag X. Therefore, this flag must be cleared before the instruction is used on the low-order BCD bytes. As an example, consider the following instruction

```
ABCD D1,D2
```

If $[D1]_{\text{low byte}} = 02$, $[D2] = 03$ and $X = 1$, then after the execution of this instruction, the low-order byte of D2 will contain 06 ($03 + 02 + 01 = 06$). All BCD instructions can operate only on byte operands, and thus the suffix .B may be omitted. Also, the BCD add instruction can be used to add two memory operands, as shown next:

```
ABCD - (A1), - (A0)
```

In this case, it is assumed that the address registers A1 and A0 are used as pointers to the stored data. It is worth mentioning that with respect to the ABCD instruction, only the following two possibilities are allowed:

```
ABCD Di, Dj    or
ABCD -(Ai), -(Aj)
```


The preceding result illustrates that the MC68000 instruction set is not truly orthogonal. In particular, the second possibility is very useful in carrying out multiprecision addition operation on BCD operations.

Program Control Instructions

In the MC68000, there are two unconditional branch instructions: JMP and BRA. The JMP instruction operates in the absolute mode and thus the branch range is unlimited. On the other hand, the BRA instruction is a relative branch instruction. In this case, the branch address is the sum of the contents of the PC and the offset field d. The offset may be 8 or 16 bits. This implies that with a 16-bit offset, a branch range of -32768 to 32767 bytes is realized.

All MC68000 conditional branch instructions operate in the relative mode, and the general format of this instruction is

$$B_{cc}$$

In this format, the symbol *cc* represents one of the possible 14 conditions. A complete summary of these instructions is provided in Figure B.11.

Mnemonic	Condition	Status Flag Condition
BEQ	Equal	$Z = 1$
BNE	Not equal	$Z = 0$
BPL	Plus	$N = 0$
BMI	Minus	$N = 1$
*BGT	Greater	$Z + (N \oplus V) = 0$
*BLT	Less	$N \oplus V = 1$
*BGE	Greater or equal	$N \oplus V = 0$
*BLE	Less or equal	$Z + (N \oplus V) = 1$
BHI	Higher	$C + Z = 0$
BLS	Lower or same	$C + Z = 1$
BCS	Carry set	$C = 1$
BCC	Carry clear	$C = 0$
*BVS	Overflow	$V = 1$
*BVC	No overflow	$V = 0$

*2'S complement arithmetic

Figure B.11 MC68000 Branch Instructions (Courtesy of Motorola)

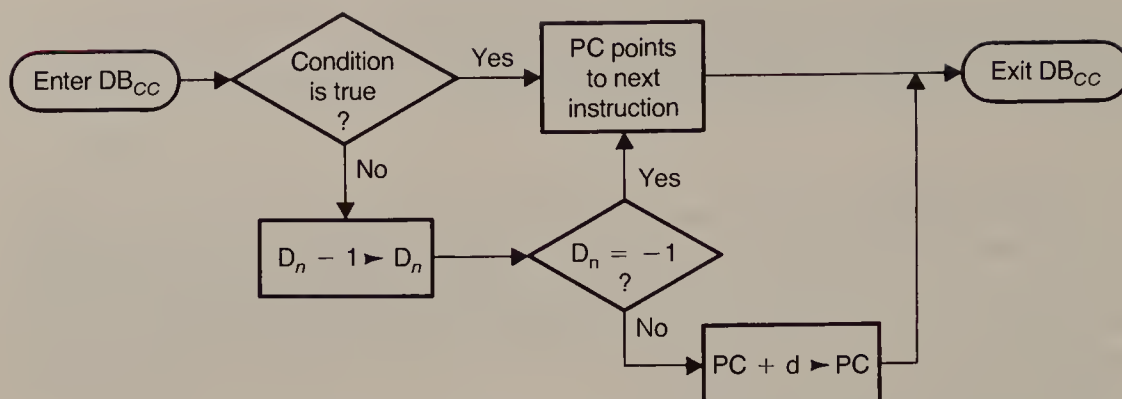


Figure B.12 Flowchart Corresponding to the MC68000 DB_{cc} Instruction (Courtesy of Motorola)

In order to describe program loops, the MC68000 instruction set provides a special test, decrement, and branch instruction. The general format of this instruction is

DB_{cc}

where *cc* is any one of the 14 possible conditions discussed earlier. A flowchart that describes the mechanics of this instruction is shown in Figure B.12.

The behavior of this instruction can be modeled using the following Pascal segment:

```

While Count ≥ 0 and not (cc) do
begin
    Evaluate the condition cc
    . . . . .
    . . . . .
    . . . . .
    count := count - 1
end
  
```

Here it is assumed that the loop count is held in the data register D_n.

In MC68000, subroutine calls are handled by the JSR (absolute mode) and BSR (relative mode) instructions. In either case, PC is automatically used as the linkage register. A return from the subroutine is accomplished via the RTS instruction.

System-control Instructions

The MC68000 operation can be controlled by system-control instructions, which include privileged instructions, trap-generating instructions, and instructions that use or modify the status register. Note that the MC68000 can execute the privileged instructions only in the supervisory mode. As an example, consider ORI #0050H, SR where SR logically ORs 0050₁₆ with the contents of SR.

The general format of MC68000 assembly language program is the same as the 8085 or PDP-11 program. Many MC68000 assemblers interpret the directives ORG, EQU, and END in the same way as the MACRO-11 directives such as =V+LC, <sym>

= $\langle \text{exp} \rangle$, and END directives, respectively. However, the directives that are used to define constants and reserve space are slightly different from the corresponding PDP-11 directives. They are summarized as follows:

<i>Directive</i>	<i>Purpose</i>
DC.B	To define a byte constant
DC.W	To define a word constant
DC.L	To define a long-word constant
DS.B n	To reserve n consecutive bytes
DS.W n	To reserve n consecutive words
DS.L n	To reserve n consecutive long words



EXAMPLE B-1

Write a MC68000 assembly language program that is equivalent to the following Pascal program segment:

```
sum := 0 ;
for i := 0 to 9 do
    sum := sum + X[i] * Y[i]
```

Assume that X and Y are addresses of integer arrays and each element requires 16 bits of storage. In particular, initialize the array elements as shown:

i	X_i	Y_i
0	20	-4
1	10	3
2	18	7
3	-9	16
4	22	11
5	19	5
6	32	-12
7	25	14
8	29	17
9	-34	40

SOLUTION

```

N      ORG      $1000      ;
      EQU      9          ; Array length
      MOVEQ    #N,D0      ; Use D0 as loop counter
      LEA      X, A0      ; Point A0 to X
      LEA      Y, A1      ; Point A1 to Y
      LEA      SUM, A2    ; Point A2 to SUM
      CLR.L    D1          ; Use D1 as accumulator
                        ; and clear it at the beginning
LOOP   MOVE     (A0)+,D2   ; Move X[i] in D2
      MULS     (A1)+,D2   ; Compute X[i] * Y[i]
      ADD.L    D2, D1     ; Update sum
      DBF      D0, LOOP   ; Repeat until D0 = -1
      MOVE.L   D1,(A2)    ; Save the result
HLT    BRA      HLT       ; Stay here forever

X      DC.W     20,10,18,-9,22 ; Elements of the
      DC.W     19,32,25,29,-34; array X

Y      DC.W     -4,3,7,16,11   ; Elements of the
      DC.W     5,-12,14,17,40 ; array Y

SUM    DS.L     1              ; Reserve 4 bytes
                        ; for the result

      END

```

EXAMPLE B-2

Write an MC68000 subroutine in order to compute the trace of an $N \times N$ integer matrix A . Assume that each matrix element takes 16 bits of storage. Write a main program that uses this subroutine to compute the trace of the following matrix:

$$a = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} = \begin{bmatrix} 18 & 48 & 22 & 19 \\ 78 & 94 & 37 & 29 \\ 43 & 52 & 88 & 34 \\ 57 & 61 & 72 & 95 \end{bmatrix}$$

SOLUTION

The trace of an $N \times N$ matrix is defined as the sum of the elements on the leading diagonal. More formally,

$$\text{trace } T = \sum_{i=1}^N a_{ii}$$

Suppose that the 4×4 matrix is stored in the row-major order form. That is, elements are stored row by row and within a row, elements are stored column by column. This idea is shown next:

Word address

S	a_{11}
S + 2	a_{12}
S + 4	a_{13}
S + 6	a_{14}
S + 8	a_{21}
S + 10	a_{22}
S + 12	a_{23}
S + 14	a_{24}
S + 16	a_{31}
S + 18	a_{32}
S + 20	a_{33}
S + 22	a_{34}
S + 24	a_{41}
S + 26	a_{42}
S + 28	a_{43}
S + 30	a_{44}

In this organization, the address of the array element a_{ij} , $1 \leq i \leq N$, $1 \leq j \leq N$, can be determined using the following formula:

$$\text{address of } (a_{ij}) = S + B[(i - 1) + N(j - 1)]$$

where B is the number of bytes required to store an array element. In our case, $B = 2$ and $N = 4$. Therefore, it follows that

$$\begin{aligned} \text{address of } (a_{ij}) &= S + 2[(i - 1) + 4(j - 1)] \\ &= S + 2i - 2 + 8j - 8 \\ &= S + 2i + 8j - 10 \end{aligned}$$

Suppose that the address register A0 contains the start address S and that the data register D1 contains the sum $2i + 8j$; then the MC68000 instruction

MOVE.W $-10(A0,D1.W), D3$

will transfer the array element a_{ij} into the data register D3. This is the key idea in the subroutine design. The complete solution to this problem is obtained as follows:

```

N      ORG      $1000
      EQU      4
      LEA      A,A0          ; Point A0 to a [1,1]
      LEA      TRVAL,A1      ; Point A1 to the result
      MOVEQ    #N-1,D0       ; Use D0 as loop counter
      BSR      TRACE         ; Call the subroutine
      MOVE.W   D4,(A1)       ; Transfer the result
FEVER  BRA      FEVER        ; Stay here forever
A      DC.W    18,44,22,19   ; 1st row of the matrix A
      DC.W    78,94,37,29   ; 2nd row of the matrix A
      DC.W    43,52,88,34   ; 3rd row of the matrix A
      DC.W    57,61,72,95   ; 4th row of the matrix A
TRVAL  DC.W    1
;      subroutine to compute the trace
;
TRACE  CLR.W    D4           ; Clear D4 to zero
      MOVEQ    #1,D5        ; Set i = 1
LOOP   MOVE     D5,D2        ; Get j into D2
      LSL.W    #1,D1        ; Compute 2 * i by shifting left once
      LSL.W    #3,D2        ; Compute 8 * j by shifting left 3 times
      ADD      D2,D1         ; Compute 2 * i + 8 * j
      pMOVE.W  -10(A0,D1.W),D3 ; Retrieve a [i,i]
      ADD      D3,D4         ; Update trace
      ADDQ.B   #1,D5        ; Increment i
      DBF      D0,LOOP      ; Repeat until D0 = -1
      RTS                    ; Return.

```

EXAMPLE B-3

Write a MC68000 subroutine in order to implement the following FORTRAN assignment statement:

$$P = P + Q$$

where P and Q are two 8-digit BCD numbers. For example,

$$P = 8877665544332211$$

$$Q = 1122334455667788$$

$$P + Q = 9999999999999999$$

SOLUTION

Let us assume that the numbers P and Q are stored in the memory, as shown next:

2000	88		77
2002	66		55
2004	44		33
2006	22		11

Number P (occupies 4 words)

3000	11		22
3002	33		44
3004	55		66
3006	77		88

Number Q (occupies 4 words)

For this initialization we write the MC68000 code shown next.

```

* main program
PROGRAM      EQU          $4000          ; Program origin
NUMP         EQU          $2000          ; Start address of P
NUMQ         EQU          $3000          ; Start address of Q
              ORG          PROGRAM
              MOVEA.L #NUMP, A0          ; Point A0 to P
              MOVEA.L #NUMQ, A1         ; Point A1 to Q
              MOVEQ    #7 , D1          ; Let D1 holds the loop count
              BSR      BCDADD
STAY          BRA          STAY
*
*subroutine to add two 16 digit BCD numbers
*
BCDADD        LEA    1 (A0, D1.W),A0      ; Point A0 and A, to
              LEA    1 (A1, D1.W),A1      ; the first two least-significant
                                              ; digits of P and Q
                                              ; respectively
              MOVE    #0,CCR              ; Clear X flag
ALOOP         ABCD    -(A1), -(A0)        ; Add the numbers
              DBF     D1, ALOOP           ; until D1 = -1
              RTS

```



EXAMPLE B-4

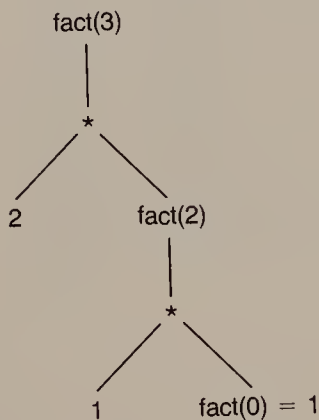
Write an MC68000 subroutine that recursively computes the factorial of n , where n is positive integer in the range 0 to B.

SOLUTION

From combinational mathematics, we know that

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times \text{fact}(n - 1) & \text{otherwise} \end{cases}$$

where $\text{fact}(n)$ is the factorial of n . A tree that illustrates the recursive computation of factorial 3 is shown next.



In this arrangement the recursive definition is first applied; then go down all the way until the root of the definition is reached, that is, the $\text{fact}(0) = 1$. After this back up in the reverse order to find the answer for $\text{fact}(3)$. In other words, the result of $\text{fact}(0)$ is used to obtain $\text{fact}(1)$ ($\text{fact}(1) = 1 \times \text{fact}(0)$), and this result is used in turn to find $\text{fact}(3)$. As mentioned before, a recursive computation in general involves a descent followed by an ascent exactly in the reverse order. This implies that the use of stack offers a clear solution for implementing recursive subroutines. A 68000 program that realizes this idea is shown next.

```

        MOVEQ    #N,D0
        MOVE.L   D0,-(SP)    ; Push N into the stack
        BSR     FACT
A       MOVE.L   (SP)+,D1    ; POP N! into D1
  
```

*subroutine to compute N!

*

```

FACT  MOVE.L  (SP)+, A4    ; Save return address
      MOVE.L  (SP)+, D0    ; Get N into D0
      TST     D0
      BEQ     FACT0        ; If N = 0 GOTO root part
      MOVE.L  D0, -(SP)    ; Otherwise push n
      MOVE.L  A4, -(SP)    ; Push return address
      SUBQ.L  #1, D0       ; Compute N - 1
      MOVE.L  D0, -(SP)    ; Push N - 1 and
      BSR     FACT         ; Invoke a recursive call
B      MOVE.L  (SP)+, D1    ; Retrieve N - 1
      MOVE.L  (SP)+, A4    ; Retrieve return address
      MOVE.L  (SP)+, D0    ; Retrieve fact(n)
      MULU    D1, D0       ; Compute n*fact(n - 1)
      MOVE.L  D0, -(SP)    ; And push this result
      MOVE.L  A4, -(SP)    ; Restore return address
      BRA     EXIT
FACT0 ADDQ.L  #1, D0       ; Compute fact(0)
      MOVE.L  D0, -(SP)    ; And push it into the stack
      MOVE.L  A4, -(SP)    ; Restore return address
EXIT  RTS

```

It is interesting to note that in this program, the stack size varies in a dynamic fashion. Shown next is a complete trace of the stack contents when $N = 3$.

			B	B			
			0	1			fact (0) = 1
		B	B	B	B		
		1	1	1	1		fact (1) = 1
	B	B	B	B	B	B	
	2	2	2	2	2	2	fact (2) = 2
A	A	A	A	A	A	A	
3	3	3	3	3	3	6	fact (3) = 6
	Descent			Ascent			

In this sketch, A and B denote the return addresses.

This example shows that in order to support recursive subroutines, a large memory space is required; in addition the system has to perform extensive housekeeping functions.

REFERENCES

1. Rafiquzzaman, M. *Microprocessors and Microcomputer Development Systems*. New York: Wiley, 1984.
2. Tanenbaum, A. S. *Structured Computer Organization*. Englewood Cliffs, N.J.: Prentice-Hall, 1984.
3. Triebel, W. A., and A. Singh. *The 68000 Microprocessor Architecture, Software, and Interfacing Techniques*. Englewood Cliffs, N.J.: Prentice-Hall, 1986.

INDEX

- Accumulator based processors 12
- A/D converter 279
- Access time (of memory) 204
- Adders 74
- Adder design
 - adder/subtractor 84
 - carry look-ahead adder 77–79
 - carry save addition 80
 - multi-operand summation 81
 - ripple carry adder 75
 - Wallace tree 82
- Address extension 210
- Addressing modes 31
- Arithmetic logic unit (ALU) 2
- ALU design techniques
 - ALU design 84–88
 - AM 2901 processor slice 88–94
- AM2909 microprogram sequencer 178–182
- Arithmetic pipelines 317–320
- Array multiplier 96
- Assembler
 - basic concepts 6, 39
 - MC68000 assembly language programming 380–406
- Associative memory 205, 232–242
- Bank switching 210
- Barrel shifter 68–70
- Basic gates
 - AND 346
 - Exclusive-OR 349–350
 - NAND 348
 - NOR 349
 - NOT 345
- Belady's anomaly 263
- Best-fit method 258–260
- Bit-slice processor (AM2901) 88–94
- Boolean algebra 344–353
- Booth's Algorithm 104–106
- Bus 10
- Bus structures 134–137
- Cache memory 203, 220–232
 - cache hit 221

- Cache memory (*continued*)
 - cache miss 221
 - direct mapped cache 224
 - fully associative mapped cache 223
 - MC68020 on chip cache 230–232
 - set associative cache 226
 - write back policy 229
 - write through policy 229
- Canonical recording 110
- Carry look-ahead adder 77–79
- Carry save addition 80
- Cassettes 80
- CLI (Command Line Interpreter) 4
- Co-processors 118–121
- Combinational circuit design 353–372
 - comparators 355–356
 - decoders 357–360
 - encoders 360–362
 - full adder 353–356
 - half adder 353–354
 - multiplexers 362–364
 - PLAs (Programmable Logic Arrays) 370–372
 - PROMs (Programmable ROMs) 368–369
- Combinational shifter 66
- Conditional I/O 279
- Control unit 2
- Control unit design
 - design methods 139–140
 - hardwired control design 155–190
 - microprogrammed control design 155–190
 - nanomemory 186–189
 - sequencers 174–182
- CPU (Central Processing Unit) 61
- Cycle time (of memory) 206
- Daisy chain interrupts 287–289
- Data flow computers 334–338
- Delayed branch 325
- Demand paging 255
- Direct access memory 206
- Direct mapped cache 224
- Division techniques
 - non-restoring division 113–115
 - Newton Raphson division 116–117
 - restoring division 111–112
- DMA (Direct memory access) 50, 277, 289–291, 297–300
 - MC68000 DMA 297–300
- DRO (Destructive Read Out) 207
- Dynamic address translation 240, 248
- Dynamic memories 207
- EAROM (Electrically Alterable ROM) 207, 370
- Emulation 140
- ENIAC Computer 1
- EPROM (Erasable Programmable ROM) 207, 370
- Expanding Op-code technique 25
- External fragmentation 250
- Fetch policy 247
- FIFO replacement policy 261–262
- Firmware 5
- First-fit method 259–260
- Flip-flops
 - D flip-flop 373
 - J-K flip-flop 373
 - R-S flip-flop 372
 - T flip-flop 373
- Floppy disk 216
- Flynn's classification 306–307
- Full adder 353–356
- Fully associative mapped cache 223
- Garbage collector 261
- General purpose register design 62
- General register processors 10
- Hardware 5
- Hardwired control design 140–155
- Half adder 353–354
- Huffman encoding 25, 27–31
- I/O Processor 300–302
 - Intel 8089 I/O processor 302
- Input/Output (I/O)
 - direct memory access (DMA) 277, 289–291
 - interrupt I/O 277, 282–289
 - MC68000 I/O 291–300
 - memory mapped I/O 279
 - programmed I/O 277, 278–279
 - Virtual I/O 277
- Inclusion property 265
- Instructions formats 23
 - MC68000 instruction formats 385
- Instruction pipelines 320–325
- Instruction set encoding
 - expanding op-code technique 25
 - Huffman encoding 25, 27–31
- Instruction types 39–50
 - MC68000 instruction types 384–406
- Intel 8089 I/O processor 301–302
- Internal fragmentation 250
- Interrupt I/O 277, 282–289
 - daisy chain interrupts 287–289
 - MC68000 interrupt system 295–298

- Linear segmented virtual memory system 252
- Locality of reference 203, 256
- Lockout (Software) 49
- Loosely coupled system 204
- LRU replacement policy 263–265

- Magnetic disk 214
- Magnetic tape 213
- Main memory 202
- Mapping ROM 182–183
- Mask programmable ROM 206
- MC68000 architecture, instruction set, and programming 380–406
 - addressing modes 383–384, 386–387
 - flags 381–382
 - instruction formats 385
 - instruction set 384–399
 - programming model 381
- MC68000 I/O 291–300
 - MC68000 DMA 297–300
 - MC68000 Exception handling 299–300
 - MC68000 Interrupt system 295–298
 - MC68000 Programmed I/O 291–295
 - MC68230 291–295
- MC68020 on chip cache 230–232
- Mealy machine 375
- Memory 2
- Memory array design 208
- Memory compaction 250, 261
- Memory hierarchy 216–220
- Memory management 242–267
 - address extension 211
 - address translation 240, 248
 - best-fit method 258–260
 - Belady's anomaly 263
 - demand paging 255
 - dirty bit 266
 - dynamic address translation 240, 248
 - external fragmentation 250
 - fetch policy 247
 - first-fit method 258–260
 - FIFO replacement policy
 - garbage collector 261
 - inclusion property 265
 - internal fragmentation 250
 - LRU replacement policy 263–265
 - memory compaction 250, 261
 - optimal replacement policy 263
 - overlays 242
 - page fault 245–246
 - page table 244–245
 - paging 243–246
 - segment descriptor 247
 - segment table 247
 - thrashing 267
 - TLB (Translation look-aside buffer) 252–254
 - working set model 257–258
 - worst-fit method 258–260
- Memory mapped I/O 50, 279
- Micro-instruction encoding 184–186
- Micro-instruction formats 158–160
- Microcomputer 17
- Microprocessor 3
- Microprogram 5
- Microprogram sequencers 174–182
- Microprogrammed control unit design 155–192
 - LSI sequencers 174–182
- Mapping ROM 182–183
 - micro-instruction encoding 184–186
 - micro-instruction formats 158–160
 - multiple micro-instruction format 185–186
 - nanomemory 186–191
- Minicomputers 17
- Modified Booth's algorithm 107–109
- Modular programming 45
- Multi-operand summation 81
- Multiple micro-instruction format 185–186
- Multiplication techniques
 - array multiplier 96
 - Booth's multiplication algorithm 104–106
 - canonical recoding 110
 - modified Booth's algorithm 107–109
 - ROM based multipliers 97
 - Wallace tree multiplier 98–99
- Multiprocessors 327–333
- Mutual exclusion problem 49

- Nanomemory 186–191
- Newton Raphson division 116–117
- Non-restoring division 113–115

- Op-code encoding techniques 24–31
 - expanding op-code technique 25
 - Huffman encoding technique, 25, 27–31
- Operating system 3, 6, 277
 - Virtual I/O 277
- Optimal replacement policy 263
- Overlays 242

- Page fault 245–246
- Page table 244–245
- Paging 243–246
- Parallel processing 305–343
 - arithmetic pipelines 317–320
 - array processors 308–314
 - data flow computers 334–338
 - Flynn's classification 306–307
 - instruction pipelines 320–325

- Parallel processing (*continued*)
 - multiprocessors 327–333
 - pipeline processing 314–325
 - score board 305
 - systolic arrays 310
 - wave front array processors 311
- Parameter passing convention 46
- Passive memory system 204
- Peripherals 276
- Pipeline processing 314–325
 - arithmetic pipelines 317–320
 - delayed branch 325
 - instruction pipelines 320–325
 - target prefetch 323
- Placement strategies 258–261
 - best-fit method 259
 - first-fit method 258
 - garbage collector 261
 - memory compaction 250, 261
 - worst-fit method 258–260
- Polled interrupts 286–287
- Position independent code 39
- Primary memory 203
- Processor Architecture
 - accumulator based processors 12
 - general register processors 10
 - stack machine 16
- Programmed I/O 277–279
- Random access memory 9, 206
- Refresh logic 207
- Registers
 - barrel shifter 68–70
 - combinational shifter 66
 - general purpose register design 62
 - MC68000 registers 381
- Register transfer notation 128–133
- Replacement policies 261–267
 - Belady's anomaly 263
 - dirty bit 266
 - FIFO replacement policy 261–262
 - inclusion property 265
 - LRU replacement policy 263–265
- Restoring division 111–112
- Ripple carry adder 75
- RISC (Reduced Instruction Set Computer) 51–56
- ROM (Read only memory) 9
 - memory array design 208
- ROM based multiplier 97
- Rotational delay (or latency) 215
- RWM (Read/Write memory) 9
 - address extension 210
 - bank switching 210
 - memory array design 208
- Score board 305
- Seek time 215
- Segment descriptor 247
- Segment table 247
- Sequential circuit design 372–379
 - flip-flops 372–373
 - systematic design approach 373–379
 - state diagram 375
 - state table 377
- Serial access memory 206
- Set associative mapped cache 226
- Software 5
- Stack 13
- Stack machine 16
- Standard I/O 279
- Static memories 207
- Status flag interpretation 73
- Stored program computer 7
- Subprogram 7
- Subroutines
 - linkage convention 46
 - nesting 46
 - parameter passing convention 46
- Supercomputers 17
- Superminicomputer 17
- Systolic arrays 310
- Tagged memory unit 305
- Target prefetch 323
- Thrashing 267
- Tightly coupled system 204
- Timing signal generation 137–139
- TLB (Translation Look-aside Buffer) 252–253
- Unconditional I/O 279
- Valid bit 229
- Virtual I/O 277
- Virtual memory 242–267
- Volatility (of memory) 207
- Von Neumann architecture 2
- Wallace tree multiplier 98–99
- Wallace tree structure 82
- Wave front array processors 311
- Working set model 257–258
- Worst-fit method 260
- Write back policy 229
- Write through policy 229

ISBN 0-314-60174-0



90000



9 780314 601742